

Verilog & SystemVerilog

for SoC & ASIC Design

The Complete Guide - From Beginner to Expert

Lexical Rules · Data Types · 4-State Logic · Sizing & Signedness · The Event Scheduler · Parameters & Generate · Directives · Gate-Level & Timing Checks · System Tasks · Synthesizable Subset · SV Types & Arrays · Packages · Interfaces · OOP · Constrained Random · IPC · Clocking Blocks · SVA · Coverage · DPI · Testbenches · UVM · Pitfalls · Roadmap

Contents	28 chapters in 5 parts, plus 4 appendices
Level	From your first <code>module</code> to classes, constraints, assertions, coverage and DPI - the whole language, design and verification
Standards	Verilog IEEE 1364-2005 and SystemVerilog IEEE 1800 (2017, with 2023 changes noted)
Every example runs	Where an open-source tool can run it, the example was run with Icarus Verilog 12 or Verilator 5.020 ; the output on green cards is what the tool actually printed
Honest about tools	Code that needs a commercial simulator (constraint solving, covergroups, full SVA, UVM) is marked as such and shown without an output card
Companion	<i>RTL Design for SoC & ASIC</i> in this repository covers micro-architecture; this book covers the languages

How To Use This Book

Two languages carry a chip from idea to silicon. **Verilog** is the language almost every piece of synthesizable RTL is still written in, and the language of gate-level netlists and standard-cell models. **SystemVerilog** is its superset: it makes design code safer and more compact (**logic**, **always_ff**, enums, structs, packages, interfaces), and it adds a complete verification language - classes, constrained random stimulus, processes, assertions and coverage - on which UVM and nearly every modern verification environment is built.

Most engineers learn these languages by copying code that works, and pay for it later in bugs that come from rules they never learned: an expression evaluated at the wrong width, a signed value silently treated as unsigned, a race between two **initial** blocks, an **x** that a 2-state type hid, a loop variable captured by a **fork**. This book teaches the rules themselves, precisely, and shows every one of those failures happening in a real simulator.

The five parts

Part	Chapters	What you get out of it
I - Verilog Fundamentals	1-6	History, standards and tools; lexical rules, modules and ports; nets, variables and 4-state values; operators with the exact sizing and signedness rules; assignments, timing controls and the scheduler; control flow, functions and tasks.
II - Verilog in Depth	7-11	Parameters and generate; compiler directives and macros; gate-level modelling, UDPs, specify blocks and timing checks; system tasks and file I/O; the synthesizable subset and simulation/synthesis mismatches.
III - SystemVerilog for Design	12-17	New data types, casting and assignment patterns; every kind of array; always_comb/always_ff , unique/priority and new operators; packages and scoping; interfaces and modports; idioms used in real SoC RTL.
IV - SystemVerilog for Verification	18-24	Classes and OOP; constrained random; fork/join, events, semaphores and mailboxes; clocking and program blocks; assertions (SVA); functional coverage; DPI-C, VPI and Python co-simulation.
V - Putting It Together	25-28	A complete layered testbench; how UVM is built from the language; a catalogue of pitfalls and portability issues; the learning roadmap for each SoC/ASIC role.

Reading paths

- **Complete beginner:** Parts I and II in order, running every example. Chapters 4 and 5 are the ones that separate people who know Verilog from people who have used it.
- **RTL designer:** 1-7, 11, then 12-17. Skip Part IV until you need to write assertions (Chapter 22).
- **Verification engineer:** 3-5 and 12-16 for the language core, then all of Part IV and Chapters 25-26.
- **Gate-level, DFT or library work:** 3, 5, 8, 9 and 10.
- **Interview preparation:** 4, 5, 11, 14, 18-20, 22, 27, then Appendix C.

Conventions

Style	Meaning
A grey card	Code exactly as you would save it
A green card	The output that code produced when run with Icarus Verilog 12 or Verilator 5.020 while this book was written
No green card	Either a fragment, or code that needs a commercial simulator; the text says which

Style	Meaning
Coloured boxes	KEY IDEA = the one thing to remember. MATH = a rule or equation. INTUITION = the mental picture. PRACTICAL TIP = what to do. PITFALL = a mistake that reaches silicon or sign-off. EXPERT CORNER = depth you can skip on a first read.
LRM	The IEEE language reference manual: 1364-2005 for Verilog, 1800-2017/2023 for SystemVerilog

RUN THE EXAMPLES YOURSELF

Icarus Verilog ([iverilog -g2012](#)) and Verilator ([verilator --binary --timing](#)) are free and install with one package-manager command. EDA Playground gives browser access to commercial simulators for the constraint, coverage and assertion examples no open-source tool fully runs yet.

TWO QUESTIONS TO ASK OF EVERY LINE OF CODE

What hardware does this describe? - for design code, and **when does this execute, relative to everything else?** - for every line. The first question is answered by Chapters 11 and 17, the second by Chapters 5 and 21. Almost every bug in this book is a wrong answer to one of them.

Table of Contents

PART I - Verilog Fundamentals	14
1. HDLs, Verilog and SystemVerilog: History, Standards, Tools and the SoC/ASIC Flow	15
1.1 What a hardware description language is	15
1.2 A short history of Verilog	16
1.3 From Verilog to SystemVerilog	16
1.4 Verilog versus VHDL, and the other HDLs	17
1.5 The tool landscape	18
1.6 Where each part of the language is used in the SoC/ASIC flow	18
1.7 Your first module and testbench	20
1.8 The same example in SystemVerilog	21
1.9 Conventions used in this book	22
1.10 Summary	22
1.11 Exercises	23
2. Lexical Conventions, Modules, Ports and Hierarchy	24
2.1 Lexical conventions	24
2.2 Modules	25
2.3 Port declarations: ANSI and non-ANSI style	26
2.4 Bidirectional ports and tri-state buses	27
2.5 Connecting ports	28
2.6 Implicit nets and the default_nettype directive	29
2.7 Hierarchy, top-level modules and hierarchical references	30
2.8 Multiple instances and arrays of instances	31
2.9 Summary	31
2.10 Exercises	32
3. Data Types: Nets, Variables, 4-State Logic, Strengths, Vectors, Arrays and Literals	33
3.1 Two kinds of data object: nets and variables	33
3.2 The four-state value set	33
3.3 Net types and resolution	34
3.4 Drive strength and charge strength	35
3.5 Variables: reg, integer, real, time and realtime	36
3.6 Vectors, bit-selects and part-selects	37
3.7 Arrays and memories	38
3.8 Integer literals: formats, sizing, padding and truncation	39
3.9 Summary	41
3.10 Exercises	41
4. Operators and Expressions: Sizing, Signedness and Evaluation Rules	42
4.1 The operators and their precedence	42
4.2 Arithmetic operators, division, modulus and power	43

4.3 Equality, relational and logical operators	44
4.4 Bitwise, reduction and logical operators	45
4.5 Shift operators	46
4.6 Concatenation and replication	47
4.7 The conditional operator and unknown conditions	47
4.8 Expression bit-length rules	48
4.9 Signedness rules	49
4.10 Evaluation order and short-circuiting	51
4.11 Summary	51
4.12 Exercises	51
5. Assignments, Procedural Blocks, Timing Controls and the Event Scheduler	53
5.1 Processes	53
5.2 Continuous assignments	53
5.3 initial and always blocks	54
5.4 Blocking and non-blocking assignments	55
5.5 Delays: regular and intra-assignment	56
5.6 Event control: @, edges, @*, wait and named events	57
5.7 The stratified event scheduler	58
5.8 Races, demonstrated	60
5.9 Summary	61
5.10 Exercises	62
6. Control Flow, Functions and Tasks	63
6.1 if / else	63
6.2 case, casez and casex	63
6.3 Loops	65
6.4 Named blocks and disable	66
6.5 Functions	66
6.6 Tasks	68
6.7 fork ... join in Verilog	69
6.8 Summary	70
6.9 Exercises	70

PART II - Verilog in Depth

71

7. Parameters, Generate and Configurations	72
7.1 parameter and localparam	72
7.2 Overriding parameters: #(), named association and defparam	74
7.3 Parameter dependencies and constant expressions	75
7.4 Generate constructs	76
7.5 Configurations and libraries	79
7.6 Where this is used on the SoC/ASIC roadmap	80
7.7 Summary	80

7.8 Exercises	81
8. Compiler Directives and Macros	82
8.1 Text macros: define, arguments and undef	82
8.2 Conditional compilation	84
8.3 include and include guards	84
8.4 timescale: time units, precision and compile-order dependence	85
8.5 default_nettype, resetall, celldefine and unconnected_drive	86
8.6 line, __FILE__ and __LINE__	87
8.7 Macro pitfalls and good practice	87
8.8 Summary	88
8.9 Exercises	88
9. Gate-Level Modelling: Primitives, UDPs, Delays, Specify Blocks and Timing Checks	90
9.1 Built-in gate and switch primitives	90
9.2 User-defined primitives (UDPs)	92
9.3 Delays on gates, nets and assignments	94
9.4 Specify blocks and module path delays	95
9.5 Timing checks and notifiers	96
9.6 SDF back-annotation	98
9.7 Gate-level simulation issues	99
9.8 Summary	100
9.9 Exercises	100
10. System Tasks and Functions, File I/O, VCD and an Introduction to PLI/VPI	101
10.1 Display tasks: \$display, \$write, \$strobe and \$monitor	101
10.2 Simulation time and control: \$time, \$realtime, \$timeformat, \$finish and \$stop	103
10.3 File I/O	104
10.4 Random numbers	106
10.5 Plusargs: configuring a test from the command line	107
10.6 Conversion, math and bit-vector system functions	107
10.7 Value change dump (VCD)	108
10.8 PLI and VPI: extending the simulator in C	110
10.9 Summary	111
10.10 Exercises	111
11. Synthesizable Verilog: Coding Patterns and Simulation/Synthesis Mismatches	113
11.1 The synthesizable subset	113
11.2 Templates for combinational logic	114
11.3 Templates for sequential logic	114
11.4 Simulation/synthesis mismatches	117
11.5 Coding for reuse	122
11.6 Summary	123
11.7 Exercises	123

PART III - SystemVerilog for Design**124**

12. SystemVerilog Data Types: logic, 2-State Types, Enums, Typedefs, Structs, Unions, Strings and Casting	125
12.1 logic, reg and wire: kind versus data type	125
12.2 Integer and real types: 2-state versus 4-state	126
12.3 Signedness defaults	127
12.4 Enumerated types	128
12.5 typedef	129
12.6 Structures: packed and unpacked	130
12.7 Unions: packed, unpacked and tagged	131
12.8 The string type	132
12.9 void, chandle and event	133
12.10 Casting	133
12.11 Summary	134
12.12 Exercises	135
13. Arrays: Packed, Unpacked, Dynamic, Associative, Queues and Array Methods	136
13.1 Packed versus unpacked dimensions	136
13.2 Dynamic arrays	137
13.3 Associative arrays	138
13.4 Queues	140
13.5 Array manipulation methods	141
13.6 Arrays in RTL: what synthesizes	142
13.7 Summary	143
13.8 Exercises	143
14. Procedural Enhancements: always_comb/ff/latch, unique/priority, New Operators, Loops, inside, Streaming and Argument Passing	144
14.1 always_comb versus always @*	144
14.2 unique, unique0 and priority	145
14.3 Assignment operators, ++ and -- and their blocking nature	147
14.4 Loops and jump statements	147
14.5 Functions and tasks: the SystemVerilog additions	149
14.6 Set membership: inside	149
14.7 Streaming operators	151
14.8 Summary	152
14.9 Exercises	152
15. Packages, Compilation Units, Scoping and Name Resolution	153
15.1 Packages	153
15.2 Explicit versus wildcard import: the exact rules	154
15.3 Where to put an import	155
15.4 Compilation units, \$unit and compile order	155
15.5 Name resolution	156
15.6 The timescale directive versus timeunit and timeprecision	158

15.7 Best practices for SoC projects	158
15.8 Summary	159
15.9 Exercises	159
16. Interfaces, Modports and Virtual Interfaces	160
16.1 Declaring and instantiating an interface	160
16.2 Modports	162
16.3 Generic interface ports and interface arrays	162
16.4 Tasks and functions in interfaces: bus-functional models	163
16.5 Virtual interfaces	163
16.6 Protocol assertions inside the interface	165
16.7 Interfaces and synthesis	165
16.8 Summary	166
16.9 Exercises	166
17. SystemVerilog Idioms for SoC RTL: Type Parameters, Struct-Based Bus Bundles, let, and the Synthesizable Subset Across Tools	167
17.1 Type parameters	167
17.2 Struct-based bus bundles and packages of bus typedefs	169
17.3 let: a scoped, typed macro	170
17.4 Assertions embedded in RTL	170
17.5 Enum FSMs and struct register files	171
17.6 Generate, \$bits and \$clog2 in SystemVerilog style	172
17.7 The synthesizable SystemVerilog subset across tools	173
17.8 A checklist for SoC-grade SystemVerilog RTL	174
17.9 Summary	174
17.10 Exercises	174
PART IV - SystemVerilog for Verification	176
18. Object-Oriented Programming: Classes, Inheritance, Polymorphism, Parameterized and Interface Classes.	177
18.1 Classes, objects and handles	177
18.2 Copying objects: handle copy, shallow copy, deep copy	179
18.3 Inheritance, super and method overriding	180
18.4 Abstract classes, pure virtual methods, extern and visibility	182
18.5 Parameterized classes	184
18.6 Forward declarations with typedef class	185
18.7 Interface classes	186
18.8 Design patterns you will meet: the singleton	187
18.9 Summary	187
18.10 Exercises	188
19. Constrained Random Stimulus	189
19.1 rand, randc and randomize()	189
19.2 Constraint blocks	190

19.3 Shaping the distribution: dist	191
19.4 Solution order and solve...before	191
19.5 Controlling constraints: soft, override, static, constraint_mode, rand_mode	192
19.6 pre_randomize() and post_randomize()	193
19.7 System random functions and seeding	194
19.8 When the solver fails: common causes and debugging	195
19.9 Summary	196
19.10 Exercises	196
20. Processes and Inter-Process Communication: fork/join, Events, Semaphores, Mailboxes	197
20.1 fork...join, join_any and join_none	197
20.2 The classic loop-variable bug	198
20.3 wait fork, disable fork and their scope	199
20.4 The process class	200
20.5 Named events	201
20.6 Semaphores	202
20.7 Mailboxes	203
20.8 Summary	205
20.9 Exercises	205
21. Clocking Blocks, Program Blocks and Race-Free Testbenches	206
21.1 Anatomy of a race	206
21.2 The SystemVerilog scheduling regions	206
21.3 Clocking blocks	208
21.4 Program blocks	210
21.5 final blocks	211
21.6 timeunit, timeprecision and the timescale directive	211
21.7 A race-free testbench checklist	212
21.8 Summary	213
21.9 Exercises	213
22. SystemVerilog Assertions (SVA) in Depth	214
22.1 Two families: immediate and concurrent assertions	214
22.2 Immediate assertions: simple and deferred	214
22.3 Concurrent assertions and the scheduler	216
22.4 Sequences: delays and repetition	217
22.5 Sequence operators	218
22.6 Implication and vacuity	219
22.7 Property operators	220
22.8 Sampled-value functions	221
22.9 disable iff and reset	221
22.10 Local variables	222
22.11 Multi-clock sequences and properties	222
22.12 assert, assume, cover, restrict and action blocks	223

22.13 bind, checkers and assertion control	223
22.14 A library of SoC protocol assertions	225
22.15 Tool support at a glance	227
22.16 Summary	227
22.17 Exercises	228
23. Functional Coverage	229
23.1 Code coverage versus functional coverage	229
23.2 Declaring and sampling a covergroup	229
23.3 Coverpoints and bins	230
23.4 Transition bins	232
23.5 Cross coverage	232
23.6 A worked example of the coverage calculation	233
23.7 Options and type options	234
23.8 Querying and controlling coverage	234
23.9 Covergroups in classes	235
23.10 cover property versus covergroup	236
23.11 Coverage closure and the verification plan	236
23.12 Summary	237
23.13 Exercises	237
24. DPI-C, VPI and Co-Simulation with C and Python	238
24.1 Importing C functions and tasks	238
24.2 Data type mapping	239
24.3 Open arrays and the svdpi.h API	240
24.4 Worked example: calling a C reference model from SystemVerilog	240
24.5 Exporting SV functions and tasks to C	242
24.6 Compiling and linking across simulators	243
24.7 VPI basics	244
24.8 cocotb: Python testbenches over VPI	246
24.9 Co-simulation with SystemC and golden models in SoC DV flows	248
24.10 Summary	249
24.11 Exercises	249
PART V - Putting It Together	250
25. A Layered Class-Based Testbench from Scratch	251
25.1 The layers and their responsibilities	251
25.2 The design under test	252
25.3 The interface and the virtual interface	253
25.4 Transactions, and where the constraints would go	253
25.5 The generator and mailboxes	254
25.6 The driver	255
25.7 The monitor	256

25.8 The scoreboard and reference model	256
25.9 The environment: construction, threads and end of test	257
25.10 Tests and the top module	258
25.11 Building and running	259
25.12 Catching a bug	260
25.13 From here to a production environment	261
25.14 Summary	262
25.15 Exercises	262
26. How UVM Uses the Language	263
26.1 The map: UVM mechanism -> SystemVerilog feature	263
26.2 Class hierarchy and polymorphism	263
26.3 Parameterized classes: drivers, sequencers and ports	265
26.4 Static registries and the factory	265
26.5 What the utility macros expand to	266
26.6 config_db: a parameterized class with static storage	267
26.7 Phases: virtual methods called by a scheduler	268
26.8 Sequences and sequencer arbitration	269
26.9 The register layer (RAL) in one page	270
26.10 A compact but complete UVM environment	270
26.11 Demystified: a mini UVM in plain SystemVerilog	272
26.12 Summary	277
26.13 Exercises	277
27. Language Pitfalls, Portability and Lint	278
27.1 Expression width and signedness	278
27.2 X, 4-state and 2-state pitfalls	280
27.3 Races and scheduling	281
27.4 Testbench-language pitfalls	283
27.5 The catalogue	284
27.6 Tool portability	286
27.7 Lint tools and rule categories	287
27.8 Running lint on a buggy block	288
27.9 Summary	289
27.10 Exercises	290
28. The Verilog/SystemVerilog Learning Roadmap for SoC and ASIC Roles	291
28.1 Stages of growth	291
28.2 Which language subsets each role needs	291
28.3 A 22-week study plan	292
28.4 Practice projects	293
28.5 Free tools setup	293
28.6 Reading the IEEE 1800 LRM efficiently	294
28.7 Interview preparation	295

28.8 Final checklist	296
28.9 Summary	296
28.10 Exercises	297

APPENDICES 298

Appendix A. Keywords and Syntax Quick Reference 299

Verilog-2005 reserved words (IEEE 1364-2005, Annex B)	299
Words SystemVerilog added (IEEE 1800)	299
Lexical cheat sheet	300
Template: package and module	301
Template: interface, modport and clocking block	302
Template: typedef, enum, struct and union	302
Template: always_comb, always_ff and always_latch	303
Template: function and task	304
Template: generate	305
Template: class	305
Template: program block	306
Template: covergroup	307
Template: sequence, property and assertion statements	307
Template: fork variants	308

Appendix B. Operators, Sizing Rules and System Tasks Reference 309

Operator precedence and associativity	309
Operator reference	309
Expression bit-length rules (IEEE 1800-2017 Table 11-21)	310
Signedness rules (IEEE 1800-2017 11.8)	311
Four-state truth tables	312
Equality and matching variants	313
Casting and conversion	313
System tasks and functions by group	314
Format specifiers and escapes	316
Array methods (IEEE 1800-2017 7.12)	317
String methods (IEEE 1800-2017 6.16)	317

Appendix C. 100 Verilog/SystemVerilog Interview Questions with Answers 319

Verilog basics	319
Data types and sizing	320
Blocking, non-blocking and scheduling	321
Synthesis	322
SystemVerilog design features	323
Object-oriented programming	324
Constrained randomization	325
Processes and inter-process communication	327

SystemVerilog Assertions	328
Functional coverage	329
DPI and how UVM uses the language	329
Coding puzzles	331
Appendix D. Standards, Tools and Resources	333
Verilog: IEEE 1364	333
SystemVerilog: IEEE 1800	333
Related standards	334
Simulators	334
Front ends, lint and formal	335
Online playgrounds and references	335
Books	336
Clifford Cummings' SNUG papers worth reading	336
Open-source SystemVerilog code bases to read	336
How to use these resources	337

PART I

Verilog Fundamentals

Every SystemVerilog design, testbench and UVM environment still rests on the Verilog core: modules and ports, nets and variables, four-state values, the sizing and signedness rules of expressions, and the event scheduler that decides what a simulation means. This part teaches that core at the level of the IEEE standard, with every rule demonstrated by code that was actually simulated, so that later parts can build the SystemVerilog design and verification layers on solid ground.

1. HDLs, Verilog and SystemVerilog: History, Standards, Tools and the SoC/ASIC Flow

A modern system-on-chip contains billions of transistors, yet almost none of them was drawn by hand. They were *described* - in a hardware description language (HDL) - and then simulated, checked, synthesized into gates, placed and routed by tools that read that description. For digital design and verification in industry, that language is, in the overwhelming majority of projects, **Verilog** and its successor **SystemVerilog**. Every RTL designer writes it, every verification engineer writes it (more of it, in fact), and DFT, physical design, emulation and silicon-validation engineers read it every day.

This chapter sets the scene: what an HDL is and how it differs from a programming language, where Verilog and SystemVerilog came from and what each revision of the standard added, how they compare with VHDL, which tools consume them, and - most importantly for your career - which *subset* of the language is used at each step of the SoC/ASIC flow. It ends with a first module and testbench, simulated with two different simulators and synthesized to gates.

1.1 What a hardware description language is

An HDL looks like a programming language - it has variables, operators, **if**, **for** and functions - but it describes something fundamentally different: a set of hardware blocks that all operate **at the same time**, connected by wires, whose values change as **simulated time** advances. Three ideas separate an HDL from C or Python, and all three appear in the very first Verilog you write:

- **Concurrency.** Every **assign**, every **always** block and every module instance is an independent process. There is no "next line" between two **always** blocks; they run in parallel, and the simulator interleaves them according to precise scheduling rules (Chapter 5).
- **Time.** Statements can wait: **#10** waits ten time units, **@(posedge clk)** waits for a clock edge, **wait (ready)** waits for a condition. A program runs as fast as it can; a model advances through simulated nanoseconds.
- **Hardware values.** A Verilog bit is not a Boolean. It is one of four values - **0**, **1**, **x** (unknown) and **z** (high impedance, undriven) - and nets carry drive strengths so that several drivers on one wire can be resolved (Chapter 3). Vectors have an exact bit width that the language tracks through every expression (Chapter 4).

Aspect	Software language (C, Python)	Verilog / SystemVerilog
Execution model	Sequential; one thread unless you ask for more	Thousands of concurrent processes; sequential only inside a procedural block
Time	Wall-clock time is incidental	Simulated time is explicit (# , @ , wait) and ordered by a scheduler
Data	Integers, floats, pointers	Bit vectors of any width, 4-state values, strengths; SV adds classes, queues, strings
Result	An executable program	(a) a simulation model, and (b) for the synthesizable subset, a netlist of gates and flip-flops
Typical bug	Crash, wrong answer	Wrong answer only in some cycle, X-propagation, race between processes, simulation/synthesis mismatch

The last row of that table matters most. Verilog has **two readers with different semantics**: a simulator executes the full language event by event, while a synthesis tool recognises a restricted set of coding patterns and maps them onto hardware. The same line of code can mean slightly different things to the two - a missing signal in a sensitivity list, an **x** in a **case**, an initial value on a register. Much of this book is about knowing, for each construct, exactly what both readers will do.

DESIGN SUBSET VERSUS VERIFICATION SUBSET

Roughly a quarter of SystemVerilog is synthesizable: modules, nets, logic and packed types, `always_ff/always_comb`, functions, parameters, generate, interfaces, packages. The rest - delays, `initial` blocks with timing, classes, dynamic arrays, queues, constrained random, coverage, most assertions, DPI - exists to **verify** hardware and runs only in simulation (or in formal and emulation tools). Knowing which side of this line a construct lives on is the first skill of a professional.

1.2 A short history of Verilog

Verilog was created in 1983-1984 at **Gateway Design Automation** by Phil Moorby, with Prabhu Goel, as the input language for a fast logic simulator, **Verilog-XL**. It borrowed its expression syntax from C, which helped its adoption by engineers who already knew C, and its key technical asset was a gate-level simulation engine fast enough for sign-off. When logic synthesis arrived (Synopsys Design Compiler, late 1980s) and accepted Verilog as input, the language became the natural bridge from description to gates.

Year	Event	Why it mattered
1984	Gateway Design Automation releases Verilog and the Verilog-XL simulator	A single language for gate-level and behavioural models
1989-90	Cadence acquires Gateway	Verilog becomes the language of the market-leading simulator
1990-91	Cadence places the language in the public domain; Open Verilog International (OVI) is formed to maintain it	Other vendors can build Verilog simulators and synthesis tools
1995	IEEE 1364-1995	First IEEE standard; basis of every tool for the next decade
2001	IEEE 1364-2001 ("Verilog-2001")	The largest revision: ANSI ports, <code>signed</code> , <code>generate</code> , multi-dimensional arrays, <code>@*</code> , <code>**</code> , <code>+:/- :</code> part-selects, automatic tasks, configurations, file I/O
2005	IEEE 1364-2005	Clarifications plus <code>uwire</code> , <code>\$clog2</code> , <code>pragma</code> , <code>begin_keywords</code> ; the last stand-alone Verilog standard
2009	1364 merged into IEEE 1800-2009	Verilog is now formally a subset of SystemVerilog; 1364 is no longer maintained

Standards bodies merged as well: in 2000 OVI and VHDL International combined to form **Accellera**, which incubated SystemVerilog before handing it to the IEEE. Accellera still develops UVM, SystemC, the Portable Stimulus Standard and IP-XACT, so you will meet its name throughout a verification career.

1.3 From Verilog to SystemVerilog

By the early 2000s, Verilog-2001 was adequate for RTL but weak for two jobs that had grown enormously: describing large designs concisely (no structs, no enums, no interfaces, no packages) and verifying them (no classes, no random constraints, no coverage, no assertions). Verification teams had moved to separate **hardware verification languages** - Synopsys **Vera** and Verisity **e** - coupled to the Verilog simulator through PLI. SystemVerilog was created to put all of that back into one language:

- **SystemVerilog 3.0** (Accellera, 2002) added design features largely from **Superlog**, donated by Co-Design Automation: `logic`, 2-state types, `typedef`, `enum`, `struct`, `always_ff/always_comb`, interfaces.
- **SystemVerilog 3.1 / 3.1a** (2003-2004) added the verification layer, drawing on Synopsys' donations of **OpenVera** (classes, constrained random, coverage), **OpenVera Assertions** (the basis of SVA) and **DirectC** (the basis of DPI).
- **IEEE 1800-2005** standardised SystemVerilog as an extension document on top of IEEE 1364-2005.

- **IEEE 1800-2009** merged the two documents; from then on there is one language, SystemVerilog, and "Verilog" names its Verilog-2005-compatible subset.

Standard	Main additions (not exhaustive)
1800-2005	Everything above: data types, OOP, randomization, covergroups, SVA, DPI, interfaces, packages, program blocks, clocking blocks
1800-2009	Merge with 1364; checkers, <code>let</code> , <code>unique0</code> , <code>global clocking</code> , improved assertion semantics (<code>s_eventually</code> , <code>accept_on...</code>), <code>\$fatal</code> elaboration tasks
1800-2012	Interface classes (multiple inheritance of interfaces), soft constraints, <code>uniqueness</code> constraints, coverage enhancements, <code>timeunit</code> rules clarified
1800-2017	Corrections and clarifications only; no major new features. The version most tools claim to support, and available at no cost through the IEEE GET program
1800-2023	Further clarifications plus some enhancements (for example triple-quoted multi-line string literals and class method qualifiers such as <code>:initial</code> , <code>:extends</code> , <code>:final</code>); tool support is still arriving

WHICH STANDARD DOES THIS BOOK FOLLOW?

The Verilog chapters (Parts I and II) follow IEEE 1364-2005, which is also the Verilog subset of IEEE 1800. The SystemVerilog chapters follow IEEE 1800-2017, the version every major tool supports, and call out 1800-2023 differences where they matter. When this book says "the LRM" (Language Reference Manual) it means these documents. UVM is a separate standard, IEEE 1800.2 (2017, revised 2020), built on top of the language (Chapter 26).

1.4 Verilog versus VHDL, and the other HDLs

VHDL (IEEE 1076, first standardised in 1987 from the US Department of Defense VHSIC programme; revised 1993, 2002, 2008 and 2019) is the other mainstream HDL. Both describe the same hardware and both are fully supported by commercial simulation and synthesis tools; the choice is mostly regional and industrial. VHDL is strong in Europe, in aerospace and defence and in some FPGA houses; Verilog and SystemVerilog dominate ASIC and SoC work, especially verification, where UVM is SystemVerilog-only.

Topic	Verilog / SystemVerilog	VHDL
Typing	Weak: widths and signedness converted implicitly (Chapter 4 exists because of this)	Strong: explicit conversions (<code>resize</code> , <code>unsigned()</code>), fewer silent surprises
Syntax style	C-like, compact	Ada-like, verbose
Four-state logic	Built in (<code>0 1 x z</code>)	<code>std_logic</code> is a 9-value enumerated type from a library (IEEE 1164)
Scheduling	Stratified event queue; races possible (Chapter 5)	Delta cycles with signal/variable separation; deterministic by construction
Verification	Classes, constraints, coverage, SVA, UVM in the language	OSVVM and UVVM libraries; PSL for assertions
Gate-level / SDF	Native gate primitives, specify blocks, SDF back-annotation (Chapter 9)	VITAL (IEEE 1076.4)
Market	Most ASIC/SoC design and nearly all ASIC verification	FPGA, defence, parts of Europe

Mixed-language simulation is routine: commercial simulators compile Verilog, SystemVerilog and VHDL into one model, so a VHDL IP block can sit inside a SystemVerilog SoC testbench. Other languages you may encounter compile to Verilog: **Chisel** (Scala), **SpinalHDL**, **Amaranth** (Python) and high-level synthesis from C++/SystemC all emit Verilog netlists, so reading generated Verilog is a useful skill even for engineers who never write it. **Verilog-AMS** extends Verilog for analog and mixed-signal modelling, and **SystemC** (IEEE 1666) is a C++ library used for transaction-level architecture models.

1.5 The tool landscape

No single tool implements the whole of IEEE 1800, and every tool implements a different part of it. The table lists the main categories. The open-source tools in the last column are the ones used to run every example in this book; the commercial tools are what you will use on a production chip.

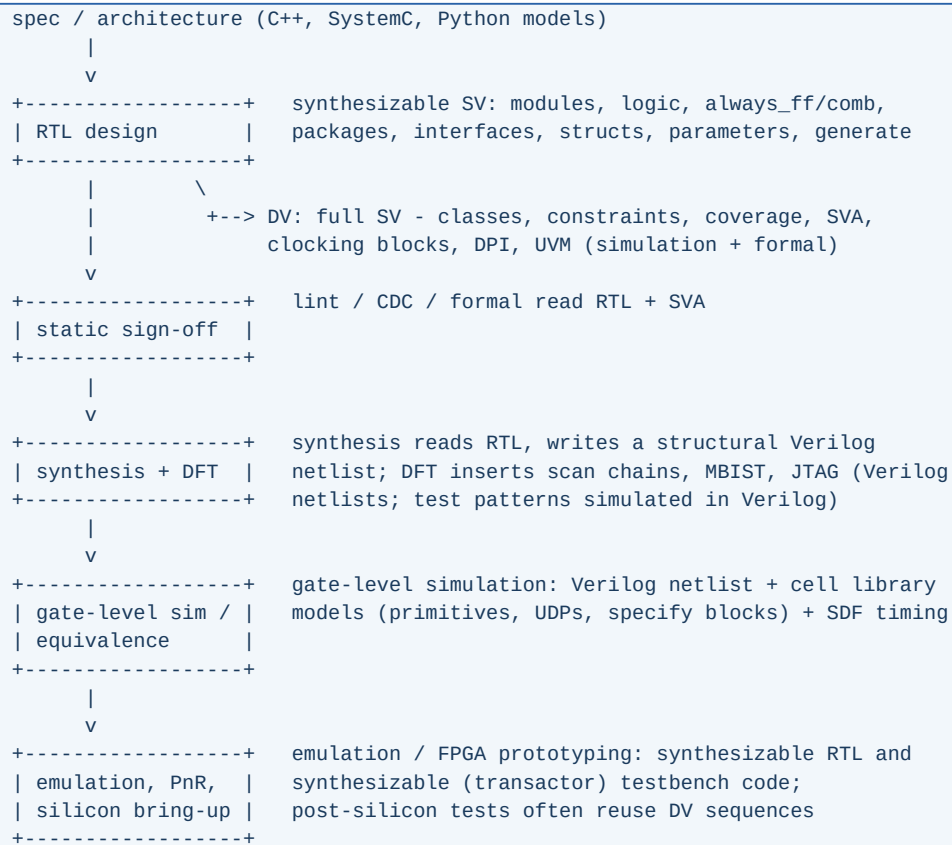
Category	What it does with the language	Commercial	Open source
Event-driven simulator	Executes the full language, 4-state, with timing	Synopsys VCS, Cadence Xcelium, Siemens Questa	Icarus Verilog
Cycle-based / compiled simulator	Translates RTL to C++; fast, 2-state, limited timing	(modes of the above)	Verilator
Logic synthesis	Maps the synthesizable subset to gates	Synopsys Design Compiler / Fusion Compiler, Cadence Genus	Yosys
Lint	Static rule checks on coding style and semantics	Synopsys SpyGlass, Cadence JasperGold Superlint, Siemens Questa Lint	Verilator --lint-only, Verible
Formal verification	Proves SVA properties exhaustively	Cadence Jasper, Synopsys VC Formal, Siemens Questa Formal	SymbiYosys (Yosys)
Equivalence checking	Proves RTL == netlist	Synopsys Formality, Cadence Conformal	Yosys (equiv, limited)
CDC / RDC	Structural clock/reset-domain crossing checks	SpyGlass CDC, Questa CDC, Jasper CDC	-
Emulation / prototyping	Maps synthesizable RTL (and synthesizable testbench parts) to hardware	Cadence Palladium and Protium, Synopsys ZeBu and HAPS, Siemens Veloce	FPGA boards with open flows

PITFALL: "IT COMPILES" IS NOT PORTABILITY

Tools differ in what they accept and in what they warn about. Icarus Verilog is a faithful 4-state event-driven simulator but supports only part of SystemVerilog (no classes with constraints, no covergroups). Verilator accepts much of the verification language but is 2-state (it turns `x` and `z` into 0/1 values), and version 5.020 compiles `randomize()` while ignoring constraints. Synthesis tools silently ignore delays and `initial` blocks for ASIC targets. Production teams therefore run the same RTL through several tools - lint, two simulators, synthesis, equivalence - and code in the portable subset. Chapter 27 is devoted to this problem.

1.6 Where each part of the language is used in the SoC/ASIC flow

The same language serves very different engineers. The flow below shows the main steps of a digital ASIC project and, for each step, which part of Verilog/SystemVerilog is written or read. The companion RTL Design guide covers the flow itself in depth; here the question is *which language features* each step depends on.



The SoC/ASIC flow and the language subset consumed at each step.

Flow step	Language subset used	Chapters
RTL design	Verilog-2005 core + synthesizable SV: logic , always_ff/comb , enums, structs, packages, interfaces, parameters, generate	2-7, 11-17
Design verification (DV)	Full SV: classes, randomization, coverage, SVA, clocking blocks, mailboxes, DPI; UVM library	18-26
Formal verification	SVA properties (assert/assume/cover property) bound to RTL	22
Gate-level simulation	Structural Verilog netlists, gate primitives, UDPs, specify blocks, timing checks, SDF, timescale	8-10
DFT	Verilog netlists with scan/MBIST; pattern testbenches (often generated) in Verilog	9-11
Emulation / prototyping	Synthesizable RTL plus synthesizable transactors (SCE-MI, DPI-based)	11, 17, 24
Architecture / modelling	Behavioural SV, DPI-C to C/C++ models	24

The roadmap through this book

```

Part I   Verilog fundamentals ..... every role (ch 1-6)
|
Part II  Verilog in depth ..... design, DFT, gate-level (ch 7-11)
|
Part III SystemVerilog for design ... RTL designers (ch 12-17)
|           \
|           +-- (designers may stop here,
v               then read ch 22 and 27)
Part IV  SystemVerilog for verification ... DV engineers (ch 18-24)
|
Part V   putting it together: TB from scratch, UVM, pitfalls, roles (ch 25-28)

```

Suggested reading paths. Parts I-III are needed by everyone; verification engineers continue through Part IV.

1.7 Your first module and testbench

Every Verilog description is built from **modules**. A module has a name, a list of **ports** (its pins), and a body describing its behaviour or structure. A **testbench** is simply another module - usually with no ports - that instantiates the design under test (DUT), drives its inputs and checks its outputs. Below is a one-bit full adder and a self-checking testbench that applies all eight input combinations.

```
`timescale 1ns/1ns
// A 1-bit full adder: the "hello world" of hardware
module full_adder (
    input  wire a, b, cin,
    output wire sum, cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule

module tb_full_adder;
    reg a, b, cin;
    wire sum, cout;
    integer i, errors = 0;

    full_adder dut (.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));

    initial begin
        for (i = 0; i < 8; i = i + 1) begin
            {a, b, cin} = i[2:0];
            #1; // let the assigns settle
            if ({cout, sum} !== a + b + cin) begin
                errors = errors + 1;
                $display("MISMATCH a=%b b=%b cin=%b -> %b%b", a, b, cin, cout, sum);
            end
            $display("t=%0t a=%b b=%b cin=%b | cout=%b sum=%b", $time, a, b, cin, cout, sum);
        end
        if (errors == 0) $display("PASS"); else $display("FAIL: %0d errors", errors);
        $finish;
    end
endmodule
```

File c1_hello.v: a full adder (the design) and its exhaustive, self-checking testbench.

Reading it top to bottom:

- The compiler directive **timescale** (all compiler directives start with a grave accent) sets the time unit to 1 ns and the precision to 1 ns, so **#1** means one nanosecond (Chapter 8).
- **module full_adder (...)** declares the ports in the **ANSI style** of Verilog-2001: direction, type and name together (Chapter 2).
- The two **assign** statements are **continuous assignments**: they behave like gates, re-evaluating whenever an operand changes (Chapter 5). This is pure combinational hardware.
- In the testbench, **reg** variables hold the stimulus and **wire** nets carry the DUT outputs. The instance **full_adder dut (...)** connects ports **by name**.
- The **initial** block is a process that runs once from time 0. **{a, b, cin} = i[2:0]** assigns three bits at once through a **concatenation** on the left-hand side.
- **#1** lets simulated time advance so that the continuous assignments settle before the check. Without it, the check would read the old outputs - the most common beginner bug.
- The check compares **{cout, sum}** with **a + b + cin**. It works only because of Verilog's **expression sizing** rule: the comparison is evaluated in the width of its widest operand - two bits - so the carry of the addition is kept (Chapter 4). **!==** is the 4-state case inequality, so an **x** output also counts as a mismatch.

Compile and run it with Icarus Verilog, the event-driven simulator used for most examples in this book:

```
$ iverilog -g2005 -Wall -o sim c1_hello.v      # compile + elaborate
$ vvp -n sim                                  # run the simulation
```

```
t=1 a=0 b=0 cin=0 | cout=0 sum=0
t=2 a=0 b=0 cin=1 | cout=0 sum=1
t=3 a=0 b=1 cin=0 | cout=0 sum=1
t=4 a=0 b=1 cin=1 | cout=1 sum=0
t=5 a=1 b=0 cin=0 | cout=0 sum=1
t=6 a=1 b=0 cin=1 | cout=1 sum=0
t=7 a=1 b=1 cin=0 | cout=1 sum=0
t=8 a=1 b=1 cin=1 | cout=1 sum=1
PASS
c1_hello.v:29: $finish called at 8 (1ns)
```

Output (Icarus Verilog 12). The testbench decides PASS/FAIL by itself - nobody has to read waveforms.

The same source runs unchanged on Verilator, which translates the design into C++ and compiles it. The `--timing` option enables delays and event controls in the testbench:

```
$ verilator --binary --timing -Wall --top-module tb_full_adder fa_tb.v
$ ./obj_dir/Vtb_full_adder
```

```
t=8 a=1 b=1 cin=1 | cout=1 sum=1
PASS
- fa_tb.v:29: Verilog $finish
```

Last lines of the Verilator 5.020 run of the same testbench (copied to fa_tb.v): identical result.

Finally, a synthesis tool reads only the design module and produces a gate-level **netlist**, itself written in (structural) Verilog. Asking Yosys to map onto simple AND/OR/XOR gates gives:

```
$ yosys -q -p "read_verilog fa.v; synth -top full_adder; abc -g AND,OR,XOR; opt_clean; write_verilog -noattr fa_net.v"
```

```
assign cout = _0_ | _2_;
assign _0_ = b & a;
assign _1_ = b ^ a;
assign _2_ = cin & _1_;
assign sum = cin ^ _1_;
```

Excerpt of the netlist written by Yosys 0.33: five gates. The testbench, `initial`, `$display` and `#1` are not synthesizable and are simply not part of this flow.

ONE SOURCE, THREE READERS

The adder was consumed by an event-driven simulator, a cycle-based simulator and a synthesis tool, and each produced something consistent with the others. That consistency is not automatic - it holds because the design uses only constructs whose simulation semantics match their synthesis semantics. Chapter 11 catalogues the constructs where they do not.

1.8 The same example in SystemVerilog

SystemVerilog is a superset, so the Verilog above is already legal SystemVerilog. Rewritten in the style used in modern projects, the same idea looks like this - a parameterized adder and a random-stimulus testbench with an immediate assertion:

```
module adder #(parameter int W = 8) (
    input  logic [W-1:0] a, b,
    output logic [W:0]    sum
);
    always_comb sum = a + b;           // SV: always_comb, logic
endmodule

module tb_adder;
    localparam int W = 8;
```

```

logic [W-1:0] a, b;
logic [W:0] sum;
int errors = 0; // SV: 2-state int
adder #(.W(W)) dut (.*); // SV: .* port connection
initial begin
  repeat (1000) begin
    a = W'($urandom); b = W'($urandom); // SV: size cast
    #1;
    assert (sum == a + b) // SV: immediate assertion
      else $error("a=%0d b=%0d sum=%0d", a, b, sum);
    errors += int'(sum != a + b); // SV: += and a type cast
  end
  $display("%s: 1000 random vectors, %0d errors", (errors != 0) ? "FAIL" : "PASS", errors);
  $finish;
end
endmodule

```

File c1_sv.sv. Features new in SystemVerilog are marked; each is explained in Parts III and IV.

```

PASS: 1000 random vectors, 0 errors
c1_sv.sv:23: $finish called at 1000 (1s)

```

Output with Icarus Verilog 12 (iverilog -g2012 -Wall).

```

PASS: 1000 random vectors, 0 errors
- c1_sv.sv:23: Verilog $finish

```

Output with Verilator 5.020 (verilator --binary --timing --assert).

PITFALL: TOOL BUGS ARE REAL

While preparing this example, an earlier version that incremented `errors++` inside the assertion's `else begin ... end` action block failed to compile in Verilator 5.020 with `--assert` (a C++ code generation error) and reported 1000 false failures without it. The version shown is legal in both forms, but the lesson is general: when a result looks impossible, reproduce it on a second simulator before you debug the design.

1.9 Conventions used in this book

- Code in grey boxes is complete and was compiled; code followed by a green box was also *run*, and the green box is the verbatim output of the tool named in its caption.
- Code that needs a commercial simulator (constraint solving, covergroups, concurrent assertions, UVM) is shown without output and says so.
- `reg` and `wire` are used in the Verilog chapters; from Part III on, designs use `logic`. Both styles appear in real projects, and you must read both fluently.
- PITFALL boxes flag simulation/synthesis mismatches and portability problems; EXPERT boxes give the interview-grade detail behind a rule.

1.10 Summary

- An HDL describes concurrent hardware in simulated time with 4-state, fixed-width values; it has two readers - simulation and synthesis - whose semantics must agree.
- Verilog (Gateway, 1984) was standardised as IEEE 1364-1995, greatly extended in 1364-2001 and finalised in 1364-2005.
- SystemVerilog (Accellera 3.0/3.1a from Superlog, Vera, OVA and DirectC; IEEE 1800-2005) merged with Verilog in 1800-2009; 2012 added interface classes and soft constraints; 2017 is the reference version; 2023 is the newest.
- VHDL is the strongly typed alternative; SystemVerilog dominates ASIC design and nearly all ASIC verification through UVM.
- Each flow step uses a different language subset: synthesizable RTL, full SV for DV, SVA for formal, structural Verilog with primitives and specify blocks for gate-level and DFT.

- A testbench is just a module; a self-checking one decides pass or fail itself.

1.11 Exercises

- 1 List four constructs that belong only to the verification subset and four that are synthesizable. For each, say which flow step (Section 1.6) consumes it.
- 2 Remove the `#1` from the testbench in Section 1.7 and predict what is printed. Run it and explain the result in terms of processes and time.
- 3 Change the check to `if ({cout, sum} !== {a + b + cin})` (note the extra braces). Predict the outcome before running it, then explain it using the sizing rule you will meet in Chapter 4.
- 4 Which revision of the standard introduced each of: ANSI port lists, `signed`, `uwire`, classes, interface classes, soft constraints?
- 5 Your team receives an IP block written in VHDL and a UVM testbench. Describe how the two are simulated together and which tools can do it.
- 6 Synthesize the full adder with Yosys without the `abc -g` option and compare the cell count with Section 1.7. Why can the counts differ?

2. Lexical Conventions, Modules, Ports and Hierarchy

Before a simulator can do anything with a design it must **parse** the text into tokens, **elaborate** the module hierarchy - create every instance, resolve every parameter, connect every port - and only then **simulate**. This chapter covers the first two steps: the lexical rules that decide what a token is, the structure of a module, the many ways to declare and connect ports, and the hierarchy that results. These are the rules you use on every line you write, and most of the confusing compile errors and silent connection bugs of a beginner come from them.

2.1 Lexical conventions

A Verilog source file is a stream of **tokens**: white space, comments, operators, numbers, strings, identifiers and keywords. The language is **case-sensitive** (`Data` and `data` are different names) and **free-format**: spaces, tabs and newlines separate tokens but are otherwise ignored, except inside strings and where they terminate an escaped identifier.

Token class	Rules	Examples
White space	Space, tab, newline, form feed; separates tokens	-
Comments	// to end of line; /* ... */ block comments do not nest (a /* inside a block comment is ignored, the first */ ends it)	// note, /* a */
Operators	One, two or three characters	+ ~& === <<< ?:
Numbers	Sized or unsized, with optional base and sign (Chapter 3)	8'hFF, 'b1x0z, 12, 4'sd3, 1.5e-3
Strings	Double-quoted, one line; escapes \n \t \\ \" \ddd (octal)	"hello\n"
Identifiers	Letters, digits, \$ and _; must start with a letter or _; up to at least 1024 characters	clk, _tmp, data_\$1, u_core
Escaped identifiers	Start with a backslash, end at the first white space; may contain any printable ASCII character	\bus[0] , \a+b , \cpu/alu/q_reg
Keywords	Reserved, always lower case	module wire reg always begin end
System tasks / functions	Identifier starting with \$	\$display, \$time, \$signed
Compiler directives	Grave accent followed by a name	timescale, define, ifdef (Chapter 8)


```

module tb_lex;
  reg [7:0] \bus[0] ;           // escaped identifier: the name is bus[0]
  reg      \a+b ;              // name is a+b (terminated by whitespace)
  reg [7:0] data_$1;           // $ is legal after the first character
  reg [31:0] _big;
  initial begin
    \bus[0] = 8'hA5;
    \a+b    = 1'b1;
    data_$1 = 8'b1010_0101;    // underscores are ignored in numbers
    _big     = 32'd1_000_000;
    $display("%s=%h %s=%b data_$1=%h _big=%0d",
             "\\bus[0]", \bus[0] , "\\a+b", \a+b , data_$1, _big);
    /* block comment
       // nested line comment inside a block comment is fine */
    $display("12'o7_7 = %b   'hF = %b", 12'o7_7, 'hF);
  end
endmodule

```

Identifiers, escaped identifiers, underscores in numbers and comments (c2_lex.v).

```

\bus[0]=a5 \a+b=1 data_$1=a5 _big=1000000
12'o7_7 = 000000111111 'hF = 000000000000000000000000000000001111

```

Output (Icarus Verilog 12). The backslash and the terminating space are not part of the name.

Three details in this example trip people up:

- An escaped identifier **must** be followed by white space: `\a+b ;` is the identifier `a+b` followed by `;`. Without the space, the semicolon would become part of the name.
- `\cpu` and `cpu` are the **same** identifier - escaping a name that would be legal anyway changes nothing.
- The octal literal `12'o7_7` is `000_000_111_111`: underscores are ignored inside numbers, and each octal digit is three bits.

WHERE ESCAPED IDENTIFIERS COME FROM

You will rarely type an escaped identifier, but you will read thousands of them. Synthesis tools flatten the hierarchy and keep the original names in the netlist: a flip-flop that was bit 3 of `q_reg` inside instance `u_alu` of `u_cpu` becomes a cell named `\u_cpu/u_alu/q_reg[3]`. Gate-level simulation, SDF back-annotation, DFT scan-chain reports and ECO scripts all refer to such names, and a missing trailing space in a hand-written force or SDF path is a classic gate-level debugging trap.

PITFALL: SYSTEMVERILOG KEYWORDS IN OLD VERILOG CODE

SystemVerilog reserves many new keywords: `logic`, `bit`, `byte`, `int`, `string`, `class`, `interface`, `program`, `priority`, `unique`, `do`, `break`, `final`, `ref`, `type`, `chandle` and more. Legacy Verilog that uses them as signal names (`wire bit;`, `reg final;`) fails to compile in SystemVerilog mode. The directive pair `begin_keywords "1364-2005" / end_keywords` tells the compiler which keyword set applies to a region of code (Chapter 8); the robust fix is to rename the signals.

2.2 Modules

A **module** is the unit of design. It is a *definition*, like a class or a schematic symbol: nothing exists until the module is **instantiated**, and it can be instantiated any number of times, each instance having its own copy of every net, variable and process inside it. The general shape is:

```

module name #(parameter_list)      // optional parameters (Chapter 7)
    (port_list);                  // ports
    // module items, in any order (declare before use):
    // net and variable declarations    wire, reg, integer ...
    // parameter / localparam          Chapter 7
    // continuous assignments          assign ...
    // procedural blocks                initial, always
    // module / primitive instances    and u1 (y, a, b); sub u2 (...);
    // generate regions                Chapter 7
    // task and function definitions    Chapter 6
    // specify blocks                  Chapter 9
endmodule

```

Module skeleton (Verilog-2005).

- Module definitions cannot be nested in Verilog (SystemVerilog allows nested module declarations, rarely used). All modules live in one global name space, the **definitions name space**, so two modules with the same name anywhere in the compiled files collide.
- Declaration order matters in one direction only: a net or variable must be declared before it is referenced (implicit nets aside, Section 2.6). Modules themselves may be instantiated before their definition appears, even in a later file.
- Everything inside a module runs **concurrently**: the order of **assign** statements, **always** blocks and instances in the text has no meaning for the hardware described (and should have none in simulation - Chapter 5 shows where it leaks).

Compilation, elaboration, simulation

Phase	What happens	Typical errors
Parse / compile	Tokenise, check syntax, build a model of each module	Syntax errors, undeclared identifiers (with default_nettype none)
Elaborate	Pick the top module(s), instantiate the tree, evaluate parameters and generate blocks, connect ports, resolve hierarchical names	Unknown module, port mismatch, width warnings, multiple drivers on a uwire or variable
Simulate	Execute processes under the event scheduler	Functional bugs, races, x propagation, \$fatal

Icarus runs compile and elaborate in **iverilog** and simulation in **vvp**; commercial simulators expose the same split as separate commands (for example **xrun** does all three, **vcs** compiles and elaborates into an executable **simv**).

2.3 Port declarations: ANSI and non-ANSI style

Verilog-1995 declared ports twice: the header lists only the names, and the body gives each port a direction, and optionally a width and a type. Verilog-2001 introduced the **ANSI style** (named after ANSI C function prototypes), where direction, type, signedness and width all go in the header. A module must use one style or the other; mixing them in one header is illegal.

```

// Non-ANSI (Verilog-1995) header: port list, then declarations in the body
module mux2_1995 (y, a, b, sel);
    output y;
    input  a, b, sel;
    wire   y;
    assign y = sel ? b : a;
endmodule

// ANSI (Verilog-2001) header: direction, type and width in the port list
module mux2 (
    input wire [3:0] a, b,
    input wire       sel,

```

```

    output wire [3:0] y
);
    assign y = sel ? b : a;
endmodule

module tb_ports;
    reg [3:0] a = 4'h3, b = 4'hC;
    reg      sel = 1'b1;
    wire [3:0] y_ord, y_name, y_dot, y_star;
    wire      y1;

    mux2      u_ord (a, b, sel, y_ord);           // by position (fragile)
    mux2      u_name (.y(y_name), .sel(sel), .a(a), .b(b)); // by name (any order)
    mux2      u_dot  (.a, .b, .sel, .y(y_dot));    // SV .name implicit
    mux2      u_star (.y(y_star), .*);           // SV .* wildcard
    mux2_1995 u_old  (.y(y1), .a(a[0]), .b(b[0]), .sel(sel));

    initial begin
        #1 $display("ord=%h name=%h dot=%h star=%h old=%b", y_ord, y_name, y_dot, y_star, y1);
    end
endmodule

```

Both header styles, and the four ways to connect ports (c2_ports.sv, compiled with -g2012 for the SV `.name` and `.*` forms).

```
ord=c name=c dot=c star=c old=0
```

Output (Icarus Verilog 12): all connection styles produce the same hardware.

Direction	Inside the module it may be	Driven from outside by	Notes
input	A net (Verilog). SV also allows a variable, which then may not be written inside	Any expression	Default kind is a net (wire); even input logic a is a net in SV
output	A net or a variable (output reg [7:0] q)	Must connect to a net (Verilog) or net/variable (SV)	Driving an output from two places is a multiple-driver bug
inout	A net only - never a variable	A net only	Bidirectional; resolved like any multi-driver net

HOUSE STYLE FOR NEW CODE

Use ANSI headers, one port per line, grouped by interface (clock and reset first, then each bus), with a comment per port. Declare outputs as **output logic** (SV) or **output reg/output wire** (Verilog) explicitly rather than relying on the default. Lint tools and IP-XACT generators parse this layout easily, and code review is far easier when the header is the documentation.

2.4 Bidirectional ports and tri-state buses

An **inout** port is a net that both the module and the outside world may drive. When several drivers are active the net **resolves** their values (Chapter 3): a driver that outputs **z** - high impedance - is effectively disconnected. The classic use is an I/O pad.

```

module bidir_pad (
    inout wire    pad,      // bidirectional: must be a net
    input wire    oe,       // output enable
    input wire    dout,     // value to drive when oe=1
    output wire    din      // what is on the pad
);
    assign pad = oe ? dout : 1'bz; // tri-state driver
    assign din = pad;
endmodule

module tb_inout;

```

```

wire bus;
reg  oe_a, d_a, oe_b, d_b;
wire in_a, in_b;
bidir_pad A (.pad(bus), .oe(oe_a), .dout(d_a), .din(in_a));
bidir_pad B (.pad(bus), .oe(oe_b), .dout(d_b), .din(in_b));
initial begin
    $display("oe_a d_a oe_b d_b | bus in_a in_b");
    {oe_a, d_a, oe_b, d_b} = 4'b0000; #1 show;
    {oe_a, d_a, oe_b, d_b} = 4'b1100; #1 show;
    {oe_a, d_a, oe_b, d_b} = 4'b0010; #1 show;
    {oe_a, d_a, oe_b, d_b} = 4'b1110; #1 show;    // contention
end
task show;
    $display(" %b    %b    %b    %b | %b    %b    %b",
             oe_a, d_a, oe_b, d_b, bus, in_a, in_b);
endtask
endmodule

```

Two bidirectional pads on one shared net (c2_inout.v).

```

oe_a d_a oe_b d_b | bus in_a in_b
0    0    0    0 | z    z    z
1    1    0    0 | 1    1    1
0    0    1    0 | 0    0    0
1    1    1    0 | x    x    x

```

Output (Icarus Verilog 12). With nobody driving, the bus floats at **z**; two conflicting drivers give **x**.

PITFALL: TRI-STATES INSIDE AN ASIC

Internal tri-state buses are almost never used in modern ASIC design: they complicate timing analysis, testability (scan) and equivalence checking, and a floating bus burns power. Synthesis tools typically convert internal tri-states into multiplexers or flag them. **inout** belongs at the chip boundary (pads, where the library provides bidirectional I/O cells) and in testbenches that model open-drain buses such as I2C. Inside the core, use separate input and output buses with an explicit mux.

2.5 Connecting ports

An instance connects each port of the child module to an expression in the parent. Verilog offers two styles and SystemVerilog adds two shorthands:

Style	Syntax	Rules and risks
By position (ordered)	<code>mux2 u (a, b, sel, y);</code>	Matches the order of the port list. Legal but fragile: reordering the module's ports silently reconnects every instance. Acceptable only for gate primitives and tiny cells
By name	<code>mux2 u (.a(a), .y(y), ...);</code>	Order-independent, self-documenting; the industry default. <code>.p()</code> leaves port p explicitly unconnected
Implicit .name (SV)	<code>mux2 u (.a, .b, .sel, .y(y2));</code>	<code>.a</code> means <code>.a(a)</code> . A net or variable named a must exist in the parent and be of a compatible type and equal width ; no implicit net is created
Wildcard .* (SV)	<code>mux2 u (.y(y2), .*);</code>	Connects every remaining port to a same-named signal. Same checks as <code>.name</code> . Explicit connections listed with it take precedence. Great for testbenches and wrappers; some teams ban it in RTL because the connectivity is invisible in the text

A port connection behaves like a continuous assignment from the driver side to the receiving side. Consequently:

- If widths differ, the value is **zero-extended or truncated** exactly as in an assignment (sign-extended if the source is signed). The LRM makes this legal; every lint tool warns about it, and so should your review checklist.

- An unconnected **input** floats at **z**; inside the module a **z** on a gate input reads as **x**, so a forgotten connection usually shows up as **x** propagation in simulation. Synthesis may tie it off - or not - so fix every unconnected input warning.
- An unconnected **output** is simply unused, which is legal and common (**.overflow()**).

INTERVIEW QUESTION: CAN AN OUTPUT PORT BE A VARIABLE IN THE PARENT?

In Verilog, the parent side of an **output** or **inout** connection must be a net, because a port connection is a continuous assignment and only nets can be continuously driven. SystemVerilog relaxes this: a variable may be connected to an output port, provided it has no other driver (a variable written by a port connection or a continuous assignment may have only that one driver). This is one reason **logic** is so convenient: it can be used on either side.

2.6 Implicit nets and the default_nettype directive

Verilog creates a net **implicitly** when an undeclared identifier appears (a) in a port connection of an instance, or (b) on the left-hand side of a continuous assignment. The implicit net is a scalar (1-bit) **wire**. This was a convenience for gate-level netlists, but in RTL it turns every typo into a silent bug:

```
module and_or (input wire [3:0] a, b, output wire [3:0] y);
    wire [3:0] t;
    assign tmp = a & b;           // typo: 'tmp' instead of 't' -> implicit 1-bit wire!
    assign y = t | 4'b0000;
endmodule
module tb_implicit;
    reg [3:0] a = 4'b1100, b = 4'b1010; wire [3:0] y;
    and_or u (.a(a), .b(b), .y(y));
    initial #1 $display("y = %b (expected 1000)", y);
endmodule
```

A one-character typo (c2_implicit.v).

```
c2_implicit.v:3: warning: implicit definition of wire 'tmp'.
y = xxxx (expected 1000)
```

Icarus warns under -Wall, but the simulation runs and produces x.

The typo creates a 1-bit wire **tmp** that receives the truncated AND result, while the intended **t** is never driven, stays **z**, and **z | 0** evaluates to **x**. In a larger design, the effect is often subtler - a bus that works for bit 0 only. The cure is the compiler directive **default_nettype none**, which turns implicit net creation into an error:

```
`default_nettype none
module and_or (input wire [3:0] a, b, output wire [3:0] y);
    wire [3:0] t;
    assign tmp = a & b;           // typo: 'tmp' instead of 't' -> implicit 1-bit wire!
    assign y = t | 4'b0000;
```

The same file with **default_nettype none** at the top (c2_implicit_none.v); the directive is set back to **wire** before the testbench.

```
c2_implicit_none.v:4: error: Net tmp is not defined in this context.
1 error(s) during elaboration.
```

Output (Icarus Verilog 12): the typo is now a compile error.

```
%Warning-IMPLICIT: c2_implicit.v:3:10: Signal definition not found, creating implicitly: 'tmp'
```

Verilator 5.020 --lint-only -Wall on the original file flags it too (first message shown).

PITFALL: DEFAULT_NETTYPE NONE LEAKS ACROSS FILES

Compiler directives are not scoped to a file or module: they remain in effect for all following source text in the same compilation, including files compiled afterwards. Once `default_nettype none` is active, every port must be declared with an explicit type in the ANSI header (`input wire a`, not just `input a`) and third-party code that relies on implicit nets breaks. The usual convention is to put `default_nettype none` at the top of each of your files and `default_nettype wire` at the bottom.

2.7 Hierarchy, top-level modules and hierarchical references

Elaboration builds a tree of instances. Every instance has a **hierarchical path** made of instance names separated by dots, starting with a **top-level module**: any module that is compiled but never instantiated. A simulation may have several tops (a testbench and a separate monitor or a `bind` target), and the format specifier `%m` prints the path of the scope that executes it.

```
module counter (input wire clk, rst, output reg [3:0] q);
    reg [3:0] next;                // internal, not a port
    always @* next = q + 4'd1;
    always @(posedge clk) q <= rst ? 4'd0 : next;
    initial $display("instance path: %m");
endmodule

module top;
    reg clk = 0, rst = 1;
    wire [3:0] q0, q1;
    counter c0 (.clk(clk), .rst(rst), .q(q0));
    counter c1 (.clk(clk), .rst(rst), .q(q1));
    always #5 clk = ~clk;
    initial begin
        #12 rst = 0;
        #30 $display("c0.q=%0d c0.next=%0d (peeked)", q0, top.c0.next); // downward ref
        force top.c1.next = 4'd9; // DV: poke an internal node
        #10 $display("c1.q=%0d after force", q1);
        release top.c1.next;
        $finish;
    end
endmodule

module monitor_top;                // never instantiated -> 2nd top
    initial #20 $display("%m sees top.c0.q = %0d", top.c0.q); // cross-top reference
endmodule
```

Two instances of one module, a second top-level module, downward and cross-top hierarchical references, and `force/release` (c2_hier.v).

```
instance path: top.c0
instance path: top.c1
monitor_top sees top.c0.q = 1
c0.q=3 c0.next=4 (peeked)
c1.q=9 after force
c2_hier.v:20: $finish called at 52 (1s)
```

Output (Icarus Verilog 12). `monitor_top` is a second top: nothing instantiates it.

Any object can be referenced by its full path from anywhere in the design: `top.c0.next` reads a signal that is not a port. Names are resolved in a precise order: a reference `a.b.c` is first looked up **downward** from the current scope (is there a child instance `a`?), then **upward** - the name `a` is searched for in the parent, grandparent and so on - and finally as a top-level module name. Upward resolution is what lets `c0.next` inside `top` and `top.c0.next` anywhere else refer to the same object.

Use	Where hierarchical references are normal
Testbench probes	Reading internal state for checking or debug (<code>top.dut.u_fifo.count</code>)
<code>force / release</code>	Fault injection, bypassing a PLL or a clock gate in simulation, gate-level debugging
Backdoor memory access	<code>\$readmemh</code> into <code>tb.dut.u_rom.mem</code> , UVM register backdoor paths
Assertions	SVA <code>bind</code> attaches a checker module inside an RTL instance without editing the RTL (Chapter 22)

PITFALL: HIERARCHICAL REFERENCES IN RTL

Synthesis tools generally do not support hierarchical references across module boundaries in design code (some accept limited downward references), and they break encapsulation: a renamed instance silently breaks a testbench probe only at elaboration time. Keep them out of RTL; in testbenches, collect them in one place (a probe module, or an interface bound into the design) so that a hierarchy change is fixed once.

2.8 Multiple instances and arrays of instances

Instantiating a module several times is just several instance statements. For regular structures, Verilog-2001 added **arrays of instances**: a range after the instance name creates one instance per index. Port connections are distributed by width: if a connected expression has exactly `N` times the port width, each instance gets its own slice (the leftmost instance index gets the leftmost slice); if it has exactly the port width, it is replicated to every instance; any other width is an error.

```
module nand2 (output wire y, input wire a, b);
    assign y = ~(a & b);
endmodule

module tb_array;
    reg [3:0] a = 4'b1100, b = 4'b1010;
    reg      en = 1'b1;
    wire [3:0] y, g;
    // Array of instances: u[3], u[2], u[1], u[0]; vectors are split one bit per instance
    nand2 u [3:0] (.y(y), .a(a), .b(b));
    // A 1-bit signal on an array port is replicated to every instance
    nand2 gate [3:0] (.y(g), .a(a), .b(en));
    initial #1 $display("y = %b    g = %b    u[2].y = %b", y, g, u[2].y);
endmodule
```

Arrays of instances (c2_array.v).

```
y = 0111    g = 0011    u[2].y = 1
```

Output (Icarus Verilog 12). With `en` replicated, `g = ~(a & 1) = ~a`.

Arrays of instances are compact but limited: every instance must be identical and connections follow the slicing rule above. The `generate` construct (Chapter 7) is the general tool - it can instantiate conditionally, compute per-instance parameters and connect arbitrary expressions - and is what most RTL uses today. Arrays of instances remain common in gate-level netlists and pad rings.

2.9 Summary

- Verilog is case-sensitive and free-format; block comments do not nest; escaped identifiers begin with a backslash and end at white space, and appear everywhere in netlists.
- A module is a definition; instances create copies. Compilation, elaboration and simulation are separate phases with their own errors.
- Use ANSI port headers. Inputs are nets, `inout` ports are always nets, outputs may be variables.
- Connect by name (or `.name/.*` in SV); port connections behave like continuous assignments, so widths are silently extended or truncated.

- Implicit nets turn typos into 1-bit wires; use `default_nettype none` and reset it at the end of each file.
- Hierarchical paths reach any object; they are for testbenches, `force`, backdoors and binds - not for RTL.
- Arrays of instances split or replicate connections by width; `generate` is the general mechanism.

2.10 Exercises

- 1 Which of these are legal identifiers, and what name does each declare: `2fast`, `_2fast`, `$fast`, `fast$`, `\2fast`, `\fast`, `\a b` ?
- 2 Rewrite `mux2_1995` from Section 2.3 in ANSI style with a 4-bit data path. Then connect a 3-bit signal to its `a` port and an 8-bit signal to `y`; what happens to the missing and extra bits, and what does your lint tool report?
- 3 Explain why `.*` is safe against the typo of Section 2.6 while positional connection is not. What error do you get if a port has no same-named signal in the parent?
- 4 In `c2_hier.v`, a module `monitor_top` refers to `top.c0.q`. Explain how the name is resolved. What changes if you also add a local instance named `top` inside `monitor_top`?
- 5 Build a 16-bit bitwise NAND from `nand2` using (a) an array of instances and (b) a `generate for` loop (look ahead to Chapter 7). Compare the hierarchical names of the resulting instances.
- 6 Write an open-drain I2C-style bus model with two devices and a pull-up using `inout` ports, and show that either device can pull the line low.

3. Data Types: Nets, Variables, 4-State Logic, Strengths, Vectors, Arrays and Literals

Verilog's data types model two physical things: **wires**, which carry whatever their drivers put on them, and **storage**, which keeps a value until something writes a new one. On top of that it layers a four-valued logic system, a strength system for resolving conflicts between drivers, arbitrary-width vectors and arrays, and a literal syntax with precise sizing rules. SystemVerilog adds many more types (Chapters 12 and 13), but every one of them is defined in terms of the concepts in this chapter.

3.1 Two kinds of data object: nets and variables

	Nets (<i>wire</i> , <i>tri</i> , <i>wand</i> , ...)	Variables (<i>reg</i> , <i>integer</i> , <i>real</i> , <i>time</i> , ...)
Models	A physical connection	A storage element in the model (not necessarily a flip-flop)
Assigned by	Continuous assignments, primitive and module outputs, port connections - i.e. drivers	Procedural assignments in <i>initial/always</i> blocks, tasks and functions
Holds a value?	No: its value is recomputed from its drivers (except <i>triereg</i>)	Yes: until the next procedural assignment
Multiple sources	Yes: resolved by the net type and strengths	Last write wins (multiple processes writing one variable is a race)
Undriven value	<i>z</i> (or 0/1 for <i>tri0/tri1</i> , charge for <i>triereg</i>)	<i>x</i> for <i>reg/integer/time</i> ; 0.0 for <i>real</i>
Default width	1 bit	<i>reg</i> 1 bit; <i>integer</i> 32; <i>time</i> 64

REG DOES NOT MEAN REGISTER

A *reg* is simply a variable - the name is a historical accident. Whether it becomes a flip-flop, a latch or plain combinational logic depends entirely on how it is assigned: *always @(posedge clk) q <= d;* infers a flip-flop; *always @* y = a & b;* gives an AND gate whose output is declared *reg*. SystemVerilog replaced the misleading keyword with *logic* (Chapter 12), which may be used for both variables and, with a net keyword, nets.

3.2 The four-state value set

Value	Meaning	Typical origin
0	Logic zero, false	Driver outputs 0
1	Logic one, true	Driver outputs 1
x	Unknown: could be 0, 1, or in transition; the simulator cannot tell	Uninitialised variable, contention between drivers, <i>z</i> read by a gate input, out-of-range index, division by zero, timing-check violation in gate-level sim
z	High impedance: no driver is driving	Tri-state driver disabled, unconnected port, undriven net

x is a **simulation** concept. Real hardware always has a 0 or a 1 (or an analog value in between), and a synthesis tool treats an *x* you assign as a don't-care it may optimise. That asymmetry is the root of X-optimism and X-pessimism (Chapter 11): simulation can hide a bug that silicon exhibits, or show *x* where silicon would be fine. The operators of Chapter 4 each have exact rules for *x* and *z* inputs; the general principle is that a result is *x* whenever the unknown bits could change it.

3.3 Net types and resolution

Net type	Resolution of multiple drivers	Use
<code>wire</code> , <code>tri</code>	Standard: equal strength 0 vs 1 gives <code>x</code> ; <code>z</code> loses to anything	Default; <code>tri</code> documents intended tri-state use (identical semantics)
<code>wand</code> , <code>triand</code>	Wired-AND: any 0 wins	Open-collector buses (rare)
<code>wor</code> , <code>trior</code>	Wired-OR: any 1 wins	Wired-OR buses (rare)
<code>tri0</code> , <code>tri1</code>	Like <code>wire</code> , but reads 0 / 1 (pull strength) when all drivers are <code>z</code>	Pull-down / pull-up modelling, testbenches
<code>supply0</code> , <code>supply1</code>	Constant 0 / 1 at supply strength	Ground and power in netlists and switch-level models
<code>triereg</code>	Stores its last driven value as a charge when all drivers go to <code>z</code> ; charge decays only if a decay time is given	Switch-level models of dynamic nodes, precharged buses
<code>uwire</code> (2005)	Unresolved: more than one driver is an elaboration error	Enforcing single-driver connectivity

The following testbench drives every net type from two continuous assignments and walks through all combinations of driven values:

```

module tb_nets;
    reg d1, d2;                                // values placed on two drivers
    wire w; wand wa; wor wo; tri0 t0; tri1 t1;
    assign w = d1; assign w = d2; // two continuous drivers on each net
    assign wa = d1; assign wa = d2;
    assign wo = d1; assign wo = d2;
    assign t0 = d1; assign t0 = d2;
    assign t1 = d1; assign t1 = d2;
    integer i, j;
    reg [3:0] v [0:3];
    initial begin
        v[0] = 0; v[1] = 1; v[2] = 1'bx; v[3] = 1'bz;
        $display("d1 d2 | wire wand wor tri0 tri1");
        for (i = 0; i < 4; i = i + 1)
            for (j = i; j < 4; j = j + 1) begin
                d1 = v[i]; d2 = v[j]; #1;
                $display(" %b %b | %b %b %b %b %b", d1, d2, w, wa, wo, t0, t1);
            end
        end
    end
endmodule

```

Two drivers on each net type (c3_nets.v).

```

d1 d2 | wire wand wor tri0 tri1
0 0 | 0 0 0 0 0
0 1 | x 0 1 x x
0 x | x 0 x x x
0 z | 0 0 0 0 0
1 1 | 1 1 1 1 1
1 x | x x 1 x x
1 z | 1 1 1 1 1
x x | x x x x x
x z | x x x x x
z z | z z z 0 1

```

Output (Icarus Verilog 12): the resolution functions of wire, wand, wor, tri0 and tri1.

For a plain `wire` with two drivers of equal strength, the result is given by the table below - the one every DV engineer should be able to write from memory. It is symmetric, `z` is the identity, and any disagreement produces `x`.

wire	0	1	x	z
0	0	x	x	0
1	x	1	x	1
x	x	x	x	x
z	0	1	x	z

Resolution of two equal-strength drivers on a wire/tri net.

A **uwire** refuses a second driver at elaboration time, which is exactly what you want for a signal that must have one source:

```
module tb_uwire;
  reg a = 1, b = 0;
  uwire u;
  assign u = a;
  assign u = b;      // second driver on a uwire is illegal
endmodule
```

Two drivers on a uwire (c3_uwire.v).

```
c3_uwire.v:5: error: Unresolved net/uwire u cannot have multiple drivers.
1 error(s) during elaboration.
```

Icarus Verilog 12 rejects the design. (Verilator 5.020 --lint-only accepted this file silently.)

WHY SYSTEMVERILOG MADE SINGLE DRIVERS THE DEFAULT FOR VARIABLES

Resolved nets make multiple drivers legal, so a copy-paste error that drives a bus from two **assign** statements simply produces **x** at run time - if the two ever disagree during the test. SystemVerilog attacks this from the other side: a variable (**logic** without a net keyword) may be written by **either** any number of procedural blocks **or** exactly one continuous driver, and **always_comb/always_ff** outputs may not be written by any other process. The tool then reports multiple drivers at compile time. This is the main reason modern RTL declares almost everything as **logic** and reserves **wire/tri** for genuinely multi-driven nets.

3.4 Drive strength and charge strength

When drivers conflict, Verilog compares **strengths** before values. Each driven value carries a strength for its 0 and its 1; the stronger driver wins, and equal strengths with opposite values give **x** (carrying that strength). Eight levels exist:

Level	Name	Keyword (0 / 1)	Kind	Typical source
7	Supply	supply0 / supply1	Drive	Power rails
6	Strong	strong0 / strong1	Drive	Default for gates and continuous assignments
5	Pull	pull0 / pull1	Drive	Pull resistors, tri0/tri1 , pullup/pulldown primitives
4	Large	large	Charge	triereg with large capacitance
3	Weak	weak0 / weak1	Drive	Weak keepers
2	Medium	medium	Charge	Default triereg charge
1	Small	small	Charge	Small triereg
0	High impedance	highz0 / highz1	-	No drive

Strengths are written in parentheses after **assign** or a gate keyword, one for 0 and one for 1. The format **%v** prints a net's strength and value as a three-character code (**St0**, **We1**, **PuX**, **HiZ**...).

```

module tb_strength;
  reg en, d;
  wire bus;
  // An open-drain style bus: a weak pull-up plus a driver that only pulls low
  assign (weak0, weak1) bus = 1'b1; // pull-up resistor model
  assign (strong0, highz1) bus = (en & ~d) ? 1'b0 : 1'b1; // '1' becomes highz1
  wire x_bus;
  assign (pull0, pull1) x_bus = 1'b0; // two equal-strength
  assign (pull0, pull1) x_bus = 1'b1; // opposite drivers
  supply0 gnd; supply1 vdd;
  initial begin
    en = 0; d = 0; #1 $display("en=0 : bus=%b %v", bus, bus);
    en = 1; d = 0; #1 $display("en=1 d=0 : bus=%b %v", bus, bus);
    en = 1; d = 1; #1 $display("en=1 d=1 : bus=%b %v", bus, bus);
    $display("pull0 vs pull1: x_bus=%b %v", x_bus, x_bus);
    $display("supply nets : gnd=%b %v vdd=%b %v", gnd, gnd, vdd, vdd);
  end
endmodule

```

An open-drain bus with a weak pull-up, a conflict between equal strengths, and supply nets (c3_str.v).

```

en=0      : bus=1 We1
en=1 d=0  : bus=0 St0
en=1 d=1  : bus=1 We1
pull0 vs pull1: x_bus=x PuX
supply nets : gnd=0 Su0 vdd=1 Su1

```

Output (Icarus Verilog 12) using %v to show strength.

The open-drain driver specifies **highz1**, so when it "drives" a 1 it actually drives nothing, and the weak pull-up makes the bus a weak 1 (**We1**). When it drives 0 at strong strength, it overpowers the pull-up (**St0**). Two pull-strength drivers of opposite value resolve to **PuX**: unknown, at pull strength - which would still lose to a strong driver.

trireg and charge storage

```

trireg (medium) #(0, 0, 50) node; // charge decays to x after 50 time units
assign node = en ? d : 1'bz;      // when en falls, node keeps d as a charge

```

A charge-storage net. Shown without output: Icarus Verilog 12 reports 'sorry: trireg nets not supported'; commercial simulators support it.

A **trireg** holds its last driven value, at its charge strength, when all drivers turn off; the optional third delay is the charge decay time after which it becomes **x**. Charge strengths (**large**, **medium**, **small**) only compete with other charge strengths; any driven value overrides them.

WHO USES STRENGTHS TODAY

RTL designers essentially never do: synthesis ignores strengths, and RTL uses one driver per signal. Strengths matter in **gate-level** and **switch-level** models - standard-cell and I/O library Verilog models, pad rings, pull-ups on bidirectional pins, analog-ish behavioural models - and in testbenches that model open-drain buses (I2C, 1-Wire, MDIO). Chapter 9 revisits them with the **nmos**/**pmos**/**tran** switch primitives.

3.5 Variables: reg, integer, real, time and realtime

Type	Width / encoding	States	Signed?	Initial value	Typical use
reg [n:0]	Any width vector	4	No, unless reg signed	x	All RTL storage and combinational outputs
integer	32 bits (at least)	4	Yes	x	Loop counters, testbench arithmetic

Type	Width / encoding	States	Signed?	Initial value	Typical use
<code>real</code>	IEEE 754 double	2 (no x/z)	Yes	0.0	Behavioural and analog models, statistics
<code>realtime</code>	Same as <code>real</code>	2	Yes	0.0	Storing <code>\$realtime</code> values
<code>time</code>	64 bits	4	No	<code>x</code>	Storing <code>\$time</code> values

- A variable declaration may include an initial value (`reg [3:0] q = 4'd5;`). In Verilog this behaves like an `initial` assignment at time 0 - whose ordering relative to other time-0 processes is not defined (Chapter 5 shows a consequence). In SystemVerilog the initialiser is applied before any process starts.
- `real` variables cannot be bit- or part-selected, cannot be used as edge-event operands (`@(posedge r)`), and are not synthesizable. Assigning a real to an integral variable **rounds** to the nearest integer, ties away from zero (Chapter 4).
- An `integer` is equivalent to `reg signed [31:0]`; use it for loop variables in Verilog-2005 (SystemVerilog code uses `int`, which is 2-state).
- In an ASIC, initial values on variables are **not** implemented (there is no power-on value); only a reset gives a register a known value. FPGA flows do honour initial values.

3.6 Vectors, bit-selects and part-selects

A vector is declared with a range `[msb:lsb]`. The **left** index is always the most significant bit, whatever its numeric value: `[7:0]` is the usual "little-endian" numbering (MSB has the highest index), `[0:7]` is "big-endian" (MSB is bit 0), and ranges need not start at zero (`[15:8]`). Values are always transferred MSB to MSB, so the index numbering matters only when you select bits.

Syntax	Meaning	Constraints
<code>v[i]</code>	Bit-select	<code>i</code> may be a variable; out of range or <code>x/z</code> index reads <code>x</code> , writes are ignored
<code>v[m:l]</code>	Constant part-select	Both bounds constant; must follow the declaration direction (<code>[7:0]</code> allows <code>v[5:2]</code> , not <code>v[2:5]</code>)
<code>v[base +: w]</code>	Indexed part-select, ascending: <code>w</code> bits starting at <code>base</code> going up	<code>w</code> must be a positive constant; <code>base</code> may be variable
<code>v[base -: w]</code>	Indexed part-select, descending: <code>w</code> bits starting at <code>base</code> going down	Same

```

module tb_vec;
    reg [7:0]  le;           // "little endian": MSB is bit 7
    reg [0:7]  be;           // "big endian"    : MSB is bit 0
    reg [31:0] word;
    reg [15:8] hi;           // ranges need not start at 0
    integer    k;
    initial begin
        le = 8'b1100_0000; be = 8'b1100_0000; // assignment is always MSB-to-MSB
        $display("le[7]=%b le[0]=%b | be[0]=%b be[7]=%b", le[7], le[0], be[0], be[7]);
        word = 32'hDEAD_BEEF;
        $display("word[15:8]=%h word[8+:8]=%h word[15 -:8]=%h",
            word[15:8], word[8+:8], word[15 -:8]);
        for (k = 0; k < 4; k = k + 1)
            $display("byte %0d = %h", k, word[k*8 +: 8]); // variable base, constant width
        be = 8'b0000_1111;
        $display("be[0+:4]=%b (bits 0..3) be[7 -:4]=%b (bits 4..7)", be[0+:4], be[7 -:4]);
        hi = 8'hA5;
        $display("hi[15]=%b hi[8]=%b out-of-range hi[3]=%b", hi[15], hi[8], hi[3]);
        k = 40;
    end
endmodule

```

```

    $display("word[k] with k=40 -> %b ; word[k+:4] -> %b", word[k], word[k+:4]);
end
endmodule

```

Endianness, indexed part-selects and out-of-range selects (c3_vec.v).

```

c3_vec.v:18: warning: Constant bit select [3] is before vector hi[15:8].
c3_vec.v:18:      : Replacing select with a constant 1'bx.
le[7]=1 le[0]=0 | be[0]=1 be[7]=0
word[15:8]=be word[8+:8]=be word[15-:8]=be
byte 0 = ef
byte 1 = be
byte 2 = ad
byte 3 = de
be[0+:4]=0000 (bits 0..3) be[7-:4]=1111 (bits 4..7)
hi[15]=1 hi[8]=1 out-of-range hi[3]=x
word[k] with k=40 -> x ; word[k+:4] -> xxxx

```

Output (Icarus Verilog 12), including its compile-time warning for the constant out-of-range select.

For a little-endian vector, $v[b +: w]$ is $v[b+w-1 : b]$ and $v[b -: w]$ is $v[b : b-w+1]$. The indexed forms exist because Verilog does not allow a part-select with two variable bounds ($v[k*8+7 : k*8]$ is illegal - the width must be constant); $v[k*8 +: 8]$ expresses the same byte lane with a constant width and a variable base, which synthesizes to a multiplexer.

PITFALL: +: ON A BIG-ENDIAN VECTOR

For `reg [0:7] be`, `be[0 +: 4]` is `be[0:3]` - the four **most** significant bits - because `+:` counts upward in index, and index 0 is the MSB of a big-endian vector. The output above shows it. Mixing endianness is legal but a rich source of byte-lane bugs; most coding standards require `[N-1:0]` everywhere, with big-endian numbering allowed only when matching a protocol specification.

Two more vector properties: the `signed` keyword (`reg signed [7:0] s;`) makes arithmetic on the vector two's complement (Chapter 4), and a selection of a vector - even of all its bits, `s[7:0]` - is always **unsigned**. The rarely used keywords `vector` and `scalared` on a net declaration tell the tool whether bit-selects of the net may be forced or driven individually.

3.7 Arrays and memories

Verilog-2001 allows **arrays** of nets or variables with any number of **unpacked** dimensions, declared after the name. A one-dimensional array of `reg` vectors is traditionally called a **memory**. The dimensions before the name (the vector range) and after it (the array range) are fundamentally different: the vector is one value you can operate on as a whole; an array in Verilog can only be accessed one element at a time.

```

module tb_mem;
  reg [7:0] mem [0:15];           // memory: 16 words of 8 bits (unpacked dimension)
  reg [7:0] cube [0:1][0:2];      // 2-D array of bytes (Verilog-2001)
  reg [3:0] nib;
  integer i, j;
  initial begin
    for (i = 0; i < 16; i = i + 1) mem[i] = i * 17; // 0x00, 0x11, 0x22, ...
    $display("mem[3]=%h mem[3][7:4]=%h mem[3][0]=%b", mem[3], mem[3][7:4], mem[3][0]);
    for (i = 0; i < 2; i = i + 1)
      for (j = 0; j < 3; j = j + 1) cube[i][j] = 8'h10 * i + j;
    $display("cube[1][2]=%h", cube[1][2]);
    $display("mem[16] (out of range read) = %h", mem[16]);
    nib = 4'bxx01;
    $display("mem[nib] with x in index = %h", mem[nib]);
    $writememh("mem.hex", mem, 0, 3);
    $readmemh("mem.hex", mem, 8, 11); // copy words 0..3 to 8..11
    $display("after readmemh: mem[8..11] = %h %h %h %h", mem[8], mem[9], mem[10], mem[11]);
  end
end

```

endmodule

Memories, a 2-D array, out-of-range and x-index reads, and \$readmemh/\$writememh (c3_mem.v).

```

c3_mem.v:12: warning: returning 'bx for out of bounds array access mem[16].
mem[3]=33 mem[3][7:4]=3 mem[3][0]=1
cube[1][2]=12
mem[16] (out of range read) = xx
mem[nib] with x in index = xx
after readmemh: mem[8..11] = 00 11 22 33

```

Output (Icarus Verilog 12).

- `mem[3][7:4]` first selects element 3, then a part-select of it; Verilog-2001 allows bit- and part-selects of array elements.
- Reading an address outside the declared range, or an address containing `x/z`, returns `x` for 4-state types; writing to such an address does nothing. Synthesized hardware has no such check - an out-of-range address aliases to some real location (Chapter 11).
- Verilog has no whole-array assignment, comparison or port connection (`mem = other;` is illegal); SystemVerilog adds all of these for unpacked arrays (Chapter 13).
- `$readmemh/$readmemb` load an array from a text file of hex/binary words (with optional `@address` lines); `$writememh/$writememb` (1364-2005) dump one. They are the standard way to initialise ROMs and instruction memories in simulation, and FPGA synthesis tools honour `$readmemh` in an `initial` block to fill block RAM (Chapter 10).

HOW A MEMORY BECOMES HARDWARE

A synthesis tool maps an array with a clocked write and a (clocked or combinational) read either to flip-flops plus multiplexers (a register file) or, in an FPGA, to block RAM. In an ASIC, large memories are never synthesized from RTL arrays: they are SRAM macros from a memory compiler, instantiated as black boxes, with the RTL array used only as the simulation model. Chapter 11 and the companion RTL Design guide cover the coding templates.

3.8 Integer literals: formats, sizing, padding and truncation

An integer literal has up to four parts: an optional **size** in bits (a decimal number), an apostrophe, an optional **s** for signed, a **base** letter (**b**, **o**, **d**, **h**, upper or lower case), and the digits, which may include **x**, **z**, **?** (a synonym for **z**) and **_** separators.

Literal	Size	Signed?	Value / rule
12	32 (at least)	Yes	A plain decimal number is a signed 32-bit integer
'd12, 'hc	32 (at least)	No	Unsize based literal: unsigned
8'd12	8	No	Sized based literal
8'sd12	8	Yes	s makes it signed (two's complement)
-8'd12	8	No	Unary minus applied to 8'd12: the bit pattern of -12 in 8 bits, but unsigned
8'hx, 8'bz	8	No	All bits x / z
4'b1?0?	4	No	? is z (useful in casez)
1.5e3, 2.0	-	-	Real literals; must have a digit on both sides of the point
"AB"	16	No	A string used as a number: 8 bits per character, first character in the MSBs

The value of a sized literal is adjusted to its declared size by three rules:

- **Padding.** If the digits specify fewer bits than the size, the value is extended on the left with 0 - unless the leftmost digit is **x** or **z**, in which case it is extended with **x** or **z**.
- **Truncation.** If the digits specify more bits than the size, the leftmost bits are dropped (tools warn).

- **Unsigned x/z.** An unsigned literal whose leftmost digit is **x** or **z** ('hx', 'bz') is extended to the full width of the expression it appears in. (In Verilog-1995 it stopped at 32 bits - a known incompatibility for wide buses.) SystemVerilog adds the fill literals '0, '1, 'x, 'z, which fill any width.

```
module tb_lit;
  reg [11:0] r;
  reg [7:0] b;
  reg signed [7:0] s;
  initial begin
    r = 8'hA5;          $display("8'hA5 into 12 bits      -> %b (zero-padded)", r);
    r = 'hx;           $display("'hx into 12 bits       -> %b (x-extended)", r);
    r = 4'bz01;        $display("4'bz01 into 12 bits    -> %b", r);
    r = 8'bx1;         $display("8'bx1                 -> %b", r);
    b = 12'hABC;        $display("12'hABC into 8 bits    -> %h (MSBs truncated)", b);
    b = 3'd9;          $display("3'd9                  -> %d (value too big for size)", b);
    s = -8'd3;         $display("-8'd3                 -> %b = %d", s, s);
    s = 4'shF;         $display("4'shF (signed, = -1)    -> %b = %d (sign-extended)", s, s);
    s = 4'hF;          $display("4'hF (unsigned)         -> %b = %d (zero-extended)", s, s);
    $display("unsigned 'd5 is %0d bits; 12 is %0d bits", $bits('d5), $bits(12));
    $display("?-as-z: 4'b1?0? = %b", 4'b1?0?);
  end
endmodule
```

Padding, truncation, x/z extension and signed literals (c3_lit.v; compiled with -g2012 for \$bits).

```
c3_lit.v:11: warning: Numeric constant truncated to 3 bits.
8'hA5 into 12 bits      -> 000010100101 (zero-padded)
'hx into 12 bits       -> xxxxxxxxxxxx (x-extended)
4'bz01 into 12 bits    -> 00000000zz01
8'bx1                  -> 0000xxxxxx1
12'hABC into 8 bits    -> bc (MSBs truncated)
3'd9                   -> 1 (value too big for size)
-8'd3                  -> 11111101 = -3
4'shF (signed, = -1)   -> 11111111 = -1 (sign-extended)
4'hF (unsigned)        -> 00001111 = 15 (zero-extended)
unsigned 'd5 is 32 bits; 12 is 32 bits
?-as-z: 4'b1?0? = 1z0z
```

Output (Icarus Verilog 12).

Read each line carefully - these are standard interview questions:

- **4'bz01**: three digits for four bits; the leftmost digit is **z**, so the literal is **zz01**. Assigned to 12 bits, that **4-bit unsigned value** is then zero-extended: the **z** extension stops at the literal's own size.
- **8'bx1** becomes **xxxxxxx1** (x-extended to 8 bits), then zero-extended to 12 bits.
- **3'd9** truncates 1001 to 001. Icarus warns; many tools do not.
- **4'shF** is the signed 4-bit value -1, so it is **sign-extended** to **11111111**; **4'hF** is unsigned 15 and zero-extended. The only difference is the **s**.
- Unsigned literals are 32 bits wide in every tool used here (the LRM says "at least 32"), which is why **'d5** in a 64-bit context is safe but **'hFFFF_FFFF_F** (36 bits of digits, unsigned) is not portable.

PITFALL: A NEGATIVE LITERAL IS NOT A SIGNED LITERAL

-8'd3 is unary minus applied to the **unsigned** literal **8'd3**. Its bit pattern is **11111101**, but the expression is unsigned, so in a wider context it zero-extends to 253, not -3. For a genuinely negative constant write **-8'sd3** or use a plain decimal **-3** (which is signed 32-bit). Chapter 4 shows how one unsigned operand turns a whole expression unsigned.

3.9 Summary

- Nets model connections and are driven continuously; variables store values written by procedural code. `reg` is a variable, not necessarily a register.
- Values are 0, 1, x (unknown) and z (undriven). x is a simulation artefact that synthesis treats as don't-care.
- Net types differ in how they resolve multiple drivers: wire/tri (conflict -> x), wand/wor, tri0/tri1 (pulled), supply0/1, trireg (charge), uwire (single driver enforced).
- Strength levels (supply > strong > pull > weak, plus charge levels) decide conflicts before values; used in gate-level, pad and open-drain models.
- Vector ranges fix which index is the MSB; `+:/-:` give variable-base, constant-width part-selects; any select is unsigned; out-of-range reads return x.
- Arrays are accessed element by element in Verilog; `$readmemh` loads them.
- Literals: plain decimal is signed 32-bit; based literals are unsigned unless marked `s`; values are zero-padded (x/z-padded if the leftmost digit is x/z) or truncated to their size.

3.10 Exercises

- 1 Write the resolution table for a `wand` and a `wor` net with two drivers, and check it against the output in Section 3.3.
- 2 A bus has a `pull1` pull-up, a device driving `strong0` and a second device driving `weak1`. What is the resolved value and strength? Verify with `%v`.
- 3 Given `reg [31:0] w = 32'h12345678;` and `integer k = 2;`, evaluate `w[k*8 +: 8]`, `w[31 -: 4]`, `w[k]`, and `w[k*16 +: 16]`. Which of them is illegal, if any, and why?
- 4 What values do `12'hx5`, `12'h5x`, `6'o7z`, `4'b1` and `'bz` take when assigned to a 16-bit `reg`?
- 5 Declare a 256 x 32-bit memory, load it from a hex file with `$readmemh`, and write a loop that computes a checksum of its words. What does the checksum become if one line of the file is missing, and why?
- 6 Explain the difference between `reg signed [7:0] a = -3;` and `reg [7:0] b = -3;` when each is added to a 16-bit `reg signed` variable holding 0.

4. Operators and Expressions: Sizing, Signedness and Evaluation Rules

Verilog's operators look like C's, and that is the trap. In C, an `int` is an `int`. In Verilog, every operand has an exact bit width, a signedness and four-state bits, and the language silently extends, truncates and re-interprets them according to rules that are precise but not intuitive. More RTL bugs, more lint warnings and more interview questions come from this chapter than from any other. Every rule below is demonstrated with a simulation, including the surprising results - learn to predict them before reading the output.

4.1 The operators and their precedence

Precedence	Operators	Kind
1 (highest)	<code>+ - ! ~ & ~& ~ ^ ~^ ^~</code> (unary)	Unary arithmetic, logical and bitwise negation, reduction
2	<code>**</code>	Power
3	<code>* / %</code>	Multiplicative
4	<code>+ -</code> (binary)	Additive
5	<code><< >> <<< >>></code>	Logical and arithmetic shifts
6	<code>< <= > >=</code> (SV adds <code>inside, dist</code>)	Relational
7	<code>== != === !==</code> (SV adds <code>==? !=?</code>)	Equality: logical, case (4-state), wildcard
8	<code>&</code> (binary)	Bitwise AND
9	<code>^ ~^ ^~</code> (binary)	Bitwise XOR / XNOR
10	<code> </code> (binary)	Bitwise OR
11	<code>&&</code>	Logical AND
12	<code> </code>	Logical OR
13	<code>?:</code>	Conditional (right-associative)
(SV) 14	<code>-> <-></code>	Logical implication / equivalence
(lowest)	<code>{ } { }</code>	Concatenation and replication are delimiters, not ranked operators

Operator precedence (IEEE 1364-2005 Table 5-4, IEEE 1800-2017 Table 11-2). All binary operators are left-associative; `?:` is right-associative.

```

module tb_prec;
  reg [3:0] a, b, c;
  initial begin
    a = 4'd3; b = 4'd2; c = 4'd2;
    $display("a & b == c    -> %b  (== binds tighter: a & (b==c))", a & b == c);
    $display("(a & b) == c    -> %b", (a & b) == c);
    $display("a | b ^ c      -> %b  (^ before |)", a | b ^ c);
    $display("a + b << 1      -> %0d    (+ before <<)", a + b << 1);
    $display("!a == 0        -> %b    ((!a) == 0)", !a == 0);
    $display("-2 ** 2        -> %0d    (unary minus binds tighter than **)", -2 ** 2);
    $display("2 ** 3 ** 2      -> %0d    (** is left-associative in 1364/1800-2017)", 2 ** 3 ** 2);
    $display("a ? b : c ? 4'd7 : 4'd8 -> %0d (?: is right-associative)", a ? b : c ? 4'd7 : 4'd8);
  end
endmodule

```

Precedence surprises (c4_prec.v).

```

a & b == c      -> 0001  (== binds tighter: a & (b==c))
(a & b) == c    -> 1
a | b ^ c       -> 0011  (^ before |)
a + b << 1      -> 10    (+ before <<)
!a == 0         -> 1     ((!a) == 0)
-2 ** 2         -> 4     (unary minus binds tighter than **)
2 ** 3 ** 2     -> 64    (** is left-assoc in 1364/1800-2017)
a ? b : c ? 4'd7 : 4'd8 -> 2  (?: is right-assoc)

```

Output (Icarus Verilog 12).

- `a & b == c` parses as `a & (b == c)`: equality binds more tightly than bitwise AND - the reverse of what most people expect, and inherited from C. The 1-bit result of `==` is zero-extended to four bits before the AND.
- `a | b ^ c` is `a | (b ^ c)`: XOR binds more tightly than OR.
- `-2 ** 2` is 4, because unary minus has the highest precedence of all.
- `2 ** 3 ** 2` is `(2 ** 3) ** 2 = 64` because the standards list `**` as left-associative, unlike mathematics and Python (512).
- `?:` is right-associative, so chains of conditionals read as if/else-if ladders.

PARENTHESISE

Nobody reviewing your code should need this table. Put parentheses around every mix of bitwise, equality and logical operators, around shifts inside arithmetic, and around every chained `**`. Lint tools flag many of these (for example Verilator's WIDTH warnings show where the `==` result is being extended).

4.2 Arithmetic operators, division, modulus and power

`+` `-` `*` `/` `%` work on integral operands of any width. The rule for unknowns is blunt: **if any bit of any operand is x or z, the entire result is x**. The arithmetic itself is performed in the width and signedness determined by Sections 4.8 and 4.9; overflow simply wraps.

```

module tb_div;
  integer i;
  reg [3:0] u;
  initial begin
    $display(" 7/2=%0d  -7/2=%0d  7/-2=%0d  -7/-2=%0d", 7/2, -7/2, 7/-2, -7/-2);
    $display(" 7%%2=%0d  -7%%2=%0d  7%-2=%0d  -7%-2=%0d (sign follows the dividend)",
              7%2, -7%2, 7%-2, -7%-2);
    u = 4'd9;
    $display(" u/0=%b  u%%0=%b (division by zero gives x)", u/4'd0, u%4'd0);
    $display(" 2**10=%0d  2**-1=%0d  (-2)**3=%0d  (-1)**-3=%0d  0**-1=%0d",
              2**10, 2**-1, (-2)**3, (-1)**-3, 0**-1);
    $display(" 3'd7**2 in 8 bits: %0d ; 4'd3 ** 2'd3 = %0d",
              {8'd0 + 3'd7**2}, 4'd3 ** 2'd3);
    $display(" 2.0**0.5 = %f  7.0/2 = %f  7/2.0 = %f", 2.0**0.5, 7.0/2, 7/2.0);
    i = 6.5; $write(" real->integer rounds half away from zero: 6.5->%0d", i);
    i = -2.5; $display(" -2.5->%0d", i);
  end
endmodule

```

Integer division and modulus with negative operands, division by zero, the power operator and reals (c4_div.v).

```

7/2=3  -7/2=-3  7/-2=-3  -7/-2=3
7%2=1  -7%2=-1  7%-2=1  -7%-2=-1 (sign follows the dividend)
u/0=xxxx  u%%0=xxxx (division by zero gives x)
2**10=1024  2**-1=0  (-2)**3=-8  (-1)**-3=-1  0**-1=x
3'd7**2 in 8 bits: 49 ; 4'd3 ** 2'd3 = 11
2.0**0.5 = 1.414214  7.0/2 = 3.500000  7/2.0 = 3.500000
real->integer rounds half away from zero: 6.5->7  -2.5->-3

```

Output (Icarus Verilog 12).

- Integer division **truncates toward zero** ($-7/2 = -3$), and the result of `%` takes the **sign of the first operand** ($-7 \% 2 = -1$, $7 \% -2 = 1$). This matches C99, but not Python, whose `//` and `%` floor toward minus infinity - a classic mismatch between a Python reference model and RTL.
- Division or modulus by zero yields `x` for integral types.
- The power operator's result has the width of its **left** operand (context-determined); the right operand is self-determined. That is why `4'd3 ** 2'd3` in a self-determined context prints 11: 27 does not fit in four bits.
- For integer operands, `**` with a negative exponent follows a small table: `0 ** -n` is `x`, `1 ** -n` is 1, `(-1) ** -n` is +1 or -1 (by parity), anything else to a negative power is 0.
- If either operand is `real`, the operation is performed in real arithmetic ($7/2.0 = 3.5$). A real assigned to an integral variable rounds to nearest, ties away from zero.

PITFALL: DIVISION IN RTL

`/` and `%` by a non-power-of-two are synthesizable in principle, but produce a huge combinational divider that will rarely meet timing; most coding standards forbid them in RTL except with constant power-of-two divisors (which become shifts and masks) or in constant expressions evaluated at elaboration. A real design uses a multi-cycle divider (see the companion RTL Design guide). `**` is likewise restricted to constant operands or a power-of-two base (`2 ** n` is a shift).

4.3 Equality, relational and logical operators

There are two families of equality. **Logical equality** `==/!=` models hardware comparators: it returns `x` whenever an `x` or `z` bit could affect the answer. **Case equality** `===/!==` compares all four states literally and always returns 0 or 1. Relational operators `< <= > >=` behave like `==`: any `x/z` bit gives `x`.

```
module tb_eq;
  reg [3:0] a, b;
  initial begin
    a = 4'b10x1; b = 4'b10x1;
    $display("a==b -> %b   a===b -> %b   a!=b -> %b   a!==b -> %b",
             a==b, a===b, a!=b, a!==b);
    a = 4'b10x1; b = 4'b0000;
    $display("a==0 -> %b   (a differs in a known bit, so result is known)", a==b);
    a = 4'b0101; b = 4'b0011;
    $display("logical  a&&b=%b  a||b=%b  !a=%b", a && b, a || b, !a);
    $display("bitwise  a&b=%b  a|b=%b  a^b=%b  ~a=%b  a~^b=%b", a & b, a | b, a ^ b, ~a, a ~^ b);
    $display("reduce   &a=%b  |a=%b  ^a=%b  ~&a=%b  ~|a=%b  ~^a=%b", &a, |a, ^a, ~&a, ~|a, ~^a);
    a = 4'b00x0;
    $display("x cases  |a=%b  &a=%b  a&&1=%b  !a=%b  4'b10x0||0=%b",
             |a, &a, a && 1'b1, !a, 4'b10x0 || 1'b0);
    if (a == 4'b0000) $display("if (x) took the THEN branch");
    else               $display("if (a == 0) with x -> condition is x -> ELSE branch taken");
  end
endmodule
```

Equality with unknowns, and logical, bitwise and reduction operators side by side (c4_eq.v).

```
a==b -> x   a===b -> 1   a!=b -> x   a!==b -> 0
a==0 -> 0   (a differs in a known bit, so result is known)
logical  a&&b=1  a||b=1  !a=0
bitwise  a&b=0001 a|b=0111 a^b=0110 ~a=1010 a~^b=1001
reduce   &a=0  |a=1  ^a=0  ~&a=1  ~|a=0  ~^a=1
x cases  |a=x  &a=0  a&&1=x  !a=x  4'b10x0||0=1
if (a == 0) with x -> condition is x -> ELSE branch taken
```

Output (Icarus Verilog 12).

Operator	Result width	x/z handling	Synthesizable?
<code>==, !=</code>	1 bit	x if any x/z bit could decide the result; known if a known bit differs (<code>4'b10x1 == 4'b0000</code> is 0)	Yes: comparator
<code>===, !==</code>	1 bit	Exact 4-state match, never x	No (tools treat x/z literally or error); testbench only
<code>==?, !=? (SV)</code>	1 bit	x/z bits in the right operand are wildcards	Yes with constant wildcards
<code>< <= > >=</code>	1 bit	x if any operand bit is x/z	Yes
<code>&& !</code>	1 bit	Operands reduced to true/false/unknown: a vector is true if any bit is 1, false if all bits are 0, else x	Yes

IF (X) TAKES THE ELSE BRANCH

An **if** condition that evaluates to **x** or **z** is treated as **false**: the **else** branch executes. This is X-optimism in its purest form. Hardware would pick one branch or the other; the simulator always picks the **else**, which can hide an uninitialised control signal. The same holds for **while**, **?:** is different (Section 4.7), and **case** has its own rules (Chapter 6). Use `===` or `$isunknown` (SV) in testbench checks, and assertions such as `assert (!$isunknown(sel))` in RTL to catch unknown controls.

4.4 Bitwise, reduction and logical operators

The same symbols play three different roles, and confusing them is a top source of bugs:

Kind	Syntax	Operands -> result	Example (a=0101, b=0011)
Bitwise	<code>a & b, a b, a ^ b, ~a, a ~^ b</code>	Vectors -> vector, bit by bit	<code>a & b = 0001, ~a = 1010</code>
Reduction	<code>&a, a, ^a, ~&a, ~ a, ~^a</code>	One vector -> 1 bit, folding all its bits	<code>&a = 0, a = 1, ^a = 0</code> (even parity)
Logical	<code>a && b, a b, !a</code>	Operands as truth values -> 1 bit	<code>a && b = 1, !a = 0</code>

The 4-state truth tables of the bitwise operators follow the "dominant value" principle: 0 dominates AND, 1 dominates OR, and XOR is unknown whenever either input is unknown. A **z** input behaves as **x**.

&	0	1	x	z
0	0	0	0	0
1	0	1	x	x
x	0	x	x	x
z	0	x	x	x

Bitwise AND.

 	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

Bitwise OR.

^	0	1	x	z
0	0	1	x	x

$\wedge$	0	1	x	z
1	1	0	x	x
x	x	x	x	x
z	x	x	x	x

Bitwise XOR (XNOR is its complement; $\sim x = x$).

The output line **x cases** in the previous run shows the consequences for reduction and logical operators: for **a** = **4'b00x0**, **|a** is **x** (the unknown bit could be the only 1), but **&a** is 0 (a known 0 decides the AND). **4'b10x0 || 0** is 1 because the vector has a known 1 bit and is therefore true.

PITFALL: ! VERSUS ~ ON VECTORS

if (!status) tests whether the whole vector is zero; **if (~status)** inverts every bit and then tests whether the *result* is non-zero - true unless **status** is all ones. For a 1-bit signal the two agree, which is why the bug survives until someone widens the signal. Use **!** (or an explicit **== 0**) for conditions and **~** for data.

4.5 Shift operators

Operator	Name	Vacated bits filled with
<<	Logical left shift	0
>>	Logical right shift	0
<<<	Arithmetic left shift	0 (identical to <<)
>>>	Arithmetic right shift	Copies of the MSB if the expression is signed, otherwise 0

```

module tb_shift;
  reg      [7:0] u;
  reg signed [7:0] s;
  initial begin
    u = 8'b1001_0110; s = 8'sb1001_0110; // s = -106
    $display("u >> 2 = %b u >>> 2 = %b (unsigned: both logical)", u >> 2, u >>> 2);
    $display("s >> 2 = %b s >>> 2 = %b (signed: >>> copies the sign)", s >> 2, s >>> 2);
    $display("s >>> 2 = %0d (-106/4 = -26.5 -> floor -> -27)", s >>> 2);
    $display("s <<< 1 = %b same as s << 1", s <<< 1);
    $display("u >> 9 = %b u << 4'bx = %b", u >> 9, u << 4'bx);
    $display("{u[3:0], u[7:4]} = %b {3{2'b10}} = %b", {u[3:0], u[7:4]}, {3{2'b10}});
    $display("{2{u[7]}, u[7:2]} (sign-ext manual) = %b", {2{u[7]}, u[7:2]});
  end
endmodule

```

Logical and arithmetic shifts, concatenation and replication (c4_shift.v).

```

u >> 2 = 00100101 u >>> 2 = 00100101 (unsigned: both logical)
s >> 2 = 00100101 s >>> 2 = 11100101 (signed: >>> copies the sign)
s >>> 2 = -27 (-106/4 = -26.5 -> floor -> -27)
s <<< 1 = 00101100 same as s << 1
u >> 9 = 00000000 u << 4'bx = xxxxxxxx
{u[3:0], u[7:4]} = 01101001 {3{2'b10}} = 101010
{2{u[7]}, u[7:2]} (sign-ext manual) = 11100101

```

Output (Icarus Verilog 12).

- >>>** is an arithmetic shift only when the left operand - more precisely, the whole expression it belongs to - is **signed**. On the unsigned **u** it behaves exactly like **>>**. This is the most common reason for a broken fixed-point scaler.
- An arithmetic right shift is division by a power of two that rounds toward **minus infinity** ($-106 >>> 2 = -27$), whereas **/** truncates toward zero ($-106 / 4 = -26$). A reference model must use the same rounding as the RTL.

- The shift amount is always treated as unsigned and is self-determined; shifting by more than the width gives 0 (or all sign bits), and a shift amount containing `x/z` gives an all-`x` result.
- A shift by a constant is free in hardware (wiring); a shift by a variable is a barrel shifter (log2 N levels of multiplexers).

4.6 Concatenation and replication

`{a, b, c}` joins operands MSB-first into one vector whose width is the sum of their widths; `{n{a}}` repeats `a` `n` times. Rules:

- Every operand of a concatenation must be **sized**: `{a, 1}` is illegal because `1` has no defined width; write `{a, 1'b1}`.
- The replication count must be a non-negative constant expression. A count of zero is legal only inside a larger concatenation with at least one positive-width operand (1364-2005; SystemVerilog clarifies the same rule), which is useful in parameterized code.
- Operands are **self-determined**: an expression inside `{}` is evaluated in its own width, which is how `{a + b}` loses the carry in Section 4.8.
- The result of a concatenation is always unsigned.
- A concatenation may be the target of an assignment: `{cout, sum} = a + b + cin;` splits the result, the idiom used in Chapter 1.

Replication of the sign bit is the manual way to sign-extend: `{{2{u[7]}}, u[7:2]}` above produces the same bits as `s >>> 2`. Note the doubled braces: the replication `2{u[7]}` is itself an operand of the outer concatenation.

4.7 The conditional operator and unknown conditions

`c ? a : b` selects `a` if `c` is true and `b` if it is false. If `c` is `x` or `z`, the LRM requires both `a` and `b` to be evaluated and combined **bit by bit**: where the corresponding bits are equal the result takes that value, otherwise it is `x`.

```
module tb_cond;
  reg      sel;
  reg [3:0] a, b;
  initial begin
    a = 4'b1100; b = 4'b1010;
    sel = 1'b0; $display("sel=0 : %b", sel ? a : b);
    sel = 1'b1; $display("sel=1 : %b", sel ? a : b);
    sel = 1'bx; $display("sel=x : %b   (bits where a and b agree survive)", sel ? a : b);
    sel = 1'bz; $display("sel=z : %b", sel ? a : b);
  end
endmodule
```

The conditional operator with an unknown select (c4_cond.v).

```
sel=0 : 1010
sel=1 : 1100
sel=x : 1xx0   (bits where a and b agree survive)
sel=z : 1xx0
```

Output (Icarus Verilog 12).

This is a realistic model of a multiplexer with an unknown select: if both inputs agree, the output does not depend on the select. It is therefore **less** optimistic than `if (sel) y = a; else y = b;`, which would silently return `b`. This difference - `?:` merges, `if` picks else - is an important one in X-propagation analysis and is why some teams prefer `?:` (or **unique case** with assertions) for data-path muxes. Commercial simulators also offer X-propagation modes that make `if/case` behave like `?:` (Chapter 11).

4.8 Expression bit-length rules

This is the heart of the chapter. Verilog evaluates every expression at a single width, chosen **before** evaluation from the widths of the operands and, for an assignment, of the left-hand side. Each operand is either **context-determined** (it takes part in choosing the width and is extended to it) or **self-determined** (it is evaluated in its own width and then used as a value).

Expression	Bit length	Notes
Unsigned constant	Same as integer (32)	
Sized constant	As given	
i op j, op is + - * / % & ^ ^~ ~^	max(L(i), L(j))	Both operands context-determined
op i, op is + - ~	L(i)	Context-determined
i op j, op is === !== == != > >= < <=	1 bit	Operands sized to max(L(i), L(j)) between themselves
i op j, op is && 	1 bit	Each operand self-determined
op i, op is & ~& ~ ^ ~^ ^~ !	1 bit	Operand self-determined
i op j, op is >> << >>> <<< **	L(i)	j is self-determined
i ? j : k	max(L(j), L(k))	i is self-determined
{i, ..., j}	L(i) + ... + L(j)	All self-determined
{i{j}, ..., k{j}}	i x (L(j) + ... + L(k))	All self-determined

Bit lengths of expressions (IEEE 1364-2005 Table 5-22; the same table appears as Table 11-21 in IEEE 1800-2017).

The width of an expression is found in two passes. **Upward**: from the leaves, compute the maximum width of all context-determined operands, including the left-hand side of an assignment. **Downward**: extend every context-determined operand to that width, then evaluate. Finally, the result is truncated (or extended) to the left-hand side. Crucially, the left-hand side takes part in the first pass - which is why `sum9 = a + b` keeps its carry.

```

module tb_size;
  reg [7:0] a, b;
  reg [8:0] sum9;
  reg [7:0] avg8;
  reg [15:0] w;
  initial begin
    a = 8'd200; b = 8'd100;
    sum9 = a + b;
    $display("1) sum9 = a + b           = %0d (context is 9 bits)", sum9);
    $display("2) a + b in $display      = %0d (self-determined: 8 bits)", a + b);
    $display("3) {a + b}                  = %0d (concat operand: self-determined)", {a + b});
    avg8 = (a + b) >> 1;
    $display("4) avg8 = (a+b)>>1           = %0d (8-bit context: carry lost)", avg8);
    avg8 = (a + b + 9'd0) >> 1;
    $display("5) avg8 = (a+b+9'd0)>>1       = %0d (a 9-bit operand widens all)", avg8);
    w = a * b;
    $display("6) w = a * b                     = %0d (16-bit context)", w);
    w = {a * b};
    $display("7) w = {a * b}                   = %0d (20000 mod 256)", w);
    $display("8) (a + b) > 8'd255                = %b (compared in 8 bits)", (a + b) > 8'd255);
    $display("9) (a + b) > 9'd255                = %b (9 bits: carry kept)", (a + b) > 9'd255);
    $display("10) ~a == 8'd55                    = %b (8-bit compare)", ~a == 8'd55);
    $display("11) ~a == 9'd55                    = %b (a widened to 9 bits BEFORE ~)", ~a == 9'd55);
  end
endmodule

```

Eleven sizing cases with a = 200 and b = 100 (c4_size.v).


```

1) sum9 = a + b           = 300 (context is 9 bits)
2) a + b in $display      = 44  (self-determined: 8 bits)
3) {a + b}                = 44  (concat operand: self-determined)
4) avg8 = (a+b)>>1        = 22  (8-bit context: carry lost)
5) avg8 = (a+b+9'd0)>>1   = 150 (a 9-bit operand widens all)
6) w = a * b              = 20000 (16-bit context)
7) w = {a * b}            = 32  (20000 mod 256)
8) (a + b) > 8'd255        = 0   (compared in 8 bits)
9) (a + b) > 9'd255        = 1   (9 bits: carry kept)
10) ~a == 8'd55            = 1   (8-bit compare)
11) ~a == 9'd55            = 0   (a widened to 9 bits BEFORE ~)

```

Output (Icarus Verilog 12).

Case	Why
1) 300	LHS is 9 bits, so a and b are extended to 9 bits before the add; the carry is kept
2) 44	Arguments of \$display (and of any system task) are self-determined: 8 bits, 300 mod 256 = 44
3) 44	Concatenation operands are self-determined
4) 22	>> does not widen its left operand beyond the context: the context is max(8, 8, LHS 8) = 8, so the carry is lost before the shift - the classic averaging bug
5) 150	A 9-bit operand anywhere in the context widens every operand to 9 bits; the carry survives the shift
6) 20000	The 16-bit LHS widens a and b before the multiply
7) 32	Braces make the product self-determined: 8 bits
8) 0	A comparison sizes its operands only among themselves: max(8, 8, 8) = 8, so a+b is 44
9) 1	9'd255 makes the comparison 9 bits wide
10) 1	8-bit compare: ~200 = 55
11) 0	a is extended to 9 bits before ~ is applied, so ~a = 9'b1_0011_0111 = 311, not 55

INTERVIEW QUESTION: WHY DOES ~A == 9'D55 FAIL?

Case 11 is the favourite trick question on sizing. The unary **~** is context-determined, so its operand is extended to the context width (9 bits, from **9'd55**) first - with a 0 - and the inversion then turns that 0 into a 1. The same happens with **-a**. Whenever an inverted or negated signal is compared against, or assigned to, something wider, the extra MSBs are ones. Lint tools report it as a width mismatch; the fix is to make widths equal or to isolate the operand: **{~a} == 9'd55**, or **8'(~a)** in SystemVerilog.

RULES OF THUMB FOR CORRECT WIDTHS

(1) Size the left-hand side to hold the full result (N+1 bits for an N-bit add, 2N for a multiply). (2) Never rely on the LHS to widen an intermediate that is shifted, compared or concatenated - widen explicitly: **({1'b0, a} + {1'b0, b}) >> 1**. (3) Treat every lint WIDTH warning as a bug until proven otherwise.

4.9 Signedness rules

Verilog-2001 added signed arithmetic via the **signed** keyword, signed literals (**'s**), **\$signed()** and **\$unsigned()**. The rules deciding whether an expression is evaluated as signed are short, strict and the cause of countless bugs:

- The signedness of an expression depends **only on its operands**, never on the left-hand side of the assignment.
- **If any operand is unsigned, the whole expression is unsigned.** Only when all context-determined operands are signed is the operation signed.
- Bit-selects, part-selects (even of the whole vector), concatenations, and the results of comparison and reduction operators are **unsigned**.

- Based literals are unsigned unless they carry **s**; plain decimal numbers (**12**, **-3**) and **integer** variables are signed.
- Once the expression type is known it is propagated down to the context-determined operands, which are then extended: sign-extended if the expression is signed, zero-extended if not - regardless of each operand's own declaration.
- **\$signed(x)** and **\$unsigned(x)** return the same bits with the other interpretation; they do not change the width.

```

module tb_sign;
  reg signed [7:0]  s;
  reg      [7:0]  u;
  reg signed [15:0] r;
  reg      [3:0]  n;
  integer i;
  initial begin
    s = -8'sd4;  u = 8'd4;
    $display("1) s < u  (s=-4, u=4)   -> %b  (u unsigned => s read as 252)", s < u);
    $display("2) s < $signed(u)       -> %b", s < $signed(u));
    r = s;          $display("3) r = s                -> %0d", r);
    r = s + u;      $display("4) r = s + u            -> %0d (unsigned: s zero-ext)", r);
    r = s + $signed(u); $display("5) r = s + $signed(u) -> %0d", r);
    n = 4'd3;
    r = s + n;      $display("6) r = s + n (n=3)       -> %0d", r);
    r = s + $signed(n); $display("7) r = s + $signed(n) -> %0d", r);
    n = 4'b1101;
    r = s + $signed(n); $display("8) n=13: s + $signed(n) -> %0d (4'b1101 read as -3)", r);
    r = s + $signed({1'b0, n});
    $display("9) s + $signed({1'b0,n})-> %0d (n kept positive)", r);
    r = s * 2'sd3;  $display("10) s * 2'sd3           -> %0d  (2'sd3 is -1)", r);
    r = s[7:0];     $display("11) r = s[7:0]          -> %0d (part-select: UNSIGNED)", r);
    i = -12 / 3;    $display("12) -12 / 3             -> %0d", i);
    i = -'d12 / 3;  $display("13) -'d12 / 3           -> %0d", i);
    i = -4'sd12 / 3; $display("14) -4'sd12 / 3         -> %0d  (4'sd12 is -4)", i);
  end
endmodule

```

Fourteen signedness cases with s = -4 (signed) and u = 4 (unsigned) (c4_sign.v).

```

1) s < u  (s=-4, u=4)   -> 0  (u unsigned => s read as 252)
2) s < $signed(u)       -> 1
3) r = s                -> -4
4) r = s + u            -> 256 (unsigned: s zero-ext)
5) r = s + $signed(u)   -> 0
6) r = s + n (n=3)      -> 255
7) r = s + $signed(n)   -> -1
8) n=13: s + $signed(n) -> -7 (4'b1101 read as -3)
9) s + $signed({1'b0,n})-> 9 (n kept positive)
10) s * 2'sd3           -> 4  (2'sd3 is -1)
11) r = s[7:0]          -> 252 (part-select: UNSIGNED)
12) -12 / 3             -> -4
13) -'d12 / 3           -> 1431655761
14) -4'sd12 / 3         -> 1  (4'sd12 is -4)

```

Output (Icarus Verilog 12).

Case	Explanation
1) 0	u is unsigned, so the comparison is unsigned: s is read as 252, and 252 < 4 is false
3) -4	All operands signed; s is sign-extended to 16 bits
4) 256	u makes the add unsigned; s is zero -extended to 16 bits (252) and 252 + 4 = 256
5) 0	\$signed(u) makes all operands signed: -4 + 4

Case	Explanation
6) 255	Same trap with a 4-bit <code>n</code> : <code>252 + 3</code>
8) -7	<code>\$signed(4'b1101)</code> is -3: casting reinterprets the bits, it does not add a sign bit
9) 9	Prepending a 0 (<code>{1'b0, n}</code>) before <code>\$signed</code> keeps a positive unsigned value positive - the correct way to mix signed and unsigned operands
10) 4	<code>2'sd3</code> is the two-bit signed pattern 11, i.e. -1
11) 252	A part-select is unsigned even if it covers the whole signed vector
12-14	<code>-12/3</code> is signed; <code>- 'd12</code> is an unsigned 32-bit pattern (4294967284) divided by 3; <code>4'sd12</code> is the 4-bit signed pattern 1100 = -4, negated to 4 and divided to 1

PITFALL: ONE UNSIGNED OPERAND POISONS THE EXPRESSION

Mixing a signed coefficient with an unsigned sample, a signed accumulator with an unsigned `count`, or a signed value with an unsigned-but-based constant (`acc + 'd1`) silently turns the whole computation unsigned. The simulator gives no warning; the result is wrong only for negative values. Discipline: make every operand of a signed expression signed, convert unsigned operands with `$signed({1'b0, x})`, use plain decimal or `'sd` constants, and watch lint warnings about signed/unsigned mixing.

4.10 Evaluation order and short-circuiting

- The LRM does not define the order in which the operands of most operators are evaluated. That matters only when an operand has a side effect, for example a function call that modifies a variable - so do not write such expressions.
- `&&`, `||` and `?:` short-circuit: IEEE 1800 requires that the right operand of `&&` is not evaluated when the left is false (and similarly for `||` when the left is true), which matters for function calls with side effects and for `null` handle checks in SystemVerilog testbenches (`if (h != null && h.valid)`).
- Integer expressions in constant contexts (parameters, ranges) are evaluated at elaboration time with exactly the same sizing and signedness rules - a parameter computed as `WIDTH-1` where `WIDTH` is an unsigned decimal is a signed 32-bit value.

4.11 Summary

- Know the precedence traps: `==` binds tighter than `&`, `^` tighter than `|`, unary minus tighter than `**`; `**` is left-associative; `?:` is right-associative. Parenthesise.
- Arithmetic with any `x/z` bit gives all-`x`; `/` truncates toward zero, `%` takes the sign of the dividend, division by zero gives `x`.
- `==` returns `x` on unknowns, `===` never does; an `if` on `x` takes the else branch; `?:` with an `x` select merges both inputs.
- Bitwise, reduction and logical operators share symbols but not semantics; use `!` for conditions and `~` for data.
- `>>>` is arithmetic only in a signed expression, and rounds toward minus infinity.
- Expression width is the maximum of all context-determined operands and the LHS; concatenation operands, system-task arguments, shift amounts and conditions are self-determined; unary `~/-` extend before operating.
- One unsigned operand makes the whole expression unsigned; selects and concatenations are unsigned; `$signed/$unsigned` reinterpret without resizing.

4.12 Exercises

- With `reg [3:0] a = 4'b1010`, `b = 4'b0110`; evaluate: `a & b`, `a && b`, `&a`, `a ^ b`, `^a`, `!a`, `~a`, `a | b` == `4'b1110`, `(a | b) == 4'b1110`.
- `reg [7:0] x = 8'hF0`; `reg [15:0] y`; What is `y` after each of `y = x << 4`; `y = {x << 4}`; `y = (x << 4) >> 4`; `y = ~x`; `y = -x`;

- 3 Write a signed 8-bit average `avg = (p + q) / 2` that is correct for all inputs, where `p` and `q` are `reg signed [7:0]`. Then write it with `>>>` and explain why the two differ for negative odd sums.
- 4 A designer writes `if (count - 1 < 0)` with `reg [3:0] count`. Explain why the branch is never taken, and give two fixes.
- 5 Explain why `$display("%0d", a + b)` and `sum = a + b; $display("%0d", sum)` can print different values, citing the rule from the bit-length table.
- 6 Predict, then verify, the result of `-4'sd1 >>> 1`, `$unsigned(-4'sd1) >> 1`, `4'sb1000 / -1` and `8'sd100 + 8'sd100` in an 8-bit signed context.

5. Assignments, Procedural Blocks, Timing Controls and the Event Scheduler

A Verilog model is a collection of **processes** that wake up, compute, update values and go back to sleep, all coordinated by an **event scheduler** that advances simulated time. This chapter explains the two families of assignment - continuous and procedural - the blocking and non-blocking forms, every kind of timing control, and the stratified event queue that gives them meaning. It ends with the races that appear when code depends on details the standard leaves unspecified, demonstrated by running the same source on two simulators and getting different answers.

5.1 Processes

Process	Starts	Runs	Typical use
Continuous assignment <code>assign y = f(x);</code>	Time 0	Re-evaluates whenever an operand changes	Combinational logic, wiring
Gate / primitive instance	Time 0	Re-evaluates on input change	Netlists (Chapter 9)
initial block	Time 0	Once, from top to bottom, then terminates	Testbench stimulus, initialisation
always block	Time 0	Forever: at the end it loops back to the top	Flip-flops, combinational logic, clock generators
Port connection	-	Behaves as a continuous assignment	-
(SV) <code>always_comb</code>, <code>always_ff</code>, <code>always_latch</code>, <code>final</code>, <code>forked threads</code>	See Chapters 14 and 20		

All processes run concurrently. Inside one **initial** or **always** block, statements execute sequentially until the block reaches a **timing control** (`#`, `@`, `wait`), where it suspends and lets other processes run. An **always** block without any timing control is an infinite zero-delay loop that hangs the simulator at time 0: `always clk = ~clk;` is the classic example (the fix is `always #5 clk = ~clk;`).

5.2 Continuous assignments

A continuous assignment drives a **net** (in SystemVerilog also a variable, as its single driver) with the value of an expression, permanently. It can be written as a separate `assign` statement or folded into the net declaration (a **net declaration assignment**: `wire y = a & b;`). A net declaration assignment is legal only once per net and is *not* an initialisation - it is a driver, exactly like `assign`. (For a variable, `reg r = 0;` is an initialisation - the same syntax means two different things.)

A delay on a continuous assignment models a gate's propagation delay with **inertial** semantics: when the right-hand side changes, the update is scheduled `d` time units later, and any pending update is cancelled. Pulses shorter than the delay therefore vanish. Up to three delays can be given: rise (to 1), fall (to 0) and turn-off (to `z`); a transition to `x` uses the smallest.

```

`timescale 1ns/1ns
module tb_ca;
    reg a = 0;
    wire #3 y_net = a;          // net declaration assignment with a 3 ns delay
    wire y_ca;
    assign #3 y_ca = a;         // continuous assignment with a 3 ns delay
    wire #(2,4) y_rf = a;       // rise delay 2, fall delay 4
    initial begin
        $monitor("t=%2t a=%b y_net=%b y_ca=%b y_rf=%b", $time, a, y_net, y_ca, y_rf);
        #5 a = 1;               // long pulse: propagates
        #10 a = 0;
        #5 a = 1;               // 2 ns pulse: shorter than 3 ns -> filtered (inertial)
        #2 a = 0;
        #10 $finish;
    end
endmodule

```

Continuous-assignment and net-declaration delays; a 2 ns pulse on a 3 ns path is filtered (c5_ca.v).

```

t= 0 a=0 y_net=x y_ca=x y_rf=x
t= 3 a=0 y_net=0 y_ca=0 y_rf=x
t= 4 a=0 y_net=0 y_ca=0 y_rf=0
t= 5 a=1 y_net=0 y_ca=0 y_rf=0
t= 7 a=1 y_net=0 y_ca=0 y_rf=1
t= 8 a=1 y_net=1 y_ca=1 y_rf=1
t=15 a=0 y_net=1 y_ca=1 y_rf=1
t=18 a=0 y_net=0 y_ca=0 y_rf=1
t=19 a=0 y_net=0 y_ca=0 y_rf=0
t=20 a=1 y_net=0 y_ca=0 y_rf=0
t=22 a=0 y_net=0 y_ca=0 y_rf=1
t=26 a=0 y_net=0 y_ca=0 y_rf=0
c5_ca.v:14: $finish called at 32 (1ns)

```

Output (Icarus Verilog 12).

- At t=0 all outputs are **x**: the nets have no value until their delayed updates mature. **y_rf** settles at t=4 because **x** -> 0 uses the fall delay.
- The rising edge at t=5 appears at t=8 on **y_net/y_ca** (3 ns) and at t=7 on **y_rf** (rise delay 2).
- The pulse from t=20 to t=22 is shorter than 3 ns and never reaches **y_net** or **y_ca** - inertial filtering. It is exactly as long as the rise delay of **y_rf**, so it passes there (t=22 to 26, stretched by the longer fall delay).

DELAYS AND SYNTHESIS

Synthesis ignores every **#** delay (often with a warning). Delays in RTL are for simulation only: to model I/O timing in a testbench, to make waveforms readable, or in gate-level models where the real values come from SDF back-annotation (Chapter 9). Putting **#1** on RTL assignments to "fix" a race hides the real problem and slows simulation.

5.3 initial and always blocks

- **initial** runs once. Testbenches are mostly **initial** blocks; in ASIC RTL, **initial** is ignored by synthesis (FPGA tools use it for power-up values and memory initialisation).
- **always @(posedge clk)** is the flip-flop template; **always @*** (or an explicit complete sensitivity list) is the combinational template; **always #5 clk = ~clk;** is a clock generator.
- Multiple **initial** and **always** blocks all start at time 0 in an **unspecified order**. Code that depends on which one runs first has a race (Section 5.8).
- A block contains one statement; **begin ... end** groups several sequentially, **fork ... join** groups them in parallel (Chapter 6).

5.4 Blocking and non-blocking assignments

Procedural code assigns variables in two ways. A **blocking** assignment `a = b;` evaluates the right-hand side and updates `a` immediately, before the next statement runs - like a software assignment. A **non-blocking** assignment `a <= b;` evaluates the right-hand side immediately but **schedules** the update for later in the same time step (the NBA region, Section 5.7), so statements that follow still see the old value of `a`.

```
module tb_ba;
  reg clk = 0;
  reg [3:0] in = 4'd5;
  reg [3:0] b1, b2, b3;          // pipeline written with blocking '='
  reg [3:0] n1, n2, n3;          // pipeline written with non-blocking '<='
  reg [3:0] x = 1, y = 2;
  always #5 clk = ~clk;
  always @(posedge clk) begin b1 = in; b2 = b1; b3 = b2; end      // collapses!
  always @(posedge clk) begin n1 <= in; n2 <= n1; n3 <= n2; end    // 3 flops
  initial begin
    @(posedge clk); #1
    $display("after edge 1: blocking b3=%0d | nonblocking n1=%0d n2=%0d n3=%0d",
             b3, n1, n2, n3);
    in = 4'd9;
    @(posedge clk); #1
    $display("after edge 2: blocking b3=%0d | nonblocking n1=%0d n2=%0d n3=%0d",
             b3, n1, n2, n3);
    // swap: NBA reads all right-hand sides before any update
    x <= y; y <= x;
    #1 $display("swap with <= : x=%0d y=%0d", x, y);
    x = y; y = x;
    $display("swap with = : x=%0d y=%0d", x, y);
    $finish;
  end
endmodule
```

A three-stage pipeline written with blocking and with non-blocking assignments, and a swap (c5_ba.v).

```
after edge 1: blocking b3=5 | nonblocking n1=5 n2=x n3=x
after edge 2: blocking b3=9 | nonblocking n1=9 n2=5 n3=x
swap with <= : x=2 y=1
swap with = : x=1 y=1
c5_ba.v:23: $finish called at 17 (1s)
```

Output (Icarus Verilog 12).

With blocking assignments, `b1 = in;` `b2 = b1;` `b3 = b2;` passes the new input straight through all three variables in one edge: the "pipeline" collapses into a single register (and synthesis will build exactly that). With non-blocking assignments all right-hand sides are sampled before any variable changes, so each edge shifts the data by exactly one stage - three flip-flops. The swap shows the same effect in two lines.

Guideline	Reason
Use <code><=</code> in every clocked (posedge) block	All flops sample their inputs before any output changes, independent of the order of <code>always</code> blocks - this is how real flip-flops behave
Use <code>=</code> in combinational blocks (<code>always @*</code>)	The block must compute its outputs from its inputs in one pass; <code><=</code> would need extra evaluation passes and can model latches wrongly
Do not mix <code>=</code> and <code><=</code> on the same variable	Lint error; simulation and synthesis may disagree
Assign each variable from one <code>always</code> block only	Otherwise a write-write race and, in synthesis, multiple drivers
Temporaries inside a clocked block may use <code>=</code>	Legal if they are read only later in the same block (they become wires); risky if read elsewhere

The standard coding guidelines (after C. Cummings, "Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!", SNUG 2000).

5.5 Delays: regular and intra-assignment

A delay can appear in two positions in a procedural assignment, with very different meanings:

Form	Samples RHS	Updates LHS	Process blocked?
<code>#d a = b;</code> (regular delay)	After the delay	After the delay	Yes, for d
<code>a = #d b;</code> (intra-assignment, blocking)	Now	After d	Yes, for d
<code>#d a <= b;</code>	After the delay	NBA region after the delay	Yes, for d
<code>a <= #d b;</code> (intra-assignment, non-blocking)	Now	d later, in the NBA region	No : the next statement runs at once

```
`timescale 1ns/1ns
module tb_delay;
  reg a = 0, r1, r2, r3;
  initial begin
    #1 a = 1;
    #2 a = 0;                // a is 1 only during [1,3)
  end
  initial begin
    #2 r1 = a;               // regular delay: wait 2, THEN sample a (=1)
  end
  initial begin
    r2 = #2 a;               // intra-assignment: sample a NOW (t=0 -> 0), assign at t=2
  end
  initial begin
    #1 r3 <= #5 a;           // sample at t=1 (a=1), update at t=6; process does NOT block
    $display("t=%0t: the process continued immediately after r3 <= #5", $time);
  end
  initial begin
    $monitor("t=%0t a=%b r1=%b r2=%b r3=%b", $time, a, r1, r2, r3);
    #10 $finish;
  end
endmodule
```

The four delay forms around a pulse on **a** from t=1 to t=3 (c5_delay.v).

```
t=0 a=0 r1=x r2=x r3=x
t=1: the process continued immediately after r3 <= #5
t=1 a=1 r1=x r2=x r3=x
t=2 a=1 r1=1 r2=0 r3=x
t=3 a=0 r1=1 r2=0 r3=x
t=6 a=0 r1=1 r2=0 r3=1
c5_delay.v:20: $finish called at 10 (1ns)
```

Output (Icarus Verilog 12).

r1 waited 2 ns and then read **a** (1); **r2** read **a** at time 0 (0) and wrote it 2 ns later; **r3** read **a** at t=1 and was updated at t=6 while its process carried on immediately. A sequence of `a <= #d b;` statements therefore models a **transport** delay line - every change is delivered, however short - unlike the inertial delay of a continuous assignment.

PITFALL: Q <= #1 D IN RTL

Some legacy RTL writes `q <= #1 d;` in flip-flops so that waveforms show a small clock-to-Q delay and to paper over races with zero-delay clock gating in the testbench. It is ignored by synthesis, it slows simulation, it hides genuine race bugs, and it makes RTL and gate-level behaviour differ. Modern coding standards ban it; fix races with correct blocking/non-blocking usage and clocking blocks (Chapter 21) instead.

5.6 Event control: @, edges, @*, wait and named events

`@(expression)` suspends a process until the expression **changes value**. `@(posedge e)` and `@(negedge e)` wait for specific transitions of the least significant bit of `e`. Several events can be combined with `or` or a comma: `@(posedge clk or negedge rst_n)`. The edges are defined over all four values:

	to 0	to 1	to x	to z
from 0	-	posedge	posedge	posedge
from 1	negedge	-	negedge	negedge
from x	negedge	posedge	-	-
from z	negedge	posedge	-	-

Transitions that count as posedge and negedge (IEEE 1364-2005 clause 9.7.2; unchanged in IEEE 1800).

```
`timescale 1ns/1ns
module tb_events;
    reg clk;                // starts at x
    reg ready = 0;
    reg [1:0] v = 0;
    event done;              // named event: no value, only an occurrence
    always @(posedge clk) $display("t=%0t posedge seen (clk=%b)", $time, clk);
    always @(negedge clk) $display("t=%0t negedge seen (clk=%b)", $time, clk);
    always @(v[0])           $display("t=%0t @(v[0]) woke: v=%b", $time, v); // any change
    initial begin
        #1 clk = 0;          // x -> 0 : a negedge
        #1 clk = 1;          // 0 -> 1 : a posedge
        #1 clk = 1'bz;       // 1 -> z : a negedge
        #1 clk = 1;          // z -> 1 : a posedge
        #1 v = 2'b10;        // v[0] did not change: no wake-up
        #1 v = 2'b11;        // v[0] changed
        #1 ready = 1;
    end
    initial begin
        wait (ready);        // level-sensitive: blocks until ready is true
        $display("t=%0t wait(ready) released", $time);
        -> done;             // trigger the named event
    end
    initial begin
        @done $display("t=%0t named event 'done' received", $time);
        wait (ready) $display("t=%0t wait(ready) with ready already 1: no blocking", $time);
    end
endmodule
```

Edges involving x and z, a value-change event on one bit, wait and a named event (c5_events.v).

```
t=0 @(v[0]) woke: v=00
t=1 negedge seen (clk=0)
t=2 posedge seen (clk=1)
t=3 negedge seen (clk=z)
t=4 posedge seen (clk=1)
t=6 @(v[0]) woke: v=11
t=7 wait(ready) released
t=7 named event 'done' received
t=7 wait(ready) with ready already 1: no blocking
```

Output (Icarus Verilog 12).

- `x -> 0` and `1 -> z` are **negedges**, `z -> 1` is a **posedge**. A clock that starts at `x` and goes to 1 produces a posedge at the first transition - one reason flip-flops can capture on the very first clock edge of a simulation.
- `@(v[0])` ignores changes to other bits of `v`; `@(v)` would wake on any bit.

- The first line is instructive: the declaration `reg [1:0] v = 0;` changed `v[0]` from x to 0 at time 0, and in Icarus that initialisation ran after the `always` block had started waiting, so the block woke up. Verilog does not define that order; SystemVerilog does - declaration initialisers are applied before any process starts, generating no event.
- `wait (expr)` is **level-sensitive**: it does not block at all if `expr` is already true. `@` is **edge-sensitive**: it always waits for the next change. Confusing them causes hangs (`@(ready)` when `ready` is already 1) or missed synchronisation.
- A named **event** carries no value. `-> e;` triggers it; processes blocked on `@e` resume. A trigger that happens before the waiter reaches `@e` is lost (SystemVerilog adds the persistent `e.triggered` property, Chapter 20).

@* and incomplete sensitivity lists

`@*` (also written `@(*)`) builds the sensitivity list automatically from every net and variable that is **read** by the statement: right-hand sides, `if/case` expressions, index expressions and function-call arguments. It does not include variables read only inside a called function's body, nor anything written only on the left-hand side.

```
module tb_star;
  reg a = 0, b = 0;
  reg y_list, y_star;
  always @(a) y_list = a & b;      // incomplete list: b missing
  always @*   y_star = a & b;      // @* = @(a or b): all RHS/condition signals
  initial begin
    #1 a = 1;
    #1 b = 1;                      // only y_star re-evaluates
    #1 $display("a=%b b=%b y_list=%b (stale latch-like) y_star=%b", a, b, y_list, y_star);
  end
endmodule
```

An incomplete sensitivity list versus `@*` (c5_star.v).

```
a=1 b=1 y_list=0 (stale latch-like) y_star=1
```

Output (Icarus Verilog 12).

PITFALL: SENSITIVITY LISTS ARE IGNORED BY SYNTHESIS

Synthesis builds an AND gate from `always @(a) y = a & b;` - it does not model the missing `b`. The simulation shows a stale, latch-like output that the gates will never produce: a simulation/synthesis mismatch that equivalence checking will flag only if someone runs it on RTL-to-gates. Use `@*` in Verilog and `always_comb` in SystemVerilog (which also covers variables read inside functions and runs once at time zero).

5.7 The stratified event scheduler

Simulated time is divided into **time slots**. Within a slot, the Verilog standard (IEEE 1364-2005, clause 11) sorts pending events into **regions** that are processed in a fixed order:

Region	Contains	Created by
Active	Events to process now, in any order	Process resumptions, continuous-assignment and primitive updates, blocking assignments, RHS evaluation of <code><=</code> , <code>\$display</code>
Inactive	Events to process after all active events	Explicit zero delays: <code>#0</code>
NBA (non-blocking assign update)	Variable updates from <code><=</code>	Non-blocking assignments issued in this slot
Monitor	End-of-slot observation	<code>\$monitor</code> , <code>\$strobe</code>
Future	Events for later slots	Delays <code>#d</code> , <code><= #d</code>

```

time slot t
+-----+
| +-----+ empty +-----+ empty +-----+ |
+-->| Active |----->| Inactive |----->| NBA |---+
^   +-----+         +-----+         +-----+ |
|   ^   (moving an event into Active restarts the loop) |
+-----+
      when Active, Inactive and NBA are all empty:
          +-----+
          | Monitor | $monitor / $strobe print final values
          +-----+
            |
            v   advance to the next time slot with events

```

The Verilog scheduling loop. NBA updates can wake new processes, so the loop iterates (delta cycles) until the slot is quiescent.

The algorithm is: while there are active events, execute one. When the active region is empty, move all inactive events to active; when both are empty, move all NBA updates to active. Updates wake more processes, which may issue more events, and the loop continues until nothing is left; then the monitor region runs and time advances. Each pass through Active -> NBA is informally called a **delta cycle**.

```

module tb_regions;
  reg [3:0] q = 0;
  initial begin
    #10;
    q <= 4'd5;                                // NBA region
    $display ("display : q=%0d (active region: old value)", q);
    $strobe  ("strobe  : q=%0d (postponed/monitor region: final value)", q);
    #0 $display("after #0: q=%0d (inactive region: NBA not yet done)", q);
    q = 4'd7;                                // blocking, same time step
    $display ("display : q=%0d after blocking q=7", q);
  end
  initial #10 $monitor("monitor : q=%0d at t=%0t (printed once per time step)", q, $time);
endmodule

```

What \$display, #0, \$strobe and \$monitor see when a variable is written with <= and = in the same slot (c5_regions.v).

```

display : q=0 (active region: old value)
after #0: q=0 (inactive region: NBA not yet done)
display : q=7 after blocking q=7
strobe  : q=5 (postponed/monitor region: final value)
monitor : q=5 at t=10 (printed once per time step)

```

Output (Icarus Verilog 12).

- **\$display** runs in the active region, before the NBA update: it prints the old value 0.
- **#0** moves the process to the inactive region, which still runs before NBA: still 0. This is why **#0** does not fix races involving non-blocking assignments.
- The blocking **q = 7** takes effect immediately, but the NBA update to 5 scheduled earlier is applied afterwards and wins: the final value is 5 - mixing = and <= on one variable gives results that depend on region order, not text order.
- **\$strobe** and **\$monitor** run in the monitor region and see the final, settled value. Use them (or sample after the edge) when printing values of registers updated with <=.

WHAT THE STANDARD DOES AND DOES NOT GUARANTEE

Guaranteed: statements inside one **begin-end** execute in order; NBA updates from one process are applied in the order they were issued; regions are processed in the order above. Not guaranteed: the order in which different processes that are active at the same time execute, and even whether a process runs to its next timing control without being interleaved (the LRM explicitly allows a simulator to suspend a process and run another). Any result that depends on the unguaranteed part is a **race**.

Preview: the SystemVerilog scheduler

IEEE 1800 refines the scheduler into many more regions so that testbench code and assertions can run at well-defined points relative to the design (details in Chapters 21 and 22):

```

Preponed    sample values for assertions and clocking blocks (read-only)
|
Active -> Inactive -> NBA      design evaluation (same as Verilog)
|      ^_____| iterate
Observed    evaluate concurrent assertion properties
|
Reactive -> Re-Inactive -> Re-NBA  program blocks, assertion action blocks,
|      ^_____| testbench drives (iterate; may re-enter Active)
Postponed   $monitor, $strobe, end of slot (read-only)

(Pre-/Post- regions exist for the PLI/VPI and are not used by SV code.)

```

Simplified IEEE 1800 time slot.

5.8 Races, demonstrated

A race occurs when the result depends on the order in which simultaneously active processes execute. The following example has two of them: two **always** blocks at the same edge, one writing **a** with **=** and one reading it; and a testbench that changes the DUT's input with **=** on the same edge that the DUT samples it. A macro swaps the textual order of the first two blocks.

```

module tb_race;
  reg clk = 0;
  reg [3:0] a = 0, b = 0, d = 0, q = 0;
  always #5 clk = ~clk;

  `ifdef SWAP
    always @(posedge clk) b = a;          // reader first in the source
    always @(posedge clk) a = a + 1;      // writer second
  `else
    always @(posedge clk) a = a + 1;      // writer first in the source
    always @(posedge clk) b = a;          // reader: old or new a?  Unspecified!
  `endif
  always @(posedge clk) q <= d;           // a "DUT" flop
  initial begin
    @(posedge clk) d = 4'd9;              // racy stimulus: blocking on the sampling edge
    #1 $display("after edge 1: a=%0d b=%0d q=%0d (flop saw %s d)",
               a, b, q, (q == 9) ? "NEW" : "OLD");
    $finish;
  end
endmodule

```

Two classic races (c5_race.v). Compiled with and without -DSWAP.

Simulator	Source order	b (reader of a)	q (flop fed by racy d)
Icarus Verilog 12	writer first	0 (old value)	9 (new)
Icarus Verilog 12	reader first (SWAP)	1 (new value)	9 (new)
Verilator 5.020	writer first	1 (new value)	9 (new)
Verilator 5.020	reader first (SWAP)	0 (old value)	9 (new)

Results of the four runs; each is a verbatim **after edge 1** line below.

```

after edge 1: a=1 b=0 q=9 (flop saw NEW d)
after edge 1: a=1 b=1 q=9 (flop saw NEW d)

```

Icarus Verilog 12: default order, then with -DSWAP.

```
after edge 1: a=1 b=1 q=9 (flop saw NEW d)
after edge 1: a=1 b=0 q=9 (flop saw NEW d)
```

Verilator 5.020: default order, then with -DSWAP.

The two simulators give **opposite** answers for the same source, and each changes its answer when two independent blocks are reordered - a change that means nothing in hardware. Both behaviours are legal. In the second race, both simulators happened to let the testbench write **d** before the flop sampled it, so the flop captured the new value on the same edge - one cycle earlier than real hardware would. That kind of "luck" typically flips when an unrelated change is made to the testbench, or when moving to a different simulator at the end of the project. Driving **d** with a non-blocking assignment removes the race:

```
@(posedge clk) d <= 4'd9; // fixed: NBA
```

The one-line fix in c5_race_fix.v.

```
after edge 1: a=1 b=0 q=0 (flop saw OLD d)
```

Output (Icarus Verilog 12): the flop now sees the old value of d on edge 1, as real hardware would.

Race	Symptom	Cure
Write-read between <i>always</i> blocks at one edge	Reader sees old or new value depending on order	<= for all clocked assignments
Write-write: two processes assign one variable	Last writer wins, unpredictably	One <i>always</i> block per variable
Testbench vs DUT at the sampling edge	DUT sees stimulus a cycle early, sometimes	Drive with <= , drive on the opposite edge, or use clocking blocks (Chapter 21)
Time-0 initialisation	<i>always</i> blocks see or miss the initial x -> value event	Reset logic; SV initialisers; avoid relying on time-0 events
\$display of <= targets	Printed value is one cycle old	\$strobe , or sample after the edge

INTERVIEW QUESTION: DOES A RACE EXIST IN ALWAYS @(POSEDGE CLK) A = B; ALWAYS @(POSEDGE CLK) B = A;?

Yes. If the first block runs first, both end up with the old **b**; if the second runs first, both end up with the old **a**. Neither is a swap. With non-blocking assignments, both right-hand sides are evaluated in the active region and both updates happen in the NBA region, so the values are swapped every clock regardless of order - which is also what the synthesized pair of flip-flops does.

5.9 Summary

- Every *assign*, primitive, *initial* and *always* is a concurrent process; procedural blocks suspend only at timing controls.
- Continuous-assignment delays are inertial (short pulses are filtered); **a <= #d b** gives transport behaviour.
- Blocking **=** updates immediately; non-blocking **<=** evaluates now and updates in the NBA region. Use **<=** for clocked logic and **=** for combinational logic.
- Regular delays wait before sampling; intra-assignment delays sample first and assign later.
- posedge/negedge** include transitions through x and z; *wait* is level-sensitive, **@** edge-sensitive; named events are lost if nobody is waiting.
- The Verilog time slot runs Active -> Inactive -> NBA until quiescent, then Monitor; SystemVerilog adds Preponed, Observed, Reactive and Postponed regions.
- Process order within a region is unspecified; code that depends on it is a race and can give different results on different simulators, as demonstrated.

5.10 Exercises

- 1 Rewrite the blocking pipeline of Section 5.4 so that it works with blocking assignments only, by reordering the statements. Why is this still a bad idea?
- 2 What does `wire #5 y = a;` do with a 4 ns pulse on `a`? With a 6 ns pulse? Write a model of a 5 ns transport delay line and show that it passes the 4 ns pulse.
- 3 Predict the output of `initial begin a = 0; a <= 1; $display(a); #0 $display(a); $strobe(a); a = 2; end.`
- 4 A testbench generates a clock with `always #5 clk = ~clk;` and drives inputs with `@(posedge clk) data = $random;`. Explain the race and give three different fixes.
- 5 Explain why `@(ready)` can hang where `wait (ready)` does not, and construct a case where the opposite causes a bug (the process fails to wait when it should).
- 6 Draw the regions visited in one time slot for a design with one flip-flop (`q <= d`), a continuous assignment `assign d = ~q`, and a `$monitor` on `q`, starting from the posedge of the clock.

6. Control Flow, Functions and Tasks

Inside **initial** and **always** blocks, Verilog offers the familiar statements of a structured language - **if**, **case**, loops, subroutines - with two twists: every condition is evaluated in four-state logic, and code that describes hardware must avoid constructs that cannot be unrolled into a fixed circuit. This chapter covers each statement with its exact semantics for **x** and **z**, its use in RTL and in testbenches, and the pitfalls that make functions and tasks behave differently from their software counterparts.

6.1 if / else

```
if (cond) statement;                // cond is x or z -> treated as false
else if (cond2) begin ... end
else statement;

// dangling else: an 'else' binds to the NEAREST preceding 'if'
if (a)
    if (b) y = 1;
else    y = 0;    // belongs to 'if (b)', despite the indentation!
```

Syntax fragment; the last lines show the dangling-else rule.

- The condition is true if it is non-zero and known; if it is zero, **x** or **z** the **else** branch runs (Chapter 4 demonstrated this). **if (a == b)** with an unknown bit in **a** therefore silently takes the else path.
- In combinational RTL, an **if** without **else** (or a variable not assigned on every path) implies storage: synthesis infers a **latch**. Assign defaults at the top of the block or cover every path.
- An **if-else if** chain describes a **priority** structure: the first true condition wins, and synthesis builds a chain of multiplexers (or equivalent logic) in that order.
- Always use **begin ... end** for multi-line branches; the dangling-else fragment above is a real bug pattern that the indentation hides.

6.2 case, casez and casex

A **case** statement compares its **case expression** with each **case item** in order and executes the first match; there is no fall-through and no **break**. Several items may share a statement (**2'b01, 2'b10: ...**), and **default** catches everything else. The three variants differ only in how they compare:

Statement	Comparison	x in expression or item	z / ? in expression or item
case	Exact 4-state match (like ===)	Must match an x literally	Must match a z literally
casez	z and ? bits are don't-care	Must match literally	Don't care - in either the item or the expression
casex	x, z and ? bits are don't-care	Don't care - in either	Don't care - in either

The phrase "in either" is what makes the variants dangerous: a don't-care bit in the *expression*, which is usually a signal, matches anything. The testbench below runs four selector values through all three variants:

```
module tb_case;
    reg [1:0] s;
    integer i;
    reg [1:0] vals [0:3];
    task run;
        begin
            case (s)                // exact 4-state match (=== per item)
                2'b00: $write("case :00    ");
                2'b1x: $write("case :1x    ");
                2'bzz: $write("case :zz    ");
                default: $write("case :dflt ");
            endcase
        end
    endtask
endmodule
```

```

    endcase
    casez (s) // z / ? in item OR selector = don't care
        2'b0?: $write("casez:0? ");
        2'b1?: $write("casez:1? ");
        default: $write("casez:dflt ");
    endcase
    casex (s) // x AND z in item OR selector = don't care
        2'b00: $display("casex:00");
        2'b1x: $display("casex:1x");
        default: $display("casex:dflt");
    endcase
end
endtask
initial begin
    vals[0] = 2'b10; vals[1] = 2'b1x; vals[2] = 2'bzz; vals[3] = 2'bx0;
    for (i = 0; i < 4; i = i + 1) begin
        s = vals[i]; $write("s=%b | ", s); run;
    end
end
endmodule

```

case, casez and casex with known, x and z selectors (c6_case.v).

s=10	case :dflt	casez:1?	casex:1x
s=1x	case :1x	casez:1?	casex:1x
s=zz	case :zz	casez:0?	casex:00
s=x0	case :dflt	casez:dflt	casex:00

Output (Icarus Verilog 12).

- **s=10**: plain **case** finds no exact match (the item **2'b1x** requires a literal x) and takes the default; **casez** and **casex** match on the don't-care bits.
- **s=1x**: **case** matches the literal **2'b1x** - useful in testbenches to detect unknowns, meaningless in hardware.
- **s=zz**: in **casez** the selector's own z bits are wildcards, so it matches the **first** item **2'b0?**; in **casex** it matches **2'b00**.
- **s=x0**: an **unknown** selector makes **casex** match **2'b00** and execute that branch as if everything were fine. In RTL simulation this hides an uninitialised or corrupted control signal - the gates would do something else entirely.

PITFALL: CASEX IN RTL

Because **casex** treats an **x** in the selector as a wildcard, it turns X-propagation into X-masking: the simulation proceeds down a branch chosen by the item order, while real hardware might take any branch. Most coding standards ban **casex** in RTL. Use **casez** with **?** in the items for wildcard decoding (it still masks **z**, which is rare in RTL), or SystemVerilog's **case (...)** **inside** with **?** wildcards, which treats x/z in the **expression** as ordinary values (Chapter 14). Add an assertion that the selector is known.

FULL_CASE AND PARALLEL_CASE

A case statement whose items do not cover all values implies a latch in a combinational block; one whose items overlap describes priority logic. The synthesis pragmas **// synopsys full_case** and **// synopsys parallel_case** tell synthesis to assume the case is complete or mutually exclusive *without telling the simulator* - the textbook source of simulation/synthesis mismatch (Chapter 11). SystemVerilog's **unique** and **priority** keywords (Chapter 14) give the same optimisation with simulator checks, and replace the pragmas in modern code.

6.3 Loops

Loop	Form	Notes
<code>for</code>	<code>for (i = 0; i < N; i = i + 1) stmt</code>	Verilog-2005 has no <code>++</code> , no <code>+=</code> and no loop-local declaration (all added by SV); the variable is usually an integer or a genvar in generate
<code>while</code>	<code>while (cond) stmt</code>	Condition re-evaluated before each iteration; x/z counts as false
<code>repeat</code>	<code>repeat (n) stmt</code>	n is evaluated once at the start; x or z means zero iterations
<code>forever</code>	<code>forever stmt</code>	Infinite; must contain a timing control; left only with disable or \$finish
(SV) <code>do ... while,</code> <code>foreach,</code> <code>break,</code> <code>continue</code>	-	Chapter 14

```

module tb_loops;
  reg [7:0] data = 8'b0010_1100;
  integer i, n, first_one;
  reg clk = 0;
  initial begin
    // for: count ones
    n = 0;
    for (i = 0; i < 8; i = i + 1) n = n + data[i];
    $display("for    : popcount = %0d", n);
    // while + disable of a named block = "break"
    first_one = -1; i = 0;
    begin : search
      while (i < 8) begin
        if (data[i]) begin first_one = i; disable search; end
        i = i + 1;
      end
    end
    $display("while  : lowest set bit = %0d", first_one);
    // disable of the inner named block = "continue"
    n = 0;
    for (i = 0; i < 8; i = i + 1) begin : body
      if (i % 2) disable body;          // skip odd i
      n = n + data[i];
    end
    $display("for    : ones in even positions = %0d", n);
    // repeat: fixed count, expression evaluated once
    n = 3; i = 0;
    repeat (n) begin n = n + 1; i = i + 1; end
    $display("repeat : ran %0d times although n grew to %0d", i, n);
  end
  // forever: clock generator stopped by disable from another process
  initial begin : clkgen
    forever #5 clk = ~clk;
  end
  initial begin
    #23 disable clkgen;
    #50 $display("forever: clk frozen at %b, t=%0t", clk, $time);
  end
endmodule

```

Loops, and named blocks with disable used as break and continue (c6_loops.v).

```

for      : popcount = 3
while    : lowest set bit = 2
for      : ones in even positions = 1
repeat   : ran 3 times although n grew to 6
forever: clk frozen at 0, t=73

```

Output (Icarus Verilog 12). data = 8'b0010_1100.

Verilog-2005 has no `break` or `continue`. Both are built from a **named block** and `disable`: `disable search`; terminates the named block `search` that encloses the loop (break), while `disable body`; inside the loop body ends only the current iteration, and the `for` continues with its step statement (continue). `disable clkgen`; from another process kills a `forever` loop.

Loops in synthesizable code

Synthesis **unrolls** loops: every iteration becomes a copy of the hardware. A loop is therefore synthesizable only if its iteration count is fixed at elaboration (constant bounds), and its body must not contain timing controls. The popcount above synthesizes to an adder tree of eight 1-bit inputs; the `while` search to a priority encoder. A loop bound that depends on a signal (`while (x != 0)`) does not describe hardware of fixed size and is rejected - or worse, unrolled to a tool limit.

6.4 Named blocks and disable

Any `begin ... end` or `fork ... join` can be named with `: name`. A named block creates a new **scope**: it may declare local variables (in Verilog-2005 only in named blocks), appears in the hierarchy (`tb.search.k`), and can be terminated by `disable name` from inside or outside. `disable task_name` terminates a running task. Disabling a block that is not currently executing does nothing.

PITFALL: DISABLE KILLS EVERY ACTIVATION

`disable name` terminates all currently active executions of that block or task, from every caller - not just the one you meant. In a static task called from two processes, or a named block inside a task that runs concurrently, one `disable` stops them all. SystemVerilog's `disable fork`, process handles (`process::self()`, `kill()`) and `break/continue/return` provide finer control (Chapter 20).

6.5 Functions

A function computes a value in zero simulated time. It is called in an expression, returns a value through its own name (or `return` in SystemVerilog), and in synthesizable code becomes combinational logic inlined at every call.

Rule (Verilog-2005)	Consequence
At least one input argument; no output or inout (SV allows them, and void functions)	Functions return exactly one value
No timing controls (<code>#</code> , <code>@</code> , <code>wait</code>), no task calls	Always executes in zero time; can be used in continuous assignments
No non-blocking assignments, no event triggers	It cannot schedule side effects
Return type given by a range, integer , real , signed ...	<code>function [7:0] f</code> ; default 1 bit - a classic truncation bug
Static by default: arguments and locals are allocated once per module instance	Concurrent or recursive calls share storage
<code>function automatic</code> : new storage for each call	Recursion and reentrancy work; locals cannot be accessed hierarchically or assigned with <code><=</code>

```

module fifo_ctrl #(parameter DEPTH = 20) (output wire [AW-1:0] addr_w);
    // Constant function: evaluated at elaboration to size a port/localparam

```

```

function integer clog2 (input integer value);
    integer v;
    begin
        v = value - 1;
        for (clog2 = 0; v > 0; clog2 = clog2 + 1) v = v >> 1;
    end
endfunction
localparam AW = clog2(DEPTH);
assign addr_w = {AW{1'b0}};
endmodule

module tb_func;
    wire [4:0] w;                                // 5 = clog2(20)
    fifo_ctrl #(.DEPTH(20)) u (.addr_w(w));

    function automatic integer fact (input integer n); // recursion needs automatic
        fact = (n <= 1) ? 1 : n * fact(n - 1);
    endfunction

    function integer fact_static (input integer n);    // static: one copy of n
        fact_static = (n <= 1) ? 1 : fact_static(n - 1) * n;
    endfunction

    function [7:0] reverse (input [7:0] x);           // result width = 8
        integer k;
        for (k = 0; k < 8; k = k + 1) reverse[k] = x[7-k];
    endfunction

    initial begin
        $display("AW for DEPTH=20 : %0d", u.AW);
        $display("fact(5)           : %0d", fact(5));
        $display("fact_static(5)    : %0d (static: recursive calls share n)", fact_static(5));
        $display("reverse(8'b1101_0000) = %b", reverse(8'b1101_0000));
    end
endmodule

```

A constant function sizing a port, an automatic recursive function, the same function left static, and a bit reverser (c6_func.v).

```

AW for DEPTH=20 : 5
fact(5)         : 120
fact_static(5)  : 1 (static: recursive calls share n)
reverse(8'b1101_0000) = 00001011

```

Output (Icarus Verilog 12).

- **clog2** is a **constant function**: called with constant arguments in a constant expression (a **localparam**), it is evaluated during elaboration. Constant functions may use only their arguments, locals, parameters and other constant functions - no hierarchical references and no global variables. This is how legacy code computed address widths; IEEE 1364-2005 added the built-in **\$clog2**, which new code should use.
- **fact** is **automatic**, so each recursive call has its own **n**: 120.
- **fact_static** is identical except that it is static. Every recursive call overwrites the single shared **n**; by the time the multiplications run, **n** is 1 at every level, and the result is 1. (With the operands in the other order, **n * f(n-1)**, Icarus happens to read **n** before the call and prints 120 - the result depends on operand evaluation order, which is unspecified.)
- **reverse** shows the declared return width **[7:0]**; without it the function would return 1 bit.

6.6 Tasks

A task is a subroutine that may consume time. It is called as a statement, may have any number of **input**, **output** and **inout** arguments, may contain timing controls, may call other tasks and functions, and returns no value (SystemVerilog adds **return** and **ref** arguments). Tasks are the backbone of Verilog testbenches: bus functional models are sets of tasks such as **write(addr, data)** and **read(addr, data)** (Chapter 25).

Two semantic details cause most task bugs. First, arguments are passed **by value**: inputs are copied in when the task is called, and outputs are copied out **only when it returns**. Second, tasks are **static** by default, like functions: all concurrent callers share the same argument and local storage.

```
`timescale 1ns/1ns
module tb_task;
    // STATIC task: one shared copy of 'id' and 'dly' for all callers
    task send_static (input [7:0] id, input integer dly);
        begin
            #dly $display("t=%0t static    : finished id=%0d", $time, id);
        end
    endtask
    // AUTOMATIC task: each call gets its own stack frame
    task automatic send_auto (input [7:0] id, input integer dly);
        begin
            #dly $display("t=%0t automatic : finished id=%0d", $time, id);
        end
    endtask
    // output arguments are copied back only when the task returns
    task pulse (output reg q, input integer width);
        begin
            q = 1; #width q = 0;
        end
    endtask
    reg p = 0;
    initial begin
        fork
            send_static(8'd1, 10);           // both calls run concurrently...
            send_static(8'd2, 3);           // ...the second overwrites id and dly
        join
        fork
            send_auto(8'd1, 10);
            send_auto(8'd2, 3);
        join
        fork
            pulse(p, 5);
            #2 $display("t=%0t p=%b inside pulse (output not copied yet)", $time, p);
        join
            $display("t=%0t p=%b after pulse returned", $time, p);
        end
    end
endmodule
```

A static task called concurrently, the same task declared automatic, and an output argument that never shows its pulse (c6_task.v).

```
t=3 static    : finished id=1
t=10 static   : finished id=1
t=13 automatic : finished id=2
t=20 automatic : finished id=1
t=22 p=0 inside pulse (output not copied yet)
t=25 p=0 after pulse returned
```

Output (Icarus Verilog 12).

- Both concurrent calls of the static `send_static` report `id=1`: the two calls share one `id` variable, and whichever call wrote it last wins (here the fork happened to start the second call first). The delays still differ because each `#dly` was evaluated when its call started.
- With `automatic`, each call has its own frame and both report their own `id` correctly.
- The `pulse` task sets its output `q` to 1 and back to 0, but the caller's `p` is updated only on return, with the final value 0: the pulse never appears. A task that must wiggle a signal over time has to assign the module-level signal directly (a side effect), or, in SystemVerilog, take it as a `ref` argument or drive it through a virtual interface.

MAKE TESTBENCH TASKS AND FUNCTIONS AUTOMATIC

Any subroutine that may be called from more than one process at a time - which is every BFM task in a testbench with parallel threads - must be `automatic`. In SystemVerilog, subroutines declared inside classes are always automatic, and a whole module or program can be made automatic with `module automatic m;`. Static subroutines remain the default in modules for backward compatibility, and they remain appropriate for synthesizable functions, where static and automatic behave identically.

	Function	Task
Called as	Expression operand	Statement
Timing controls	Not allowed	Allowed
Arguments	Inputs only (Verilog); SV adds output/inout/ref	input, output, inout (SV adds ref)
Returns	One value	Nothing (values through outputs)
Can call	Functions	Functions and tasks
Synthesizable	Yes (combinational logic)	Only without timing controls; rarely used in RTL
Default lifetime	Static in modules	Static in modules

6.7 fork ... join in Verilog

`begin ... end` executes its statements in sequence; `fork ... join` starts all of them **in parallel** at the same simulation time and continues only when **all** have finished. Each statement in the fork is a separate process, so delays inside different branches are all measured from the moment the fork started. Verilog has only `join`; SystemVerilog adds `join_any`, `join_none` and `disable fork` (Chapter 20).

```
`timescale 1ns/1ns
module tb_fork;
    reg ack = 0;
    initial #35 ack = 1;                // the "DUT" answers at t=35
    initial begin
        fork                            // all statements start at t=0
            #10 $display("t=%0t branch A done", $time);
            #30 $display("t=%0t branch B done", $time);
            begin #5 $display("t=%0t branch C step 1", $time);
                #5 $display("t=%0t branch C step 2", $time); end
        join                            // waits for the LAST branch
        $display("t=%0t join complete", $time);
        // Verilog timeout idiom: race a wait against a timer, kill the loser
        fork : wait_or_timeout
            begin @(posedge ack) $display("t=%0t ack received", $time);
                disable wait_or_timeout; end
            begin #50 $display("t=%0t TIMEOUT waiting for ack", $time);
                disable wait_or_timeout; end
        join
        $display("t=%0t continuing after the race", $time);
    end
endmodule
```

Parallel branches, and the Verilog idiom for a timeout: two branches race and the winner disables the named fork (c6_fork.v).

```
t=5 branch C step 1
t=10 branch A done
t=10 branch C step 2
t=30 branch B done
t=30 join complete
t=35 ack received
t=35 continuing after the race
```

Output (Icarus Verilog 12).

- The `join` completes at t=30, when the slowest branch (B) finishes. Branch C is itself sequential: its two steps run at t=5 and t=10.
- In the timeout idiom, whichever branch finishes first executes `disable wait_or_timeout`, which kills the other branch and ends the fork. Here `ack` rises at t=35, before the 50 ns timeout.
- The order in which simultaneous branches start is unspecified - the static task example above depended on it.
- `fork ... join` is not synthesizable; it belongs to testbenches and behavioural models.

6.8 Summary

- `if` treats x/z as false and takes the else branch; an incomplete `if` in combinational logic infers a latch; `else` binds to the nearest `if`.
- `case` compares with 4-state exactness; `casez` treats z/? as don't-care and `casex` also x - in the item **and** the selector, which makes `casex` mask unknowns. Prefer `casez` with `?`, or SV `case inside`.
- `for`, `while`, `repeat` (count evaluated once) and `forever`; `break/continue` are built with named blocks and `disable` in Verilog-2005. Synthesizable loops must have constant bounds.
- Functions: zero time, at least one input, one return value, static by default; `automatic` for recursion; constant functions run at elaboration (\$clog2 replaces the hand-written ones).
- Tasks: may consume time; arguments copied in at call and out at return; static by default, so concurrent calls corrupt each other unless declared `automatic`.
- `fork ... join` runs branches in parallel and waits for all; a named fork plus `disable` implements timeouts.

6.9 Exercises

- 1 Write a priority encoder for an 8-bit request vector three ways - `if-else if`, `casez` with `?` items, and a `for` loop - and verify that they agree on all 256 inputs. Which one gives an x-free output for an input containing x?
- 2 Show a selector value for which `case`, `casez` and `casex` all choose different branches of the same item list.
- 3 Rewrite `fact_static` so that it returns 120 while remaining static, without recursion. Then explain why a static function is fine in synthesizable code.
- 4 Write a testbench task `apply_reset(input integer cycles)` and a task `send_byte` that drives a serial line bit by bit. Which of them must be automatic if two instances of a UART model call `send_byte` concurrently? What goes wrong otherwise?
- 5 Implement a watchdog: a `fork` that waits for `done`, a timeout branch, and a third branch that prints a progress message every 100 ns. Make sure all branches stop when the fork is left.
- 6 A `repeat (count)` loop is used while `count` is modified inside the loop body. How many iterations run? What if `count` is x?

PART II

Verilog in Depth

Part I taught the core of the language. This part covers the rest of IEEE 1364 that every professional meets sooner or later: parameterisation and generate, the preprocessor, the gate-level and timing constructs that standard-cell libraries and gate-level simulation are built on, the system tasks that testbenches live on, and - most important of all for a designer - the synthesizable subset and the ways RTL simulation can disagree with the hardware that synthesis actually builds.

7. Parameters, Generate and Configurations

A block of silicon IP is almost never written for one size. The same FIFO is instantiated 40 times in an SoC with a dozen different depths and widths; the same AXI interconnect is built with 3 masters in one chip and 11 in the next; a DSP datapath is generated with 4, 8 or 16 lanes. Verilog supports this with three mechanisms that all act at **elaboration time** - after parsing, before simulation or synthesis starts: **parameters** (named constants that can differ per instance), **generate constructs** (structure that is replicated or selected according to parameters) and **configurations** (which source description is bound to which instance). Understanding exactly when and how these are evaluated is what separates a reusable IP from a block that has to be edited by hand for every project.

```

source files --> parse --> ELABORATION -----> simulate
                        | 1. build hierarchy from the top      | or
                        | 2. resolve parameter values per inst. | synthesizer
                        |   (#() overrides, defparam)          |
                        | 3. evaluate constant expressions      |
                        |   ($clog2, constant functions)        |
                        | 4. expand generate for/if/case         |
                        | 5. bind cells via config/libraries     |
                        v                                       v
                        one flat, fully-sized design (no parameters left)

```

Everything in this chapter is resolved during elaboration. Nothing here exists at run time: a parameter is never a register.

7.1 parameter and localparam

A **parameter** is a named constant of a module (or interface, program or class in SystemVerilog) whose value may be changed for each instance when the design is elaborated. A **localparam** is identical except that it can **never** be overridden from outside; it is normally derived from other parameters. Both may only be assigned **constant expressions**: literals, other parameters, **localparams**, **genvars**, constant function calls and constant system functions such as **\$clog2**, **\$bits**, **\$signed** and the math functions. A variable, a net or a hierarchical reference is never allowed.

Parameters may be declared in two places. The Verilog-2001 **parameter port list** **#(...)** after the module name is the modern style and the only one you should write; body declarations are the Verilog-1995 style. IEEE 1800 adds a rule that surprises people: if a module has a parameter port list, **every parameter** declared in its body is treated as a **localparam** and cannot be overridden. SystemVerilog also allows **localparam** inside the port list, which keeps derived values next to the parameters they depend on (Icarus, Verilator and all commercial tools accept it).

```

module fifo #(
    parameter integer DEPTH = 16,          // typed parameter
    parameter [7:0]  WIDTH = 8,           // ranged parameter (8-bit, unsigned)
    parameter        NAME  = "fifo",      // untyped: takes type of its value
    localparam integer AW    = $clog2(DEPTH) // derived; cannot be overridden
) ();
    initial $display("%m: DEPTH=%0d WIDTH=%0d AW=%0d NAME=%0s", DEPTH, WIDTH, AW, NAME);
endmodule

module top;
    fifo                                u_default ();
    fifo #(64, 32)                      u_pos      (); // positional: DEPTH=64, WIDTH=32
    fifo #(.DEPTH(1000), .NAME("rx")) u_named (); // named: WIDTH keeps its default
    fifo                                u_defp     ();
    defparam u_defp.DEPTH = 5;           // legal but deprecated
endmodule

```

p1.v - the three ways of giving an instance its own parameter values.


```
top.u_default: DEPTH=16 WIDTH=8 AW=4 NAME=fifo
top.u_pos: DEPTH=64 WIDTH=32 AW=6 NAME=fifo
top.u_named: DEPTH=1000 WIDTH=8 AW=10 NAME=rx
top.u_defp: DEPTH=5 WIDTH=8 AW=3 NAME=fifo
```

iverilog -g2012 -Wall p1.v && vvp - note that AW is recomputed for every instance.

The type and size of a parameter

A parameter declaration may carry a data type, a range, a sign, all of these or none. The rules (IEEE 1800-2017 6.20.2, same as 1364-2005 12.2) decide what an overriding value turns into, and they are a classic source of silent truncation:

Declaration	Type/size of the parameter	Effect of an override
<code>parameter P = 8'd200</code>	Untyped: takes the type, size and sign of the final value	Takes the size of the overriding value (pass 12'd10 and P becomes 12 bits)
<code>parameter [3:0] P = ...</code>	Unsigned, exactly the range given	Override is converted: truncated or zero-extended to 4 bits
<code>parameter signed [7:0] P</code>	Signed, range given	Converted to 8-bit signed
<code>parameter signed P = ...</code>	Signed, size from the final value	Size follows value, sign forced signed
<code>parameter integer P</code>	32-bit signed integer	A real value is rounded, not truncated
<code>parameter real P</code>	Real	An integer value is converted to real
<code>parameter type T = logic</code>	SystemVerilog type parameter	A type, not a value (Chapter 17)

```
module p2;
  parameter      P_U  = 8'd200;      // untyped: 8-bit unsigned from value
  parameter [3:0] P_R  = 8'd200;      // ranged: value truncated to 4 bits
  parameter signed [7:0] P_S = 8'd200; // signed 8-bit: reads as -56
  parameter real  P_F  = 2.5;
  parameter integer P_I = 2.9;        // real converted to integer: rounds
  initial begin
    $display("P_U=%0d bits=%0d", P_U, $bits(P_U));
    $display("P_R=%0d bits=%0d", P_R, $bits(P_R));
    $display("P_S=%0d", P_S);
    $display("P_F=%0.2f P_I=%0d", P_F, P_I);
  end
endmodule
```

p2.v - what the declared type does to a value.

```
P_U=200 bits=8
P_R=8 bits=4
P_S=-56
P_F=2.50 P_I=3
```

Icarus Verilog 12. 200 = 1100_1000b: truncated to 4 bits it is 8; as signed 8-bit it is -56.

PITFALL: UNTYPED PARAMETERS CHANGE WIDTH WHEN OVERRIDDEN

A bare `parameter W = 8` is a 32-bit integer by default - but if an integrator overrides it with a sized literal such as `4'd12`, W becomes a 4-bit unsigned value and every expression using W is now evaluated with different sizing rules (Chapter 4). Arithmetic such as `W*2` or `W-1` can then wrap. Give parameters that are used as numbers an explicit type (`parameter int W = 8` in SystemVerilog, `parameter integer W = 8` in Verilog) and give parameters that are used as bit patterns an explicit range (`parameter [31:0] RST_VAL`).

7.2 Overriding parameters: #(), named association and defparam

There are two legal ways to change a parameter of an instance. The **instance parameter value assignment** `#( ... )` is part of the instantiation and comes in two forms, exactly like port connections: **positional** (`#(64, 32)` - values assigned in declaration order, you cannot skip one) and **named** (`#(.DEPTH(64))` - any subset, in any order). Named association is the only style that survives a reordering of the parameter list in the next IP release, so every coding guideline requires it. A parameter that is not mentioned keeps its default, and **localparams** cannot be named at all:

```
module m #(parameter A = 4, localparam B = A * 2) ();
    initial $display("%m A=%0d B=%0d", A, B);
endmodule
module top3;
    m #(.A(3), .B(9)) u ();
endmodule
```

p3.v - attempting to override a localparam.

```
p3.v:5: error: Cannot override localparam `B` in `top3.u`.
2 error(s) during elaboration.
```

iverilog -g2012 -Wall p3.v

The second mechanism is the **defparam** statement, which assigns a new value to any parameter anywhere in the design through a **hierarchical name**. It was the only override mechanism in the earliest Verilog and it still works in every tool, but IEEE 1800-2017 lists it in Annex C as **deprecated**, and it is banned by virtually every company coding standard. The example shows why:

```
module leaf #(parameter W = 1) ();
    initial $display("%m W=%0d", W);
endmodule
module mid;
    leaf u_leaf ();
endmodule
module top4;
    mid u_a ();
    mid u_b ();
    defparam u_a.u_leaf.W = 8;           // reaches down two levels, from outside
endmodule
module other;                          // another top-level module...
    defparam top4.u_b.u_leaf.W = 16;    // ...silently changes a design it does not own
endmodule
```

p4.v - defparam is action at a distance.

```
top4.u_a.u_leaf W=8
top4.u_b.u_leaf W=16
```

Icarus: both overrides take effect, one of them from a completely unrelated top-level module.

- **Action at a distance.** Reading `mid` or `leaf` tells you nothing about the value `W` will have; it can be changed from any file in the compile list, including a testbench file that the synthesis tool never sees - a guaranteed simulation/synthesis mismatch.
- **Precedence surprises.** If a parameter is set both by `#()` and by `defparam`, the `defparam` wins; if two `defparams` target the same parameter the result depends on source order.
- **Elaboration complexity.** A `defparam` can change a parameter that controls a generate construct, which creates new hierarchy that may contain further `defparams`. The LRM needs special rules (a `defparam` inside a generate block or instance array may only affect its own sub-hierarchy) and tools elaborate it iteratively.
- **Tool support.** Several synthesis and formal tools reject it, or only accept it when it targets a direct child instance.

HOUSE RULE

Override parameters only with named `#(.NAME(value))` association at the point of instantiation. Derived constants are `localparam`. Never write `defparam` in new code; when you inherit it in legacy IP, convert it to `#()` overrides before the first synthesis run.

7.3 Parameter dependencies and constant expressions

A parameter's default may refer to parameters declared before it. The default expression is evaluated **after** overrides are applied, so a dependent parameter automatically follows an overridden one - unless the dependent parameter is itself overridden, at which point the relationship is broken silently. This is the real reason derived values should be `localparam`:

```
module cfg_reg #(
    parameter W      = 8,           // untyped: becomes whatever is passed in
    parameter MAX    = (1 << W) - 1, // depends on W: re-evaluated after override
    parameter [W-1:0] RST = MAX     // its range also follows W
) ();
    initial $display("%m: W=%0d MAX=%0d RST=%h $bits(W)=%0d", W, MAX, RST, $bits(W));
endmodule

module top5;
    cfg_reg          a ();
    cfg_reg #(.W(4)) b (); // MAX and RST follow W
    cfg_reg #(.W(4), .MAX(100)) c (); // MAX overridden -> no longer tied to W
    cfg_reg #(.W(12'd10)) d (); // untyped W now has 12 bits
endmodule
```

p5.v - dependent parameters.

```
top5.a: W=8 MAX=255 RST=ff $bits(W)=32
top5.b: W=4 MAX=15 RST=f $bits(W)=32
top5.c: W=4 MAX=100 RST=4 $bits(W)=32
top5.d: W=10 MAX=1023 RST=3ff $bits(W)=12
```

Instance c: 100 does not fit in 4 bits, so RST silently becomes 4. Instance d: W changed its own width to 12 bits.

Constant functions

When a derived constant needs more than an expression - a log2, a maximum over a list, a CRC table entry - Verilog-2001 lets you call a **constant function**: an ordinary function of the same module that is called with constant arguments in a place where a constant is required. The LRM (1364-2005 10.4.5, 1800-2017 13.4.3) restricts it so the elaborator can execute it: it may use only its arguments, its own locals, parameters and other constant functions; no hierarchical references, no timing controls, no non-constant system functions, and (Verilog) it must be declared in the module that calls it. The built-in `$clog2` (ceiling of log2, added in 1364-2005) replaced the hand-written version in most code, but the pattern is still useful and appears in older IP:

```
module enc #(parameter N = 8) (input [N-1:0] req, output [$clog2(N)-1:0] idx);
    // Constant function: legal in a constant expression because it is
    // side-effect free and uses only its arguments and parameters.
    function integer clog2(input integer v);
        integer k;
        begin
            clog2 = 0;
            for (k = v - 1; k > 0; k = k >> 1) clog2 = clog2 + 1;
        end
    endfunction
    localparam W = clog2(N);
    reg [W-1:0] r; integer j;
    always @* begin
        r = {W{1'b0}};
    end
```

```

    for (j = N-1; j >= 0; j = j - 1) if (req[j]) r = j[W-1:0];    // lowest index wins
end
assign idx = r;
initial $display("%m: N=%0d -> W=%0d (builtin $clog2=%0d)", N, W, $clog2(N));
endmodule

```

g3.v (part 1) - a constant function sizing a localparam. The port uses the built-in \$clog2, because referring to a function before its declaration is not portable.

You need a width that can hold...	Expression	Gotcha
an index 0 .. N-1 (address of an N-entry RAM)	<code>\$clog2(N)</code>	N = 1 gives 0: a zero-width vector [-1:0]. Use <code>(N > 1) ? \$clog2(N) : 1</code>
a count 0 .. N (FIFO occupancy, N inclusive)	<code>\$clog2(N+1)</code>	<code>\$clog2(16)</code> = 4 cannot hold 16
a value up to MAX	<code>\$clog2(MAX+1)</code>	Same off-by-one
N items, N a power of two, one extra wrap bit	<code>\$clog2(N)+1</code>	Classic FIFO pointer (companion RTL Design guide)

Sizing with \$clog2.

7.4 Generate constructs

A **generate construct** is structure that is created, replicated or discarded during elaboration according to constant expressions. Since 1364-2005 a generate construct can appear directly in the module body; the `generate ... endgenerate` keywords that enclose it (a **generate region**) are optional and have no semantic effect. There are three kinds:

Construct	Chooses/replicates	Typical SoC use
Loop generate <code>for (genvar ...)</code>	N copies of the body, one scope per iteration	Per-lane datapaths, bit-slices, arrays of FIFOs, per-master arbitration logic, synchronizer banks
Conditional generate <code>if</code>	At most one of the branches	Optional features (ECC on/off), technology selection (behavioural model vs. hard macro), small/large implementations
Case generate <code>case</code>	At most one of the items	Selecting among several architectures by an integer or string parameter

A generate block may contain almost any **module item**: net and variable declarations, continuous assignments, `always` and `initial` blocks, module/primitive instances, functions, tasks, `localparams` and nested generate constructs. It may **not** contain port declarations, `specify` blocks or `specparams`, and a parameter declared inside it can never be overridden - write `localparam`.

Loop generate and genvar

The loop variable must be a **genvar**: an integer that exists only during elaboration. It can be declared before the loop (`genvar i;`) or, in SystemVerilog, inline (`for (genvar i = 0; ...)`). The initialisation, condition and step must be constant expressions, the genvar may not be assigned anywhere else, and two nested loops may not use the same genvar. Each iteration produces a separate **generate block instance** named `name[i]`; declarations inside the block are local to that iteration, which is what makes `wire p` below a different net for every bit:

```

// Ripple-carry adder built with a generate-for loop
module rca #(parameter N = 4) (
    input  [N-1:0] a, b, input cin,
    output [N-1:0] s, output cout);
    wire [N:0] c;
    assign c[0] = cin;
    genvar i;
    generate
        for (i = 0; i < N; i = i + 1) begin : g_bit    // named block -> g_bit[0..N-1]

```

```

        wire p = a[i] ^ b[i]; // local net, one per iteration
        assign s[i] = p ^ c[i];
        assign c[i+1] = (a[i] & b[i]) | (p & c[i]);
    end
endgenerate
assign cout = c[N];
endmodule

module tb;
    reg [3:0] a, b; reg cin; wire [3:0] s; wire co;
    rca #(4) dut (.a(a), .b(b), .cin(cin), .s(s), .cout(co));
    initial begin
        a = 4'd9; b = 4'd8; cin = 1'b1; #1;
        $display("9+8+1 = %0d (cout=%b s=%0d)", {co, s}, co, s);
        $display("propagate of bit 3 = %b (tb.dut.g_bit[3].p)", dut.g_bit[3].p);
    end
endmodule

```

g1.v - a loop generate and a hierarchical reference into one of its iterations.

```

9+8+1 = 18 (cout=1 s=2)
propagate of bit 3 = 0 (tb.dut.g_bit[3].p)

```

Icarus Verilog 12.

LOOP GENERATE OR PROCEDURAL FOR LOOP?

A **for** loop inside an **always @*** block is also unrolled by synthesis, and is often simpler (the priority encoder above uses one). Use a **generate** loop when each iteration needs its own **instances**, its own **always** block or its own continuous assignments, or when you need a hierarchical name per iteration (for debug, for SDF back-annotation, for a UPF power domain or for a DFT insertion script). Use a procedural loop for pure combinational functions of vectors.

Conditional generate, case generate and block names

In a conditional generate the condition is evaluated once; only the chosen branch exists in the elaborated design. A useful idiom is to give **every branch the same block name**: because only one branch survives, the name is unique, and scripts, assertions and hierarchical references work whichever implementation was chosen. An **else if** chain does not create an extra level of hierarchy (the LRM's "direct nesting" rule), so all branches below are siblings:

```

module mult #(parameter W = 8, parameter IMPL = "FAST") (
    input [W-1:0] a, b, output [2*W-1:0] y);
    generate
        if (W <= 4) begin : g_impl // 'generate' keyword is optional
            assign y = a * b; // same name in every branch:
            // the path is g_impl whichever
            initial $display("%m: small LUT multiplier"); // branch is chosen
        end else if (IMPL == "FAST") begin : g_impl
            assign y = a * b;
            initial $display("%m: fast (Booth/Wallace) multiplier");
        end else begin : g_impl
            assign y = a * b;
            initial $display("%m: slow iterative multiplier");
        end
    endgenerate
    if (W > 16) begin // unnamed: tool names it genblkN
        initial $display("%m: wide!");
    end
endmodule

module tb2;
    wire [7:0] y4; wire [15:0] y8; wire [47:0] y24;
    mult #(4) m4 (4'd3, 4'd5, y4);
endmodule

```

```

    mult #(8, "SLOW") m8 (8'd3, 8'd5, y8);
    mult #(24) m24 (24'd3, 24'd5, y24);
    initial #1 $display("y4=%0d y8=%0d y24=%0d", y4, y8, y24);
endmodule

```

g2.v - conditional generate; a string parameter compared at elaboration time.

```

tb2.m4.g_impl: small LUT multiplier
tb2.m8.g_impl: slow iterative multiplier
tb2.m24.g_impl: fast (Booth/Wallace) multiplier
tb2.m24.genblk2: wide!
y4=15 y8=15 y24=15

```

Icarus Verilog 12.

The last line shows the rule for **unnamed** generate blocks (1800-2017 27.6): each generate construct in a scope is numbered 1, 2, 3... in source order - named or not - and an unnamed block of construct number n is called **genblk<n>**. Our **if (W > 16)** is the second generate construct in **mult**, hence **genblk2**. If that name clashes with a user identifier, leading zeros are inserted (**genblk02**). Because inserting a new construct above it renumbers the block, and because older tools used different schemes, **never leave a generate block that contains declarations or instances unnamed**: its hierarchical path appears in waveform setups, SDF files, UPF, synthesis constraints and formal scripts.

The generate **case** works the same way and is the natural choice when a parameter selects among more than two architectures:

```

module rst_sync #(parameter STYLE = 1) (input clk, rst_n, output q);
    generate
        case (STYLE)                                // generate-case
            0:      begin : g_s  assign q = rst_n;      end
            1:      begin : g_s  reg [1:0] f = 0;
                    always @(posedge clk) f <= {f[0], rst_n};
                    assign q = f[1];                  end
            default: begin : g_s  reg [2:0] f = 0;
                    always @(posedge clk) f <= {f[1:0], rst_n};
                    assign q = f[2];                  end
        endcase
    endgenerate
    initial $display("%m: STYLE=%0d", STYLE);
endmodule

module tb3;
    reg [11:0] req; wire [3:0] idx; wire [2:0] i5;
    enc #(12) e12 (.req(req), .idx(idx));
    enc #(5) e5 (.req(5'b10100), .idx(i5));
    reg clk = 0; wire q0, q2;
    rst_sync #(0) r0 (clk, 1'b1, q0);
    rst_sync #(3) r3 (clk, 1'b1, q2);
    always #5 clk = ~clk;
    initial begin
        req = 12'b0010_1000_0000;
        #1 $display("idx=%0d i5=%0d", idx, i5);
        #30 $display("q0=%b q2=%b bits(r3.g_s.f)=%0d", q0, q2, $bits(r3.g_s.f));
        $finish;
    end
endmodule

```

g3.v (part 2) - generate-case; the testbench also exercises the enc module shown earlier.

```
tb3.e12: N=12 -> W=4 (builtin $clog2=4)
tb3.e5: N=5 -> W=3 (builtin $clog2=3)
tb3.r0: STYLE=0
tb3.r3: STYLE=3
idx=7 i5=2
q0=1 q2=1 bits(r3.g_s.f)=3
g3.v:48: $finish called at 31 (1s)
```

Icarus Verilog 12. For STYLE=3 the default item built a 3-flop synchronizer; \$bits sees its 3-bit register through the g_s scope.

PITFALL: A GENERATE CONDITION IS NOT A RUN-TIME CONDITION

`if (MODE == 1)` at module level is a **generate** if - MODE must be a constant and the branch is chosen once. The same text inside an **always** block is a procedural if and synthesizes to a multiplexer. If MODE is accidentally a signal, the module-level form is an elaboration error, which is good; but a parameter that was meant to be a run-time register field silently becomes hard-wired logic.

Instance arrays

For the common case of N identical instances with sliced ports, Verilog also has **arrays of instances**: `inv u_inv [7:0] (.a(a), .y(y));` connects an 8-bit `a` bit-by-bit to eight 1-bit instances named `u_inv[7]` ... `u_inv[0]`. The port-slicing rule is simple (a vector whose width equals N x port width is divided evenly; a signal of exactly the port width is broadcast to all), but a generate loop is more flexible and more readable, and it is what most teams use.

7.5 Configurations and libraries

Parameters decide the *values* inside a module; a **configuration** decides *which description* of a module is used for each instance. Verilog-2001 introduced the concept to solve a real problem of mixed-level simulation: you want the CPU subsystem as gates (to check the netlist from synthesis) while the rest of the SoC stays at RTL, and both descriptions define a module called `cpu_core`.

Two pieces are involved. A **library map file** (tool option, e.g. `-libmap lib.map`) assigns source files to named **libraries**; a module compiled from those files becomes the **cell** `lib.module`. The **config** block then gives binding rules that the elaborator applies while it builds the hierarchy from the top:

```
// lib.map - which files go into which library
library rtl_lib  "rtl/*.v";
library gate_lib "netlist/*.v";
library tb_lib   "tb/*.sv";

// cfg.v - the binding rules
config soc_gate_cpu;
  design tb_lib.tb_top;                // the top-level cell(s)
  default liblist rtl_lib gate_lib;    // search order for every instance
  instance tb_top.dut.u_cpu liblist gate_lib; // this instance (and below): netlist
  cell      sram_32x1024 use rtl_lib.sram_model; // every sram_32x1024 -> a model
  instance tb_top.dut.u_dma use rtl_lib.dma_v2; // bind one instance to another cell
endconfig
```

A library map and a configuration. Simulating the configuration (instead of a top module) elaborates `tb_top` with the CPU from the netlist and everything else from RTL.

Clause	Meaning
<code>design lib.cell</code>	Top-level cell(s) the configuration elaborates
<code>default liblist a b</code>	Libraries searched, in order, for any cell not bound by a more specific rule
<code>instance path liblist ...</code>	Search list for one instance and, by inheritance, its sub-hierarchy
<code>instance path use lib.cell</code>	Bind one instance to a specific cell (optionally <code>:config</code> to use another configuration hierarchically)

Clause	Meaning
<code>cell name use lib.cell</code>	Bind every instance of a cell name
<code>cell name liblist ...</code>	Search list for every instance of a cell name

```
cfg.v:3: sorry: config declarations are not supported and will be skipped.
```

Icarus Verilog 12 (Verilator 5.020 reports "Unsupported: Verilog 2001-config reserved word not implemented"). Configurations need a commercial simulator such as VCS, Xcelium or Questa.

In practice many teams achieve the same effect with simpler mechanisms: compiling different file lists (the netlist file instead of the RTL file), tool-specific library options (`-y`, `-v`, `-L lib`), or a macro that selects which `include` or instance to use. Configurations remain valuable in large mixed-signal and gate-level regressions where many variants of the same testbench must be elaborated from one set of compiled libraries. SystemVerilog extended them only slightly (for example with parameter overrides inside a config in 1800-2009).

7.6 Where this is used on the SoC/ASIC roadmap

Role	What you do with parameters and generate
RTL designer	Write IP once with typed, named, documented parameters; derive every width with <code>localparam</code> and <code>\$clog2</code> ; use generate for per-lane/per-port structure; add elaboration-time checks for illegal combinations
SoC integrator	Instantiate IP with named overrides; generate the top level (often from IP-XACT or a Python script) with the right parameter sets
DV engineer	Run the regression over several parameter sets; bind assertions into generate scopes; read hierarchical names like <code>g_lane[3].u_fifo</code> in failures
Synthesis / STA	Constraints and reports use elaborated names (<code>u_dma/g_ch_2_u_fifo/...</code> after uniquification) - unnamed genblks make scripts fragile
Gate-level / DFT	Configurations or file lists swap RTL for netlists; SDF instance paths must match generate block names

INTERVIEW INSIGHT: CHECKING PARAMETERS AT ELABORATION

A robust IP rejects illegal parameter values instead of silently building broken hardware. In SystemVerilog use an elaboration-time system task inside a generate if: `if (DEPTH < 2) $error("DEPTH must be >= 2");` (1800-2009 onwards, reported during elaboration). In plain Verilog the classic trick is to instantiate a non-existent module inside the failing branch so that elaboration stops with a readable module name such as `DEPTH_must_be_a_power_of_2 u_err();`.

7.7 Summary

- Parameters, generate and configurations are all resolved during **elaboration**; afterwards the design is flat and fully sized.
- `parameter` can be overridden per instance; `localparam` cannot. With a parameter port list, body `parameters` become local (1800).
- The declared type/range decides what an override becomes; untyped parameters take the width of whatever value is passed in.
- Override with named `#(.P(v)).defparam` is deprecated (1800 Annex C), acts at a distance and takes precedence over `#()`.
- Dependent defaults are re-evaluated after overrides; derive with `localparam` so the relationship cannot be broken.
- Constant functions and `$clog2` compute widths; watch the N=1 and count-to-N off-by-one cases.
- Loop/if/case generate create named scopes; name every block (`begin : g_name`) - unnamed blocks become `genblk<n>`.

- Configurations bind instances to library cells for mixed RTL/gate simulation; open-source tools do not support them.

7.8 Exercises

- 1 Write a parameterized `onehot_mux #(N, W)` using a generate loop that ANDs each input with its select bit and ORs the results. Verify with N=1, 3 and 8.
- 2 A colleague writes `parameter WIDTH = 8; parameter MAXV = 2**WIDTH - 1;` and instantiates with `#(.WIDTH(4'd12))`. What are WIDTH, MAXV and \$bits(WIDTH)? Check your answer with Icarus.
- 3 Explain what `genblk3` in a waveform refers to, and why adding an `if generate` at the top of the module changes the name of an unrelated block. How do you prevent this?
- 4 Implement a FIFO whose depth may be any integer ≥ 2 and add an elaboration-time check that rejects $DEPTH < 2$ (both the SV \$error way and the Verilog-2001 missing-module trick).
- 5 Rewrite a legacy testbench that uses three `defparam` statements so that it uses only `#()` overrides. What changes if one of the targets is inside a generate loop?
- 6 Write the config block that simulates `tb.dut` at RTL except for every instance of cell `pll_wrap`, which must use `ams_lib.pll_wrap`.

8. Compiler Directives and Macros

Before a Verilog compiler parses a single module, a **preprocessor** reads the source text and acts on **compiler directives** - lines introduced by the grave accent character (ASCII 0x60, the "back-tick"). Directives define text macros, include other files, select code conditionally, and set compilation state such as the time unit or the default net type. They are not part of any module and they do not respect module boundaries: a directive stays in effect from the point where it is read **until it is changed, across file boundaries**, for the rest of the compilation unit. That single fact explains most of the pitfalls in this chapter.

NOTATION IN THIS CHAPTER

In running text the leading grave accent of a directive is omitted and the name is set in code font: the `define` directive, the `timescale` directive. Code listings show the real syntax, including the grave accent.

Directive	Purpose	Standard
<code>define, undef, undefineall</code>	Text macros (with arguments since 1364-1995, default arguments since 1800-2009)	1364 / 1800-2009
<code>ifdef, ifndef, elsif, else, endif</code>	Conditional compilation	1364-2001
<code>include</code>	Textual file inclusion	1364
<code>timescale</code>	Time unit and precision for delays	1364
<code>default_nettype</code>	Type of implicitly declared nets (<code>wire</code> , <code>none</code> , ...)	1364-2001
<code>resetall</code>	Reset all directives except macros to defaults	1364
<code>celldefine, endcelldefine</code>	Mark modules as library cells	1364
<code>unconnected_drive, nounconnected_drive</code>	Pull unconnected input ports up or down	1364
<code>line</code>	Set file name and line number for messages	1364-2005
<code>__FILE__, __LINE__</code>	Current file name / line number	1800-2009
<code>pragma</code>	Tool-specific directives with standard syntax (e.g. <code>protect</code> encryption envelopes)	1364-2005
<code>begin_keywords, end_keywords</code>	Select the reserved-word set of a given standard	1364-2005 / 1800

The complete directive set.

8.1 Text macros: define, arguments and undef

`define NAME text` defines a **text macro**; every later use of the name preceded by a grave accent is replaced by the text. The text runs to the end of the line; a backslash immediately before the newline continues it on the next line. A macro may take **formal arguments** in parentheses directly after its name (no space is allowed between the name and the parenthesis, otherwise the parenthesis becomes part of the text). SystemVerilog adds **default argument values**: a formal written as `msg = "text"` may be omitted at the call site. Macro substitution is purely textual - there is no type checking, no scoping and no evaluation, which gives both the power and the danger of macros.

```
`ifndef DEFS_VH
`define DEFS_VH
`define DATA_W 32
`define MAX(a, b) ((a) > (b) ? (a) : (b))
`define BAD_MAX(a, b) a > b ? a : b
`define SQ(x) ((x) * (x))
`define BAD_SQ(x) x * x
`endif
```

defs.vh - a header with an include guard and two good and two bad macros.

```

`include "defs.vh"
`include "defs.vh"           // second include is harmless thanks to the guard
`define CHECK(cond, msg = "check failed") \
    if (!(cond)) $display("ERROR %s:%0d: %s", `__FILE__, `__LINE__, msg);
`define STR(x) `"x"`
`define REG(name, w) reg [w-1:0] name``_q, name``_d;
module m1;
    `REG(cnt, 4)              // declares cnt_q and cnt_d
    initial begin
        cnt_q = 4'd9; cnt_d = 4'd3;
        $display("MAX=%0d 1+MAX*2=%0d 1+BAD_MAX*2=%0d",
            `MAX(cnt_q, cnt_d), 1 + `MAX(cnt_q, cnt_d) * 2, 1 + `BAD_MAX(cnt_q, cnt_d) * 2);
        $display("SQ(1+2)=%0d BAD_SQ(1+2)=%0d", `SQ(1+2), `BAD_SQ(1+2));
        $display("DATA_W=%0d stringified: %s", `DATA_W, `STR(cnt_q + 1));
        `CHECK(cnt_q < 8, "cnt_q out of range")
        `CHECK(cnt_d == 0)           // uses the default message
    `ifdef FAST_SIM
        $display("FAST_SIM build");
    `elsif GATE_SIM
        $display("GATE_SIM build");
    `else
        $display("default RTL build");
    `endif
    end
endmodule

```

m1.sv - macros with arguments, a default argument, stringification, token pasting and conditional compilation.

```

MAX=9 1+MAX*2=19 1+BAD_MAX*2=9
SQ(1+2)=9 BAD_SQ(1+2)=5
DATA_W=32 stringified: cnt_q + 1
ERROR m1.sv:15: cnt_q out of range
ERROR m1.sv:16: check failed
default RTL build

```

iverilog -g2012 -Wall m1.sv && vvp. With -DGATE_SIM the last line becomes "GATE_SIM build".

Read the results carefully - each line illustrates a rule:

- **BAD_MAX** expands to `1 + cnt_q > cnt_d ? cnt_q : cnt_d * 2`. Because `+` binds tighter than `>` and `?:` binds loosest, this is `(10 > 3) ? 9 : 6 = 9`. **Parenthesise the whole body and every use of every argument**, as **MAX** and **SQ** do.
- **BAD_SQ(1+2)** expands to `1+2 * 1+2 = 5`, not 9. The argument is text, not a value.
- The SystemVerilog **stringification** escape (a grave accent followed by a double quote) lets the macro argument be substituted inside a string literal; without it, text inside quotes is never substituted. The escape (grave accent + backslash + double quote) produces a literal quote inside such a string.
- The **token-pasting** escape (two grave accents) joins text without whitespace: `name`, the escape and `_q` with `name = cnt` become the single identifier `cnt_q`. It is the basis of register-bank and UVM field macros.
- `__FILE__` and `__LINE__` expand to the file and line **where the macro is used** (lines 15 and 16), which makes them ideal in checker macros.

Redefinition and undef

A macro may be redefined; the new text applies from that point on (most tools warn). `undef NAME` removes one definition and `undefineall` (1800-2009) removes all of them. Because macros are global across files, a macro defined in one IP's file is visible - and can be clobbered - in every file compiled after it:

```

`define W 8
module ud1; initial $display("ud1 sees W=%0d", `W); endmodule
`undef W
`define W 16 // redefinition (tools may warn if not undef'd)
module ud2; initial $display("ud2 sees W=%0d", `W); endmodule
module udt; ud1 a(); ud2 b(); endmodule

```

ud.v - the value of a macro depends on where in the text it is used, not on the hierarchy.

```

ud1 sees W=8
ud2 sees W=16

```

Icarus Verilog 12.

PITFALL: GLOBAL MACRO NAMES COLLIDE BETWEEN IPS

Two vendors' IPs both define `WIDTH` or `ADDR_BITS`. Whichever file is compiled last wins for every file after it, and the result changes when the file list is reordered. Rules used by every SoC team: prefix every macro with the IP name (`DMA_ADDR_W`); prefer parameters and package constants (Chapter 15) for anything that is a value; `undef` local helper macros at the end of the file that defines them.

8.2 Conditional compilation

`ifdef NAME` compiles the following text if a macro called `NAME` is defined (its value is irrelevant); `ifndef` is the opposite; `elsif` and `else` provide alternatives and `endif` closes the group. Groups may nest. The text in a group that is not selected is skipped by the preprocessor and need only be lexically valid. Macros are usually defined from the command line: `+define+NAME=value` (VCS, Xcelium, Questa) or `-DNAME=value` (Icarus, Verilator, Yosys).

Typical guard	Used for
<code>ifdef SYNTHESIS / ifndef SYNTHESIS</code>	Excluding simulation-only code (assertions, <code>\$display</code> , models) from synthesis. Most synthesis tools define <code>SYNTHESIS</code> automatically; Yosys does, and Verilator defines <code>VERILATOR</code>
<code>ifdef GATE_SIM</code>	Different stimulus or checks for gate-level simulation (e.g. SDF annotation, X-handling)
<code>ifdef ASIC / ifdef FPGA</code>	Technology selection: SRAM macro wrapper vs. inferred block RAM, clock gating cell vs. enable
<code>ifdef NO_ASSERT</code>	Turning checkers off in long runs
Include guard <code>ifndef X_VH define X_VH ... endif</code>	Making a header safe to include more than once

PITFALL: IFDEF-ED RTL IS A SIM/SYNTH MISMATCH GENERATOR

Code under `ifndef SYNTHESIS` is simulated but never built. If it contains anything that affects functionality - an initial value, a forced signal, a behavioural model - the netlist behaves differently from the RTL you verified. Keep such regions to pure observation (messages, assertions, coverage), and make lint check both configurations. Prefer generate-if on a parameter for legitimate architectural variants: both branches are then parsed and type-checked by every tool, and the choice is visible in the hierarchy.

8.3 include and include guards

`include "file"` inserts the complete contents of the file in place of the directive, exactly as if it had been typed there. The file is searched first relative to the current directory (tool dependent: the current working directory or the including file's directory) and then in the include directories given with `+incdir+dir` or `-I dir`. The angle-bracket form `include <file>` (1800-2009) searches only the tool's own library directories. Includes may nest (the LRM requires at least 15 levels).

Use includes for **declarations shared textually**: macro headers, register address maps generated from IP-XACT, and - in SystemVerilog - class files that are included into one package. Do **not** include module definitions into other modules' files; list them on the command line instead, so that every module is compiled exactly once. Every header must have an **include guard**, as `defs.vh` above does; otherwise a second inclusion redefines every macro (a warning) or declares every item twice (an error).

8.4 timescale: time units, precision and compile-order dependence

`timescale unit / precision` gives delays in the following modules a **time unit** (what `#1` means) and a **precision** (to what granularity delays are rounded). Both are 1, 10 or 100 followed by s, ms, us, ns, ps or fs, and the precision must be at least as fine as the unit. The simulator's global time step is the finest precision of any module in the design. Delays are rounded to the module's precision; `$time` returns the current time **rounded to the module's unit** as a 64-bit integer, `$realtime` returns it as a real in units, and `%t` formats a time according to `$timeformat` (Chapter 10).

```
`timescale 1ns/100ps
module ts1;
  initial begin
    #1.55 $display("ts1: $time=%0d $realtime=%0.2f", $time, $realtime);
    #0.04 $display("ts1: after #0.04 more, $realtime=%0.2f", $realtime);
  end
endmodule
```

ts1.v - rounding to the precision.

```
module ts2;                                // no `timescale of its own!
  initial #10 $display("ts2: #10 done; $realtime=%0.1f in ts2 units", $realtime);
endmodule
```

ts2.v - a file that relies on whatever timescale is in effect when it is compiled.

```
module tstop;
  ts1 a(); ts2 b();
  initial begin
    $timeformat(-9, 1, " ns", 12);
    #20 $display("tstop: simulation time is %t", $realtime);
  end
endmodule
```

tstop.v - also has no timescale.

```
== iverilog ts1.v ts2.v tstop.v ==
ts1: $time=2 $realtime=1.60
ts1: after #0.04 more, $realtime=1.60
ts2: #10 done; $realtime=10.0 in ts2 units
tstop: simulation time is      20.0 ns
== iverilog ts2.v tstop.v ts1.v ==
warning: Found both default and explicit timescale based delays. Use
       : -Wtimescale to find the design element(s) with no explicit
       : timescale.
ts1: $time=2 $realtime=1.60
ts1: after #0.04 more, $realtime=1.60
ts2: #10 done; $realtime=10.0 in ts2 units
tstop: simulation time is 20000000000.0 ns
```

Same three files, two compile orders, different simulations.

The first run shows the rounding rules: `#1.55` with 100 ps precision becomes 1.6 ns, `$time` rounds 1.6 to 2, and `#0.04` rounds to zero delay. The second run shows the **compile-order dependence**. In the first order, ts2 and tstop inherit `1ns/100ps` from ts1 because the directive is still in effect. In the second order they are compiled before any `timescale` and receive the tool's default (1 s in Icarus), so `#20` in tstop now means 20 seconds. The LRM says it is an error for some modules to have a timescale and others not; most tools only warn, and the behaviour of modules without one is tool-specific.

ROBUST TIME UNITS

Put a `timescale` directive in **every** file that contains delays (or in none, and give the default on the command line - for example Verilator's `--timescale 1ns/1ps` or the `-timescale` option of the commercial simulators - which applies only to files without one). In SystemVerilog, prefer the `timeunit 1ns; timeprecision 1ps;` declarations inside the module or package: they belong to the scope, not to the compilation order. Choose a precision fine enough for your fastest clock (1 ps for multi-GHz designs) but no finer than needed - a 1 fs global precision slows some simulators measurably.

Symptom	Likely timescale cause
Clock runs 1000x too slow or fast in one sub-block	That file compiled with a different inherited timescale
Gate-level sim: all cell delays are zero or huge	Library cell files carry their own timescale (often 1ns/1ps) that differs from the SDF TIMESCALE or the testbench
<code>#0.5</code> behaves like <code>#0</code> or <code>#1</code>	Precision coarser than the delay: rounded
Waveform timestamps disagree with printed \$time	\$time is rounded to the unit of the module that calls it

8.5 default_nettype, resetall, celldefine and unconnected_drive

Verilog creates an **implicit net** whenever an undeclared identifier is used in a port connection or on the left of a continuous assignment (Chapter 3). By default the implicit net is a 1-bit `wire`, which turns every typo into a silent floating signal. `default_nettype none` disables implicit nets so that the typo becomes a compile error:

```
`ifdef STRICT
`default_nettype none
`endif
module and2 (input a, b, output y); assign y = a & b; endmodule
module dn;
    reg a = 1, b = 1;
    wire y;
    and2 u (.a(a), .b(b), .y(y_out)); // typo: y_out was never declared
    initial #1 $display("y=%b", y);
endmodule
`default_nettype wire // restore for files compiled after this one
```

dn.v - an implicit net hides a typo.

```
== default ==
dn.v:8: warning: implicit definition of wire 'y_out'.
y=z
== -DSTRICT ==
dn.v:8: error: Net y_out is not defined in this context.
dn.v:8: error: Output port expression must support continuous assignment.
dn.v:8:      : Port 3 (y) of and2 is connected to y_out
2 error(s) during elaboration.
```

Icarus Verilog 12 with -Wall.

Note the last line of `dn.v`: because the directive stays in effect for the rest of the compilation, a file that sets `none` should restore `wire` at its end, otherwise third-party files compiled later - which often rely on implicit nets in port lists of the Verilog-1995 style - stop compiling. Other values are `tri`, `tri0`, `tri1`, `wand`, `wor`, `triand`, `trior`, `triereg` and (1364-2005) `uwire`.

Directive	Effect and use
<code>resetall</code>	Resets every directive except text macros to its default (timescale, default_nettype, celldefine, unconnected_drive). Some teams start every file with it; it must not appear inside a module.

Directive	Effect and use
<code>celldefine ... endcelldefine</code>	Marks the enclosed modules as cells . Tools use it to treat them as leaves: <code>\$dumpvars</code> and coverage can exclude them, PLI's <code>acc_object_of_type(accCell)</code> and VPI's <code>vpiCellInstance</code> report them. Every standard-cell simulation library uses it (Chapter 9).
<code>unconnected_drive pull1 / pull0 ... nounconnected_drive</code>	Unconnected input ports of modules in the region are pulled to 1 or 0 instead of floating to z. Found in old libraries; explicit tie-offs are better practice.
<code>begin_keywords "1364-2005" ... end_keywords</code>	Parses the enclosed code with the reserved words of that standard, so that legacy code using identifiers such as <code>logic</code> or <code>bit</code> still compiles in a SystemVerilog tool.
<code>pragma name ...</code>	Standard syntax for tool pragmas; IEEE 1364-2005 defines <code>pragma protect</code> for IP encryption envelopes.

8.6 line, FILE and LINE

`line number "file" level` tells the compiler to report subsequent lines as coming from another file and line - used by code generators (register-map generators, SystemVerilog-to-Verilog converters, Chisel/SpinalHDL emitters) so that error messages point to the original source. The level is 0, 1 (entering an include file) or 2 (leaving one). SystemVerilog's `__FILE__` and `__LINE__` expand to a string literal and a decimal number and follow any `line` directive:

```
module ln;
  initial begin
    $display("A: %s line %0d", `__FILE__, `__LINE__);
    `line 100 "generated_regs.v" 0
    $display("B: %s line %0d", `__FILE__, `__LINE__);
  end
endmodule
```

ln.v

```
A: ln.v line 3
B: generated_regs.v line 100
```

Icarus Verilog 12.

8.7 Macro pitfalls and good practice

Because macros are text, a macro that expands to several statements behaves differently from a function call in control flow:

```
`define INC2_BAD(v)  v = v + 1; v = v + 1;
`define INC2(v)      begin v = v + 1; v = v + 1; end
module mp;
  integer x, y;
  initial begin
    x = 0; y = 0;
    if (0) `INC2_BAD(x)      // expands to: if (0) x = x + 1; x = x + 1;
    if (0) `INC2(y)          // whole body is guarded
    $display("x=%0d (expected 0) y=%0d", x, y);
  end
endmodule
```

mp.v - a multi-statement macro under an if.

```
x=1 (expected 0) y=0
```

Icarus Verilog 12.

Pitfall	Remedy
Operator precedence inside the expansion	Parenthesise the body and each argument use

Pitfall	Remedy
Argument evaluated twice (<code>MAX(i++, j)</code>) - side effects doubled	Use a function; macros are not functions
Multi-statement body under <code>if/else</code>	Wrap the body in <code>begin ... end</code> (the C idiom <code>do begin ... end while (0)</code> is also legal SystemVerilog)
Trailing semicolon: <code>MACRO(x);</code> adds an empty statement, which breaks <code>if (c) M(x); else ...</code>	Decide once whether the macro includes the semicolon and be consistent
Name collisions between IPs; redefinition order	Prefix names; <code>undef</code> local helpers; use packages
Comments inside a macro body: a <code>//</code> comment ends the macro text at the end of that line	Use <code>/* */</code> inside multi-line macros, or put comments outside
Space between name and <code>(</code> in the definition	<code>define F (x) x</code> defines F with no argument whose text is <code>(x) x</code>
Macros are invisible in waveforms and hard to debug	Keep them small; most tools can write out the preprocessed text (for example <code>iverilog -E</code> , <code>verilator -E</code>)

WHERE MACROS ARE THE RIGHT TOOL

UVM's `uvm_info`, `uvm_error` and field-automation macros; assertion shorthands such as `ASSERT_KNOWN(name, sig, clk, rst)` (used in OpenTitan and many SoC code bases) that expand into a labelled concurrent assertion and capture `__FILE__/__LINE__`; register-bank declarations built with token pasting; and portable wrappers around vendor-specific constructs. Everything that is merely a constant should be a parameter or a package `localparam`: it has a type, a scope, and it shows up in the debugger.

8.8 Summary

- Directives are processed before parsing and stay in effect across file boundaries until changed; compile order therefore matters.
- Macros are pure text substitution: parenthesise, wrap statements in `begin/end`, prefix names, `undef` helpers.
- SystemVerilog adds default macro arguments, stringification with the grave-accent-quote escape, token pasting with a double grave accent, `__FILE__/__LINE__` and `undefineall`.
- `ifdef` selects code; keep `ifndef SYNTHESIS` regions free of functional behaviour.
- Every header needs an include guard; include declarations, not modules.
- The `timescale` directive sets unit/precision; modules without one inherit from whatever was compiled earlier - use it in every file or use `timeunit/timeprecision`.
- `default_nettype none` turns implicit-net typos into errors - restore `wire` at the end of the file.
- `celldefine` marks library cells; `line` and `resetall` exist for code generators and hygiene.

8.9 Exercises

- 1 Write a macro `CLAMP(x, lo, hi)` that is safe against precedence problems. Show a call for which it still misbehaves because an argument has a side effect.
- 2 A macro `DBL` takes one argument `x` and its text is `x+x`. Predict the value of the expression `3*DBL(2)` (`DBL` invoked with its grave accent) and verify it with Icarus.
- 3 Two files are compiled in the order a.v, b.v. a.v starts with `timescale 1ns/1ps` and b.v has no timescale but contains `#5`. What does `#5` mean in b.v? What if the order is reversed? How do you make it order-independent?
- 4 Write a `DECLARE_REG(name, width, rst)` macro that uses token pasting to declare `name_q` and `name_d` and a flop with asynchronous reset to `rst`. Use it twice in a module and simulate.
- 5 Explain why wrapping a flop's power-up value `initial q = 0;` in an `ifndef SYNTHESIS` region is dangerous, and what the correct design fix is.

- 6 Your SoC compiles 300 third-party files. After adding a new IP, a previously clean block reports "implicit net" errors. What directive is the most likely culprit and how do you confirm it?

9. Gate-Level Modelling: Primitives, UDPs, Delays, Specify Blocks and Timing Checks

Nobody designs a modern SoC by instantiating AND gates. Yet the constructs in this chapter are executed billions of times a day: every **standard-cell simulation library** is written with them, every **synthesized netlist** is a gate-level Verilog file that instantiates those cells, and every **gate-level simulation (GLS)** with **SDF back-annotation** relies on specify blocks and timing checks. A DV or DFT engineer who debugs GLS, a physical-design engineer who reads a netlist, and a library engineer who writes cell models all need to read this part of the language fluently. It is also a favourite source of interview questions (inertial delay, UDP tables, \$setuphold, notifiers).

```
RTL --synthesis--> gate-level netlist (.v) -- instantiates --> std-cell library (.v)
    |                                                         | primitives, UDPs,
    | place & route, STA                                     | specify blocks,
    v                                                         | timing checks
    SDF file (delays + limits) ---$sdf_annotate---+
    v
    gate-level simulation with real timing
```

Where gate-level constructs live in the ASIC flow.

9.1 Built-in gate and switch primitives

Verilog has 26 built-in **primitives**. They are instantiated like modules but with three differences: the instance name is optional, the **output terminal(s) come first**, and ports can only be connected by position. Inputs may be driven by expressions. Every primitive terminal is scalar (an array of instances can be used for vectors).

Group	Primitives	Terminal order	Notes
N-input gates	<code>and nand or nor xor xnor</code>	(out, in1, in2, ...)	Any number of inputs; z on an input is treated as x
N-output gates	<code>buf not</code>	(out1, ..., outN, in)	One input fans out to several outputs
Three-state	<code>bufif0 bufif1 notif0 notif1</code>	(out, in, ctrl)	Output z when not enabled; L/H values when ctrl is x
MOS switches	<code>nmos pmos rnmos rpmos</code>	(out, data, gate)	Unidirectional; r-versions reduce strength
CMOS switches	<code>cmos rcmos</code>	(out, data, ngate, pgate)	Transmission gate
Bidirectional	<code>tran rtran tranif0 tranif1 rtranif0 rtranif1</code>	(io1, io2 [, ctrl])	Pass values both ways; no delay on tran/rtran
Pull sources	<code>pullup pulldown</code>	(out)	Drive pull1/pull0 strength

```
`timescale 1ns/1ps
module mux2_gates (output y, input a, b, s);    // output first for primitives style
    wire sn, t0, t1;
    not #1      g0 (sn, s);
    and #(2, 1) g1 (t0, a, sn);                // rise 2, fall 1
    and #(2, 1) g2 (t1, b, s);
    or #1       g3 (y, t0, t1);
endmodule

module gp;
    reg a, b, s, en;
    wire y, bus;
    mux2_gates m (y, a, b, s);
    bufif1 #(1, 1, 3) d0 (bus, a, en);          // rise, fall, turn-off (to z)
```

```

bufif1          d1 (bus, b, ~en);          // second driver: enabled when en=0
initial begin
    $timeformat(-9, 0, "ns", 5);
    $monitor("%4t a=%b b=%b s=%b en=%b | y=%b bus=%b", $time, a, b, s, en, y, bus);
    a = 1; b = 0; s = 0; en = 1;
    #10 s = 1;
    #10 en = 0;
    #10 b = 1'bx;
    #10 $finish;
end
endmodule

```

gp.v - a gate-level multiplexer with rise/fall delays and a two-driver three-state bus.

```

0ns a=1 b=0 s=0 en=1 | y=x bus=x
1ns a=1 b=0 s=0 en=1 | y=x bus=1
4ns a=1 b=0 s=0 en=1 | y=1 bus=1
10ns a=1 b=0 s=1 en=1 | y=1 bus=1
13ns a=1 b=0 s=1 en=1 | y=0 bus=1
20ns a=1 b=0 s=1 en=0 | y=0 bus=x
23ns a=1 b=0 s=1 en=0 | y=0 bus=0
30ns a=1 b=x s=1 en=0 | y=0 bus=x
32ns a=1 b=x s=1 en=0 | y=x bus=x
gp.v:23: $finish called at 40000 (1ps)

```

Icarus Verilog 12.

Trace the delays: at 0 ns **sn** becomes 1 after 1 ns, **t0** rises 2 ns later and **y** 1 ns after that - so y is x until 4 ns. At 10 ns s rises: **sn** falls at 11, **t0** falls (fall delay 1) at 12, y falls at 13. At 20 ns the second driver switches on immediately (no delay) while the first needs its **turn-off delay** of 3 ns to release the bus: for 3 ns two strong drivers fight and the bus is x - the gate-level picture of **bus contention**, which real hardware turns into crowbar current. Finally, x on b propagates through **and** because s selects b.

Logic tables and switch-level modelling

```

module andx;
    reg a, b; wire y_and, y_or, y_xor;
    and (y_and, a, b); or (y_or, a, b); xor (y_xor, a, b);
    initial begin
        $display(" a b | and or xor");
        a = 0; b = 1'bx; #1 $display(" %b %b | %b %b %b", a, b, y_and, y_or, y_xor);
        a = 1; b = 1'bx; #1 $display(" %b %b | %b %b %b", a, b, y_and, y_or, y_xor);
        a = 1; b = 1'bz; #1 $display(" %b %b | %b %b %b", a, b, y_and, y_or, y_xor);
    end
endmodule

```

andx.v - controlling values beat x; z inputs act as x.

```

a b | and or xor
0 x | 0   x   x
1 x | x   1   x
1 z | x   1   x

```

Icarus Verilog 12.

Switch primitives model transistors and propagate **strength** (Chapter 3) as well as value. **nmos** conducts when its gate is 1, **pmos** when it is 0; a **supply** or **strong** input passes through as **strong**, while the resistive **r** versions reduce the strength by one or more levels (strong to pull, pull to weak, ...). This is how library developers model pass-gate logic, and how analog-ish structures such as pads with keepers or wired-OR open-drain buses are simulated.

```

module sw;
    reg in, ctl;
    wire n_out, p_out, c_out;
    supply1 vdd; supply0 gnd;

```

```

nmos (n_out, in, ctl);          // passes 'in' when ctl=1
pmos (p_out, in, ctl);          // passes 'in' when ctl=0
cmos (c_out, in, ctl, ~ctl);    // transmission gate: nmos gate ctl, pmos gate ~ctl
// CMOS inverter from two switches
wire inv_y;
pmos (inv_y, vdd, in);
nmos (inv_y, gnd, in);
// bidirectional pass switch with strength reduction on the resistive version
wire l, r; reg drv;
assign (strong0, strong1) l = drv;
rtran (l, r);                  // r sees drv with strength reduced (strong->pull)
initial begin
    in = 1; ctl = 1; drv = 1;
    #1 $display("ctl=1: nmos=%b pmos=%b cmos=%b inv(1)=%b r=%v",
               n_out, p_out, c_out, inv_y, r);
    ctl = 0; in = 0;
    #1 $display("ctl=0: nmos=%b pmos=%b cmos=%b inv(0)=%b l=%v",
               n_out, p_out, c_out, inv_y, l);
end
endmodule

```

sw.v - switch-level primitives; %v prints the strength.

```

ctl=1: nmos=1 pmos=z cmos=1 inv(1)=0 r=Pu1
ctl=0: nmos=z pmos=0 cmos=z inv(0)=1 l=St1

```

Icarus Verilog 12.

SWITCH-LEVEL IN PRACTICE

Synthesis tools ignore switch primitives, and Verilator does not model strengths. Switch-level code survives in standard-cell and I/O-pad simulation models, in custom-circuit (full-custom SRAM, register file) behavioural views, and in mixed-signal testbenches. For digital SoC work, being able to read it is enough.

9.2 User-defined primitives (UDPs)

A **UDP** describes a new primitive with a truth table. UDPs are evaluated very efficiently by simulators, which is why vendors model the internal state element of every flop and latch in a cell library as a UDP. The rules (1364-2005 clause 8, 1800-2017 clause 29):

- A UDP has exactly **one scalar output**, listed first, and one or more scalar inputs (at least 9 must be supported for sequential, 10 for combinational UDPs). Ports cannot be vectors or **inout**.
- Table values are 0, 1 and x only; a z on an input is treated as x. The output can never be z.
- **Combinational** UDP rows are **inputs : output**; . Any input combination that matches no row produces x.
- **Sequential** UDPs declare the output as **reg**, may give it an **initial** value, and have rows **inputs : current_state : next_state**; . A next state of - means "no change".
- Sequential rows may be **level-sensitive** or **edge-sensitive**: an edge such as (**01**) or the shorthand **r** (rise), **f** (fall), **p** (potential positive edge: 01, 0x, x1), **n** (potential negative edge) or ***** (any change). At most one edge per row. Level-sensitive rows take precedence over edge rows when both match.
- **?** matches 0, 1 or x; **b** matches 0 or 1.

```

primitive udp_mux2 (y, s, a, b);    // combinational UDP: output first, scalar ports
output y; input s, a, b;
table
// s  a  b  :  y
  0  0  ?  :  0;
  0  1  ?  :  1;
  1  ?  0  :  0;
  1  ?  1  :  1;
  x  0  0  :  0;
  x  1  1  :  1;
// pessimism reduction: equal inputs -> known

```

```

    endtable                                // any other combination -> x
endprimitive

primitive udp_dff (q, clk, d, rst_n); // sequential, edge-sensitive UDP
    output q; reg q; input clk, d, rst_n;
    initial q = 1'b0;
    table
    // clk  d  rst_n : q(now) : q(next)
    ?    ?  0      : ?      : 0;    // async reset (level)
    (01) 0  1      : ?      : 0;    // rising edge captures d
    (01) 1  1      : ?      : 1;
    (0?) 1  1      : 1      : 1;    // 0->x with d==q: stays (pessimism reduction)
    (0?) 0  1      : 0      : 0;
    (?0) ?  1      : ?      : -;    // falling edge: no change
    ?    *  1      : ?      : -;    // d changes without clock edge: no change
    ?    ?  (?1)   : ?      : -;    // reset release: no change
    endtable
endprimitive

```

udp.v (part 1) - a combinational and a sequential UDP.

```

module tb_udp;
    reg s, a, b, clk, d, rst_n; wire y, q;
    udp_mux2 m (y, s, a, b);
    udp_dff f (q, clk, d, rst_n);
    initial begin
        s = 1'bx; a = 1; b = 1; #1 $display("mux s=x a=b=1 -> y=%b", y);
        b = 0; #1 $display("mux s=x a=1 b=0 -> y=%b", y);
        clk = 0; d = 1; rst_n = 1;
        #1 clk = 1; #1 $display("after posedge d=1: q=%b", q);
        d = 0; #1 $display("d=0, no edge: q=%b", q);
        clk = 0; #1 $display("negedge: q=%b", q);
        rst_n = 0; #1 $display("rst_n=0: q=%b", q);
    end
endmodule

```

udp.v (part 2) - testbench.

```

mux s=x a=b=1 -> y=1
mux s=x a=1 b=0 -> y=x
after posedge d=1: q=1
d=0, no edge: q=1
negedge: q=1
rst_n=0: q=0

```

Icarus Verilog 12.

The two **x** rows of the multiplexer are a deliberate **X-pessimism reduction**: real hardware outputs 1 when both data inputs are 1, whatever the select, and a good library model says so. The **(0?)** rows of the flop do the same for a clock going to x when D already equals Q. Library quality is judged partly by how carefully these rows are written, because every missing row becomes an x in GLS.

PITFALL: INCOMPLETE SEQUENTIAL TABLES

Any combination of input values and edges that matches no row sets the output to **x**. Forgetting the "data changes while the clock is stable: no change" row (**? * 1 : ? : -;**) makes the flop go x on every data toggle - a bug that only shows up in gate-level simulation. Cover every edge of every input, or use ***** and **?** rows for the no-change cases.

9.3 Delays on gates, nets and assignments

A delay can be attached to a gate instance, a continuous assignment or a net declaration with `#`. It may have one, two or three values (for three-state and MOS devices), each of which may be a **min:typ:max** triplet:

Delay spec	0 -> 1	1 -> 0	-> z	-> x
<code>#d</code>	d	d	d	d
<code> #(r, f)</code>	r	f	min(r, f)	min(r, f)
<code> #(r, f, t)</code>	r	f	t	min(r, f, t)

Which delay is used for which output transition (1364-2005 7.14). Transitions from x use the pessimistic (shortest) rule analogously.

```
`timescale 1ns/1ps
module mtm;
  reg a = 0; wire y; realtime t0;
  buf #(1:2:3, 4:5:6) g (y, a);          // (rise min:typ:max, fall min:typ:max)
  initial begin
    #10 a = 1; t0 = $realtime; @(y) $display("rise delay = %0.1f ns", $realtime - t0);
    #10 a = 0; t0 = $realtime; @(y) $display("fall delay = %0.1f ns", $realtime - t0);
  end
endmodule
```

mtm.v - one source, three corners. The simulator chooses the corner (Icarus -Tmin|-Ttyp|-Tmax; commercial tools +mindelays/+typdelays/+maxdelays).

```
-Tmin: rise delay = 1.0 ns fall delay = 4.0 ns
-Ttyp: rise delay = 2.0 ns fall delay = 5.0 ns
-Tmax: rise delay = 3.0 ns fall delay = 6.0 ns
```

for T in min typ max; do iverilog -g2012 -T\$T -o mtm mtm.v && echo "-T\$T:" \$(vvp mtm); done

A **net delay** (`wire #3 w;`) delays every change of the net's resolved value, in addition to the delays of its drivers - a crude model of wire delay. It is rarely written by hand; in GLS, interconnect delays come from SDF **INTERCONNECT** entries instead.

Inertial versus transport delay

Gate delays, continuous-assignment delays and net delays are **inertial**: when the input changes again before the scheduled output change has happened, the pending change is cancelled, so pulses shorter than the delay are **filtered** - like a real gate whose output cannot swing in less than its propagation time. **Transport** delay passes every pulse, however short, and is modelled with a non-blocking assignment with an intra-assignment delay. A delay placed **before** a blocking assignment behaves like neither: the process sleeps and misses input changes while it waits.

```

`timescale 1ns/1ns
module dly;
  reg a = 0;
  wire      w_inertial;
  reg       r_transport, r_blocking;
  wire      g_out;
  assign #5 w_inertial = a;           // continuous assign: inertial
  buf      #5 g (g_out, a);          // gate: inertial
  always @(a) r_transport <= #5 a;    // NBA intra-delay: transport
  always @(a) #5 r_blocking = a;     // delay BEFORE sampling: misses events
  initial begin
    $monitor("%3t a=%b inertial=%b gate=%b transport=%b blocking=%b",
             $time, a, w_inertial, g_out, r_transport, r_blocking);
    #10 a = 1; #3 a = 0;              // 3 ns pulse (< 5 ns delay)
    #10 a = 1; #7 a = 0;              // 7 ns pulse (> 5 ns delay)
    #20 $finish;
  end
endmodule

```

dly.v - the four delay behaviours side by side.

```

0 a=0 inertial=x gate=x transport=x blocking=x
5 a=0 inertial=0 gate=0 transport=x blocking=x
10 a=1 inertial=0 gate=0 transport=x blocking=x
13 a=0 inertial=0 gate=0 transport=x blocking=x
15 a=0 inertial=0 gate=0 transport=1 blocking=0
18 a=0 inertial=0 gate=0 transport=0 blocking=0
23 a=1 inertial=0 gate=0 transport=0 blocking=0
28 a=1 inertial=1 gate=1 transport=1 blocking=1
30 a=0 inertial=1 gate=1 transport=1 blocking=1
35 a=0 inertial=0 gate=0 transport=0 blocking=0
dly.v:16: $finish called at 50 (1ns)

```

Icarus Verilog 12.

- The 3 ns pulse at 10-13 ns disappears on the inertial outputs and appears, shifted by 5 ns, on the transport output (15-18 ns).
- The blocking version woke at 10, slept 5 ns and sampled `a` at 15 - by then `a` was 0 again, so the pulse is lost and the 13 ns change was never seen by the waiting process.
- The 7 ns pulse passes through all four models.
- `transport` and `blocking` stay x until 15/28 ns: `reg a = 0` initialises `a` at time 0 **without an event** that the `always @(a)` processes can see (a time-0 race, Chapter 5).

PULSE CONTROL IN SPECIFY PATHS

Module path delays (next section) are inertial too, but with finer control: a pulse shorter than the **reject limit** is filtered, a pulse between the reject and **error limit** produces an x, and longer pulses pass. By default both limits equal the path delay; they can be set per path with `PATHPULSE$` specparams, globally with options such as `+pulse_r/percent` and `+pulse_e/percent`, or from SDF (`PATHPULSE`). `pulstyle_onevent` / `pulstyle_ondetect` and `showcancelled` (1364-2001) control when that x appears.

9.4 Specify blocks and module path delays

Distributed delays on the gates inside a cell are hard to characterise. Library vendors instead give each cell **module path delays** - pin-to-pin delays from an input port to an output port - in a `specify ... endspecify` block, and keep the functional model zero-delay. The characterised numbers come from the Liberty (.lib) file and, after layout, from SDF.

Construct	Syntax	Meaning
Parallel connection	<code>(A ==> Y) = d;</code>	Bit i of A to bit i of Y; source and destination must have the same width
Full connection	<code>(A, B ==> Y) = d;</code>	Every bit of every source to every bit of every destination
Edge-sensitive path	<code>(posedge CK ==> (Q += D)) = d;</code>	Path from a clock edge; <code>+/:-</code> give the data polarity (non-inverting / inverting) for the tool
State-dependent path	<code>if (S) (A ==> Y) = d1; and ifnone (A ==> Y) = d2;</code>	Different delays depending on the state of other inputs (e.g. XOR, mux select)
Polarity	<code>(A ==> Y), (A ==> Y)</code>	Declares a (non-)inverting path
Delay values	1, 2, 3, 6 or 12 values	rise/fall/z; 6 values for 01,10,0z,z1,1z,z0; 12 add the x transitions
specparam	<code>specparam tpd = 0.12;</code>	Constants for the specify block (not overridable like parameters)

If a module has both distributed delays and a path delay for the same input-output pair, the simulator uses the larger of the path delay and the distributed delay along that path (1364-2005 14.4). Icarus applies specify-block delays only when invoked with `-gspecify`; without it, and in Verilator and synthesis, specify blocks are ignored.

9.5 Timing checks and notifiers

Timing checks are system tasks that may appear only inside a specify block. They monitor the ports of the cell and report a **violation** when a timing limit from the library is broken; they do not change any value by themselves. To make the violation affect simulation, each check can toggle a **notifier** - a 1-bit `reg` passed as the last argument - and the cell model uses the notifier to drive its state element to x. This is how a setup violation in GLS turns into an x that propagates through the design and makes the test fail.

Check	Arguments	Violation when
<code>\$setup</code>	<code>(data, ref_edge, limit[, notifier])</code>	data changes less than limit before the reference edge
<code>\$hold</code>	<code>(ref_edge, data, limit[, notifier])</code>	data changes less than limit after the reference edge (note the swapped argument order)
<code>\$setuphold</code>	<code>(ref, data, setup, hold[, notifier, ...])</code>	Either; allows negative setup or hold values with the optional <code>delayed_ref</code> / <code>delayed_data</code> arguments
<code>\$recovery</code>	<code>(ref, data, limit[, notifier])</code>	An asynchronous control (e.g. reset) is released too close before the clock edge (reset-release "setup")
<code>\$removal</code>	<code>(ref, data, limit[, notifier])</code>	Released too close after the clock edge (reset-release "hold")
<code>\$crem</code>	<code>(ref, data, rec, rem[, notifier, ...])</code>	Either, with negative-limit support
<code>\$width</code>	<code>(ref_edge, limit[, threshold, notifier])</code>	A pulse starting with <code>ref_edge</code> is narrower than limit (min pulse width)
<code>\$period</code>	<code>(ref_edge, limit[, notifier])</code>	Two consecutive reference edges are closer than limit
<code>\$skew</code> , <code>\$timeskew</code> , <code>\$fullskew</code>	<code>(ref, data, limit, ...)</code>	Skew between two signals exceeds the limit
<code>\$nochange</code>	<code>(ref, data, start, end[, notifier])</code>	data changes while <code>ref</code> is at a level (e.g. memory address stable while <code>WE</code> is high)

Timing check events may carry a condition with `&&&`, e.g. `$setup(D, posedge CK &&& SE_n, 0.05, ntfr);` checks D only when scan enable is inactive - library models use this to switch between functional and

scan-path checks.

How a standard-cell library looks in Verilog

```
`timescale 1ns/1ps
`celldefine
module NAND2X1 (output Y, input A, B);
    nand g (Y, A, B);                // zero-delay functional model
    specify
        specparam tpd_rise = 0.12, tpd_fall = 0.08;
        (A => Y) = (tpd_rise, tpd_fall);    // parallel path, rise/fall
        (B => Y) = (0.10, 0.07);
    endspecify
endmodule
`endcelldefine

`celldefine
module DFFX1 (output reg Q, input D, CK);
    reg notifier;
    always @(posedge CK) Q <= D;
    always @(notifier) Q <= 1'bx;        // violation -> corrupt state
    specify
        (posedge CK => (Q += D)) = (0.20, 0.18);    // edge-sensitive path
        $setup(D, posedge CK, 0.05, notifier);
        $hold (posedge CK, D, 0.03, notifier);
        $width(posedge CK, 0.25);
    endspecify
endmodule
`endcelldefine
```

spec.v (part 1) - two simplified library cells. Production libraries model the flop's state with a sequential UDP that takes the notifier as an extra input, and add scan, set/reset and power pins.

```
module tb_spec;
    reg a, b, d, ck; wire y, q;
    NAND2X1 u1 (.Y(y), .A(a), .B(b));
    DFFX1 u2 (.Q(q), .D(d), .CK(ck));
    initial begin
        $timeformat(-9, 3, "ns", 9);
        $monitor("%t a=%b b=%b y=%b | d=%b ck=%b q=%b", $realtime, a, b, y, d, ck, q);
        a = 1; b = 1; d = 0; ck = 0;
        #1 a = 0;                // y rises after 0.12
        #1 a = 1;                // y falls after 0.08
        #1 d = 1; #1 ck = 1;      // clean capture
        #1 ck = 0; #1 d = 0; #0.02 ck = 1;    // setup violation: D changed 0.02 before CK
        #1 $finish;
    end
endmodule
```

spec.v (part 2) - testbench with a deliberate setup violation.

```

0.000ns a=1 b=1 y=x | d=0 ck=0 q=x
0.070ns a=1 b=1 y=0 | d=0 ck=0 q=x
1.000ns a=0 b=1 y=0 | d=0 ck=0 q=x
1.120ns a=0 b=1 y=1 | d=0 ck=0 q=x
2.000ns a=1 b=1 y=1 | d=0 ck=0 q=x
2.080ns a=1 b=1 y=0 | d=0 ck=0 q=x
3.000ns a=1 b=1 y=0 | d=1 ck=0 q=x
4.000ns a=1 b=1 y=0 | d=1 ck=1 q=x
4.200ns a=1 b=1 y=0 | d=1 ck=1 q=1
5.000ns a=1 b=1 y=0 | d=1 ck=0 q=1
6.000ns a=1 b=1 y=0 | d=0 ck=0 q=1
6.020ns a=1 b=1 y=0 | d=0 ck=1 q=1
6.200ns a=1 b=1 y=0 | d=0 ck=1 q=0
spec.v:39: $finish called at 7020 (1ps)

```

iverilog -g2012 -gspecify: path delays are applied (0.12 / 0.08 / 0.20 / 0.18 ns) but Icarus parses and ignores timing checks.

The path delays are exactly the specify values (at time 0 both inputs switch together and the fall at 0.070 ns uses the faster B arc). The setup violation at 6.020 ns is **not** reported, because Icarus does not execute timing checks, and the flop cleanly captures 0. **Expected behaviour in a commercial simulator** (VCS, Xcelium, Questa): a message naming `$setup`, the instance `tb_spec.u2`, the two event times and the 0.05 ns limit is printed at 6.020 ns, the notifier toggles, and q becomes x instead of 0 - and stays x until the next clean capture. Those simulators also offer switches to turn checks off globally (`+notimingcheck`), to keep the message but not the x (`+no_notifier`), or to disable them per instance.

9.6 SDF back-annotation

The **Standard Delay Format** (IEEE 1497) is the text file in which static timing analysis (PrimeTime, Tempus) or place-and-route writes the actual delays of every cell instance and wire in a netlist, for a specific corner. The simulator reads it with the system task `$sdf_annotate("file.sdf", scope [, config, log, mtm, scale, type])` - usually at time 0 in the testbench - and overwrites the specify-block values of the matching instances. `IOPATH` entries map to module paths, `INTERCONNECT` to port-to-port wire delays, and `TIMINGCHECK` entries (`SETUP`, `HOLD`, `SETUPHOLD`, `WIDTH`, `RECOVERY`, ...) to the limits of the corresponding checks.

```

(DELAYFILE
  (SDFVERSION "3.0")
  (DESIGN "tb_sdf")
  (TIMESCALE 1ns)
  (CELL (CELLTYPE "NAND2X1") (INSTANCE u1)
    (DELAY (ABSOLUTE
      (IOPATH A Y (0.30:0.35:0.40) (0.20:0.25:0.30))
      (IOPATH B Y (0.30:0.35:0.40) (0.20:0.25:0.30))))))
)

```

top.sdf - rise and fall triplets for both arcs of one NAND instance.

```

`timescale 1ns/1ps
module NAND2X1 (output Y, input A, B);
  nand g (Y, A, B);
  specify
    (A => Y) = (0.12, 0.08);
    (B => Y) = (0.10, 0.07);
  endspecify
endmodule
module tb_sdf;
  reg a, b; wire y;
  NAND2X1 u1 (.Y(y), .A(a), .B(b));
  initial begin
`ifdef SDF
    $sdf_annotate("top.sdf", tb_sdf);
`endif

```

```

$timeformat(-9, 3, "ns", 9);
a = 1; b = 1;
#1 a = 0; @(y) $display("%t y rose (A->Y)", $realtime);
#1 a = 1; @(y) $display("%t y fell (A->Y)", $realtime);
end
endmodule

```

sdf.v

```

== without SDF ==
1.120ns y rose (A->Y)
2.200ns y fell (A->Y)
== with -DSDF ==
1.350ns y rose (A->Y)
2.600ns y fell (A->Y)

```

iverilog -g2012 -gspecify [-DSDF] sdf.v; vvp (headers echoed by the run script). The typical values 0.35/0.25 replaced the library's 0.12/0.08.

PITFALL: SILENT SDF MISMATCHES

An SDF entry whose instance path or pin names do not match the netlist is skipped with a warning buried among thousands of lines, leaving that cell at its library default delay (often zero). Always check the annotation log/coverage report (tools print how many IOPATHs and timing checks were annotated), make sure the SDF TIMESCALE and the library's `timescale` directives agree, and annotate at the scope that matches the SDF's DESIGN hierarchy.

9.7 Gate-level simulation issues

GLS is slow and painful, so it is run on a small set of tests - reset and boot, clock and power-mode switching, DFT patterns, and anything timing-sensitive such as asynchronous interfaces. Its value lies exactly in the problems RTL simulation cannot see. The recurring issues are:

Issue	What happens	Typical fix
X-pessimism	A mux or AND-OR tree in the netlist outputs x where the RTL (and silicon) gives a known value; flops without reset start at x and never clear	Reset or initialise all control flops; review x sources; tool X-propagation options
X-optimism in RTL hidden	RTL <code>if (x)</code> took the else branch; the netlist propagates x and fails	A real bug - fix the RTL (Chapter 11)
Zero-delay races	Without SDF, clock-tree buffers make the clock arrive a delta later than data at some flops: data races through two stages	Unit-delay mode or SDF; never hand-insert #1 in RTL to "fix" it
Timing checks on synchronizers	The first flop of a CDC synchronizer legitimately violates setup; its notifier makes the whole design x	Disable checks on those instances (tool option or SDF/timing-check exclusion list)
Negative timing checks	Libraries use negative setup/hold; without delayed-signal support the checks are wrong	Enable negative timing check support in the simulator
Unannotated or mis-annotated SDF	Some cells at library defaults	Check annotation reports; same netlist version as the SDF
Memories and macros	SRAM/PLL/analog models have their own timing and x behaviour	Use the vendor's verification models and their documented options

WHAT GLS VERIFIES THAT RTL CANNOT

Synthesis correctness in the presence of sim/synth mismatches (Chapter 11), X-propagation from uninitialised state, reset sequencing, DFT/scan-chain and test-mode logic inserted after RTL, clock-gating and power-switch cells, and - with SDF - real timing on paths that STA constrains as exceptions. Equivalence checking (LEC) covers most of the logical risk far faster, which is why GLS runs are few but still mandatory before tape-out.

9.8 Summary

- 26 built-in primitives: gates, three-state buffers, MOS/CMOS and bidirectional switches, pullup/pulldown. Output first, positional connections, optional instance name.
- UDPs: one scalar output, table of 0/1/x; unmatched rows give x; sequential UDPs add current/next state and edge entries; libraries use them for flops and latches.
- Delays: 1/2/3 values, min:typ:max chosen by a tool option; gate, net and assign delays are inertial; NBA intra-assignment delay is transport.
- Specify blocks give pin-to-pin path delays (=> parallel, *> full, edge and state-dependent paths) and hold timing checks.
- Timing checks (\$setup, \$hold, \$setuphold, \$recovery, \$removal, \$width, \$period, ...) report violations and toggle a notifier that the model turns into x.
- SDF (IEEE 1497) back-annotates real delays and limits via \$sdf_annotate; check the annotation report.
- GLS catches x-propagation, reset, DFT and real-timing problems; its classic traps are x-pessimism, zero-delay races and synchronizer timing checks.

9.9 Exercises

- 1 Build a 2:1 multiplexer from `nand` gates only, with a delay of 1 ns per gate, and measure (in simulation) the glitch on the output when the select toggles with both data inputs at 1. Add the consensus term and show that the glitch disappears.
- 2 Write a sequential UDP for a positive-level latch with active-low asynchronous clear, including pessimism-reduction rows. Test the clear-while-enabled case.
- 3 What does `bufif1 #(2, 3, 5)` do when its control goes from 1 to 0? From 0 to x? Check the LRM table and confirm with Icarus.
- 4 A 4 ns pulse drives `assign #5 y = a;` and `always @(a) y2 <= #5 a;`. Draw both waveforms. Which one models a real buffer, and which one models a delay line?
- 5 Write the specify block for a 2-input XOR cell whose A-to-Y delay depends on B (state-dependent paths with `if` and `ifnone`), and a \$setuphold check with a negative hold limit for a flop.
- 6 Your GLS reset test goes all-x shortly after reset release, but only with SDF. List three plausible causes and how you would distinguish them.

10. System Tasks and Functions, File I/O, VCD and an Introduction to PLI/VPI

Identifiers that start with a dollar sign are **system tasks** (used as statements, like `$display`) and **system functions** (return a value, like `$clog2`). They are the testbench's interface to the outside world: printing, reading stimulus files, writing logs and waveforms, random numbers, command-line options and simulation control. The standard set is defined in 1364-2005 clause 17 and 1800-2017 clauses 20-21; simulators add their own (`$fsdbDumpvars` for Verdi waveforms, for example) and users add more through the **VPI**, the C interface at the end of this chapter. Except for the conversion and math functions, system tasks are ignored or rejected by synthesis.

10.1 Display tasks: \$display, \$write, \$strobe and \$monitor

Task	When it prints	Newline	Typical use
<code>\$display</code>	Immediately, when executed (Active region)	Yes	Messages, checks
<code>\$write</code>	Immediately	No	Building a line in pieces
<code>\$strobe</code>	At the end of the current time step (Postponed region), with the values after all updates of this step	Yes	Printing settled values of NBA-updated registers
<code>\$monitor</code>	At the end of every time step in which any of its arguments changed (except \$time)	Yes	Quick console traces; only one \$monitor is active at a time
<code>\$monitoron / \$monitoroff</code>	Enable/disable the active monitor; <code>on</code> prints immediately	-	Quiet phases
<code>\$displayb/h/o, \$writeb...</code>	As above with a different default radix for unformatted arguments		Rarely used

```

`timescale 1ns/1ns
module strb;
  reg [3:0] c = 0; reg clk = 0;
  always #5 clk = ~clk;
  always @(posedge clk) c <= c + 1;
  initial begin
    $monitor("%0t monitor : c=%0d", $time, c);      // end of time step, on change only
    @(posedge clk);
    $display("%0t display : c=%0d (before the NBA update)", $time, c);
    $strobe ("%0t strobe : c=%0d (Postponed region)", $time, c);
    @(posedge clk) $monitoroff;
    @(posedge clk) $display("%0t display : c=%0d (monitor off)", $time, c);
    $monitoron;                                   // prints current values immediately
    #1 $finish;
  end
endmodule

```

strb.v - the same variable seen by three display tasks.

```

0 monitor : c=0
5 display : c=0 (before the NBA update)
5 strobe : c=1 (Postponed region)
5 monitor : c=1
25 display : c=2 (monitor off)
25 monitor : c=3
strb.v:14: $finish called at 26 (1ns)

```

Icarus Verilog 12.

At 5 ns `$display` runs in the Active region, before the NBA `c <= c + 1` has been applied, and prints the old value 0; `$strobe` and `$monitor` print in the Postponed region and see 1 (Chapter 5). The same reasoning explains the classic interview question "why does my testbench print the value from the previous cycle?".

Format specifiers

Spec	Meaning	Spec	Meaning
<code>%b %o %h %d</code>	Binary, octal, hex, decimal	<code>%e %f %g</code>	Real: exponent, fixed, shorter of the two
<code>%0d, %0h</code>	Minimum width (no padding)	<code>%t</code>	Time, formatted by <code>\$timeformat</code>
<code>%5d, %-5d</code>	Field width; left-justify	<code>%s</code>	String (8 bits per character)
<code>%c</code>	ASCII character	<code>%m</code>	Hierarchical name of the scope (no argument)
<code>%v</code>	Net strength (St1, Pu0, HiZ ...)	<code>%l</code>	Library binding (config)
<code>%u, %z</code>	Unformatted 2-/4-state binary (for files)	<code>%p (SV)</code>	Assignment-pattern format of any aggregate: struct, array, queue, class handle
<code>%%</code>	A literal percent sign	<code>\n \t \\\ \"</code>	Newline, tab, backslash, quote

```

module fmt;
  reg [11:0] v = 12'hA5x;
  reg signed [7:0] s = -8'sd5;
  real r = 3.14159e3;
  wire (weak1, highz0) w = 1'b1;
  initial begin
    $display("%b=%b %h=%h %o=%o %d=%d", v, v, v, v);
    $display("width: [%d] [%0d] [%5d] [%-5d]|", 8'd7, 8'd7, 8'd7, 8'd7);
    $display("signed: %d %0d %h", s, s, s);
    $display("real: %f %0.2f %e %g %10.3f|", r, r, r, r, r);
    $display("char/string: %c %s [%0s]", 8'h41, "hello", "x");
    $display("scope: %m strength: %v", w);
    $write("no newline... ");
    $write("then newline\n");
  end
endmodule

```

fmt.sv

```

%b=10100101xxxx %h=a5x %o=51Xx %d=  X
width: [ 7] [7] [ 7] [7  ]|
signed: -5 -5 fb
real: 3141.590000 3141.59 3.141590e+03 3141.59 3141.590|
char/string: A hello [x]
scope: fmt strength: We1
no newline... then newline

```

Icarus Verilog 12.

- **Default width.** Without a width, `%d` pads to the number of characters needed for the largest value of the expression (3 for 8 bits: `[ 7]`), `%h` and `%b` print all digits. `%0d` removes the padding - use it in almost every message.
- **Unknown digits.** A lower-case `x` or `z` means all bits of that digit are unknown/high-impedance; an upper-case `X` or `Z` means only some are. The octal digit `X` above covers bits 01x. With `%d`, any x bit makes the whole number `X`.
- **Signedness.** `%d` of a signed variable prints the negative value; `%h` always shows the raw two's-complement bits.
- **Strings** are packed 8-bit characters; `%s` right-justifies in the full width of the argument, so a string held in a wide `reg` gets leading spaces - `%0s` removes them.

`%p` is SystemVerilog-only and not implemented in Icarus 12; Verilator handles it:

```

module pfmt;
    typedef struct { logic [3:0] op; logic [7:0] imm; } instr_t;    // unpacked struct
    typedef enum logic [1:0] {IDLE, BUSY, DONE} st_t;
    instr_t  ins = '{op: 4'h3, imm: 8'h2A};
    int      q[$] = '{1, 2, 3};
    st_t     st   = BUSY;
    initial begin
        $display("ins=%p q=%p", ins, q);
        $display("st=%s (%0d)  $sformatf -> \"%s\"", st.name(), st,
                $sformatf("op%0d_%h", ins.op, ins.imm));
        $finish;
    end
endmodule

```

pfmt.sv - %p, enum names and \$sformatf.

```

ins='{op:'h3, imm:'h2a} q='{ 'h1, 'h2, 'h3}
st=BUSY (1)  $sformatf -> "op3_2a"
- pfmt.sv:11: Verilog $finish

```

verilator --binary -Wall pfmt.sv && ./obj_dir/Vpfmt

10.2 Simulation time and control: \$time, \$realtime, \$timeformat, \$finish and \$stop

\$time returns the current time as a 64-bit integer **in the time unit of the module that calls it, rounded**; **\$stime** returns the same truncated to 32 bits; **\$realtime** returns a *real*, also in the caller's unit but without rounding. **\$timeformat(units, precision, suffix, min_width)** sets how **%t** prints: **units** is the exponent (-9 = ns, -12 = ps), **precision** the number of decimals. Before any **\$timeformat** call, **%t** prints in the global simulation precision with a width of 20.

```

`timescale 1ns/10ps
module tm;
    initial begin
        #12.345;
        $display("$time=%0d $stime=%0d $realtime=%0.3f", $time, $stime, $realtime);
        $display("%t default   : [%t]", $realtime);
        $timeformat(-6, 4, " us", 12);           // units=us, 4 decimals, suffix, min width
        $display("%t after fmt : [%t]", $realtime);
        $timeformat(-12, 0, " ps", 0);
        $display("%t in ps    : [%t]", $time);
    end
endmodule

```

tm.v

```

$time=12 $stime=12 $realtime=12.350
%t default   : [                  1235]
%t after fmt : [    0.0123 us]
%t in ps     : [12000 ps]

```

Icarus Verilog 12.

The delay **#12.345** was rounded to the 10 ps precision (12.35 ns). The default **%t** shows 1235 **units of the global precision** (10 ps). The last line is the trap: **\$time** had already rounded 12.35 ns to 12 ns, so printing it in picoseconds shows 12000 ps. Pass **\$realtime** to **%t** in **any** design with sub-unit delays.

Task	Effect
\$finish(n)	Ends the simulation. n = 0 prints nothing, 1 (default) prints time and location, 2 adds memory/CPU statistics. Final blocks (SV) run.
\$stop(n)	Suspends the simulation and enters the interactive prompt (in batch mode many tools just stop)
\$exit (SV)	Ends a program block (Chapter 21)

Task	Effect
<code>\$fatal(n, msg)</code> (SV)	Prints an error and calls <code>\$finish(n)</code> ; with <code>\$error</code> , <code>\$warning</code> , <code>\$info</code> the severity tasks of Chapter 22

EXIT STATUS IN REGRESSIONS

Continuous-integration scripts judge a test by the simulator's exit code and by grepping for a PASS/FAIL line. `$finish` normally exits with 0 even if the test failed; `$fatal` makes most simulators exit non-zero. End every test with an explicit summary line and use `$fatal` for failures, or have the run script parse the log.

10.3 File I/O

Verilog-2001 added C-like file I/O. `$fopen(name, mode)` returns a 32-bit **file descriptor** (0 on failure) with modes "r", "w", "a" and their "+" and "b" variants. The older form without a mode returns a **multichannel descriptor** (MCD): one bit per file, bit 0 = standard output, so several files can be written with one call by OR-ing descriptors. The output tasks `$fdisplay`, `$fwrite`, `$fstrobe` and `$fmonitor` take the descriptor as their first argument; the input functions mirror C: `$fgetc`, `$ungetc`, `$fgets`, `$fscanf`, `$fread`, plus `$feof`, `$ferror`, `$fseek`, `$ftell`, `$rewind`, `$fflush` and `$fclose`. `$sscanf` parses a string, `$sformat/$swrite` (and SV's `$sformatf` function) format into one.

```
# a    b    expected_sum
0003 0004 0007
00ff 0001 0100
ffff 0001 0000
```

vectors.txt - a stimulus file with a comment line.

```
module fio;
  integer fin, fout, n, errs, line_no;
  reg [15:0] a, b, exp_sum, sum;
  reg [8*80-1:0] line; // 80-character line buffer
  initial begin
    errs = 0; line_no = 0;
    fin = $fopen("vectors.txt", "r");
    fout = $fopen("results.log", "w");
    if (fin == 0) begin $display("cannot open vectors.txt"); $finish; end
    while (!$feof(fin)) begin
      n = $fgets(line, fin); // read one whole line (incl. newline)
      line_no = line_no + 1;
      if (n != 0 && line[8*n-1 -: 8] != "#") begin // first character of the line
        n = $sscanf(line, "%h %h %h", a, b, exp_sum);
        if (n == 3) begin
          sum = a + b;
          $fdisplay(fout, "line %0d: %h + %h = %h %0s", line_no, a, b, sum,
                    (sum == exp_sum) ? "OK" : "MISMATCH");
          if (sum != exp_sum) errs = errs + 1;
        end
      end
    end
    $fclose(fin);
    $fwrite(fout, "errors=%0d\n", errs);
    $fclose(fout);
    fin = $fopen("results.log", "r"); // read the log back and echo it
    while ($fgets(line, fin)) $write("%0s", line);
    $fclose(fin);
  end
endmodule
```

fio.v - a file-driven self-checking test.


```

line 2: 0003 + 0004 = 0007 OK
line 3: 00ff + 0001 = 0100 OK
line 4: ffff + 0001 = 0000 OK
errors=0

```

Icarus Verilog 12.

- `$fgets` fills the variable **right-justified**: the first character of an n-character line sits in bits `[8*n-1 -: 8]`. It returns the number of characters read (0 at end of file).
- `$fscanf/$sscanf` return the number of items matched; always check it - a malformed line otherwise leaves the old values in place and the test silently re-checks the previous vector.
- `$feof` becomes true only after a read has hit the end, so the loop above also processes one empty read; the `n != 0` check handles it.
- Multichannel descriptors are limited to 31 files and ordinary descriptors by the tool and the OS: close files, especially in tests that open one per transaction.

Memory initialisation: \$readmemh, \$readmemb and \$writememh

`$readmemh(file, mem [, start [, end]])` loads a memory array from a text file of hex numbers (`$readmemb` for binary) separated by white space, with `//` and `/* */` comments and `@addr` directives that jump to a (hex) address. Unmentioned words keep their value. It is the standard way to load boot ROM images, coefficient tables and test programs into processor memories, and - uniquely among system tasks - **synthesis tools honour it** in an initial block to set ROM/RAM contents on FPGAs. SystemVerilog adds `$writememh` and `$writememb` to dump a memory.

```

// boot ROM image: one hex word per line, // comments allowed
DEAD BEEF
@4          // jump to address 4
1234
5678
@7 FFFF

```

rom.hex

```

module rm;
  reg [15:0] rom [0:7];
  integer i;
  initial begin
    for (i = 0; i < 8; i = i + 1) rom[i] = 16'h0;
    $readmemh("rom.hex", rom);
    for (i = 0; i < 8; i = i + 1) $write("%h ", rom[i]);
    $write("\n");
    $writememh("dump.hex", rom, 4, 7);          // write addresses 4..7
    $readmemb("bits.bin", rom, 0, 1);          // file missing -> warning, rom unchanged
  end
endmodule

```

rm.v

```

dead beef 0000 0000 1234 5678 0000 ffff
ERROR: rm.v:10: $readmemb: Unable to open bits.bin for reading.
// 0x00000000
1234
5678
0000
ffff

```

Icarus Verilog 12, followed by cat dump.hex (the first line of the dump is Icarus' own address comment).

PITFALL: A MISSING MEMORY FILE IS NOT FATAL

Most simulators only warn when `$readmemh` cannot open its file or when the file has fewer words than the memory, and the simulation continues with x or zero contents - the processor then executes garbage and the failure is reported thousands of cycles later. Check the file in the testbench (`$fopen` it first and `$fatal` if that returns 0) and pass the path through a plusarg rather than relying on the current working directory.

10.4 Random numbers

Function	Returns	Notes
<code>\$random[(seed)]</code>	Signed 32-bit	1364 algorithm is defined in the LRM, so a given seed gives the same sequence on every compliant simulator; seed is an inout integer variable
<code>\$urandom[(seed)]</code> (SV)	Unsigned 32-bit	Per-thread RNG (random stability, Chapter 19); sequence is tool-specific
<code>\$urandom_range(max[, min])</code> (SV)	Unsigned in [min, max]	Arguments may be given in either order
<code>\$dist_uniform(seed, lo, hi)</code>	Integer	Also: <code>\$dist_normal(seed, mean, sd)</code> , <code>\$dist_exponential</code> , <code>\$dist_poisson</code> , <code>\$dist_chi_square</code> , <code>\$dist_t</code> , <code>\$dist_erlang</code>

```

module rnd;
  integer seed, i, r;
  integer hist [0:3];
  initial begin
    seed = 42;
    $write("$random(seed)      :");
    for (i = 0; i < 4; i = i + 1) $write(" %0d", $random(seed));
    $write("\n$random %% 10      :");
    for (i = 0; i < 4; i = i + 1) $write(" %0d", $random(seed) % 10); // can be negative!
    $write("\n{$random} %% 10      :");
    for (i = 0; i < 4; i = i + 1) $write(" %0d", {$random(seed)} % 10); // unsigned: 0..9
    $write("\n$urandom_range(3,1) :");
    for (i = 0; i < 6; i = i + 1) $write(" %0d", $urandom_range(3, 1));
    $write("\n");
    for (i = 0; i < 4; i = i + 1) hist[i] = 0;
    for (i = 0; i < 1000; i = i + 1) begin
      r = $dist_poisson(seed, 1); // mean 1
      if (r > 3) r = 3;
      hist[r] = hist[r] + 1;
    end
    $display("$dist_poisson(mean=1) histogram 0,1,2,>=3: %0d %0d %0d %0d",
      hist[0], hist[1], hist[2], hist[3]);
  end
endmodule

```

rnd.sv

```

$random(seed)      : -2144582656 646214477 38602500 -975846261
$random % 10       : 7 -9 4 0
{$random} % 10     : 5 2 8 9
$urandom_range(3,1) : 2 1 1 1 2 3
$dist_poisson(mean=1) histogram 0,1,2,>=3: 359 355 191 95

```

Icarus Verilog 12. The Poisson histogram is close to the theoretical 368/368/184/80 per 1000.

PITFALL: \$RANDOM % N CAN BE NEGATIVE

`$random` is signed, so `$random % 10` lies in -9..9. Wrap it in a concatenation (`{ $random } % 10`), which is always unsigned, or use `$urandom_range`. Also, `$random` without a seed argument uses one global seed for the whole simulation, so adding one call anywhere changes every random value after it; pass a per-component seed variable, or use SystemVerilog's per-thread RNG and constrained `randomize()` (Chapter 19).

10.5 Plusargs: configuring a test from the command line

Arguments that start with `+` on the simulator command line are **plusargs**, visible to the design. `$test$plusargs("NAME")` returns 1 if a plusarg beginning with NAME is present; `$value$plusargs("NAME=%d", var)` also converts the rest of the argument with a format (`%d %h %o %b %s %f %e %g`) into a variable and returns 1 on success. Every regression system uses them for the seed, the test name, verbosity, timeouts and feature switches - UVM's `+UVM_TESTNAME` and `+UVM_VERBOSITY` are plusargs.

```
module pa;
  integer seed, n_pkts; reg [8*32-1:0] test; real freq;
  initial begin
    if ($test$plusargs("VERBOSE")) $display("verbose mode on");
    if (!$value$plusargs("SEED=%d", seed)) seed = 1;           // default if absent
    if (!$value$plusargs("NPKT=%d", n_pkts)) n_pkts = 10;
    if (!$value$plusargs("TEST=%s", test)) test = "smoke";
    if (!$value$plusargs("FREQ=%f", freq)) freq = 100.0;
    $display("seed=%0d n_pkts=%0d test=%0s freq=%0.1f MHz", seed, n_pkts, test, freq);
  end
endmodule
```

pa.v

```
$ vvp pa
seed=1 n_pkts=10 test=smoke freq=100.0 MHz
$ vvp pa +VERBOSE +SEED=7 +TEST=dma_stress +FREQ=312.5
verbose mode on
seed=7 n_pkts=10 test=dma_stress freq=312.5 MHz
```

Icarus Verilog 12 (command lines shown with \$).

PITFALL: PREFIX MATCHING

Both functions match a **prefix**: `$test$plusargs("DEBUG")` is also true for `+DEBUG_AXI=0`. Choose names that are not prefixes of each other, or always use the `NAME=` form.

10.6 Conversion, math and bit-vector system functions

```
module cv;
  real r; reg [63:0] bits; integer i;
  reg [7:0] v;
  initial begin
    r = 2.7;
    $display("$rtoi(2.7)=%0d int'(2.7)=%0d $itor(5)/2=%0.1f",
             $rtoi(r), integer'(r), $itor(5) / 2);
    bits = $realtobits(1.0);
    $display("$realtobits(1.0)=%h back=%0.1f", bits, $bitstoreal(bits));
    $display("$clog2: 1->%0d 2->%0d 5->%0d 1024->%0d 1025->%0d",
             $clog2(1), $clog2(2), $clog2(5), $clog2(1024), $clog2(1025));
    v = 8'b0010_1100;
    $display("v=%b $bits=%0d $countones=%0d $onehot=%b $onehot0=%b",
             v, $bits(v), $countones(v), $onehot(v), $onehot0(v));
    v = 8'b0000_1000;
    $display("v=%b $onehot=%b $isunknown=%b", v, $onehot(v), $isunknown(v));
    v = 8'b0000_1z00;
```

```

    $display("v=%b $isunknown=%b (v == 8) is %b", v, $isunknown(v), v == 8);
    $display("$signed(4'b1111)=%0d $unsigned(-1) as 4 bits=%0d",
              $signed(4'b1111), 4'($unsigned(-1)));
end
endmodule

```

CV.SV

```

$rtoi(2.7)=2 int'(2.7)=3 $itor(5)/2=2.5
$realtobits(1.0)=3ff0000000000000 back=1.0
$clog2: 1->0 2->1 5->3 1024->10 1025->11
v=00101100 $bits=8 $countones=3 $onehot=0 $onehot0=0
v=00001000 $onehot=1 $isunknown=0
v=00001z00 $isunknown=1 (v == 8) is x
$signed(4'b1111)=-1 $unsigned(-1) as 4 bits=15

```

Icarus Verilog 12.

Function	Meaning	Synthesizable?
<code>\$rtoi(r)</code>	Real to integer by truncation	No (real)
<code>int'(r)</code> , assignment	Real to integer by rounding (away from zero at .5)	No (real)
<code>\$itor(i)</code>	Integer to real	No
<code>\$realtobits / \$bitstoreal</code>	Real <-> 64-bit IEEE-754 pattern (to pass reals through ports)	No
<code>\$shortrealtobits / \$bitstoshortreal (SV)</code>	32-bit float pattern	No
<code>\$signed(x), \$unsigned(x)</code>	Reinterpret signedness (no bit change)	Yes
<code>\$clog2(n)</code>	Ceiling log2 (0 for n = 0 or 1)	Yes, as a constant
<code>\$bits(x)</code>	Number of bits of an expression or type	Yes, constant
<code>\$countones, \$countbits(x, '1, ...)</code>	Population count (countbits: of given values, 1800-2012)	Yes (adder tree)
<code>\$onehot, \$onehot0</code>	Exactly one bit set / at most one	Yes
<code>\$isunknown(x)</code>	Any bit is x or z	Simulation only
<code>\$ln \$log10 \$exp \$sqrt \$pow \$floor \$ceil \$sin ...</code>	Real math (1364-2005)	Only in constant expressions

`$onehot`, `$onehot0`, `$isunknown` and `$countones` were introduced for assertions (Chapter 22) - `assert property (@(posedge clk) $onehot0(grant));` is the canonical arbiter check - but they are equally useful in immediate checks and, apart from `$isunknown`, in RTL.

10.7 Value change dump (VCD)

VCD is the waveform format defined by the Verilog standard (1364 clause 18) and read by every waveform viewer (GTKWave, Surfer, and the commercial debuggers). It is a text file: a header that declares scopes and variables with short identifier codes, then a list of time stamps (**#t**) each followed by the value changes at that time. The tasks that control it:

Task	Effect
<code>\$dumpfile("f.vcd")</code>	Name the file (default dump.vcd); call before \$dumpvars
<code>\$dumpvars(depth, scope, ...)</code>	Select variables: depth 0 = all levels below the scope, 1 = only that scope, n = n levels. No arguments = entire design
<code>\$dumpoff / \$dumpon</code>	Pause and resume; at a pause all variables are dumped as x, at resume their current values
<code>\$dumpall</code>	Checkpoint all current values

Task	Effect
<code>\$dumplimit(bytes)</code>	Stop dumping when the file reaches a size
<code>\$dumpflush</code>	Flush the OS buffer (useful before a crash or while viewing a running simulation)
<code>\$dumpports(...)</code> (extended VCD)	Port values with strength and direction, used for test-pattern exchange

```

`timescale 1ns/1ns
module cnt (input clk, output reg [1:0] q = 0);
    always @(posedge clk) q <= q + 1;
endmodule
module vcd;
    reg clk = 0;
    wire [1:0] q;
    cnt u (.clk(clk), .q(q));
    always #5 clk = ~clk;
    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0, vcd);           // depth 0 = this scope and everything below it
        #20 $dumpoff;               // stop recording (signals dumped as x)
        #20 $dumpon;                // resume: current values are dumped again
        #10 $finish;
    end
endmodule

```

vcd.v

```

$timescale
    1ns
$end
$scope module vcd $end
$var wire 2 ! q [1:0] $end
$var reg 1 " clk $end
$scope module u $end
$var wire 1 " clk $end
$var reg 2 # q [1:0] $end
$upscope $end
$upscope $end
$enddefinitions $end
...
#15
b10 !
b10 #
1"
#20
$dumppoff
bx #
x"
bx !
$end
#40
$dumpon
b0 #
1"
b0 !
$end

```

Excerpts of wave.vcd written by Icarus Verilog 12 (tab shown as spaces; "..." marks omitted lines).

Note that `vcd.clk` and `vcd.u.clk` share the identifier code " - VCD stores a net once however many scopes it appears in. VCD is simple and universal but large and slow for SoC-size designs; production flows use compressed proprietary formats (FSDB for Verdi, SHM for Xcelium, WLF for Questa, VPD for VCS) or the open FST format that Icarus (`-fst`) and Verilator (`--trace-fst`) can write. Their dump tasks follow the same

pattern: select a scope and depth, then window the dump in time to keep files manageable.

10.8 PLI and VPI: extending the simulator in C

The **Programming Language Interface** lets C code run inside the simulator, read and write signals, traverse the design hierarchy, and register new system tasks. It evolved in three generations, all still found in the field:

Generation	Routines	Status
PLI 1.0 TF routines	<code>tf_getp</code> , <code>tf_putp</code> , <code>tf_nump</code> ...: access to the arguments of a user system task	Deprecated, removed from 1364-2005
PLI 1.0 ACC routines	<code>acc_handle_object</code> , <code>acc_fetch_value</code> ...: access to design objects	Deprecated, removed from 1364-2005
PLI 2.0 = VPI	<code>vpi_handle</code> , <code>vpi_iterate</code> , <code>vpi_scan</code> , <code>vpi_get_value</code> , <code>vpi_put_value</code> , <code>vpi_register_cb</code> ...	The current standard (1364 clauses 26-27, 1800 clauses 36-38)
DPI-C (SV)	Direct import/export of C functions	The preferred way to call C models from SystemVerilog (Chapter 24)

VPI is **object-oriented over handles**: every design object (module, net, reg, port, the call of a system task, even a callback) is reached through an opaque `vpiHandle`; relationships are navigated with `vpi_handle(relation, obj)` (one-to-one) and `vpi_iterate` + `vpi_scan` (one-to-many); properties are read with `vpi_get` (integers) and `vpi_get_str` (strings); values with `vpi_get_value` / `vpi_put_value` in a chosen format. New system tasks are registered by routines listed in the `vlog_startup_routines` array, which the simulator calls when it loads the shared library. Here is a complete, working example:

```
#include <vpi_user.h>

/* $show_value(sig): print the full hierarchical name and value of a signal */
static PLI_INT32 show_value_calltf(PLI_BYTE8 *user_data) {
    (void)user_data;
    vpiHandle call = vpi_handle(vpiSysTfCall, NULL); /* this $show_value call */
    vpiHandle args = vpi_iterate(vpiArgument, call); /* its argument list */
    vpiHandle sig = vpi_scan(args); /* first argument */
    s_vpi_value v; v.format = vpiBinStrVal;
    vpi_get_value(sig, &v);
    s_vpi_time t; t.type = vpiSimTime;
    vpi_get_time(NULL, &t);
    vpi_printf("[VPI] t=%u %s (%d bits) = %s\n", t.low,
               vpi_get_str(vpiFullName, sig), vpi_get(vpiSize, sig), v.value.str);
    vpi_free_object(args);
    return 0;
}

static void register_tasks(void) {
    s_vpi_systf_data tf = {0};
    tf.type = vpiSysTask;
    tf.tfname = "$show_value";
    tf.calltf = show_value_calltf;
    vpi_register_systf(&tf);
}

/* the simulator calls every routine in this null-terminated table at startup */
void (*vlog_startup_routines[])(void) = { register_tasks, 0 };
```

hello_vpi.c - a user-defined system task in 30 lines of C.

```

module vpi_tb;
  reg [7:0] data = 8'hA5;
  initial begin
    #3 $show_value(data);
    data = data + 1;
    #2 $show_value(data);
  end
endmodule

```

vpi_tb.v

```

$ iverilog-vpi hello_vpi.c
Compiling hello_vpi.c...
Making hello_vpi.vpi from hello_vpi.o...
$ iverilog -g2012 -o vpi_tb vpi_tb.v && vvp -M. -mhello_vpi vpi_tb
[VPI] t=3 vpi_tb.data (8 bits) = 10100101
[VPI] t=5 vpi_tb.data (8 bits) = 10100110

```

Built with iverilog-vpi and loaded into vvp with -m.

Beyond new system tasks, VPI is how a large part of the verification ecosystem works: **callbacks** (`vpi_register_cb` with `cbValueChange`, `cbReadWriteSynch`, `cbEndOfSimulation` ...) let C code react to signal changes and scheduling events; waveform dumpers such as the FSDB writer, code-coverage tools and debuggers are VPI applications; and the Python co-simulation framework **cocotb** drives Icarus, Verilator, VCS, Xcelium and Questa through VPI (or VHPI). Chapter 24 compares VPI with the simpler and faster DPI-C and shows cocotb-style co-simulation.

INTERVIEW INSIGHT: WHY DPI REPLACED MOST PLI CODE

VPI is powerful because it can see everything, and slow for the same reason: every value access goes through handle lookups and format conversions, and it forces the simulator to keep objects visible (debug access, which disables optimisations). DPI-C passes arguments directly as C types, with no handles, and is the right choice for calling reference models. Use VPI when you need introspection: walking the hierarchy, callbacks on arbitrary signals, or tools that must work without modifying the HDL.

10.9 Summary

- `$display` prints immediately; `$strobe` and `$monitor` print settled values in the Postponed region; only one `$monitor` is active.
- Use `%0d/%0h/%0s` to avoid padding; upper-case `X/Z` in a digit means partly unknown; `%m` prints the scope, `%p` (SV) any aggregate.
- `$time` rounds to the caller's unit; pass `$realtime` to `%t` and set `$timeformat` once in the testbench.
- File I/O: `$fopen` returns 0 on failure; check `$fscanf/$sscanf` return counts; `$fgets` right-justifies; close what you open.
- `$readmemh/$readmemb` load memories (and FPGA ROM contents); a missing file only warns.
- `$random` is signed and LRM-defined; `$urandom/$urandom_range` are the SV per-thread RNG; `$dist_*` give distributions.
- `$test$plusargs / $value$plusargs` configure tests (prefix match!).
- `$clog2`, `$bits`, `$signed`, `$countones`, `$onehot` are synthesizable; `$rtoi` truncates while a cast rounds.
- VCD tasks select scope/depth and window time; production uses FSDB/FST. VPI is the C API behind custom tasks, dumpers and cocotb.

10.10 Exercises

- 1 Write a testbench that reads a CSV file of `addr, data` pairs, writes them into a RAM model, and dumps the RAM with `$writememh`. Handle a missing file with `$fatal` and a malformed line with an error message that includes the line number.
- 2 Predict, then check, what `$display("%d|%0d|%h|%s", 8'sh80, 8'sh80, 8'sh80, 16'h4142)` prints.

- 3 A testbench prints `$time` with `%t` after `$timeformat(-12, 0, " ps", 10)` in a module with `timescale 1ns/1ps` and shows 100000 ps where the waveform shows 100.4 ns. Explain and fix.
- 4 Use `$value$plusargs` to implement `+TIMEOUT_NS=<n>` with a default of 1 ms, and a watchdog that ends the test with `$fatal`.
- 5 Generate 10,000 values with `$dist_normal(seed, 100, 15)` and print a 10-bin histogram. Compare the mean and standard deviation with the requested values.
- 6 Extend `hello_vpi.c` with a `$count_changes(sig)` task that registers a `cbValueChange` callback and prints the number of changes at the end of simulation (`cbEndOfSimulation`).

11. Synthesizable Verilog: Coding Patterns and Simulation/Synthesis Mismatches

Verilog was designed as a simulation language; synthesis came later and adopted a **subset** of it, interpreted according to templates. The simulator executes your code literally, event by event. The synthesis tool instead **recognises patterns** - "an always block triggered by a clock edge whose outputs are assigned with `<=` is a bank of flip-flops" - and builds the hardware those patterns stand for. When the code matches the templates, simulation and hardware agree. When it does not, the tool may still produce a netlist, sometimes with only a warning, and the chip no longer behaves like the RTL you verified. This chapter gives the templates and then demonstrates, with real synthesis and real re-simulation of the netlist, each classic way the two can disagree.

HOW THE DEMOS IN THIS CHAPTER WERE PRODUCED

Each mismatch example was simulated as RTL with Icarus Verilog, synthesized with Yosys 0.33 (`synth -top <module>; write_verilog -noattr net.v`), and the resulting gate-level netlist was simulated with the same testbench against Yosys' cell library (`/usr/share/yosys/simcells.v`). Commercial synthesis tools (Design Compiler, Genus) behave the same way on these examples, usually with more warnings. The IEEE standard for the synthesizable subset is IEEE 1364.1-2002 (Verilog RTL synthesis); for SystemVerilog each tool documents its own subset (Chapter 17).

11.1 The synthesizable subset

Construct	Synthesis treatment
module, ports, parameters, localparam, generate, functions	Fully supported (functions become combinational logic, inlined)
<code>assign, always @*, always @(posedge clk ...)</code>	Supported by template: combinational logic, flops, latches
Tasks	Supported if they contain no timing controls; inlined
<code>for</code> loops with constant bounds	Unrolled; <code>while/repeat</code> only with constant iteration counts (tool-dependent)
Operators <code>+ - * & ^ << >> ?: == < ...</code>	Supported; <code>/</code> and <code>%</code> usually only by constants or powers of two; <code>**</code> only with constant operands or base 2
<code>case, casez, if</code>	Supported; <code>casex</code> supported but dangerous (see below)
Memories <code>reg [7:0] m [0:N-1]</code>	Inferred as flops or mapped to RAM macros / FPGA block RAM
<code>initial</code> blocks	Ignored for ASIC (except that FPGA tools use them as power-up values and \$readmem contents)
Delays <code>#n</code>	Ignored (with or without a warning)
<code>x</code> and <code>z</code> literals	<code>x</code> = don't care (optimiser picks); <code>z</code> = three-state driver
<code>wait, @</code> inside a block, <code>fork/join</code> , events	Not synthesizable
<code>force/release, deassign, \$display</code> , file I/O, <code>real</code>	Not synthesizable (system tasks ignored, <code>real</code> only in constant expressions)
<code>==, !=</code>	Treated as <code>==/!=</code> or rejected - <code>x</code> cannot be detected in hardware
UDPs, switch primitives, <code>specify</code> blocks	Gate primitives are mapped; UDPs/switches/specify are for simulation libraries
Hierarchical references, <code>defparam</code>	Usually rejected or limited

The Verilog synthesizable subset (based on IEEE 1364.1 and common tool practice).

11.2 Templates for combinational logic

Combinational logic is written with continuous assignments or with `always @*` (SystemVerilog: `always_comb`, Chapter 14). Inside an `always @*` block three rules guarantee a clean result: (1) use **blocking** = assignments; (2) **assign every output on every path** through the block - most easily with default assignments at the top; (3) never read a variable before it is assigned in the block unless you intend a latch.

```
module mux4 #(parameter W = 8) (input [1:0] sel, input [W-1:0] d0, d1, d2, d3,
                                output reg [W-1:0] y);

    always @* begin
        case (sel)
            2'd0: y = d0;
            2'd1: y = d1;
            2'd2: y = d2;
            default: y = d3;          // default covers 3 (and x/z in simulation)
        endcase
    end
endmodule

module dec #(parameter N = 3) (input [N-1:0] a, input en, output [(1<<N)-1:0] y);
    assign y = en ? ({((1<<N)-1){1'b0}}, 1'b1) << a : {(1<<N){1'b0}};
endmodule

module prienc (input [7:0] req, output reg [2:0] idx, output reg vld);
    integer i;
    always @* begin
        idx = 3'd0; vld = 1'b0;
        for (i = 0; i < 8; i = i + 1) // loop unrolls into a priority chain
            if (req[i] && !vld) begin idx = i[2:0]; vld = 1'b1; end
    end
endmodule
```

more.v (part 1) - multiplexer, decoder and priority encoder templates. All compile warning-free in Icarus and synthesize to pure logic in Yosys.

- **Multiplexers:** `case` with a `default`, or `?:` for two inputs. A `case` whose items are mutually exclusive synthesizes to a parallel mux; an `if/else if` chain to a priority mux - modern tools optimise both, but the priority form states intent.
- **Decoders:** a shifted one (`1 << a`) is the clearest; the replication above keeps the constant as wide as the output, so no bits are lost to sizing (Chapter 4).
- **Loops** are unrolled; the loop variable must have constant bounds. The `!vld` term makes the lowest index win; without it the last matching iteration - the highest index - would win.
- **Arithmetic:** `+`, `-`, `*` and comparisons map to library adders/multipliers (DesignWare in DC); size the result explicitly to keep the carry (`{c, s} = a + b;`).

11.3 Templates for sequential logic

```
module ff_async (input clk, rst_n, en, input [7:0] d, output reg [7:0] q);
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q <= 8'h00;          // async reset: in sensitivity list, tested first
        else if (en) q <= d;             // enable: no else -> hold (mux or ICG, not a latch)
endmodule

module ff_sync (input clk, rst, input [7:0] d, output reg [7:0] q);
    always @(posedge clk)
        if (rst) q <= 8'h00;             // sync reset: just another data input
        else q <= d;
endmodule
```

tmpl.v (part 1) - flip-flop templates.

The asynchronous-reset template is rigid, and the rules are LRM-independent conventions that every synthesis tool enforces: the sensitivity list contains **exactly** the clock edge and the reset edge(s); the first **if** tests the reset with the polarity matching the edge (**negedge rst_n -> if (!rst_n)**); the reset branch assigns constants only; and everything else goes in the final **else**. A register that is missing from the reset branch but assigned in the **else** branch gets its **reset used as an enable** (the tool adds logic to hold the value during reset) - a frequent lint warning.

```
module counter #(parameter W = 8) (input clk, rst_n, en, load, input [W-1:0] din,
                                   output reg [W-1:0] cnt, output tc);
    always @(posedge clk or negedge rst_n)
        if (!rst_n)    cnt <= {W{1'b0}};
        else if (load) cnt <= din;
        else if (en)    cnt <= cnt + 1'b1;
    assign tc = en & (&cnt);          // terminal count, for cascading
endmodule

module shreg #(parameter N = 4) (input clk, en, sin, output sout, output reg [N-1:0] q);
    always @(posedge clk)
        if (en) q <= {q[N-2:0], sin}; // one NBA per clock: no ordering issue
    assign sout = q[N-1];
endmodule

module lat (input g, input [7:0] d, output reg [7:0] q);
    always @*                // intentional latch: transparent while g=1
        if (g) q = d;        // (use only when the methodology allows latches)
endmodule
```

more.v (part 2) - counter, shift register and an intentional latch. Yosys maps them to 8 x \$_DFFE_PN0P_ (enable flop, async reset to 0), 4 x \$_DFFE_PP_ and 8 x \$_DLATCH_P_ respectively.

Memories: inferring RAM and ROM

```
module ram_sp #(parameter AW = 8, DW = 32) (
    input clk, we, input [AW-1:0] addr, input [DW-1:0] wdata, output reg [DW-1:0] rdata);
    reg [DW-1:0] mem [0:(1<<AW)-1];
    always @(posedge clk) begin
        if (we) mem[addr] <= wdata;
        rdata <= mem[addr];          // synchronous read -> maps to SRAM/BRAM
    end
endmodule

module rom_case (input [1:0] a, output reg [7:0] d);
    always @* case (a)
        2'd0: d = 8'h3C; 2'd1: d = 8'hA5; 2'd2: d = 8'h0F; default: d = 8'hFF;
    endcase
endmodule
```

tmpl.v (part 2) - a single-port RAM with synchronous read (read-first) and a case ROM.

```
=== ff_async ===
    $adffe                1
=== ff_sync ===
    $sdff                 1
=== ram_sp ===
    $mem_v2                1
    $mux                   3
=== rom_case ===
    $eq                    2
    $logic_not             1
    $pmux                  1
```

yosys -p "read_verilog tmpl.v; hierarchy -check; proc; opt; memory -nomap; opt; stat" | grep: the async-reset enable flop, the sync-reset flop, a recognised memory and a ROM as a parallel mux.

A memory is recognised when the array is accessed with the templates the target supports: synchronous write, and either synchronous read (registered address or data - maps to SRAM macros and FPGA block RAM) or asynchronous read (only small register files or LUT RAM). Read-during-write behaviour is part of the template: `rdata <= mem[addr]` in the same block as the write reads the **old** data (read-first); writing the read from a variable updated with blocking assignment gives write-first. In ASIC flows, large memories are not inferred at all: they are instantiated as compiler-generated SRAM macros through a wrapper, with the behavioural array used only in the simulation model. ROMs are written as a `case` statement or as an array initialised with `$readmemh` (FPGA) and are turned into logic.

Finite state machines

```
// "101" sequence detector, Moore, overlapping, two-process style
module det101 (input clk, rst_n, din, output reg hit);
    localparam [1:0] S0 = 2'd0, S1 = 2'd1, S10 = 2'd2, S101 = 2'd3;
    reg [1:0] state, nxt;
    always @(posedge clk or negedge rst_n) // state register
        if (!rst_n) state <= S0;
        else state <= nxt;
    always @* begin // next-state logic
        nxt = state; // default: stay
        case (state)
            S0: if (din) nxt = S1;
            S1: if (!din) nxt = S10;
            S10: nxt = din ? S101 : S0;
            S101: nxt = din ? S1 : S10;
            default: nxt = S0;
        endcase
    end
    always @* hit = (state == S101); // Moore output
endmodule

module tb_fsm;
    reg clk = 0, rst_n = 0, din = 0; wire hit;
    reg [11:0] pattern = 12'b1101_0110_1010; // shifted in MSB first
    integer i;
    det101 dut (.clk(clk), .rst_n(rst_n), .din(din), .hit(hit));
    always #5 clk = ~clk;
    initial begin
        #12 rst_n = 1;
        for (i = 11; i >= 0; i = i - 1) begin
            @(negedge clk) din = pattern[i];
            @(posedge clk) #1 $write("%b", hit);
        end
        $write(" <- hit after each bit of %b\n", pattern);
        $finish;
    end
endmodule
```

fsm.v - the canonical two-process FSM: a state register and a combinational next-state block with a default.

```
000101001010 <- hit after each bit of 110101101010
```

Icarus Verilog 12: hits after bits 4, 6, 9 and 11 (overlapping matches).

State encodings are `localparams` (SystemVerilog: an `enum`, Chapter 12). The `nxt = state;` default makes every path assign `nxt`, so no latch can appear, and the `default` item recovers from illegal states. Synthesis tools extract the FSM and may re-encode it (one-hot, Gray) unless told otherwise; the micro-architecture of FSMs, Mealy vs. Moore and output registering are covered in the companion RTL Design guide.

Three-state buffers and bidirectional pins

```

module gpio_pad (inout pad, input oe, input dout, output din);
    assign pad = oe ? dout : 1'bz;    // drive or release the pin
    assign din = pad;                 // always sample the pin
endmodule

module tb_tri;
    reg oe, dout, ext_en, ext_val;
    wire pad, din;
    gpio_pad u (.pad(pad), .oe(oe), .dout(dout), .din(din));
    assign pad = ext_en ? ext_val : 1'bz; // the other chip on the board
    pullup (pad);                        // board pull-up resistor
    initial begin
        oe = 1; dout = 0; ext_en = 0; ext_val = 0;
        #1 $display("we drive 0      : pad=%b (%v) din=%b", pad, pad, din);
        oe = 0;
        #1 $display("nobody drives   : pad=%b (%v) din=%b", pad, pad, din);
        ext_en = 1;
        #1 $display("other chip: 0    : pad=%b (%v) din=%b", pad, pad, din);
        oe = 1; dout = 1;
        #1 $display("both drive 1 vs 0: pad=%b (%v) din=%b  <- contention", pad, pad, din);
    end
endmodule

```

tri.v - a bidirectional pad model.

```

we drive 0      : pad=0 (St0) din=0
nobody drives   : pad=1 (Pu1) din=1
other chip: 0    : pad=0 (St0) din=0
both drive 1 vs 0: pad=x (StX) din=x  <- contention

```

Icarus Verilog 12.

THREE-STATES BELONG ONLY AT THE CHIP BOUNDARY

Inside an ASIC, internal three-state buses are forbidden by almost every methodology (they are hard to test, create contention and floating nodes, and complicate timing); use multiplexers. FPGAs have no internal three-states at all - tools convert them to muxes. The `inout + ? : 1'bz` pattern is used only in the pad ring, where it maps onto a bidirectional I/O cell (the I/O cell's OE, A and Y pins correspond to `oe`, `dout` and `din`).

11.4 Simulation/synthesis mismatches

A **mismatch** exists when the netlist, simulated with the same stimulus, behaves differently from the RTL. Formal **equivalence checking** (Conformal LEC, Formality) catches many of them - it compares the RTL as the synthesis tool interprets it with the netlist - but it cannot catch cases where the RTL simulation itself differs from that interpretation. Only good coding style, lint and gate-level simulation catch those. The following demos each show one mechanism.

1. Incomplete sensitivity list

```

module sens (input a, b, c, output reg y);
    always @(a or b)          // BUG: c is missing from the sensitivity list
        y = (a & b) | c;
endmodule

```

sens.v

```

module tb_sens;
  reg a, b, c; wire y;
  sens dut (.a(a), .b(b), .c(c), .y(y));
  initial begin
    a = 0; b = 0; c = 0;
    #1 c = 1; #1 $display("a=%b b=%b c=%b -> y=%b", a, b, c, y);
    #1 a = 1; #1 $display("a=%b b=%b c=%b -> y=%b", a, b, c, y);
  end
endmodule

```

tb_sens.v

```

== RTL ==
a=0 b=0 c=1 -> y=0
a=1 b=0 c=1 -> y=1
== gate netlist ==
a=0 b=0 c=1 -> y=1
a=1 b=0 c=1 -> y=1

```

Icarus on sens.v, then on the Yosys netlist (assign y = (b & a) | c). Yosys synthesized the intended logic without any warning.

The simulator only re-evaluates the block when a or b changes, so the RTL behaves like a latch on c; synthesis ignores the sensitivity list and builds the AND-OR. The RTL was wrong, the silicon right - but it was the RTL that was verified. **Fix:** `always @*` (or `always_comb`), which the LRM defines to include every variable read in the block. Verilator's lint flags this block (as a sequential process with blocking assignments); commercial lint reports "incomplete sensitivity list".

2. Incomplete assignment: the unintended latch

```

module latch_bug (input [1:0] sel, input a, b, output reg y);
  always @* begin
    case (sel)
      // sel == 2'b11 not covered, no default
      2'b00: y = a;
      2'b01: y = b;
      2'b10: y = a ^ b;
    endcase
    // -> y must hold its value -> LATCH
  end
endmodule

module latch_fix (input [1:0] sel, input a, b, output reg y);
  always @* begin
    y = 1'b0; // default assignment first: every path assigns y
    case (sel)
      2'b00: y = a;
      2'b01: y = b;
      2'b10: y = a ^ b;
    endcase
  end
endmodule

```

latch.v

```

No latch inferred for signal `latch_fix.y' from process
`latch_fix.$proc$latch.v:11$3'.
Latch inferred for signal `latch_bug.y' from process
`latch_bug.$proc$latch.v:2$1': $auto$proc_dlatch.cc:427:proc_dlatch$37

```

yosys -p "read_verilog latch.v; proc; stat" (lines wrapped).

This one is **not** a mismatch - the simulator also holds y when sel = 11 - but it is almost always a bug: a latch in a flop-based design is transparent while the enable is high, is hard to time and to test, and its enable is a glitch-prone combinational signal. Every synthesis log should be searched for "latch inferred". SystemVerilog's `always_comb` makes tools warn, and `always_latch` documents an intentional one.

3. full_case and parallel_case

// **synopsys full_case** (or the attribute (*** full_case ***)) tells synthesis that the listed case items are **all that can occur**, so unlisted values are don't-cares and no latch is needed. **parallel_case** tells it the items are mutually exclusive, so no priority logic is needed. Both are **invisible to the simulator**, which is exactly the problem:

```
module fc (input [1:0] sel, input a, b, output reg y);
  always @* begin
    (* full_case *)           // same meaning as the comment // synopsys full_case
    case (sel)
      2'b00: y = a;
      2'b01: y = b;
      2'b10: y = a ^ b;
    endcase
  end
endmodule
```

fc.v - the latch_bug case with a full_case directive.

```
module tb_fc;
  reg [1:0] sel; reg a, b; wire y;
  fc dut (.sel(sel), .a(a), .b(b), .y(y));
  initial begin
    a = 1; b = 0;
    sel = 2'b00; #1 $display("sel=%b a=%b b=%b -> y=%b", sel, a, b, y);
    sel = 2'b11; #1 $display("sel=%b a=%b b=%b -> y=%b", sel, a, b, y);
    a = 0;      #1 $display("sel=%b a=%b b=%b -> y=%b", sel, a, b, y);
  end
endmodule
```

tb_fc.v

```
No latch inferred for signal `fc.y' from process `fc.$proc$fc.v:2$1'.
== RTL ==
sel=00 a=1 b=0 -> y=1
sel=11 a=1 b=0 -> y=1
sel=11 a=0 b=0 -> y=1
== gates ==
sel=00 a=1 b=0 -> y=1
sel=11 a=1 b=0 -> y=1
sel=11 a=0 b=0 -> y=0
```

Yosys honours full_case: no latch. RTL still holds y for sel = 11; the netlist (y = sel==11 ? a : ...) follows a. Mismatch.

PITFALL: FULL_CASE AND PARALLEL_CASE ARE EVIL TWINS

Cliff Cummings' well-known paper of that title summarises the industry verdict: these directives change the hardware without changing the simulation, so any value outside the listed items (or any overlap between items) produces a netlist that nobody simulated. Instead: add a **default** (with a real value, or with 'x' if you deliberately want a don't-care, see below); write mutually exclusive items; and in SystemVerilog use **unique case** or **priority case** (Chapter 14), which give the same optimisation **and** make the simulator check the assumption at run time.

4. casex and casez hide unknowns

```
module cx (input [1:0] op, input [7:0] a, b, output reg [7:0] y);
  always @* begin
    casex (op)
      2'b00: y = a + b;
      2'b01: y = a - b;
      2'b1?: y = a & b;
      default: y = 8'hEE;
    endcase
  end
endmodule
```

```

    end
endmodule
module cz (input [1:0] op, input [7:0] a, b, output reg [7:0] y);
    always @* begin
        case (op)
            // plain case: x in op matches no item
            2'b00: y = a + b;
            2'b01: y = a - b;
            2'b10,
            2'b11: y = a & b;
            default: y = 8'hxx; // make unknowns visible in simulation
        endcase
    end
endmodule
module tb_cx;
    reg [1:0] op; reg [7:0] a = 8'd12, b = 8'd10; wire [7:0] y1, y2;
    cx u1 (op, a, b, y1);
    cz u2 (op, a, b, y2);
    initial begin
        op = 2'b01; #1 $display("op=%b casex:y=%h case:y=%h", op, y1, y2);
        op = 2'bx1; #1 $display("op=%b casex:y=%h case:y=%h <- uninitialised op", op, y1, y2);
        op = 2'bxx; #1 $display("op=%b casex:y=%h case:y=%h", op, y1, y2);
    end
endmodule

```

CX.V

```

op=01 casex:y=02 case:y=02
op=x1 casex:y=02 case:y=xx <- uninitialised op
op=xx casex:y=16 case:y=xx

```

Icarus Verilog 12.

casex treats x and z as wildcards **in the case expression too**, not only in the items. An uninitialised or corrupted **op = xx** matches the first item and the simulation happily computes $a + b = 0x16$, hiding the bug that silicon would expose. **casez** has the same problem for z (a floating input). Use plain **case**, or **casez** with ? only in the items (the accepted idiom for priority decoders), or SystemVerilog's **case inside**, which treats x/z in the expression as non-matching.

5. Blocking assignments in sequential logic

```

module pipe2 (input clk, input [3:0] d, output reg [3:0] q2);
    reg [3:0] q1;
    always @(posedge clk) q1 = d; // BUG: blocking assignment in clocked logic
    always @(posedge clk) q2 = q1; // reads q1 - old or new value? order-dependent
endmodule

```

race.v (race_swapped.v has the two always blocks in the opposite order).

```

module tb_race;
    reg clk = 0; reg [3:0] d = 0; wire [3:0] q2;
    pipe2 dut (.clk(clk), .d(d), .q2(q2));
    always #5 clk = ~clk;
    initial begin
        @(negedge clk) d = 4'd1;
        @(negedge clk) d = 4'd2;
        @(negedge clk) d = 4'd3;
        @(negedge clk) $display("after 3 edges: q2=%0d (a 2-stage pipe should show 2)", q2);
        $finish;
    end
endmodule

```

tb_race.v


```

== RTL (iverilog) ==
after 3 edges: q2=3 (a 2-stage pipe should show 2)
== gates ==
after 3 edges: q2=2 (a 2-stage pipe should show 2)
== RTL, blocks swapped ==
after 3 edges: q2=2 (a 2-stage pipe should show 2)

```

The same RTL gives different results depending on the order of two always blocks; the netlist always has two flops.

Both blocks wake on the same edge; the LRM leaves their order undefined (Chapter 5). If the first block runs first, q2 sees the **new** q1 and the pipeline collapses to one stage in simulation; synthesis always builds two flops. Another simulator, another compile option or an unrelated edit can flip the order. **Rule:** in clocked blocks use only non-blocking <=; in combinational blocks use only blocking =; never mix them for the same variable. (Blocking assignments inside a single clocked block, e.g. `q1 = d; q2 = q1;`, are not a race: both sim and synthesis collapse them to one flop - a different, but equally unintended, result.)

6. Initial values, delays and x-assignments

```

module cnt_noreset (input clk, output reg [3:0] cnt);
    initial cnt = 4'd0;           // works in RTL sim; an ASIC flop has no power-up value
    always @(posedge clk) cnt <= cnt + 4'd1;
endmodule

```

init.v - relying on an initial block instead of a reset.

```

== RTL ==
after 3 clocks: cnt=0011
== gates (ASIC: init values dropped) ==
after 3 clocks: cnt=xxxx

```

Yosys synth; setattr -unset init (as an ASIC flow does); re-simulated with Icarus. The x never clears: `x + 1 = x`.

```

module xa (input [1:0] sel, input a, b, output reg y, output d);
    assign #2 d = a;           // delay: ignored by synthesis
    always @* begin
        case (sel)
            2'b00: y = a;
            2'b01: y = b;
            default: y = 1'bx; // "don't care": synthesis may pick 0 or 1
        endcase
    end
endmodule

```

xa.v - a delay and an x-assignment.

```

assign _0_ = sel[0] & ~(sel[1]);
assign y = _0_ ? b : a;
assign d = a;

```

The body of the Yosys netlist: the #2 is gone, and the don't-care was resolved to `y = a` for `sel = 1x`.

```

== RTL ==
t=1 sel=10 a=1 b=0 -> y=x d=x
t=3 sel=10 a=1 b=0 -> y=x d=1
== netlist ==
t=1 sel=10 a=1 b=0 -> y=1 d=1
t=3 sel=10 a=1 b=0 -> y=1 d=1

```

Same testbench on RTL and netlist: RTL shows x (and a 2 ns delay on d); the gates show a definite value immediately.

Assigning x is legitimate - it gives the optimiser freedom and makes the simulation show x if the "impossible" case happens - provided that the case really is impossible and a checker watches for it. The opposite RTL hazard is **X-optimism**: `if (sel_x) ... else ...` takes the else branch when `sel_x` is x, so RTL produces a clean value that the hardware may not. Delays in RTL are ignored by synthesis and only serve to hide races; the one accepted use is a small #1 on non-blocking assignments in some legacy code bases to make waveforms readable, which modern teams avoid.

7. Functions with side effects and other traps

A function that writes a module-level variable (a counter of calls, a debug flag) is executed by the simulator **each time** its expression is evaluated - which depends on events - while synthesis inlines it once per call site and turns the side effect into ordinary logic (Yosys built a register for such a counter). Keep functions pure: outputs depend only on inputs, no global writes, no static locals relied upon between calls (declare them `automatic`). Other classic mismatch sources:

Mechanism	Simulation	Synthesis	Prevention
Missing sensitivity entry	Latch-like on the missing input	Combinational	<code>always @*</code> / <code>always_comb</code>
<code>full_case</code> / <code>parallel_case</code>	Holds value / priority	Don't care / parallel	<code>default; unique/priority case</code>
<code>casex</code> / <code>casez</code> with x/z input	Wildcard match	Real logic	<code>case, case inside</code> , x-checks
Blocking in clocked blocks	Order-dependent race	Flops	<code><=</code> in clocked logic
initial values	Known power-up	Ignored (ASIC)	Reset every control flop
Delays <code>#n</code>	Delayed	Ignored	No delays in RTL
x-assignment / <code>if (x)</code>	x or else-branch	0 or 1	Assertions, X-prop simulation
<code>===</code> / <code>!==</code> in RTL	Compares x/z exactly	Treated as <code>==</code>	Use only in testbenches
Function side effects	Per evaluation	Per call site	Pure, automatic functions
<code>ifdef SYNTHESIS</code> regions	Code present	Code absent	Only non-functional code inside
Out-of-range index	Read gives x, write ignored	Some value / aliasing	Size arrays to powers of two or check

THE MISMATCH SAFETY NET

1) **Lint** every file (Verilator `--lint-only -Wall`, Spyglass, Ascent) with rules for sensitivity lists, latches, blocking/non-blocking use, case completeness and x-assignments. 2) Read the **synthesis log** for latches, ignored delays and constant registers. 3) Run **equivalence checking** RTL vs. netlist on every netlist. 4) Run a small set of **gate-level simulations** (Chapter 9), which catch what the first three cannot - notably reset and x-propagation problems.

11.5 Coding for reuse

Code that is synthesizable is necessary but not sufficient: IP is reused across projects, technologies and teams for a decade. The habits that make that possible:

- **One module per file**, file named after the module; ANSI port lists with directions and widths; `default_nettype none` at the top and `wire` restored at the end (Chapter 8).
- **Parameterise** widths and depths with typed, named parameters; derive everything else with `localparam` and `$clog2`; check illegal parameter values at elaboration (Chapter 7).
- **Register outputs** of a block (or document which are combinational), so timing budgets at integration are predictable.
- **No technology cells in RTL**: clock gates, synchronizers, SRAMs and pads go behind thin wrapper modules with a behavioural model for simulation and a technology-specific implementation.
- **Explicit reset strategy**: reset control/state flops; leave pure datapath flops without reset if the methodology allows (saves area and routing), but never rely on initial values.
- **One clock per always block**, one edge; clock-domain crossings only through the approved synchronizer cells.
- **Naming conventions** that tools and people can parse: `_n` for active-low, `_q/_d` for flop outputs/inputs, `i/_o` or `_i/_o` for ports, `u` for instances, `g` for generate blocks.
- **Keep the synthesizable subset of the target tools**, including the weakest one in the flow (lint, formal, emulation, FPGA prototyping).

INTERVIEW INSIGHT: WHAT IS REALLY WRONG WITH THIS CODE?

Interviewers love to show a 10-line always block and ask what hardware it builds. Answer in the order the tool thinks: (1) is it clocked (edge in the sensitivity list) or combinational? (2) for combinational: is every output assigned on every path (else latch)? is the sensitivity list complete? (3) for clocked: are assignments non-blocking? is the reset template exact? (4) what do case statements do with uncovered values and x? (5) is any variable assigned in two always blocks (multiple drivers - an error in synthesis, a race in simulation)?

11.6 Summary

- Synthesis recognises templates; simulation executes literally. Code to the templates so that both mean the same hardware.
- Combinational: `always @*`, blocking `=`, defaults first, every output on every path, case with default.
- Sequential: edge-triggered block, non-blocking `<=`, exact async/sync reset template, enable as `else if (en)`.
- Memories follow the target's RAM templates; ASIC SRAMs are instantiated macros behind wrappers. Three-states only in the pad ring.
- Mismatch mechanisms demonstrated: incomplete sensitivity lists, `full_case`, `case` with x, blocking races, initial values, delays, x-assignment; plus side-effect functions, `===`, `ifdef` regions.
- The safety net is lint + synthesis-log review + equivalence checking + targeted gate-level simulation.

11.7 Exercises

- 1 Write an 8-bit up/down counter with synchronous load, asynchronous active-low reset and saturation at 0 and 255. Synthesize it with Yosys, simulate RTL and netlist with the same testbench and compare.
- 2 Take `latch_bug` from this chapter and fix it three different ways (default assignment, default item, SystemVerilog `always_comb` with `unique case`). Which fix still lets simulation detect `sel = 11`?
- 3 Modify `pipe2` so that it uses blocking assignments inside a single always block in the order `q2 = q1; q1 = d;`. Does it work? Is it a race? Why do coding guidelines still forbid it?
- 4 Write a module whose RTL simulation differs from its netlist because of an `ifdef SYNTHESIS` region. Show the mismatch with Yosys (which defines SYNTHESIS) and Icarus.
- 5 Infer a 64 x 16 dual-port RAM (one write port, one synchronous read port on different addresses) and check with Yosys `stat` that it is recognised as a memory. What changes if the read is asynchronous?
- 6 Review a given always block and list, in order, the five questions of the interview box above, answering each one.

PART III

SystemVerilog for Design

SystemVerilog turned Verilog from a netlist-era description language into a modern design language: a single `logic` type, strong enums, structs and unions, packages, interfaces, intent-carrying always blocks and a rich set of operators. This part covers every design-side feature at the level of the IEEE 1800 rules, shows what each one becomes in hardware, which tools accept it, and how the open-source RTL of real SoCs (lowRISC, PULP, OpenTitan) uses it.

12. SystemVerilog Data Types: logic, 2-State Types, Enums, Typedefs, Structs, Unions, Strings and Casting

Chapter 3 described the Verilog-2005 type system: nets and variables, four-valued logic, vectors and arrays. That system has two famous weaknesses. First, the keyword `reg` suggests a register but only means "a variable", so every beginner has to unlearn it. Second, everything is an untyped bit vector: there is no way to say that a signal is a state of an FSM, an AXI response code or a packet header, so the compiler cannot catch mistakes. SystemVerilog fixes both. It separates the **kind** of an object (net or variable) from its **data type**, adds 2-state integer types for fast testbenches, and adds user-defined types: enumerations, structures, unions and typedefs. It also adds a real `string` type and a disciplined casting system.

Everything in this chapter is used daily on an SoC project. RTL designers use `logic`, enums, packed structs and typedefs; DV engineers additionally use the 2-state types, strings, unpacked structs and `$cast`. Examples were run with Icarus Verilog 12 where it supports the construct and with Verilator 5.020 otherwise; the chapter notes where the two tools differ, because those differences are exactly the portability traps you will meet in industry.

12.1 logic, reg and wire: kind versus data type

In IEEE 1800 every declared object has a **kind** - net or variable - and a **data type** - the set of values it can hold. `logic` is a data type: the 4-state (0, 1, x, z) single bit, extended to vectors with a packed range. `reg` is simply a synonym for `logic` kept for backward compatibility. `wire` is a net kind whose default data type is `logic`. The rules that decide what a declaration is are:

Declaration	Kind	Data type	Notes
<code>logic [7:0] a;</code>	variable	<code>logic [7:0]</code>	<code>logic</code> alone (outside a port) always declares a variable
<code>reg [7:0] a;</code>	variable	<code>logic [7:0]</code>	identical to the line above
<code>wire [7:0] a;</code>	net	<code>logic [7:0]</code>	<code>wire</code> is shorthand for <code>wire logic</code>
<code>wire logic [7:0] a;</code>	net	<code>logic [7:0]</code>	explicit form
<code>var logic [7:0] a;</code>	variable	<code>logic [7:0]</code>	explicit form; <code>var</code> alone implies logic
<code>input logic [7:0] a</code>	net	<code>logic [7:0]</code>	input/inout ports with only a data type are nets (default net type)
<code>output logic [7:0] y</code>	variable	<code>logic [7:0]</code>	output ports with a data type are variables
<code>wire bit b;</code>	-	-	illegal: a net's data type must be 4-state

The practical difference between a net and a variable is **how it may be driven**. A net resolves any number of drivers (continuous assignments, primitive outputs, port connections) with the resolution function of its net type - this is how tri-state buses and `inout` pads work. A variable has no resolution function, so IEEE 1800 (6.5) allows a variable to be written **either** by any number of procedural assignments **or** by exactly one continuous assignment or one output port - never both, and never two continuous drivers. `always_comb`, `always_ff` and `always_latch` tighten this further: a variable written by one of them may not be written by any other process (Chapter 14).

```
module t12_multi(input logic a, b, output logic y);
    assign y = a;          // first continuous driver
    assign y = b;          // second continuous driver: illegal on a variable
endmodule
```

Two continuous drivers on a `logic` variable.

```
t12_multi.sv:3: error: Unresolved net/uwire y cannot have multiple drivers.
1 error(s) during elaboration.
```

Icarus Verilog 12: a hard elaboration error, as IEEE 1800 requires.

```
%Warning-MULTIDRIVEN: t12_multi.sv:1:49: Bits [0:0] of signal 'y' have multiple
combinational drivers
```

Verilator 5.020: only a lint warning; the model is still built.

PITFALL: THE SINGLE-DRIVER RULE IS ENFORCED UNEVENLY

The rule "a variable may not have multiple continuous drivers" is what makes **logic** safer than **wire**: an accidental second **assign** or two instances driving the same signal becomes a compile error instead of a silent x in simulation. Commercial simulators and synthesis tools report it as an error; Verilator only warns (MULTIDRIVEN), so a design that is clean in a Verilator-only flow can fail the first time it meets VCS, Xcelium or Design Compiler. Treat MULTIDRIVEN as an error in lint.

HOUSE STYLE USED ON MOST SOC PROJECTS

Declare everything as **logic** - ports, internal signals, flops. Use **wire** (or **tri**) only for signals that genuinely have several drivers: bidirectional pads, tri-state test buses and analog-ish top-level connections. Put the directive **default_nettype none** (written with a leading grave accent, as in the listings) at the top of every file so that a typo cannot create an implicit 1-bit wire (Chapter 8). lowRISC and PULP style guides both mandate **logic**.

12.2 Integer and real types: 2-state versus 4-state

SystemVerilog adds C-like integer types. The 2-state types store only 0 and 1; they need half the memory of 4-state types and let simulators use native machine arithmetic, which is why testbench code uses them. The table lists every built-in integral and real type.

Type	States	Width	Default signedness	Typical use
bit	2	1 (vector with range)	unsigned	TB flags, 2-state vectors
byte	2	8	signed	characters, byte buffers in TB
shortint	2	16	signed	C interoperability (DPI)
int	2	32	signed	loop counters, TB arithmetic
longint	2	64	signed	addresses, cycle counts, DPI
logic / reg	4	1 (vector with range)	unsigned	all RTL
integer	4	32	signed	legacy Verilog loop variables
time	4	64	unsigned	simulation time values
real / realtime	-	64-bit IEEE double	-	models, delays, analog
shortreal	-	32-bit IEEE float	-	C float in DPI

```
module t12_types;
    logic [3:0] l;           // 4-state, starts as 'x'
    bit  [3:0] b;           // 2-state, starts as 0
    integer  ig;            // 4-state, 32-bit signed
    int      i;             // 2-state, 32-bit signed
    byte     by;            // 2-state, 8-bit signed
    byte unsigned ub;       // 2-state, 8-bit unsigned
    shortint si;
    longint  li;
    real     r;
    shortreal sr;
    initial begin
        $display("init : l=%b b=%b ig=%0d i=%0d", l, b, ig, i);
        l = 4'b1x0z;  b = 1;           // x/z become 0 in a 2-state variable
        $display("x->2s: l=%b b=%b", l, b);
    end
endmodule
```

```

by = 8'hFF; ub = 8'hFF;
$display("byte : by=%0d ub=%0d (by>0)=%0d", by, ub, by > 0);
si = 16'h8000; li = 64'sd1 <<< 62;
$display("width: $bits(si)=%0d si=%0d $bits(li)=%0d", $bits(si), si, $bits(li));
i = -7; $display("int : i/2=%0d i>>>1=%0d i>>1=%0h", i/2, i >>> 1, i >> 1);
r = 1.0/3; sr = r;
$display("real : r=%.10f sr=%.10f", r, sr);
$finish;
end
endmodule

```

Initial values, 4-to-2-state conversion, signedness and arithmetic of the built-in types.

```

init : l=xxxx b=0000 ig=x i=0
x->2s: l=1x0z b=1000
byte : by=-1 ub=255 (by>0)=0
width: $bits(si)=16 si=-32768 $bits(li)=64
int : i/2=-3 i>>>1=-4 i>>1=7fffffff
real : r=0.3333333333 sr=0.3333333333
t12_types.sv:23: $finish called at 0 (1s)

```

Icarus Verilog 12 (a 4-state simulator).

```

init : l=0000 b=0000 ig=0 i=0
x->2s: l=0000 b=0000

```

Verilator 5.020 (a 2-state simulator) - the same code, first two lines.

Read the Icarus output line by line. A 4-state variable starts at x; a 2-state variable starts at 0. Assigning `4'b1x0z` to a `bit [3:0]` converts each x and z to 0, giving `1000` - IEEE 1800 defines this conversion and no warning is issued. `byte` is signed, so `8'hFF` stored in it is -1 and `by > 0` is false; the `unsigned` keyword changes that. Integer division truncates toward zero ($-7/2 = -3$) while the arithmetic shift `>>>` of a signed value rounds toward minus infinity (-4); the logical shift `>>` shifts in zeros. Verilator, which models every type in 2 states, prints `l=0000` where every 4-state simulator prints `xxxx` - a first hint that it cannot be your only simulator for reset and X-propagation checks.

PITFALL: 2-STATE TYPES HIDE MISSING RESETS

If a flop is declared `bit` and its reset is forgotten, simulation starts it at 0 and everything looks fine; real silicon powers up with a random value. With `logic` the missing reset shows up as x that propagates to outputs and assertions. For the same reason a DUT output sampled into a `bit` or `int` in the testbench silently turns x into 0, so a scoreboard can pass while the DUT drives x. Rule: RTL uses 4-state `logic`; testbench code checks `$isunknown(dut_sig)` before copying a DUT value into a 2-state variable. In Verilator flows, use `--x-assign unique` and `--x-initial unique` to randomize would-be-x values.

INTERVIEW INSIGHT: WHAT DOES 2-STATE BUY YOU?

Speed and memory in the testbench (a large `int` array needs half the memory of an `integer` one), exact matching of C types across DPI (`int` is `int32_t`, `longint` is `int64_t`, `byte` is `char`), and no x-pessimism in reference models. What it costs: the loss of x as a bug detector. Gate-level simulation, X-propagation ("X-prop") modes of VCS and Xcelium, and formal reset checks exist precisely to recover that detector.

12.3 Signedness defaults

Signedness is part of the data type, and it changes the result of comparisons, extensions, division, `>>>` and the conversion to real (Chapter 4). The defaults are easy to forget: `byte`, `shortint`, `int`, `longint` and `integer` are **signed**; `bit`, `logic`, `reg` and `time` are **unsigned**; user types built from them inherit the base type's signedness unless overridden.

```

int unsigned count;           // 32-bit 2-state unsigned
logic signed [15:0] sample;   // 16-bit 4-state two's complement
byte unsigned ch;             // 0..255
typedef logic signed [7:0] s8_t;

// Mixed expressions are unsigned if ANY operand is unsigned (Ch 4):
// int i = -1; bit [31:0] u = 1;
// (i < u) is FALSE: i is reinterpreted as 32'hFFFF_FFFF

```

Overriding default signedness.

PITFALL: COMPARING AN INT LOOP INDEX WITH AN UNSIGNED SIZE

`for (int i = 0; i < n - 1; i++)` with `int unsigned n = 0` does not run zero times: `n - 1` wraps to 4294967295 and the loop never terminates. Keep loop bounds and indices of the same signedness, or cast explicitly: `int'(n) - 1`. Lint rules (Verilator WIDTH, the width and sign rules of Spyglass or Ascent Lint) exist for exactly this.

12.4 Enumerated types

An enumeration declares a set of named values together with a base type. It is the single most useful SystemVerilog feature for RTL readability: FSM states, opcodes and response codes become names that appear in waveforms and log messages, and the compiler refuses to assign a raw number to an enum variable without a cast.

```

typedef enum {RED, GREEN, BLUE} color_e;           // base type int: 0,1,2
typedef enum logic [1:0] {IDLE, RUN, HALT} st_e;   // explicit 2-bit base
typedef enum logic [2:0] {A = 3'd1, B, C = 3'd6, D} e3_e; // 1,2,6,7
typedef enum bit [3:0] {OFF = 4'h0, ON = 4'hF} pwr_e;
typedef enum logic [1:0] {S0 = 2'b00, SX = 2'bxx} xs_e; // x only with 4-state base
typedef enum {R0, R[2:4], REG[3]} regs_e;         // R0, R2,R3,R4, REG0,REG1,REG2
typedef enum {P[4] = 10} p_e;                     // P0=10, P1=11, P2=12, P3=13

```

Enum declarations: default, explicit base types, explicit values and ranges.

The rules (IEEE 1800 6.19): the default base type is `int`. Unassigned names take the previous value plus one, starting at 0. Every value must fit in the base type and all values must be unique - `enum logic [1:0] {A = 3, B}` is an error because B would be 4. An x or z value is allowed only with a 4-state base type, and the name following it must have an explicit value. `name[N]` generates N names `name0..name(N-1)` and `name[N:M]` generates `nameN..nameM`. Enum literals live in the scope where the enum is declared, so two enums in the same scope cannot both define `IDLE` - hence the common prefixing convention (`ST_IDLE`, `AXI_OKAY`) or putting each enum in its own package.

Method	Returns
<code>first()</code> / <code>last()</code>	the first / last member (declaration order)
<code>next(N=1)</code> / <code>prev(N=1)</code>	the member N positions later / earlier, wrapping around; for a value that is not a member the result is the default initial value of the base type (x for a 4-state base, 0 for 2-state)
<code>num()</code>	number of members
<code>name()</code>	the member's name as a string; "" if the value is not a member

```

module t12_enum;
  typedef enum logic [1:0] {IDLE, BUSY, DONE = 2'd3} state_t;
  typedef enum {R0, R[2:4], REG[3]} regs_t;           // enum ranges
  state_t s;
  regs_t r;
  initial begin
    $display("uninit s=%b name='%s'", s, s.name());
    s = s.first();
  do begin

```



```

    $display("s=%0d %-4s next=%s prev=%s", s, s.name(), s.next().name(), s.prev().name());
    s = s.next();
end while (s != s.first());
s = IDLE; s = s.next(2);
$display("num=%0d last=%s IDLE.next(2)=%s", s.num(), s.last().name(), s.name());
for (r = r.first(); ; r = r.next()) begin
    $write("s=%0d ", r.name(), r);
    if (r == r.last()) break;
end
$display("");
s = state_t'(2); // static cast: no check
$display("state_t'(2) -> %0d name='%s'", s, s.name());
if (!$cast(s, 2)) $display("$cast(s,2) failed, s still %0d", s);
$finish;
end
endmodule

```

Enum methods, ranges, and static versus dynamic casts to an enum.

```

uninit s=00 name='IDLE'
s=0 IDLE next=BUSY prev=DONE
s=1 BUSY next=DONE prev=IDLE
s=3 DONE next=IDLE prev=BUSY
num=3 last=DONE IDLE.next(2)=DONE
R0=0 R2=1 R3=2 R4=3 REG0=4 REG1=5 REG2=6
state_t'(2) -> 2 name=''
$cast(s,2) failed, s still 2
- t12_enum.sv:23: Verilog $finish

```

Verilator 5.020.

Several details deserve attention. `next()` and `prev()` follow **declaration order**, not numerical order, and wrap around - after DONE comes IDLE even though value 2 is skipped. The static cast `state_t'(2)` compiles and stores 2, which is not a member, so `name()` returns an empty string: static casts to an enum are not checked. `$cast(s, 2)` checks at run time, returns 0 and leaves `s` unchanged. Finally, the first line shows a Verilator artefact: an uninitialized 2-bit 4-state enum is `2'bx` in any 4-state simulator, whose `name()` is "", but Verilator starts it at 0 and reports IDLE.

STRONG TYPING RULES FOR ENUMS

An enum value is automatically converted to its base type in expressions (`s + 1` is an int). The reverse is not automatic: assigning an integer expression, a different enum type or even `s + 1` back to an enum variable is a compile error without a cast. Use `s.next()` for sequencing, `st_e'(expr)` when you know the value is legal, and `$cast` when it comes from outside (a register, a file, a random number). Some tools accept the illegal assignment with a warning - do not rely on it.

WHAT AN ENUM IS IN HARDWARE

Nothing more than its base type: an `enum logic [1:0]` is a 2-bit vector and synthesizes to 2 flops. The encoding is exactly the values you wrote, unless the synthesis tool's FSM extraction re-encodes the state register (one-hot, Gray) - Design Compiler and Genus do that only when told to; FPGA tools such as Vivado and Quartus do it by default for recognized FSMs. If the encoding must be fixed (a state visible on a debug bus or in a register), give explicit values and disable FSM re-encoding for that register.

12.5 typedef

`typedef` gives a name to any type. Its value is not saving keystrokes but making the type a single point of truth: change `addr_t` in one package and every port, register and function using it follows. Common conventions are `_t` for types, `_e` for enums and `_s` for structs (lowRISC), or `_t` for everything (PULP).

```

typedef logic [31:0]          word_t;
typedef logic [AW-1:0]       addr_t;      // AW from a package parameter
typedef word_t               regfile_t [32];
typedef struct packed {logic v; addr_t a;} req_t;
typedef struct req_fwd_s;                // forward declaration
typedef class my_driver;                 // forward class declaration (Ch 18)
typedef enum logic [1:0] {OKAY, EXOKAY, SLVERR, DECERR} resp_e;

```

Typical typedefs.

Typedefs obey normal scoping: a typedef in a module is local to that module and cannot be used in its port list unless the type comes from a package or from `$unit`. That is why real projects put every shared type in a package (Chapter 15).

12.6 Structures: packed and unpacked

A `struct` groups named members. The keyword `packed` makes the struct a single contiguous bit vector; without it, the struct is a collection of independent variables, like a C struct.

Property	struct packed	struct (unpacked)
Storage	one vector; first member = most significant bits	separate members; layout is tool-defined
Member types	only packed/integral types (logic, bit, int, enums, packed structs/unions, packed arrays)	any type: real, string, class handles, unpacked arrays, queues
Use as a vector	yes: slicing, arithmetic, <code>==</code> , assign to/from vectors	no; whole-struct copy and <code>==</code> only
signed qualifier	allowed (whole vector signed)	not applicable
Member default values	not allowed	allowed: <code>int n = 4;</code>
Synthesis	fully supported by all mainstream tools	supported by most ASIC/FPGA tools for simple members; avoid for ports
Typical use	bus payloads, register maps, pipeline bundles	testbench transactions, configuration records

```

module t12_struct;
  typedef enum logic [1:0] {RD, WR, RMW} op_t;
  typedef struct packed {                // first member = most significant bits
    op_t      op;                        // [15:14]
    logic     err;                       // [13]
    logic [4:0] id;                      // [12:8]
    logic [7:0] data;                   // [7:0]
  } req_t;
  typedef struct {                       // unpacked: members are separate variables
    string    name;
    int       addr;
    logic [7:0] mask [4];
  } cfg_t;
  typedef union packed {                 // all members must have the same width
    logic [15:0] raw;
    req_t      req;
    struct packed { logic [7:0] hi, lo; } b;
  } word_u;

  req_t r;  cfg_t c;  word_u w;
  initial begin
    r = '{op: WR, id: 5'd3, data: 8'hA5, default: '0};
    $display("r=%h $bits=%0d op=%s id=%0d r[15:14]=%b", r, $bits(r), r.op.name(),
              r.id, r[15:14]);
    r = 16'h8123;                        // a packed struct is also a plain vector
    $display("r.op=%s r.err=%b r.id=%0d r.data=%h", r.op.name(), r.err, r.id, r.data);
    c = '{name: "uart0", addr: 32'h4000_0000, mask: '{default: 8'hFF}};
  end

```

```

    $display("c.name=%s addr=%h mask[3]=%h", c.name, c.addr, c.mask[3]);
    w.raw = 16'h4A5B;
    $display("w.req.op=%s w.req.data=%h w.b.hi=%h", w.req.op.name(), w.req.data, w.b.hi);
    $finish;
end
endmodule

```

Packed struct layout, unpacked struct, packed union and assignment patterns.

```

r=43a5 $bits=16 op=WR id=3 r[15:14]=01
r.op=RMW r.err=0 r.id=1 r.data=23
c.name=uart0 addr=40000000 mask[3]=ff
w.req.op=WR w.req.data=5b w.b.hi=4a
- t12_struct.sv:31: Verilog $finish

```

Verilator 5.020 (Icarus 12 does not support unpacked structs).

With `{op: WR, id: 5'd3, data: 8'hA5, default: '0}` the packed struct is `01 0 00011 10100101` = `16'h43A5`: `op` occupies bits [15:14] because it is declared first. Assigning `16'h8123` to the same struct and reading the members back decodes that vector: bits [15:14] = `2'b10` = `RMW`. The packed union `word_u` gives three views of the same 16 bits - raw, as a request, or as two bytes - which is how RTL overlays different formats onto one register or bus field.

Assignment patterns

An **assignment pattern** `{...}` builds a struct or array value from its elements. It is not a concatenation: each element is matched to a member or array element and converted to its type as if by assignment, so widths need not match exactly. Four forms exist and may be mixed (IEEE 1800 10.9):

Form	Example	Meaning
positional	<code>{2'b01, 1'b0, 5'd3, 8'hA5}</code>	members in declaration order
member label	<code>{op: WR, data: 8'hA5, ...}</code>	by name, any order
type key	<code>{logic: '0, int: -1}</code>	every member of that type
default	<code>{default: '0}</code>	every remaining member (recursively)
array index	<code>{0: 8'h1, 3: 8'hFF, default: 0}</code>	for arrays
replication	<code>{4{8'h00}}</code>	for arrays: N copies

PITFALL: {} VERSUS { }

`{a, b}` is a **concatenation**: it produces a self-determined packed vector whose width is the sum of the operand widths, and unsized constants are not allowed in it. `{a, b}` is an **assignment pattern** whose elements are assigned to members. For a packed struct both may work, but for an unpacked struct or unpacked array only the pattern is legal. Writing `{default: 0}` versus `'0` also differs: `'0` fills a packed value with zeros; `{default: 0}` works for unpacked types too.

12.7 Unions: packed, unpacked and tagged

A union stores several members in the same storage. A **packed union** requires every member to have the same width (IEEE 1800-2023 adds `union soft`, which relaxes this and right-justifies narrower members); writing one member and reading another is well defined because they are all views of the same bits. Packed unions are fully synthesizable and are the right tool for multi-format fields. An **unpacked union** may hold members of any type, but reading a member other than the one last written is undefined; it is rarely used and poorly supported by synthesis.

A **tagged union** stores a hidden tag recording which member is valid, and the language checks every read against the tag. Members can be `void`, giving a type-safe "optional" value:

```

typedef union tagged {
    void      Invalid;
    logic [31:0] Valid;
} maybe_word_t;

maybe_word_t m;
initial begin
    m = tagged Valid 32'hCAFE;           // sets tag + value
    case (m) matches                     // pattern matching on the tag
        tagged Invalid:    $display("empty");
        tagged Valid .v:   $display("value %h", v);
    endcase
    if (m matches tagged Valid .v &&& v > 0) $display("positive");
end

```

Tagged union and pattern matching (IEEE 1800 7.3.2, 12.6). Neither Icarus 12 nor Verilator 5.020 supports tagged unions; run on a commercial simulator. Expected output: "value 0000cafe" then "positive".

Tagged unions come from the Bluespec heritage of SystemVerilog and are almost never used in industrial RTL, because tool support is patchy; know that they exist for interviews and code reading.

12.8 The string type

A **string** variable holds a dynamically sized sequence of bytes. There is no terminating null character and no fixed length; assignment copies the value. Indexing `s[i]` returns a byte (0 if `i` is out of range). A string **literal** such as "abc" is not a string variable: it is a packed integral constant with 8 bits per character, which is why `logic [8*5:1] name = "hello";` is legal Verilog-2005.

Method	Effect
<code>len()</code>	number of characters
<code>putc(i, c) / getc(i)</code>	write / read the byte at index <code>i</code>
<code>toupper() / tolower()</code>	return a converted copy
<code>compare(s) / icompare(s)</code>	like C <code>strcmp</code> (negative, 0, positive); <code>icompare</code> ignores case
<code>substr(i, j)</code>	characters <code>i..j</code> inclusive ("" if out of range)
<code>atoi() atohex() atooct() atobin()</code> <code>atoreal()</code>	parse a number from the start of the string; stop at the first bad character
<code>itoa(i) hextoa(i) octtoa(i) bintoa(i)</code> <code>realtoa(r)</code>	set the string to the text form of a number
operators	<code>==, !=, <, ></code> (lexical), <code>{a, b}</code> concatenation, <code>{N{s}}</code> replication

```

module t12_str;
  string s, t;
  int    n;
  initial begin
    s = "uart";
    t = {s, "_", $sprintf("%0d", 3)};          // concatenation builds a new string
    $display("t=%s len=%0d t[0]=%s upper=%s", t, t.len(), t[0], t.toupper());
    $display("substr(0,3)=%s compare=%0d icompare=%0d", t.substr(0, 3),
              s.compare("uarT"), s.icompare("UART"));
    s = "0x1F"; n = s.substr(2, 3).atohex();
    t = "42abc";
    $display("atohex=%0d atoi=%0d", n, t.atoi());
    s.itoa(-17); t = {4{"ab"}};
    $display("itoa=%s replicate=%s ==: %0d", s, t, t == "abababab");
    s = ""; $display("empty len=%0d", s.len());
  $finish;
end
endmodule

```

String methods.

```

t=uart_3 len=6 t[0]=u upper=UART_3
substr(0,3)=uart compare=32 icompare=0
atohex=31 atoi=42
itoa=-17 replicate=abababab ==: 1
empty len=0
- t12_str.sv:16: Verilog $finish

```

Verilator 5.020.

Note that `compare("uarT")` returns 32, the difference between 't' and 'T': only the sign of the result is specified. `$sprintf` is the workhorse for building messages and is how UVM builds every report. Strings are not synthesizable as signals, but `parameter string` is accepted by most synthesis tools for names and modes (for example a Xilinx primitive's `SIM_DEVICE` parameter).

12.9 void, chandle and event

- **void** - the absence of a value: the return type of a function with no result (`function void f();`, Chapter 14), a member of a tagged union, and the cast `void'(f())` that discards a result.
- **chandle** - an opaque pointer-sized handle that can hold a C pointer passed through DPI (Chapter 24). It can only be compared, tested against `null`, assigned and passed back to C.
- **event** - a synchronization object; `->e` triggers it and `@e` or `wait(e.triggered)` waits for it. In SystemVerilog an event variable is a handle, so events can be assigned, passed to tasks and compared (Chapter 20).

12.10 Casting

SystemVerilog has two kinds of cast. A **static cast** `T'(expr)` is resolved at compile time and never fails; it converts the value as if it were assigned to a variable of the target type. A **dynamic cast** `$cast(dest, src)` is checked at run time: it is needed for enums and for class handles (downcasting, Chapter 18).

Form	Meaning	Example
<code>type'(e)</code>	convert to a type	<code>int'(r), state_e'(v), addr_t'(x)</code>
<code>N'(e)</code>	change width to N bits (truncate or extend by the signedness of e)	<code>12'(a), \$bits(t)'(0)</code>
<code>signed'(e) / unsigned'(e)</code>	change signedness, same bits	<code>signed'(a)</code>
<code>const'(e)</code>	treat as constant (for const ref arguments)	rare

Form	Meaning	Example
<code>\$cast(d, s)</code> as function	returns 1 on success, 0 on failure; d unchanged on failure	<code>if (!\$cast(st, v)) ...</code>
<code>\$cast(d, s);</code> as task	run-time error on failure	<code>\$cast(h_ext, h_base);</code>

```

module t12_cast;
  typedef enum logic [1:0] {IDLE, RUN, STOP} st_t;
  typedef logic [11:0] u12_t;
  logic [7:0] a = 8'hF0;
  logic [3:0] n = 4'hC;
  int i;
  st_t st;
  initial begin
    $display("size' : 12'(a)=%h 4'(a)=%h", 12'(a), 4'(a));
    $display("signed' : %0d unsigned': %0d", signed'(a), unsigned'(signed'(n)));
    $display("sext : %h", u12_t'(signed'(n))); // sign-extend 4 -> 12
    $display("type' : int'(3.7)=%0d int'(-3.5)=%0d", int'(3.7), int'(-3.5));
    $display("width : $bits(a+n)=%0d $bits(9'(a+n))=%0d", $bits(a + n), $bits(9'(a + n)));
    i = 2; st = st_t'(i); // compile-time, unchecked
    $display("static : st=%s", st.name());
    i = 3;
    if ($cast(st, i)) $display("$cast ok: %s", st.name());
    else $display("$cast(st, 3) failed");
    $finish;
  end
endmodule

```

Static size, sign and type casts, and \$cast.

```

size' : 12'(a)=0f0 4'(a)=0
signed' : -16 unsigned': 12
sext : ffc
type' : int'(3.7)=4 int'(-3.5)=-4
width : $bits(a+n)=8 $bits(9'(a+n))=9
static : st=STOP
$cast(st, 3) failed
- t12_cast.sv:19: Verilog $finish

```

Verilator 5.020 (Icarus 12 does not implement \$cast).

`12'(a)` zero-extends because `a` is unsigned, while `u12_t'(signed'(n))` sign-extends `4'hC` to `12'hFFC` because the operand of the outer cast is signed - the extension follows the **source** signedness, exactly like an assignment. `int'(3.7)` is 4: real to integer conversion **rounds** to nearest, with ties away from zero (-3.5 becomes -4); it does not truncate like C. `$bits(a + n)` is 8, the self-determined width of the addition; a size cast `9'(a + n)` is the idiomatic way to keep the carry of an addition inside a larger expression. Verilator's width warnings about these lines are exactly the lint checks you want on in an RTL flow.

CASTING IN RTL

Size casts replace the old trick of padding with concatenations: `sum = 9'(a) + 9'(b);` keeps the carry and documents the intent. Width casts to a parameter-dependent size are legal as long as the size is a constant expression: `AW'(base + ofs)`. Keep `$cast` in testbench code; synthesis tools do not support it.

12.11 Summary

- `logic` is a 4-state data type; `reg` is a synonym. Nets (`wire`) resolve multiple drivers; variables allow procedural writers or one continuous driver. Use `logic` everywhere except true multi-driver nets.
- 2-state types (`bit`, `byte`, `shortint`, `int`, `longint`) are fast and match C types, but convert x/z to 0 and can hide missing resets; keep RTL 4-state.
- `byte`, `shortint`, `int`, `longint` and `integer` are signed by default; `bit`, `logic` and `reg` are unsigned.

- Enums carry a base type and explicit or implicit values, provide first/last/next/prev/num/name, and are strongly typed: integer to enum requires a cast; `$cast` checks membership.
- Packed structs and unions are vectors with named fields - the backbone of SoC bus payloads; unpacked structs hold arbitrary types for testbenches. Assignment patterns `{...}` build them.
- `string` is a dynamic byte sequence with rich methods; string literals are packed constants.
- Static casts (`T'()`, `N'()`, `signed'()`) never fail; real-to-int rounds; `$cast` performs run-time checked conversion.

12.12 Exercises

- 1 Declare a packed struct for a 32-bit RISC-V I-type instruction (`imm[11:0]`, `rs1`, `funct3`, `rd`, `opcode`) so that assigning the instruction word to it decodes all fields. Verify with a test of `addi x1, x0, 5` (32'h00500093).
- 2 Write an enum for the AXI response codes with explicit values and a function that converts a raw 2-bit value to the enum using `$cast`, returning `SLVERR` for anything illegal. Why is the static cast not enough if the base type is `logic [2:0]`?
- 3 Predict and then check the output of: `int i = -1; bit [31:0] u = 1; $display(i < u, int'(i) < int'(u));`.
- 4 An FSM register is declared `bit [2:0] state;` and its reset branch is accidentally deleted. What does RTL simulation show, what does gate-level simulation show, and which lint rule catches it?
- 5 Use string methods to parse the plusarg string "addr=0x40001000" into an `int unsigned` (hint: `substr` and `atohex`).
- 6 Explain the difference between `{default: 0}`, `'0` and `{0}` when assigned to (a) a packed struct and (b) an unpacked array.

13. Arrays: Packed, Unpacked, Dynamic, Associative, Queues and Array Methods

Verilog-2005 had one kind of array: a fixed-size collection of elements indexed by integers, used for memories and register files. SystemVerilog keeps it, generalizes packed (vector) dimensions to any number of levels, and adds three **dynamic** collections - dynamic arrays, associative arrays and queues - plus a library of built-in methods for searching, sorting and reducing arrays. The fixed-size forms are the RTL designer's tools; the dynamic forms are the verification engineer's, and they are what makes scoreboards, reference models and sparse memory models a few lines long instead of a few hundred.

Kind	Declaration	Size	Index	Synthesizable
packed array	<code>logic [3:0][7:0] w;</code>	fixed	integer	yes
fixed unpacked	<code>logic [7:0] m [256];</code>	fixed	integer	yes (memories, regfiles)
dynamic	<code>int d[];</code>	set at run time with <code>new[]</code>	integer from 0	no
associative	<code>int aa[string];</code>	grows per written key	any key type	no
queue	<code>int q[\$]; / int q[\$:15];</code>	grows/shrinks at both ends	integer from 0; <code>\$ = last</code>	no (bounded queues: a few tools, avoid)

13.1 Packed versus unpacked dimensions

Dimensions written **before** the identifier are **packed**; those written **after** it are **unpacked**. A packed array is a single contiguous vector: it can be used anywhere an integral value can - arithmetic, comparison, assignment to a vector of another shape - and only single-bit types (`logic`, `bit`, `reg`), other packed arrays, packed structs and enums can be its elements. An unpacked array is a collection of separate elements of any type; the whole array can only be copied, compared for equality, or passed to a task or function.

```

logic  [3:0][7:0]  mem  [0:1][0:2];
      \_____/      \_____/
      packed        unpacked
      dims 3,4      dims 1,2

```

```

Index order in an expression:  mem [u1][u2] [p3][p4]
                                |   |   |   |
                                1   2   3   4
                                dimension:

```

Unpacked dimensions first (left to right), then packed (left to right).
 The RIGHTMOST packed dimension varies fastest: `mem[i][j][p3]` is a byte,
`mem[i][j][p3][p4]` is a bit, and `mem[i][j]` is one 32-bit word.

Dimension numbering and index order of a mixed array.

Each dimension is written either as a range `[msb:lsb]` or, for unpacked dimensions only, as a C-style size `[N]` meaning `[0:N-1]`. The query functions `$dimensions`, `$unpacked_dimensions`, `$size`, `$left`, `$right`, `$low`, `$high` and `$increment` take a dimension number counted as in the figure, and `$bits` returns the total number of bits.

```

module t13_fixed;
  //      packed dims      unpacked dims
  logic [3:0][7:0] mem  [0:1][0:2];  // 2x3 array of 32-bit words made of 4 bytes
  logic [3:0][7:0] w;
  logic [7:0]      a [4], b [4];      // C-style size [4] == [0:3]
  initial begin
    $display("$dimensions=%0d $unpacked_dimensions=%0d $bits(mem)=%0d",
             $dimensions(mem), $unpacked_dimensions(mem), $bits(mem));
    $display("$size(mem,1)=%0d $size(mem,2)=%0d $size(mem,3)=%0d $size(mem,4)=%0d",

```



```

        $size(mem, 1), $size(mem, 2), $size(mem, 3), $size(mem, 4));
    w = 32'hDDCC_BBAA;
    mem[1][2] = w; // unpacked indices first, then packed
    $display("mem[1][2][3]=%h mem[1][2][0][3:0]=%h w[2:1]=%h", mem[1][2][3],
        mem[1][2][0][3:0], w[2:1]); // w[2:1] is a 2-byte packed slice
    a = '{8'h10, 8'h20, 8'h30, 8'h40};
    b = a; // whole-array copy (same shape)
    $display("a==b: %0d", a == b);
    b[1:2] = '{8'hEE, 8'hFF}; // unpacked slice assignment
    $display("a==b: %0d b=%p", a == b, b);
    $finish;
end
endmodule

```

Mixed packed/unpacked array, slicing, whole-array assignment and comparison.

```

$dimensions=4 $unpacked_dimensions=2 $bits(mem)=192
$size(mem,1)=2 $size(mem,2)=3 $size(mem,3)=4 $size(mem,4)=8
mem[1][2][3]=dd mem[1][2][0][3:0]=a w[2:1]=ccbb
a==b: 1
a==b: 0 b='{ 'h10, 'hee, 'hff, 'h40}
- t13_fixed.sv:20: Verilog $finish

```

Verilator 5.020. Icarus 12 reports "sorry: Assignment to an entire array or to an array slice is not yet supported".

`mem[1][2][3]` selects unpacked element `[1][2]` (a 32-bit word) and then its packed byte 3, `8'hDD`. A **slice** selects a contiguous range of one dimension: `w[2:1]` is the two middle bytes of the packed word (`16'hCCBB`); `b[1:2]` is a two-element unpacked slice that can be assigned from another two-element unpacked array or pattern. Whole unpacked arrays of the same shape can be assigned and compared with `==/!=`; the element types must be equivalent and the number of elements in each dimension equal (the ranges need not be identical: `[0:3]` and `[4:1]` match element by element from the left).

WHEN TO PACK, WHEN TO UNPACK

Pack a dimension when the elements form one value that is moved, compared or operated on as a whole: bytes of a data word, lanes of a SIMD vector, fields of a bus. Unpack a dimension when elements are addressed individually by an index that changes at run time: memory words, register-file entries, per-channel state. Synthesis maps unpacked arrays that are read with a variable index to RAM or to a flop array plus a read mux, and packed arrays to plain wires. Simulators also store them differently: a large packed vector is slow to update bit by bit.

PITFALL: MEMORIES AND RESET

A `logic [31:0] mem [1024]` written in an `always_ff` with a reset branch that clears every word cannot be mapped to SRAM - RAM macros have no reset. Synthesis then builds 32768 flops. Reset only the valid bits or pointers, never the storage, and keep RAM inference templates (single write port, registered or unregistered read) exactly as the target library expects (companion RTL Design guide, memories chapter).

13.2 Dynamic arrays

A dynamic array is an unpacked array whose single (rightmost unpacked) dimension is sized at run time. It starts empty. `new[N]` allocates N default-valued elements; `new[N](src)` allocates N elements and copies as many as fit from `src` - the idiom for resizing while keeping contents. Assignment from another array or pattern resizes automatically; `size()` returns the length and `delete()` empties it.

```

module t13_dyn;
  int d[], e[];
  initial begin
    $display("empty: size=%0d", d.size());
    d = new[4]; // 4 elements, default value 0
    foreach (d[i]) d[i] = i * 10;
    e = new[6](d); // resize-and-copy: old 4 kept, 2 new = 0
    $display("d=%p e=%p", d, e);
    d = new[2](d); // shrink: keeps d[0:1]
    e = d; // assignment copies elements (not a handle)
    e[0] = 99;
    $display("d=%p e=%p size=%0d", d, e, e.size());
    d = '{1, 2, 3}; // assignment pattern resizes automatically
    d.delete();
    $display("after delete: size=%0d", d.size());
    $finish;
  end
endmodule

```

Dynamic array allocation, copy-resize, value semantics and delete.

```

empty: size=0
d='{h0, 'ha, 'h14, 'h1e} e='{h0, 'ha, 'h14, 'h1e, 'h0, 'h0}
d='{h0, 'ha} e='{h63, 'ha} size=2
after delete: size=0
- t13_dyn.sv:16: Verilog $finish

```

Verilator 5.020 (%p prints elements in hex; the format of %p is tool-specific).

Notice `e = d`; `e[0] = 99`; did not change `d`: array assignment copies elements. Arrays are values, not references - only class objects (Chapter 18) are handled by reference. Dynamic arrays are the natural container for a packet payload whose length is only known after randomization, and for the `data[]` field of a bus transaction class.

13.3 Associative arrays

An associative array stores only the elements that have been written, indexed by a key of any type: `int aa[string]`, `logic [31:0] mem [bit [63:0]]`, `txn_c by_id [int]`, or even class-handle keys. It behaves as an ordered map: iteration always visits keys in ascending order, as the standard requires. The classic use is a **sparse memory model** - a 64-bit address space where only the touched words cost memory - and a scoreboard keyed by transaction ID.

Method	Effect
<code>num()</code> / <code>size()</code>	number of entries
<code>exists(k)</code>	1 if key <code>k</code> has an entry
<code>delete()</code> / <code>delete(k)</code>	remove all entries / the entry <code>k</code>
<code>first(k)</code> / <code>last(k)</code>	set <code>k</code> to the smallest / largest key; return 0 if empty
<code>next(k)</code> / <code>prev(k)</code>	set <code>k</code> to the next larger / smaller key; return 0 at the end

```

module t13_assoc;
  logic [31:0] mem [bit [31:0]];      // sparse memory: index type bit[31:0]
  int      cnt [string];              // string-indexed
  bit  [31:0] k;
  string  s;
  initial begin
    mem[32'h0000_0010] = 32'hAAAA;  mem[32'h8000_0000] = 32'hBBBB;
    mem[32'h0000_0004] = 32'hCCCC;
    $display("num=%0d exists(4)=%0d exists(8)=%0d", mem.num(), mem.exists(4), mem.exists(8));
    if (mem.first(k)) do $display("  mem[%h] = %h", k, mem[k]); while (mem.next(k));
    mem.delete(32'h10);
    $display("after delete(10h): num=%0d  %p", mem.num(), mem);
    cnt["apb"]++; cnt["axi"] += 3; cnt["apb"]++;
    foreach (cnt[name]) $display("  cnt[%s] = %0d", name, cnt[name]);
    if (cnt.last(s)) $display("last key = %s", s);
    $finish;
  end
endmodule

```

A sparse memory and a string-keyed counter.

```

num=3 exists(4)=1 exists(8)=0
  mem[00000004] = 0000cccc
  mem[00000010] = 0000aaaa
  mem[80000000] = 0000bbbb
after delete(10h): num=2  {'h4:'hcccc, 'h80000000:'hbbbb}
  cnt[apb] = 2
  cnt[axi] = 3
last key = axi
- t13_assoc.sv:16: Verilog $finish

```

Verilator 5.020.

Iteration with `first/next` or `foreach` visits keys in ascending order (numerical for integral keys, lexicographical for strings). `cnt["apb"]++` on a missing key works because reading a missing entry returns the default value of the element type and the write then creates it. The index type matters: with `[int]` keys negative numbers sort before positive ones; with a `string` key, "10" sorts before "9".

The **wildcard index** `int aa[*]` accepts any integral key and is a Verilog-era relic kept for compatibility: the key type is not known, so `foreach` and the iteration methods cannot declare the loop variable safely, and string keys are not allowed. Prefer an explicit key type in new code.

PITFALL: READING A MISSING KEY

IEEE 1800 (7.8.6) says that reading a nonexistent entry returns the default value of the element type (x for 4-state, 0 for 2-state), may issue a warning, and does **not** create the entry. Tools differ in practice. Verilator 5.020 creates it:

```

module t13_x;
  byte by[] = '{100, 100, 100};
  int s1, s2, s3;
  logic [31:0] mem [bit [31:0]];
  logic [31:0] v;
  initial begin
    s1 = by.sum();
    s2 = by.sum() with (int'(item));
    s3 = by.sum(x) with (32'(x));
    $display("s1=%0d s2=%0d s3=%0d", s1, s2, s3);
    mem[4] = 1;
    $display("num=%0d", mem.num());
    v = mem[8];
    $display("num=%0d exists(8)=%0d v=%h", mem.num(), mem.exists(8), v);
    $finish;
  end
endmodule

```

Two tool-dependent corners: the width of a byte-array sum and a read of a missing associative-array key.

```

s1=44 s2=44 s3=44
num=1
num=2 exists(8)=1 v=00000000

```

Verilator 5.020. The first line belongs to the sum() pitfall later in this chapter; for the last line a compliant simulator prints num=1 exists(8)=0.

Portable code always tests `exists()` before reading: `v = mem.exists(a) ? mem[a] : '0`;. The same rule makes a memory model return a predictable value for uninitialized addresses instead of relying on the tool.

13.4 Queues

A queue `T q[$]` is an ordered, variable-length collection with constant-time insertion and removal at both ends - a deque. It is indexed like an array, with `0` the first element and `$` the last. A **bounded** queue `T q[$:N]` holds at most `N+1` elements; writes that would exceed the bound are ignored (with a warning in most tools).

Operation	Method form	Operator form
append	<code>q.push_back(x)</code>	<code>q = {q, x};</code>
prepend	<code>q.push_front(x)</code>	<code>q = {x, q};</code>
remove first / last	<code>x = q.pop_front() / q.pop_back()</code>	<code>x = q[0]; q = q[1:\$];</code>
insert at i	<code>q.insert(i, x)</code>	<code>q = {q[0:i-1], x, q[i:\$]};</code>
delete element i / all	<code>q.delete(i) / q.delete()</code>	<code>q = {};</code>
length	<code>q.size()</code>	<code>\$size(q)</code>

```

module t13_queue;
  int q[$] = '{2, 4, 6};
  int b[$:3]; // bounded queue: at most 4 elements
  int x;
  initial begin
    q.push_front(0); q.push_back(8); q.insert(2, 3);
    $display("q=%p size=%0d q[$]=%0d", q, q.size(), q[$]);
    x = q.pop_front(); $display("pop_front=%0d q=%p", x, q);
    x = q.pop_back(); $display("pop_back=%0d q=%p", x, q);
    $display("q[1:$]=%p q[0:1]=%p", q[1:$], q[0:1]);
    q = {q, 10}; q = {-1, q}; // concatenation also works
    q.delete(1); $display("q=%p", q);
    for (int i = 0; i < 6; i++) b.push_back(i);
    $write("bounded b size=%0d:", b.size()); foreach (b[i]) $write(" %0d", b[i]);
    $display("");
  end
endmodule

```

```

    q = {}; $display("empty size=%0d", q.size());
    $finish;
end
endmodule

```

Queue operations, slices and a bounded queue.

```

q={'h0, 'h2, 'h3, 'h4, 'h6, 'h8} size=6 q[$]=8
pop_front=0 q={'h2, 'h3, 'h4, 'h6, 'h8}
pop_back=8 q={'h2, 'h3, 'h4, 'h6}
q[1:$]='{'h3, 'h4, 'h6} q[0:1]='{'h2, 'h3}
q={'hfffffff, 'h3, 'h4, 'h6, 'ha}
bounded b size=4: 0 1 2 3
empty size=0
- t13_queue.sv:17: Verilog $finish

```

Verilator 5.020 (-1 prints as 'hfffffff in %p).

Queue slices `q[a:b]` return a new queue; indices out of range are clipped, and `q[a:b]` with `a > b` is empty. Popping an empty queue returns the default value and may issue a warning - always check `size()` first. Queues are the standard container for expected-transaction lists in scoreboards (push the prediction, pop and compare when the DUT produces output) and for the `mailbox`-free communication inside one component.

INTERVIEW INSIGHT: QUEUE VERSUS DYNAMIC ARRAY VERSUS ASSOCIATIVE ARRAY

Queue: in-order streams, FIFO/LIFO models, scoreboards of in-order protocols. **Dynamic array:** a payload or table whose size is chosen once and then indexed randomly; cheapest element access. **Associative array:** sparse or keyed data - memory models, out-of-order scoreboards keyed by AXI ID, coverage of seen values. A classic question is how to model an out-of-order AXI scoreboard: an associative array keyed by ID whose elements are queues, `txn_c by_id [int][$]`, an associative array of queues.

13.5 Array manipulation methods

IEEE 1800 (7.12) defines built-in methods on every unpacked array (fixed, dynamic, queue and, for most, associative). Many take an optional **with clause**: an expression evaluated for each element, in which the element is called `item` by default or the name given in parentheses (`find(x) with (x > 5)`); `item.index` gives its index.

Group	Methods	Result
Locator	<code>find</code> , <code>find_index</code> , <code>find_first</code> , <code>find_first_index</code> , <code>find_last</code> , <code>find_last_index</code> (with clause required)	a queue of elements or of indices (empty if nothing matches)
Locator	<code>min</code> , <code>max</code> , <code>unique</code> , <code>unique_index</code> (with clause optional)	a queue (min/max: one element, or empty for an empty array)
Ordering	<code>sort</code> , <code>rsort</code> (with clause optional), <code>reverse</code> , <code>shuffle</code>	reorders in place; not for associative arrays
Reduction	<code>sum</code> , <code>product</code> , <code>and</code> , <code>or</code> , <code>xor</code> (with clause optional)	a single value

```

module t13_meth;
    int a[] = '{7, 3, 9, 3, 1, 12};
    int r[$];
    byte by[] = '{100, 100, 100};
    typedef struct { string n; int lat; } ip_t;
    ip_t ips[] = '{{"dma", 12}, {"uart", 3}, {"ddr", 40}};
    ip_t ir[$];
    int acc;
    initial begin
        r = a.find(x) with (x > 5);          $display("find >5          %p", r);
        r = a.find_index with (item == 3);   $display("find_index ==3 %p", r);
        r = a.find_first with (item < 5);    $display("find_first <5 %p", r);
        r = a.min(); $display("min %p", r);  r = a.max(); $display("max %p", r);
        r = a.unique();                      $display("unique          %p", r);
    end
endmodule

```

```

$display("sum=%0d product=%0d xor=%0d sum(x>5)=%0d", a.sum(), a.product(),
        a.xor(), a.sum() with (int'(item > 5)));
$display("byte sum=%0d sum with int'=%0d", by.sum(), by.sum() with (int'(item)));
acc = 0; foreach (by[i]) acc += by[i]; // portable: accumulate in an int
$display("foreach accumulate=%0d", acc);
a.sort(); $display("sort %p", a);
a.rsort(); $display("rsort %p", a);
a.reverse(); $display("reverse %p", a);
ips.sort(x) with (x.lat);
foreach (ips[i]) $write("s:%0d ", ips[i].n, ips[i].lat);
ir = ips.find(x) with (x.lat > 10); $display("\nslow IPs: %0d", ir.size());
$finish;
end
endmodule

```

Locator, reduction and ordering methods, including sort by a struct member.

```

find >5      {'h7, 'h9, 'hc}
find_index ==3 {'h1, 'h3}
find_first <5 {'h3}
min {'h1}
max {'hc}
unique      {'h7, 'h3, 'h9, 'h1, 'hc}
sum=35 product=6804 xor=3 sum(x>5)=3
byte sum=44 sum with int'=44
foreach accumulate=300
sort      {'h1, 'h3, 'h3, 'h7, 'h9, 'hc}
rsort     {'hc, 'h9, 'h7, 'h3, 'h3, 'h1}
reverse   {'h1, 'h3, 'h3, 'h7, 'h9, 'hc}
uart:3 dma:12 ddr:40
slow IPs: 2
- t13_meth.sv:26: Verilog $finish

```

Verilator 5.020.

Locator methods always return a **queue**, even `min()` and `find_first` - assign them to a queue, not to a scalar. `unique()` keeps the first occurrence of each value in original order. `sort(x) with (x.lat)` sorts an array of structs by a key expression - the standard way to order transactions by time stamp or priority. The reductions return the array's element type, and that is the trap explained next.

PITFALL: THE WIDTH OF SUM() AND PRODUCT()

A reduction method returns a value of the **element type** unless a `with` clause is given, in which case the result has the type of the `with` expression (IEEE 1800 7.12.3). Summing three `byte` values of 100 overflows the 8-bit signed result: $300 \bmod 256 = 44$. The standard fix is `by.sum() with (int'(item))`, which a compliant simulator evaluates in 32 bits and returns 300. As the output above shows, Verilator 5.020 still returns 44 for the `with` form (first line of the `t13_x` output, where the result was even assigned to an `int`). The only fully portable form is an explicit loop into a wide accumulator, as the `foreach` line does. The same trap applies to `bit` arrays: `flags.sum()` over `bit flags[16]` is 1 bit wide, so counting set flags needs `flags.sum() with (int'(item))`.

13.6 Arrays in RTL: what synthesizes

Construct	Synthesis	Notes
Multi-dimensional packed arrays	yes, all tools	pure wiring
Fixed unpacked arrays	yes	RAM or flop inference depending on coding
Unpacked arrays as ports	yes in DC/Genus/Vivado/Quartus; Yosys 0.33 rejects them	prefer packed arrays or structs for ports
Whole-array assignment / ==	yes (DC, Genus, Vivado)	unrolled into per-element logic

Construct	Synthesis	Notes
Array assignment patterns '{...}'	yes in commercial tools	Yosys 0.33 rejects them (Chapter 17)
foreach over a fixed array	yes	unrolled like a for loop
Dynamic arrays, queues, associative arrays	no	testbench only
Array methods (sum, find, ...)	generally no	some tools accept reductions on fixed arrays; write loops in RTL

RTL IDIOM: THE REDUCTION YOU ACTUALLY WANT

In RTL write reductions over fixed arrays as a loop in an `always_comb`, with an explicitly sized accumulator: `cnt = '0; foreach (req[i]) cnt += CW'(req[i]);`. Synthesis turns it into an adder tree and the width is under your control. For packed vectors the unary reduction operators (`&v`, `|v`, `^v`) remain the simplest form.

13.7 Summary

- Packed dimensions (before the name) form one vector; unpacked dimensions (after the name) form a collection. Index order: unpacked left to right, then packed left to right.
- Fixed unpacked arrays support slices, whole-array copy and equality; they are the RTL memory and register-file construct.
- Dynamic arrays (`new[N]`, `new[N](old)`), associative arrays (`exists`, `first/next`, `delete`) and queues (`push/pop`, `insert`, `[a:$]` slices, bounded `[$:N]`) are testbench containers with value semantics.
- Locator methods return queues; ordering methods work in place; reductions return the element type unless a `with` clause changes it - beware 8-bit sums.
- Tools differ at the edges (missing associative keys, `with` result width, `%p` format): write code whose result does not depend on them.

13.8 Exercises

- For `logic [1:0][3:0] x [2][3]`; give `$bits(x)`, `$dimensions(x)`, `$size(x, 3)` and the width of `x[1][2][0]`. Check your answers in Verilator.
- Write a sparse 64-bit-address memory model class with `write`, `read` (returning 32'hDEAD_BEEF for never-written words, without creating entries) and `dump` (ascending address order).
- Build an out-of-order scoreboard: expected transactions go into `exp_q [int][$]` keyed by ID; actual ones are matched against `exp_q[id].pop_front()`. Report leftovers at the end of the test.
- Given `int lat[] = '{12, 40, 3, 40, 7}'`; use array methods only to compute: the index of every maximum, the number of values above 10, and the average as a real.
- Show that `bit flags[16]` summed with `flags.sum()` gives a wrong count, then fix it two ways. Which fix works in Verilator 5.020?
- Write synthesizable RTL for a 32-entry population count of `logic [31:0] v` using `foreach` and a 6-bit accumulator; synthesize it with Yosys and report the cell count.

14. Procedural Enhancements: `always_comb`/`ff`/`latch`, `unique`/`priority`, New Operators, Loops, `inside`, Streaming and Argument Passing

Verilog's procedural language has one general-purpose `always` block, a C-like but incomplete set of statements, and functions whose arguments can only be copied in. SystemVerilog adds constructs that state the designer's **intent** - this block is combinational, this one is a flop, this case is one-hot - so that simulators, synthesis and lint can check it; it completes the C statement set; and it adds operators for set membership, wildcard comparison and bit-stream packing. This chapter covers them in the order an RTL engineer meets them, then the subroutine features that testbench code relies on.

14.1 `always_comb` versus `always @*`

`always_comb` models combinational logic. Compared with the Verilog-2001 `always @*`, IEEE 1800 (9.2.2.2) gives it four extra properties:

- 1 **Time-zero evaluation.** It runs once at time 0, after all `initial` and `always` processes have started, so its outputs are consistent with its inputs even if no input ever changes. `always @*` waits for the first event and may never run.
- 2 **Complete sensitivity, including functions.** It is sensitive to every variable read in the block **and in any function it calls**; `always @*` only looks inside the block, so a function that reads a module-level signal directly leaves the block stale.
- 3 **Single writer.** Variables written by an `always_comb` may not be written by any other process - multiple drivers become a compile error rather than a race.
- 4 **No timing controls.** `#`, `@`, `wait` and blocking calls that consume time are illegal inside it; it must be pure logic.

```
module t14_comb;
  localparam logic K = 1'b0;
  logic a = 1'b0, inv = 1'b0;          // 'inv' is read only inside the function
  logic y_star, y_comb, z_star, z_comb;
  function automatic logic f(input logic x);
    return x ^ inv;                    // reads a signal that is not an argument
  endfunction
  always @*      z_star = ~K;           // no signals in the expression: never runs
  always_comb    z_comb = ~K;           // always_comb runs once at time 0
  always @*      y_star = f(a);         // sensitive to 'a' only
  always_comb    y_comb = f(a);         // sensitive to 'a' AND 'inv' (read in f)
  initial begin
    #1 $display("t=1 z_star=%b z_comb=%b y_star=%b y_comb=%b", z_star, z_comb, y_star, y_comb);
    a = 1; #1 $display("a=1 : y_star=%b y_comb=%b", y_star, y_comb);
    inv = 1; #1 $display("inv=1 : y_star=%b y_comb=%b <- always @* is stale", y_star, y_comb);
    $finish;
  end
endmodule
```

Time-zero evaluation and function sensitivity.

```
t14_comb.sv:8: warning: @* found no sensitivities so it will never trigger.
t14_comb.sv:9: warning: always_comb process has no sensitivities.
t=1 z_star=x z_comb=1 y_star=x y_comb=0
a=1 : y_star=1 y_comb=1
inv=1 : y_star=1 y_comb=0 <- always @* is stale
t14_comb.sv:16: $finish called at 3 (1s)
```

Icarus Verilog 12: the IEEE-defined behaviour.

`z_star` stays `x` forever because `~K` contains no signal to trigger `always @*` (Icarus even warns about it); `always_comb` evaluated it at time 0. When `inv` changes, only `always_comb` re-evaluates, because it is

sensitive to what `f` reads. In silicon both blocks are the same XOR gate, so the `always @*` version is a genuine **simulation/synthesis mismatch** that gate-level simulation or equivalence checking would expose.

```
t=1 z_star=1 z_comb=1 y_star=0 y_comb=0
a=1 : y_star=1 y_comb=1
inv=1 : y_star=0 y_comb=0 <- always @* is stale
- t14_comb.sv:16: Verilog $finish
```

Verilator 5.020 on the same file: no x, and no stale value.

Verilator does not reproduce either symptom: it is a 2-state, cycle-based simulator that schedules combinational logic by static analysis rather than by the event-driven rules. That is convenient but means a Verilator-only flow can never catch this class of bug - one more reason sign-off regressions run on an event-driven simulator.

ALWAYS_LATCH AND ALWAYS_FF

`always_latch` has exactly the semantics of `always_comb` (time-0 run, inferred sensitivity, single writer) but documents that a latch is intended; tools warn if the logic is not a latch, and warn on `always_comb` blocks that do infer one. `always_ff` must contain exactly one event control, at the top, and no other timing control; its variables must not be written elsewhere, and synthesis reports an error or warning if it does not infer flip-flops. Neither adds anything to simulation semantics beyond these checks - their value is that intent becomes checkable.

```
always_ff @(posedge clk or negedge rst_n) // async active-low reset flop
  if (!rst_n) q <= '0;
  else if (en) q <= d;

always_latch // transparent-high latch
  if (g) lq <= d;

always_comb begin // default first: no latch
  nxt = st;
  if (go) nxt = st.next();
end
```

Canonical forms of the three intent-specific blocks.

14.2 unique, unique0 and priority

The keywords `unique`, `unique0` and `priority` in front of `if` or `case` make two claims about the decision: whether the branch conditions are **mutually exclusive** and whether they are **complete** (some branch always matches). The simulator checks the claims at run time and reports a **violation**; synthesis uses them to simplify logic. Since IEEE 1800-2009 the checks are deferred: a violation is only reported if it is still present at the end of the time step, so zero-delay glitches while inputs settle are ignored.

Qualifier	Claims exclusive	Claims complete	Synthesis meaning	Violation reported when
(none)	no	no	priority logic in branch order; latch if incomplete	never
<code>unique</code>	yes	yes	<code>parallel_case</code> + <code>full_case</code>	no branch matches (and no else/default), or more than one matches
<code>unique0</code>	yes	no	<code>parallel_case</code>	more than one branch matches
<code>priority</code>	no (first wins)	yes	<code>full_case</code>	no branch matches (and no else/default)

```
module t14_uniq;
  logic [2:0] sel;
  logic [1:0] u, p;
  always_comb begin
    u = 2'd0;
    unique casez (sel) // claims: items are mutually exclusive AND complete
```

```

        3'b1???: u = 2'd3;
        3'b?1?:  u = 2'd2;
        3'b??1:  u = 2'd1;
    endcase
end
always_comb begin
    p = 2'd0;
    priority casez (sel)                // claims: complete; first match wins
        3'b1???: p = 2'd3;
        3'b?1?:  p = 2'd2;
        3'b??1:  p = 2'd1;
    endcase
end
initial begin
    sel = 3'b010; #1 $display("sel=%b u=%0d p=%0d", sel, u, p);
    sel = 3'b110; #1 $display("sel=%b u=%0d p=%0d", sel, u, p);
    sel = 3'b000; #1 $display("sel=%b u=%0d p=%0d", sel, u, p);
    $finish;
end
endmodule

```

A unique and a priority casez over overlapping patterns.

```

t14_uniq.sv:6: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
sel=010 u=2 p=2
sel=110 u=3 p=3
WARNING: t14_uniq.sv:14: value is unhandled for priority or unique case statement
      Time: 2   Scope: t14_uniq
WARNING: t14_uniq.sv:6: value is unhandled for priority or unique case statement
      Time: 2   Scope: t14_uniq
sel=000 u=0 p=0
t14_uniq.sv:24: $finish called at 3 (1s)

```

Icarus Verilog 12.

For sel = 3'b110 the first two items of the **unique casez** both match, a uniqueness violation - but Icarus announces at compile time that it ignores the exclusivity check and only reports the no-match case (sel = 000) for both statements. Verilator, run with **--assert**, does check exclusivity and stops on the first violation with "Assertion failed in TOP.t14_uniq: synthesis parallel_case, but multiple matches found" at time 1. Commercial simulators (VCS, Xcelium, Questa) report both kinds of violation as run-time warnings with the file, line and time.

PITFALL: UNIQUE IS A PROMISE TO SYNTHESIS, NOT A CHECK IN SILICON

With **unique case**, synthesis may build an AND-OR mux that assumes one-hot select - for the overlapping input 110 the hardware ORs two branch values, while RTL simulation picks the first matching item. With a missing default, **full_case** lets synthesis treat unlisted values as don't-care, so the gates can output anything where simulation holds the previous or default value. Both are simulation/synthesis mismatches that only the run-time violation reports protect you from - so never ignore those reports, and never use the **// synopsys parallel_case full_case** comments, which give synthesis the same freedom with no simulation check at all.

WHEN TO USE WHICH

unique case for decoders whose select is one-hot or fully decoded by design (FSM state decode with an enum covering all values, one-hot mux selects). **priority if** for arbitration where order matters but some request is guaranteed. Plain **case** with a **default** when unsure - it is always safe. **unique0** for one-hot selects that may legally be all zero.

14.3 Assignment operators, ++ and -- and their blocking nature

SystemVerilog adds the C operators `++`, `--`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`, `<<<=` and `>>>=`. IEEE 1800 (11.4.1-11.4.2) specifies that they **behave as blocking assignments**. That is harmless in `always_comb` and in testbench code, and wrong in `always_ff`, where every other process reading the variable at the same clock edge races with the update.

```
module t14_ffpit;
  logic clk = 0;
  int cnt_b = 0, cnt_nb = 0, snap_b, snap_nb;
  always #5 clk = ~clk;
  always_ff @(posedge clk) cnt_b++;           // ++ is a BLOCKING assignment
  always_ff @(posedge clk) cnt_nb <= cnt_nb + 1; // nonblocking: the flop idiom
  always_ff @(posedge clk) begin
    snap_b <= cnt_b;                          // race: old or new value?
    snap_nb <= cnt_nb;                        // always the old value
  end
  initial begin
    repeat (3) @(negedge clk)
      $display("cnt_b=%0d snap_b=%0d | cnt_nb=%0d snap_nb=%0d", cnt_b, snap_b,
              cnt_nb, snap_nb);
    $finish;
  end
endmodule
```

`cnt_b++` in an `always_ff` versus the nonblocking idiom.

```
cnt_b=1 snap_b=0 | cnt_nb=1 snap_nb=0
cnt_b=2 snap_b=2 | cnt_nb=2 snap_nb=1
cnt_b=3 snap_b=2 | cnt_nb=3 snap_nb=2
t14_ffpit.sv:15: $finish called at 30 (1s)
```

Icarus Verilog 12: `snap_b` sees the old value on cycles 1 and 3 but the new value on cycle 2.

The nonblocking pair (`cnt_nb`, `snap_nb`) behaves like two flops: the snapshot is always one behind. The blocking pair gives an answer that depends on the order in which the simulator happens to run the two processes - Icarus even changes its mind between cycles, and Verilator prints `snap_b` equal to `cnt_b` on every cycle. Synthesis builds two ordinary flops in both cases, so the RTL simulation of the blocking version does not match the netlist. Lint catches it (Verilator warns on blocking assignments in `always_ff` via BLKSEQ).

PITFALL: ++ INSIDE ALWAYS_FF

Write `cnt <= cnt + 1'b1`; in sequential blocks, never `cnt++` or `cnt += 1`. The same applies to `i++` in a `for` loop only if `i` is a module-level variable read by another process - a loop variable declared in the `for` header is local and safe.

14.4 Loops and jump statements

SystemVerilog completes the loop set: `do ... while` (body runs at least once), `foreach` over any array (the index variables are declared implicitly, are automatic and read-only), loop variables declared in the `for` header (`for (int i = 0; ...)`, local to the loop), and the jump statements `break`, `continue` and `return`. `disable` is still available for named blocks but `break/continue` are clearer and synthesizable in loops with static bounds.

```
module t14_sub;
  int data[8] = '{3, -1, 7, 0, 5, -4, 9, 2};

  // default argument values, return, void function
  function automatic int clip(int v, int lo = 0, int hi = 7);
    if (v < lo) return lo;
    if (v > hi) return hi;
```

```

    return v;
endfunction : clip

function automatic void show(string tag, const ref int a[8]); // no copy, read-only
    $write("%s:", tag);
    foreach (a[i]) $write(" %0d", a[i]);
    $display("");
endfunction

task automatic bump(ref int a[8], input int by = 1);          // modifies caller's array
    foreach (a[i]) a[i] += by;
endtask

initial begin : main
    int sum, i;
    sum = 0; i = 0;
    show("orig ", data);
    foreach (data[k]) begin : scan
        if (data[k] < 0) continue;          // skip negatives
        if (data[k] == 0) break;           // stop at the first zero
        sum += data[k];
    end : scan
    $display("sum before first 0 (positives only) = %0d", sum);
    do i++; while (data[i] != 5);           // body runs at least once
    $display("first 5 at index %0d", i);
    $display("clip(9)=%0d clip(-2)=%0d clip(9,.hi(4))=%0d clip(.v(2),.lo(3))=%0d",
        clip(9), clip(-2), clip(9, .hi(4)), clip(.v(2), .lo(3)));
    bump(data, 10);
    show("bump ", data);
    void'(clip(100));                      // explicitly discard a return value
    $finish;
end : main
endmodule

```

foreach, continue/break, do-while, default and named arguments, const ref and ref, named end labels.

```

orig : 3 -1 7 0 5 -4 9 2
sum before first 0 (positives only) = 10
first 5 at index 4
clip(9)=7 clip(-2)=0 clip(9,.hi(4))=4 clip(.v(2),.lo(3))=3
bump : 13 9 17 10 15 6 19 12
- t14_sub.sv:38: Verilog $finish

```

Verilator 5.020 (Icarus 12 rejects **const ref** and **break**).

The listing also shows **named end labels** (**end : scan**, **endfunction : clip**, **end : main**): the name after the colon must match the block's name, and the compiler checks it - worthwhile on long generate blocks and FSMs. For a multidimensional array, **foreach (m[i, j])** iterates both dimensions with i outermost.

Statement	Synthesizable?	Note
for , foreach with constant bounds	yes	unrolled
break , continue in such loops	commercial tools; Yosys 0.33 rejects break	becomes enable logic on later iterations
while , do-while	only if the tool can bound the iterations	avoid in RTL
return in a function	yes	early exit logic
repeat (N)	yes with constant N	mostly used in testbenches

14.5 Functions and tasks: the SystemVerilog additions

- **Void functions** return nothing and are called as statements; a non-void function called as a statement must be cast to void: `void'(f(x));`.
- **return** exits a function (with a value) or a task (without).
- **Default argument values**: `int lo = 0` - the argument may be omitted in a call; a missing positional argument can be skipped with an empty comma, `f(1, , 3)`.
- **Named argument binding**: `clip(.v(2), .lo(3))`, like port connections; positional and named arguments may be mixed with the positional ones first.
- **Functions with output/inout arguments** and functions that call tasks through `fork ... join_none` are legal, but a function still cannot consume time.
- **begin/end is optional** around multiple statements in a function or task body.
- **Lifetime**: in modules, interfaces and packages, tasks and functions are static by default (Verilog heritage) - declare them `automatic` unless you need shared storage. Methods of classes are **automatic by default** and cannot be static-lifetime.

Argument directions and passing by reference

Direction	Mechanism	Typical use
<code>input</code> (default)	copied in at the call	values
<code>output</code>	copied out when the subroutine returns	results of a task
<code>inout</code>	copied in at the call and out at the return	rare
<code>ref</code>	passed by reference: the subroutine sees and changes the caller's variable immediately, even while it waits for time to pass	large arrays, signals a task must watch while it runs
<code>const ref</code>	by reference, read-only	large arrays and objects passed for reading, without a copy

Two rules matter for `ref`. First, it is only allowed in subroutines with **automatic** lifetime. Second, the actual argument must be a variable of an equivalent type - not a net, not an expression, and not a part-select. The difference with `output` is visible when time passes: an `output` argument updates the caller's variable only at the end of the task, whereas with `ref` every assignment is seen immediately, and a task waiting on a `ref` argument (`@(sig)`) sees changes the caller makes.

INTERVIEW INSIGHT: WHY CAN'T REF BE USED WITH STATIC TASKS?

A static task has one set of argument variables shared by all calls; a reference would have to be rebound on every call while another caller might still be using it. Automatic subroutines allocate a fresh frame per call, so each call can hold its own reference. The same reason makes UVM declare every task and function in classes, which are automatic by default.

14.6 Set membership: inside

`expr inside {set}` is 1 if the expression equals any member of the set. Members can be single values, ranges `[lo:hi]` (inclusive, `$` for an open end), and whole arrays, whose elements are all members. Comparison uses wildcard equality in one direction: `x` and `z` bits in a **set member** are don't-cares, `x` or `z` in the tested expression are not. The result is 1 on a match, 0 if nothing matches, and `x` if nothing matches but some comparison was `x`.

```

module t14_inside;
  logic [7:0] op;
  logic [3:0] a, b;
  int      lst[] = '{2, 3, 5, 7};
  initial begin
    op = 8'h25;
    $display("op inside {[8'h20:8'h2F], 8'h40} = %b", op inside {[8'h20:8'h2F], 8'h40});
    $display("3 inside lst=%b 4 inside lst=%b", 3 inside {lst}, 4 inside {lst});
    $display("op inside {8'b0010_??01} = %b", op inside {8'b0010_??01});
    a = 4'b1010;
    $display("a ==? 4'b1?1? : %b   a !=? 4'b0??? : %b", a ==? 4'b1?1?, a !=? 4'b0???);
    $display("a == 4'b1x10 : %b   a ==? 4'b1x10 : %b", a == 4'b1x10, a ==? 4'b1x10);
    b = 4'b1x10;
    $display("b ==? 4'b1?10 : %b   b ==? 4'b1010 : %b", b ==? 4'b1?10, b ==? 4'b1010);
    $display("b inside {4'b1010, 4'b1110} : %b", b inside {4'b1010, 4'b1110});
    $finish;
  end
endmodule

```

inside with ranges, arrays and wildcards; wildcard equality.

```

op inside {[8'h20:8'h2F], 8'h40} = 1
3 inside lst=1 4 inside lst=0
op inside {8'b0010_??01} = 1
a ==? 4'b1?1? : 1   a !=? 4'b0??? : 1
a == 4'b1x10 : 1   a ==? 4'b1x10 : 1
b ==? 4'b1?10 : 1   b ==? 4'b1010 : 1
b inside {4'b1010, 4'b1110} : 1
- t14_inside.sv:16: Verilog $finish

```

Verilator 5.020 (Icarus 12: "sorry: inside expressions not supported yet").

inside replaces long OR-chains of comparisons in RTL decode (`addr inside {[BASE:BASE+SIZE-1]}`) and is synthesizable in all commercial tools and Verilator; constant ranges become comparators. It is also the membership operator of constraints (Chapter 19).

Wildcard equality ==? and !=?

a ==? b compares bit by bit, treating x and z bits **of the right-hand operand** as wildcards. An x or z in the left operand in a non-wildcard position makes the result x. This is the operator that **casez**-style matching should have been, and it is synthesizable when the right-hand side is a constant.

```

module t14_wild4;
  logic [3:0] a = 4'b1010, b = 4'b1x10;
  initial begin
    $display("a == 4'b1x10 : %b   a ==? 4'b1x10 : %b   a === 4'b1x10 : %b",
            a == 4'b1x10, a ==? 4'b1x10, a === 4'b1x10);
    $display("b ==? 4'b1?10 : %b   b ==? 4'b1010 : %b   b !=? 4'b1010 : %b",
            b ==? 4'b1?10, b ==? 4'b1010, b !=? 4'b1010);
  end
endmodule

```

Wildcard equality with 4-state operands.

```

a == 4'b1x10 : x   a ==? 4'b1x10 : 1   a === 4'b1x10 : 0
b ==? 4'b1?10 : 1   b ==? 4'b1010 : x   b !=? 4'b1010 : x

```

Icarus Verilog 12 (4-state). Verilator, being 2-state, turns the x of b into 0 and prints 1 for the same comparisons (last lines above).

Operator	x/z in left operand	x/z in right operand	Result can be x?
==, !=	unknown -> result x	unknown -> result x	yes
===, !==	compared literally	compared literally	no
==?, !=?	unknown -> result x	wildcard (don't care)	yes

Operator	x/z in left operand	x/z in right operand	Result can be x?
<code>inside</code>	unknown -> result x	set members: wildcard	yes

14.7 Streaming operators

The streaming operators pack the bits of one or more expressions into a stream and unpack it into a target, optionally reordering. `{>> slice {a, b}}` streams left to right; `{<< slice {a, b}}` takes the stream in blocks of `slice` bits (default 1, or the width of a type given as slice) and reverses the order of the blocks. The operator is a bit-stream cast: sources and targets can be vectors, packed or unpacked structs and arrays, and, in commercial simulators, dynamic arrays and queues.

```
module t14_stream;
  logic [7:0] b = 8'b1100_0101;
  logic [31:0] w = 32'h1122_3344;
  logic [3:0] ver, crc;
  logic [7:0] len;
  logic [3:0] n1, n2;
  logic [15:0] pkt;
  initial begin
    $display("{<<{b}} = %b (bit reverse)", {<<{b}});
    $display("{<<8{w}} = %h (byte swap / endian)", {<<8{w}});
    $display("{<<16{w}} = %h (half-word swap)", {<<16{w}});
    $display("{>>{w}} = %h (no reordering)", {>>{w}});
    {n1, n2} = {<<4{8'hA5}}; // stream into a concatenation of targets
    $display("{n1,n2} = %h %h", n1, n2);
    pkt = {>>4'hF, 8'h0C, 4'h3}; // concatenate fields into a packet
    $display("pkt = %h", pkt);
    {>>{ver, len, crc}} = 16'h7A53; // unpack: stream on the left-hand side
    $display("unpack : ver=%h len=%h crc=%h", ver, len, crc);
    $finish;
  end
endmodule
```

Bit reversal, byte swap, packing and unpacking with streaming operators.

```
{<<{b}} = 10100011 (bit reverse)
{<<8{w}} = 44332211 (byte swap / endian)
{<<16{w}} = 33441122 (half-word swap)
{>>{w}} = 11223344 (no reordering)
{n1,n2} = 5 a
pkt = f0c3
unpack : ver=7 len=a5 crc=3
- t14_stream.sv:19: Verilog $finish
```

Verilator 5.020 (Icarus 12: "sorry: Streaming concatenation not supported").

Note the rules. With `<<`, the **order of the slices** is reversed, not the bits within a slice: `{<<8{w}}` is a byte swap and `{<<{b}}` (slice 1) a bit reversal. `{>>{...}}` does not reorder and behaves like a concatenation that is allowed to be the target of an assignment. If the stream is wider than a fixed-size target, it is an error. If the target is wider, IEEE 1800 (11.4.14) left-justifies the stream and fills the remaining bits on the right with zeros - unlike an ordinary assignment, which zero-extends on the left. Tools do not agree on this corner:

```
module t14_pad;
  logic [15:0] y;
  initial begin
    y = {<<8'hA5};
    $display("y=%h", y);
    $finish;
  end
endmodule
```

An 8-bit stream assigned to a 16-bit target.

```
y=00a5
- t14_pad.sv:6: Verilog $finish
```

Verilator 5.020 right-justifies (16'h00A5, where the standard gives 16'hA500). Size streams to their targets exactly.

PITFALL: STREAMING INTO UNPACKED AND DYNAMIC CONTAINERS

Unpacking a word into a byte array or queue (`bytes = {>>8{w}};`) is the typical testbench use - serializing a transaction into a payload. It is legal IEEE 1800 and supported by VCS, Xcelium and Questa, but Verilator 5.020 rejects it ("Unsupported: Stream operation on a variable of a type 'logic[7:0]\${0:3}'" for an unpacked array, and a C++ compile error for a queue). In RTL, `{<<8{w}}` for byte swapping is synthesizable in all major tools; streaming with a variable slice size is not.

14.8 Summary

- `always_comb` runs at time 0, is sensitive to everything read including inside called functions, and enforces a single writer; `always @*` does none of that. `always_ff` and `always_latch` document and check intent.
- `unique` (exclusive + complete), `unique0` (exclusive) and `priority` (complete, first wins) produce run-time violation reports and give synthesis parallel/full-case freedom - mismatches if the claims are false.
- `++`, `--` and the compound assignments are blocking: do not use them on flops in `always_ff`.
- SystemVerilog adds do-while, foreach, break, continue, return, named end labels, void functions, default and named arguments, and `ref/const ref` (automatic subroutines only).
- `inside` tests set membership with ranges and wildcards; `==?/!=?` treat x/z of the right operand as don't-care.
- Streaming operators pack, unpack and reorder bit streams: `{<<8{w}}` swaps bytes, `{<<{v}}` reverses bits.

14.9 Exercises

- 1 Write a function `parity_ok()` that reads a module-level signal directly and call it from both `always @*` and `always_comb`. Show in Icarus which block goes stale and explain how a lint tool would flag the `always @*` version.
- 2 Code a 4-requester fixed-priority arbiter with `priority casez` and a one-hot grant decoder with `unique case`. Add a stimulus that violates each claim and record the reports from Icarus and from Verilator `--assert`.
- 3 Rewrite `for (i = 0; i < 16; i++) if (v[i]) begin idx = i; break; end` for synthesis, and explain whether it finds the first or the last set bit. Synthesize with Yosys.
- 4 Write a task `wait_for(ref logic sig, input int max_cycles)` that returns early when `sig` rises. Why would it not work with `input logic sig`?
- 5 Using only streaming operators, convert a 32-bit little-endian word to big-endian, reverse the bit order of each byte while keeping the byte order, and pack three fields (4, 8 and 4 bits) into a 16-bit header.
- 6 Predict the result of `4'b10x1 inside {4'b1??1, 4'b0000}` and of `4'b10x1 ==? 4'b1001`, then check them in Icarus.

15. Packages, Compilation Units, Scoping and Name Resolution

In Verilog-2005 there was no clean way to share a type, a constant or a function between modules: engineers used `include` files and `define` macros, which are textual, global and order-dependent. SystemVerilog adds **packages** - named, compiled scopes whose contents are imported by name - and precisely defines how an identifier is resolved when several scopes could supply it. On a large SoC with hundreds of IPs these rules decide whether the build is robust or breaks every time someone adds a file, so this chapter treats them with LRM precision.

15.1 Packages

A package is declared at the outermost level of a source file, between `package name;` and `endpackage`. It may contain parameters and localparams, typedefs (enums, structs, unions), functions and tasks, classes, variable declarations, `let` declarations, covergroups, DPI imports and imports of other packages. It may **not** contain processes (`initial`, `always`), module, interface or program instances, or hierarchical references - a package must be self-contained apart from references to other packages.

Package item	Notes
<code>parameter</code> / <code>localparam</code>	a <code>parameter</code> in a package behaves as a <code>localparam</code> : packages cannot be parameterized or overridden
<code>typedef</code>	the main content: bus structs, enums, register maps
functions and tasks	declare them <code>automatic</code> ; static ones share storage across every caller in the design
classes	verification packages: transactions, drivers, UVM components
variables	legal but global state shared by all importers; not synthesizable; avoid
<code>import</code> / <code>export</code>	packages can build on other packages

```
package bus_pkg;
  parameter int AW = 32;
  typedef logic [AW-1:0] addr_t;
  typedef enum logic [1:0] {OKAY, EXOKAY, SLVERR, DECERR} resp_t;
  function automatic bit is_err(resp_t r);
    return r inside {SLVERR, DECERR};
  endfunction
endpackage

package uart_pkg;
  parameter int AW = 12; // same name as bus_pkg::AW
  localparam int FIFO_D = 16;
endpackage

package soc_pkg;
  import bus_pkg::*;
  export bus_pkg::*; // re-export what soc_pkg uses from bus_pkg
  typedef struct packed { addr_t addr; resp_t resp; } txn_t;
endpackage

typedef int unsigned u32_t; // $unit scope: visible to later code in this unit

module t15_pkg;
  import soc_pkg::*; // gets txn_t and (re-exported) addr_t, resp_t
  import uart_pkg::FIFO_D; // explicit import of one name
  import uart_pkg::*; // wildcard: AW is only a *candidate*
  localparam int AW = 8; // local declaration hides the wildcard candidate
```

```

    txn_t t;
    u32_t n;
    initial begin
        t = '{addr: 32'h4000_1000, resp: bus_pkg::SLVERR};
        n = $bits(t);
        $display("AW=%0d bus_pkg::AW=%0d uart_pkg::AW=%0d FIFO_D=%0d", AW, bus_pkg::AW,
                uart_pkg::AW, FIFO_D);
        $display("txn=%h bits=%0d is_err=%0d", t, n, bus_pkg::is_err(t.resp));
        $finish;
    end
endmodule

```

Three packages, a re-export, a \$unit typedef, and the four ways of referring to package items.

```

AW=8 bus_pkg::AW=32 uart_pkg::AW=12 FIFO_D=16
txn=100004002 bits=34 is_err=1
- t15_pkg.sv:36: Verilog $finish

```

Verilator 5.020 (Icarus 12 supports packages and imports but not **export** or **inside**).

The listing uses all four referencing mechanisms. `bus_pkg::AW` is a **scope-resolution** reference: always unambiguous, needs no import, and is the recommended style for occasional use of a name. `import uart_pkg::FIFO_D;` is an **explicit import**. `import uart_pkg::*;` is a **wildcard import**. `export bus_pkg::*;` inside `soc_pkg` **re-exports** the names that `soc_pkg` itself imported from `bus_pkg`, so that users of `soc_pkg` see `addr_t` and `resp_t` without importing `bus_pkg` themselves.

15.2 Explicit versus wildcard import: the exact rules

The two import forms look similar but follow different rules (IEEE 1800 26.3), and most package-related build failures come from not knowing the difference.

	Explicit: <code>import p::x;</code>	Wildcard: <code>import p::*;</code>
What is imported	the name x, immediately	nothing yet: every name of p becomes a candidate
When a name is actually imported	at the import	when it is referenced and not found as a local declaration or explicit import of this scope
Local declaration of the same name	error (two declarations)	legal if it comes before any reference: the local name hides the candidate
Same name from two packages	error, even if never used	error only if the name is referenced (ambiguous)
Search order	like a local declaration	after local declarations of the scope, before enclosing scopes

In the listing, `localparam int AW = 8;` is legal because `AW` is only a wildcard candidate from `uart_pkg` and has not been referenced yet; the local declaration wins. The next two files show the two classic errors.

```

package a_pkg; parameter int W = 8; endpackage
package b_pkg; parameter int W = 16; endpackage
module t15_amb;
    import a_pkg::*;
    import b_pkg::*;
    logic [W-1:0] x;           // W is a candidate from both packages: ambiguous
    initial $display("%0d", $bits(x));
endmodule

```

Two wildcard imports supply the same referenced name.

```

t15_amb.sv:6: error: Ambiguous use of 'W'. It is exported by both 'a_pkg' and by 'b_pkg'.

```

Icarus Verilog 12.

```
package a_pkg; parameter int W = 8; endpackage
module t15_late;
  import a_pkg::*;
  logic [W-1:0] x;          // W referenced: a_pkg::W is now imported
  localparam int W = 4;     // too late: conflicts with the imported W
  initial $display("%0d", $bits(x));
endmodule
```

A local declaration after the wildcard candidate was already used.

```
t15_late.sv:5: error: 'W' has already been imported into this scope from package 'a_pkg'.
```

Icarus Verilog 12.

PITFALL: TOOLS THAT ACCEPT ILLEGAL IMPORTS

Verilator 5.020 builds both files above without an error - the ambiguous one silently, the late declaration with only a VARHIDDEN warning - and picks one of the candidates. Code that is "clean" in one tool fails in the next. The robust style avoids the question: import explicitly the few names a module needs, use `pkg::name` for the rest, and never wildcard-import two packages that may define the same identifier (every `_pkg` in an SoC defines `AW`, `DW` or `IDLE` sooner or later).

PITFALL: EXPORT AND ENUM LITERALS

`export pkg::*;` exports only the names that were actually imported into the exporting package - and with a wildcard import that means only the names that package referenced. `soc_pkg` references the type `resp_t` but none of its literals, so `SLVERR` is **not** exported: the listing uses `bus_pkg::SLVERR`. Exporting or importing an enum type never brings its literals along; they are separate names. Use `export *::*;` together with explicit imports of the literals, or tell users to import the defining package.

15.3 Where to put an import

```
// 1. Inside the module: visible in the body only (NOT in the port list)
module a (input logic [31:0] x);
  import bus_pkg::*;
  ...

// 2. In the module header (IEEE 1800-2009): visible in the parameter
//    list, the port list and the body - the recommended form
module b import bus_pkg::*; #(parameter int N = 2) (input addr_t addr);

// 3. At file level, outside any module: goes into $unit and leaks into
//    every later module of the same compilation unit - avoid
import bus_pkg::*;
module c (input addr_t addr);
```

The three places an import can appear.

Form 2 is what lowRISC, PULP and OpenTitan use for module ports typed with package types. Form 3 works, but its effect depends on which files end up in the same compilation unit, which is a tool option, so the same file list can compile under one simulator and fail under another.

15.4 Compilation units, \$unit and compile order

A **compilation unit** is the set of source files compiled together as one unit. IEEE 1800 (3.12.1) lets the tool decide: either each file is its own unit or all files on one command line form a single unit. Questa, for example, defaults to one unit per file and merges them with `-mfcu`; other tools default the other way. Declarations made outside any module, interface, program or package - typedefs, functions, parameters, imports - belong to the compilation-unit scope, called `$unit`, and are visible to the design elements that follow them in the same unit (the `u32_t` typedef in the listing is one). `$unit::name` refers to such a declaration explicitly.

COMPILE-ORDER RULES

A package must be compiled before any code that refers to it, and a package that imports another must come after it; the file list is therefore ordered: packages (in dependency order), then interfaces, then modules and the testbench. Modules and interfaces themselves may be compiled in any order, because instance names are resolved at elaboration. **define** macros are different again: they are text, visible from the point of definition onward in compile order across files, regardless of compilation units (Chapter 8). Build systems such as FuseSoC and Bender exist largely to compute this order from per-IP dependency files.

15.5 Name resolution

A **simple identifier** (no dots) is resolved lexically, in this order (IEEE 1800 23.9 and 26.3):

- 1 declarations and explicit imports in the current scope (block, task, function, module...);
- 2 wildcard-import candidates of the current scope;
- 3 the same two steps in each enclosing lexical scope, up to the module, interface or package;
- 4 the compilation-unit scope **\$unit** (and its imports);
- 5 for task and function calls, and for names used as the first part of a hierarchical name, an **upward search** through the instance hierarchy (see below).

```
package cfg_pkg;
  localparam int DEPTH = 16;
  localparam int WIDTH = 32;
endpackage

localparam int DEPTH = 4;           // $unit declaration
localparam int LANES = 2;           // $unit declaration

module t15_scope;
  import cfg_pkg::*;                // candidates: DEPTH, WIDTH
  localparam int WIDTH = 8;          // local declaration hides cfg_pkg::WIDTH
  initial begin : outer
    int LANES;                       // block-local, hides $unit::LANES
    LANES = 99;
    $display("DEPTH=%0d (wildcard candidate beats the outer $unit scope)", DEPTH);
    $display("WIDTH=%0d (local declaration beats the wildcard candidate)", WIDTH);
    $display("LANES=%0d $unit::LANES=%0d cfg_pkg::WIDTH=%0d", LANES, $unit::LANES,
      cfg_pkg::WIDTH);
  begin : inner
    int WIDTH;                       // nested block: hides module WIDTH
    WIDTH = 1;
    $display("inner WIDTH=%0d outer.LANES=%0d t15_scope.WIDTH=%0d", WIDTH,
      outer.LANES, t15_scope.WIDTH);
  end
  $finish;
end
endmodule
```

Local declarations, wildcard candidates, \$unit and nested block scopes competing for the same names.

```
DEPTH=16 (wildcard candidate beats the outer $unit scope)
WIDTH=8 (local declaration beats the wildcard candidate)
LANES=99 $unit::LANES=2 cfg_pkg::WIDTH=32
inner WIDTH=1 outer.LANES=99 t15_scope.WIDTH=8
t15_scope.sv:25: $finish called at 0 (1s)
```

Icarus Verilog 12.

Each line exercises one rule. **DEPTH** is not declared in the module, so the module's wildcard candidates are searched **before** the enclosing **\$unit** scope: the package value 16 wins over the **\$unit** value 4 - a surprise for anyone who expects "the closer declaration" to win, because textually the **\$unit** one is in the same file. **WIDTH** is declared locally, which hides the candidate. Inside the named blocks, block-level declarations hide

module-level ones, and the hidden names remain reachable through the block or module name (`outer.LANES`, `t15_scope.WIDTH`) or `$unit::` - hierarchical names are resolved from the scope names outward, as described next.

READING A NAME-RESOLUTION ERROR

When a tool reports an unexpected width or value, find which declaration it bound: most simulators can print the resolved declaration (for example with a cross-reference or debug-info option), and lint tools warn when a local declaration hides a package or outer name (Verilator: VARHIDDEN). Treat such warnings as errors in shared RTL.

A **hierarchical name** (`a.b.c`) is resolved by first finding `a`. If `a` is a scope visible from the current scope (a local instance, generate block or named block), the reference is **downward**. If not, the search moves up the instance tree: `a` may be the instance name or the **module name** of any ancestor, and the reference continues from there - this is **upward name referencing** (23.8). A name starting with `$root.` is absolute from the top of the hierarchy.

```
module leaf;
  int v = 1;
  function void hello();
    $display("hello from %m, mid.tag=%0d", mid.tag); // upward name reference
  endfunction
endmodule
module mid;
  int tag = 7;
  leaf u_leaf();
endmodule
module t15_hier;
  mid u_mid0();
  mid u_mid1();
  initial begin
    u_mid1.tag = 9;
    #1;
    $display("down: u_mid0.u_leaf.v=%0d", u_mid0.u_leaf.v); // downward reference
    u_mid0.u_leaf.hello();
    u_mid1.u_leaf.hello();
    $finish;
  end
endmodule
```

Downward references from the top, and an upward reference by module name from inside `leaf`.

```
down: u_mid0.u_leaf.v=1
hello from t15_hier.u_mid0.u_leaf.hello, mid.tag=7
hello from t15_hier.u_mid1.u_leaf.hello, mid.tag=9
t15_hier.sv:20: $finish called at 1 (1s)
```

Icarus Verilog 12.

`mid.tag` inside `leaf` does not name any local scope, so the simulator walks up: the parent of `u_leaf` is an instance of module `mid`, so `mid.tag` resolves to that instance's `tag` - 7 for `u_mid0` and 9 for `u_mid1`. The same text refers to different variables in different instances. `%m` prints the full hierarchical path of the current scope.

Use of hierarchical names	Where	Synthesizable?
Probing internal signals from a testbench	DV, debug	no
Backdoor register/memory access (UVM HDL paths)	DV	no
<code>bind</code> of assertion modules into RTL	DV, formal	the bound checker only
Forcing and releasing internal nets	DV, gate-level debug	no
Upward references in RTL	legacy code	no - most synthesis tools reject them

PITFALL: HIERARCHICAL REFERENCES IN RTL

Synthesis tools either reject cross-module references or silently create unwanted ports, and they break when the hierarchy is flattened, uniquified or renamed by the backend. Keep them out of RTL; in testbenches, collect them in one place (an interface or a `bind` module) so that a hierarchy change is fixed once.

15.6 The timescale directive versus timeunit and timeprecision

`timescale 1ns/1ps` is a compiler directive: it applies to every design element that follows it in compile order, across file boundaries, until the next `timescale`. A file without its own directive therefore inherits whatever the previous file set - the classic order-dependent bug. SystemVerilog adds the declarations `timeunit` and `timeprecision` (or the combined `timeunit 1ns / 1ps;`), which must appear first inside a module, interface, program or package, or at `$unit` level, and apply only to that scope. They take precedence over `timescale`.

```
`timescale 1ns/1ns
module tu_a;
    timeunit 1ns; timeprecision 1ps;          // overrides `timescale for this module
    initial begin
        #1.2345; $display("tu_a: $realtime=%0.4f (unit 1ns, precision 1ps)", $realtime);
    end
endmodule
module t15_time;                             // uses `timescale 1ns/1ns
    tu_a u_a();
    initial begin
        #1.2345; $display("top : $realtime=%0.4f (unit 1ns, precision 1ns)", $realtime);
        $prnttimescale(t15_time); $prnttimescale(t15_time.u_a);
        #2 $finish;
    end
endmodule
```

timeunit/timeprecision override the timescale directive for one module.

```
top : $realtime=1.0000 (unit 1ns, precision 1ns)
Time scale of (t15_time) is 1ns / 1ns
Time scale of (t15_time.u_a) is 1ns / 1ps
tu_a: $realtime=1.2350 (unit 1ns, precision 1ps)
t15_time.sv:13: $finish called at 3000 (1ps)
```

Icarus Verilog 12.

The top module works with 1 ns precision, so the delay 1.2345 ns is rounded to 1 ns; `tu_a` has 1 ps precision and waits 1.235 ns (1234.5 ps rounded). The simulator's global time step is the finest precision of any module, 1 ps here, which is why `$finish` reports time 3000 in ps units. A finer global precision slows simulation, which is a real concern on large SoC gate-level runs.

TIME-UNIT HYGIENE ON AN SOC

Put `timeunit 1ns; timeprecision 1ps;` in every package and in the top-level testbench, or pass one `-timescale=1ns/1ps` default to the simulator for files without any declaration - but never rely on file order. Delays in RTL (`#1` in flop models) are a simulation-only convenience that synthesis ignores.

15.7 Best practices for SoC projects

- **One package per IP**, named after it: `uart_pkg` for types and parameters, `uart_reg_pkg` for the (usually generated) register map. A top-level `soc_pkg` holds the address map, interrupt numbers and global widths.
- **One package per file**, file named after the package (`uart_pkg.sv`), compiled before its users; the IP's dependency file (FuseSoC `.core`, Bender.yml) lists packages first.
- **Prefix exported names** with the IP (`UART_FIFO_DEPTH`, `uart_cfg_t`) or rely on `uart_pkg::` qualification; enum literals especially need unique names.

- **Import in the module header** for port types; import explicitly or qualify with `::` inside bodies; never import at file level.
- **Packages cannot be parameterized.** A per-instance width belongs in a module parameter, a type parameter (Chapter 17), a parameterized class, or a typedef macro - not in a package.
- **Functions in packages are automatic and pure:** no global state, so they synthesize and can be reused in RTL and testbench.
- **Separate RTL and DV packages:** RTL packages must be synthesizable; verification packages (classes, `string`, queues) import them, never the reverse.

15.8 Summary

- Packages hold shared parameters, types, functions and classes; they cannot contain processes or instances and cannot be parameterized.
- `pkg::name` always works; explicit imports behave like local declarations; wildcard imports create candidates that are imported on first reference and are searched before enclosing scopes.
- Ambiguous wildcard references and local declarations after use are errors in compliant tools - Verilator 5.020 accepts both.
- `export` re-exports only names actually imported; enum literals need their own import.
- Compilation units and `$unit` are tool-dependent; order packages first and avoid file-level declarations.
- Hierarchical names resolve downward or upward (by instance or module name); keep them out of RTL.
- `timeunit/timeprecision` are scoped and order-independent, unlike `timescale`.

15.9 Exercises

- 1 Create `a_pkg` and `b_pkg`, both defining `typedef logic [7:0] data_t;` and `parameter int N.` Write a module that wildcard-imports both and uses only `data_t`. Is it legal? What if it also declares `localparam int N = 4;` before any use? After a use?
- 2 Build `soc_pkg` that re-exports `axi_pkg` so that `import soc_pkg::*;` alone gives access to `axi_pkg::resp_e` and all its literals. Test it in Icarus and in Verilator.
- 3 Two files each start with a different `timescale`; a third has none. Show that reordering the file list changes the behaviour of the third, then fix it with `timeunit`.
- 4 Write a checker module that is bound into every instance of a FIFO and reports its depth through an upward reference to the parent module's parameter. Why is this acceptable in a testbench but not in RTL?
- 5 Sketch the package structure (names, contents, dependencies and compile order) for an SoC with a RISC-V core, an AXI crossbar, a UART and a DMA.

16. Interfaces, Modports and Virtual Interfaces

A bus is a set of signals that always travel together and obey one protocol. In Verilog each module on the bus declares every signal as a separate port and every instantiation connects every signal again; a 40-signal AXI port becomes 40 lines in the module header, 40 in each instantiation and 40 more in the testbench. A SystemVerilog **interface** declares the bundle once. Beyond wiring, an interface can hold the protocol's **modports** (the direction of each signal as seen by each kind of participant), **tasks and functions** (for example a bus-functional model), **assertions** checking the protocol, and **clocking blocks** for race-free testbench timing. Through **virtual interfaces** it is also the bridge between the static module world and class-based testbenches, which is why every UVM agent starts from one.

16.1 Declaring and instantiating an interface

An interface is a design element like a module: it has an optional parameter list, an optional port list (usually just the clock and reset shared by all participants), and a body declaring the bundled signals. It is instantiated like a module, and an instance is connected to a module port whose type is the interface name.

```
interface vr_if #(parameter int W = 8) (input logic clk, input logic rst_n);
    logic        valid, ready;
    logic [W-1:0] data;

    modport src (output valid, data, input  ready, input clk, rst_n); // producer side
    modport dst (input  valid, data, output ready, input clk, rst_n); // consumer side
    modport mon (input  valid, ready, data, clk); // passive

    // Bus-functional-model tasks live with the signals they drive. Called at a
    // falling edge: drive at the falling edge, sample 1 time unit later (all
    // logic settled), so the DUT sees exactly the sampled values at the next
    // rising edge - race-free in any simulator.
    task automatic put(input logic [W-1:0] d);
        logic hs;
        data = d; valid = 1'b1;
        do begin #1 hs = ready; @(negedge clk); end while (!hs);
        valid = 1'b0;
    endtask
    task automatic get(output logic [W-1:0] d);
        logic hs;
        ready = 1'b1;
        do begin #1 hs = valid; d = data; @(negedge clk); end while (!hs);
        ready = 1'b0;
    endtask
endinterface

// DUT: a registered pipeline stage that adds one; ports are modports
module incr_stage #(parameter int W = 8) (vr_if.dst s, vr_if.src m);
    assign s.ready = !m.valid || m.ready; // accept when output slot free/draining
    always_ff @(posedge s.clk or negedge s.rst_n)
        if (!s.rst_n) m.valid <= 1'b0;
        else if (s.ready) begin
            m.valid <= s.valid;
            if (s.valid) m.data <= s.data + 1'b1;
        end
endmodule

// Passive monitor on the mon modport (a generic "interface i" port also works in
// commercial tools; Verilator 5.020 rejects generic interface ports)
module hs_counter (vr_if.mon i);
    int n = 0;
```



```

    always @(posedge i.clk) if (i.valid && i.ready) n++;
endmodule

module t16_vr;
    logic clk = 0, rst_n = 0;
    always #5 clk = ~clk;
    vr_if #(8) a (.clk, .rst_n);           // into the DUT
    vr_if #(8) b (.clk, .rst_n);           // out of the DUT
    incr_stage #(8) dut (.s(a.dst), .m(b.src));
    hs_counter cnt_b (.i(b.mon));
    logic [7:0] got;
    initial begin
        a.valid = 1'b0; b.ready = 1'b0;
        repeat (2) @(negedge clk); rst_n = 1'b1;
        fork
            for (int k = 0; k < 4; k++) a.put(8'h10 * k);           // producer thread
            for (int k = 0; k < 4; k++) begin                         // consumer thread
                b.get(got);
                $display("[%0t] got %h", $time, got);
            end
        join
        @(negedge clk) $display("handshakes on b: %0d", cnt_b.n);
        $finish;
    end
endmodule

```

A parameterized valid/ready interface with modports and BFM tasks, a DUT whose ports are modports, a passive monitor and a testbench.

```

[40] got 01
[50] got 11
[60] got 21
[70] got 31
handshakes on b: 4
- t16_vr.sv:64: Verilog $finish

```

Verilator 5.020 (Icarus 12 does not support interface ports: "syntax error ... Errors in port declarations").

Each value leaves the one-stage pipeline incremented by one, and the monitor counts four handshakes. Note what the listing does not contain: no port list with individual valid/ready/data signals anywhere, even though three modules and a testbench share them. Changing **W** or adding a signal (say, a **last** flag) touches only the interface and the logic that uses the new signal.

Element	Syntax	Notes
Declaration	<code>interface vr_if #(parameter int W = 8) (input logic clk, rst_n);</code>	ports are shared by all users; body signals are normally logic
Instance	<code>vr_if #(8) a (.clk, .rst_n);</code>	like a module instance
Port of interface type	<code>module m (vr_if s);</code>	any modport; all signals accessible
Port with modport	<code>module m (vr_if.dst s);</code>	only the modport's signals, with its directions
Connection	<code>m u (.s(a));</code> or <code>.s(a.dst)</code>	the modport may be chosen in the port or at the connection, not differently in both
Generic port	<code>module m (interface s);</code>	any interface type; modport chosen at connection

16.2 Modports

A **modport** lists the interface signals a participant may access and their direction **from that participant's point of view**. The producer's modport makes **valid** and **data** outputs and **ready** an input; the consumer's modport is the mirror image; a monitor's modport makes everything an input. Accessing a signal that is not in the modport, or writing an **input**, is a compile error - this is how interfaces recover the direction checking that loose ports had.

- Modport ports can be interface signals, interface ports (the **clk** above) or **modport expressions** that rename or slice: `modport lo (input .d(data[3:0]));`.
- **import** in a modport gives the participant access to tasks or functions declared in the interface: `modport src (output valid, data, input ready, import put);`.
- **export** declares that the **module** connected through that modport provides the task or function body (`task s.put(...)` in the module, where **s** is its interface port), which the interface's other users can then call. Rarely used, poorly supported in synthesis.
- A modport may also name a clocking block (`modport tb (clocking cb);`) to give the testbench a view that only allows synchronous access (Chapter 21).

PITFALL: INTERFACES WITHOUT MODPORTS

If a module port is declared with the bare interface type and no modport is chosen anywhere, every signal is accessible in any direction. Simulation does not care, but synthesis must infer directions from use and may create inout ports or fail, and lint cannot detect two modules driving the same signal. Always give synthesizable interfaces modports and connect through them.

16.3 Generic interface ports and interface arrays

A port declared with the keyword **interface** instead of an interface name accepts an instance of **any** interface type; the module works as long as the names it references exist in whatever is connected. It is the interface counterpart of a type parameter and is used for protocol-agnostic monitors. Verilator 5.020 does not implement it:

```
module hs_counter (interface i);      // generic interface port
    int n = 0;
    always @(posedge i.clk) if (i.valid && i.ready) n++;
endmodule
```

The generic form of the monitor in the listing above.

```
%Error-UNSUPPORTED: t16_vr.sv:34:20: Unsupported: generic interfaces
%Error: Exiting due to 1 error(s)
```

Verilator 5.020 on the generic version (commercial simulators and DC/Genus accept it).

Interfaces can be instantiated as arrays, and a module port can be an array of interfaces - the natural description of an N-port crossbar or an N-lane datapath:

```
vr_if #(8) lane [4] (.clk, .rst_n);      // four instances lane[0..3]

module rr_arbiter #(parameter int N = 4)
    (vr_if.dst req [N], vr_if.src gnt);    // array of interface ports
    ...
endmodule

rr_arbiter #(4) u_arb (.req(lane), .gnt(out_if));
for (genvar i = 0; i < 4; i++) begin : gen_src
    src_model u_src (.m(lane[i]));
end
```

Interface arrays and array-of-interface ports (fragment). Indices used to select an element must be constant (genvars or literals).

PITFALL: INTERFACE-ARRAY ELEMENTS NEED CONSTANT INDICES

`lane[i]` with a genvar or literal `i` is fine; a variable index (`lane[sel]` in procedural code) is illegal, because each element is a separate instance, not an element of a memory. To select among interfaces at run time, copy their signals into an unpacked array in a generate loop, or use virtual interfaces in a testbench. Tools are also uneven here: Verilator 5.020 failed to build the model (internal fault or C++ compile error) when an interface-array element was passed to a class constructor as a virtual interface, which is why the next example uses two named instances.

16.4 Tasks and functions in interfaces: bus-functional models

The `put` and `get` tasks in `vr_if` are a **bus-functional model** (BFM): they turn a transaction ("send this byte") into cycle-accurate pin activity, and they live in the interface next to the signals, so every testbench that instantiates the interface gets them. A task called through an interface instance (`a.put(8'h10)`) executes in the context of that instance and drives its signals. Declare them `automatic` so that concurrent calls do not share locals.

The tasks follow a strict timing discipline that makes them race-free in every simulator: they are called at a falling edge, drive signals there, sample the handshake one time unit later, and return at a falling edge. The DUT samples at the rising edge in the middle, when all these values are stable. The naive version - drive and sample right at the rising edge - produced a **different result** in Verilator when this example was first written (the DUT appeared to accept data one cycle early) because the order between the testbench process resuming after `@(posedge clk)` and the DUT's `always_ff` is not something to rely on.

```
clocking cb @(posedge clk);          // inside vr_if (Chapter 21)
    default input #1step output #1;  // sample just before, drive just after
    output valid, data;
    input  ready;
endclocking

task automatic put(input logic [W-1:0] d);
    cb.data <= d;  cb.valid <= 1'b1;
    do @(cb); while (!cb.ready);      // cb.ready = value sampled before the edge
    cb.valid <= 1'b0;
endtask
```

The same BFM with a clocking block - the standard solution. Verilator 5.020 rejected this version (BLKANDNBLK on valid and data); it runs on commercial simulators.

CLOCKING BLOCKS IN INTERFACES - PREVIEW OF CHAPTER 21

A clocking block declares, for one clock, which signals the testbench samples and drives and with what skew. Inputs are sampled in the Preponed region (`#1step`: the value just before the edge, exactly what the DUT's flops see); outputs are driven after the edge. Placing the clocking block in the interface and exposing it through a testbench modport makes every driver and monitor race-free by construction, which is why UVM agents access DUT pins only through clocking blocks.

16.5 Virtual interfaces

Classes (Chapter 18) are created at run time and cannot have ports, so a class-based driver cannot be connected to an interface instance the way a module is. A **virtual interface** is a variable that holds a **reference to an interface instance**: `virtual apb_if vif;`. It is assigned from an instance (`vif = bus0;`), passed around like any handle, and used with the dot notation to read and drive the instance's signals and call its tasks.

```
interface apb_if (input logic pclk);
    logic      psel, penable, pwrite, pready;
    logic [7:0] paddr;
    logic [31:0] pwrite, prdata;
    modport slv (input psel, penable, pwrite, paddr, pwrite, pclk, output pready, prdata);
```

```

endinterface

module apb_ram (apb_if.slv bus);          // tiny APB slave: 4 words, no wait states
    logic [31:0] mem [4];
    assign bus.pready = 1'b1;
    always_ff @(posedge bus.pclk) begin
        if (bus.psel && bus.penable && bus.pwrite) mem[bus.paddr[3:2]] <= bus.pwdata;
        if (bus.psel && !bus.penable && !bus.pwrite) bus.prdata <= mem[bus.paddr[3:2]];
    end
endmodule

class apb_master;                        // class-based driver
    virtual apb_if vif;                  // handle to a real interface instance
    string name;
    logic [31:0] rd_unused;
    function new(string name, virtual apb_if vif);
        this.name = name; this.vif = vif;
    endfunction
    task write(logic [7:0] a, logic [31:0] d); // call at a falling edge
        vif.psel = 1; vif.pwrite = 1; vif.paddr = a; vif.pwdata = d; // setup
        @(negedge vif.pclk) vif.penable = 1; // access
        wait_ready(rd_unused);
        vif.psel = 0; vif.penable = 0;
    endtask
    task read(logic [7:0] a, output logic [31:0] d);
        vif.psel = 1; vif.pwrite = 0; vif.paddr = a;
        @(negedge vif.pclk) vif.penable = 1;
        wait_ready(d);
        vif.psel = 0; vif.penable = 0;
    endtask
    task wait_ready(output logic [31:0] rd); // sample 1 unit after the drive
        logic rdy;
        do begin #1 rdy = vif.pready; rd = vif.prdata; @(negedge vif.pclk); end while (!rdy);
    endtask
endclass

module t16_vif;
    logic clk = 0;
    always #5 clk = ~clk;
    apb_if bus0 (.pclk(clk)), bus1 (.pclk(clk)); // two APB bus instances
    apb_ram u_ram0 (.bus(bus0));
    apb_ram u_ram1 (.bus(bus1));
    apb_master m[2];
    logic [31:0] d;
    initial begin
        m[0] = new("m0", bus0); // bind class handles to instances
        m[1] = new("m1", bus1);
        foreach (m[i]) begin m[i].vif.psel = 0; m[i].vif.penable = 0; end
        @(negedge clk);
        m[0].write(8'h4, 32'hCAFE_0000);
        m[1].write(8'h4, 32'h0000_BEEF);
        m[0].read(8'h4, d); $display("[%0t] %s read %h", $time, m[0].name, d);
        m[1].read(8'h4, d); $display("[%0t] %s read %h", $time, m[1].name, d);
        $finish;
    end
endmodule

```

An APB interface, a small APB slave, and a class-based APB master that drives two bus instances through virtual interfaces.

```
[70] m0 read cafe0000
[90] m1 read 0000beef
- t16_vif.sv:59: Verilog $finish
```

Verilator 5.020.

The same class code drives two different buses; which one is decided by the handle passed to `new`. That is the whole idea of UVM's `uvm_config_db#(virtual apb_if)::set(...)` - the top-level module publishes each interface instance, and each agent's driver and monitor retrieve the handle for the bus they serve (Chapter 26).

Rule	Detail
Type	<code>virtual [interface] name[#(params)][.modport]</code> ; the parameters must match the instance's
Default value	<code>null</code> ; using a null virtual interface is a run-time fatal error - check it after retrieving it from a config database
Assignment	from an interface instance, another virtual interface of the same type, or <code>null</code>
With a modport	<code>virtual apb_if.mst vif</code> ; restricts access to that modport's signals
Clocking blocks	<code>vif.cb.psel <= 1</code> ; and <code>@(vif.cb)</code> are the normal way UVM drivers access pins (not supported by Verilator 5.020, which reported an internal error)
Not allowed	virtual interfaces in synthesizable code or as module ports

16.6 Protocol assertions inside the interface

Because every participant of a bus connects through the interface, the interface is the ideal home for the protocol's rules: written once, they check every master and slave in every testbench, and formal tools pick them up as properties. The valid/ready rules of AXI-style handshakes are the classic example.

```
// inside interface vr_if (concurrent assertions: Chapter 22)
property p_hold_until_ready;           // once valid, hold it and the data
  @(posedge clk) disable iff (!rst_n)
    valid && !ready | => valid && $stable(data);
endproperty
a_hold: assert property (p_hold_until_ready)
      else $error("%m: valid dropped or data changed before ready");
a_known: assert property (@(posedge clk) disable iff (!rst_n)
      !$isunknown(valid) && !$isunknown(ready));
c_stall: cover property (@(posedge clk) (valid && !ready) [*3]); // 3-cycle stall
```

Handshake rules as concurrent assertions and a cover property. Shown without output: run them on a commercial simulator or a formal tool (Verilator 5.020 supports only a small subset of SVA).

Assertions in an interface are ignored by synthesis tools (or removed with `ifndef SYNTHESIS` guards), so they cost nothing in silicon. The same properties are often packaged separately as a checker bound to the interface, which lets the verification team own them without editing the design's interface file.

16.7 Interfaces and synthesis

Synthesis tools treat an interface instance as a bundle of wires that is **flattened** during elaboration: each signal becomes an ordinary net named after the instance and the signal (the exact naming - `bus_psel`, `bus.psel`, `\bus.psel` - depends on the tool). Modports provide the directions. Functions in an interface are synthesizable like any function; tasks with timing controls and clocking blocks are not and are ignored or rejected.

Aspect	Status	Advice
Interfaces with modports between RTL modules	DC, Genus, Vivado, Quartus Pro: supported; Yosys 0.33: basic cases	fine inside an IP or subsystem
Interfaces on the top-level ports of a chip or FPGA design	often unsupported or produces awkward port names	use plain ports or structs at the top level

Aspect	Status	Advice
Parameterized interfaces	supported by the major ASIC tools	keep parameters to widths
Interface arrays and generic interface ports	tool-dependent	test with every tool in the flow first
Netlist / gate-level simulation	the interface is gone after synthesis	gate-level testbenches need a wrapper that maps flattened ports back to an interface

INTERVIEW INSIGHT: INTERFACES OR STRUCTS FOR RTL BUSES?

Both appear in industry. Interfaces carry modports, tasks and assertions and read naturally; request/response **struct pairs** (Chapter 17) work in every tool, can be registered, multiplexed and stored in FIFOs with one statement, and have no direction ambiguity. Many teams therefore use structs for RTL ports (lowRISC, PULP) and interfaces for the testbench side, where virtual interfaces and clocking blocks are indispensable. A good answer mentions this split and the synthesis/netlist-naming concerns.

16.8 Summary

- An interface bundles signals, parameters, modports, tasks, assertions and clocking blocks; it is instantiated once and connected to module ports of interface type.
- Modports define per-participant directions and access, and can import interface tasks; connect through them in synthesizable code.
- Generic `interface` ports and interface arrays add flexibility but are unevenly supported (Verilator 5.020 rejects generic ports); interface-array selects need constant indices.
- BFM tasks in interfaces must follow a race-free timing discipline; clocking blocks provide it by construction.
- A virtual interface is a handle to an interface instance - the link between classes and the DUT, and the basis of every UVM agent.
- Interfaces synthesize by flattening; avoid them on top-level ports, and prefer struct pairs where every tool in the flow must agree.

16.9 Exercises

- 1 Write a complete `apb_if` with `mst`, `slv` and `mon` modports and a `mon` modport that imports a function `bit is_access()`. Use it to connect an APB master model, the `apb_ram` of this chapter and a monitor that prints every transfer.
- 2 Add a `last` signal to `vr_if` and extend the BFM with `put_pkt(byte q[$])`. How many files change?
- 3 Remove the `#1` sampling delay from `vr_if.put` and `get` and run the example again in Verilator (and, if available, a commercial simulator). Explain every difference in the output.
- 4 Build a 4-lane datapath with an interface array and a generate loop, then a monitor that counts handshakes on each lane. Why can't the monitor loop over `lane[i]` with a run-time index?
- 5 Write a class `vr_driver` holding a `virtual vr_if #(8)` handle and a mailbox of bytes; it pulls bytes and calls the interface's `put` task. Instantiate two drivers for two interface instances.
- 6 Synthesize `incr_stage` with Yosys 0.33 (with a wrapper that instantiates the interfaces) and inspect the flattened port and net names.

17. SystemVerilog Idioms for SoC RTL: Type Parameters, Struct-Based Bus Bundles, let, and the Synthesizable Subset Across Tools

The previous chapters described the language feature by feature. This chapter shows how the features combine in the RTL of real SoCs. Open-source projects such as lowRISC Ibex and OpenTitan, the PULP platform (CVA6, the `axi` and `common_cells` libraries) and many commercial code bases have converged on a small set of idioms: packages of bus typedefs, request/response struct pairs instead of dozens of loose wires, modules parameterized by **types** as well as by numbers, enum-typed state machines, register files described as structs, and assertions written next to the logic they check. The chapter ends with the question every team must answer before using any of this: which constructs do all the tools in the flow accept?

17.1 Type parameters

A module parameter can be a **type**: `parameter type T = logic [7:0]`. Inside the module, `T` is used like any typedef - for ports, variables and arrays - and each instance can override it with any type, including a packed struct from a package. Derived constants follow from the type with `$bits(T)`. A FIFO, a pipeline register, a clock-domain-crossing synchronizer or an arbiter written once this way serves every payload in the SoC.

```
// ----- bus package: one package per protocol / IP -----
package axil_pkg;
  localparam int AW = 32, DW = 32, SW = DW / 8;
  typedef logic [AW-1:0] addr_t;
  typedef logic [DW-1:0] data_t;
  typedef logic [SW-1:0] strb_t;
  typedef enum logic [1:0] {OKAY = 2'b00, SLVERR = 2'b10, DECERR = 2'b11} resp_e;
  typedef struct packed { addr_t addr; logic [2:0] prot; } ax_t; // AW / AR payload
  typedef struct packed { data_t data; strb_t strb; } w_t; // W payload
  typedef struct packed { data_t data; resp_e resp; } r_t; // R payload
  typedef struct packed { // master -> slave
    ax_t aw; logic aw_valid; w_t w; logic w_valid; logic b_ready;
    ax_t ar; logic ar_valid; logic r_ready;
  } req_t;
  typedef struct packed { // slave -> master
    logic aw_ready; logic w_ready; resp_e b_resp; logic b_valid;
    logic ar_ready; r_t r; logic r_valid;
  } rsp_t;
endpackage

// ----- generic FIFO with a type parameter -----
module fifo_t #(parameter type T = logic [7:0], parameter int DEPTH = 4,
               localparam int AW = $clog2(DEPTH)) // derived, not overridable
(
  input logic clk, rst_n,
  input logic push, input T wdata, output logic full,
  input logic pop, output T rdata, output logic empty);
  T mem [DEPTH];
  logic [AW:0] wp, rp; // extra MSB wrap bit
  let ptr_eq(a, b) = (a[AW-1:0] == b[AW-1:0]); // let: a scoped macro
  assign empty = (wp == rp);
  assign full = ptr_eq(wp, rp) && (wp[AW] != rp[AW]);
  assign rdata = mem[rp[AW-1:0]];
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin wp <= '0; rp <= '0; end
    else begin
      if (push && !full) begin mem[wp[AW-1:0]] <= wdata; wp <= wp + 1'b1; end
      if (pop && !empty) rp <= rp + 1'b1;
    end
end
```

```

`ifndef SYNTHESIS
  always_ff @(posedge clk) if (rst_n) begin                                // RTL-embedded checks
    a_no_ovf: assert (!(push && full)) else $error("%m: push while full");
    a_no_udf: assert (!(pop && empty)) else $error("%m: pop while empty");
  end
`endif
endmodule

module t17_soc;
  import axil_pkg::*;
  logic clk = 0, rst_n = 0;
  always #5 clk = ~clk;
  logic push = 0, pop = 0, full, empty;
  ax_t  wd, rd;
  fifo_t #(T(ax_t), .DEPTH(4)) u_awq (.clk, .rst_n, .push, .wdata(wd), .full,
                                       .pop, .rdata(rd), .empty);

  initial begin
    $display("$bits: ax_t=%0d req_t=%0d rsp_t=%0d  fifo AW=%0d", $bits(ax_t), $bits(req_t),
             $bits(rsp_t), u_awq.AW);
    @(negedge clk) rst_n = 1;
    for (int i = 0; i < 5; i++) begin                                     // 5th push overflows
      @(negedge clk) push = 1; wd = '{addr: 32'h1000 + 4 * i, prot: 3'(i)};
    end
    @(negedge clk) push = 0; pop = 1;
    repeat (4) begin
      @(posedge clk) $display("[%0t] pop addr=%h prot=%0d", $time, rd.addr, rd.prot);
    end
    @(negedge clk) $finish;
  end
endmodule

```

An AXI4-Lite style bus package, a FIFO parameterized by type, a let declaration and embedded immediate assertions.

```

$bits: ax_t=35 req_t=111 rsp_t=41  fifo AW=2
[65] %Error: t17_soc.sv:41: Assertion failed in TOP.t17_soc.u_awq.a_no_ovf:
      TOP.t17_soc.u_awq.a_no_ovf: push while full
-Info: t17_soc.sv:41: Verilog $stop, ignored due to +verilator+error+limit
[75] pop addr=00001000 prot=0
[85] pop addr=00001004 prot=1
[95] pop addr=00001008 prot=2
[105] pop addr=0000100c prot=3
- t17_soc.sv:66: Verilog $finish

```

Verilator 5.020 with --assert (run with +verilator+error+limit+100 so the first assertion failure does not stop the test). Icarus 12 does not support let.

Points to notice in the listing:

- **localparam int AW = \$clog2(DEPTH)** sits in the **parameter port list** (legal since IEEE 1800-2009): it is visible to the ports but cannot be overridden by an instance. The pointer scheme with an extra wrap bit is only correct for power-of-two DEPTH - a real IP would add an elaboration-time check (**if (DEPTH != 2**AW) \$error(...)** in a generate block, IEEE 1800 20.11).
- The instance overrides the type by name: **#(.T(ax_t), .DEPTH(4))**. The memory **T mem [DEPTH]** becomes 35 bits wide automatically.
- Every channel payload is a packed struct built from package typedefs, so **\$bits(req_t) = 111** is computed, never hand-counted.
- The fifth push while full triggers the embedded assertion at time 65; the FIFO correctly drops it, and the four stored entries come out in order.

RULES FOR TYPE PARAMETERS

A type parameter's default and override must be data types; a value parameter cannot be passed where a type is expected, or vice versa. Two instances with different type overrides are different modules to elaboration and synthesis (uniquified). Inside the module, only operations valid for every intended type should be used: assignment and `==` work for any packed type, arithmetic only for integral types. `type(expr)` gives the type of an expression, useful for declaring a temporary of "the same type as this port".

17.2 Struct-based bus bundles and packages of bus typedefs

An AXI4 master port has five channels and about 40 signals; an SoC interconnect has dozens of such ports. Declaring them signal by signal makes every port list hundreds of lines long and every protocol change a global edit. The dominant industrial idiom bundles the signals into **two packed structs per protocol**, one per direction: `req_t` (master to slave: all address, write data and ready-for-response signals) and `rsp_t` (slave to master). A port is then just `input req_t req_i, output rsp_t rsp_o`.

```

      req_t  (aw, aw_valid, w, w_valid, b_ready, ar, ar_valid, r_ready)
+-----+-----+-----+-----+-----+-----+-----+-----+
| master |                                     | slave |
+-----+-----+-----+-----+-----+-----+-----+-----+
      rsp_t  (aw_ready, w_ready, b_resp, b_valid, ar_ready, r, r_valid)

OpenTitan TL-UL:  tl_h2d_t / tl_d2h_t          PULP AXI:  axi_req_t / axi_resp_t

```

Two structs, one per direction, carry a whole protocol.

Why two structs rather than one or an interface? A packed struct port has a single direction, so signals flowing both ways must be split. Structs work with every synthesis tool, cross module boundaries without modports, can be registered with one statement (`req_q <= req_d;`), multiplexed with one `?`, stored in a FIFO with a type parameter, and compared in assertions. OpenTitan names them `tl_h2d_t/tl_d2h_t`; PULP names them `axi_req_t/axi_resp_t` and generates them with typedef macros, because packages cannot be parameterized:

```

// A typedef macro (style of PULP's axi/typedef.svh) builds the channel
// structs for any address/data/ID width in the scope where it is used.
`define AXIL_TYPEDEF_ALL(pfx, addr_t, data_t, strb_t)
    typedef struct packed { addr_t addr; logic [2:0] prot; } pfx``_ax_t;
    typedef struct packed { data_t data; strb_t strb; }      pfx``_w_t;
    typedef struct packed { pfx``_ax_t aw; logic aw_valid;
                           pfx``_w_t w; logic w_valid; logic b_ready;
                           pfx``_ax_t ar; logic ar_valid; logic r_ready;
                           } pfx``_req_t;

module soc_top;
    typedef logic [31:0] addr32_t; typedef logic [63:0] data64_t;
    typedef logic [7:0] strb8_t;
    `AXIL_TYPEDEF_ALL(periph, addr32_t, data64_t, strb8_t) // periph_req_t ...
    periph_req_t periph_req;
    // A generic crossbar takes the types as parameters:
    // axil_xbar #(.req_t(periph_req_t), .rsp_t(periph_rsp_t)) u_xbar (...);
endmodule

```

Typedef macros generate protocol structs for arbitrary widths (sketch; a double grave accent joins text in macro bodies, Chapter 8).

PITFALL: STRUCT PORTS IN WAVEFORMS AND AT THE NETLIST BOUNDARY

After synthesis, a struct port becomes one flat vector (`req_i[110:0]`) or a set of escaped names, depending on the tool's naming options; gate-level testbenches and UPF/SDC constraints written against RTL member names may no longer match. Agree on the netlist naming convention early (for DC, `change_names` rules; for Genus, `hdl_*` naming attributes), and write SDC against clocks and top-level ports, not struct members. Older waveform viewers show struct ports as one long vector; current ones decode the members.

17.3 let: a scoped, typed macro

`let name(args) = expression;` (IEEE 1800 11.12) declares a named expression that is expanded at each use, like a macro - but it obeys scoping (it can live in a package or module and be imported), its arguments are checked expressions rather than text, and it cannot silently capture neighbouring code the way `define` expansion can. The FIFO's `ptr_eq(a, b)` is a typical use: a small helper that would otherwise be a macro or a function with fixed argument widths. `let` arguments are untyped by default, so the same `let` works for any width; they can also be given types.

```
package util_pkg;
  let max(a, b)      = (a > b) ? a : b;
  let onehot0(v)     = ((v & (v - 1)) == '0);
  let in_range(x, lo, hi) = (x >= lo) && (x <= hi);
endpackage

// in a module: assign ok = util_pkg::onehot0(gnt);
//               localparam int W = util_pkg::max(AW, DW);
```

A package of let helpers.

Tool support is the catch: Verilator handles `let` (the FIFO above), Icarus 12 and Yosys 0.33 reject it, and support in commercial synthesis tools depends on the release. Where portability matters, a package function declared `automatic` is the safe alternative for fixed widths.

17.4 Assertions embedded in RTL

The designer knows the assumptions a block makes - a FIFO is never pushed when full, a one-hot select is one-hot, an index is in range. Writing them as assertions **inside the RTL** turns them into checks that run in every simulation of every testbench that ever instantiates the block, and into properties a formal tool can prove. Two forms matter for RTL:

Form	Example	Where it runs
Immediate assertion (in a procedural block)	<code>a_ovf: assert (!(push && full)) else \$error(...);</code>	every simulator, Verilator with <code>--assert</code> , Yosys formal flow
Deferred immediate (<code>assert final, assert #0</code>)	<code>assert final (onehot0(gnt));</code>	same, but only reports the value at the end of the time step
Concurrent assertion (Chapter 22)	<code>assert property @(posedge clk) disable iff (!rst_n) push -> !full);</code>	commercial simulators and formal; limited in Verilator

Synthesis tools ignore assertions or reject the system tasks in their action blocks, so RTL assertions are wrapped in `ifndef SYNTHESIS` (a macro every synthesis tool defines or can be told to define) or placed in a separate checker module attached with `bind` (Chapter 22), which keeps the RTL file clean. Labels (`a_no_ovf:`) give each assertion a stable hierarchical name for reports, waivers and coverage.

WHAT TO ASSERT IN RTL

Interface assumptions (no push when full, no pop when empty, valid stays high until ready), encoding invariants (one-hot state, legal enum values after reset), parameter legality at elaboration time (`$error` in a generate `if`), and X-checks on control signals (`assert (!$isunknown(valid))`). Do not duplicate the testbench's scoreboard - assertions check local rules, the scoreboard checks end-to-end function.

17.5 Enum FSMs and struct register files

The second listing combines three idioms: a register map described as a packed struct in a (normally generated) package, including a typed reset value built with an assignment pattern; a register block whose entire state is one struct-typed output port; and an FSM whose state variable has an enum type, so every log message and waveform shows state names.

```
package tmr_reg_pkg;                                // generated-style register package
    typedef struct packed {
        logic [7:0] prescale;                        // [15:8]
        logic [5:0] rsvd;                            // [7:2]
        logic      irq_en;                           // [1]
        logic      en;                               // [0]
    } ctrl_t;
    typedef struct packed {
        ctrl_t      ctrl;                            // 0x0
        logic [15:0] reload;                          // 0x4
    } regs_t;
    localparam regs_t RESET = '{ctrl: '{prescale: 8'd1, default: '0}, reload: 16'hFFFF};
endpackage

module tmr_regs import tmr_reg_pkg::*; (            // package import in the header
    input  logic      clk, rst_n,
    input  logic      wr, input logic [2:0] addr, input logic [15:0] wdata,
    output regs_t      q);                          // whole register file as a port
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q <= RESET;
        else if (wr)
            unique case (addr)
                3'h0:   q.ctrl   <= ctrl_t'(wdata & 16'hFF03); // mask reserved bits
                3'h4:   q.reload <= wdata;
                default: ;
            endcase
endmodule

module t17_regs;
    import tmr_reg_pkg::*;
    typedef enum logic [1:0] {IDLE, LOAD, COUNT} st_e; // enum-typed FSM state
    logic clk = 0, rst_n = 0, wr = 0;
    logic [2:0] addr; logic [15:0] wdata;
    regs_t q;
    st_e st;
    tmr_regs u_regs (.*);                          // .* works because names match
    always #5 clk = ~clk;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) st <= IDLE;
        else unique case (st)
            IDLE:   if (q.ctrl.en) st <= LOAD;
            LOAD:   st <= COUNT;
            COUNT:  if (!q.ctrl.en) st <= IDLE;
            default: st <= IDLE;
        endcase
    initial begin
        @(negedge clk) $display("reset: ctrl=%h reload=%h st=%s", q.ctrl, q.reload, st.name());
        rst_n = 1;
        @(negedge clk) begin wr = 1; addr = 3'h0; wdata = 16'h04FF; end
        @(negedge clk) wr = 0;
        $display("wrote : ctrl=%h prescale=%0d en=%b irq_en=%b", q.ctrl, q.ctrl.prescale,
            q.ctrl.en, q.ctrl.irq_en);
        repeat (3) @(negedge clk) $display("st=%s", st.name());
        $finish;
    end
endmodule
```

```
end
endmodule
```

Struct register file with a typed reset constant, package import in the module header, `.*` connection and an enum-typed FSM.

```
reset: ctrl=0100 reload=ffff st=IDLE
wrote : ctrl=0403 prescale=4 en=1 irq_en=1
st=LOAD
st=COUNT
st=COUNT
- t17_regs.sv:54: Verilog $finish
```

Verilator 5.020 (Icarus 12 does not parse the nested assignment pattern of RESET).

`q.ctrl <= ctrl_t'(wdata & 16'hFF03)` writes a whole register with the reserved bits masked; reading `q.ctrl.prescale` needs no bit numbers. Adding a field means editing the struct only - the decode, the reset value (built with `default: '0`) and every reader follow. Register generators (OpenTitan's reggen, PeakRDL, Agnisys, Magillem) emit exactly this kind of package, plus the matching UVM register model, from one register description (SystemRDL or IP-XACT).

PITFALL: VARIABLE INITIALIZERS AND ASYNCHRONOUS RESET AT TIME 0

In this testbench `rst_n` is declared `logic rst_n = 0`. A declaration initializer is executed before any process starts and generates **no** event, so `always_ff @(posedge clk or negedge rst_n)` does not see a falling edge at time 0. The registers are reset here only because `rst_n` is still 0 at the first rising clock edge. A block with a truly asynchronous reset and no clock during reset would stay unreset in simulation. Drive resets from an `initial` block (`rst_n = 1; #1 rst_n = 0;`) or keep the clock running during reset.

17.6 Generate, \$bits and \$clog2 in SystemVerilog style

Generate constructs (Chapter 7) combine naturally with the SV types. Loop variables are declared in the loop (`for (genvar i = 0; ...)`), generate blocks should be named so that their instances have stable hierarchical names (`gen_lane[2].u_fifo`), and conditional generate can test type-derived constants.

```
module lanes #(parameter int N = 4, parameter type T = logic [31:0],
               localparam int IW = (N > 1) ? $clog2(N) : 1) // never 0 wide
(input logic      clk, rst_n,
 input T          in_data [N],
 input logic [IW-1:0] sel,
 output T         out_data);
T stage [N];
for (genvar i = 0; i < N; i++) begin : gen_lane
    if ($bits(T) > 64) begin : gen_wide           // pipeline wide payloads
        always_ff @(posedge clk) stage[i] <= in_data[i];
    end else begin : gen_narrow
        assign stage[i] = in_data[i];
    end
end
assign out_data = stage[sel];
endmodule
```

Named generate blocks, a type-dependent generate condition and a guarded `$clog2` (IEEE 1800 constant functions in parameters).

PITFALL: \$CLOG2(1) IS 0

`$clog2(N)` is the number of bits needed to encode N distinct values - 0 for $N = 1$. A select `logic [$clog2(N)-1:0]` then becomes `[-1:0]`, a 2-bit vector with a reversed range, and the design silently misbehaves. Guard every such parameter (`(N > 1) ? $clog2(N) : 1`) and remember that `$clog2(5)` is 3 but a counter to 5 inclusive needs `$clog2(5 + 1)`.

17.7 The synthesizable SystemVerilog subset across tools

"Synthesizable SystemVerilog" is not one subset: each tool accepts a different one, and an SoC flow usually passes the same RTL through five or six of them (lint, simulation, FPGA prototyping, ASIC synthesis, equivalence checking, formal). The table combines two kinds of information. The **Yosys 0.33** and **Verilator 5.020** columns were measured in this book's environment with small test modules (Yosys: `read_verilog -sv` then `synth`). The commercial columns summarize the vendors' documented support in recent releases; check the exact version your project uses.

Construct	DC / Genus	Vivado	Quartus Pro	Yosys 0.33	Verilator
<code>logic, always_ff/comb/latch</code>	yes	yes	yes	yes	yes
Enums, enum-typed FSM	yes	yes	yes	yes	yes
Packed structs and unions as ports	yes	yes	yes	yes	yes
Multi-dimensional packed arrays	yes	yes	yes	no	yes
Unpacked arrays as ports	yes	yes	yes	no	yes
Package <code>p::name</code> references	yes	yes	yes	yes	yes
<code>import p::*</code> (body or module header)	yes	yes	yes	no	yes
Functions in packages	yes	yes	yes	no	yes
Assignment patterns <code>{...}</code>	yes	yes	yes	no	yes
<code>'0</code> , size casts <code>N'(x)</code> , <code>\$clog2</code> params	yes	yes	yes	yes	yes
Type parameters	yes	yes	yes	no	yes
Interfaces with modports	yes	yes	yes	basic	yes
Generic interface ports	yes	check	check	no	no
<code>inside, ==?</code>	yes	yes	yes	no	yes
Streaming <code>{<<8{x}}</code> (constant slice)	yes	yes	yes	no	yes
<code>foreach, break, continue</code>	yes	yes	yes	no	yes
<code>let</code>	check	check	check	no	yes
<code>unique/priority case</code>	yes (parallel/full)	yes	yes	accepted	yes; checked with <code>--assert</code>
Immediate assertions	ignored	ignored	ignored	used by formal	checked with <code>--assert</code>
Classes, strings, queues, dynamic arrays	no	no	no	no	simulation only

Synthesizable-subset matrix. Yosys and Verilator columns: tested here. Commercial columns: vendor documentation for recent releases ("check": support depends on version).

Two observations follow. First, the built-in Yosys frontend accepts less than half of the idioms in this chapter; open-source ASIC and FPGA flows therefore put a converter or a stronger frontend in front of it. Second, even among commercial tools the edges differ (generic interface ports, `let`, interfaces at the top level of an FPGA design), so the safe core for IP meant to be reused everywhere is: `logic`, the three `always_*` blocks, packages with typedefs, packed structs and enums, type and value parameters, and interfaces only inside the IP, never on its top-level ports.

SV2V, SYNLIG AND YOSYS-SLANG

sv2v (open source, Haskell) converts synthesizable SystemVerilog - packages, interfaces, structs, enums, type parameters, assignment patterns - into plain Verilog-2005 that Yosys, Icarus or an old FPGA tool accept: `sv2v -DSYNTHESIS pkg.sv rtl.sv > rtl_v2005.v`. It is not installed in this environment. **Synlig** (formerly yosys-systemverilog, based on Surelog/UHDM) and the **yosys-slang** plugin replace the Yosys frontend with full IEEE 1800 parsers. OpenTitan and PULP designs are routinely pushed through these paths into Yosys-based flows such as OpenROAD.

17.8 A checklist for SoC-grade SystemVerilog RTL

Before an IP is released to integration

- ☐ Every shared type, width and enum lives in the IP's package; no file-level declarations, no wildcard imports in `$unit`.
- ☐ Ports use package types or type parameters; bundles are `req_t/rsp_t` struct pairs where a protocol has many signals.
- ☐ Every derived constant is computed (`$bits`, `$clog2` with a guard); parameters have legality checks at elaboration.
- ☐ Only `always_ff`, `always_comb` and (intentionally) `always_latch`; nonblocking in sequential blocks; no `++` on flops.
- ☐ FSM states are enums with explicit base type; `unique case` only where exclusivity really holds, with a `default`.
- ☐ Assertions for local assumptions, labelled and guarded by `SYNTHESIS` or bound from a checker module.
- ☐ The RTL passes lint (Verilator `-Wall` or a commercial linter), compiles in the event-driven simulator, and has been read by the synthesis tool (or sv2v) used downstream.

17.9 Summary

- Type parameters (`parameter type T`) let one FIFO, register slice or arbiter serve every payload; derived constants come from `$bits(T)` and guarded `$clog2`.
- Industrial SoC RTL bundles protocols into two packed structs per direction (`req_t/rsp_t`), defined in packages or generated by typedef macros because packages cannot be parameterized.
- `let` is a scoped, checked replacement for expression macros - well supported in simulators, not yet everywhere in synthesis.
- Immediate assertions embedded in RTL check local assumptions in every simulation and feed formal tools; guard them from synthesis.
- Enum-typed FSMs and struct register files make RTL self-documenting and generator-friendly.
- Tool support defines the usable subset: Yosys 0.33's own frontend rejects packages imports, patterns, type parameters, `inside` and more - use sv2v or an SV frontend plugin; check commercial tool versions for `let` and generic interfaces.

17.10 Exercises

- 1 Extend `fifo_t` with an elaboration-time check that rejects a non-power-of-two DEPTH, then generalize the pointers so that any DEPTH ≥ 2 works.
- 2 Write a type-parameterized register slice `#(parameter type T = logic) (input T d, input logic valid_i, output logic ready_o, ...)` with a full valid/ready handshake, and instantiate it for the `ax_t` and `w_t` channels of `axil_pkg`.
- 3 Complete `axil_pkg` with a `rsp_t` and write a 4-register AXI4-Lite slave whose ports are only `input req_t req_i, output rsp_t rsp_o`.
- 4 Replace the `let` in `fifo_t` with an automatic function in a package. What must the function's argument type be, and why is the `let` version more general?

- 5 Take the FIFO and the register block from this chapter and find, by trial with Yosys 0.33, the minimal set of edits that makes them synthesize with the built-in frontend. Compare with what sv2v would do automatically.
- 6 For the table in this chapter, add a column for the simulator you use at work (or Icarus 12) by writing one test module per row.

PART IV

SystemVerilog for Verification

SystemVerilog became the language of chip verification because it added a real programming language to Verilog: classes with inheritance and polymorphism, a constraint solver that generates legal random stimulus, threads that communicate through events, semaphores and mailboxes, clocking blocks and a scheduler with dedicated regions for race-free testbenches, a temporal assertion language (SVA), functional coverage, and the DPI to call C, C++ and Python models. This part teaches each of these features at language-reference depth (Chapters 18-24): what the LRM guarantees, what simulators actually do, and how each construct is used by the verification engineers who sign off SoCs and by UVM, which is built entirely from them.

18. Object-Oriented Programming: Classes, Inheritance, Polymorphism, Parameterized and Interface Classes

Everything in Part III describes hardware: modules, nets and variables that exist from time zero to the end of simulation and whose number is fixed at elaboration. A testbench needs something different. It creates thousands of transactions, keeps some of them in a scoreboard, throws others away, builds different components depending on the test, and wants to reuse one driver across ten projects by overriding only a few methods. SystemVerilog answers with **classes**: dynamically created, garbage-collected objects with single inheritance, virtual methods, parameterization and interface classes - essentially the object model of Java, grafted onto Verilog and its event scheduler.

Classes are a **verification** feature. No synthesis tool accepts them in RTL (a handful accept classes used purely as compile-time namespaces for static functions, but do not rely on it). Every UVM component, sequence item, register model and scoreboard is a class, so fluency with the rules in this chapter - handle semantics, copy semantics, virtual dispatch, `$cast`, parameterized classes - is a hard prerequisite for Chapters 25 and 26. All examples below were compiled and run with Verilator 5.020 (`--binary --timing`); the output cards are pasted from those runs.

18.1 Classes, objects and handles

A **class** is a type that bundles data (**properties**) with the code that operates on it (**methods**: tasks and functions). A class declaration allocates nothing. Memory is allocated only when `new` constructs an **object** (an instance), and objects are reached exclusively through **handles** - typed, safe references that behave like pointers without pointer arithmetic. A handle that points nowhere holds the special value `null`, which is also the default value of every class variable.

Concept	Module	struct	class
Created	At elaboration, fixed count	With the variable that holds it	At run time by <code>new</code> , any number
Lifetime	Whole simulation	Scope of the variable	Until no handle refers to it (garbage collected)
Assignment copies	n/a (instances are not values)	The whole value	Only the handle ; both then name one object
Contains	Processes, nets, instances	Data only	Data + tasks/functions; no always/initial, no nets
Inheritance	No	No	Single inheritance + interface classes
Synthesizable	Yes	Yes (packed or unpacked)	No

Modules, structs and classes compared.

```
class Packet;
    // properties (class members); default lifetime: one copy per object
    bit [7:0]  addr;
    bit [31:0] data;
    static int count = 0;      // one copy shared by ALL Packet objects
    int       id;

    function new(bit [7:0] addr = 8'h00, bit [31:0] data = 0);
        this.addr = addr;      // 'this' disambiguates member vs argument
        this.data = data;
        id = count++;          // unique id per object
    endfunction

    function string sprint();
```

```

    return $sprintf("Packet#%0d addr=%02h data=%08h", id, addr, data);
endfunction

static function int get_count();    // may touch only static members
    return count;
endfunction
endclass

module top;
    Packet p1, p2, p3;              // handles, all null initially
    initial begin
        if (p1 == null) $display("p1 is null before new()");
        p1 = new(8'h10, 32'hCAFE_F00D);
        p2 = new();                 // default arguments
        p3 = p1;                   // copy the HANDLE: p3 and p1 name ONE object
        p3.addr = 8'h99;
        $display("%s", p1.sprint());
        $display("%s", p2.sprint());
        $display("p1==p3: %0b  p1==p2: %0b", p1 == p3, p1 == p2);
        $display("objects created: %0d (via class scope: %0d)",
            p1.get_count(), Packet::get_count());
        p1 = null; p3 = null;       // object 0 is now unreachable -> reclaimed
        $display("p2 still valid: %s", p2.sprint());
        $finish;
    end
endmodule

```

c18_basic.sv - properties, constructor with default arguments, **this**, a static counter and a static method, and handle copying.

```

p1 is null before new()
Packet#0 addr=99 data=cafef00d
Packet#1 addr=00 data=00000000
p1==p3: 1  p1==p2: 0
objects created: 2 (via class scope: 2)
p2 still valid: Packet#1 addr=00 data=00000000
- top.sv:38: Verilog $finish

```

Verilator 5.020 output.

Read the output line by line, because each line is a rule:

- **Handles start as null.** Calling a method or touching a property through a null handle is a run-time fatal error (Verilator 5.020 aborts with 'Null pointer dereferenced' and the source line; commercial simulators report a null object access and stop). Always construct before use, and test `h == null` when a handle is optional.
- **p3 = p1 copies the handle, not the object.** Writing `p3.addr` changed what `p1` sees, because there is only one object. Handle equality (`==`) compares identity, not contents.
- **The constructor** is the function called `new`. It has no return type (it implicitly returns the new object), may have arguments with defaults, and if you do not write one the compiler provides an implicit `new()` that only runs the property initializers. Inside `new`, property declaration initializers (`int count = 0;`) execute first - after the implicit or explicit `super.new()` - and then the body.
- **this** is a handle to the current object. It is needed only when a local name (here the arguments `addr` and `data`) hides a property; Verilator warns about the hiding (VARHIDDEN), which is exactly the situation where `this` is required.
- **Static properties** exist once per class, not per object - `count` is shared, and is how the objects got ids 0 and 1. A **static method** has no `this`, so it may only use static members; it can be called through a handle or, more clearly, with the class scope operator: `Packet::get_count()`.

METHODS OF A CLASS ARE ALWAYS AUTOMATIC

Tasks and functions declared inside a class have automatic lifetime: every call gets fresh local variables, so methods are re-entrant and can be called from many threads at once. The LRM makes it illegal to give a class method a static lifetime (`function static ...`). Do not confuse this with a **static method** (`static function ...`), which is about belonging to the class instead of an object, not about variable lifetime.

Object lifetime and garbage collection

SystemVerilog has no `delete` and no destructor. An object is reclaimed automatically once no handle anywhere refers to it; in the example, the first Packet became garbage when both `p1` and `p3` were set to `null`. You never free memory explicitly, but you can still leak it: every handle kept in a queue, an associative array, a mailbox or a static variable keeps its object alive. A scoreboard that pushes expected transactions and never pops the matched ones is the classic memory leak of long regressions - the simulation slows down and eventually runs out of memory after hours, not seconds.

WHERE THIS SHOWS UP IN DV

UVM sequence items are created by the thousand (`new` or the factory's `create`), passed by handle through TLM ports, and dropped when the scoreboard is done with them. Because only handles move, passing a large transaction through five components costs nothing - but it also means every component sees the *same* object. A monitor that reuses one object for every observed transaction will corrupt everything its subscribers stored. The rule: construct a new object per transaction, or clone before storing.

18.2 Copying objects: handle copy, shallow copy, deep copy

There are three different things people mean by 'copy', and confusing them is one of the most common testbench bugs:

Syntax	What is copied	Constructor called?
<code>b = a;</code>	The handle only. One object, two names.	No
<code>b = new a;</code>	Shallow copy: a new object whose properties are copied from <code>a</code> . Properties that are handles are copied as handles, so nested objects are shared .	No - <code>new a</code> copies memory; <code>new()</code> is not executed
<code>b = a.clone();</code>	Deep copy: user-written method that creates a new object and recursively copies nested objects.	Yes, explicitly, inside <code>clone()</code>

```

class Header;
    int len;
    function new(int len = 0); this.len = len; endfunction
endclass

class Frame;
    int seq;
    Header hdr; // a handle to another object
    function new(); hdr = new(4); endfunction
    // deep copy: copy scalars AND clone every object we point to
    function Frame clone();
        clone = new();
        clone.seq = seq;
        clone.hdr.len = hdr.len; // new() already built a fresh Header
    endfunction
endclass

module top;
    Frame a, s, d;
    initial begin

```

```

a = new(); a.seq = 1;
s = new a; // SHALLOW copy (built-in, no constructor call)
d = a.clone(); // DEEP copy (user-written)
a.seq = 2; a.hdr.len = 99; // modify the original
$display("orig : seq=%0d hdr.len=%0d", a.seq, a.hdr.len);
$display("shallow: seq=%0d hdr.len=%0d (hdr shared: %0b)",
        s.seq, s.hdr.len, s.hdr == a.hdr);
$display("deep : seq=%0d hdr.len=%0d (hdr shared: %0b)",
        d.seq, d.hdr.len, d.hdr == a.hdr);
$finish;
end
endmodule

```

c18_copy.sv - shallow copy with **new** a versus a hand-written deep copy.

```

orig : seq=2 hdr.len=99
shallow: seq=1 hdr.len=99 (hdr shared: 1)
deep : seq=1 hdr.len=4 (hdr shared: 0)
- top.sv:30: Verilog $finish

```

Verilator 5.020 output.

After the original was modified, the shallow copy still has its own **seq** (scalars are copied) but reports **hdr.len=99** because its **hdr** handle points to the original's Header. The deep copy is fully independent. Note also that **new a** is resolved from **declared** types at compile time, not from the object's run-time type, so it is not polymorphic: shallow-copying through a base-class handle is not a reliable way to duplicate a derived object. That is why real code uses a virtual **copy/clone** method (UVM: **uvm_object::clone()** and the user-overridden **do_copy()**), which works polymorphically.

18.3 Inheritance, super and method overriding

A class can **extend** exactly one parent (base) class. The child (derived) class inherits every property and method, may add new ones, and may **override** inherited methods by redeclaring them with the same name. Inside the child, **super.** refers to the parent's version of an overridden member. Constructors chain: the first statement of a derived **new** must be **super.new(...)**; if you omit it, the compiler inserts **super.new()** with no arguments, which is a compile error when the parent's constructor has arguments without defaults. SystemVerilog-2012 also allows passing the arguments in the declaration: **class Derived extends Base(5);**.

```

class Base;
  int x = 1;
  function new(int x = 1); this.x = x; endfunction
  function string kind(); return "Base"; endfunction // NOT virtual
  virtual function string vkind(); return "Base"; endfunction // virtual
  virtual function void show();
    $display(" %s/%s x=%0d", kind(), vkind(), x);
  endfunction
endclass

class Derived extends Base;
  int y;
  function new(int x = 5, int y = 7);
    super.new(x); // must be the first statement
    this.y = y;
  endfunction
  function string kind(); return "Derived"; endfunction
  virtual function string vkind(); return "Derived"; endfunction
  virtual function void show();
    super.show(); // reuse the parent's version
    $display(" ...plus y=%0d", y);
  endfunction
endclass

```

```

module top;
  Base    b;
  Derived d, d2;
  initial begin
    d = new();
    b = d;                                // upcast: always legal, implicit
    $display("b.kind()=%s  b.vkind()=%s", b.kind(), b.vkind());
    $display("b.show():"); b.show();
    // downcast: must be checked at run time with $cast
    if ($cast(d2, b)) $display("$cast ok, d2.y=%0d", d2.y);
    b = new(3);                            // now b points to a real Base
    if (!$cast(d2, b)) $display("$cast failed: b holds a Base, not a Derived");
    $finish;
  end
endmodule

```

c18_poly.sv - **super.new**, **super.show()**, a non-virtual versus a virtual method, and up/down-casting.

```

b.kind()=Base  b.vkind()=Derived
b.show():
  Base/Derived x=5
  ...plus y=7
$cast ok, d2.y=7
$cast failed: b holds a Base, not a Derived
- top.sv:37: Verilog $finish

```

Verilator 5.020 output.

Virtual methods and polymorphism

The handle **b** has declared type Base but refers to a Derived object. The two calls behave differently, and this is the single most important rule of SystemVerilog OOP:

- For a **non-virtual** method the method is chosen at compile time from the **type of the handle**: **b.kind()** runs Base::kind and prints 'Base'.
- For a **virtual** method it is chosen at run time from the **type of the object**: **b.vkind()** runs Derived::vkind. This is **polymorphism** (dynamic dispatch).
- b.show()** dispatched to Derived::show (virtual), which called **super.show()**. Inside Base::show, the call to **kind()** (non-virtual) bound to Base::kind while **vkind()** (virtual) again reached Derived::vkind - hence 'Base/Derived'.
- Once a method is virtual it stays virtual in every descendant, whether or not the keyword is repeated. An override must match the prototype: same argument names, types, directions and qualifiers, and the same return type (or, for a class return type, a type derived from it).

WHY UVM IS BUILT ON VIRTUAL METHODS

A UVM environment holds components through base-class handles (**uvm_component**, **uvm_driver #(REQ)**) and calls **build_phase**, **run_phase**, **do_copy**, **convert2string** on them. Because those methods are virtual, your overrides run. The **factory** goes one step further: it lets a test replace the *class* that gets constructed (**set_type_override**), and virtual dispatch makes the rest of the environment use the new behaviour without editing a line. Forgetting **virtual** on a method you intend to override is a silent bug: the code compiles and the base version runs.

Upcasting, downcasting and \$cast

Assigning a derived handle to a base handle (**upcast**) is always legal and implicit, because every Derived *is* a Base. The reverse (**downcast**) is legal only if the object really is of the target type (or derived from it), which is known only at run time, so it requires **\$cast**:

Form	On failure	Use when
<code>if (\$cast(dst, src)) ... (function)</code>	Returns 0, <code>dst</code> unchanged, no message	You want to test the type (e.g. 'is this a WriteTxn?')
<code>\$cast(dst, src); (task)</code>	Run-time error ; simulation continues or stops per tool	Failure is a bug; you want to hear about it
<code>dst = src; with base-typed src</code>	Compile error	Never; this is what <code>\$cast</code> is for

`$cast` also converts between enum and integral types with a range check (Chapter 12). With a `null` source, the class form of `$cast` succeeds and assigns `null`. In UVM you will see it in every `do_copy` and `do_compare`: the argument arrives as a `uvm_object` handle and is downcast to the concrete item type.

18.4 Abstract classes, pure virtual methods, extern and visibility

A class declared `virtual class` is **abstract**: it cannot be constructed, only extended. It typically declares **pure virtual** methods - prototypes without a body, `pure virtual function ...;` - that every non-abstract descendant must implement. This is how a framework specifies what a user must provide (UVM's `uvm_object`, `uvm_component`, `uvm_sequence_base` are all virtual classes). Methods may also be declared `extern` in the class and defined later with the class scope operator, `function void Shape::report();`, which keeps long class declarations readable and lets a header-like declaration sit in a package while bodies follow.

```

virtual class Shape;                                // abstract: cannot be constructed
    protected string name;
    function new(string name); this.name = name; endfunction
    pure virtual function real area(); // no body: every child MUST implement
    extern function void report();      // body defined outside the class
endclass

function void Shape::report();                // out-of-block (extern) definition
    $display("%-8s area = %6.2f", name, area());
endfunction

class Rect extends Shape;
    local real w, h;                            // visible only inside Rect
    function new(real w, real h); super.new("rect"); this.w = w; this.h = h; endfunction
    virtual function real area(); return w * h; endfunction
endclass

class Circle extends Shape;
    local real r;
    function new(real r); super.new("circle"); this.r = r; endfunction
    virtual function real area(); return 3.14159 * r * r; endfunction
endclass

module top;
    Shape shapes[$];                            // queue of BASE handles
    real total;
    initial begin
        Rect r;
        Circle c;
        r = new(2.0, 3.5);
        c = new(1.0);
        shapes.push_back(r);
        shapes.push_back(c);
        foreach (shapes[i]) shapes[i].report(); // dynamic dispatch per object
        total = 0;
        foreach (shapes[i]) total += shapes[i].area();
        $display("total area = %6.2f", total);
    $finish;
endmodule

```

```
end
endmodule
```

c18_abstract.sv - an abstract base with a pure virtual method, an extern method, protected and local members, and a queue of base handles dispatched polymorphically.

```
rect    area = 7.00
circle  area = 3.14
total   area = 10.14
- top.sv:38: Verilog $finish
```

Verilator 5.020 output.

The base method `report()` calls `area()`, which does not exist in `Shape`; at run time the call reaches `Rect::area` or `Circle::area`. The loop over `shapes` knows nothing about rectangles or circles - add a `Triangle` class and the loop does not change. That is the payoff of polymorphism for a testbench: a scoreboard iterates over `uvm_sequence_item` handles; a sequencer arbitrates between sequences it knows only by base type.

Qualifier	Visible in the class	Visible in derived classes	Visible outside
(none) = public	Yes	Yes	Yes
protected	Yes	Yes	No
local	Yes	No	No

Member visibility. **const** (read-only after construction) and **static** combine freely with these qualifiers.

Violations are compile-time errors. Here is what Verilator reports for the two classic mistakes, constructing an abstract class and reading a **local** property from outside (two separate runs, one error each):

```
virtual class Shape;
    pure virtual function real area();
endclass
class Rect extends Shape;
    local real w = 2.0;
    virtual function real area(); return w * w; endfunction
endclass
module top;
    initial begin
        Shape s;
        Rect r;
        r = new();
        s = new();           // ERROR 1: abstract class
        $display("%f", r.w); // ERROR 2: local member
    end
endmodule
```

c18_bad.sv - two deliberate errors.

```
%Error: top.sv:13:9: Illegal to call 'new' using an abstract virtual class 'Shape' (IEEE 1800-2017 8.21)
%Error-ENCAPSULATED: top.sv:14:22: 'w' is hidden as 'local' within this context (IEEE 1800-2017 8.18)
```

verilator --lint-only, first error line of each run.

LOCAL IS PER CLASS, NOT PER OBJECT

Access control is checked against the *class* in which the code is written, not the object. A method of `Rect` may read `other.w` of another `Rect` object even though `w` is local. That is what makes `compare(Rect other)` and `copy(Rect rhs)` implementable without public getters. Interview variant: 'Can a derived class read a local member of its parent?' - No, only protected ones; it can, however, call a public or protected parent method that reads it.

CONSTRUCTORS ARE NEVER VIRTUAL - AND VIRTUAL CALLS INSIDE NEW()

`new` cannot be declared virtual: an object's type is fixed by which class's `new` is called. A virtual method called from inside a base-class constructor does dispatch to the derived override, but the derived part of the object has not been initialized yet (its property initializers and constructor body run after `super.new()` returns). Keep constructors trivial; this is exactly why UVM moves construction of sub-components into `build_phase`.

18.5 Parameterized classes

Classes take parameters like modules: **value parameters** (`int DEPTH = 4`) and **type parameters** (`type T = int`). A parameterized class is a *generic class*; each distinct set of actual parameters creates a **specialization**, which is a separate type. Two specializations are assignment-incompatible even if they look alike, and - a favourite interview question - each specialization has its **own copy of the static members**.

```
// A generic bounded stack: T is a TYPE parameter, DEPTH a VALUE parameter
class Stack #(type T = int, int DEPTH = 4);
    static int n_stacks = 0;          // one copy PER SPECIALIZATION
    local T items[$];
    function new(); n_stacks++; endfunction
    function bit push(T v);
        if (items.size() == DEPTH) return 0;    // full
        items.push_back(v); return 1;
    endfunction
    function T pop(); return items.pop_back(); endfunction
    function int size(); return items.size(); endfunction
endclass

typedef Stack#(string, 2) StrStack;    // name a specialization

module top;
    Stack #()      si1, si2;          // defaults: Stack#(int,4)
    Stack #(int, 4) si3;              // SAME specialization as #()
    StrStack       ss;
    initial begin
        si1 = new(); si2 = new(); si3 = new(); ss = new();
        void'(ss.push("a")); void'(ss.push("b"));
        $display("push to full StrStack returns %0d", ss.push("c"));
        $display("pop -> %s, size now %0d", ss.pop(), ss.size());
        $display("Stack#(int,4)::n_stacks = %0d", Stack#(int,4)::n_stacks);
        $display("StrStack::n_stacks      = %0d", StrStack::n_stacks);
        $finish;
    end
endmodule
```

c18_param.sv - a generic stack with a type and a value parameter; static members per specialization.

```
push to full StrStack returns 0
pop -> b, size now 1
Stack#(int,4)::n_stacks = 3
StrStack::n_stacks      = 1
- top.sv:27: Verilog $finish
```

Verilator 5.020 output.

`Stack #()`, `Stack #(int, 4)` and the default parameter values name the same specialization, so three objects were counted there, while the string specialization counted one. A `typedef` gives a specialization a short name - essential when the same specialized type appears in many places (UVM: `typedef uvm_sequencer #(bus_item) bus_sequencer;`). Outside the class, reach static members and typedefs of a specialization with `Stack#(int,4)::name`; the bare `Stack::name` refers to the generic class and is legal only inside the class itself or in its out-of-block (extern) method definitions.

Use of ::	Example
Static property / method	<code>Packet::count, Config::get()</code>
Typedef or enum inside a class	<code>bus_item::kind_e</code>
Parameter of a specialization	<code>Stack#(int,4)::DEPTH</code>
Out-of-block method body	<code>function void Shape::report();</code>
Package item (Chapter 15)	<code>uvm_pkg::uvm_report_info</code>
Parent version (inside class)	<code>super.new(), super.show()</code> - uses <code>.</code> not <code>::</code>

The class scope resolution operator.

TYPE PARAMETERS ARE HOW UVM STAYS TYPE-SAFE

`uvm_driver #(REQ, RSP)`, `uvm_sequencer #(REQ)`, `uvm_analysis_port #(T)`, `uvm_tlm_fifo #(T)` and `uvm_config_db #(T)` are all parameterized classes. Getting the parameter wrong (a driver of `bus_item` connected to a sequencer of `base_item`) is a compile-time type error, not a run-time surprise - which is the whole point. And because statics are per specialization, `uvm_config_db#(int)` and `uvm_config_db#(virtual bus_if)` are separate databases: a `set` with one type is invisible to a `get` with another.

18.6 Forward declarations with typedef class

A class can only be used after it is declared. Two classes that refer to each other - a driver and its sequencer, a register and its block, a producer and a consumer - need a **forward declaration**: `typedef class Name;` announces that a class of that name will be defined later in the same scope.

```
typedef class Consumer;           // forward declaration: name used before defined

class Producer;
    Consumer peer;               // legal thanks to the typedef above
    function void send(int v); peer.take(v); endfunction
endclass

class Consumer;
    Producer peer;
    int total;
    function void take(int v); total += v; endfunction
endclass

module top;
    initial begin
        Producer p;
        Consumer c;
        p = new();
        c = new();
        p.peer = c; c.peer = p;   // two objects that reference each other
        p.send(3); p.send(4);
        $display("consumer total = %0d", c.total);
        $finish;
    end
endmodule
```

c18_fwd.sv - mutually referencing classes.

```
consumer total = 7
- top.sv:23: Verilog $finish
```

Verilator 5.020 output.

Circular references like this one are also why garbage collection is not simple reference counting: the two objects refer to each other, and are still reclaimed once nothing *outside* them does. Forward typedefs are also

used for parameterized classes (`typedef class Stack; then Stack #(int) s; later`).

18.7 Interface classes

Single inheritance cannot express 'this transaction is both printable and comparable', and deep hierarchies that try to make one base class do everything become brittle. SystemVerilog-2012 added **interface classes** (unrelated to the `interface` construct of Chapter 16): a declaration of a set of pure virtual methods with no data. A class **implements** any number of them in addition to extending at most one class.

```
interface class Printable;           // only pure virtual methods (+ params, typedefs)
    pure virtual function string to_string();
endclass

interface class Comparable #(type T = int);
    pure virtual function int compare(T other);    // <0, 0, >0
endclass

// a class may implement ANY number of interface classes
class BusTxn implements Printable, Comparable#(BusTxn);
    int id, addr;
    function new(int id, int addr); this.id = id; this.addr = addr; endfunction
    virtual function string to_string();
        return $sprintf("BusTxn(id=%0d, addr=%0h)", id, addr);
    endfunction
    virtual function int compare(BusTxn other); return addr - other.addr; endfunction
endclass

function automatic void log_all(Printable items[$]);    // works for ANY Printable
    foreach (items[i]) $display(" %s", items[i].to_string());
endfunction

module top;
    initial begin
        BusTxn a, b;
        Printable q[$];
        a = new(1, 'h40);
        b = new(2, 'h10);
        q.push_back(a); q.push_back(b);    // implicit upcast to the interface type
        log_all(q);
        $display("a.compare(b) = %0d", a.compare(b));
        $finish;
    end
endmodule
```

c18_iface.sv - two interface classes (one parameterized) implemented by one class, used through an interface-class handle.

```
BusTxn(id=1, addr=40)
BusTxn(id=2, addr=10)
a.compare(b) = 48
- top.sv:32: Verilog $finish
```

Verilator 5.020 output.

- An interface class may contain only pure virtual methods, typedefs and parameters - no properties, no constraints, no method bodies, no constructor.
- An interface class can **extends** one or more other interface classes (multiple inheritance of **interface**, never of implementation, so the 'diamond problem' of C++ cannot occur).
- A class that **implements** an interface class must implement every method with a matching virtual prototype, unless the class is a **virtual class**, which may leave them as pure virtual for its children.
- A handle of an interface-class type can refer to any object whose class implements it, and **\$cast** works between an interface class and any class that implements it.

TOOL PORTABILITY: COMBINING EXTENDS AND IMPLEMENTS

`class BusTxn extends Txn implements Printable, Comparable#(BusTxn);` is perfectly legal IEEE 1800-2017. In this environment, Verilator 5.020 compiled that form but failed in its C++ build when a `BusTxn` handle was assigned to a `Printable` handle, and a `$cast` workaround failed at run time; removing the `extends Txn` (as in the listing above) made it work. Interface classes are newer than the rest of SV OOP, and support is still uneven across simulators and linters - test your pattern on every tool in your flow before building a library on it.

18.8 Design patterns you will meet: the singleton

Some objects must exist exactly once: a global configuration, a report server, a factory. The **singleton** pattern hides the constructor (`local function new`) and offers a static `get()` that constructs the single instance on first use.

```
class Config;
    local static Config m_inst;          // the one and only instance
    int verbosity = 1;
    local function new(); $display(" Config constructed"); endfunction // no outside new
    static function Config get();
        if (m_inst == null) m_inst = new();    // lazy construction on first use
        return m_inst;
    endfunction
endclass

module top;
    initial begin
        Config c1, c2;
        c1 = Config::get();
        c1.verbosity = 3;
        c2 = Config::get();                // same object, no second construction
        $display("same object: %0b, c2.verbosity=%0d", c1 == c2, c2.verbosity);
        $finish;
    end
endmodule
```

c18_single.sv - a lazily constructed singleton.

```
Config constructed
same object: 1, c2.verbosity=3
- top.sv:18: Verilog $finish
```

Verilator 5.020 output.

UVM is full of singletons: `uvm_root` (the implicit top of the component tree, reached with `uvm_root::get()`), `uvm_factory`, the report server and the resource pool behind `uvm_config_db`. Other patterns you will recognize later: the **factory** (construct by type name, override per test), the **observer** (analysis ports broadcasting to subscribers), the **strategy** (a driver delegating to a swappable protocol object) and **callbacks** (user hooks registered at run time).

18.9 Summary

- A class is a type; `new` creates objects at run time; handles refer to objects and default to `null`. Assignment copies handles, not objects.
- Objects are garbage collected when unreachable; memory leaks come from handles kept in containers.
- Static members are shared per class (per **specialization** for parameterized classes); static methods have no `this`. Class methods always have automatic lifetime.
- `new a` is a shallow copy that does not run the constructor and shares nested objects; write a virtual clone/copy for deep copies.
- Non-virtual methods bind by handle type at compile time; virtual methods bind by object type at run time. Once virtual, always virtual. Constructors cannot be virtual.

- Upcasts are implicit; downcasts need `$cast`, whose function form returns 0 on failure.
- `virtual class` + `pure virtual` define abstract frameworks; `extern` separates declaration from definition; `local` and `protected` restrict visibility (per class, not per object).
- `typedef class` forward-declares; interface classes provide multiple inheritance of interface via `implements`.
- These rules are the vocabulary of UVM: base-class handles, virtual phase methods, parameterized TLM classes, singletons and the factory.

18.10 Exercises

- 1 Write a class `Counter` with a static `total` and a per-object `count`, and a method `incr()` that updates both. Create three objects, call `incr()` a different number of times on each, and print all values. Then explain what changes if `total` is not static.
- 2 Extend `c18_copy.sv` with a queue of Header handles inside Frame. Show that `new a` shares the queue contents' objects, and write `clone()` so that it does not.
- 3 Remove the `virtual` keyword from `vkind` in `c18_poly.sv` and predict every output line before running it. Then make `show` non-virtual in Base only and predict again.
- 4 Implement `virtual function bit compare(Shape other)` in Shape and override it in Rect using `$cast` so that comparing a Rect with a Circle returns 0 instead of crashing.
- 5 Create a parameterized class `Pool #(type T = int)` with a static method `get_count()`. Prove that `Pool#(int)` and `Pool#(byte)` count independently. Why does this matter for `uvm_config_db#(T)`?
- 6 Interview drill: explain, without code, why a monitor that calls `new` once and then reuses that object for every observed bus transfer produces a scoreboard in which all stored transactions look identical.

19. Constrained Random Stimulus

A directed test checks the scenario its author thought of. The bugs that escape to silicon live in the scenarios nobody thought of: a burst that crosses a 4 KB boundary while the FIFO is one entry from full, during a clock-gating request. **Constrained random verification** (CRV) attacks that space statistically: you describe what a *legal* stimulus is, with declarative constraints, and a **constraint solver** built into the simulator picks random values that satisfy all of them. Combined with self-checking and functional coverage (Chapter 23), this is the coverage-driven methodology used on essentially every commercial SoC.

WHICH TOOLS IN THIS BOOK CAN RUN THIS CHAPTER

Verilator 5.020 accepts the full constraint syntax used here but **ignores the constraints** - `randomize()` fills every `rand` variable with unconstrained random bits (a constraint `a inside {[10:20]}` produced 'hc4 in our test). Icarus Verilog does not support `rand` at all. The constraint listings in this chapter therefore need a commercial simulator (Questa, VCS, Xcelium, Riviera-PRO) and are shown **without** output cards; the expected behaviour is stated in the text. The parts that do not depend on the solver - `$urandom`, seeding, the `randomize` hooks and the return value - were run with Verilator.

19.1 rand, randc and randomize()

Declaring a class property `rand` makes it a **random variable**: each call of the object's built-in `randomize()` method assigns it a new value that satisfies every active constraint. `randc` is **random-cyclic**: the variable walks through a random permutation of all its possible values before any value repeats (a 2-bit `randc` returns, e.g., 2, 0, 3, 1, then a new permutation). Only integral types can be random: bit/logic vectors, integer types, enums (which take only their named values), and arrays and queues of those - whose **size** is also random unless constrained. `real` variables cannot be `rand` in IEEE 1800-2017 (IEEE 1800-2023 adds random `real` support, not yet widely implemented). A handle declared `rand` makes `randomize()` descend into the object it points to, which must not be null.

Property	rand	randc
Distribution	Uniform over the solution space (unless shaped by dist / solve-before)	Permutation: every value once per cycle
Solved	Together with all other rand variables	Before the rand variables; its value is then a fixed input for them
Width	Any	Tools may limit the width (keep it small, e.g. <= 16 bits)
Typical use	Addresses, data, lengths, modes	Tags/IDs, register indices, 'visit every opcode' tests

`randomize()` is a virtual function that returns **1 on success** and **0 when the constraints cannot be satisfied**. On failure the random variables keep their previous values and the simulator usually prints only a warning, so a test that ignores the return value silently keeps driving stale stimulus. Always check it:

```
// correct: the check cannot be switched off
if (!tr.randomize()) `uvm_fatal("RAND", "tr.randomize() failed")
if (!tr.randomize()) $fatal(1, "randomize failed"); // plain SV

// WRONG: randomize() is called INSIDE the assertion
assert (tr.randomize());
```

Checking the result of `randomize()`.

NEVER CALL RANDOMIZE() INSIDE ASSERT()

An immediate assertion's expression is not evaluated at all when assertions are disabled (`$assertoff`, a `-nosva`-style switch, or a regression that turns assertions off for speed). `assert(tr.randomize());` then **never randomizes** - every transaction keeps its last value and the test still 'passes'. Put the call in an `if`, as above.

19.2 Constraint blocks

A **constraint block** is a named set of Boolean expressions over random (and non-random) variables. Constraints are **declarative and bidirectional**: they are relations the solver must make true, not assignments executed in order. `len == 4` in a constraint does not 'set' `len`; it restricts the solution space, and the solver may satisfy `a + b == 10` by choosing `b` first. All active constraints of the object, including those inherited and those of nested `rand` objects, are solved **simultaneously**.

```
typedef enum bit [1:0] {READ, WRITE, IDLE} kind_e;

class BusTxn;
  rand kind_e    kind;
  rand bit [31:0] addr;
  rand bit [7:0] len;           // burst length in beats
  randc bit [3:0] tag;          // cycles through all 16 values before repeating
  rand bit [31:0] data[];       // dynamic array: its SIZE is random too

  constraint c_addr { addr inside {[32'h0000_0000 : 32'h0000_FFFF], // RAM window
                                [32'h4000_0000 : 32'h4000_00FF]}; // CSR window
                    addr[1:0] == 2'b00; }                          // word aligned
  constraint c_len  { len inside {1, 2, 4, 8, 16}; }
  constraint c_data { data.size() == len;                          // size tracks len
                    foreach (data[i]) data[i] != 0; }
  constraint c_csr  { addr >= 32'h4000_0000 -> len == 1; }          // CSRs: single beat
  constraint c_idle { if (kind == IDLE) { len == 1; } else { len >= 1; } }
endclass
```

k19_basic.sv - set membership, implication, if-else, foreach and array-size constraints (needs a commercial simulator).

What a constraint-solving simulator does with this class:

- **inside** restricts `addr` to the union of two ranges; `[lo:hi]` is inclusive and ranges and single values may be mixed. Every legal value is equally likely, so the 64 KB RAM window is hit about 256 times more often than the 256-byte CSR window - a distribution effect you will usually want to reshape with `dist`.
- **Implication P -> Q** means 'if P then Q' (it is equivalent to `!P || Q`): every CSR address forces `len == 1`, and because constraints are bidirectional, `len != 1` also forbids CSR addresses.
- **if-else** is the block form of implication and can hold several constraints per branch.
- **Array constraints**: `data.size() == len` sizes the dynamic array (the size is solved before the elements), and `foreach` applies a constraint to every element. Sum, product, and, or and xor reductions are also allowed: `data.sum() with (int'(item)) < 1000`.
- **randc tag** is solved first and cycles through all 16 values.
- Every `rand` variable not mentioned in any constraint is simply uniform over its type.

For comparison, here is what Verilator reports when it meets a constraint block - it is a warning, not an error, and the constraint is dropped:

```
%Warning-CONSTRAINTIGN: top.sv:4:14: Constraint ignored (unsupported)
  4 |   constraint c_a { a dist { 0 := 40, [1:3] := 20, [4:15] := 20 }; }
%Warning-CONSTRAINTIGN: top.sv:8:14: Constraint ignored (unsupported)
```

verilator --lint-only -Wall on k19_dist.sv (excerpt). A reminder to check what your tool supports before trusting a random test.

WIDTH AND SIGN IN CONSTRAINTS

Constraint expressions follow the normal SystemVerilog expression rules of Chapter 4 - including silent truncation. With `rand bit [7:0] a, b;` the constraint `a + b == 300` is evaluated in the width of its largest operand, the 32-bit literal 300, so it works; but `a + b < 8'd50` is evaluated in 8 bits and wraps, so `a=200, b=100` (sum 300 mod 256 = 44) satisfies it. The same trap hits `data.sum() < 1000` on a byte array: the sum is computed in the element width. Cast explicitly: `data.sum() with (int'(item)) < 1000`. Mixing signed and unsigned operands makes the comparison unsigned, so `x > -1` with an unsigned `x` is always false.

19.3 Shaping the distribution: dist

Uniform random is rarely what you want: corner cases such as address 0, the maximum length or back-to-back errors need to be hit far more often than their share of the value space. A `dist` constraint assigns **weights**. Two operators distinguish how a weight applies to a range:

```
class DistDemo;
  rand bit [3:0] a, b;
  // := weight applies to EACH value in the item; := divides the weight across the range
  constraint c_a { a dist { 0 := 40, [1:3] := 20, [4:15] := 20 }; }
  // total weight = 40 + 3*20 + 20 = 120
  // P(a==0) = 40/120 = 1/3      P(a==1) = P(a==2) = P(a==3) = 20/120 = 1/6
  // P(a==k), k in 4..15 = (20/12)/120 = 1/72      (the range SHARES 20)
  constraint c_b { b dist { [0:7] := 1, [8:15] := 3 }; } // upper half 3x more likely
endclass
```

k19_dist.sv - := (weight per value) versus := (weight shared by the range). The comments give the exact probabilities.

Item	Operator	Weight of each value	Probability
0	:= 40	40	40/120 = 33.3%
[1:3]	:= 20	20 (each of 3 values)	3 x 20/120 = 50%
[4:15]	:= 20	20/12 = 1.67 (shared by 12 values)	20/120 = 16.7% in total

How the weights of c_a add up.

- A `dist` is still a hard constraint on **membership**: the value must be one of the listed values. If other constraints exclude some of them, the weights apply among the values that remain. A weight of 0 excludes a value.
- `dist` cannot be applied to a `randc` variable.
- Distributions are a property of **many** calls, not one; with other constraints active, the observed frequencies can differ noticeably from the nominal weights.

19.4 Solution order and solve...before

The solver picks uniformly among **complete solutions**, not variable by variable. That has a surprising consequence for implications, captured in the classic example below.

```

class SolveDemo;
    rand bit      mode;                // 1 = "long" mode
    rand bit [7:0] len;
    constraint c_impl { mode -> len == 0; } // only ONE of 257 solutions has mode==1
    constraint c_order { solve mode before len; } // pick mode FIRST: P(mode==1) = 1/2
endclass

class Arrays;
    rand bit [7:0] q[$];
    rand bit [7:0] ids[4];
    constraint c_size { q.size() inside {[2:5]}; }
    constraint c_sum { q.sum() with (int'(item)) < 200; } // widen before summing!
    constraint c_sort { foreach (q[i]) if (i > 0) q[i] > q[i-1]; } // strictly ascending
    constraint c_uniq { unique {ids}; } // all four differ
endclass

```

k19_solve.sv - solve-before, and array constraints: size, sum, ordering and unique (needs a commercial simulator).

Without `c_order`, the legal (mode, len) pairs are 256 pairs with mode=0 plus **one** pair (1, 0). Each is equally likely, so $P(\text{mode}==1) = 1/257$: the 'long mode' is almost never tested. `solve mode before len` tells the solver to choose `mode` first (from the values for which *some* solution exists), then `len`: $P(\text{mode}==1)$ becomes 1/2. **`solve...before` never changes which solutions are legal** - only their probabilities. It cannot involve `randc` variables (they are always solved first anyway), and circular orderings are an error.

The `Arrays` class shows the array toolbox: `q.size()` bounds the queue length, `sum() with` constrains a reduction (widened to `int` to avoid the wrap-around described above), the `foreach` with `i > 0` guard produces a strictly ascending list (the guard keeps `q[i-1]` in bounds), and `unique {ids}` makes all elements of the array pairwise different - a one-line replacement for an $O(n^2)$ foreach over pairs.

19.5 Controlling constraints: soft, override, static, constraint_mode, rand_mode

```

class Cfg;
    rand bit [7:0] burst;
    rand bit [3:0] prio;
    constraint c_def { soft burst == 4; } // default, may be overridden
    constraint c_prio { prio inside {[0:7]}; }
    static constraint c_global { prio != 5; } // static: mode shared by all objects
endclass

class Stress extends Cfg;
    constraint c_def { burst inside {[16:64]}; } // SAME name: overrides the parent block
endclass

module top;
    initial begin
        Cfg c;
        int unsigned max_burst;
        c = new();
        max_burst = 8;
        // inline constraint beats the soft default; name resolution inside with{}:
        // unqualified names are looked up in the OBJECT first; local:: forces the caller
        if (!c.randomize() with { burst == local::max_burst; prio < 3; })
            $error("randomize failed");
        c.c_prio.constraint_mode(0); // switch a block off
        c.prio.rand_mode(0); // prio becomes a state variable
        void'(c.randomize()); // only burst is randomized now
        void'(std::randomize(max_burst) with { max_burst inside {[1:16]}; });
    end
endmodule

```


k19_soft.sv - soft defaults, overriding a block by name, static constraints, inline constraints with local::, constraint_mode, rand_mode and std::randomize (needs a commercial simulator).

Mechanism	Effect
<code>soft expr;</code>	A default that is dropped (not an error) when it conflicts with a hard constraint. Among conflicting soft constraints, later / higher-priority ones win (inline beats class, derived beats base).
Same-named block in a child class	Replaces the parent's block entirely (<code>Stress::c_def</code> removes the soft burst == 4 and requires 16..64). This is how tests reshape stimulus without editing the transaction class.
<code>static constraint</code>	<code>constraint_mode()</code> on it affects every object of the class, not just one.
<code>h.blk.constraint_mode(0/1)</code>	Turns a block off/on for this object; <code>h.constraint_mode(0)</code> turns all blocks off. <code>h.blk.constraint_mode()</code> returns the current state.
<code>h.var.rand_mode(0/1)</code>	Makes a rand variable a state variable (keeps its value, still usable in constraints) or random again. <code>h.rand_mode(0)</code> affects all rand variables.
<code>h.randomize(v1, v2)</code>	Randomizes only the listed variables; all other rand variables act as state variables for this call.
<code>h.randomize(null)</code>	Randomizes nothing; only checks whether the current values satisfy the constraints (returns 1/0).

The **inline constraint** `randomize() with { ... }` adds constraints for one call only; it is how a sequence says 'give me a random write to this address'. Name resolution inside the braces is the source of a well-known bug: unqualified names are looked up **in the object being randomized first**, and only then in the calling scope. If the class has a property called `max_burst`, `burst == max_burst` compares with the object's own property, not with your local variable. `local::max_burst` forces the caller's scope (IEEE 1800-2012 and later); `this.` or no prefix means the object. In UVM, the constraint blocks passed to macros such as `uvm_do_with` follow the same rule.

`std::randomize(vars) with {...}` randomizes ordinary variables (module, function or class scope) without a class: useful for a quick random delay or a local choice. The `std::` prefix is optional unless a `randomize` in scope would hide it.

19.6 pre_randomize() and post_randomize()

Every class has two empty void functions that `randomize()` calls automatically: `pre_randomize()` before solving and `post_randomize()` after a **successful** solve (not after a failure). Override them to prepare state (switch constraint modes, size arrays, record statistics) and to compute derived fields that should not be solved - CRCs, parity, ECC, packed encodings - which is also much faster than expressing them as constraints. They are called for nested rand objects too, and a child override should call `super.pre_randomize()` / `super.post_randomize()` if the parent's version does work.

```
class Pkt;
  rand bit [7:0] len;
  rand bit [31:0] payload;
  bit [7:0] crc; // NOT rand: derived, never randomized
  int n_pre, n_post;
  function void pre_randomize(); // called BEFORE the solver
    n_pre++;
  endfunction
  function void post_randomize(); // called AFTER a SUCCESSFUL solve
    n_post++;
    crc = payload[31:24] ^ payload[23:16] ^ payload[15:8] ^ payload[7:0];
  endfunction
endclass

module top;
  initial begin
```

```

Pkt p;
int ok;
p = new();
ok = p.randomize();
$display("randomize() returned %0d, pre=%0d post=%0d, payload=%h crc=%h",
        ok, p.n_pre, p.n_post, p.payload, p.crc);
repeat (3) begin
    if (!p.randomize()) $fatal(1, "randomize() failed"); // ALWAYS check
    $display("len=%3d payload=%h crc=%h", p.len, p.payload, p.crc);
end
$display("pre_randomize calls=%0d, post_randomize calls=%0d", p.n_pre, p.n_post);
$finish;
end
endmodule

```

c19_hooks.sv - a derived CRC computed in post_randomize (no constraints, so it runs on Verilator).

```

randomize() returned 1, pre=1 post=1, payload=0b1e5d9c crc=d4
len= 2 payload=48c2e0e4 crc=8e
len=120 payload=bdd58ebc crc=5c
len=182 payload=2a14ffe4 crc=25
pre_randomize calls=4, post_randomize calls=4
- top.sv:28: Verilog $finish

```

Verilator 5.020 output (random values depend on the seed).

Check the arithmetic: $0b \wedge 1e \wedge 5d \wedge 9c = d4$, so the CRC was computed from the freshly randomized payload, once per call. One honest note on tool support: when this example also exercised `p.len.rand_mode(0)`, Verilator 5.020 still reported `rand_mode()` as 1 and kept randomizing `len`, so `rand_mode` is another feature to test only on a full-featured simulator.

19.7 System random functions and seeding

Function	Returns	Notes
<code>\$urandom</code> / <code>\$urandom(seed)</code>	32-bit unsigned	Uses the calling thread's RNG (thread stability); the seed form reseeds that call
<code>\$urandom_range(max, min=0)</code>	Unsigned in [min, max]	Inclusive; arguments may be given in either order
<code>\$random</code> / <code>\$random(seed_var)</code>	32-bit signed	Verilog-2001 legacy: one global generator, <code>inout</code> seed variable; <code>\$random % 10</code> can be negative
<code>\$dist_uniform, \$dist_normal, ...</code>	Integer	Verilog distribution functions; rarely used in modern DV
<code>obj.srandom(seed),</code> <code>process::self().srandom(seed)</code>	-	Seed one object's / one thread's RNG
<code>get_randstate()</code> / <code>set_randstate(s)</code>	string / -	Save and restore an RNG state (replay one object's stream)

```

module top;
    int unsigned a[6];
    initial begin
        foreach (a[i]) a[i] = $urandom_range(9, 0); // inclusive; args may be swapped
        $display("$urandom_range(9,0): %p", a);
        foreach (a[i]) a[i] = $urandom_range(3); // one argument: min defaults to 0
        $display("$urandom_range(3) : %p", a);
        $display("$urandom          : %0d (32-bit unsigned)", $urandom);
        $display("$random          : %0d (32-bit signed)", $random);
        $finish;
    end
endmodule

```

c19_urand.sv

```

$ ./obj_dir/Vtop
$urandom_range(9,0): '{h6, h4, h2, h2, h2, h2}'
$urandom_range(3) : '{h2, h0, h3, h3, h1, h1}'
$ ./obj_dir/Vtop
$urandom_range(9,0): '{h6, h4, h2, h2, h2, h2}'
$urandom_range(3) : '{h2, h0, h3, h3, h1, h1}'
$ ./obj_dir/Vtop +verilator+seed+1
$urandom_range(9,0): '{h0, h9, h9, h1, h6, h6}'
$urandom_range(3) : '{h0, h3, h0, h0, h2, h1}'
$ ./obj_dir/Vtop +verilator+seed+2
$urandom_range(9,0): '{h6, h3, h7, h4, h1, h4}'
$urandom_range(3) : '{h1, h3, h1, h3, h1, h2}'

```

Verilator 5.020: the same binary run with the default seed (twice), and with +verilator+seed+1 and +2.

The first two runs are identical: a random simulation is fully **reproducible** from its seed, which is what makes it debuggable. Every simulator takes the seed on the command line - Questa `-sv_seed N` (or `random`), VCS `+ntb_random_seed=N`, Xcelium `-svseed N`, Verilator `+verilator+seed+N` - and a regression system runs each test with many seeds and records the seed of every failure. In Verilator 5.020 we also observed that `process::self().srandom(42)` did not reset the `$urandom` sequence of the thread, contrary to the LRM, so seed through the command line when using it.

Random stability

IEEE 1800 defines **random stability**: every thread and every object has its own random number generator (RNG), seeded from its parent's RNG when the thread or object is created. Consequences:

- **Thread stability**: the values a thread gets from `$urandom` do not depend on what other threads do, as long as threads are created in the same order.
- **Object stability**: `obj.randomize()` uses the object's own RNG, so adding a `randomize()` call on some other object does not change this object's values.
- Stability is **hierarchical**, not absolute: inserting a new `fork` or constructing an extra object *before* existing ones changes the seeds handed out afterwards. A small testbench edit can therefore make a failing seed stop failing. Keep the failing seed's full environment (source revision + seed) to reproduce bugs, and add new components after existing ones where possible.
- `$random` is **not** stable: it has a single global stream shared by everyone; prefer `$urandom` and class randomization.

19.8 When the solver fails: common causes and debugging

Symptom	Typical cause	Fix
<code>randomize()</code> returns 0 every time	Two constraints contradict (e.g. inherited <code>len <= 8</code> and a test's <code>len == 16</code>)	Bisect with <code>constraint_mode(0)</code> on blocks; override the block by name instead of adding a conflicting one
Fails only with an inline <code>with{}</code>	Name captured from the object instead of the caller	Use <code>local::</code> for caller variables
Value outside the intended range	Width/sign wrap-around in the expression	Widen with casts, compare with sized literals
A mode is (almost) never generated	Implication skews the solution count	<code>solve a before b</code> or a <code>dist</code> on the mode
Null object error in <code>randomize()</code>	<code>rand</code> handle to a nested object that was never constructed	Construct in <code>new()</code>
Very slow <code>randomize()</code>	Large arrays with pairwise foreach, multiplication/division/modulo on wide random variables	Use <code>unique</code> , move derived values to <code>post_randomize</code> , reduce widths, split into stages

Symptom	Typical cause	Fix
Different result on another simulator	Solver implementations differ; the LRM does not define the exact sequence	Never compare random streams across tools; compare coverage

Commercial simulators print a report that names the smallest set of conflicting constraints when a solve fails (Questa's `-solvefaildebug`, and equivalents in VCS and Xcelium) - learn the option for your tool on day one. Without it, the bisection method works everywhere: call `randomize()` in a loop while disabling constraint blocks one at a time until it succeeds.

INTERVIEW FAVOURITES

(1) 'Constrain a 32-bit value to have exactly 4 bits set' - `$countones(v) == 4`. (2) 'Generate 10 unique values without unique' - `foreach (a[i]) foreach (a[j]) if (i < j) a[i] != a[j];` (then mention `unique` and `randc`). (3) 'Why does my 1-in-257 mode never appear?' - solution-space uniformity and `solve before`. (4) 'Difference between `rand_mode(0)` and removing `rand`?' - the former is dynamic per object and the variable still participates in constraints as a state variable. (5) 'What happens to `rand` variables when `randomize()` fails?' - they keep their old values and `post_randomize()` is not called.

19.9 Summary

- `rand` variables are solved together, uniformly over all solutions; `randc` cycles through permutations and is solved first.
- `randomize()` returns 1/0; check it with `if`, never inside `assert`.
- Constraints are declarative, bidirectional Boolean relations: `inside`, `dist`, `->`, `if-else`, `foreach`, `size()`, `reductions`, `unique`, `soft`.
- `:=` weights each value, `:/` shares the weight across a range; `solve before` changes probabilities, never legality.
- Tests reshape stimulus by overriding blocks by name, inline `with {}` (with `local::` for caller names), `constraint_mode` and `rand_mode`.
- `pre_randomize` / `post_randomize` prepare state and compute derived fields; `post` runs only on success.
- Random stimulus is reproducible from the seed; thread and object stability limit, but do not eliminate, seed perturbation from testbench edits.
- Verilator 5.020 ignores constraints: use a commercial simulator for constrained random.

19.10 Exercises

- Write a class for an AXI-like burst with `addr`, `len` (1-256) and `size` (1, 2, 4, 8 bytes) such that the burst never crosses a 4 KB boundary, `addr` is aligned to `size`, and bursts of length 1 appear 30% of the time.
- For `rand bit [2:0] x; rand bit y; constraint c { y -> x == 0; }` compute $P(y==1)$ with and without `solve y before x`.
- Constrain a queue of 8 bytes to be a permutation of 0..7 in two different ways (with `unique` and without it).
- A derived test class adds `constraint c_len { len == 32; }` but the base class has `constraint c_len_max { len <= 16; }`. What happens, and give two different ways to make the test work.
- Explain why `if (!tr.randomize() with { addr == addr; })` does not constrain anything, and fix it.
- Run `c19_urand.sv` with three seeds on Verilator, then add a second `initial` block that also calls `$urandom` before the first one. Which printed values change, and what does that tell you about Verilator's thread stability?

20. Processes and Inter-Process Communication: fork/join, Events, Semaphores, Mailboxes

A testbench is a concurrent program. The driver waits for the bus to be free while the monitor samples every cycle, the scoreboard waits for results, a watchdog counts down, and the test decides when everything is done. Verilog gave us static processes (`initial`, `always`) and a basic `fork...join`. SystemVerilog adds dynamic process creation (`join_any`, `join_none`), process control (`wait fork`, `disable fork`, the `process` class) and three communication primitives - **events**, **semaphores** and **mailboxes** - that are the foundation of UVM's TLM ports, sequencer arbitration and objections.

All processes in a simulation run in one thread of the simulator: 'parallel' means **interleaved at blocking points**, according to the scheduling rules of Chapters 5 and 21. A process runs until it blocks (a delay, an event control, a `wait`, a blocking `get`), and the order among processes that are ready in the same region is **not defined** by the LRM. Code whose result depends on that order is a race. Examples in this chapter were run with Verilator 5.020 (`-timing`) unless the caption says Icarus Verilog.

20.1 fork...join, join_any and join_none

Each statement directly inside `fork ... join` becomes a separate child process (wrap several statements in `begin...end` to make one child). The three closing keywords differ only in **when the parent continues**:

Closing keyword	Parent continues	Children afterwards
<code>join</code>	When all children have finished	All done
<code>join_any</code>	When any one child has finished	The others keep running
<code>join_none</code>	Immediately, without blocking	All keep running; they start when the parent next blocks (or ends)

```
`timescale 1ns/1ns
module top;
  task automatic job(string name, int d);
    #d $display("[%0t] %s done (took %0d)", $time, name, d);
  endtask
  initial begin
    $display("-- join: wait for ALL children");
    fork job("A", 10); job("B", 30); job("C", 20); join
    $display("[%0t] after join", $time);

    $display("-- join_any: wait for the FIRST child; others keep running");
    fork job("D", 10); job("E", 30); join_any
    $display("[%0t] after join_any", $time);

    $display("-- join_none: do not wait at all");
    fork job("F", 5); join_none
    $display("[%0t] after join_none", $time);
    wait fork;           // wait for E and F (all children of this thread)
    $display("[%0t] after wait fork", $time);
    $finish;
  end
endmodule
```

c20_fork.sv - the three join variants and wait fork.

```
-- join: wait for ALL children
[10] A done (took 10)
[20] C done (took 20)
[30] B done (took 30)
[30] after join
-- join_any: wait for the FIRST child; others keep running
[40] D done (took 10)
[40] after join_any
-- join_none: do not wait at all
[40] after join_none
[45] F done (took 5)
[60] E done (took 30)
[60] after wait fork
- top.sv:20: Verilog $finish
```

Verilator 5.020 output (Icarus Verilog 12 prints the same lines).

- After `join_any` the parent resumed at 40 while E was still running; E completed at 60, long after the fork statement 'ended'. Leftover children are the source of many end-of-test bugs.
- With `join_none`, the child F did not start at the `fork`: a `join_none` child is **scheduled**, and runs only when the parent thread blocks or terminates. That detail is what makes the loop-variable bug below possible.
- `wait fork` blocks until all **immediate** child processes of the current process have finished - not their own descendants, and not unrelated threads. Here those were E and F.

20.2 The classic loop-variable bug

The most famous fork pitfall: spawning one thread per loop iteration with `join_none`, and letting each thread use the loop variable.

```
`timescale 1ns/1ns
module top;
    int i;                                // ONE static variable shared by every thread
    initial begin
        for (i = 0; i < 3; i++)
            fork
                #1 $display("[%0t] child sees i=%0d", $time, i);
            join_none
            $display("[%0t] loop finished, i=%0d", $time, i);
            #5 $finish;
    end
endmodule
```

c20_loopbug.sv - threads that read a shared variable after the loop has finished.

```
[0] loop finished, i=3
[1] child sees i=3
[1] child sees i=3
[1] child sees i=3
c20_loopbug.sv:10: $finish called at 5 (1ns)
```

Icarus Verilog 12 output: the IEEE-1800 behaviour.

The loop spawns three children but none of them runs until the parent blocks at `#5`, by which time the loop has finished and `i == 3`. All three children read the **same** static variable, so all see 3. The same happens with `for (int i ...)` in a static context: the LRM makes the loop variable a single variable of the loop, not one per iteration.

VERILATOR 5.020 HIDES THIS BUG

Compiled with Verilator 5.020, the very same file printed 'child sees i=0', 'i=1', 'i=2' - Verilator captured the value of `i` when each child was created, which is not IEEE 1800 behaviour. A testbench that 'works' on Verilator can therefore fail on Questa, VCS or Xcelium. When porting testbenches between simulators, fork-in-loop code is the first place to review.

The fix is to give every child its **own automatic copy**, made while the loop variable still holds the right value, i.e. before the parent moves on:

```
`timescale 1ns/1ns
module top;
  initial begin
    for (int i = 0; i < 3; i++) begin
      automatic int k = i;          // a NEW variable per iteration, set BEFORE the fork
      fork
        #1 $display("[%0t] child sees k=%0d", $time, k);
      join_none
    end
    wait fork;                      // wait for all three children
    $display("[%0t] all children done", $time);
    $finish;
  end
endmodule
```

c20_loopfix.sv - one automatic variable per iteration.

```
[1] child sees k=0
[1] child sees k=1
[1] child sees k=2
[1] all children done
- top.sv:12: Verilog $finish
```

Verilator 5.020 output.

An **automatic** variable declared in a **begin...end** block is created afresh each time the block is entered, i.e. once per iteration, and the child's reference keeps it alive after the iteration ends. An equivalent, LRM-sanctioned form declares the copy in the **declaration part of the fork itself** - **fork automatic int k = i; ... join_none** - because such fork declarations are initialized when the fork statement executes, before any child is spawned. (Verilator 5.020 failed with a C++ compilation error on that second form in our test, so the listing uses the first.) In a class method or **automatic** task everything is automatic already, and the explicit copy is still needed because the loop variable itself is still shared.

20.3 wait fork, disable fork and their scope

disable fork terminates **all active child processes of the calling process**, together with their descendants. It is the standard companion of **join_any**: race a response against a timeout, continue on whichever comes first, then kill the loser. The pitfall is the word *all*: children forked earlier by the same thread, for completely different purposes, die too.

```
`timescale 1ns/1ns
module top;
  initial begin
    fork
      forever #20 $display("[%0t] heartbeat", $time);
    join_none
    fork
      // later: a timeout race in the SAME thread
      #25 $display("[%0t] response arrived", $time);
      #50 $display("[%0t] TIMEOUT", $time);
    join_any
    disable fork;                  // PITFALL: also kills the heartbeat!
    #60 $display("[%0t] no heartbeat since 20", $time);
    $finish;
  end
endmodule
```

c20_disable_bug.sv - disable fork also kills an unrelated background thread.

```
[20]  heartbeat
[25] response arrived
[85] no heartbeat since 20
- top.sv:13: Verilog $finish
```

Verilator 5.020 output.

The heartbeat stopped after 20 ns, because it was a child of the same `initial` process. The robust pattern wraps the race in an extra `fork begin ... end join`, so that the thread executing `disable fork` is a fresh one whose only children are the two racing branches:

```
`timescale 1ns/1ns
module top;
  bit timed_out;
  initial begin
    fork
      // background thread started EARLIER
      forever #20 $display("[%0t]  heartbeat", $time);
    join_none
      // Guard the race in its own thread: 'disable fork' then only sees ITS children
      fork begin : guarded
        fork
          begin #25 timed_out = 0; $display("[%0t] response arrived", $time); end
          begin #50 timed_out = 1; $display("[%0t] TIMEOUT", $time); end
        join_any
          disable fork; // kills the timeout thread only
        end join
        $display("[%0t] timed_out=%0b", $time, timed_out);
        #50 $display("[%0t] no TIMEOUT at 50, heartbeat still alive", $time);
        $finish;
      end
    end
  endmodule
```

c20_disable.sv - guarding disable fork with an isolating fork...join.

```
[20]  heartbeat
[25] response arrived
[25] timed_out=0
[40]  heartbeat
[60]  heartbeat
[75] no TIMEOUT at 50, heartbeat still alive
- top.sv:18: Verilog $finish
```

Verilator 5.020 output (identical under Icarus Verilog 12).

DISABLE IS STATIC AND GLOBAL

The Verilog `disable label;` statement kills **every** active execution of the named block, in every thread - if ten drivers run the same task, `disable` on a block inside it stops all ten. It predates dynamic processes and does not know which thread you meant. In class-based code use `disable fork` (with the guard above) or the `process` class instead.

20.4 The process class

`process` is a built-in class (declared in the `std` package; it cannot be extended or constructed with `new`) that gives a handle to a running thread. A thread obtains its own handle with the static function `process::self()` and can hand it to others.

Method	Meaning
<code>process::self()</code>	Handle to the calling process
<code>status()</code>	Enum <code>process::state</code> : RUNNING, WAITING, SUSPENDED, FINISHED, KILLED

Method	Meaning
<code>kill()</code>	Terminate the process and all its descendants
<code>await()</code>	Block until that (other) process finishes or is killed; a process may not await itself
<code>suspend()</code> / <code>resume()</code>	Pause and continue it
<code>srandom()</code> , <code>get_randstate()</code> , <code>set_randstate()</code>	Control the thread's random generator (Chapter 19)

```

`timescale 1ns/1ns
module top;
  process p[3];
  initial begin
    fork
      // each child records its own handle first
      begin p[0] = process::self(); #10 $display("[%0t] worker 0 finished", $time); end
      begin p[1] = process::self(); #20 $display("[%0t] worker 1 finished", $time); end
      begin p[2] = process::self(); #30 $display("[%0t] worker 2 finished", $time); end
    join_none
    #15;
    foreach (p[n]) $display("[%0t] worker %0d status=%s", $time, n, p[n].status().name());
    p[2].kill();
    p[1].await();
    $display("[%0t] after await: w1=%s w2=%s", $time,
      p[1].status().name(), p[2].status().name());
    #20 $display("[%0t] worker 2 never printed", $time);
    $finish;
  end
endmodule

```

c20_proc.sv - recording process handles, querying status, kill and await.

```

[10] worker 0 finished
[15] worker 0 status=FINISHED
[15] worker 1 status=WAITING
[15] worker 2 status=WAITING
[20] worker 1 finished
[20] after await: w1=FINISHED w2=KILLED
[40] worker 2 never printed
- top.sv:17: Verilog $finish

```

Verilator 5.020 output.

At 15 ns worker 0 has finished and the others are WAITING (blocked on their delays). Killing worker 2 removes it before it can print; `await` returned exactly when worker 1 finished at 20. UVM uses `process` internally to kill the `run_phase` threads of all components when the phase ends, and to implement sequence `kill()`.

20.5 Named events

An `event` is a synchronization object with no value. -> **ev triggers** it, unblocking every process currently waiting with `@ev`. Three details make events subtle:

- `@ev` is **edge-sensitive**: it waits for a trigger that happens *after* the wait begins. A process that reaches `@ev` in the same time step but just after the trigger misses it and may wait forever.
- `ev.triggered` is a **level** that stays true from the trigger until the end of the current time step. `wait (ev.triggered)` therefore succeeds whether it executes before or after the trigger within that time step - the standard cure for the race.
- -> `ev` is the **nonblocking trigger**: the trigger is scheduled in the NBA region of the time step, after all active processes (including ones about to wait) have run. Events can also be assigned (`ev_a = ev_b` merges them), compared, and set to `null`; `wait_order(a, b, c)` waits for triggers in a given order.

```

`timescale 1ns/1ns
module top;
  event done;
  initial begin
    // waiter 1: edge-sensitive @, waiting early
    @done $display("[%0t] waiter @done woke up", $time);
  end
  initial begin
    // triggers at t=10
    #10 -> done;
    $display("[%0t] triggered done", $time);
  end
  initial begin
    // waiter 2: arrives in the SAME time step, too late
    #10 @done $display("[%0t] late @done woke up (never printed)", $time);
  end
  initial begin
    // waiter 3: level-sensitive triggered state
    #10 wait (done.triggered) $display("[%0t] wait(done.triggered) woke up", $time);
  end
  initial #50 $display("[%0t] end of test", $time);
endmodule

```

c20_event.sv - an early waiter, a late waiter and a wait(triggered) waiter, all in the trigger's time step.

```

[10] triggered done
[10] wait(done.triggered) woke up
[10] waiter @done woke up
[50] end of test

```

Verilator 5.020 output.

Waiters 2 and 3 and the triggering process are all ready at 10 ns and run in an unspecified order. The run shows the dangerous case: the trigger ran first, so waiter 2's `@done` started *after* the trigger and never woke (its message never appears), while waiter 3's `wait (done.triggered)` succeeded. On another simulator the order may differ and waiter 2 might wake - which is exactly why the code is wrong: its behaviour depends on scheduling order. Always use `wait (ev.triggered)` when the waiter might arrive in the same time step as the trigger.

20.6 Semaphores

A **semaphore** is a built-in class holding a bucket of **keys**. `new(n)` creates it with `n` keys (default 0); `get(k)` removes `k` keys, blocking until enough are available; `put(k)` returns keys; `try_get(k)` takes them only if available, returning a positive value on success and 0 otherwise, without blocking. With one key it is a **mutex** that serializes access to a shared resource: a bus that several masters' drivers use, a shared memory model, a log file.

```

`timescale 1ns/1ns
module top;
  semaphore bus = new(1); // one key: a mutex for a shared bus
  task automatic master(int id, int n_cycles);
    bus.get(1); // blocks until a key is available
    $display("[%0t] M%0d owns the bus", $time, id);
    #(n_cycles);
    $display("[%0t] M%0d releases", $time, id);
    bus.put(1); // return the key
  endtask
  initial begin
    fork
      master(0, 10);
      master(1, 5);
    begin
      #1 if (!bus.try_get(1)) $display("[%0t] M2 try_get failed, not blocking", $time);
    end
  join
  $finish;
endmodule

```

```
end
endmodule
```

c20_sem.sv - a one-key semaphore arbitrating a bus between masters.

```
[0] M0 owns the bus
[1] M2 try_get failed, not blocking
[10] M0 releases
[10] M1 owns the bus
[15] M1 releases
- top.sv:19: Verilog $finish
```

Verilator 5.020 output.

- M0 and M1 both called `get` at time 0; M0 won, M1 blocked until M0's `put` at 10. The LRM specifies that waiting `get` requests are served in FIFO order, but which of two same-time callers queues first is scheduling order.
- `put` does not check ownership: any thread may put keys, and putting more keys than were taken silently grows the bucket. Always pair get/put in the same task, and put in every exit path.
- Forgetting a `put` (for example after an early `return` or a `disable fork` that kills the owner) deadlocks every later user - a common cause of simulations that 'hang' until a watchdog fires.

20.7 Mailboxes

A `mailbox` is a built-in FIFO for passing messages between processes - the SystemVerilog equivalent of a channel. It is the natural connection between a generator and a driver, or a monitor and a scoreboard.

Method	Blocking?	Behaviour
<code>new(bound = 0)</code>	-	0 = unbounded; >0 = capacity
<code>put(msg)</code>	Yes, when full	Append to the tail
<code>try_put(msg)</code>	No	Returns 1 if stored, 0 if full
<code>get(ref msg)</code>	Yes, when empty	Remove the head
<code>try_get(ref msg)</code>	No	Returns >0 if a message was removed, 0 if empty (negative on type mismatch)
<code>peek(ref msg)</code>	Yes, when empty	Copy the head without removing it
<code>try_peek(ref msg)</code>	No	As peek, returns 0 if empty
<code>num()</code>	No	Number of messages currently stored

A **parameterized** mailbox, `mailbox #(Txn)`, accepts only that type and is checked at compile time. A plain `mailbox` is typeless: it accepts anything, and a `get` into a variable of the wrong type is a **run-time** error. Always use the parameterized form.

```
`timescale 1ns/1ns
class Txn;
  int id; bit [7:0] data;
  function new(int id); this.id = id; data = 8'(id * 17); endfunction
endclass

module top;
  mailbox #(Txn) mb = new(2);          // parameterized, BOUNDED to 2 entries
  initial begin : producer
    for (int n = 0; n < 4; n++) begin
      automatic Txn t = new(n);        // automatic: a fresh handle per iteration
      mb.put(t);                        // blocks while the mailbox is full
      $display("[%0t] put   id=%0d (num=%0d)", $time, n, mb.num());
    end
  end
  initial begin : consumer
```

```

Txn t;
#10;                                // start late: the producer fills up and blocks
mb.peek(t);                          // look without removing
$display("[%0t] peek id=%0d (num=%0d)", $time, t.id, mb.num());
repeat (4) begin
    mb.get(t);                       // blocks while the mailbox is empty
    $display("[%0t] get id=%0d data=%0d", $time, t.id, t.data);
    #5;
end
if (mb.try_get(t) == 0) $display("[%0t] try_get on empty mailbox returns 0", $time);
$finish;
end
endmodule

```

c20_mbox.sv - producer and consumer connected by a bounded, parameterized mailbox.

```

[0] put id=0 (num=1)
[0] put id=1 (num=2)
[10] peek id=0 (num=2)
[10] get id=0 data=0
[10] put id=2 (num=2)
[15] get id=1 data=17
[15] put id=3 (num=2)
[20] get id=2 data=34
[25] get id=3 data=51
[30] try_get on empty mailbox returns 0
- top.sv:27: Verilog $finish

```

Verilator 5.020 output.

The bound of 2 gives **back-pressure**: the producer put ids 0 and 1 at time 0 and then blocked in **put** until the consumer freed a slot at 10; from then on, the two threads advance in lock-step with the consumer's rate. An unbounded mailbox would have let the producer run arbitrarily far ahead, which is sometimes what you want (monitor to scoreboard) and sometimes a problem (a generator that creates a million transactions before the driver sends one).

A MAILBOX STORES HANDLES, NOT COPIES

mb.put(t) stores the **handle** **t**. If the producer then modifies the same object and puts it again, the consumer receives two handles to one object whose contents are whatever the producer wrote last. The producer above constructs a new Txn per iteration (and declares **t** *automatic* so that each iteration really has its own variable: a declaration with an initializer inside a loop of a static block would otherwise be a static variable, initialized only once). Copy-on-send or new-per-transaction is mandatory.

Choosing a synchronization primitive

Need	Use	UVM equivalent
Signal 'something happened', no data	event, wait(ev.triggered)	uvm_event , objections for end of test
Mutual exclusion / limited resource	semaphore	Sequencer arbitration, lock()/grab()
Pass transactions with flow control	mailbox (bounded)	uvm_tlm_fifo , seq_item_port get/put
Broadcast transactions to many	Loop over mailboxes, or callbacks	uvm_analysis_port (write to all subscribers)
Wait for a set of threads	fork...join , wait fork	uvm_objection drop / phase_ready_to_end

INTERVIEW FAVOURITES

(1) 'Print 0..9 from ten `join_none` threads' - automatic copy per iteration. (2) 'Difference between `@ev` and `wait(ev.triggered)`' - edge vs level within the time step. (3) 'Implement a mailbox with a semaphore and a queue' - two semaphores (slots and items) plus a queue. (4) 'What does `disable fork kill?`' - all children of the current process, including unrelated earlier ones; isolate with `fork begin ... end join`. (5) 'Can a function contain `fork?`' - only `fork...join_none`, because a function must not block.

20.8 Summary

- `join` waits for all children, `join_any` for the first, `join_none` for none; `join_none` children start only when the parent blocks.
- Threads spawned in a loop must use a per-iteration automatic copy of the loop variable; Verilator 5.020 masks this bug, IEEE simulators expose it.
- `wait fork` waits for all immediate children; `disable fork` kills **all** descendants of the current process; isolate with an extra `fork...join`.
- The `process` class exposes `self`, `status`, `kill`, `await`, `suspend` and `resume`.
- Events: `->` triggers, `@` is edge-sensitive, `.triggered` is a level for the rest of the time step, `->>` triggers in the NBA region.
- Semaphores serialize access with keys (`get/put/try_get`); mailboxes pass typed messages with optional bound and back-pressure (`put/get/peek/try_*/num`).
- Mailboxes pass handles: construct a new object per message.

20.9 Exercises

- 1 Rewrite `c20_loopbug.sv` so that each child prints its index using the fork-declaration form `fork automatic int k = i; ... join_none`, and run it on a simulator that accepts it.
- 2 Write a task `with_timeout(int ns)` that runs a transaction task and reports an error if it takes longer than `ns`, without killing any other thread of the caller.
- 3 Build a bounded mailbox yourself from a queue and two semaphores and verify that it behaves like `c20_mbox.sv`.
- 4 Modify `c20_sem.sv` so that masters need 2 of 3 keys. Which masters can overlap, and what happens if one master calls `get(4)`?
- 5 Create a monitor thread that triggers an event on every transaction and a checker that uses `@`. Construct a case in which the checker misses a transaction, then fix it.
- 6 Explain why a function may contain `fork...join_none` but not `fork...join`, and give a real use for the former.

21. Clocking Blocks, Program Blocks and Race-Free Testbenches

A testbench and a DUT that both react to the same clock edge are two groups of processes woken in the same time step. If the testbench writes an input with a blocking assignment at `@(posedge clk)`, whether the DUT's flip-flops see the old or the new value depends on which process the simulator happens to run first - a **race**. Races are the most expensive testbench bugs: the test passes on one simulator, fails on another, and changes behaviour when an unrelated line is added. This chapter explains the complete SystemVerilog scheduling algorithm, then the two language features designed to remove TB-DUT races by construction - **clocking blocks** and **program blocks** - and the housekeeping constructs every testbench uses: `final`, `$exit`, `timeunit` and `timeprecision`.

21.1 Anatomy of a race

```
`timescale 1ns/1ns
module top;
  logic clk = 0, d = 0, q1, q2;
  always #5 clk = ~clk;
  always @(posedge clk) q1 <= d;           // "DUT" flop 1: declared BEFORE the stimulus
  initial begin                          // stimulus with a BLOCKING write at the edge
    @(posedge clk) d = 1;               // RACE: same region as the flops
    @(posedge clk) d = 0;
    #1 $finish;
  end
  always @(posedge clk) q2 <= d;         // "DUT" flop 2: declared AFTER the stimulus
  always @(negedge clk) $display("[%0t] d=%b q1=%b q2=%b", $time, d, q1, q2);
endmodule
```

c21_race.sv - stimulus written with a blocking assignment at the clock edge.

```
[10] d=1 q1=1 q2=0
c21_race.sv:9: $finish called at 16 (1ns)
```

Icarus Verilog 12 output.

Both flip-flops are identical and sample the same `d` at the same edge, yet at 10 ns they disagree: `q1` caught the new value and `q2` the old one. At the edge, three processes are ready in the Active region - flop 1, flop 2 and the stimulus - and the LRM lets the simulator run them in any order. Icarus happened to run flop 2 before the stimulus and flop 1 after it. Another simulator, another version, or a reordered source file gives a different answer. (Verilator 5.020 printed `q1=1 q2=1` for this file.) Real hardware has no such ambiguity: a flop samples its input just before the edge. The goal of every technique in this chapter is to make the testbench behave like that hardware.

21.2 The SystemVerilog scheduling regions

Chapter 5 introduced the Verilog event queue (Active, Inactive, NBA, Monitor). IEEE 1800 refines each **time slot** into a fixed sequence of regions, adding a sampling region before everything, an Observed region for assertions, and a mirror-image **reactive** region set for testbench code. The PLI-only regions (Pre-Active, Pre-NBA, Post-NBA, Pre-Observed, Post-Observed, Pre-Re-NBA, Post-Re-NBA, Pre-Postponed) are omitted from the figure; they exist for VPI callbacks.

```

time slot t
+-----+
| PREPONED   sample #1step inputs, assertion/coverage values |
+-----+
|
| v          ACTIVE REGION SET (design: modules, interfaces)
+-----+
| ACTIVE     always/initial in modules, assign, blocking =,    |<-+
|            RHS of <=, $display, gate evaluation              | |
| INACTIVE   processes resumed after #0                        | |
| NBA        nonblocking-assignment updates (LHS of <=)        |--+
+-----+
| (loop until Active, Inactive and NBA are all empty)
| v
+-----+
| OBSERVED   concurrent assertions evaluated (on sampled values) |
+-----+
|
| v          REACTIVE REGION SET (testbench: programs)
+-----+
| REACTIVE   program code, assertion action blocks              |<-+
| RE-INACTIVE #0 inside programs                                | |
| RE-NBA     <= updates from programs, clocking #0 drives      |--+
+-----+
| (if anything scheduled Active events: back to ACTIVE)
| v
+-----+
| POSTPONED  $strobe, $monitor, read-only; then advance time   |
+-----+

```

The IEEE 1800 time slot. Design code iterates in the active set; testbench code in the reactive set runs after the design has settled; everything re-iterates until no events remain in the time slot.

Region	What executes there	Why it matters
Preponed	Sampling for #1step clocking inputs and for concurrent assertions and coverage	Values 'just before' the edge, immune to races
Active	Module processes, continuous assignments, blocking assignments, RHS of <=	Order within the region is undefined
Inactive	Threads resumed from #0	#0 is a hack for ordering; avoid it
NBA	LHS updates of <= from module code	Makes flops order-independent
Observed	Evaluation of concurrent assertions (Chapter 22)	Uses Preponed samples
Reactive	Program processes, assertion pass/fail action blocks	Testbench reacts after design settled
Re-Inactive / Re-NBA	#0 and <= in program code; clocking-block drives with #0 output skew	Mirror of Inactive/NBA for the TB
Postponed	\$strobe , \$monitor , final sampling	Sees the final values of the time slot; nothing may change them

```

`timescale 1ns/1ns
module top;
  int a = 0;
  initial begin
    #10;
    a = 1; // Active: immediate update
    a <= 2; // NBA: update scheduled for later
    $strobe("[%0t] $strobe (Postponed) a=%0d", $time, a);
    $display("[%0t] $display (Active) a=%0d", $time, a);
    #0 $display("[%0t] after #0 (Inactive) a=%0d", $time, a);
    #1 $display("[%0t] next time step a=%0d", $time, a);
  end
endmodule

```

c21_regions.sv - which value each kind of statement sees.

```
[10] $display (Active)      a=1
[10] after #0 (Inactive) a=1
[10] $strobe  (Postponed) a=2
[11] next time step      a=2
```

Icarus Verilog 12 output.

`$display` executes immediately in Active and sees the blocking value 1; the process resumed from `#0` runs in Inactive, which comes **before** NBA, so it still sees 1; the NBA update to 2 happens next, and `$strobe` prints in Postponed, after all updates. The same file compiled with Verilator 5.020 printed `a=2` for all three lines of time 10: Verilator does not implement the Active/Inactive/NBA distinction inside `initial` blocks as the LRM does. Use an event-driven simulator (Icarus, or any commercial one) to reason about regions.

With the regions in hand, the classic fix for the race is clear: drive DUT inputs with **nonblocking** assignments, so the new value is written in NBA, after every flop has read the old one in Active.

```
initial begin                                // stimulus with a NONBLOCKING write at the edge
    @(posedge clk) d <= 1;                    // NBA: flops sampled the OLD d
    @(posedge clk) d <= 0;
```

c21_norace.sv - the only change to c21_race.sv.

```
[10] d=1 q1=0 q2=0
c21_norace.sv:9: $finish called at 16 (1ns)
```

Icarus Verilog 12 output: both flops agree.

Nonblocking stimulus removes the race on **inputs**, but a testbench that *samples* DUT outputs at `@(posedge clk)` still races with the NBA updates of those outputs, and every engineer must remember the discipline. Clocking blocks move the discipline into the language.

21.3 Clocking blocks

A **clocking block** declares, for one clock, which signals a testbench samples (**input**), which it drives (**output**, **inout**), and **when**: an input is sampled a given **skew** before the clocking event, and an output is driven a given skew after it. The testbench then reads `cb.sig` and writes `cb.sig <= value` instead of touching the raw signals, and is automatically synchronized to the clock like the flip-flops around it.

```
module dut (input logic clk, rst_n, input logic [7:0] d, output logic [7:0] q);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q <= '0; else q <= d + 8'd1;    // registered "increment"
endmodule

module top;
    timeunit 1ns; timeprecision 100ps;
    logic clk = 0, rst_n = 0;
    logic [7:0] d = 0, q;
    always #5 clk = ~clk;                          // posedges at 5, 15, 25, ...
    dut u (.*);
```

```
default clocking cb @(posedge clk);                // 'default' enables ##n delays
default input #1step output #2;                    // sample just before edge, drive 2ns after
output d, rst_n;
input q;
endclocking

initial begin
    ##1 cb.rst_n <= 1;                               // clocking drive (always nonblocking)
    for (int n = 10; n < 13; n++) begin
        ##1 cb.d <= 8'(n);
        $display("[%4.1f] edge: cb.q=%0d q=%0d; drive d=%0d", $realtime, cb.q, q, n);
    end
    repeat (2) begin
        ##1;                                          // wait one clock cycle
```



```

    $display("[%4.1f] edge: cb.q=%0d q=%0d", $realtime, cb.q, q);
end
$finish;
end
always @(d) $display("[%4.1f]   pin d <- %0d", $realtime, d);
endmodule

```

c21_cb.sv - a default clocking block with #1step input and #2 output skews, ##n cycle delays, and clocking drives.

```

[ 0.0]   pin d <- 0
[15.0] edge: cb.q=0 q=0; drive d=10
[17.0]   pin d <- 10
[25.0] edge: cb.q=1 q=1; drive d=11
[27.0]   pin d <- 11
[35.0] edge: cb.q=11 q=11; drive d=12
[37.0]   pin d <- 12
[45.0] edge: cb.q=12 q=12
[55.0] edge: cb.q=13 q=13
- top.sv:29: Verilog $finish

```

Verilator 5.020 output.

Walk through the timeline. At each edge the testbench reads `cb.q`, which is the value of `q` sampled in the **Preponed** region of that time slot (**#1step**) - the value a real flop would capture. `cb.d <= n` does not change `d` now: it schedules the drive for this clocking event plus the output skew, so `d` changes at 17, 27, 37, exactly 2 ns after each edge, well clear of the edge in both directions. The DUT registers `d + 1`, so the value sampled at edge N is what the DUT computed at edge N-1: `cb.q` becomes 11 at 35 because the DUT captured `d = 10` at 25. The testbench sees exactly the waveform a logic analyser would show.

Syntax	Meaning
<code>clocking cb @(posedge clk); ... endclocking</code>	Named clocking block; the event can be any event expression
<code>default input #1step output #2;</code>	Default skews for the block
<code>input #1step</code>	Sample in the Preponed region: value just before the edge (default input skew)
<code>input #0</code>	Sample in the Observed region of the edge's time slot (after the design settled - rarely what you want)
<code>input #3ns / output #2</code>	Explicit time skews (in the block's time unit when unitless)
<code>output #0 (default output skew)</code>	Drive in the Re-NBA region of the edge's time slot
<code>output negedge</code>	Skew given as an edge of the clock
<code>default clocking cb ...;</code>	One per module/interface/program; enables <code>##n</code> and is the default for assertions
<code>##n</code>	Wait n clocking events of the default clocking
<code>@(cb)</code>	Wait for the clocking event itself
<code>cb.sig <= expr; / cb.sig <= ##2 expr;</code>	Synchronous drive, optionally n cycles later; <code>=</code> is illegal on clocking outputs
<code>global clocking</code>	Names the primary system clock for formal/assertions (<code>\$global_clock</code> , Chapter 22)

Clocking block syntax and semantics.

- A clocking block does not create signals; `cb.q` is a view of `q`. Reading `q` directly bypasses the sampling (the example prints both; they agree only because the display runs before the NBA update of that slot).
- Clocking blocks are declared inside modules, interfaces or programs; in UVM they live in the **interface**, and the driver and monitor reach them through a virtual interface: `vif.drv_cb.valid <= 1; @(vif.drv_cb);` (Chapter 16 shows how `modport` exports them).

- Several clocking blocks may use the same clock with different directions and skews - typically `drv_cb` and `mon_cb` in one interface.
- Inputs and outputs of a clocking block can be hierarchical expressions: `input en = top.dut.u_ctrl.en;`

TOOL PORTABILITY: CLOCKING-BLOCK CORNER CASES

Verilator 5.020 rejected the two-step form `clocking cb ...; ... default clocking cb;` ('Unsupported: default clocking identifier'), so the listing uses `default clocking cb @(...)` directly. In an earlier version of this example, an `@(cb)` executed right after a `##1` in the same time step resumed in **that same time step** on Verilator and read post-NBA values; the LRM says it must wait for the next clocking event. Commercial simulators follow the LRM here, but mixing `@(cb)`, `##n` and raw `@(posedge clk)` in one thread is a known source of off-by-one-cycle bugs on every tool: pick one style per driver.

21.4 Program blocks

A **program** is a module-like container for testbench code whose processes run in the **Reactive** region set. Because reactive code runs only after the design's active set has settled, a program can read DUT outputs and even write DUT inputs with blocking assignments at the clock edge without racing the design. Program blocks also give an automatic end of simulation.

```
`timescale 1ns/1ns
module top;
  logic clk = 0, q;
  wire d;                                // driven by the program's output port
  int n_edges;
  always #5 clk = ~clk;
  always @(posedge clk) q <= d;           // DUT flop (design region: Active/NBA)
  always @(posedge clk) n_edges++;
  test t (.clk(clk), .d(d), .q(q));
  final $display("final block: %0d clock edges simulated", n_edges);
endmodule

program automatic test (input logic clk, output logic d, input logic q);
  initial begin                          // runs in the REACTIVE region set
    d = 0;
    repeat (2) begin
      @(posedge clk);
      d = ~d;                            // even a blocking write cannot race the flop
      $display("[%0t] program: drove d=%b, q=%b", $time, d, q);
    end
    #1 $display("[%0t] program ends -> $exit", $time);
    $exit;                               // ends this program; last program ends the sim
  end
endprogram
```

c21_prog.sv - a program driving a module-based design, with final and \$exit.

```
[5] program: drove d=1, q=0
[15] program: drove d=0, q=1
[16] program ends -> $exit
- top.sv:22: Verilog $finish
final block: 2 clock edges simulated
```

Verilator 5.020 output.

- At 5 ns the program read `q=0`, the flop's value **after** its NBA update in that slot; the flop had already captured `d=0` before the program's blocking write, so there is no race even though `d = ~d` is blocking.
- A program may contain `initial` and `final` procedures, data, classes, tasks, functions, clocking blocks and continuous assignments. It may **not** contain `always` procedures, module instances, interfaces or other programs, and it cannot be instantiated inside a program.

- When every **initial** procedure of every program has finished, the simulation ends as if **\$finish** were called. **\$exit** ends the calling program immediately (all of its threads); here that was the last program, so simulation ended at 16 and ran the **final** block.
- Tasks called from a program run in the reactive region; module code called from anywhere runs in the caller's region.

PROGRAM BLOCKS IN TODAY'S INDUSTRY

Program blocks were introduced with Vera/VMM-style testbenches, and you will meet them in legacy code and interview questions. Modern UVM testbenches do **not** use them: UVM's **run_test()** is called from a module, components are classes, and race-freedom comes from clocking blocks in interfaces plus nonblocking drives. Programs also complicate things: a module task called from a program executes in the reactive region, which surprises code written for Active-region timing, and a class method executes in the region of whatever thread calls it. Verilator 5.020 accepts **program** and **\$exit** but does not model the reactive region set separately, so the run above demonstrates syntax, not scheduling.

21.5 final blocks

A **final** procedure runs once, at the end of simulation - when **\$finish** is called (explicitly, by **\$fatal**, or implicitly by the end of all programs) or when the event queue becomes empty. It executes in zero time, like a function: no delays, no event controls, no blocking calls. Multiple final blocks run in an undefined order. It is the right place for end-of-test reports: error counts, scoreboard leftovers (expected items never seen), coverage summaries and 'TEST PASSED/FAILED' lines that regression scripts grep for. UVM's report phase is the class-based equivalent.

END-OF-TEST CHECKS BELONG IN FINAL - BUT NOT ONLY THERE

A scoreboard that still holds expected transactions at the end means the DUT dropped data. Checking **expected.size() == 0** in **final** catches it even when the test ended through a timeout or a **\$fatal** elsewhere. But **final** cannot wait for in-flight transactions to drain; drain first (wait for idle, then a few cycles), then **\$finish**, then let **final** report.

21.6 timeunit, timeprecision and the timescale directive

Delays such as **#5** are multiples of the **time unit** of the design element that contains them, rounded to its **time precision**. Verilog sets both with the compiler directive **timescale**, which is a textual, file-order-dependent setting that leaks into every file compiled after it (Chapter 8). SystemVerilog adds the **timeunit** and **timeprecision** keywords, which are part of the module (or interface, program, package, compilation unit) itself.

```
module fast;                                // local time unit and precision
    timeunit 1ns;
    timeprecision 100ps;
    initial begin
        #1.26;                               // 1.26 ns rounds to the 100ps precision -> 1.3 ns
        $display("fast: $time=%0d $realtime=%0.2f (units of 1ns)", $time, $realtime);
        $printrtimescale;
    end
endmodule

module slow;
    timeunit 1us / 1ns;                       // combined form: unit / precision
    initial begin
        #0.0025;                             // 2.5 ns -> 3 ns at 1ns precision
        $display("slow: $realtime=%0.4f (units of 1us)", $realtime);
        $printrtimescale;
    end
end
```

```

endmodule

module top;
  timeunit 1ns; timeprecision 1ps;
  fast f(); slow s();
  initial #10 $display("top : $realtime=%0.3f ns", $realtime);
endmodule

```

c21_tu.sv - per-module time units, rounding to precision, and \$printtimescale.

```

fast: $time=1 $realtime=1.30 (units of 1ns)
Time scale of (top.f) is 1ns / 100ps
slow: $realtime=0.0030 (units of 1us)
Time scale of (top.s) is 1us / 1ns
top : $realtime=10.000 ns

```

Icarus Verilog 12 output.

- **#1.26** in a 1ns/100ps module is rounded to 1.3 ns; `$time` returns the time **rounded to the module's unit** (1), `$realtime` keeps the fraction (1.30).
- **#0.0025** in a 1us/1ns module is 2.5 ns, rounded to 3 ns, and `$realtime` reports it in microseconds (0.0030).
- `timeunit / timeprecision` must be the first items in the design element; the combined form `timeunit 1us / 1ns`; is also legal. A declaration in the element beats any `timescale` in effect, which beats a compilation-unit-level declaration, which beats the tool's default.
- The simulator's global time precision is the finest precision of all design elements, and `#1step` equals one unit of that global precision.
- `$printtimescale` and `$timeformat` (Chapter 10) help debug timing mismatches. Verilator 5.020 ignored the per-module `timeprecision` in this example (it printed 1.26 and '1ns / 1ps'), another reason to use an event-driven simulator for timing questions.

MIXED TIME UNITS IN A TESTBENCH

A classic integration bug: an IP's testbench file has no time declaration and silently inherits a `timescale` of 1ps/1ps from a file compiled before it, so `#10` becomes 10 ps instead of 10 ns and the clock runs 1000x faster. Put `timeunit/timeprecision` (or a `timescale`) in **every** file, or use the tool option that errors on missing time units (most simulators have one).

21.7 A race-free testbench checklist

Race-free TB-DUT interaction

- ☐ Drive DUT inputs through a clocking block (`cb.sig <= v`) or, without one, with nonblocking assignments.
- ☐ Sample DUT outputs through a clocking block (`cb.sig, #1step`) or in a monitor that uses the same edge and reads values updated by NBA only.
- ☐ Never use `#0` to 'fix' ordering; it moves the race, it does not remove it.
- ☐ Generate clocks with a single process; derive other clocks with nonblocking or in the same process to avoid delta-cycle skew between clock domains of a synchronous design.
- ☐ Keep reset release synchronous to the clock (drive it through the clocking block).
- ☐ Do not mix `@(posedge clk)`, `@(cb)` and `##n` in one thread.
- ☐ Use `wait (ev.triggered)` rather than `@ev` for same-time-step handshakes (Chapter 20).
- ☐ Put `timeunit/timeprecision` in every file; report with `final`.
- ☐ Confirm race-sensitive behaviour on an event-driven, LRM-compliant simulator - Verilator deviates in several of the cases shown in this chapter.

21.8 Summary

- A race is a result that depends on the undefined order of processes in one region; TB-DUT races at the clock edge are the common case.
- Each time slot runs Preponed, the active set (Active, Inactive, NBA), Observed, the reactive set (Reactive, Re-Inactive, Re-NBA) and Postponed, iterating until no events remain.
- Clocking blocks sample inputs a skew before the clock event (`#1step` = Preponed) and drive outputs a skew after it (`#0` = Re-NBA); `default clocking` enables `##n`.
- Program blocks run TB code in the reactive set and end the simulation when all their initial blocks finish; `$exit` ends a program early. UVM uses modules + clocking blocks instead.
- `final` blocks run once, in zero time, at the end of simulation.
- `timeunit/timeprecision` bind time units to the design element; `#1step` is one global precision unit.

21.9 Exercises

- 1 Add a third flop to `c21_race.sv` that uses `always_ff` and place it between the other two. Predict whether the result changes on Icarus, then run it.
- 2 Rewrite the stimulus of `c21_race.sv` with a clocking block (`output #1`) and show that both flops always agree.
- 3 In `c21_cb.sv`, change the input skew to `#0` and predict the sampled values at every edge. Why is `#1step` the default?
- 4 Draw the regions in which each statement of `c21_regions.sv` executes, and predict the output if `a <= 2` is replaced by `#0 a = 2`.
- 5 Write an interface `apb_if` with a `drv_cb` (outputs: `psel`, `penable`, `pwrite`, `paddr`, `pdata`; inputs: `pready`, `prdata`) and a `mon_cb` (all inputs), plus modports for driver and monitor.
- 6 Interview drill: explain in three sentences why a program block can use blocking assignments to DUT inputs without racing the DUT, and why UVM testbenches nevertheless do not use programs.

22. SystemVerilog Assertions (SVA) in Depth

An assertion is an executable statement of intent: a property the design must always satisfy, written in a form a simulator can check every cycle and a formal tool can try to prove for every possible input. SystemVerilog Assertions (SVA, IEEE 1800 clauses 16 and 17) are a small temporal language embedded in SystemVerilog. They are the single most productive verification feature of the language: a ten-line assertion bound into a block catches the bug at the cycle and the signal where it happens, instead of hundreds of cycles later as a scoreboard mismatch at the far end of the chip.

This chapter covers the language precisely: the two families of assertions and where they execute in the event scheduler (Chapter 5), sequences and their repetition operators, the sequence and property operators, implication and vacuity, sampled-value functions, reset handling, local variables, multi-clock properties, the four verification directives, and the packaging constructs (**bind**, **checker**) that let a DV engineer attach checks to RTL without touching it. It ends with a reusable library of SoC protocol assertions. Examples that the open-source tools can execute were run with Verilator 5.020; the rest are written to the IEEE 1800-2017 grammar and need a commercial simulator (VCS, Xcelium, Questa, Riviera-PRO) or a formal tool (JasperGold, VC Formal, Questa Formal, SymbiYosys with a commercial front end).

22.1 Two families: immediate and concurrent assertions

Aspect	Immediate assertion	Concurrent assertion
Syntax	<code>assert (expr) [pass] [else fail];</code>	<code>assert property (prop) [pass] [else fail];</code>
Where	Procedural code (and, for deferred forms, also as a module item)	Module, interface, program, checker, or procedural code (clock then inferred)
When evaluated	When execution reaches it (simple) or at the end of the time step (deferred)	On every tick of its clock
Values used	Current values	Values sampled in the Preponed region
Temporal?	No - a single instant	Yes - sequences span many clock ticks, many attempts overlap
Formal tools	Mostly ignored (some treat as an invariant)	The native input of property checkers
Typical use	Checking function arguments, TB invariants, combinational sanity	Protocol rules, latency, stability, FSM legality, data integrity

Both families share the same verification directives: **assert** (the property must hold), **assume** (the property is a constraint on the environment), **cover** (tell me when this happens) and, for concurrent properties only, **restrict** (a formal-only constraint). Every assertion may carry a label (**a_req_ack:**), which becomes its hierarchical name in reports, waveform viewers and coverage databases - always label your assertions.

22.2 Immediate assertions: simple and deferred

Simple immediate assertions

A **simple immediate assertion** is a procedural statement. When control reaches it, the expression is evaluated like the condition of an **if**: a value of 0, X or Z is a failure, any known non-zero value is a success. The optional pass statement runs on success; the statement after **else** runs on failure. With no **else**, a failure calls **\$error** with a tool-specific message. The action statements may be any procedural statement, but in practice they are one of the four **severity tasks**: **\$fatal(finish_number, fmt, ...)** (terminates the run; **finish_number** 0, 1 or 2 selects the statistics printed as for **\$finish**), **\$error** (run-time error, counted), **\$warning** and **\$info**. All four print the file, line, scope and time in a tool-specific format.

```
module imm;
```

```

logic [3:0] a, b;
logic      sel;
logic [3:0] y;
assign y = sel ? a : b;

// deferred immediate assertion: evaluated after the glitches settle
always_comb a_def: assert #0 (y == (sel ? a : b)) else $error("mux mismatch");

initial begin
    a = 4'd3; b = 4'd9; sel = 1'b0;
    #1;
    a_imm: assert (y == 4'd9) $info("y=%0d as expected", y);
           else $error("y=%0d", y);
    sel = 1'b1; #1;
    assert (y == 4'd9) else $warning("warning: y=%0d, sel=%b", y, sel);
    assert (!$isunknown(y)) else $fatal(1, "X on y");
    $display("done at %0t", $time);
    $finish;
end
endmodule

```

imm.sv - simple and deferred immediate assertions with severity tasks. Built with `verilator --binary --timing --assert imm.sv`.

```

[1] -Info: imm.sv:13: TOP.imm.a_imm: y=9 as expected
[2] %Warning: imm.sv:16: TOP.imm: warning: y=3, sel=1
done at 2
- imm.sv:19: Verilog $finish

```

Verilator 5.020 output. The labelled assertion reports its hierarchical name `TOP.imm.a_imm`; the unlabelled one only its scope.

PITFALL: ASSERTIONS ARE SILENTLY OFF UNLESS YOU ENABLE THEM

Verilator compiles assertions only with `--assert` (without it the run above prints nothing but `done at 2`). Commercial simulators enable them by default but offer switches to disable them (`-assert disable`, `-nosva`, `+nosva...`), and regressions have shipped for months with assertions off. Add a deliberately failing smoke assertion to a sanity test so the flow proves the checkers are alive. Icarus Verilog 12 accepts unlabelled simple immediate assertions (and prints `INFO:/WARNING:` lines for the same test) but rejects labels on them and the deferred forms `assert #0` and `assert final` with a syntax error.

Deferred immediate assertions: #0 and final

A simple immediate assertion inside `always_comb` is fragile: a combinational process may execute several times in one time step while its inputs settle (a 0-delay glitch), and an intermediate evaluation can fail even though the final, settled value is correct. **Deferred** immediate assertions solve this. When a deferred assertion executes, its result is not reported at once: the failure (or pass) report is placed in a deferred queue for that process. If the process executes again in the same time step, the queue is **flushed** and the new evaluation replaces the old one. Only the evaluation that survives to the end matures and is reported.

Form	Since	Report matures in	Action block rules
<code>assert #0 (e) (observed deferred)</code>	1800-2009	Observed region; the action runs in the Reactive region, so it sees values after the Active/NBA settling of this time slot	A single subroutine call (e.g. <code>\$error(...)</code>), arguments are evaluated when the assertion executes
<code>assert final (e) (final deferred)</code>	1800-2012	Postponed region - after everything, including program/testbench reactions, has settled	A single subroutine call; no time-consuming or state-changing actions are allowed (it runs in Postponed)
<code>assert (e) (simple)</code>	1800-2005	Immediately, when executed	Any statement

A deferred assertion may also be written directly as a module item, outside any procedural block; it then behaves as if it were inside its own `always_comb`. That makes `a_onehot: assert final ($onehot0(grant));` a one-line combinational checker that is immune to delta-cycle glitches. In the SoC flow, deferred assertions are the right tool for combinational invariants in RTL (one-hot selects, mutually exclusive enables, legal encodings) that are not tied to any clock, e.g. inside an interconnect decoder.

INTERVIEW INSIGHT: WHY THE ACTION BLOCK IS RESTRICTED

Because a deferred report can be flushed, its action cannot be an arbitrary statement whose side effects might already have happened for an evaluation that is later cancelled. The LRM therefore allows only one subroutine call, whose arguments are captured when the assertion executes and which is called only when the report matures.

22.3 Concurrent assertions and the scheduler

A concurrent assertion describes behaviour over time relative to a **clock**. At every tick of that clock a new **attempt** starts; an attempt may take many ticks to complete, so many attempts are in flight at once. Three regions of the time slot (Chapter 5) matter:

one time slot of the clock edge					
Preponed	Active	NBA	Observed	Reactive	Postponed
sample all values used by properties	RTL runs, clock edge happens	flops update	evaluate properties (on samples)	pass/fail action blocks (TB reacts)	\$strobe, assert final reports

Concurrent assertions use the values sampled in Preponed (the values just before the clock edge), evaluate in Observed and run their action blocks in Reactive.

Because of **sampling**, a concurrent assertion sees exactly what a flip-flop sees: the value a signal had just before the clock edge. This makes assertions race-free with respect to the RTL they check and consistent with formal tools, which reason on the same cycle-based model. The flip side is the most common beginner confusion: an assertion that fails 'one cycle late' in the waveform is usually correct - it reports at the edge where the sampled values violate the rule, and the failing values are the ones plotted just **before** that edge.

Clocking a property

How the clock is specified	Example	Notes
Explicit clocking event	<code>assert property @(posedge clk) a > b;</code>	Most portable; any event expression, e.g. <code>@(posedge clk iff en)</code>
Inside a named property/sequence	<code>property p; @(posedge clk) a > b; endproperty</code>	The declaration carries its clock
Default clocking	<code>default clocking cb @(posedge clk); endclocking</code>	Applies to all un-clocked assertions in the module, interface or checker (one default per scope)
Inferred from a procedure	<code>always @(posedge clk) if (en) assert property (a > b);</code>	Procedural concurrent assertion: clock is inferred from the <code>always</code> , the enclosing <code>if</code> becomes an implicit antecedent condition
Clocking block event	<code>assert property @(cb) ...)</code>	Uses the event of a clocking block (Chapter 21)

The example below uses the explicit form and the constructs that Verilator 5.020 executes: Boolean antecedents and consequents joined by `|>` or `|>=`, `disable iff`, `$past`, `$rose` and `$stable`. The stimulus deliberately breaks each rule once.

```
module top;
  logic clk = 0, rst_n = 0;
  logic req = 0, ack = 0, valid = 0, ready = 0;
```



```

logic [7:0] data = 0;
always #5 clk = ~clk;

// req must be answered by ack on the next cycle
a_req_ack: assert property (@(posedge clk) disable iff (!rst_n) req |=> ack)
    else $error("req at %0t not acked", $time);

// valid held until ready, data stable while stalled
a_stable: assert property (@(posedge clk) disable iff (!rst_n)
    valid && !ready |=> valid && $stable(data))
    else $error("stall rule broken at %0t", $time);

// on a rising edge of ack, previous-cycle req must have been 1
a_rose: assert property (@(posedge clk) $rose(ack) |-> $past(req))
    else $error("spurious ack at %0t", $time);

c_hs: cover property (@(posedge clk) valid && ready);

initial begin
    repeat (2) @(negedge clk); rst_n = 1;
    @(negedge clk) req = 1;
    @(negedge clk) begin req = 0; ack = 1; end
    @(negedge clk) ack = 0;
    @(negedge clk) req = 1;           // cycle: no ack follows -> failure
    @(negedge clk) req = 0;
    @(negedge clk) begin valid = 1; data = 8'hA5; end
    @(negedge clk) data = 8'h5A;     // changes while stalled -> failure
    @(negedge clk) ready = 1;
    @(negedge clk) begin valid = 0; ready = 0; end
    @(negedge clk) ack = 1;           // spurious ack
    @(negedge clk) ack = 0;
    #20 $finish;
end
endmodule

```

top.sv - three concurrent assertions and a cover. Run: `verilator --binary --timing --assert top.sv && ./obj_dir/Vtop +verilator+error+limit+100.`

```

[75] %Error: top.sv:9: Assertion failed in TOP.top.a_req_ack: req at 75 not acked
[95] %Error: top.sv:14: Assertion failed in TOP.top.a_stable: stall rule broken at 95
[125] %Error: top.sv:18: Assertion failed in TOP.top.a_rose: spurious ack at 125
- top.sv:35: Verilog $finish

```

Verilator 5.020 output (the '-Info: ... \$stop ignored' line printed after each failure is omitted). Without `+verilator+error+limit` the first `$error` stops the run.

Read the times carefully. The clock rises at 5, 15, 25, ... The second `req` pulse is driven at 60 and sampled high at the edge at 65; `|=>` requires `ack` at the **next** edge, 75, where the sampled `ack` is 0 - so the attempt that started at 65 fails at 75. The data is changed at 90 while `valid && !ready` was sampled at 85, so `$stable(data)` is false at 95. The spurious `ack` is driven at 120: at 125 `$rose(ack)` is true but `$past(req)` (the value sampled at 115) is 0. Note also that `c_hs` produced no text: `cover` directives are counted into the coverage database, not printed.

22.4 Sequences: delays and repetition

A **sequence** is a pattern of Boolean expressions over consecutive clock ticks. A sequence **matches** over an interval of ticks (possibly several different intervals: a sequence can have multiple matches from one start point). The building blocks are cycle delays and repetition operators.

Operator	Meaning	Example and match
<code>a ##1 b</code>	b one tick after a	<code>a@t, b@t+1</code>

Operator	Meaning	Example and match
<code>a ##0 b</code>	Fusion: b at the same tick as the end of a	a and b both @t
<code>a ##n b</code>	b exactly n ticks after a (n constant >= 0)	<code>req ##2 gnt: gnt@t+2</code>
<code>a ##[m:n] b</code>	b between m and n ticks after a; one match per tick where b holds	<code>req ##[1:3] gnt</code>
<code>a ##[m:\$] b</code>	Unbounded upper limit: b at some tick >= m later	Weak by default in assertions (see strong/weak)
<code>##[*] / ##[+]</code>	Shorthand for <code>##[0:\$] / ##[1:\$]</code>	<code>start ##[+] done</code>
<code>b[*n]</code>	Consecutive repetition: b on n consecutive ticks	<code>busy[*3] = busy ##1 busy ##1 busy</code>
<code>b[*m:n], b[*], b[+]</code>	Consecutive, m to n times; <code>[*] = [*0:\$]</code> ; <code>[+] = [*1:\$]</code>	<code>[*0]</code> is the empty sequence
<code>b[->n]</code>	Goto repetition: n (not necessarily consecutive) occurrences of b; the match ends at the n-th b	<code>a ##1 b[->2] ##1 c: c right after 2nd b</code>
<code>b[=n]</code>	Non-consecutive repetition: n occurrences of b; the match may extend past the n-th b over ticks where b is false	<code>a ##1 b[=2] ##1 c: c any time after 2nd b, with no 3rd b before it</code>

tick:	1	2	3	4	5	6	7
b:	1	0	1	0	0	1	0
<code>b[*2]</code>	no match starting at 1 (b is 0 at tick 2)						
<code>b[->2]</code>	from tick 1: matches exactly [1..3] (ends ON the 2nd b)						
<code>b[=2]</code>	from tick 1: matches [1..3], [1..4] and [1..5] (may continue while b stays 0; stops before the 3rd b at 6)						
<code>a ##1 b[->2] ##1 c</code>	: c must be true at the tick right after the 2nd b						
<code>a ##1 b[=2] ##1 c</code>	: c may come later, but before a 3rd b occurs						

Goto versus non-consecutive repetition on the same waveform, starting at tick 1.

Precisely: `b[->n]` is defined as `(!b[*0:$] ##1 b)[*n]` - each occurrence is preceded by any number of ticks with `b` false and ends on a tick with `b` true. `b[=n]` is `b[->n] ##1 !b[*0:$]`, i.e. goto repetition followed by any number of ticks without `b`. Delay and repetition bounds must be elaboration-time constants (parameters, literals, \$); you cannot write `##[1:max_lat]` with a variable. When a latency is programmable, check it with a local variable counter (shown later) or generate one assertion per legal value.

NAMED SEQUENCES ARE REUSABLE, PARAMETERISED BUILDING BLOCKS

`sequence s_hs(v, r); v ##0 r; endsequence` declares a sequence with formal arguments; arguments may be untyped (substituted textually, so they can be expressions, events or other sequences) or typed. Named sequences can be instantiated inside other sequences and properties, and expose the methods `s.triggered` (the sequence reached its end point in the current time step; usable in procedural code and in `wait (s.triggered)`) and `s.matched` (for synchronising across clock domains).

22.5 Sequence operators

Operator	Semantics (both operands are sequences unless noted)
<code>s1 and s2</code>	Both start at the same tick and both match; they may end at different ticks, and the composite match ends when the later one ends
<code>s1 intersect s2</code>	Like and , but both must match over the same interval (same start and same end tick). Used to bound length: <code>s intersect (1'b1[*1:8])</code> = s completes within 8 ticks
<code>s1 or s2</code>	At least one of them matches; the composite has the union of their matches
<code>s1 within s2</code>	s1 matches somewhere inside the interval of a match of s2 (s1 starts no earlier and ends no later than s2)

Operator	Semantics (both operands are sequences unless noted)
<code>e throughout s</code>	Boolean <code>e</code> is true on every tick of the match of <code>s</code> . <code>e</code> must be an expression, not a sequence
<code>first_match(s)</code>	Keeps only the earliest-ending match(es) of <code>s</code> ; discards later matches from the same start tick
<code>s1 ##n s2</code>	Concatenation (see delays)
<code>e[*n]</code> , <code>[->]</code> , <code>[=]</code>	Repetition; <code>[->]</code> / <code>[=]</code> operate on Boolean expressions only, <code>[*]</code> works on sequences too

```
// A burst of exactly 4 data beats while 'grant' stays high the whole time
sequence s_burst;
  start ##1 (grant throughout (beat[->4]));
endsequence

// The response must arrive 1..15 cycles after the request (intersect bounds the length)
property p_resp_bound;
  @(posedge clk) req |-> (##[1:$] resp) intersect (1'b1[*1:16]);
endproperty

// Only the FIRST ack after a request counts as its completion
property p_first_ack;
  @(posedge clk) first_match(req ##[1:8] ack) |> idle;
endproperty

// An error pulse must never be seen inside a transfer
property p_no_err_in_xfer;
  @(posedge clk) not (err[->1] within (sof ##1 !eof[*0:$] ##1 eof));
endproperty
```

Sequence operators in typical DV properties (IEEE 1800-2017; needs a simulator with full SVA support - Verilator 5.020 rejects `##` delays and repetition operators as unsupported).

PITFALL: UNBOUNDED RANGES IN AN ANTECEDENT WITHOUT FIRST_MATCH

`a ##[1:$] b |-> c` starts a separate check for **every** match of the antecedent, i.e. for every later tick where `b` is true, and the attempt never finishes while `b` can still occur. That costs simulation performance and gives surprising multiple failures. Write `first_match(a ##[1:$] b) |-> c` when you mean the first occurrence, or bound the range.

22.6 Implication and vacuity

Most properties are **implications**: `antecedent |-> consequent`. The antecedent is a sequence; for each of its matches, the consequent property must hold. With `|->` (**overlapping**) the consequent starts at the tick where the antecedent match ends; with `|=>` (**non-overlapping**) it starts one tick later, i.e. `s |=> p` is exactly `s ##1 1'b1 |-> p`.

If the antecedent has no match, the implication is **vacuously true**: nothing was required, so nothing failed. Vacuity is essential (a `req |-> ##1 ack` property must not fail on cycles without `req`) but also dangerous: an assertion whose antecedent never occurs in any test passes forever and proves nothing. Tools report vacuous and non-vacuous passes separately; always pair important assertions with a `cover property` on the antecedent (or on the whole sequence) and check the cover is hit in regression.

Result of an attempt	Meaning
Non-vacuous pass	Antecedent matched and the consequent held
Vacuous pass	Antecedent never matched - no obligation arose
Failure	Antecedent matched and the consequent was violated
Disabled	<code>disable iff</code> became true during the attempt - neither pass nor fail

Result of an attempt	Meaning
Incomplete / unfinished	Simulation ended before the attempt finished: a failure for a strong property, not a failure for a weak one

WEAK AND STRONG

A property is **weak** if it only requires that no bad thing is seen within the observed trace, and **strong** if it also requires the good thing to actually happen before the trace ends. `req |-> ##[1:$] ack` is weak by default: if simulation ends while waiting for `ack`, no failure is reported. Write `req |-> strong(##[1:$] ack)` or `req |-> s_eventually ack` to require the `ack`. A sequence used as a property is weak in `assert/assume` and strong in `cover` unless wrapped in `strong()/weak()`. In formal verification strong properties are liveness properties, which need fairness assumptions and different proof engines.

22.7 Property operators

Operator	Meaning
<code>not p</code>	p does not hold (swaps weak/strong: <code>not</code> of a weak property is strong)
<code>p1 and p2, p1 or p2</code>	Property-level conjunction/disjunction
<code>p1 implies p2, p1 iff p2</code>	Property-level logic (1800-2009)
<code>if (e) p1 else p2</code>	Choose the property to check from a Boolean at the start tick; <code>case</code> is also allowed
<code>nexttime p, nexttime [n] p</code>	p holds at the next (n-th next) tick; weak - passes if there is no next tick
<code>s_nexttime p</code>	Strong: the next tick must exist
<code>always p, always [m:n] p</code>	p holds at every tick (in the range). Weak form; the range may be unbounded
<code>s_always [m:n] p</code>	Strong, bounded range only
<code>s_eventually p, s_eventually [m:\$] p</code>	p holds at some tick (strong: must actually happen)
<code>eventually [m:n] p</code>	Weak form, bounded range required
<code>p until q</code>	p holds on every tick until (not including) the tick where q holds; weak - q need never occur
<code>p s_until q</code>	Strong: q must eventually hold
<code>p until_with q, s_until_with</code>	p holds up to and including the tick where q holds
<code>s #-# p, s ==# p</code>	Followed-by: the dual of implication; requires a match of s (no vacuous success)
<code>accept_on (e) p, reject_on (e) p</code>	Abort: e true -> the attempt passes / fails immediately (sync_ variants sample e)

```
// Once 'lock' is taken it is held until 'unlock', inclusive of that cycle
a_lock: assert property (@(posedge clk) $rose(lock) |-> lock until_with unlock);

// Grant must eventually follow request (liveness - strong)
a_live: assert property (@(posedge clk) req |-> s_eventually gnt);

// Mode-dependent latency
a_mode: assert property (@(posedge clk)
    start |-> if (fast) ##1 done else ##4 done);

// After reset release, 'ready' rises within 8 cycles and then stays high forever
// (-# is 'followed-by': the sequence must match, then the property must hold)
a_boot: assert property (@(posedge clk) $rose(rst_n) |->
    first_match(##[0:8] ready) #-# always ready);
```

Property operators (IEEE 1800-2017, commercial simulator or formal tool).

22.8 Sampled-value functions

Sampled-value functions let a Boolean expression refer to the history of a signal. They use the **sampled** value at each tick of the clock of the property that contains them (or the explicit clock argument), so they are safe to use in assertions and also callable from procedural code with an explicit clock.

Function	Returns true / value	Notes
<code>\$sampled(e)</code>	The sampled value of e	Implicit inside assertions; useful in action blocks to print what was checked
<code>\$rose(e)</code>	LSB of e changed to 1 since the previous tick	Only the LSB: <code>\$rose(bus)</code> for a vector is a common bug; X->1 counts as a rise
<code>\$fell(e)</code>	LSB changed to 0	
<code>\$stable(e)</code>	Whole value unchanged since the previous tick	Compares all bits (with <code>===</code> -like 4-state comparison)
<code>\$changed(e)</code>	<code>!\$stable(e)</code>	1800-2009
<code>\$past(e, n, gate, clk)</code>	Value of e n ticks ago (n default 1)	With <code>gate</code> , counts only ticks where gate was true
<code>\$rose_gclk, \$future_gclk...</code>	Global-clocking variants	Used mainly in formal, relative to global clocking

At the first tick there is no history; `$past` then returns the value the expression would have had before the first sample (the declaration initial value, typically X for 4-state variables), and `$rose/$fell/$stable` compare against that. Guard assertions that use `$past` with the reset (`disable iff`) or with a `past_valid` flag so the first cycles after reset do not report garbage. The gating argument is how you express 'the value when the pipeline last advanced': `$past(din, 1, en)` is the value of `din` at the previous tick where `en` was high - exactly what an enable-gated register holds, so `q == $past(din, 1, en)` is a one-line reference model of such a register.

22.9 disable iff and reset

`disable iff (cond)` is the asynchronous abort of a property. Unlike every other expression in a concurrent assertion, `cond` is evaluated on **current**, not sampled, values: as soon as it becomes true, every in-flight attempt is **disabled** (reported as neither pass nor fail) and no new attempt produces a result while it stays true. It may appear only once, at the top of a property (not in sequences, not nested). A scope-wide default is available:

```
module axi_checks (input logic aclk, aresetn /* ... */);
  default clocking cb @(posedge aclk); endclocking
  default disable iff (!aresetn);           // applies to every assertion below

  a_awvalid_hold: assert property (awvalid && !awready |=> awvalid);
  a_override:      assert property (disable iff (1'b0) !aresetn |-> !awvalid);
endmodule
```

Default clocking and default disable iff (1800-2009). An explicit `disable iff` in a property overrides the default. Verilator 5.020 reports 'Unsupported: default disable iff'.

PITFALL: CHECKING RESET BEHAVIOUR WITH A RESET-DISABLED PROPERTY

A property such as `!aresetn |-> !awvalid` (AXI: VALID must be low during reset) can never fail if it inherits `disable iff (!aresetn)`; it is disabled exactly when it matters. Override with `disable iff (1'b0)`, as `a_override` does. Conversely, a `$past`-based property evaluated just after reset release may read pre-reset X values: disable it one extra cycle, or AND the antecedent with `$past(aresetn)`.

22.10 Local variables

A sequence or property can declare **local variables** that capture a value at one point of a match and compare it later. They are assigned by a comma-separated list after a sub-sequence in parentheses: `(expr, v = data, n = n + 1)`. Each attempt (indeed each thread of each attempt) has its own copy, so overlapping transactions do not interfere. This is how SVA checks data integrity through a pipeline without writing a scoreboard.

```
// The value entering a 3-stage pipeline must appear at the output 3 cycles later
property p_pipe_data;
  logic [31:0] v;
  @(posedge clk) disable iff (!rst_n)
    (in_valid, v = in_data) |-> ##3 (out_valid && out_data == v);
endproperty
a_pipe: assert property (p_pipe_data);

// Programmable latency: count cycles with a local variable (bounds need not be constants)
property p_prog_lat;
  int cnt;
  @(posedge clk) (start, cnt = cfg_lat) |->
    (cnt > 0, cnt = cnt - 1)[*0:$] ##1 (cnt == 0 && done);
endproperty

// Tag matching for out-of-order responses (ID-based)
property p_id_resp;
  logic [3:0] id;
  @(posedge clk) (arvalid && arready, id = arid) |->
    ##[1:64] (rvalid && rready && rlast && rid == id);
endproperty
```

Local variables in properties (IEEE 1800-2017; Verilator 5.020 fails to parse a local variable declaration in a property).

Rules worth knowing: local variables cannot be referenced in `disable iff` or in action blocks directly (copy to a module variable via a subroutine call in a match item, e.g. `(a, v = d, log(v))`); a local variable assigned in one branch of `or` is not visible after the `or` unless it is assigned in both branches; and they may be passed as arguments to named sequences declared with `local input` or `local inout` formals.

22.11 Multi-clock sequences and properties

A sequence can change clocks at a concatenation or implication. The boundary uses `##1` (the next tick of the new clock **strictly after** the previous match ends) or `##0` (the new clock tick that coincides with, or follows, the end point); other delay values are not allowed across a clock change.

```
// A pulse captured in the fast domain must be seen in the slow domain
a_cdc_pulse: assert property (
  @(posedge clk_fast) $rose(evt_fast) |=> @(posedge clk_slow) ##[0:2] evt_slow);

// Multiply-clocked sequence: request in domain A, ack in domain B
sequence s_req_ack_x;
  @(posedge clk_a) req ##1 @(posedge clk_b) ack;
endsequence
```

Multi-clock properties (commercial simulators and formal tools).

Multi-clock assertions are used on CDC boundaries, but they check the protocol only - they cannot detect metastability. Structural CDC checking (synchroniser recognition, reconvergence) is the job of a CDC tool; the companion RTL Design guide treats CDC design itself.

22.12 assert, assume, cover, restrict and action blocks

Directive	In simulation	In formal
<code>assert property (p)</code>	Checked; failures run the else-action (default <code>\$error</code>)	Proof target: prove it holds for all legal inputs, or produce a counter-example trace
<code>assume property (p)</code>	Checked like an assertion (reports environment violations)	Constraint: only input traces that satisfy p are explored
<code>cover property (p) / cover sequence (s)</code>	Counts matches (the pass action runs on each); reported in coverage	Reachability: find a trace that reaches p, or prove it unreachable
<code>restrict property (p)</code>	Ignored	Constraint like assume, but never checked in simulation; no action block allowed

The action block syntax is `assert property (p) pass_stmt; else fail_stmt;` - the pass statement is rarely used because it runs on every successful attempt, including vacuous ones on some tools. The fail statement typically calls a severity task with a message built from `$sampled()` values. Because the action runs in the Reactive region, it may also increment a testbench error counter, trigger an event, or call a function that dumps extra state. Assume/assert duality is the backbone of **assume-guarantee** reasoning in formal: the same property file is used as `assume` when verifying the consumer and as `assert` when verifying the producer, usually selected by a macro or a parameter.

SEVERITY POLICY IN A REAL FLOW

Protocol violations by the DUT: `$error` (counted, the test fails at the end, other checks still run). Environment/assumption violations: `$fatal` (the stimulus is illegal, nothing after it can be trusted). Informational covers: no message at all. UVM flows often route the else-branch through the `uvm_error` macro so assertion failures appear in the same report summary and obey the same verbosity/severity overrides as the rest of the testbench.

22.13 bind, checkers and assertion control

bind: attaching assertions without touching the RTL

`bind` instantiates a module, interface, program or checker **inside** another module as if the instantiation had been written in its body. The target can be a module name (every instance), specific instances (`bind fifo:u_rx_fifo,u_tx_fifo ...`) or a hierarchical instance path. Port connections are resolved in the scope of the target, so the checker can see internal signals such as a FIFO's `count`. This is how DV teams keep assertion IP in their own files, compile it only in verification builds, and never risk changing the RTL that goes to synthesis.

```
// A 4-entry FIFO with a deliberate bug: 'full' is computed one entry too late.
module fifo #(parameter int DEPTH = 4, parameter int W = 8) (
    input logic      clk, rst_n,
    input logic      push, pop,
    input logic [W-1:0] din,
    output logic [W-1:0] dout,
    output logic      full, empty
);
    logic [W-1:0] mem [DEPTH];
    logic [2:0]    count;
    logic [1:0]    wp, rp;
    assign full = (count > DEPTH);           // BUG: should be (count == DEPTH)
    assign empty = (count == 0);
    assign dout = mem[rp];
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin count <= '0; wp <= '0; rp <= '0; end
    else begin
```



```

        if (push && !full) begin mem[wp] <= din; wp <= wp + 1'b1; end
        if (pop && !empty) rp <= rp + 1'b1;
        count <= count + 3'(push && !full) - 3'(pop && !empty);
    end
endmodule

```

fifo.sv - the design under test (RTL, owned by the designer).

```

// Protocol checker: written by DV, attached with 'bind' - the RTL is untouched.
module fifo_sva #(parameter int DEPTH = 4) (
    input logic clk, rst_n, push, pop, full, empty,
    input logic [2:0] count
);
    default clocking cb @(posedge clk); endclocking

    a_no_overflow: assert property (disable iff (!rst_n) count <= DEPTH)
        else $error("overflow: count=%0d", count);
    a_full_flag:    assert property (disable iff (!rst_n) full == (count == DEPTH))
        else $error("full=%b count=%0d", full, count);
    a_not_both:     assert property (disable iff (!rst_n) !(full && empty));
    a_empty_stable: assert property (disable iff (!rst_n) empty && !push | => empty);
    c_full:         cover property (disable iff (!rst_n) full);
endmodule

bind fifo fifo_sva #(.DEPTH(DEPTH)) chk (. *);

```

sva.sv - the checker and the bind statement. `. *` connects each checker port to the signal of the same name inside `fifo`, including the internal `count`; `DEPTH` is resolved in the target's scope.

```

module tb;
    logic clk = 0, rst_n = 0, push = 0, pop = 0;
    logic [7:0] din = 0, dout;
    logic full, empty;
    always #5 clk = ~clk;
    fifo dut (. *);
    initial begin
        repeat (2) @(negedge clk); rst_n = 1;
        push = 1;
        repeat (6) @(negedge clk) din++;
        push = 0;
        @(negedge clk) $finish;
    end
endmodule

```

tb.sv - pushes six words into the 4-entry FIFO. Run: `verilator --binary --timing --assert --top-module tb tb.sv fifo.sv sva.sv`.

```

[65] %Error: sva.sv:11: Assertion failed in TOP.tb.dut.chk.a_full_flag: full=0 count=4
[75] %Error: sva.sv:9: Assertion failed in TOP.tb.dut.chk.a_no_overflow: overflow: count=5
[75] %Error: sva.sv:11: Assertion failed in TOP.tb.dut.chk.a_full_flag: full=1 count=5
[85] %Error: sva.sv:9: Assertion failed in TOP.tb.dut.chk.a_no_overflow: overflow: count=5
[85] %Error: sva.sv:11: Assertion failed in TOP.tb.dut.chk.a_full_flag: full=1 count=5
- tb.sv:12: Verilog $finish

```

Verilator 5.020 (with +verilator+error+limit+100; the '-Info' lines are omitted). The checker instance appears as `TOP.tb.dut.chk` - inside the DUT, although the DUT source never mentions it.

The first failure pinpoints the bug exactly: at the edge at 65 the FIFO holds four entries and `full` is still 0, so the fifth push is accepted and the FIFO overflows at 75 (overwriting entry 0 because the 2-bit write pointer wrapped). An end-to-end scoreboard would only have seen a missing word much later, after a pop.

The checker construct

A **checker** (1800-2009) is a container designed for assertion IP. Compared with a module it can be instantiated **inside procedural code** (the enclosing `always` supplies the clock and enable), its formal arguments may be sequences, properties or untyped expressions, it can infer the clock and reset of its context

with `$inferred_clock` and `$inferred_disable`, and it may declare **rand free variables** that formal tools treat as unconstrained inputs (useful for abstract models such as 'pick any address and track it').

```
checker c_valid_ready (logic valid, logic ready, logic [63:0] payload,
                      event clk = $inferred_clock,
                      untyped rst = $inferred_disable);
  default clocking @clk; endclocking
  default disable iff (rst);
  a_hold:  assert property (valid && !ready | => valid);
  a_stable: assert property (valid && !ready | => $stable(payload));
  c_stall:  cover  property (valid && !ready ##1 valid && ready);
endchecker

module producer (input logic clk, rst, /* ... */);
  default clocking @(posedge clk); endclocking
  default disable iff (rst);
  // clock and reset inferred from the defaults above
  c_valid_ready u_chk (m_valid, m_ready, m_data);
endmodule
```

A reusable handshake checker (IEEE 1800-2017; checkers are not supported by Icarus or Verilator 5.020).

Assertion control system tasks

Task	Effect
<code>\$assertoff(levels, scope_or_assertion, ...)</code>	Stop checking: new attempts are not started; in-flight attempts continue
<code>\$assertkill(...)</code>	Like <code>\$assertoff</code> , and also abort all attempts currently in flight
<code>\$asserton(...)</code>	Resume checking
<code>\$assertpassoff/on, \$assertfailoff/on, \$assertvacuousoff</code>	Enable/disable the pass or fail action blocks, or vacuous-pass actions (1800-2009)
<code>\$assertcontrol(control_type, assertion_type, directive_type, levels, list)</code>	The general form (1800-2012) of all of the above; can target only asserts, only covers, only concurrent, etc.

`levels` works like `$dumpvars`: 0 means the named scope and everything below it, n means n levels. A typical use is `initial begin $assertoff(0, tb.dut); @(posedge init_done); $asserton(0, tb.dut); end` to silence checks during a power-on/initialisation phase, or turning off a known-broken check for one test while a bug is fixed. Neither Icarus nor Verilator 5.020 implement these tasks (Verilator: 'Unsupported or unknown PLI call: \$assertoff').

22.14 A library of SoC protocol assertions

The same dozen rule shapes appear on every SoC interface. The block below is written to IEEE 1800-2017 and is typical of what DV teams keep in an assertion package bound onto every instance of an interface. Rule numbers refer to the AMBA AXI specification (ARM IHI 0022), whose handshake rules apply equally to AXI-Stream and most valid/ready interfaces.

```
module axi_hs_sva #(parameter int W = 64) (
  input logic aclk, aresetn,
  input logic valid, ready, last,
  input logic [W-1:0] payload
);
  default clocking cb @(posedge aclk); endclocking
  default disable iff (!aresetn);

  // AXI A3.2.1: once VALID is asserted it stays asserted until the handshake
  a_valid_hold:  assert property (valid && !ready | => valid);
```

```

// Payload (and LAST) must not change while waiting for READY
a_payload_stbl: assert property (valid && !ready |>= $stable({payload, last}));
// No X on control; no X on payload while VALID
a_valid_known: assert property (!$isunknown(valid));
a_pl_known:    assert property (valid |>= !$isunknown(payload));
// A3.1.2: VALID low during reset (reset-disabled default must be overridden)
a_reset_valid: assert property (disable iff (1'b0) !aresetn |>= !valid);
// Coverage of the interesting handshake shapes
c_back2back:   cover property (valid && ready ##1 valid && ready);
c_stall3:      cover property ((valid && !ready)[*3] ##1 ready);
endmodule

module soc_misc_sva (input logic clk, rst_n, req, ack, push, pop, full, empty,
                    input logic [7:0] gnt, input logic [2:0] fsm);
    default clocking cb @(posedge clk); endclocking
    default disable iff (!rst_n);
    a_gnt_onehot: assert property ($onehot0(gnt));           // arbiter grant
    a_req_ack_lat: assert property (req |>= ##[1:4] ack);    // bounded latency
    a_ack_has_req: assert property (ack |>= $past(req) || $past(req, 2) ||
                                   $past(req, 3) || $past(req, 4));
    a_req_held:    assert property (req && !ack |>= req);    // 4-phase handshake
    a_no_ovf:      assert property (full |>= !push || pop);
    a_no_unf:      assert property (empty |>= !pop);
    a_fsm_legal:   assert property (fsm inside {3'd0, 3'd1, 3'd2, 3'd4});
    a_fsm_nodead:  assert property (fsm == 3'd4 |>= s_eventually fsm == 3'd0);
endmodule

```

A small SoC assertion library (IEEE 1800-2017; commercial simulator or formal tool).

When only an open-source simulator is available, many of these rules can be rewritten in the subset Verilator 5.020 executes: a fixed delay `req |>= ##2 ack` becomes `$past(req, 2) |>= ack`, and pure invariants need no temporal operator at all. The run below also illustrates an important limitation of 2-state simulation.

```

module top;
    logic clk = 0, rst_n = 0, req = 0, ack = 0;
    logic [3:0] gnt = '0;
    always #5 clk = ~clk;

    // Portable form of "req |>= ##2 ack" for tools without sequence support:
    a_lat2: assert property (@(posedge clk) disable iff (!rst_n) $past(req, 2) |>= ack)
        else $error("ack missing 2 cycles after req");
    a_onehot: assert property (@(posedge clk) disable iff (!rst_n) $onehot0(gnt))
        else $error("gnt=%b not one-hot-0", gnt);
    a_known: assert property (@(posedge clk) disable iff (!rst_n) !$isunknown(gnt))
        else $error("gnt has X/Z: %b", gnt);

    initial begin
        repeat (3) @(negedge clk); rst_n = 1;
        req = 1; @(negedge clk) req = 0;
        @(negedge clk) ack = 1; @(negedge clk) ack = 0;           // latency 2: pass
        req = 1; @(negedge clk) req = 0;
        @(negedge clk); @(negedge clk) ack = 1;                 // latency 3: fail
        @(negedge clk) begin ack = 0; gnt = 4'b0110; end         // two grants
        @(negedge clk) gnt = 4'b00x0;                             // X on the bus
        @(negedge clk) gnt = 4'b0000;
        @(negedge clk) $finish;
    end
endmodule

```

Portable forms of latency and one-hot checks (run with Verilator 5.020).

```
[85] %Error: top.sv:8: Assertion failed in TOP.top.a_lat2: ack missing 2 cycles after req
[105] %Error: top.sv:10: Assertion failed in TOP.top.a_onehot: gnt=0110 not one-hot-0
- top.sv:23: Verilog $finish
```

Verilator 5.020 output ('-Info' lines omitted). The X injected on `gnt` at 110 is NOT reported.

PITFALL: X-CHECKS ARE MEANINGLESS IN A 2-STATE SIMULATOR

Verilator is a 2-state simulator: the assignment `gnt = 4'b00x0` stores 0 in place of X, so `a_known` can never fail. A 4-state simulator reports it at the edge at 115. X-propagation checks, reset-value checks and `$isunknown` assertions must run on a 4-state simulator (or on formal tools with X modelling). Also note that `$past(req,2) |-> ack` is equivalent to `req |-> ##2 ack` only away from reset: the `$past` form reads values from inside the reset window during the first two cycles after release.

22.15 Tool support at a glance

The following matrix was established by compiling one-line properties with Verilator 5.020 (`--lint-only --assert --timing`) and Icarus Verilog 12 in this environment. Commercial simulators and formal tools support the full clause 16 language.

Construct	Verilator 5.020	Icarus 12
Simple immediate <code>assert</code> + severity tasks	Yes (<code>--assert</code>)	Yes, unlabelled only
<code>assert #0, assert final</code>	Yes	No (syntax error)
Boolean <code> -></code>, <code> =></code>, <code>not</code>, <code>disable iff</code>	Yes	No
<code>\$past(e,n), \$rose, \$fell, \$stable, \$changed</code>	Yes	No
<code>\$past</code> with gating/clock arguments	No	No
<code>##n, ##[m:n], [*n], [->n], [=n]</code>	No	No
Sequence <code>and/or/intersect/within/throughout/first_match</code>	No	No
<code>always, s_eventually, until, nexttime, if/else</code>	No	No
Local variables in properties	No	No
<code>default disable iff</code>	No	No
<code>bind</code> of a module with assertions	Yes	No concurrent SVA
<code>\$assertoff/on/kill</code>	No	No

22.16 Summary

- Immediate assertions check a Boolean when executed; deferred forms (`#0`, `final`) report only the settled result of a time step and are the right choice in combinational code.
- Concurrent assertions sample in Preponed, evaluate in Observed and act in Reactive; they see what a flop sees, and a new attempt starts on every clock tick.
- Sequences combine delays (`##n`, `##[m:n]`) and repetitions (`[*n]` consecutive, `[->n]` goto ending on the last hit, `[=n]` non-consecutive that may extend past it) with `and`, `or`, `intersect`, `within`, `throughout` and `first_match`.
- `|->` checks the consequent at the antecedent's end tick, `|=>` one tick later; a missing antecedent is a vacuous pass - pair assertions with covers.
- Weak properties may remain unfinished at the end of simulation; strong ones (`s_eventually`, `strong()`, `s_until`) may not.
- `disable iff` uses current values and disables (does not pass) an attempt; local variables carry data through a pipeline check.

- `bind` and `checker` package assertion IP without editing RTL;
`$assertoff/$asserton/$assertkill/$assertcontrol` control it at run time.
- Verilator 5.020 runs only Boolean implication with sampled-value functions; it is 2-state, so X checks need a 4-state simulator.

22.17 Exercises

- 1 For the waveform `b = 1,0,1,0,0,1,0` (ticks 1-7) list every match of `b[*1:2]`, `b[->2]` and `b[=2]` starting at tick 1. Which matches does `first_match(b[=2])` keep?
- 2 Write a property that says 'after `start`, `busy` stays high until and including the cycle where `done` rises, and `done` must come within 20 cycles'. State which parts are weak and which strong.
- 3 Explain why `assert property (@(posedge clk) disable iff (!rst_n) !rst_n |-> !valid);` can never fail, and fix it.
- 4 Rewrite `a_req_ack` from the first concurrent example so that a request must be acknowledged within 1 to 3 cycles, once using `##[1:3]` and once using only constructs Verilator 5.020 accepts (hint: `$past` with several depths). Where do the two forms differ?
- 5 Write a checker with a local variable that verifies a store-and-forward buffer: every word pushed with ID `n` must pop with the same data, in order, within 32 cycles. Add a cover for a full buffer.
- 6 Your regression reports 0 failures and 100% passes for `a_axi_rlast`. What additional data do you need before trusting that result?

23. Functional Coverage

Constrained-random stimulus (Chapter 19) answers the question 'how do I generate a million interesting tests?'; assertions and scoreboards (Chapter 22) answer 'did anything go wrong?'. Neither answers the question every verification sign-off review asks: **what did we actually test?** A random test that passes has proved nothing about the corner cases it never reached. Functional coverage is the language feature that measures this. The engineer writes down, as executable code, the list of scenarios that must be exercised - the **coverage model** - and the simulator counts how often each one occurred across the whole regression.

This chapter covers the `covergroup` language of IEEE 1800 clause 19 in detail: declaration and sampling, coverpoints and every kind of bin, transitions, crosses, options, the coverage calculation and the query API, covergroups in classes, and how coverage drives closure in a real SoC project. None of the open-source tools used in this book executes covergroups (Verilator 5.020 reports 'Unsupported: covergroup'; Icarus Verilog 12 reports a syntax error), so the examples here carry no simulator output. Instead, each one is followed by the result the LRM rules require, worked out by hand and labelled as such; any commercial simulator (VCS, Xcelium, Questa, Riviera-PRO) will report the same numbers.

23.1 Code coverage versus functional coverage

	Code coverage	Functional coverage
Source	Generated automatically by the simulator from the RTL	Written by the verification engineer from the specification
Metrics	Line/statement, branch, condition/expression, toggle, FSM state and arc	Covergroup bins and crosses, <code>cover property</code> / <code>cover sequence</code> hits
Answers	Which parts of the implementation were executed?	Which features and scenarios of the spec were exercised?
Blind spot	Cannot see missing functionality: code that was never written has 100% coverage	Only as good as the plan: scenarios nobody thought of are never measured
Cost	Free (a compile switch); some simulation slowdown	Engineering effort to write and review the model
Typical sign-off target	Line ~100%, branch/condition 95-100% with reviewed exclusions, toggle on ports	100% of the verification plan items, with reviewed waivers

The two are complementary and both are required at sign-off. High code coverage with low functional coverage means the stimulus runs the code but not the scenarios (for example every FIFO line executed but never 'full while a flush arrives'). High functional coverage with low code coverage means the coverage model is incomplete, or there is dead or unreachable RTL that should be removed or waived.

23.2 Declaring and sampling a covergroup

A `covergroup` is a user-defined **type**, like a class: the declaration describes the coverage model, and each `new` creates an **instance** that counts independently. A covergroup can be declared in a module, interface, program, package, or class (Section 23.9). It contains coverpoints, crosses and options, and it specifies how it is sampled:

Sampling style	Syntax	When to use
Clocking event	<code>covergroup cg @(posedge clk);</code>	Signal-level coverage in a module or interface; sampled automatically every event
Block event	<code>covergroup cg @(begin mon.write);</code>	Sample when a named task/function begins or ends execution

Sampling style	Syntax	When to use
Explicit <code>sample()</code>	<code>covergroup cg; ... endgroup</code> then <code>cg_inst.sample();</code>	Transaction-level coverage from a monitor or scoreboard: sample exactly once per transaction
<code>sample()</code> with arguments	<code>covergroup cg with function sample(bit [3:0] len, bit wr);</code>	1800-2009: pass the values to cover, so the group need not reference any variable
Constructor arguments	<code>covergroup cg (ref logic [7:0] v, input int max) @(posedge clk);</code>	Reusable groups: <code>ref</code> arguments are read at every sample, <code>input</code> arguments once at <code>new</code>

```

module bus_cov (input logic clk, rst_n, valid, ready, input logic [1:0] kind,
               input logic [3:0] len);
    // Signal-level: sampled on every clock edge, guarded by iff
    covergroup cg_hs @(posedge clk);
        cp_kind: coverpoint kind iff (rst_n && valid && ready);
        cp_stall: coverpoint (valid && !ready) iff (rst_n);
    endgroup
    cg_hs u_cg_hs = new(); // must be instantiated to count anything

    // Transaction-level: the monitor calls sample() with the values
    covergroup cg_txn with function sample(bit [3:0] len, bit wr);
        cp_len: coverpoint len;
        cp_wr: coverpoint wr;
    endgroup
    cg_txn u_cg_txn = new();
    always @(posedge clk) if (rst_n && valid && ready) u_cg_txn.sample(len, kind[0]);
endmodule

```

Two ways of sampling. `iff (expr)` on a coverpoint (or on a cross) disables sampling of that point when the guard is false.

PITFALL: SAMPLING RACES AND THE FORGOTTEN NEW()

A covergroup sampled by `@(posedge clk)` reads the **current** values when the event fires, in the Active region - it is not Preponed-sampled like an assertion. If the same edge also updates the covered flops, the result depends on process ordering. Sample through a clocking block, on the opposite edge, or with `sample()` from a monitor that has already captured the transaction. And a covergroup that is declared but never constructed with `new` silently contributes nothing - the report still lists the type, with 0 instances.

23.3 Coverpoints and bins

A **coverpoint** names an integral expression to cover (a variable, a part-select, or any expression, e.g. `coverpoint a + b`). Its values are counted into **bins**. A bin is **covered** when its hit count reaches `option.at_least` (default 1), and the coverpoint's coverage is the percentage of its bins that are covered.

Automatic bins

With no bin specification the tool creates automatic bins. For an enum expression, one bin per named value. Otherwise, let N be the number of possible values (2^n for an n-bit expression): if $N \leq \text{option.auto_bin_max}$ (default 64) there is one bin per value, named `auto[0]`, `auto[1]`, ...; if N is larger, the value range is split into `auto_bin_max` bins of equal size (the last one absorbing any remainder), named like `auto[0:3]`. Values containing X or Z are not counted in any bin. Automatic bins are fine for a 1-bit flag or a small enum; for a 32-bit address they are useless (64 bins of 64M addresses each), so real coverage models declare explicit bins.

Explicit bins

```

covergroup cg_len with function sample(bit [3:0] len, bit wr);
  cp_len: coverpoint len {
    bins zero      = {0};           // one bin for one value
    bins lo[]      = {[1:3]};       // array: one bin per value -> lo[1], lo[2], lo[3]
    bins mid[2]    = {[4:9]};       // fixed count: values split into 2 bins
    bins hi        = {[10:14]};     // one bin for a range
    illegal_bins bad = {15};        // length 15 is reserved in this protocol
  }
  cp_wr: coverpoint wr;             // automatic bins auto[0], auto[1]
  x_len_wr: cross cp_len, cp_wr;
endgroup

```

A coverage model with explicit bins (IEEE 1800-2017; commercial simulator). Note: the bin names **small**, **medium** and **large** are illegal - they are Verilog charge-strength keywords.

Bin form	Bins created	Rule
bins zero = {0};	1	All listed values count into one bin
bins lo[] = {[1:3]};	3: lo[1], lo[2], lo[3]	[] creates one bin per value in the set
bins mid[2] = {[4:9]};	2: mid[0] = {4,5,6}, mid[1] = {7,8,9}	Values distributed evenly in order; if not divisible, the last bin gets the remainder (e.g. [3] = {[0:10]} gives {0,1,2}, {3,4,5}, {6..10})
bins hi = {[10:14]};	1	Ranges [lo:hi] ; \$ means the maximum value, e.g. {[8:\$]}
bins others = default;	1 (not counted in coverage)	Catches every value not in any other bin; excluded from the coverage percentage and from crosses
ignore_bins	0	Values removed from all other bins; never counted
illegal_bins	0	Values removed from all bins; sampling one is a run-time error ; takes precedence over all other bins

Value sets may mix values and ranges, {0, [2:5], 7, [12:\$]}, and since 1800-2009 may be given by an expression with a **with** filter, e.g. **bins even[] = {[0:15]} with (item % 2 == 0);**. A value may appear in several bins; it then counts in each of them.

ignore_bins, illegal_bins and wildcard bins

```

covergroup cg_op @(posedge clk iff valid);
  cp_opcode: coverpoint opcode {
    wildcard bins alu    = {8'b0000_????}; // ? (or x, z) = don't care in wildcard bins
    wildcard bins load   = {8'b0100_0???};
    wildcard bins store  = {8'b0100_1???};
    ignore_bins  dbg     = {8'hF0, 8'hF1}; // debug opcodes: not a DV goal
    illegal_bins undef    = {[8'hF8:8'hFF]}; // decoding error if ever seen
    bins         misc    = default;        // everything else, reported but not scored
  }
endgroup

```

Wildcard, ignore and illegal bins (commercial simulator).

ILLEGAL_BINS VERSUS ASSERTIONS

An illegal bin is a convenient way to flag a value that must never occur, but it only fires when the covergroup is sampled, coverage is enabled and the group is instantiated. For a hard design rule, write an assertion too; many teams forbid illegal_bins entirely because coverage collection is sometimes switched off in regression for speed, which silently disables the check.

23.4 Transition bins

Transition bins count sequences of **successive sampled values** of one coverpoint. They are the covergroup counterpart of an SVA sequence, and are mostly used for FSM arcs and protocol orderings.

Syntax	Meaning
<code>(IDLE => BUSY)</code>	Value IDLE at one sample, BUSY at the next
<code>(IDLE => BUSY => DONE)</code>	Three successive samples
<code>(1, 5 => 6, 7)</code>	Set of transitions: 1=>6, 1=>7, 5=>6, 5=>7 (one bin unless declared with <code>[]</code> , then four bins)
<code>(BUSY [*3])</code>	Consecutive repetition: BUSY=>BUSY=>BUSY
<code>(BUSY [*2:4])</code>	BUSY repeated 2 to 4 consecutive samples
<code>(ERR [-> 2])</code>	Goto: ... => ERR => ... => ERR, any other values in between, ending on the second ERR
<code>(ERR [= 2])</code>	Non-consecutive: as goto, but may continue with non-ERR values after the second ERR
<code>bins others = default sequence;</code>	Catches transitions not matched by any other transition bin (not scored)

```
typedef enum logic [1:0] {IDLE, BUSY, WAIT, DONE} st_e;
covergroup cg_fsm @(posedge clk iff rst_n);
  cp_st: coverpoint state {
    bins states[]      = {IDLE, BUSY, WAIT, DONE};
    bins arcs[]        = (IDLE => BUSY), (BUSY => WAIT), (WAIT => BUSY),
                        (BUSY => DONE), (DONE => IDLE);
    bins long_wait     = (WAIT [*4:8]);           // stalled 4..8 samples
    bins retry_twice   = (BUSY => WAIT => BUSY => WAIT => BUSY);
    bins illegal_bins  = (IDLE => DONE);          // must pass through BUSY
  }
endgroup
```

FSM state and arc coverage with transition bins (commercial simulator).

Expected result (by the LRM rules, not a simulation). For the sampled sequence IDLE, BUSY, WAIT, WAIT, BUSY, DONE, IDLE: the four `states[]` bins are all hit; the arcs `arcs[IDLE=>BUSY]`, `arcs[BUSY=>WAIT]`, `arcs[WAIT=>BUSY]`, `arcs[BUSY=>DONE]` and `arcs[DONE=>IDLE]` are each hit once (5 of 5); `long_wait` is not hit (only 2 consecutive WAIT samples); `retry_twice` is not hit. `cp_st` therefore has 4 + 5 + 1 + 1 = 11 bins of which 9 are covered: 9/11 = 81.82%. Note that transitions are measured between **samples**, not clock cycles: if the group is sampled only when `valid` is high, a repeated value may hide cycles where the state changed and changed back.

23.5 Cross coverage

A **cross** counts **combinations** of the bins of two or more coverpoints (or of variables, for which implicit coverpoints with automatic bins are created). By default it has one bin for every combination of the crossed coverpoints' bins, excluding their ignore, illegal and default bins. Crosses are where coverage models explode: three coverpoints with 16, 8 and 10 bins make 1280 cross bins, most of which may be meaningless. User-defined cross bins, built with `binsof` and `intersect`, select, merge or exclude combinations.

Select expression	Selects the cross products whose ...
<code>binsof(cp_len)</code>	<code>cp_len</code> component is any bin of <code>cp_len</code>
<code>binsof(cp_len.hi)</code>	<code>cp_len</code> component is the bin <code>hi</code>
<code>binsof(cp_len) intersect {[0:3]}</code>	<code>cp_len</code> bin contains at least one value in 0..3
<code>!binsof(cp_len) intersect {0}</code>	<code>cp_len</code> bin does not contain 0

Select expression	Selects the cross products whose ...
<code>binsof(a.x) && binsof(b.y)</code>	Both conditions (AND of selections)
<code>binsof(a.x) binsof(b.y)</code>	Either condition (union)
<code>with (expr), matches</code>	1800-2012 filters on the crossed values

```
covergroup cg_len2 with function sample(bit [3:0] len, bit wr);
  cp_len: coverpoint len {
    bins zero = {0}; bins lo[] = {[1:3]}; bins mid[2] = {[4:9]}; bins hi = {[10:14]};
    illegal_bins bad = {15};
  }
  cp_wr: coverpoint wr;
  x_len_wr: cross cp_len, cp_wr {
    // long writes are not supported by this slave: not a coverage goal
    ignore_bins no_long_wr = binsof(cp_len.hi) && binsof(cp_wr) intersect {1};
    // all short reads count as one scenario
    bins short_rd = binsof(cp_len.lo) && binsof(cp_wr) intersect {0};
  }
endgroup
```

User-defined cross bins (commercial simulator).

Expected bin count (by the LRM rules). `cp_len` has $1 + 3 + 2 + 1 = 7$ bins and `cp_wr` 2, so the plain cross has 14 products. The ignore bin removes (hi, 1). The user bin `short_rd` absorbs the three products (lo[1..3], 0); automatically generated cross bins exclude any product already in a user-defined bin, so those three disappear as separate bins and one bin `short_rd` appears. Total: $14 - 1 - 3 + 1 = 11$ cross bins.

23.6 A worked example of the coverage calculation

Take `cg_len` from Section 23.3 (without the user-defined cross bins: 7 `cp_len` bins, 2 `cp_wr` bins, 14 cross bins, all weights 1) and call `sample()` five times with (len, wr) = (0,1), (2,0), (2,1), (5,1), (12,0).

Item	Bins hit	Coverage (expected, per LRM)
cp_len	zero, lo[2], mid[0] (5 is in {4,5,6}), hi	$4/7 = 57.14\%$
cp_wr	auto[0], auto[1]	$2/2 = 100.00\%$
x_len_wr	(zero,1), (lo[2],0), (lo[2],1), (mid[0],1), (hi,0)	$5/14 = 35.71\%$
Instance (weighted average)	$(57.14 + 100 + 35.71) / 3$	64.29%

A sixth call `sample(15, 0)` hits `illegal_bins bad`: the simulator reports a run-time error naming the bin, and the sample is not counted in `cp_len` or in the cross. The group coverage formula is the weighted average of its coverpoints and crosses, $\sum(w_i \times C_i) / \sum(w_i)$, where each C_i is the percentage of bins whose hit count reached `at_least`. Setting `option.weight = 0` on a coverpoint keeps it in the report but removes it from the group score - a common idiom for coverpoints that exist only to be crossed.

COVERAGE IS A PERCENTAGE OF BINS, NOT OF HITS

Hitting one bin a million times and another zero times gives 50%. That is why bin design is the real work: each bin should be one scenario a reviewer agrees must be seen. Too-coarse bins (automatic bins on wide buses) give meaningless 100% scores; too-fine crosses give coverage models that never close.

23.7 Options and type options

`option` members are **per instance** (set in the declaration, which applies to every instance, or procedurally on one instance: `u_cg.option.at_least = 4;`). `type_option` members are **static**, shared by all instances of the type, and set in the declaration or with `cg_len::type_option.goal = 90;`. Most can also be specified on an individual coverpoint or cross.

Option	Default	Meaning
<code>option.name</code>	tool-generated	Instance name in reports (also <code>set_inst_name()</code>)
<code>option.comment</code>	""	Text shown in reports; link to the verification-plan item
<code>option.at_least</code>	1	Hits needed for a bin to count as covered
<code>option.weight</code>	1	Weight of this instance (or coverpoint/cross) in the enclosing average
<code>option.goal</code>	100	Target percentage
<code>option.auto_bin_max</code>	64	Maximum number of automatic bins per coverpoint
<code>option.cross_num_print_missing</code>	0	How many missing cross bins to list in reports
<code>option.detect_overlap</code>	0	Warn if two bins of a coverpoint overlap
<code>option.per_instance</code>	0	1: also keep and report per-instance coverage data (otherwise only type-level data is required)
<code>option.get_inst_coverage</code>	0	1: <code>get_inst_coverage()</code> returns instance data when <code>merge_instances</code> is 1
<code>type_option.weight</code>	1	Weight of the type in the overall coverage
<code>type_option.goal</code>	100	Target for the type
<code>type_option.comment</code>	""	Comment for the type
<code>type_option.strobe</code>	0	1: sample in the Postponed region (once per time step, like <code>\$strobe</code>)
<code>type_option.merge_instances</code>	0	0: type coverage is the average of instance coverages; 1: bins of all instances are merged before computing

PITFALL: PER_INSTANCE AND THE MEANING OF 'TYPE COVERAGE'

With default options a covergroup type instantiated in 8 ports reports one number for the type. If port 3 never saw a long burst but port 5 did, merged coverage can show the bin covered while port 3's logic was never exercised. Set `option.per_instance = 1` (and meaningful `option.names`) when each instance is a distinct piece of hardware.

23.8 Querying and controlling coverage

Method / function	Returns / does
<code>cg_inst.sample()</code>	Sample now (with arguments if declared with <code>function sample</code>)
<code>cg_inst.get_inst_coverage()</code>	Real 0-100: coverage of this instance
<code>cg_type::get_coverage()</code> or <code>cg_inst.get_coverage()</code>	Real 0-100: coverage of the type (all instances)
<code>get_coverage(covered, total)</code>	Optional <code>ref int</code> arguments receive the number of covered and total bins
<code>cg_inst.cp_len.get_coverage()</code>	Coverage of one coverpoint (or cross)

Method / function	Returns / does
<code>cg_inst.start() / stop()</code>	Enable / disable collection for this instance
<code>cg_inst.set_inst_name("rx0")</code>	Name the instance
<code>\$get_coverage()</code>	Overall coverage of all covergroup types
<code>\$set_coverage_db_name("f"), \$load_coverage_db("f")</code>	Name / preload the coverage database

```
// Stop the test as soon as the coverage goal is reached (coverage-driven test length)
initial begin
    forever begin
        repeat (1000) @(posedge clk);
        if (u_cg_txn.get_inst_coverage() >= 95.0) begin
            $display("coverage goal reached: %0.2f%%", u_cg_txn.get_inst_coverage());
            $finish;
        end
    end
end
end
```

Using the query API to end a random test when it stops being productive (commercial simulator).

23.9 Covergroups in classes

In a class-based testbench, coverage lives in monitors, scoreboards or dedicated coverage collectors. A covergroup declared inside a class is an **embedded covergroup**: its name is simultaneously the type and the single instance variable, it can reference the class properties (including `local` and `protected` ones), and it must be constructed in the class constructor - the embedded covergroup variable may only be assigned in `new`. Each class object therefore owns exactly one instance.

```
class axi_txn;
    rand bit [3:0] len;
    rand bit [1:0] burst;        // FIXED=0, INCR=1, WRAP=2
    rand bit write;
endclass

class axi_cov;
    axi_txn t;                  // the transaction being sampled

    covergroup cg_axi;
        option.per_instance = 1;
        cp_len: coverpoint t.len { bins one = {0}; bins short_b[] = {[1:3]};
                                   bins mid = {[4:6]}; bins len8 = {7};
                                   bins long_b = {[8:14]}; bins len16 = {15}; }
        cp_burst: coverpoint t.burst { bins fixed = {0}; bins incr = {1}; bins wrap = {2};
                                         illegal_bins rsvd = {3}; }
        cp_dir: coverpoint t.write;
        // WRAP bursts must have length 2, 4, 8 or 16 beats (len = 1, 3, 7, 15)
        x_burst_len: cross cp_burst, cp_len {
            ignore_bins wrap_bad = binsof(cp_burst.wrap) &&
                                   binsof(cp_len) intersect {0, 2, [4:6], [8:14]};
        }
    endgroup

    function new(string name);
        cg_axi = new();        // mandatory, and only allowed here
        cg_axi.set_inst_name(name);
    endfunction

    function void write(axi_txn tr); // called by the monitor for each transaction
        t = tr;
    endfunction
endclass
```

```

    cg_axi.sample();
endfunction
endclass

```

An embedded covergroup in a coverage collector class (commercial simulator). In UVM this class would extend `uvm_subscriber` and `write()` would be its analysis-port callback (Chapter 26).

Notice why `cp_len` has separate bins `len8` and `len16`: `binsof(cp_len) intersect {...}` selects every **whole bin** that contains at least one of the listed values. Had `mid` been `{[4:7]}`, the ignore expression would have thrown away the legal 8-beat WRAP burst together with the illegal ones. Cross bins can only be as fine as the coverpoint bins they are built from.

Two design choices here are standard practice. Coverage samples **monitored** transactions (what the DUT actually saw), never the generated ones: a randomized transaction that a driver failed to send, or that the DUT rejected, must not count. And legality constraints of the protocol (reserved burst type, legal WRAP lengths) are mirrored as illegal or ignore bins so that the cross does not contain holes that can never close.

23.10 cover property versus covergroup

	cover property / cover sequence	covergroup
Measures	Temporal scenarios: 'a stall of 3+ cycles followed by a back-to-back transfer'	Values and value combinations at sample points; simple value transitions
Strength	Precise cycle-level ordering, overlaps, latencies	Bins, crosses, ranges; compact models of large spaces
Weakness	One cover = one scenario; no binning	Poor at timing relationships between different signals
Where	Modules/interfaces/checkers, bound to RTL	Classes (monitors, collectors), modules, interfaces
Formal	Used as reachability targets	Ignored

A mature coverage model uses both: covergroups for the data space of transactions (sizes, types, addresses, IDs, response codes and their crosses) and cover properties for the timing space (back-pressure shapes, interleavings, arbitration fairness windows, reset in the middle of a burst).

23.11 Coverage closure and the verification plan

Coverage is only meaningful against a **verification plan** (vplan, testplan): a document, usually a spreadsheet or a tool-managed plan (Verdi/VC Planner, vManager, Questa Verification Management), that lists every feature of the specification, how it will be verified (random test, directed test, assertion, formal proof), and which coverage items prove it was exercised. Each coverpoint, cross or cover property carries an `option.comment` or naming convention that links it back to its plan item, and the tool back-annotates merged regression coverage onto the plan.

```

spec features --> verification plan --> coverage model (covergroups,
    ^                (items, owners,          cover properties, code cov)
    |                methods, goals)           |
    |                                           v
spec/plan review <-- hole analysis <-- merged coverage database
    |                (why is it 0?)          (all seeds, all tests)
    v                                           |
waivers / exclusions    +--> constraint tuning, new directed tests,
(reviewed, signed)      formal proof of unreachability

```

The coverage closure loop.

- **Write the model early**, from the spec, before the stimulus is mature, and review it with the designer and architect; a coverage model is as much a spec review as a DV artifact.
- **Run the random regression first** and let it saturate; then analyse holes. Most holes have one of four causes: a constraint that makes the scenario impossible, a missing sequence/test, a bin that is actually unreachable in this configuration, or a genuine RTL bug that prevents the state being reached.

- **Close deliberately:** tune constraints or distributions, write directed or semi-directed tests for the remaining holes, and use formal tools to prove unreachable bins really are unreachable before waiving them.
- **Merge** coverage across all tests and seeds (UCIS-based databases) and track it per build; coverage that drops after an RTL change is a regression signal just like a failing test.
- **Rank tests** by unique coverage contribution to build a fast smoke/sanity regression out of the few seeds that hit the most bins.
- **Sign-off** needs 100% of plan items met (with every waiver reviewed), code-coverage targets met with reviewed exclusions, all assertions active and non-vacuous, and zero open failures.

INTERVIEW INSIGHT: '100% COVERAGE BUT THE CHIP HAS A BUG'

Coverage measures what was exercised, not what was checked. A bin can be hit while the scoreboard compares the wrong field, while an assertion is disabled, or while the bug's symptom is masked. That is why sign-off requires coverage **and** checkers **and** a reviewed plan, and why mature teams also use mutation/fault insertion (e.g. Certitude) to measure whether the testbench would detect a bug if one were present.

23.12 Summary

- Code coverage is automatic and measures the implementation; functional coverage is written from the spec and measures scenarios. Sign-off needs both.
- A covergroup is a type; instances created with `new` count independently; sampling is by clocking event, block event or `sample()`, optionally with arguments.
- Bins: automatic (up to `auto_bin_max = 64`), single, arrays `[]`, fixed-count `[N]` (remainder in the last bin), `default`, `ignore_bins`, `illegal_bins` (run-time error), `wildcard`.
- Transition bins cover successive sampled values with `=>`, `[*]`, `[->]` and `[=]`; crosses cover combinations, shaped with `binsof`, `intersect`, `&&`, `||` and `!`.
- Coverage is the weighted average of coverpoint and cross percentages, each the fraction of bins with at least `at_least` hits; `option` is per instance, `type_option` static.
- Embedded covergroups live in classes and must be constructed in `new`; sample monitored, not generated, transactions.
- Closure is a loop: plan, model, regress, analyse holes, tune or direct, prove or waive, merge and track.

23.13 Exercises

- 1 For `coverpoint addr[7:0]` with no bins and `option.auto_bin_max = 10`, how many bins are created and which values are in the last one?
- 2 Write bins for a 3-bit field such that value 7 is illegal, value 6 is ignored, 0 has its own bin and 1-5 are covered by one bin per value. How many bins count toward coverage?
- 3 Given samples IDLE, BUSY, BUSY, BUSY, WAIT, BUSY, DONE, IDLE, which of `(BUSY [*3])`, `(BUSY [*2:4])`, `(BUSY [-> 2])` and `(IDLE => BUSY => DONE)` are hit? Explain each.
- 4 A cross of `cp_a` (4 bins) and `cp_b` (5 bins, one of them an `ignore_bins`) has one user bin selecting `binsof(cp_a) intersect {0}`. How many cross bins exist?
- 5 Design the coverage model (covergroups and cover properties) for a 4-requester round-robin arbiter. Which items prove fairness?
- 6 Explain three different reasons why a bin might remain at 0 after 10 000 random seeds, and the right action for each.

24. DPI-C, VPI and Co-Simulation with C and Python

No SoC is verified in SystemVerilog alone. The architecture team writes C/C++ reference models of the datapaths, the CPU team has an instruction-set simulator, the software team has drivers and firmware, and the algorithm team works in Python or MATLAB. The testbench must reuse all of them rather than re-implement them. SystemVerilog offers two standard foreign-language interfaces: the **Direct Programming Interface (DPI-C)**, which makes a C function callable like an SV function and vice versa, and the older **Verilog Procedural Interface (VPI)**, which gives C code a handle-based view of the whole simulation database. Python testbench frameworks such as cocotb are built on VPI.

This chapter explains both interfaces at LRM level (IEEE 1800 clauses 35-38 and Annex H/I), runs a DPI-C reference-model example with Verilator 5.020, a VPI application with Icarus Verilog 12 and a cocotb Python testbench with cocotb 2.1.0 on Icarus, and closes with how C/C++, SystemC and ISS golden models are co-simulated in industrial SoC DV flows. Chapter 10 introduced PLI/VPI from the Verilog side; here we go further.

Interface	Direction and granularity	Typical use
DPI-C import	SV calls C: function-call granularity, argument passing, no knowledge of the design	Reference models, checksums, file formats, OS services, ISS step()
DPI-C export	C calls SV functions/tasks (from within an imported context call)	C test code driving a bus via an SV BFM task; firmware-driven tests
VPI	C inspects and controls the simulator: hierarchy, values, callbacks, time	Debuggers, waveform dumpers, coverage tools, cocotb, custom system tasks
Co-simulation	Whole models (SystemC, ISS, Python) exchanging transactions with RTL through DPI/VPI/TLM	Virtual platforms, lockstep CPU checking, HW/SW co-verification

24.1 Importing C functions and tasks

An import declaration tells SV that a subroutine is implemented in a foreign language. It may appear wherever a function declaration may (module, interface, package, program, compilation unit), and the imported routine is then called exactly like a native SV function or task.

```
// import "DPI-C" [pure | context] function <return_type> name(<args>);
import "DPI-C" pure    function int    crc8_byte(input int crc, input int data);
import "DPI-C"        function void   c_log(input string msg);
import "DPI-C" context function int    c_calls_sv(input int x);
import "DPI-C" context task           c_run_test(input int seed); // may consume time
import "DPI-C" functionchandle        model_new(input int cfg);   // opaque C pointer
import "DPI-C" function void          model_step(inputchandle m, input int din,
                                             output int dout);

// C name different from the SV name: "DPI-C" [property] c_name = function ...
import "DPI-C" pure sin = function real sv_sin(input real x);      // libm's sin()
```

Forms of the import declaration (IEEE 1800-2017 clause 35.5).

Property	Rule
pure function	Result depends only on inputs; no side effects, no I/O, no global state, no calls to exported SV. The simulator may skip calls whose results are unused or reuse results (constant folding). Only functions with a non-void result and no output/inout arguments can be pure
context	The C code may call exported SV subroutines, use VPI, or query the scope (<code>svGetScope</code>). The simulator must preserve the calling scope, which costs a little speed; calling an export from a non-context import is an error
Neither	May have side effects (printing, allocating) but must not call back into SV or VPI

Property	Rule
Imported task	Implemented in C as a function returning <code>int</code> : 0 normally, 1 if the task was disabled while inside an exported task it called (see <code>svIsDisabledState</code>). May call exported tasks that consume simulation time
Return types	Only <code>void</code> , <code>byte</code> , <code>shortint</code> , <code>int</code> , <code>longint</code> , <code>real</code> , <code>shortreal</code> , <code>chandle</code> , <code>string</code> , scalar <code>bit</code> and <code>logic</code> . Packed vectors and other larger results are returned through <code>output</code> arguments
Arguments	Directions <code>input</code> , <code>output</code> , <code>inout</code> ; <code>ref</code> is not allowed
Name space	All DPI-C names share one global C name space: two packages importing the same C name must use the same signature

PITFALL: LYING ABOUT PURITY

Declaring a function `pure` when it keeps state (a random-number generator, a model with an internal register, a function that prints) is undefined behaviour: an optimising simulator may call it fewer times than the source suggests, reorder calls, or cache the result. Symptoms appear only at higher optimisation levels or on another vendor's simulator. Use `pure` only for true mathematical functions.

24.2 Data type mapping

The LRM (Annex H) defines exactly how each SV argument type appears in C. Small scalar types are passed by value for `input`, and by pointer for `output` and `inout`. Packed vectors and 4-state values use canonical types defined in `svdpi.h`. Do not guess: every simulator can generate a C header with the exact prototypes (Questa `vlog -dpiheader`, Verilator's `V<top>__Dpi.h`), and your C file should include it so the C compiler checks the signatures.

SystemVerilog type	C type (input)	C type (output/inout)
<code>byte</code>	<code>char</code>	<code>char*</code>
<code>shortint</code>	<code>short int</code>	<code>short int*</code>
<code>int</code>	<code>int</code>	<code>int*</code>
<code>longint</code>	<code>long long</code>	<code>long long*</code>
<code>real</code> / <code>shortreal</code>	<code>double</code> / <code>float</code>	<code>double*</code> / <code>float*</code>
<code>chandle</code>	<code>void*</code> (opaque)	<code>void**</code>
<code>string</code>	<code>const char*</code>	<code>const char**</code>
<code>bit</code> (scalar)	<code>svBit</code> (unsigned char, 0/1)	<code>svBit*</code>
<code>logic</code> , <code>reg</code> (scalar)	<code>svLogic</code> (0,1,2=Z,3=X)	<code>svLogic*</code>
<code>bit [N-1:0]</code> (packed)	<code>const svBitVecVal*</code>	<code>svBitVecVal*</code>
<code>logic [N-1:0]</code> (packed)	<code>const svLogicVecVal*</code>	<code>svLogicVecVal*</code>
<code>enum</code>	As its base type	As its base type
Packed struct/union	As a packed vector of the same width	Same
Unpacked struct	<code>const struct*</code> with matching member types	<code>struct*</code>
Sized unpacked array <code>int a[4]</code>	<code>const int*</code> to the first element	<code>int*</code>
Open array <code>int a[]</code>	<code>const svOpenArrayHandle</code>	<code>svOpenArrayHandle</code>

Packed vectors are passed as arrays of 32-bit chunks, `SV_PACKED_DATA_NELEMS(N) = (N + 31) / 32` of them, **least significant chunk first** (chunk 0 holds bits 31:0). `svBitVecVal` is a `uint32_t`. A 4-state vector uses `svLogicVecVal`, a struct of two words per chunk, `aval` and `bval`, encoding each bit as `(aval,bval) = (0,0)` for 0, `(1,0)` for 1, `(0,1)` for Z and `(1,1)` for X. Unused high bits of the last chunk are undefined on input: mask them.

Bits of a packed vector can also be accessed with helper functions (`svGetBitSelBit`, `svGetPartSelLogic`, `svPutPartSelBit`, ...).

PITFALL: 4-STATE VALUES COLLAPSE AT THE C BOUNDARY

Passing a `logic` value to an `int` or `bit` argument converts X and Z to 0 silently, so a C model can happily 'agree' with an RTL output that is actually X. When a reference model compares RTL outputs, either pass them as `logic` vectors and check `bval`, or check `$isunknown()` in SV before calling C.

24.3 Open arrays and the svdpi.h API

An **open array** formal (`int a[], bit [7:0] b[],` or even an open packed dimension `bit [] v`) lets one C function accept arrays of any size. C receives an opaque `svOpenArrayHandle` and queries it:

svdpi.h routine	Purpose
<code>svLow(h,d), svHigh(h,d), svLeft, svRight, svSize(h,d)</code>	Bounds and size of dimension d (1 = leftmost unpacked dimension)
<code>svDimensions(h)</code>	Number of unpacked dimensions
<code>svGetArrElemPtr1(h, i)</code> (and 2, 3, <code>svGetArrElemPtr</code>)	Pointer to one element, indexed with SV indices
<code>svGetArrayPtr(h), svSizeOfArray(h)</code>	Pointer to and size of the whole array if it is contiguous in memory (may be NULL)
<code>svGetBitArrElem1, svPutLogicArrElem1VecVal, ...</code>	Read/write elements of packed/4-state arrays
<code>svGetScope(), svSetScope(s), svGetNameFromScope(s)</code>	Current SV scope of a context call; select the scope before calling an export
<code>svPutUserData(s, key, data), svGetUserData(s, key)</code>	Attach C data to an SV scope (per-instance model state)
<code>svIsDisabledState(), svAckDisabledState()</code>	Handle <code>disable</code> of an imported task

24.4 Worked example: calling a C reference model from SystemVerilog

The example below exercises the most common DPI patterns: a pure C golden model (CRC-8, polynomial 0x07) checked against an RTL-style SV implementation, a 64-bit packed vector argument, an open array, a string, and a context import that calls back into an exported SV function. The C source is simply added to the Verilator command line; `--binary` compiles and links it with the model.

```
module dpi_tb;
  import "DPI-C" pure function int crc8_byte(input int crc, input int data);
  import "DPI-C" function int lane_sum(input bit [63:0] v);
  import "DPI-C" function int arr_sum(input int a[]);
  import "DPI-C" function void c_log(input string msg);
  import "DPI-C" context function int c_calls_sv(input int x);
  export "DPI-C" function sv_scale;

  function int sv_scale(input int x);
    return x * 10;
  endfunction

  // RTL-style CRC-8 the C model checks
  function automatic logic [7:0] crc8_rtl(logic [7:0] crc, logic [7:0] d);
    logic [7:0] c = crc ^ d;
    repeat (8) c = c[7] ? ((c << 1) ^ 8'h07) : (c << 1);
    return c;
  endfunction
```



```

initial begin
    int ref_crc = 0; logic [7:0] rtl_crc = 0; int errors = 0;
    int a[5] = '{1, 2, 3, 4, 5};
    byte msg[] = '{8'h31, 8'h32, 8'h33, 8'h34, 8'h35, 8'h36, 8'h37, 8'h38, 8'h39};
    foreach (msg[i]) begin
        ref_crc = crc8_byte(ref_crc, int'(msg[i]) & 'hFF);
        rtl_crc = crc8_rtl(rtl_crc, msg[i]);
        if (rtl_crc != ref_crc[7:0]) errors++;
    end
    $display("CRC-8(\"123456789\"): rtl=%h model=%h errors=%0d",
            rtl_crc, ref_crc[7:0], errors);
    $display("lane_sum = %0d", lane_sum(64'h0004_0003_0002_0001));
    $display("arr_sum = %0d", arr_sum(a));
    c_log($sformatf("hello from SV at t=%0t", $time));
    $display("c_calls_sv(7) = %0d", c_calls_sv(7));
    $finish;
end
endmodule

```

dpi_tb.sv - the SV side: four imports, one export.

```

// ref_model.c - C reference model called from SystemVerilog through DPI-C
#include <stdio.h>
#include <stdint.h>
#include "svdpi.h"

#ifdef __cplusplus           // Verilator compiles .c files with g++: keep C linkage
extern "C" {
#endif

// CRC-8 (poly 0x07) over one byte: a typical golden model for a datapath block
int crc8_byte(int crc, int data) {
    uint8_t c = (uint8_t)(crc ^ data);
    for (int i = 0; i < 8; i++)
        c = (c & 0x80) ? (uint8_t)((c << 1) ^ 0x07) : (uint8_t)(c << 1);
    return c;
}

// Packed 64-bit vector in, sum of its four 16-bit lanes out
int lane_sum(const svBitVecVal *v) {
    int s = 0;
    for (int w = 0; w < 2; w++)           // svBitVecVal = 32-bit chunks, LSB chunk first
        s += (v[w] & 0xFFFF) + (v[w] >> 16);
    return s;
}

// Open array: works for any unpacked size the SV caller passes
int arr_sum(const svOpenArrayHandle a) {
    int s = 0;
    for (int i = svLow(a, 1); i <= svHigh(a, 1); i++)
        s += *(int *)svGetArrElemPtr1(a, i);
    return s;
}

// String argument: C sees a const char *
void c_log(const char *msg) { printf("[C] %s\n", msg); }

// Imported 'context' function calling back into an exported SV function
int sv_scale(int x);           // exported from SV (see Vdpi_tb__Dpi.h)
int c_calls_sv(int x) { return sv_scale(x) + 1; }

```

```
#ifdef __cplusplus
}
#endif
```

ref_model.c - the C side.

```
$ verilator --binary -Wall -Wno-fatal dpi_tb.sv ref_model.c
$ ./obj_dir/Vdpi_tb
```

Build and run: the .c file is compiled and linked with the Verilated model.

```
CRC-8("123456789"): rtl=f4 model=f4 errors=0
lane_sum = 10
arr_sum = 15
[C] hello from SV at t=0
c_calls_sv(7) = 71
- dpi_tb.sv:35: Verilog $finish
```

Verilator 5.020 output. 0xF4 is the published check value of CRC-8/SMBUS for the string '123456789', so both the model and the RTL are right.

Verilator writes the prototypes it expects into `obj_dir/Vdpi_tb__Dpi.h`. An excerpt of the generated file shows the mapping rules of the previous section applied: the `bit [63:0]` became `const svBitVecVal*`, the open array `const svOpenArrayHandle` and the string `const char*`.

```
// DPI EXPORTS
// DPI export at dpi_tb.sv:9:16
extern int sv_scale(int x);

// DPI IMPORTS
// DPI import at dpi_tb.sv:4:31
extern int arr_sum(const svOpenArrayHandle a);
// DPI import at dpi_tb.sv:6:39
extern int c_calls_sv(int x);
// DPI import at dpi_tb.sv:5:32
extern void c_log(const char* msg);
// DPI import at dpi_tb.sv:2:36
extern int crc8_byte(int crc, int data);
// DPI import at dpi_tb.sv:3:31
extern int lane_sum(const svBitVecVal* v);
```

Excerpt of the generated `obj_dir/Vdpi_tb__Dpi.h` (Verilator 5.020).

PITFALL: C++ NAME MANGLING

The first build of this example failed at link time with 'undefined reference to `c_log`' and 'undefined reference to `sv_scale(int)`': Verilator compiles .c files with the C++ compiler, which mangles names. DPI requires C linkage, hence the `extern "C"` block guarded by `__cplusplus`. The same applies to any C++ model: wrap the DPI entry points in `extern "C"`, or include the tool-generated header, which already contains the guard. Also note that without `$finish` a Verilator `--binary` model whose `initial` block ends does not terminate by itself here - always end tests explicitly.

24.5 Exporting SV functions and tasks to C

`export "DPI-C" function name;` (or `task name;`) makes an SV subroutine callable from C. The export declaration names a subroutine defined in the **same scope**; it does not declare its signature again. Exports may only be called from C code that is running inside a `context` import call (or from a VPI callback after setting a scope), because an SV function belongs to a particular module instance: C must know **which** instance's `sv_scale` it calls. Inside a context import the scope is the one where the import was declared; `svSetScope()` switches to another instance, typically using a scope saved with `svGetScope()` during an earlier registration call.

```
// SV side: a bus-functional model that C test code can drive
module apb_bfm (input logic pclk /* ... APB pins ... */);
```

```

export "DPI-C" task apb_write;
export "DPI-C" task apb_read;
import "DPI-C" context task c_test_main();      // the C test program
import "DPI-C" context function void c_register_bfm(string name);

task apb_write(input int addr, input int data);
    @(posedge pclk) /* drive psel, paddr, pwidth ... wait for pready */;
endtask
task apb_read(input int addr, output int data);
    @(posedge pclk) /* ... */ data = 0;
endtask

initial begin
    c_register_bfm($sformatf("%m"));           // C saves svGetScope() for this instance
    c_test_main();                             // C now runs the test, consuming time
    $finish;
end
endmodule

// C side (firmware-like test): tasks return int, outputs are pointers
// #include "svdpi.h"
// extern int apb_write(int addr, int data);
// extern int apb_read(int addr, int *data);
// static svScope bfm;
// void c_register_bfm(const char *name) { bfm = svGetScope(); }
// int c_test_main(void) {
//     int v;
//     svSetScope(bfm);
//     apb_write(0x40001000, 0x1);              /* enable the UART */
//     apb_read (0x40001004, &v);              /* status */
//     return 0;                                /* 0 = not disabled */
// }

```

Exported time-consuming tasks let C test code drive an SV BFM - the basis of HW/SW co-verification (IEEE 1800-2017; exported tasks that consume time need a simulator that supports them, such as the commercial ones).

This pattern is how SoC teams run the **same** C test program on three platforms: in RTL simulation (the `apb_write` calls go through DPI into a BFM), on an emulator or FPGA prototype (the same calls go through a transactor), and on silicon (the calls become real memory-mapped accesses). Only the thin access layer changes.

24.6 Compiling and linking across simulators

Tool	How the C code gets in	Header generation
Verilator	List <code>.c/.cpp</code> files on the command line (or <code>-CFLAGS</code> , <code>-LDFLAGS</code>); compiled with the model by make	Automatic: <code>obj_dir/V<top>__Dpi.h</code>
Questa / ModelSim	<code>vlog</code> for SV; C compiled into a shared library and loaded with <code>vsim -sv_lib <lib></code> (or compiled automatically when the C file is given to <code>vlog</code>)	<code>vlog -dpiheader dpi.h</code>
Xcelium	<code>xrun top.sv model.c</code> compiles and links the C; or prebuilt libraries with <code>-sv_lib</code>	Tool option to dump the DPI header
VCS	<code>vcs -sverilog top.sv model.c</code> (C compiled and linked into simv); or <code>-sv_lib/-LDFLAGS</code>	Tool option to dump the DPI header
Any, precompiled	<code>-sv_lib name</code> loads <code>name.so</code> at run time; <code>-sv_liblist</code> reads a list of libraries (LRM Annex J)	

Build C models with `-fPIC` when they go into shared libraries, use the same compiler major version as the simulator's runtime to avoid C++ ABI surprises, and compile with `-g` so `gdb` can attach to the simulator process and break inside the model.

24.7 VPI basics

VPI (IEEE 1800 clauses 36-38, header `vpi_user.h`, and `sv_vpi_user.h` for SV extensions) is the third generation of the Verilog Programming Language Interface, replacing the older TF and ACC routines (PLI 1.0). It is object-oriented in C: every design object (module, net, reg, port, parameter, task call, even a statement) is reached through an opaque `vpiHandle`, and relationships between objects are navigated with a small set of routines.

Routine	Purpose
<code>vpi_register_systf(&s_vpi_systf_data)</code>	Define a user system task or function (<code>\$walk</code>) with <code>calltf</code> , <code>compiletf</code> , <code>sizetf</code> callbacks
<code>vpi_handle(type, ref)</code>	One-to-one relation, e.g. <code>vpi_handle(vpiSysTfCall, NULL)</code> = the current call
<code>vpi_iterate(type, ref) + vpi_scan(it)</code>	One-to-many: nets of a module, arguments of a call, sub-instances
<code>vpi_handle_by_name("top.u.q", NULL)</code>	Look up by hierarchical name
<code>vpi_get(prop, h), vpi_get_str(prop, h)</code>	Integer / string properties: <code>vpiSize</code> , <code>vpiType</code> , <code>vpiName</code> , <code>vpiFullName</code> , <code>vpiDefName</code>
<code>vpi_get_value(h, &v), vpi_put_value(h, &v, &t, flags)</code>	Read / write values in many formats (<code>vpiIntVal</code> , <code>vpiHexStrVal</code> , <code>vpiVectorVal...</code>); <code>vpiNoDelay</code> , <code>vpiInertialDelay</code> , <code>vpiForceFlag...</code>
<code>vpi_register_cb(&s_cb_data)</code>	Callbacks on value change, time, simulation phases
<code>vpi_get_time, vpi_printf, vpi_control(vpiFinish, ...)</code>	Time, printing to the simulator log, stopping/finishing
<code>vpi_chk_error, vpi_release_handle</code>	Error checking; free handles (older name <code>vpi_free_object</code>)

An application registers itself through the `vlog_startup_routines` array: the simulator calls each function in it at start-up, and these register the system tasks and any start-of-simulation callbacks. The example defines two system tasks: `$walk(scope)` recursively lists every scope with its nets and regs, and `$watch(signal)` installs a value-change callback.

```
// walk.c - a VPI application: $walk(scope) lists the hierarchy, $watch(sig) adds a callback
#include <stdio.h>
#include <vpi_user.h>

static void walk(vpiHandle scope, int depth) {
    vpiHandle it, h;
    vpi_printf("%s scope %s\n", 2 * depth, "", vpi_get_str(vpiFullName, scope));
    if ((it = vpi_iterate(vpiNet, scope)))
        while ((h = vpi_scan(it)))
            vpi_printf("%s net %s [%d bits]\n", 2 * depth, "", vpi_get_str(vpiName, h),
                vpi_get(vpiSize, h));
    if ((it = vpi_iterate(vpiReg, scope)))
        while ((h = vpi_scan(it)))
            vpi_printf("%s reg %s [%d bits]\n", 2 * depth, "", vpi_get_str(vpiName, h),
                vpi_get(vpiSize, h));
    if ((it = vpi_iterate(vpiModule, scope)))
        while ((h = vpi_scan(it))) walk(h, depth + 1);
}

static PLI_INT32 walk_calltf(PLI_BYTE8 *user) {
    vpiHandle systf = vpi_handle(vpiSysTfCall, NULL);
    vpiHandle args = vpi_iterate(vpiArgument, systf);
    vpiHandle scope = vpi_scan(args);
    vpi_free_object(args);
    walk(scope, 0);
    return 0;
}
```

```

}

static PLI_INT32 on_change(p_cb_data cb) {
    s_vpi_value v = { vpiHexStrVal };
    vpi_get_value(cb->obj, &v);
    vpi_printf(" [cb] t=%u %s = %s\n", cb->time->low,
               vpi_get_str(vpiFullName, cb->obj), v.value.str);
    return 0;
}

static PLI_INT32 watch_calltf(PLI_BYTE8 *user) {
    vpiHandle systf = vpi_handle(vpiSysTfCall, NULL);
    vpiHandle args = vpi_iterate(vpiArgument, systf);
    static s_vpi_time t = { vpiSimTime };
    static s_vpi_value v = { vpiSuppressVal };
    s_cb_data cb = { cbValueChange, on_change, NULL, &t, &v, 0, NULL };
    cb.obj = vpi_scan(args);
    vpi_free_object(args);
    vpi_register_cb(&cb);
    return 0;
}

static void register_tasks(void) {
    s_vpi_systf_data tf = { vpiSysTask, 0, "$walk", walk_calltf, NULL, NULL, NULL };
    vpi_register_systf(&tf);
    tf.tfname = "$watch"; tf.calltf = watch_calltf;
    vpi_register_systf(&tf);
}

void (*vlog_startup_routines[])(void) = { register_tasks, 0 };

```

walk.c - hierarchy traversal and a value-change callback.

```

module counter (input wire clk, input wire rst, output reg [3:0] q);
    always @(posedge clk) q <= rst ? 4'd0 : q + 4'd1;
endmodule

module top;
    reg clk = 0, rst = 1;
    wire [3:0] q;
    counter u_cnt (.clk(clk), .rst(rst), .q(q));
    always #5 clk = ~clk;
    initial begin
        $walk(top);
        $watch(q);
        #12 rst = 0;
        #30 $finish;
    end
endmodule

```

top.v - the design the VPI tasks inspect.

```

$ iverilog-vpi walk.c                # builds walk.vpi (a shared object)
$ iverilog -o top.vvp top.v
$ vvp -M. -mwalk top.vvp             # -M: search path, -m: load module walk.vpi

```

Building and loading a VPI module with Icarus Verilog.

```

scope top
  net q [4 bits]
  reg clk [1 bits]
  reg rst [1 bits]
  scope top.u_cnt
    net clk [1 bits]
    net rst [1 bits]
    reg q [4 bits]
  [cb] t=5 top.q = 0
  [cb] t=15 top.q = 1
  [cb] t=25 top.q = 2
  [cb] t=35 top.q = 3
top.v:14: $finish called at 42 (1s)

```

Icarus Verilog 12 output. Inside **counter**, **q** is a reg while in **top** it is a net; the callback fires on every change of **top.q**, including the X -> 0 change at 5.

Callbacks

Callback reason	Fires when
cbValueChange	The object (net, reg, variable, event) changes value
cbAfterDelay	After a relative delay - a C-side timer
cbReadWriteSynch	At a point in the current time step where values may still be written (before the end of the time step)
cbReadOnlySynch	At the end of the time step, when values are final; writing is not allowed
cbNextSimTime	At the start of the next time step
cbStartOfSimulation , cbEndOfSimulation	Simulation start/end: open and close files, print reports
cbEndOfCompile	After elaboration, before time 0

DPI OR VPI?

Use DPI for anything that looks like a function call: it is faster (no handle lookups, arguments go directly into C), type-checked, and portable between simulators. Use VPI when you need what DPI cannot give: discovering the hierarchy at run time, reading signals nobody passed to you, waiting on arbitrary value changes, or building a generic tool. VPI access to signals also disables some simulator optimisations: commercial tools require read/write access to be enabled explicitly (**+acc/-access +rwc/debug** switches), and Verilator needs **--vpi** and **/*verilator public*/** (or **--public-flat-rw**) on the signals VPI touches.

24.8 cocotb: Python testbenches over VPI

cocotb (COroutine based COsimulation TestBench) is an open-source framework that runs Python test code alongside any simulator with VPI, VHPI or FLI: Icarus, Verilator, Questa, Xcelium, VCS, Riviera, GHDL and others. The simulator runs the HDL top level as usual; cocotb's GPI library, loaded through VPI, embeds a Python interpreter and exposes the design hierarchy as Python objects (**dut.a**, **dut.u_core.pc**). Tests are **async** functions that **await triggers** - **RisingEdge(sig)**, **FallingEdge**, **Timer(10, unit="ns")**, **ClockCycles**, **ReadOnly()**, **ReadWrite()** - which map directly onto VPI callbacks. Python's ecosystem then provides the golden model (NumPy, a Python ISS, a network stack) without any DPI glue.

```

`timescale 1ns/1ps
// 8-bit saturating adder with a registered output
module sat_add (
    input  logic      clk,
    input  logic [7:0] a, b,
    output logic [7:0] y
);
    logic [8:0] s;
    assign s = a + b;
    always_ff @(posedge clk) y <= s[8] ? 8'hFF : s[7:0];
endmodule

```

sat_add.sv - the DUT. cocotb needs a timescale that can represent the clock period (without it, the first run failed with 'Unable to accurately represent 10(ns) with the simulator precision of 1e0').

```

# test_sat_add.py - a cocotb testbench: Python drives and checks the SV DUT through VPI
import random
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import FallingEdge, RisingEdge, ReadOnly

def model(a, b):
    # the golden model is plain Python
    return min(a + b, 255)

@cocotb.test()
async def random_adds(dut):
    cocotb.start_soon(Clock(dut.clk, 10, unit="ns").start())
    rng = random.Random(1)
    vectors = [(0, 0), (255, 1), (200, 55), (128, 128)]
    vectors += [(rng.randrange(256), rng.randrange(256)) for _ in range(96)]
    errors = 0
    for a, b in vectors:
        await FallingEdge(dut.clk)          # drive away from the active edge
        dut.a.value = a
        dut.b.value = b
        await RisingEdge(dut.clk)
        await ReadOnly()                    # sample after the flop has updated
        got = int(dut.y.value)
        if got != model(a, b):
            errors += 1
            dut._log.error("a=%d b=%d y=%d exp=%d", a, b, got, model(a, b))
    dut._log.info("checked %d vectors, %d errors", len(vectors), errors)
    assert errors == 0

```

test_sat_add.py - directed corner cases plus 96 random vectors against a Python model.

```

# run.py - build with Icarus and run the cocotb test (cocotb 2.x runner API)
from cocotb_tools.runner import get_runner

runner = get_runner("icarus")
runner.build(sources=["sat_add.sv"], hdl_toplevel="sat_add",
            build_args=["-g2012"], always=True)
runner.test(hdl_toplevel="sat_add", test_module="test_sat_add")

```

run.py - the Python runner replaces the Makefile flow of cocotb 1.x (pip install cocotb; then python run.py).

```

1000.00ns INFO cocotb.sat_add checked 100 vectors, 0 errors
1000.00ns INFO cocotb.regression test_sat_add.random_adds passed
** TESTS=1 PASS=1 FAIL=0 SKIP=0 1000.00 0.01 140698.11 **

```

cocotb 2.1.0 with Icarus Verilog 12: the relevant lines of the log (runs of spaces condensed).

To see the checker work, the saturation was removed from the RTL (`y <= s[7:0];`) and the test re-run:

```

20.00ns ERROR cocotb.sat_add a=255 b=1 y=0 exp=255
40.00ns ERROR cocotb.sat_add a=128 b=128 y=0 exp=255
70.00ns ERROR cocotb.sat_add a=253 b=230 y=227 exp=255
1000.00ns INFO cocotb.sat_add checked 100 vectors, 53 errors
1000.00ns WARNING cocotb.regression test_sat_add.random_adds failed
AssertionError: assert 53 == 0
** TESTS=1 PASS=0 FAIL=1 SKIP=0 1000.00 0.01 73906.80 **

```

Same test with the bug injected (first three of the 53 error lines shown; spaces condensed).

PITFALL: COCOTB WRITES ARE NOT IMMEDIATE

Assigning `dut.a.value = 5` schedules a write that the simulator applies in a later write phase of the time step, so reading `dut.a.value` on the next line still returns the old value. Writing after `await ReadOnly()` is an error (the read-only phase forbids writes) - hence the `FallingEdge` at the top of the loop. The API also changed between major versions (cocotb 2.0 renamed `units=` to `unit=`, and the `cocotb_tools.runner` replaced `cocotb.runner`); pin the version in your project requirements.

Where cocotb fits on the roadmap: it is widely used in open-source hardware, startups, FPGA teams and for block-level tests where a Python model already exists; large SoC teams mostly use UVM in SV (Chapter 26) and sometimes cocotb for specific blocks. Its limits are speed (every trigger crosses the VPI boundary into Python) and the absence of SV-native constraints, covergroups and assertions - which is why cocotb projects typically keep SVA in the RTL (Chapter 22) and use Python libraries for randomisation and coverage.

24.9 Co-simulation with SystemC and golden models in SoC DV flows

Model	Connected via	Used for
C/C++ reference model of a block (DSP, codec, crypto, compression)	DPI-C, transaction-level: one call per input transaction, results compared in the scoreboard	Block and subsystem DV; often bit-accurate models from the architecture team
Instruction-set simulator (e.g. Spike or a vendor ISS for a RISC-V/ARM core)	DPI: <code>iss_step()</code> after every retired instruction, compare PC, registers, memory writes (lockstep); or offline trace comparison	CPU core verification; the riscv-dv flow compares RTL and ISS instruction traces
SystemC/TLM-2.0 virtual platform	Simulator's native SystemC co-simulation, or TLM <-> RTL transactors	Early software development, performance models, HW/SW co-verification
Verilated RTL inside SystemC/C++	Verilator <code>--sc</code> or <code>--cc</code> generates a SystemC/C++ class	Fast cycle-based models of RTL in virtual platforms; open-source SoC flows
Python / MATLAB algorithm models	cocotb, DPI with an embedded interpreter, or C code generated from MATLAB	Signal processing, ML accelerators, image pipelines
Emulator / FPGA prototype	Transactors (SCE-MI 2 function-based interface is built on DPI)	Bootting OS and firmware on the full SoC at MHz speeds

The recurring design decision is the **abstraction level of the boundary**. Cross it once per transaction (a packet, an instruction, a DMA descriptor), never once per clock per signal: every DPI call has a cost, and a per-cycle, per-signal interface is both slow and tightly coupled to the RTL timing. The SV side converts pin activity into transactions (monitor), calls the model with a transaction, and compares transactions in the scoreboard. For models with state, create them through a `chandle` factory (`model_new`) so that each SV instance or UVM component owns its own model object, and free them at the end of the test.

```

+----- SystemVerilog testbench -----+ +-----+
| driver -> DUT pins -> monitor --txn--> scoreboard|<---->| C/C++ model |
|                                     ^   | DPI | (chandle per |
|                                     +-- txn --> model_step()| call | instance) |
+-----+ +-----+
one DPI call per transaction, not per clock

```

Transaction-level coupling of a C reference model through DPI.

INTERVIEW INSIGHT: LOCKSTEP ISS COMPARISON

In CPU verification the testbench observes each retired instruction (through an RVFI-style trace port or internal probes), calls the ISS via DPI to execute the same instruction, and compares architectural state immediately. The hard parts are not the DPI calls but the **non-determinism**: interrupts, timers, memory-mapped I/O and multi-core memory ordering, where the RTL's behaviour is legal but different from the ISS. Mature flows let the RTL 'teach' the ISS (inject the interrupt at the same instruction, return the same MMIO read data) so the comparison stays exact.

24.10 Summary

- DPI-C imports make C functions and tasks callable from SV; exports make SV subroutines callable from C inside a `context` call. `pure` means no side effects; `context` is required to call exports, VPI or scope routines.
- Types map by Annex H: small scalars by value, outputs by pointer, strings as `const char*`, packed vectors as `svBitVecVal`/`svLogicVecVal` 32-bit chunks (LSB chunk first, `aval/bval` for 4-state), open arrays as `svOpenArrayHandle`.
- Always compile against the tool-generated header; wrap C++ in `extern "C"`; never lie about `pure`; watch X collapsing to 0.
- Verilator compiles C/C++ files given on its command line; commercial simulators compile them or load `-sv_lib` shared libraries.
- VPI is the handle-based API to the simulation database: `vpi_register_systf`, `vpi_handle/vpi_iterate/vpi_scan`, `vpi_get_value/vpi_put_value`, and callbacks.
- cocotb runs Python coroutine testbenches on top of VPI; it is productive for Python-modelled blocks and open-source flows.
- Co-simulate models at transaction level: C models, ISS lockstep, SystemC/TLM virtual platforms and emulator transactors all use the same idea.

24.11 Exercises

- 1 Write the C prototype for `import "DPI-C" function void f(input logic [95:0] a, output bit [7:0] b, inout int c[]);` and explain how bit 70 of `a` is found in C, and how to test whether it is X.
- 2 Extend the DPI example with an imported function that returns a 128-bit result. Why can it not be the function's return value, and what does the declaration look like instead?
- 3 Build a stateful C model with `chandle model_new(int)`, `model_step(chandle, int, output int)` and `model_free(chandle)`, and use two independent instances from two SV module instances.
- 4 Extend `walk.c` with a `$force_zero(sig)` system task that uses `vpi_put_value` with `vpiForceFlag`, and a `cbEndOfSimulation` callback that prints how many value changes `$watch` observed.
- 5 Port the cocotb test to check the adder in the same cycle with `ReadWrite()` instead of `FallingEdge/ReadOnly()` and explain what goes wrong.
- 6 Sketch the DPI interface of a lockstep checker between a RISC-V core and an ISS. Which events must be passed from RTL to the ISS to keep them synchronised?

PART V

Putting It Together

The first four parts taught the language one construct at a time. This part assembles them. Chapter 25 builds a complete layered, class-based testbench from scratch and runs it; Chapter 26 shows that UVM is nothing more than a disciplined use of the same language features; Chapter 27 collects the pitfalls, portability traps and lint rules that separate working code from code that only appears to work; and Chapter 28 lays out a study roadmap for RTL, DV, DFT and formal roles on SoC and ASIC teams.

25. A Layered Class-Based Testbench from Scratch

Every industrial verification environment - hand-written, UVM, or a company's in-house methodology - is built from the same half-dozen roles: something that decides *what* to send, something that turns it into *pin activity*, something that *watches* the pins, and something that *decides* whether what it saw was right. This chapter builds that architecture in plain SystemVerilog, without any library, for a small synchronous FIFO. Every class is short, every language feature is one you have already met (classes in Chapter 18, mailboxes and fork/join in Chapter 20, interfaces and virtual interfaces in Chapter 16), and the whole thing is compiled and run with Verilator 5.020, including a deliberately broken DUT that the scoreboard catches.

Building one environment by hand is the single best preparation for UVM. When you later meet `uvm_driver`, `uvm_monitor`, `uvm_scoreboard`, `uvm_env` and `uvm_test`, you will recognise them as exactly the classes written here, plus a factory, a configuration database and a phase scheduler (Chapter 26).

25.1 The layers and their responsibilities

```
+-----+
| TEST      base_test / fill_test : picks knobs, runs the env |
+-----+
| ENVIRONMENT env : builds components, connects mailboxes,   |
|             forks threads, decides when the test is over    |
+-----+
| SCENARIO   generator --(mailbox gen2drv, bounded)--+      |
+-----+-----+
| FUNCTIONAL                                scoreboard <--+ |
|                                     (ref model: | |         |
|                                     a queue)    | |         |
+-----+-----+-----+-----+
| COMMAND    monitor --(mailbox mon2scb)-----+ v          |
|             ^                                driver          |
+-----+-----+-----+-----+
| SIGNAL      +----- virtual interface -----+          |
|             fifo_if bus <--> sync_fifo (DUT)          |
+-----+
static world: module top, interface instance, DUT, clock
```

Figure 25.1 - The classic layered testbench. Classes above the line are dynamic objects created at time 0; the signal layer is static hardware.

The split between the *static* world (modules, interfaces, the DUT, the clock) and the *dynamic* world (class objects created with `new` at run time) is fundamental. Classes cannot contain modules or interfaces, and cannot refer to hierarchical signals portably. The only bridge is a **virtual interface**: a class member that holds a handle to an interface instance, assigned once at time 0 by the static top module.

Component	Responsibility	SV features used	UVM class
transaction	One unit of stimulus: operation + data	class, rand, enum, \$urandom	<code>uvm_sequence_item</code>
generator	Creates a stream of transactions	class, mailbox put, loop	<code>uvm_sequence</code> + <code>uvm_sequencer</code>
driver	Transaction -> pin wiggles on the clock	virtual interface, NBA, @(posedge)	<code>uvm_driver #(REQ)</code>
monitor	Pins -> observed items, never drives	virtual interface sampling, mailbox	<code>uvm_monitor</code> + analysis port
scoreboard	Predicts expected results, compares	queue as reference model, <code>!==</code>	<code>uvm_scoreboard</code>
environment	Builds and connects, runs threads	constructors, fork/join_none, wait	<code>uvm_env</code>
test	Chooses configuration and scenario	inheritance, virtual methods	<code>uvm_test</code>

Table 25.1 - Roles in a layered testbench and their UVM equivalents.

WHY LAYERS PAY OFF

Each layer changes for a different reason. A new protocol changes the driver and monitor only; a new scenario changes the generator or test only; a spec change to the data path changes the reference model only. The monitor and scoreboard never depend on the stimulus, so the same checking works for directed, random and replayed traffic - and for traffic from a neighbouring block when the environment is reused at SoC level with the driver switched off (a passive agent).

25.2 The design under test

The DUT is a 4-entry, 8-bit synchronous FIFO with a first-word-fall-through read port: `rdata` always shows the oldest entry, and `pop` removes it at the clock edge. A write is accepted when `push && !full`; a read when `pop && !empty`; both may happen in the same cycle. A compile-time macro `BUG` plants a realistic defect in the write enable, which we will use later.

```
// sync_fifo.sv - DUT: synchronous FIFO, first-word-fall-through read port
module sync_fifo #(parameter int W = 8, parameter int D = 4) (
    input  logic          clk, rst_n,
    input  logic          push, pop,
    input  logic [W-1:0]  wdata,
    output logic [W-1:0]  rdata,
    output logic          full, empty,
    output logic [$clog2(D):0] count
);
    logic [W-1:0]          mem [D];
    logic [$clog2(D)-1:0]  wp, rp;
    logic                  do_push, do_pop;

`ifdef BUG
    assign do_push = push && !full && !pop;    // BUG: write lost on simultaneous push+pop
`else
    assign do_push = push && !full;
`endif
    assign do_pop = pop && !empty;

    always_ff @(posedge clk)                // storage: no reset needed
        if (do_push) mem[wp] <= wdata;

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            wp <= '0; rp <= '0; count <= '0;
        end else begin
            if (do_push) wp <= wp + 1'b1;        // D is a power of 2: pointers wrap
            if (do_pop) rp <= rp + 1'b1;
            unique case ({do_push, do_pop})
                2'b10: count <= count + 1'b1;
                2'b01: count <= count - 1'b1;
                default: ;                        // 00 or 11: occupancy unchanged
            endcase
        end

    assign rdata = mem[rp];
    assign full = (count == ($clog2(D)+1)'(D));
    assign empty = (count == '0);
endmodule
```

`sync_fifo.sv` - lint-clean under verilator --lint-only -Wall. The size cast in the full comparison avoids a WIDTHEXPAND warning against the 32-bit parameter D.

Note the language details that keep this lint-clean and synthesizable: `count` is one bit wider than the pointers so that it can represent D; the occupancy update uses a `unique case` on the two enables rather than `count +`

`do_push` - `do_pop` (legal, and correct, but it mixes 1-bit and 3-bit operands and draws width warnings); and the storage array has no reset, which is what you want for a RAM-like structure in an ASIC (Chapter 11). The FIFO's micro-architecture is treated in depth in the companion RTL Design guide; here it is just a target.

25.3 The interface and the virtual interface

```
// fifo_if.sv - the pin-level boundary between the class world and the DUT
interface fifo_if #(parameter int W = 8, parameter int D = 4) (input logic clk);
    logic          rst_n;
    logic          push, pop;
    logic [W-1:0]  wdata, rdata;
    logic          full, empty;
    logic [$clog2(D):0] count;
    modport dut (input clk, rst_n, push, pop, wdata, output rdata, full, empty, count);
    modport tb  (input clk, rdata, full, empty, count, output rst_n, push, pop, wdata);
endinterface
```

`fifo_if.sv` - one bundle of signals, instantiated once in the top module.

Inside the testbench package the interface is referred to only through a type: `typedef virtual fifo_if vif_t`; . A virtual interface variable is a *reference* - it holds nothing until the top module passes the instance `bus` into the test constructor, and from then on `vif.push <= 1` in a class drives the real signal of that instance. The rules (IEEE 1800-2017 25.9) are: the virtual interface type must match the instance, including parameter values; an uninitialised virtual interface is `null` and dereferencing it is a run-time fatal error; and assignments through it obey the normal rules for the target (variables may be assigned procedurally with `=` or `<=`; nets may not).

PITFALL (PORTABILITY): PARAMETERIZED VIRTUAL INTERFACES

`typedef virtual fifo_if #(W, D) vif_t`; is legal SystemVerilog, and commercial simulators accept it. Verilator 5.020 crashed with an internal fault on exactly that line while this chapter was being written. The portable workaround used here is to give the interface default parameter values that match the instance and declare `virtual fifo_if` without a parameter list, which denotes the default specialization. On large projects the usual alternative is an interface without parameters whose widths come from a package.

25.4 Transactions, and where the constraints would go

The transaction is the unit of communication between the upper layers. It carries an operation and a data byte, plus an id for debug. The members are declared `rand` so that the class is ready for a constraint solver, but because Verilator 5.020 ignores constraints (it compiles `randomize()` yet does not honour `constraint` blocks) this chapter fills the fields with `$urandom_range` and weights chosen by the test.

```
typedef enum bit [1:0] {IDLE, PUSH, POP, BOTH} op_e;

class fifo_txn;
    rand op_e      op;
    rand bit [W-1:0] data;
    int            id;
    // Constraints would go here (ignored by Verilator 5.020, so we use $urandom):
    // constraint c_op { op dist {PUSH := push_w, POP := pop_w, BOTH := 2, IDLE := 1}; }
    function void rand_fill(int push_w, int pop_w);
        int r = $urandom_range(push_w + pop_w + 2, 0); // weights + BOTH(2) + IDLE(1)
        if      (r < push_w)          op = PUSH;
        else if (r < push_w + pop_w)  op = POP;
        else if (r < push_w + pop_w + 2) op = BOTH;
        else                          op = IDLE;
        data = W'($urandom);
    endfunction
    function string sprint();
```

```

        return $sformatf("#%0d %s 0x%02h", id, op.name(), data);
    endfunction
endclass

```

tb_pkg.sv (excerpt) - the transaction as run. \$urandom_range(max, min) returns an unsigned value in [min:max] inclusive.

With a constraint solver (VCS, Xcelium, Questa, Riviera-PRO), the same intent is expressed declaratively and the manual weighting code disappears. The knobs become non-random members that the test sets before calling `randomize()`:

```

class fifo_txn;                                // constrained version - commercial simulator
    rand op_e      op;
    rand bit [7:0] data;
    int unsigned   push_w = 3, pop_w = 3; // knobs, set by the test (not rand)
    constraint c_op { op dist { PUSH := push_w, POP := pop_w, BOTH := 2, IDLE := 1 }; }
    constraint c_data { op == IDLE -> data == 0; } // keep idle cycles tidy in waves
endclass
// generator: fifo_txn t = new(); t.push_w = 6;
//             if (!t.randomize()) $fatal(1, "randomize failed");
//             if (!t.randomize() with { op != IDLE; }) ... // inline constraint

```

Not run: constraint solving needs a commercial simulator (Chapter 19). dist weights may be non-constant expressions of state variables.

RANDOMNESS YOU CAN REPRODUCE

\$urandom in a class is seeded from the thread that constructs it and the simulator's global seed, so a failing run is reproducible by re-running with the same seed: +verilator+seed+N for Verilator, +ntb_random_seed=N (VCS), -svseed N (Xcelium), -sv_seed N (Questa). Always print the seed at the start of a regression test. Never use \$random for stimulus in new code: it is signed, and without an explicit seed argument all calls share one global seed, so adding a call anywhere changes every later value (no random stability, Chapter 27).

25.5 The generator and mailboxes

```

class generator;
    mailbox #(fifo_txn) out;
    int n = 20, push_w = 3, pop_w = 3;
    function new(mailbox #(fifo_txn) out); this.out = out; endfunction
    task run();
        for (int i = 0; i < n; i++) begin
            fifo_txn t = new();
            t.id = i;
            t.rand_fill(push_w, pop_w); // with constraints: if (!t.randomize()) $fatal;
            if (i < 3) $display("GEN: %s", t.sprint());
            out.put(t); // blocks when the bounded mailbox is full
        end
    endtask
endclass

```

tb_pkg.sv (excerpt) - the generator.

Two details matter. First, a **new object is constructed for every item** (`fifo_txn t = new()`; inside the loop body, which is an automatic scope because a class method is automatic). Re-using one handle and re-randomizing it is a classic bug: the mailbox stores handles, not copies, so every queued entry would point at the same object and the driver would see the last values N times. Second, the mailbox is **parameterized** (`mailbox #(fifo_txn)`), so a `put` of the wrong type is a compile error instead of a run-time surprise.

Method	Blocks?	Behaviour
new(bound)	-	bound 0 (default) = unbounded; bound N = at most N items
put(item)	Yes, if full	Append in FIFO order
get(ref item)	Yes, if empty	Remove the oldest item

Method	Blocks?	Behaviour
peek(ref item)	Yes, if empty	Copy the oldest handle, do not remove
try_put / try_get / try_peek	No	Return 1 on success, 0 otherwise
num()	No	Current number of items (a snapshot; may change at once)

Table 25.2 - std::mailbox methods (IEEE 1800-2017 15.4).

The environment gives `gen2drv` a bound of 2. The generator can therefore run at most two items ahead of the driver, which keeps memory bounded, keeps the generator's `$display` roughly aligned with what is on the pins, and - more importantly - makes it possible for a *reactive* generator to base the next item on the current DUT state. UVM reaches the same effect with the sequence/driver handshake (`get_next_item/item_done`), which is effectively a mailbox of depth one.

25.6 The driver

```
class driver;
  vif_t vif;
  mailbox #(fifo_txn) in;
  int n_driven;
  function new(vif_t vif, mailbox #(fifo_txn) in);
    this.vif = vif; this.in = in;
  endfunction
  task reset();
    vif.rst_n <= 0; vif.push <= 0; vif.pop <= 0; vif.wdata <= '0;
    repeat (2) @(posedge vif.clk);
    vif.rst_n <= 1;
  endtask
  task run();
    forever begin
      fifo_txn t;
      in.get(t);
      @(posedge vif.clk);
      vif.push <= t.op inside {PUSH, BOTH};
      vif.pop <= t.op inside {POP, BOTH};
      vif.wdata <= t.data;
      n_driven++;
    end
  endtask
endclass
```

tb_pkg.sv (excerpt) - the driver: one transaction per clock cycle.

The driver waits for a rising edge and then updates the interface with **nonblocking** assignments. That is the key to a race-free testbench without clocking blocks: at a given edge, the DUT's `always_ff` blocks and the monitor all wake up in the Active region and read the *old* values of `push`, `pop` and `wdata`; the driver's new values only land in the NBA region of that same time step, exactly like the output of a flip-flop. Had the driver used blocking `=`, whether the DUT saw the old or new value at that edge would depend on which process the simulator happened to run first (Chapter 5).

CLOCKING BLOCKS: THE INDUSTRIAL VERSION OF THE SAME IDEA

Production drivers usually go through a clocking block (`vif.cb.push <= ...; @(vif.cb);`) with an output skew, and monitors sample through it with an input skew (`#1step` by default, i.e. the value just before the edge in the Preponed region). This makes the sampling point explicit and independent of process order, and is what a UVM agent normally uses (Chapter 21). Clocking-block support still varies between tools, so this chapter uses the equivalent 'drive with NBA after the edge, sample at the edge' discipline, which is race-free for a single synchronous clock and portable.

25.7 The monitor

```
class monitor;
  vif_t vif;
  mailbox #(obs_item) out;
  int n_push_full, n_pop_empty;
  function new(vif_t vif, mailbox #(obs_item) out);
    this.vif = vif; this.out = out;
  endfunction
  task run();
    forever begin
      @(posedge vif.clk);
      if (vif.rst_n) begin // sample pre-edge values: what the DUT saw
        if (vif.pop && !vif.empty) send(0, vif.rdata);
        if (vif.push && !vif.full) send(1, vif.wdata);
        if (vif.push && vif.full) n_push_full++; // poor man's coverage
        if (vif.pop && vif.empty) n_pop_empty++;
      end
    end
  endtask
  function void send(bit w, bit [W-1:0] d);
    obs_item o = new();
    o.is_write = w; o.data = d;
    void'(out.try_put(o)); // unbounded mailbox: never fails
  endfunction
endclass
```

tb_pkg.sv (excerpt) - the monitor. obs_item is a two-field class {bit is_write; bit [W-1:0] data;}.

The monitor is completely passive: it reads the interface and never drives it. It applies the **protocol** rule - a write happens when `push && !full` at a clock edge - which comes from the interface specification, not from the RTL. That independence is what allows it to detect a DUT that *claims* to accept a write and then loses it. Note also the order: the read is reported before the write of the same edge, because for a first-word-fall-through FIFO a simultaneous push and pop must return the *old* head. The monitor is a function of pin values only, so it is also correct when the DUT is driven by some other block in a larger system.

The two counters are a stand-in for functional coverage (Chapter 23), which Verilator does not implement. They answer the question every passing test must answer: did we actually exercise the interesting cases - here, pushing into a full FIFO and popping an empty one?

25.8 The scoreboard and reference model

```
class scoreboard;
  mailbox #(obs_item) in;
  bit [W-1:0] model[$]; // reference model: a queue
  int n_wr, n_rd, n_err;
  function new(mailbox #(obs_item) in); this.in = in; endfunction
  task run();
    forever begin
      obs_item o;
      in.get(o);
      if (o.is_write) begin
        model.push_back(o.data); n_wr++;
      end else begin
        bit [W-1:0] exp;
        n_rd++;
        if (model.size() == 0) error($sformatf("read 0x%02h from empty model", o.data));
        else begin
          exp = model.pop_front();
          if (o.data != exp)
            error($sformatf("data mismatch: got 0x%02h exp 0x%02h", o.data, exp));
        end
      end
    end
  endtask
endclass
```



```

        end
    end
endtask
function void error(string msg);
    n_err++;
    if (n_err <= 3) $display("[%0t] SCB ERROR: %s", $time, msg);
endfunction
function void check_final(int unsigned dut_count);    // end-of-test check
    if (dut_count != model.size())
        error($sformatf("final occupancy: DUT %0d model %0d", dut_count, model.size()));
endfunction
function void report();
    $display("SCB: writes=%0d reads=%0d left_in_model=%0d errors=%0d",
            n_wr, n_rd, model.size(), n_err);
    if (n_err == 0) $display("*** TEST PASSED ***");
    else          $display("*** TEST FAILED ***");
endfunction
endclass

```

tb_pkg.sv (excerpt) - scoreboard with a queue-based reference model.

A FIFO's reference model is a queue, which is why SystemVerilog queues ([[\\$](#)] with [push_back/pop_front](#), Chapter 13) are the most used data structure in scoreboards. For a more complex DUT the model might be a function, a transaction-level class, or a C model called through DPI (Chapter 24). Three details are worth copying:

- The comparison uses `!==`, not `!=`. If the DUT returns X (an unwritten RAM location, an uninitialised pointer), `!=` yields X, the `if` takes the false branch, and the error is silently missed (Chapter 27). `!==` compares all four states.
- Errors are counted but only the first few are printed. After the first mismatch a FIFO is usually out of step, and 500 follow-on messages hide the one that matters.
- There is an **end-of-test check** (`check_final`): data still in the model must also be in the DUT. Without it a DUT that drops the last few writes passes whenever the test ends before they would have been read.

25.9 The environment: construction, threads and end of test

```

class env;
    vif_t vif;
    generator gen;    driver drv;    monitor mon;    scoreboard scb;
    mailbox #(fifo_txn) gen2drv;
    mailbox #(obs_item) mon2scb;
    function new(vif_t vif);
        this.vif = vif;
        gen2drv = new(2);                // bounded: generator runs just ahead
        mon2scb = new();                 // unbounded
        gen = new(gen2drv);
        drv = new(vif, gen2drv);
        mon = new(vif, mon2scb);
        scb = new(mon2scb);
    endfunction
    task run();
        drv.reset();
        fork                            // long-lived component threads
            drv.run();
            mon.run();
            scb.run();
        join_none
        gen.run();                      // returns when all items are in the mailbox
        wait (drv.n_driven == gen.n);    // end-of-test: all stimulus applied
        @(posedge vif.clk);
    endtask
endclass

```

```

    vif.push <= 0; vif.pop <= 0;
    repeat (3) @(posedge vif.clk); // drain: let the monitor/scoreboard catch up
    disable fork; // kill driver, monitor, scoreboard
    scb.check_final(int'(vif.count));
    $display("MON: push-while-full=%0d pop-while-empty=%0d",
            mon.n_push_full, mon.n_pop_empty);
    scb.report();
endtask
endclass

```

tb_pkg.sv (excerpt) - the environment builds, connects and orchestrates.

The constructor is the **build and connect** step: it creates both mailboxes and hands the same mailbox handle to its producer and its consumer. `run()` is the **run** step, and it shows the standard fork/join pattern for testbench components:

- 1 Components whose `run()` loops **forever** (driver, monitor, scoreboard) are launched with `fork ... join_none`: the parent continues immediately and the children run as independent processes.
- 2 The finite activity - the generator - runs in the parent thread, so its return marks 'all stimulus created'.
- 3 **End of test** is a condition, not a delay: wait until the driver has applied every item, idle the inputs, wait a few drain cycles for in-flight results, then stop the forever-threads with `disable fork` and run the final checks. A fixed `#10000` would be either too short (false pass - checks never ran) or too long (wasted simulation), and silently wrong when the test length changes.
- 4 The final checks and the report come *after* the threads are stopped, so nothing changes the scoreboard state while it is being summarized.

time ->	reset	<----- run ----->	drain ->	report
parent	drv.reset	fork..join_none gen.run()	wait 3 clk	disable fork
driver		get / drive / get / drive		X
monitor		sample every posedge		X
scoreboard		get / compare		X

Figure 25.2 - Thread timeline of `env.run()`. X marks where `disable fork` terminates the forever loops.

PITFALL: DISABLE FORK KILLS MORE THAN YOU THINK

`disable fork` terminates *all* active descendants of the calling process, not just those created by the most recent fork. If a test had launched, say, a watchdog thread earlier from the same initial block, it would die too. The defensive idiom isolates the children in their own process: `fork begin fork a(); b(); join_none; ...; disable fork; end join`. UVM avoids the issue by ending phases with objections and killing only the run-phase processes it created. The alternative - `process` handles obtained with `process::self()` and `p.kill()` - gives precise control (Chapter 20).

25.10 Tests and the top module

```

class base_test;
  env e;
  function new(vif_t vif); e = new(vif); endfunction
  virtual function void configure(); endfunction
  task run();
    configure();
    $display("TEST %s: %0d items, push_w=%0d pop_w=%0d",
            name(), e.gen.n, e.gen.push_w, e.gen.pop_w);
    e.run();
  endtask
  virtual function string name(); return "base_test"; endfunction
endclass

class fill_test extends base_test; // push-heavy: reaches full and overflow
  function new(vif_t vif); super.new(vif); endfunction
  virtual function void configure();
    e.gen.n = 200; e.gen.push_w = 6; e.gen.pop_w = 2;

```

```

    endfunction
    virtual function string name(); return "fill_test"; endfunction
endclass
endpackage

```

tb_pkg.sv (end) - a test is a configuration of the environment, specialised by overriding virtual methods.

```

// top.sv - static world: clock, interface, DUT; then hand the vif to the test
module top;
    import tb_pkg::*;
    logic clk = 0;
    always #5 clk = ~clk;

    fifo_if bus (clk); // default W=8, D=4 matches vif_t
    sync_fifo #(W(W), D(D)) dut (
        .clk(clk), .rst_n(bus.rst_n), .push(bus.push), .pop(bus.pop), .wdata(bus.wdata),
        .rdata(bus.rdata), .full(bus.full), .empty(bus.empty), .count(bus.count));

    initial begin
        string tname;
        base_test t;
        if (!$value$plusargs("TEST=%s", tname)) tname = "base_test";
        case (tname) // a hand-made "factory" (see Chapter 26)
            "fill_test": begin fill_test f = new(bus); t = f; end
            default: t = new(bus);
        endcase
        t.run();
        $finish;
    end
endmodule

```

top.sv - the only module in the testbench.

The test is chosen at run time with a plusarg, so one compiled executable runs the whole regression. The base-class handle `t` can hold a `fill_test`, and because `configure()` and `name()` are `virtual`, `t.run()` calls the derived versions - polymorphism doing real work. The `case` statement is a hand-written factory: it maps a string to a constructor. Adding a test means editing it, which is exactly the coupling UVM's factory removes (Chapter 26). The DUT ports are connected to interface members individually because the FIFO is written with plain ports, as most IP is; a DUT with an interface port would take `bus.dut` directly.

25.11 Building and running

```

FL="--binary --timing -Wall -Wno-fatal"
FL="$FL -Wno-DECLFILENAME -Wno-VARHIDDEN -Wno-UNUSED SIGNAL -Wno-UNDRIVEN"
verilator $FL --top-module top sync_fifo.sv fifo_if.sv tb_pkg.sv top.sv -o Vtop
./obj_dir/Vtop # base_test
./obj_dir/Vtop +TEST=fill_test
./obj_dir/Vtop +verilator+seed+7 # a different random stream

```

Build and run with Verilator 5.020. File order matters: the interface before the package that names it, the package before the module that imports it.

The waived warnings are testbench noise, not design problems: `VARHIDDEN` fires on every constructor argument named like the member it initialises (`this.vif = vif`), `UNDRIVEN/UNUSED SIGNAL` fire on interface signals that are driven or read only from classes through a virtual interface, which Verilator's static analysis cannot see. The one remaining warning, `BLKSEQ` on `always #5 clk = ~clk`, is harmless for a clock generator. Here is the real output:

```

TEST base_test: 20 items, push_w=3 pop_w=3
GEN: #0 IDLE 0x9c
GEN: #1 PUSH 0xe4
GEN: #2 POP 0xbc
MON: push-while-full=0 pop-while-empty=6
SCB: writes=9 reads=9 left_in_model=0 errors=0
*** TEST PASSED ***
- top.sv:21: Verilog $finish

```

Verilator 5.020: ./obj_dir/Vtop (base_test, default seed).

```

TEST fill_test: 200 items, push_w=6 pop_w=2
GEN: #0 POP 0x9c
GEN: #1 POP 0xe4
GEN: #2 PUSH 0xbc
MON: push-while-full=72 pop-while-empty=5
SCB: writes=76 reads=73 left_in_model=3 errors=0
*** TEST PASSED ***
- top.sv:21: Verilog $finish

```

Verilator 5.020: ./obj_dir/Vtop +TEST=fill_test.

```

TEST base_test: 20 items, push_w=3 pop_w=3
GEN: #0 BOTH 0xff
GEN: #1 POP 0x02
GEN: #2 BOTH 0x28

```

Verilator 5.020: first lines with +verilator+seed+7 - a different, reproducible stream.

Read the coverage line before you believe the PASS. The base test never pushed into a full FIFO (**push-while-full=0**), so it says nothing about overflow protection; only **fill_test** exercised it (72 times). The fill test also ends with three entries in the FIFO, and the end-of-test check confirmed that the DUT's **count** agreed with the model.

25.12 Catching a bug

Now compile the same testbench against the defective RTL by defining **BUG**, which drops a write whenever **pop** is asserted in the same cycle. This is a realistic bug: it passes every directed test that pushes a batch and then pops it, because those never overlap push and pop.

```

verilator $FL -DBUG --Mdir obj_bug --top-module top sync_fifo.sv fifo_if.sv \
    tb_pkg.sv top.sv -o Vtop
./obj_bug/Vtop
for s in 1 2 3 4 5; do ./obj_bug/Vtop +verilator+seed+$s | grep -E "SCB:|*\`*\`"; done

```

Same testbench, buggy DUT.

```

TEST base_test: 20 items, push_w=3 pop_w=3
GEN: #0 IDLE 0x9c
GEN: #1 PUSH 0xe4
GEN: #2 POP 0xbc
[95] SCB ERROR: data mismatch: got 0xe1 exp 0x53
[155] SCB ERROR: data mismatch: got 0x58 exp 0xe1
[255] SCB ERROR: final occupancy: DUT 0 model 6
MON: push-while-full=0 pop-while-empty=12
SCB: writes=9 reads=3 left_in_model=6 errors=3
*** TEST FAILED ***
- top.sv:21: Verilog $finish

```

Verilator 5.020: ./obj_bug/Vtop - the scoreboard catches the lost writes three different ways.

```

SCB: writes=6 reads=3 left_in_model=3 errors=1
*** TEST FAILED ***
SCB: writes=11 reads=6 left_in_model=5 errors=7
*** TEST FAILED ***
SCB: writes=12 reads=5 left_in_model=7 errors=4
*** TEST FAILED ***
SCB: writes=8 reads=6 left_in_model=2 errors=4
*** TEST FAILED ***
SCB: writes=10 reads=5 left_in_model=5 errors=1
*** TEST FAILED ***

```

Verilator 5.020: five seeds, five failures.

Look at how the failure presents. The monitor recorded 9 writes because 9 times the pins satisfied `push && !full`; the DUT kept only some of them. The data comparison fails as soon as a lost entry should have been read, and the end-of-test check fails because the DUT reports 0 entries while the model holds 6. Note too that the first error is at 95 ns, not at the edge where the write was lost: a scoreboard detects the *symptom*. The debug step is to find the first lost write in the waveform (Verilator `--trace`, then GTKWave or Surfer) and walk back to `do_push`.

WHY RANDOM FOUND IT AND A DIRECTED TEST WOULD NOT

The bug needs push and pop in the same cycle while the FIFO is non-empty. Random stimulus with a BOTH weight produces that combination constantly; a hand-written 'write four, read four' test never does. This is the whole argument for constrained-random verification in one example - and also the argument for coverage: if a regression shows the (push, pop, not-empty) cross has never been hit, the PASS is worthless.

INTERVIEW INSIGHT: WHAT MAKES A SCOREBOARD TRUSTWORTHY

(1) The model is written from the specification, not copied from the RTL. (2) The monitor derives transactions from pins using the protocol, never from DUT internals. (3) Comparisons are 4-state (`!==`). (4) There is an end-of-test check for outstanding items. (5) The test proves it can fail - running it against a planted bug, as here, is called *mutation* or *fault seeding* and is the cheapest sanity check of a new environment.

25.13 From here to a production environment

What is missing here	Language feature	Where covered
Declarative stimulus	constraint, dist, inline with, pre/post_randomize	Ch. 19
Measured completeness	covergroup, coverpoint, cross	Ch. 23
Cycle-level protocol checks in the interface	concurrent assertions (assert property)	Ch. 22
Race-free sampling with skews	clocking blocks, program blocks	Ch. 21
Test selection without editing top.sv	static registry, factory	Ch. 26
Configuration without long constructor lists	parameterized class with static associative array (config_db)	Ch. 26
Clean, cooperative end of test	objections over phases	Ch. 26
Broadcast to several subscribers	analysis ports calling write()	Ch. 26

Table 25.3 - The distance between this chapter and an industrial testbench.

Everything in the right-hand column is a language feature you already know. A UVM environment for this FIFO would have the same seven classes, with base classes supplied by the library.

25.14 Summary

- A layered testbench separates the static world (top module, interface, DUT) from dynamic class objects; a virtual interface is the only bridge.
- Transactions flow downward through a bounded mailbox to the driver; observed items flow upward from a passive monitor through a second mailbox to the scoreboard.
- Drive with nonblocking assignments after the clock edge (or a clocking block) and sample at the edge; that makes the testbench race-free with respect to the DUT's flip-flops.
- The scoreboard's reference model is written from the spec; for a FIFO it is a queue. Compare with `!==` and check for outstanding items at the end of the test.
- Long-lived components start with `fork ... join_none`; the test ends on a condition (all items driven, drain), not a fixed delay, and `disable fork` kills every descendant of the calling process.
- Verilator 5.020 ran the whole environment; randomization used `$urandom_range` because it ignores constraints; a parameterized virtual interface typedef crashed it, so default parameters were used instead.
- A planted bug (write lost on simultaneous push and pop) was caught on every seed by random stimulus, while a simple directed test would have missed it.

25.15 Exercises

- 1 Add a `peek`-based check to the monitor: whenever `empty` is 0, `rdata` must equal the head of the model queue even when no pop happens. Which extra class of bug does this catch?
- 2 Replace the unbounded `mon2scb` mailbox by direct calls: give the monitor a scoreboard handle and call `scb.write(o)` from `send()`. What does this gain and lose? (Hint: this is how a UVM analysis port works.)
- 3 Plant a second bug: `full` asserted at `count == D-1`. Does the current scoreboard detect it? If not, add the check that does, and decide whether it belongs in the monitor, the scoreboard or an assertion in the interface.
- 4 Make the generator reactive: when the monitor has seen 10 consecutive cycles with `full` high, bias the next 20 items toward POP. Which component needs a handle to which, and how do you avoid a race on the shared counter?
- 5 Rewrite `disable fork` using `process` handles: store the three component processes in a queue and `kill()` them explicitly. Show a situation in which the two versions behave differently.
- 6 Port the testbench to a commercial simulator (EDA Playground offers free access) and replace `rand_fill` with the constrained `fifo_txn` shown earlier. Verify that the op distribution matches the weights by counting 10,000 items.

26. How UVM Uses the Language

The Universal Verification Methodology (UVM, standardised as IEEE 1800.2) is not a language extension. It is a class library - a package called `uvm_pkg` plus a file of macros, `uvm_macros.svh` - written entirely in IEEE 1800 SystemVerilog. Every mechanism that makes UVM look magical to a newcomer (the factory, `type_id::create`, the configuration database, phases, objections, sequences, TLM ports) is built from a handful of features you already know: class inheritance and virtual methods, parameterized classes, static members, associative arrays, macros, and processes. This chapter maps each mechanism to the feature beneath it, gives a complete UVM skeleton for the FIFO of Chapter 25, and ends with a 120-line 'mini UVM' in plain SystemVerilog that runs on Verilator and demystifies the factory, `config_db`, phasing and objections.

TOOL SUPPORT FOR THIS CHAPTER

UVM needs a simulator that supports the full class, randomization and process feature set: VCS, Xcelium, Questa and Riviera-PRO ship UVM pre-compiled, and EDA Playground lets you run it in a browser. Verilator 5.020 cannot compile the UVM library, and Icarus cannot either, so the UVM code here is shown without output. The mini-UVM model at the end of the chapter was run with Verilator 5.020.

26.1 The map: UVM mechanism -> SystemVerilog feature

UVM mechanism	SystemVerilog feature underneath	Book chapter
uvm_object / uvm_component hierarchy	class inheritance, virtual methods, virtual (abstract) classes	18
uvm_driver #(REQ,RSP), TLM ports, uvm_config_db #(T)	parameterized classes (type parameters)	18
factory, type_id::create, overrides	static members, singleton proxies, associative arrays keyed by string, \$cast	13, 18
uvm_component_utils and the other utility macros	text macros with argument stringification	8
config_db, resources	parameterized class with static associative arrays	13, 18
virtual interfaces via config_db	virtual interface as a type parameter	16
phases	virtual functions (zero-time) and virtual tasks (run phases), called by a scheduler	6, 18
objections	counters + events + wait, per phase	20
sequences, sequencer arbitration	tasks, fork/join, process class, semaphores/events	20
analysis ports	parameterized port class calling write() on a list of subscribers	18
uvm_reg (RAL) backdoor	DPI-C / VPI access by HDL path	24
uvm_report_* messages	\$sformatf, \$display, static report server	10

Table 26.1 - Every UVM mechanism is a SystemVerilog idiom.

26.2 Class hierarchy and polymorphism

```

uvm_void                                (virtual, empty: the common root)
+-- uvm_object                          name, copy/compare/print/pack, create()
  +-- uvm_transaction
    |  +-- uvm_sequence_item            your transactions
    |  +-- uvm_sequence_base
    |      +-- uvm_sequence #(REQ, RSP)  your sequences: body()
+-- uvm_report_object                  messaging
  +-- uvm_component                    parent/children, phases, config
    |  +-- uvm_driver #(REQ, RSP)
    |  +-- uvm_sequencer #(REQ, RSP)
    |  +-- uvm_monitor, uvm_scoreboard, uvm_agent, uvm_env
    |  +-- uvm_test
+-- uvm_reg, uvm_reg_block, ... register model (RAL)

```

Figure 26.1 - The core of the UVM class tree (IEEE 1800.2).

Two branches matter. **Objects** (`uvm_object`) are transient data: transactions, sequences, configuration objects. They are created and destroyed freely and have no position in the testbench. **Components** (`uvm_component`) are the permanent structure: they are created once during the build phase, each has a parent and a hierarchical name (for example `uvm_test_top.env.agt.drv`), and they take part in phasing. The distinction is visible in their constructors: `new(string name = "")` for objects and `new(string name, uvm_component parent)` for components.

Polymorphism is the engine. The library holds everything through base-class handles and calls virtual methods: the phase scheduler calls `comp.build_phase(phase)` on a `uvm_component` handle and your override runs; `uvm_object::copy()` is non-virtual and calls the virtual `do_copy()`, which you override. The standard idiom inside such an override uses `$cast` to recover the derived type:

```

class fifo_item extends uvm_sequence_item;
  rand bit push, pop;  rand bit [7:0] data;
  `uvm_object_utils(fifo_item)
  function new(string name = "fifo_item");  super.new(name);  endfunction
  virtual function void do_copy(uvm_object rhs);
    fifo_item that;
    if (!$cast(that, rhs)) `uvm_fatal("CAST", "do_copy: wrong type")
    super.do_copy(rhs);           // copy base-class fields first
    push = that.push;  pop = that.pop;  data = that.data;
  endfunction
  virtual function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    fifo_item that;
    return $cast(that, rhs) && super.do_compare(rhs, comparer) &&
           push == that.push && pop == that.pop && data == that.data;
  endfunction
  virtual function string convert2string();
    return $sprintf("push=%0b pop=%0b data=0x%02h", push, pop, data);
  endfunction
endclass

```

Not run (needs UVM). The `do_*` hooks are the virtual-method pattern 'non-virtual public API calls virtual protected hook'.

FIELD MACROS VERSUS DO_* METHODS

The `uvm_field_int` family of macros generates copy/compare/print code automatically, but it expands into a large generic function that is slow and hard to debug. Many companies ban field macros and require hand-written `do_copy`, `do_compare` and `convert2string`. Knowing that the macro only writes these methods for you makes the choice easy.

26.3 Parameterized classes: drivers, sequencers and ports

A driver must know the type of item it receives, but the library cannot know your item type in advance. The solution is a type parameter: `class uvm_driver #(type REQ = uvm_sequence_item, type RSP = REQ) extends uvm_component;` Inside, the library declares `REQ req;` and a port `uvm_seq_item_pull_port #(REQ, RSP) seq_item_port;`. When you write `class fifo_driver extends uvm_driver #(fifo_item);` you create a *specialization*, a distinct class in which `req` has type `fifo_item`, so `req.data` compiles without a cast.

TLM ports are parameterized the same way, and one of them uses a trick worth knowing. An analysis imp is declared `uvm_analysis_imp #(fifo_obs, fifo_scoreboard)`: the second parameter is the type of the component that implements `write()`. Inside, the imp stores a handle of that type and its own `write(t)` calls `m_imp.write(t)`. The call is resolved when the specialization is elaborated, so if `fifo_scoreboard` has no `write` method the compile fails. This is 'duck typing at elaboration time' through type parameters; the language did not have interface classes (Chapter 18) when the pattern was designed.

```
// simplified from the UVM reference implementation
class uvm_analysis_port #(type T = int) extends uvm_port_base #(uvm_tlm_if_base #(T,T));
    function void write(input T t);                // broadcast: zero time, never blocks
        uvm_tlm_if_base #(T,T) tif;
        for (int i = 0; i < this.size(); i++) begin
            tif = this.get_if(i);                  // every connected subscriber
            tif.write(t);
        end
    endfunction
endclass

class uvm_analysis_imp #(type T = int, type IMP = int)
    extends uvm_port_base #(uvm_tlm_if_base #(T,T));
    local IMP m_imp;                               // the component that owns the imp
    function new(string name, IMP imp); super.new(name, imp, UVM_IMPLEMENTATION, 1, 1);
        m_imp = imp;
    endfunction
    function void write(input T t); m_imp.write(t); endfunction // IMP must have write()
endclass
```

Not run. Simplified: the real classes add macros for the port boilerplate. Because `write()` is a function, analysis traffic cannot consume time.

When a component needs two analysis inputs (for example the expected and actual streams of a scoreboard), bothimps would call the same `write()`. The macro `uvm_analysis_imp_decl(_exp)` (invoked with a leading backtick) solves this by declaring a new imp class, `uvm_analysis_imp_exp`, whose `write()` calls `write_exp()` - text macros generating classes, nothing more. The alternative is to instantiate `uvm_tlm_analysis_fifo #(T)` members, which contain a mailbox, and `get()` from them in the run phase.

26.4 Static registries and the factory

The factory solves the problem Chapter 25 solved with a `case` statement: create an object whose concrete type is chosen at run time, without the code that creates it knowing all the possible types. The rule in UVM is **never call new() for components or items directly; call T::type_id::create()**. That one level of indirection lets a test replace any type anywhere in the hierarchy without editing the environment.

The mechanism has three pieces:

- 1 A **proxy** (wrapper) class per registered type. `uvm_component_registry #(T, "T")` is a parameterized class; its specialization for `fifo_driver` knows how to call `new` for a `fifo_driver`. It extends the abstract `uvm_object_wrapper`, whose virtual `create_component()` it implements.
- 2 A **singleton** instance of each proxy, created and registered by a static member initializer (`local static this_type me = get();` in UVM 1.2; IEEE 1800.2 releases defer the same registration to `uvm_init`). Static initialization runs before any `initial` block, so every type is registered before `run_test()` starts.

- 3 The **factory** singleton, which holds associative arrays mapping type names and proxy handles to proxies, plus override tables. `create()` looks up the requested type, follows any override chain, asks the final proxy to construct the object and returns it through a base-class handle; the static `create()` in the registry then `$casts` it back to T.

```
// In a test: replace every fifo_item created anywhere by fifo_full_item
class fill_test extends base_test;
  `uvm_component_utils(fill_test)
  function new(string name, uvm_component parent); super.new(name, parent); endfunction
  virtual function void build_phase(uvm_phase phase);
    fifo_item::type_id::set_type_override(fifo_full_item::get_type());
    // or only under one instance path:
    // fifo_item::type_id::set_inst_override(fifo_full_item::get_type(), "env.agt.*", this);
    super.build_phase(phase); // overrides must precede creation
  endfunction
endclass

class fifo_full_item extends fifo_item; // same constraint name overrides the base one
  `uvm_object_utils(fifo_full_item)
  constraint c_mix { {push, pop} dist { 2'b10 := 8, 2'b01 := 1, 2'b11 := 1 }; }
  function new(string name = "fifo_full_item"); super.new(name); endfunction
endclass
```

Not run (needs UVM). A factory override plus constraint override by name turns the base test into a fill test without touching the sequence.

PITFALL: CALLING NEW() DEFEATS THE FACTORY

`drv = new("drv", this);` compiles and works - until someone overrides `fifo_driver` and nothing happens, because `new` always constructs exactly the declared class. Code review rule: in UVM code, `new` appears only inside constructors (`super.new`) and for ports, FIFOs and covergroups, which are not factory-registered. A related pitfall: an override set *after* the object was created has no effect, so overrides go at the top of `build_phase` before `super.build_phase()`.

26.5 What the utility macros expand to

The registration code is identical for every class except the type name, which is exactly what text macros are for (Chapter 8). A simplified expansion of `uvm_component_utils(fifo_driver)`:

```
// `uvm_component_utils(fifo_driver) expands (simplified) to:
typedef uvm_component_registry #(fifo_driver, "fifo_driver") type_id;
static function type_id get_type();
  return type_id::get(); // the singleton proxy
endfunction
virtual function uvm_object_wrapper get_object_type();
  return type_id::get();
endfunction
const static string type_name = "fifo_driver";
virtual function string get_type_name();
  return type_name;
endfunction
```

The type name string is produced by stringifying the macro argument (a grave accent followed by a double quote) inside the macro body. `uvm_object_utils` is the same with `uvm_object_registry` and an extra `create()` method.

Macro	For	Adds
uvm_object_utils(T)	non-parameterized objects	<code>type_id</code> (<code>uvm_object_registry</code>), <code>get_type</code> , <code>get_type_name</code> , <code>create</code>
uvm_object_param_utils(T)	parameterized objects	same, but registered without a string name (by type only)

Macro	For	Adds
<code>uvm_component_utils(T)</code>	components	<code>type_id (uvm_component_registry)</code> , <code>get_type</code> , <code>get_type_name</code>
<code>uvm_component_param_utils(T)</code>	parameterized components	type only
<code>uvm_*_utils_begin</code> , <code>uvm_field_*</code> , <code>uvm_*_utils_end</code>	objects or components	above + generated copy/compare/print/pack code
<code>uvm_info</code> , <code>uvm_warning</code> , <code>uvm_error</code> , <code>uvm_fatal</code>	any code	a verbosity check, then <code>uvm_report_*</code> with <code>__FILE__</code> and <code>__LINE__</code>

Table 26.2 - The common UVM macros (invoked with a leading backtick).

The parameterized variants exist because a specialization such as `my_driver #(8)` has no single name string; a stringified `T` would contain the parameter text, and different specializations must be different registry entries. Such classes can be created and overridden only by type, not by name. The reporting macros exist so that the message is formatted only if its verbosity is enabled, which keeps expensive `$sformatf` calls out of fast regressions.

26.6 config_db: a parameterized class with static storage

`uvm_config_db #(type T = int)` is a class with only static methods. Each specialization - `uvm_config_db #(int)`, `uvm_config_db #(virtual fifo_if)`, `uvm_config_db #(fifo_cfg)` - is a separate class with its own static state, which is how the database is type-safe: a value set as `int` can only be retrieved as `int`. Underneath, entries live in a global resource pool whose lookup tables are associative arrays keyed by field name, each entry carrying the scope pattern (a glob or regular expression over hierarchical names) to which it applies.

```
static function void set(uvm_component cntxt, string inst_name, string field_name,
                        T value);
static function bit  get(uvm_component cntxt, string inst_name, string field_name,
                        inout T value);
// scope = {cntxt.get_full_name(), ".", inst_name} (cntxt == null -> uvm_root)
uvm_config_db #(int)::set(this, "env.agt*", "is_active", UVM_PASSIVE); // from a test
if (!uvm_config_db #(int)::get(this, "", "n_items", n)) n = 50;         // in a component
```

Not run. The signatures (IEEE 1800.2) and typical calls.

- **Precedence during build:** a `set` from a component higher in the hierarchy wins over one from lower down, so the test can override what the environment configures for its children. Among sets from the same context, the last one wins.
- **get in build_phase:** configuration is read top-down as the tree is built, so a component's parent has already run its `build_phase` (and its sets) before the child reads.
- **Failed get:** `get` returns 0 and leaves `value` unchanged; always check the return value and either default or `uvm_fatal`.
- **Typos are silent:** a wrong field name or path pattern simply never matches. `+UVM_CONFIG_DB_TRACE` prints every set and get and is the first debugging step.

Virtual interfaces through config_db

The static top module is the only place that can see the interface instance, and the driver is a class created later by the factory, so the two meet through the database, with `virtual fifo_if` as the type parameter:

```

module tb_top;
  import uvm_pkg::*;
  logic clk = 0;  always #5 clk = ~clk;
  fifo_if bus (clk);
  sync_fifo dut (.clk, .rst_n(bus.rst_n), .push(bus.push), .pop(bus.pop),
                .wdata(bus.wdata), .rdata(bus.rdata), .full(bus.full),
                .empty(bus.empty), .count(bus.count));

  initial begin
    uvm_config_db #(virtual fifo_if)::set(null, "uvm_test_top.env.agt.*", "vif", bus);
    run_test(); // test name from +UVM_TESTNAME=...
  end
endmodule

// in the driver / monitor:
// if (!uvm_config_db #(virtual fifo_if)::get(this, "", "vif", vif))
//   `uvm_fatal("NOVIF", "no virtual interface for this agent")

```

Not run. The set must happen before run_test() starts the build phase.

26.7 Phases: virtual methods called by a scheduler

Phase	Kind	Order	Typical use
build	function	top-down	create children with type_id::create, get config
connect	function	bottom-up	connect ports: drv.seq_item_port.connect(...)
end_of_elaboration	function	bottom-up	print topology, final checks of setup
start_of_simulation	function	bottom-up	print banners, open files
run (and 12 run-time sub-phases)	task	parallel	drive, monitor, run sequences - the only phases that consume time
extract	function	bottom-up	gather final state from DUT and models
check	function	bottom-up	end-of-test checks (outstanding items)
report	function	bottom-up	print results
final	function	top-down	close files, \$finish handling

Table 26.3 - The common UVM phases.

Each phase is a virtual method of `uvm_component` with an empty default body; you override the ones you need. The scheduler walks the component tree and calls the function phases in the order shown. Build is top-down because a parent must exist - and must have run its build - before it can create its children; the others are bottom-up. For the run phase the scheduler forks one process per component, calling each `run_phase(phase)` task, and all of them run concurrently. The 12 run-time sub-phases (pre_reset, reset, post_reset, pre_configure, configure, post_configure, pre_main, main, post_main, pre_shutdown, shutdown, post_shutdown) run in sequence, in parallel with run_phase; most projects use only run_phase.

Objections: ending a run phase

A task phase has no natural end - drivers and monitors loop forever. UVM ends it by **objection**: any component (or sequence) that has work to do calls `phase.raise_objection(this)` before starting and `phase.drop_objection(this)` when done. When the count for the phase returns to zero (after an optional drain time), the scheduler kills the run-phase processes it forked and moves on to extract. Underneath this is a counter per component that propagates up the tree, and events that the scheduler `waits` on.

```

class base_test extends uvm_test;
  `uvm_component_utils(base_test)
  fifo_env env;
  function new(string name, uvm_component parent); super.new(name, parent); endfunction
  virtual function void build_phase(uvm_phase phase);
    env = fifo_env::type_id::create("env", this);
  endfunction
  virtual task run_phase(uvm_phase phase);
    fifo_rand_seq seq = fifo_rand_seq::type_id::create("seq");
    phase.raise_objection(this, "running fifo_rand_seq"); // BEFORE any delay
    if (!seq.randomize()) `uvm_fatal("RAND", "sequence randomize failed")
    seq.start(env.agt.sqr); // returns when body() ends
    phase.drop_objection(this);
  endtask
endclass

```

Not run. The test owns the objection; drivers and monitors normally do not raise any.

PITFALL: THE TEST THAT ENDS AT TIME ZERO

If no objection is raised by the end of the first delta cycles of the run phase, UVM ends the phase immediately and the test 'passes' having done nothing. The usual cause is a raise placed after a delay or after a wait for reset. The mini model at the end of this chapter reproduces it. The opposite bug - an objection never dropped - hangs until the global timeout (+UVM_TIMEOUT or set_timeout) fires.

26.8 Sequences and sequencer arbitration

A sequence is an object (not a component) whose virtual task `body()` generates items. `seq.start(sqr)` runs `body()` in the *caller's* process; each item goes through a handshake with the driver, arbitrated by the sequencer:

sequence (body)	sequencer	driver (run_phase)
start_item(it) ---- wait_for_grant -->	arbitration queue	
	picks one sequence <--	get_next_item(req)
randomize it <---- grant -----		
finish_item(it) ---- send_request ----->	hands item over	----> req = it
		drive pins
<---- item_done -----		item_done()

Figure 26.2 - The sequence/sequencer/driver handshake. `start_item` blocks until the sequencer grants this sequence; `finish_item` blocks until the driver calls `item_done`.

Several sequences may run on one sequencer at once - typically started from a *virtual sequence* with `fork seq_a.start(sqr); seq_b.start(sqr); join`. Each one blocks in `start_item`, the sequencer keeps a queue of waiting requests, and when the driver asks for an item it grants one according to the arbitration mode set with `sqr.set_arbitration(): UVM_SEQ_ARB_FIFO` (default, request order), `UVM_SEQ_ARB_WEIGHTED`, `UVM_SEQ_ARB_RANDOM`, `UVM_SEQ_ARB_STRICT_FIFO` and `UVM_SEQ_ARB_STRICT_RANDOM` (highest priority first), or `UVM_SEQ_ARB_USER` (override `user_priority_arbitration`). A sequence can take exclusive access with `lock()` (queued) or `grab()` (immediate). The language machinery is ordinary: blocking tasks, events and `wait` for the handshake, and the `process` class so that `seq.kill()` and phase ends can terminate a sequence's thread.

```

class fifo_rand_seq extends uvm_sequence #(fifo_item);
  `uvm_object_utils(fifo_rand_seq)
  rand int unsigned n = 50;
  constraint c_n { n inside {[20:200]}; }
  function new(string name = "fifo_rand_seq"); super.new(name); endfunction
  virtual task body();
    repeat (n) begin
      fifo_item it = fifo_item::type_id::create("it");
      start_item(it); // wait for grant
      if (!it.randomize()) `uvm_fatal("RAND", "item randomize failed")
      finish_item(it); // wait for item_done
    end
  endtask
endclass

```

Not run. Randomizing between start_item and finish_item (late randomization) lets constraints see the latest state.

26.9 The register layer (RAL) in one page

UVM's register abstraction layer models a DUT's memory-mapped registers as a tree of objects: `uvm_reg_block` contains `uvm_reg` objects, which contain `uvm_reg_field` objects, placed at addresses by a `uvm_reg_map`. Tests then write `regmodel.ctrl.enable.set(1); regmodel.ctrl.update(status);` instead of bus transactions. The language features involved are all familiar:

RAL piece	What it is in SV
uvm_reg / uvm_reg_field	classes with a mirrored and a desired value, access policy strings ("RW", "RO", "W1C", ...)
generated register model	thousands of lines of classes generated from IP-XACT or a spreadsheet by a script
uvm_reg_adapter	a class with virtual reg2bus()/bus2reg() converting generic register operations to your bus item and back
uvm_reg_predictor #(T)	a parameterized component with an analysis imp: updates the mirror from observed bus traffic
frontdoor access	a sequence run on the bus agent's sequencer
backdoor access	HDL-path strings resolved through DPI-C/VPI (<code>uvm_hdl_deposit / uvm_hdl_read</code>) - Chapter 24
built-in sequences	<code>uvm_reg_hw_reset_seq</code> , <code>uvm_reg_bit_bash_seq</code> , <code>uvm_reg_access_seq</code> : generic tests driven by the model

Table 26.4 - RAL building blocks.

26.10 A compact but complete UVM environment

Here is a full UVM testbench for the FIFO of Chapter 25, using every mechanism above. It is deliberately minimal - one agent, one scoreboard, one sequence - and compiles as-is on a UVM-capable simulator together with `sync_fifo.sv` and `fifo_if.sv` from Chapter 25, the `fifo_item` class shown earlier in this chapter and the `tb_top` module above. Compare it line by line with Chapter 25: the roles are the same, the plumbing is the library's.

```

`include "uvm_macros.svh"
package fifo_uvm_pkg;
  import uvm_pkg::*;
  // fifo_item and fifo_rand_seq as shown earlier in this chapter

  class fifo_obs extends uvm_sequence_item; // what the monitor publishes
    bit is_write; bit [7:0] data;
    `uvm_object_utils(fifo_obs)
    function new(string name = "fifo_obs"); super.new(name); endfunction

```

```

endclass

class fifo_driver extends uvm_driver #(fifo_item);
  `uvm_component_utils(fifo_driver)
  virtual fifo_if vif;
  function new(string name, uvm_component parent); super.new(name, parent); endfunction
  virtual function void build_phase(uvm_phase phase);
    if (!uvm_config_db #(virtual fifo_if)::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "vif not set")
    endfunction
  virtual task run_phase(uvm_phase phase);
    vif.push <= 0; vif.pop <= 0;
    forever begin
      seq_item_port.get_next_item(req);          // req is a fifo_item (REQ)
      @(posedge vif.clk);
      vif.push <= req.push; vif.pop <= req.pop; vif.wdata <= req.data;
      seq_item_port.item_done();
    end
  endtask
endclass

class fifo_monitor extends uvm_monitor;
  `uvm_component_utils(fifo_monitor)
  virtual fifo_if vif;
  uvm_analysis_port #(fifo_obs) ap;
  function new(string name, uvm_component parent);
    super.new(name, parent); ap = new("ap", this);
  endfunction
  virtual function void build_phase(uvm_phase phase);
    if (!uvm_config_db #(virtual fifo_if)::get(this, "", "vif", vif))
      `uvm_fatal("NOVIF", "vif not set")
    endfunction
  virtual task run_phase(uvm_phase phase);
    forever begin
      @(posedge vif.clk);
      if (vif.rst_n) begin
        if (vif.pop && !vif.empty) publish(0, vif.rdata);
        if (vif.push && !vif.full) publish(1, vif.wdata);
      end
    end
  endtask
  function void publish(bit w, bit [7:0] d);
    fifo_obs o = fifo_obs::type_id::create("o");
    o.is_write = w; o.data = d;
    ap.write(o);          // broadcast to all subscribers
  endfunction
endclass

```

Not run (needs a UVM simulator). Part 1: items, driver, monitor.

```

class fifo_agent extends uvm_agent;
  `uvm_component_utils(fifo_agent)
  uvm_sequencer #(fifo_item) sqr;
  fifo_driver      drv;
  fifo_monitor     mon;
  function new(string name, uvm_component parent); super.new(name, parent); endfunction
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);          // reads is_active from config_db
    mon = fifo_monitor::type_id::create("mon", this);
    if (get_is_active() == UVM_ACTIVE) begin
      sqr = uvm_sequencer #(fifo_item)::type_id::create("sqr", this);
      drv = fifo_driver::type_id::create("drv", this);
    end
  endfunction
endclass

```



```

        end
    endfunction
    virtual function void connect_phase(uvm_phase phase);
        if (get_is_active() == UVM_ACTIVE) drv.seq_item_port.connect(sqr.seq_item_export);
    endfunction
endclass

class fifo_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(fifo_scoreboard)
    uvm_analysis_imp #(fifo_obs, fifo_scoreboard) imp;
    bit [7:0] model[$];
    function new(string name, uvm_component parent);
        super.new(name, parent); imp = new("imp", this);
    endfunction
    virtual function void write(fifo_obs o);          // called by the imp, zero time
        bit [7:0] exp;
        if (o.is_write) begin model.push_back(o.data); return; end
        if (model.size() == 0) begin `uvm_error("SCB", "read from empty FIFO") return; end
        exp = model.pop_front();
        if (o.data != exp)
            `uvm_error("SCB", $sformatf("got 0x%02h exp 0x%02h", o.data, exp))
    endfunction
    virtual function void check_phase(uvm_phase phase);
        `uvm_info("SCB", $sformatf("%0d items left in model", model.size()), UVM_LOW)
    endfunction
endclass

class fifo_env extends uvm_env;
    `uvm_component_utils(fifo_env)
    fifo_agent agt; fifo_scoreboard scb;
    function new(string name, uvm_component parent); super.new(name, parent); endfunction
    virtual function void build_phase(uvm_phase phase);
        agt = fifo_agent::type_id::create("agt", this);
        scb = fifo_scoreboard::type_id::create("scb", this);
    endfunction
    virtual function void connect_phase(uvm_phase phase);
        agt.mon.ap.connect(scb.imp);
    endfunction
endclass
// base_test and fill_test as shown earlier in this chapter
endpackage
// run: +UVM_TESTNAME=base_test or +UVM_TESTNAME=fill_test

```

Not run (needs a UVM simulator). Part 2: agent, scoreboard, environment.

Notice what is *not* there compared to Chapter 25: no mailboxes (the sequencer handshake and the analysis port replace them), no `fork` (the phase scheduler forks the run phases), no `disable fork` or end-of-test wait (objections), no hand-written test `case` (`run_test()` creates the class named by `+UVM_TESTNAME` through the factory), and no constructor argument lists for the virtual interface (`config_db`).

26.11 Demystified: a mini UVM in plain SystemVerilog

The best way to believe that the factory, `config_db` and phasing are just language features is to build small versions of them. The package below implements a string-keyed factory with type overrides, a type-safe `config_db`, a global objection counter and a phase scheduler (build top-down, run in parallel until objections drop, report bottom-up), plus a `comp_utils` macro that plays the role of `uvm_component_utils`. It compiles and runs on Verilator 5.020.

```

// mini_uvm.sv - a 120-line model of the UVM machinery, in plain SystemVerilog
package mini_uvm;
    typedef class component;

```



```

// ---- factory: a static registry of "proxy" objects, keyed by type name ----
virtual class proxy_base;                                // like uvm_object_wrapper
    pure virtual function component create_comp(string name, component parent);
    pure virtual function string    type_name();
endclass

class factory;
    static proxy_base registry [string];                // type name -> proxy
    static string    overrides[string];                // type name -> replacement name
    static function void register(proxy_base p);
        registry[p.type_name()] = p;
    endfunction
    static function void set_type_override(string orig, string repl);
        overrides[orig] = repl;
    endfunction
    static function component create(string tname, string name, component parent);
        while (overrides.exists(tname)) tname = overrides[tname]; // follow overrides
        if (!registry.exists(tname)) $fatal(1, "factory: type '%s' not registered", tname);
        return registry[tname].create_comp(name, parent);
    endfunction
endclass

// one specialization of this class exists per registered type T
class registry #(type T = component, string TNAME = "") extends proxy_base;
    typedef registry #(T, TNAME) this_type;
    local static this_type me = get();                // static init registers the type
    static function this_type get();
        if (me == null) begin me = new(); factory::register(me); end
        return me;
    endfunction
    virtual function component create_comp(string name, component parent);
        T obj = new(name, parent);
        return obj;
    endfunction
    virtual function string type_name(); return TNAME; endfunction
    static function T create(string name, component parent); // T::type_id::create
        component c = factory::create(TNAME, name, parent);
        if (!$cast(create, c)) $fatal(1, "factory: override of %s is not a subtype", TNAME);
    endfunction
endclass

```

mini_uvm.sv (part 1) - the factory: proxies, a static registry and overrides.

```

// ---- config_db: parameterized class, static associative array per type ----
class config_db #(type T = int);
    static T db[string];
    static function void set(string path, string field, T value);
        db[{path, ".", field}] = value;
    endfunction
    static function bit get(string path, string field, output T value);
        if (db.exists({path, ".", field})) begin value = db[{path, ".", field}]; return 1; end
        if (db.exists({"*", field})) begin value = db[{"*", field}]; return 1; end
        return 0;
    endfunction
endclass

// ---- objections: a global counter (UVM keeps one per phase and component) ----
class objection;
    static int count;
    static function void raise(); count++; endfunction
    static function void drop(); count--; endfunction
endclass

```

```

endclass

// ---- component tree and phases (virtual methods called by the "kernel") ----
class component;
    string name; component parent; component children[$];
    function new(string name, component parent);
        this.name = name; this.parent = parent;
        if (parent != null) parent.children.push_back(this);
    endfunction
    function string full_name();
        return (parent == null) ? name : {parent.full_name(), ".", name};
    endfunction
    virtual function string get_type_name(); return "component"; endfunction
    virtual function void build_phase(); endfunction
    virtual function void connect_phase(); endfunction
    virtual task run_phase(); endtask
    virtual function void report_phase(); endfunction
endclass

class kernel; // the phase scheduler
    static function void do_build(component c); // top-down
        c.build_phase();
        foreach (c.children[i]) do_build(c.children[i]);
    endfunction
    static function void do_report(component c); // bottom-up
        foreach (c.children[i]) do_report(c.children[i]);
        c.report_phase();
    endfunction
    static function void collect(component c, ref component all[$]);
        all.push_back(c);
        foreach (c.children[i]) collect(c.children[i], all);
    endfunction
endclass

task automatic run_test(string test_name);
    component top = factory::create(test_name, "test", null);
    component all[$];
    kernel::do_build(top);
    kernel::collect(top, all);
    foreach (all[i]) all[i].connect_phase();
    foreach (all[i])
        fork
            automatic int k = i; // capture the loop index per thread
            all[k].run_phase();
        join_none
    #0; // let run_phases raise objections
    wait (objection::count == 0); // run phase ends when all objections drop
    disable fork;
    kernel::do_report(top);
endtask
endpackage

`define comp_utils(T) \
    typedef mini_uvm::registry #(T, `T`) type_id; \
    virtual function string get_type_name(); return `T`; endfunction

```

mini_uvm.sv (part 2) - config_db, objections, components, the phase kernel and the registration macro.

A few language points deserve comment. `typedef class component;` is a forward declaration, needed because the factory refers to `component` before it is defined. `proxy_base` is a `virtual class` with `pure virtual` methods - an abstract base that the parameterized `registry` implements once per registered type. The line `local static this_type me = get();` is the heart of self-registration: each specialization of

`registry` has its own static `me`, and its initializer runs during static initialization, before time 0. The `$cast(create, c)` assigns to the function's implicit return variable. The kernel's recursive functions are static class methods (Verilator 5.020 hit an internal error on the same code written as recursive package functions - another portability note). The macro stringifies its argument (a grave accent followed by a double quote around `T` in the listing) so that `comp_utils(driver)` produces the string "driver".

```
// demo.sv - a "UVM-like" test built on mini_uvm
package demo_pkg;
  import mini_uvm::*;

  class driver extends component;
    `comp_utils(driver)
    int n = 2;
    function new(string name, component parent); super.new(name, parent); endfunction
    virtual function void build_phase();
      void'(config_db#(int)::get(full_name(), "n_items", n));
    endfunction
    virtual task run_phase();
      objection::raise();
      for (int i = 0; i < n; i++) begin
        #10 $display("[%0t] %s (%s) drives item %0d", $time, full_name(), get_type_name(), i);
      end
      objection::drop();
    endtask
  endclass

  class slow_driver extends driver;                                // a replacement for driver
    `comp_utils(slow_driver)
    function new(string name, component parent); super.new(name, parent); endfunction
    virtual task run_phase();
      objection::raise();                                          // raise BEFORE consuming time
      #25 super.run_phase();
      objection::drop();
    endtask
  endclass

  class env extends component;
    `comp_utils(env)
    driver drv;
    function new(string name, component parent); super.new(name, parent); endfunction
    virtual function void build_phase();
      drv = driver::type_id::create("drv", this);    // never "new" directly
    endfunction
    virtual function void report_phase();
      $display("[%0t] %s: report, drv is a %s", $time, full_name(), drv.get_type_name());
    endfunction
  endclass

  class base_test extends component;
    `comp_utils(base_test)
    env e;
    function new(string name, component parent); super.new(name, parent); endfunction
    virtual function void build_phase();
      config_db#(int)::set("test.e.drv", "n_items", 3);
      e = env::type_id::create("e", this);
    endfunction
  endclass

  class slow_test extends base_test;
    `comp_utils(slow_test)
    function new(string name, component parent); super.new(name, parent); endfunction
```

```

    virtual function void build_phase();
        factory::set_type_override("driver", "slow_driver"); // before env is built
        super.build_phase();
    endfunction
endclass
endpackage

module tb;
    import mini_uvm::*;
    import demo_pkg::*;
    initial begin
        string t;
        if (!$value$plusargs("TEST=%s", t)) t = "base_test";
        $display("registered types: %0d", factory::registry.num());
        run_test(t); // like UVM's run_test()
    end
endmodule

```

demo.sv - components, a config_db setting, and a test that overrides the driver type through the factory.

```

verilator --binary --timing -Wno-fatal -Wno-WIDTHTRUNC --top-module tb \
    mini_uvm.sv demo.sv -o Vtb
./obj_dir/Vtb # base_test
./obj_dir/Vtb +TEST=slow_test

```

Build and run with Verilator 5.020.

```

registered types: 5
[10] test.e.drv (driver) drives item 0
[20] test.e.drv (driver) drives item 1
[30] test.e.drv (driver) drives item 2
[30] test.e: report, drv is a driver

```

Verilator 5.020: base_test. Five types were registered before time 0; config_db set n_items=3 for test.e.drv.

```

registered types: 5
[35] test.e.drv (slow_driver) drives item 0
[45] test.e.drv (slow_driver) drives item 1
[55] test.e.drv (slow_driver) drives item 2
[55] test.e: report, drv is a slow_driver

```

Verilator 5.020: +TEST=slow_test. env still calls driver::type_id::create, but gets a slow_driver.

Everything UVM promises is visible here in miniature: the test was selected by a string at run time; `env` was not edited, yet it built a different driver; the driver read its configuration from a database populated by an ancestor; and the run phase ended exactly when the last objection dropped (at 55, not at a fixed time). A misspelled test name reaches the `$fatal` in the factory:

```

registered types: 5
[0] %Fatal: mini_uvm.sv:22: Assertion failed in TOP.mini_uvm.factory.create:
    factory: type 'nope' not registered

```

Verilator 5.020: ./obj_dir/Vtb +TEST=nope (first line of the fatal message, wrapped at the indent).

Finally, the objection pitfall. Remove the two objection calls from `slow_driver::run_phase` so that it relies on the base class's raise, which now happens only after the `#25`:

```

virtual task run_phase();
    #25 super.run_phase(); // base raises its objection only at t=25
endtask

```

demo_late.sv - slow_driver without its own objection.

```

registered types: 5
[0] test.e: report, drv is a slow_driver

```

Verilator 5.020: +TEST=slow_test with demo_late.sv. The run phase ended at time 0; nothing was driven, and nothing reported an error.

WHAT THE REAL LIBRARY ADDS

Compared with this model, UVM adds instance-specific overrides (path patterns), regular-expression scopes and precedence in the config database, per-phase objection objects with hierarchy propagation, drain times and timeouts, phase jumping, domains for independently phased sub-systems, the report server with verbosity and severity actions, and the sequencer. None of it needs a language feature not used here.

26.12 Summary

- UVM is a SystemVerilog class library (IEEE 1800.2); every mechanism maps onto classes, virtual methods, parameterized classes, static members, associative arrays, macros and processes.
- `uvm_object` is for transient data (items, sequences, configs); `uvm_component` is for the permanent, phased hierarchy.
- The factory is a static registry of singleton proxy objects, one per registered type, created by static initialization; `type_id::create()` looks up the proxy, applies overrides and \$casts the result. Calling `new()` bypasses it.
- The utility macros only generate the `type_id` typedef and `get_type/get_type_name` methods (and, for field macros, copy/compare/print code).
- `uvm_config_db #(T)` is a parameterized class with static storage; each T is a separate, type-safe database. Virtual interfaces are passed from the top module this way.
- Phases are virtual methods called by a scheduler; `run_phase` tasks run in parallel and end when all objections drop - raise before any delay.
- Sequences run `body()` in the caller's process; the sequencer arbitrates between them; analysis ports broadcast by calling `write()` on each subscriber; RAL is a generated class model with adapters and DPI-based backdoor access.
- A 120-line mini UVM ran on Verilator and reproduced factory overrides, `config_db` and the objection-at-time-zero bug.

26.13 Exercises

- 1 Extend the mini factory with instance overrides: `set_inst_override("test.e.drv", "driver", "slow_driver")`. What must `create()` know that it does not know now, and how does UVM supply it?
- 2 Add a `connect_phase` to the mini model that connects a tiny analysis port (a class holding a queue of subscriber handles with a `write()` function) from a monitor to two subscribers. Why must `write()` be a function rather than a task?
- 3 Write the macro expansion of `uvm_object_utils(fifo_item)` by hand, including `create()`. Explain why a parameterized class must use the `_param_` variant.
- 4 Given `uvm_config_db#(int)::set(this, "env.*", "n", 5)` in the test and `uvm_config_db#(int)::set(this, "agt", "n", 9)` in the env, which value does the agent get in `build_phase`, and why?
- 5 Add a second sequence that writes only (push) and run it in parallel with `fifo_rand_seq` from a virtual sequence. Predict the interleaving under `UVM_SEQ_ARB_FIFO` and under `UVM_SEQ_ARB_STRICT_FIFO` with priorities 100 and 200.
- 6 Run the full UVM environment of this chapter on EDA Playground with a planted bug and confirm that `uvm_error` messages make the test fail. Then remove the objection from `base_test` and observe the result.

27. Language Pitfalls, Portability and Lint

Most expensive bugs in Verilog and SystemVerilog are not in exotic features. They come from a handful of language rules that are perfectly well defined but not what the author expected: how wide an expression is, whether it is signed, what an X does in an `if`, which process runs first in a time step, and whether a variable is static. The same rules also cause code to behave differently on different tools, either because the LRM leaves something unspecified or because a tool deviates from it. This chapter is a reference catalogue. Each pitfall is stated as a rule, shown with a bad and a good form, and - wherever a tool here could run it - demonstrated with real output from Icarus Verilog 12 and Verilator 5.020. It ends with the lint tools and rule categories that catch these mistakes automatically, run on a deliberately buggy file.

Category	Typical damage	Caught by
Expression sizing and signedness	Wrong arithmetic in RTL and models, silently	Lint (width rules), careful review, simulation
4-state / X semantics	Bugs hidden in RTL simulation, found at gate level or in silicon	X-propagation tools, assertions, gate-level sim
RTL coding style	Simulation/synthesis mismatch, latches, races	Lint, synthesis reports, equivalence checking
Scheduling and races	Results that depend on the simulator or on file order	Lint (BLKSEQ etc.), code review, race-free methodology
Testbench (lifetimes, processes)	Testbench checks the wrong thing or nothing	Code review, mutation testing, multiple simulators
Portability	Code that works on one tool only	Running more than one tool in CI

Table 27.1 - The pitfall families of this chapter.

27.1 Expression width and signedness

Recall the rules of Chapter 4. An expression is sized in two passes: the **context** width is the maximum of the operand widths *and the assignment target*; context-determined operands (arithmetic, bitwise, `?:` branches) are extended to that width *before* the operation; self-determined operands (concatenation members, shift counts, reduction and relational operands' results, casts' results) keep their own width. Signedness is decided separately: an expression is signed only if *all* its context-determined operands are signed; a single unsigned operand makes the whole expression unsigned, and part-selects and concatenations are always unsigned. Extension happens after these decisions, by sign or by zero.

```
// pit_expr.sv - expression pitfalls (run on Icarus 12 and Verilator 5.020)
module pit_expr;
    logic      [7:0] a = 8'd200, b = 8'd100, avg8;
    logic      [8:0] avg9, cat9;
    logic signed [7:0] x = -8'sd4;
    logic      [7:0] u = 8'd2;
    logic signed [15:0] y1, y2;
    initial begin
        // 1. width: the LHS is part of the context
        avg8 = (a + b) >> 1;           // context = 8 bits: carry lost before the shift
        avg9 = (a + b) >> 1;           // context = 9 bits: carry kept
        cat9 = {a + b};                // concatenation operand is self-determined: 8 bits
        $display("1: avg8=%0d avg9=%0d cat9=%0d", avg8, avg9, cat9);
        // 2. one unsigned operand makes the whole expression unsigned
        $display("2: x<u is %b   x<signed'(u) is %b", x < u, x < signed'(u));
        y1 = x;                        // signed source: sign-extended
        y2 = x + u;                    // unsigned expression: zero-extended
        $display("2: y1=%0d y2=%0d", y1, y2);
        // 3. integer literal arithmetic
```

```

    $display("3: -8'd4 / 2 = %0d  -4 / 2 = %0d  8'hFF + 1 = %0d", -8'd4 / 2, -4 / 2,
            8'hFF + 1);
    $finish;
end
endmodule

```

pit_expr.sv

```

1: avg8=22 avg9=150 cat9=44
2: x<u is 0  x<signed'(u) is 1
2: y1=-4 y2=254
3: -8'd4 / 2 = 2147483646  -4 / 2 = -2  8'hFF + 1 = 256

```

Icarus 12 and Verilator 5.020 print identical results (the LRM is unambiguous here).

- **avg8 = 22:** `a + b` is evaluated at 8 bits because both operands and the target are 8 bits; 300 wraps to 44 before the shift. With a 9-bit target the same text gives 150. Moving an expression into a narrower variable can therefore change the value of its *intermediate* results, not just truncate the final one.
- **cat9 = 44:** the operand of a concatenation is self-determined, so `{a + b}` is 8 bits whatever the target. Wrapping an expression in braces 'to be safe' is a bug.
- **x < u is 0:** `x` is signed, `u` is unsigned, so the comparison is unsigned and -4 becomes 252. The fix `signed'(u)` works here but reinterprets values of `u >= 128` as negative; the robust form is `x < signed'({1'b0, u})`, which is 9 bits and signed.
- **y2 = 254:** `x + u` is unsigned, so the 8-bit result 0xFE is zero-extended to 16 bits. `y1 = x` alone is sign-extended to -4.
- **-8'd4 / 2 = 2147483646:** a sized literal written with a minus sign is still an *unsigned* value (the minus is an operator applied to 8'd4). Combined with the signed 32-bit literal 2 the expression is unsigned 32-bit, so the operand is 4294967292. Use `-8'sd4` or plain integers.
- **8'hFF + 1 = 256:** the unsized literal 1 is 32 bits, so the addition is 32 bits wide. Width surprises go both ways.

```

// pit_misc.sv - a few more silent surprises (Icarus 12 and Verilator 5.020)
module pit_misc;
    logic signed [7:0] s = -8'sd7;
    logic          [7:0] u = 8'hF0, mem [4];
    int            r;
    function int counter();          // module functions are STATIC by default
        int c = 0;                  // initialised once, at time 0 - not per call
        c++;
        return c;
    endfunction
    initial begin
        $display("-7/2=%0d  -7>>>1=%0d  8'hF0>>>2=%h  s[7:0]>>>1=%h", s / 2, s >>> 1, u >>> 2,
            s[7:0] >>> 1);
        r = int'(2.5); $display("int'(2.5)=%0d  int'(-2.5)=%0d  $rtoi(2.5)=%0d", r, int'(-2.5),
            $rtoi(2.5));
        mem[0] = 8'h11;
        $display("mem[7]=%h (index out of range reads the default value)", mem[7]);
        $display("counter() x3: %0d %0d %0d", counter(), counter(), counter());
        $finish;
    end
endmodule

```

pit_misc.sv

```

pit_misc.sv:7: warning: Static variable initialization requires explicit lifetime
in this context.
pit_misc.sv:17: warning: returning 'bx for out of bounds array access mem[7].
-7/2=-3  -7>>>1=-4  8'hF0>>>2=3c  s[7:0]>>>1=7c
int'(2.5)=3  int'(-2.5)=-3  $rtoi(2.5)=2
mem[7]=xx (index out of range reads the default value)
counter() x3: 1 2 3

```

Icarus 12 (iverilog -g2012; the first warning line is wrapped here).

```
%Warning-IMPLICITSTATIC: pit_misc.sv:6:16: Function/task's lifetime implicitly set to static
%Warning-SELRANGE: pit_misc.sv:17:75: Selection index out of range: 7 outside 3:0
-7/2=-3  -7>>>1=-4  8'hF0>>>2=3c  s[7:0]>>>1=7c
int'(2.5)=3  int'(-2.5)=-3  $rtoi(2.5)=2
mem[7]=00 (index out of range reads the default value)
counter() x3: 3 2 1
```

Verilator 5.020 (--binary). Same arithmetic; different out-of-range value and a different order of evaluating the three calls.

- Signed division truncates toward zero ($-7/2 = -3$) but an arithmetic shift rounds toward minus infinity ($-7>>>1 = -4$). Replacing a divide by a shift in signed RTL changes the answer for negative odd values.
- $>>>$ is arithmetic only on a **signed** operand. **u** is unsigned, so $8'hF0>>>2 = 8'h3C$, and a part-select of a signed variable (**s[7:0]**) is unsigned too, so it also shifts in zeros.
- Casting real to integer **rounds** (ties away from zero): $\text{int}'(2.5) = 3$. **\$rtoi** truncates. Both appear in models of DSP blocks; mixing them up gives 1-LSB mismatches.
- An out-of-range read returns the default value of the element type: X for 4-state **logic** (Icarus) - but Verilator is a 2-state simulator and returns 0, so the same bug is visible on one tool and invisible on the other. Out-of-range writes are ignored.
- Functions and tasks declared in a module are **static** unless declared **automatic**: **int c = 0**; is initialised once, and the LRM requires an explicit **static** keyword on such an initialised declaration precisely because the behaviour surprises people. Both tools warn. And the order in which the three arguments of the **\$display** were evaluated differs between tools - never write code whose result depends on it.

27.2 X, 4-state and 2-state pitfalls

```
// pit_x.sv - X-optimism, 2-state types, == vs ===, casex (Icarus 12)
module pit_x;
    logic      sel = 1'bx;
    logic [3:0] p = 4'b10x0, q = 4'b10x0;
    logic      y_if, y_case;
    logic [1:0] y_tern;
    bit        b2;
    int        i2;
    logic [1:0] op;
    initial begin
        if (sel) y_if = 1'b1; else y_if = 1'b0;          // X takes the else branch
        case (sel) 1'b1: y_case = 1'b1; default: y_case = 1'b0; endcase
        y_tern = sel ? 2'b10 : 2'b11;                  // ?: merges: differing bits -> x
        $display("x-opt:  if->%b case->%b ?:->%b", y_if, y_case, y_tern);
        b2 = sel; i2 = p;                               // 2-state targets turn X into 0
        $display("2state: bit<-x = %b  int<-4'b10x0 = %0d", b2, i2);
        $display("eq:      p==q -> %b  p===q -> %b  p!=q -> %b", p == q, p === q, p != q);
        if (p == q) $display("eq:      if (p==q) taken"); else $display("eq:      else taken");
        op = 2'bx1;                                     // e.g. an uninitialised opcode bit
        casex (op)
            2'b0x: $display("casex:  op=%b matched 2'b0x (READ)", op);
            2'b11: $display("casex:  op=%b matched 2'b11 (WRITE)", op);
            default: $display("casex:  default");
        endcase
        casez (op)
            2'b0?: $display("casez:  op=%b matched 2'b0?", op);
            2'b11: $display("casez:  op=%b matched 2'b11", op);
            default: $display("casez:  op=%b -> default (X not treated as don't-care)", op);
        endcase
    end
endmodule
```

pit_x.sv


```

x-opt:  if->0  case->0  ?!->1x
2state: bit<-x = 0    int<-4'b10x0 = 8
eq:      p==q -> x    p===q -> 1    p!=q -> x
eq:      else taken
casex:   op=x1 matched 2'b0x (READ)
casez:   op=x1 -> default (X not treated as don't-care)

```

Icarus 12 (4-state).

- **X-optimism of if/case:** an **if** whose condition is X (or Z) takes the else branch; a **case** whose selector is X matches no ordinary item and takes the default. The RTL simulation shows a clean 0 where real hardware might produce 0 or 1. The bug surfaces only in gate-level simulation or silicon. Guard critical selects with an assertion such as **assert (!\$isunknown(sel))** and consider your simulator's X-propagation mode (VCS -xprop, Xcelium -xprop).
- **?: is X-pessimistic by comparison:** with an X condition it merges both branches bit by bit, giving **1x** - closer to hardware.
- **2-state types hide X:** assigning to **bit**, **int** or **byte** converts X and Z to 0. Declaring DUT outputs in a testbench as **bit** hides exactly the X you need to see. Rule: **logic** for everything that touches the DUT; 2-state types for loop counters and pure testbench data.
- **== versus ===:** logical equality with any X/Z bit yields X, and **if (X)** is false - so **if (p == q)** takes the else branch even though the values are bit-for-bit identical. In checkers use **===** and **!==**; in RTL use **==** (case equality has no hardware meaning and synthesis tools either reject it or treat it as **==**).
- **casex hazards:** in **casex**, X and Z in the *selector* are don't-cares too, so an unknown opcode silently matches the first item (here READ). **casez** treats only Z/? as don't-care. Better still in SystemVerilog: **case (op) inside 2'b0?: ... endcase**, where only the item side has wildcards.

PITFALL (PORTABILITY): VERILATOR IS A 2-STATE SIMULATOR

Verilator maps X and Z to 0 or 1 (chosen by --x-assign and --x-initial: 0, 1, fast or unique/random). None of the X examples above behave the same way on it, and a testbench that relies on detecting X with **===** or **\$isunknown** cannot find X bugs there. Run X-sensitive tests on a 4-state simulator, and on Verilator run regressions with --x-initial unique and several seeds so that uninitialised state is randomised rather than conveniently 0.

27.3 Races and scheduling

The event scheduler (Chapter 5) executes all processes woken at the same time step in the Active region in an **unspecified** order. Any design or testbench whose result depends on that order has a race, and different simulators - or the same simulator after a harmless edit - can resolve it differently.

```

// race.sv - two classic races: order of initial blocks, blocking between always blocks
module race;
  logic clk = 0, d = 1, q1 = 0, q2 = 0;
  int a;
  initial $display("initial race: a=%0d", a);          // runs before or after the next line?
  initial a = 5;

  `ifdef SWAP
    always @(posedge clk) q1 = d;                      // stage 1 (blocking: BAD)
    always @(posedge clk) q2 = q1;                    // stage 2 (blocking: BAD)
  `else
    always @(posedge clk) q2 = q1;                    // stage 2 (blocking: BAD)
    always @(posedge clk) q1 = d;                    // stage 1 (blocking: BAD)
  `endif

  initial begin
    #1 clk = 1;
    #1 $display("after one edge: q1=%b q2=%b (a 2-stage pipe should give q2=0)", q1, q2);
    $finish;
  end
end

```

endmodule

race.sv - compiled with and without +define SWAP.

Run	initial race	q2 after one edge
Icarus 12, default order	a=0	q2=1 (wrong)
Icarus 12, -DSWAP	a=0	q2=0
Verilator 5.020, default order	a=5	q2=1 (wrong)
Verilator 5.020, -DSWAP	a=5	q2=1 (wrong)
Icarus 12, both stages changed to <=	a=0	q2=0 (correct)

Table 27.2 - Real results of race.sv. Source order and tool change the answer; nonblocking assignments remove the race.

The first race is between two `initial` blocks at time 0: whether the display sees 0 or 5 is unspecified, and the two tools disagree. The second is the classic reason for the nonblocking rule: with blocking assignments in two clocked processes, whether stage 2 reads the old or new value of `q1` depends on which process runs first. Swapping the source order flipped the Icarus result but not Verilator's (Verilator orders the whole design statically). With `<=`, all right-hand sides are sampled before any flop updates, and every simulator gives the same, hardware-accurate answer.

```
// fork_static.sv - loop variable capture and static tasks
module f;
  task automatic show(string tag, int v); $display("%s %0d", tag, v); endtask
  task static tick(input int id);
    int v;
    v = id;
    #1 $display("static task: called with %0d, sees v=%0d", id, v);
  endtask
  initial begin
    for (int i = 0; i < 3; i++) fork show("fork (bad):", i); join_none
    #1;
    fork tick(1); tick(2); join
    $finish;
  end
endmodule
```

fork_static.sv

```
fork (bad): 3
fork (bad): 3
fork (bad): 3
static task: called with 1, sees v=1
static task: called with 1, sees v=1
```

Icarus 12 - the LRM behaviour.

```
fork (bad): 0
fork (bad): 1
fork (bad): 2
static task: called with 1, sees v=1
static task: called with 2, sees v=2
```

Verilator 5.020 - which does not follow the LRM here.

By the LRM, the three processes created by `fork ... join_none` inside the loop do not start until the parent blocks at `#1`, by which time the single loop variable `i` is 3, so every thread prints 3 - Icarus does exactly this, as do the commercial simulators. The fix is to give each thread its own copy with a declaration in the fork's declarative region, which is executed at fork time: `fork automatic int k = i; show("...", k); join_none`. Verilator 5.020 happened to print 0, 1, 2 for the *buggy* form, so a testbench developed only on Verilator can carry this bug unseen until it moves to another simulator.

The static task has one copy of its arguments and locals shared by all callers. Both concurrent calls write `id` and `v`; whichever writes last wins and both threads report it. Icarus shows the sharing (the value 2 is lost

completely); Verilator 5.020 gave each call private storage, again differing from the LRM. Rule: declare every task and function in a module or interface `automatic`, or put `module automatic` on the module. Class methods are automatic already.

The timescale directive and file order

The timescale directive (Chapter 8) is a compiler directive, not a module property: it applies to every module that follows it in the compilation, across file boundaries, until the next one. A module without its own directive therefore inherits whatever came last - which depends on the order of files on the command line.

```
// ts_a.v
`timescale 1ns/1ps
module tb;
    initial begin #10 $display("tb: t=%0t", $realtime); end
    sub u();
endmodule

// ts_b.v
module sub;                                // no `timescale: inherits whatever came last
    initial begin #10 $display("sub: t=%0t", $realtime); end
endmodule
```

ts_a.v and ts_b.v - two files, one directive.

```
$ iverilog -Wall -o ts1.vvp ts_a.v ts_b.v && vvp -n ts1.vvp
ts_b.v:1: warning: timescale for sub inherited from another file.
ts_a.v:1: ...: The inherited timescale is here.
sub: t=10000
tb: t=10000
$ iverilog -Wall -o ts2.vvp ts_b.v ts_a.v && vvp -n ts2.vvp
warning: Some modules have no timescale. This may cause
       : confusing timing results. Affected modules are:
       : -- module sub declared here: ts_b.v:1
warning: Found both default and `timescale based delays. Use
       : -Wtimescale to find the module(s) with no `timescale.
tb: t=10000
sub: t=1000000000000000
```

Icarus 12 (a tab in the output is shown as two spaces). Same source, different file order: sub's #10 is 10 ns in the first run and 10 s (the default unit) in the second. Times print in the 1 ps global precision.

Verilator refuses to guess: it reports `TIMESCALEMOD` ('Timescale missing on this module as other modules have it'). The robust fix is to declare `timeunit 1ns; timeprecision 1ps;` inside every module, interface and package, which binds the units to the design element itself, or to set a project-wide default with a tool option (for example `-timescale=1ns/1ps` in VCS, `--timescale` in Verilator).

27.4 Testbench-language pitfalls

```
// pit_tb.sv - testbench-language pitfalls (Verilator 5.020 --binary)
module pit_tb;
    typedef enum logic [1:0] {IDLE, RUN, DONE} state_e;
    byte arr[$] = '{100, 100, 100, 100};
    state_e st;
    int n, total;
    initial begin
        // 1. array reduction methods return the ELEMENT type
        foreach (arr[i]) total += arr[i]; // portable: accumulate in an int
        $display("sum()=%0d sum() with (int'(item))=%0d foreach into int=%0d",
            arr.sum(), arr.sum() with (int'(item)), total);
        // 2. enum from an integer
        if (!$cast(st, 3)) $display("$cast(st, 3) failed: 3 is not a state_e value");
        st = state_e'(3); // static cast: no check at all
```

```

$display("state_e'(3): value %0d, name '%s'", st, st.name());
// 3. a 3-bit loop counter never reaches 8
n = 0;
for (bit [2:0] j = 0; j < 8; j++) begin
    n++;
    if (n == 20) break;                // safety guard
end
$display("bit [2:0] j < 8 loop: %0d iterations before the guard fired", n);
// 4. $random is signed; $urandom is unsigned
begin
    int neg = 0;
    repeat (100) if ($random % 10 < 0) neg++;
    $display("$random %% 10 < 0 in %0d of 100 calls; $urandom %% 10 never", neg);
end
// 5. string literals in ?: without a string context are just integers
$display(n > 0 ? "ok" : "fail");
$display("%s", n > 0 ? "ok" : "fail");
$finish;
end
endmodule

```

pit_tb.sv - Icarus 12 does not support array reduction methods or break, so this one ran on Verilator only.

```

sum()=-112 sum() with (int'(item))=144 foreach into int=400
$cast(st, 3) failed: 3 is not a state_e value
state_e'(3): value 3, name ''
bit [2:0] j < 8 loop: 20 iterations before the guard fired
$random % 10 < 0 in 44 of 100 calls; $urandom % 10 never
28523
ok

```

Verilator 5.020.

- **sum() width:** array reduction methods return the element type, so four bytes of 100 sum to -112 (400 wrapped to signed 8 bits). The LRM fix is `sum() with (int'(item))`, whose result type is that of the `with` expression, so it should give 400; Verilator 5.020 still returned an 8-bit result (144 = 400 mod 256) - a tool bug. The only fully portable form is an explicit `foreach` into an `int`.
- **enum from int:** `st = 3;` is a compile error (an enum variable accepts only its own type). `$cast(st, 3)` checks the value and fails; the static cast `state_e'(3)` does not check and leaves an illegal value whose `name()` is the empty string. Use `$cast` in testbenches when the integer comes from outside (a file, a DPI call, a register read).
- **loop counters:** `j < 8` is always true for a 3-bit unsigned variable (7 + 1 wraps to 0): an infinite loop. Loop variables are `int` unless there is a reason.
- **\$random vs \$urandom:** `$random` returns a signed 32-bit value, so `$random % 10` is negative almost half the time (44 of 100 here), which indexes arrays out of range. `$urandom` and `$urandom_range` are unsigned and thread-stable (each process and object has its own generator), which is what makes a regression seed reproducible after unrelated code changes.
- **string literals are integers:** a string literal is a packed array of 8-bit characters. In `$display(cond ? "ok" : "fail")` there is no format string, so the `?:` result is printed as a number ("ok" = 16'h6F6B = 28523). With a `%s` format it prints text. The same happens when a string literal is assigned to a `logic` vector: it is right-justified and truncated on the left.

27.5 The catalogue

Table 27.3 collects the pitfalls of this chapter and of the whole book in one place, each with the bad form and the good form. Use it as a code-review checklist.

#	Pitfall	Bad	Good
1	Width of intermediate results set by the target	<code>avg8 = (a+b)>>1</code>	<code>s9 = a+b; avg8 = s9[8:1]</code>

#	Pitfall	Bad	Good
2	Concatenation operand is self-determined	<code>s9 = {a+b}</code>	<code>s9 = a + b</code> or <code>{1'b0,a} + b</code>
3	Mixed signed/unsigned becomes unsigned	<code>x < u</code>	<code>x < signed'({1'b0,u})</code>
4	Negative sized literal is unsigned	<code>-8'd4 / 2</code>	<code>-8'sd4 / 2</code>
5	Part-select of a signed value is unsigned	<code>s[7:0] >>> 1</code>	<code>signed'(s[7:0]) >>> 1</code>
6	<code>>>></code> on unsigned is logical	<code>u >>> 2</code> expecting sign fill	declare <code>logic signed</code> or cast
7	Divide vs arithmetic shift rounding	<code>x >>> 1</code> as <code>x / 2</code>	use <code>/</code> or add bias for negatives
8	Real-to-int rounds, \$rtoi truncates	<code>int'(r)</code> expecting floor	<code>\$rtoi(r)</code> or <code>\$floor</code>
9	Loop counter too narrow	<code>for (bit [2:0] j=0; j<8; j++)</code>	<code>for (int j=0; j<8; j++)</code>
10	32-bit overflow in products	<code>int p = a32 * b32</code>	<code>longint p = 64'(a32) * b32</code>
11	X-optimism of if/case	<code>if (sel)</code> with X sel	<code>assert (!\$isunknown(sel)), xprop sim</code>
12	2-state types hide X	<code>bit [7:0] dout_tb</code>	<code>logic [7:0] dout_tb</code>
13	<code>==</code> with X yields X	<code>if (dout != exp) err++</code>	<code>if (dout !== exp) err++</code>
14	casex matches X in the selector	<code>casex (op)</code>	<code>case (op)</code> inside or <code>casez</code>
15	Blocking in clocked processes	<code>always_ff ... q2 = q1;</code>	<code>q2 <= q1;</code>
16	NBA for temporaries in combinational code	<code>t <= a & b; y <= t c;</code> in <code>always_comb</code>	<code>t = a & b; y = t c;</code>
17	Missing default/else -> latch	<code>case</code> without default in <code>always_comb</code>	default assignments first, or <code>default:</code>
18	Incomplete sensitivity list	<code>always @(a) y = a & b;</code>	<code>always_comb y = a & b;</code>
19	Implicit nets from typos	undeclared <code>sum_ab</code> becomes 1-bit wire	<code>default_nettype none</code> directive at the top of every file
20	Multiple drivers	two <code>assign y = ...</code>	one driver; <code>always_comb</code> / <code>always_ff</code> enforce it
21	timescale directive depends on file order	timescale directive in some files only	<code>timeunit/timeprecision</code> in every module
22	Race between initial blocks	<code>initial a = 5; initial use(a);</code>	order with events, NBA, or one initial
23	Race between always blocks (blocking)	<code>stage-to-stage =</code>	<code><=</code> for every flop
24	Loop variable captured by fork	<code>fork show(i); join_none</code> in loop	<code>fork automatic int k = i; show(k); join_none</code>
25	Static task/function variables	<code>task t();</code> in a module	<code>task automatic t();</code>
26	Initialiser in a static function runs once	<code>int c = 0;</code> in static function	<code>automatic</code> function, or assign in the body
27	Packed vs unpacked assignment	<code>p32 = u8x4;</code> (unpacked)	<code>p32 = {>>{u8x4}};</code> streaming
28	enum assigned from int	<code>st = 3;</code> or <code>state_e'(v)</code> unchecked	<code>if (!\$cast(st, v)) error</code>
29	sum()/product() in element width	<code>byte_q.sum()</code>	<code>foreach into int, or sum() with (int'(item))</code>
30	\$random is signed, unstable	<code>idx = \$random % N</code>	<code>idx = \$urandom_range(N-1)</code>

#	Pitfall	Bad	Good
31	String literal in ?: printed as number	<code>\$display(c ? "ok" : "bad")</code>	<code>\$display("%s", c ? ...)</code>
32	Variable initialisers ignored by ASIC synthesis	<code>logic [3:0] cnt = 0;</code> as reset	explicit reset in <code>always_ff</code>
33	Out-of-range index silently tolerated	<code>mem[addr]</code> with <code>addr >= depth</code>	assertion on the index; lint SELRANGE
34	Same object put in a mailbox repeatedly	one handle, <code>randomize</code> + put in a loop	<code>t = new();</code> for every item
35	disable fork kills unrelated threads	<code>disable fork</code> in a shared process	wrap in <code>fork begin ... end join</code>
36	Dependence on argument evaluation order	<code>f(g(), g())</code> with side effects	evaluate into temporaries first

Table 27.3 - Thirty-six pitfalls with bad and good forms.

PITFALL: INITIAL VALUES IN RTL (ROW 32)

`logic [3:0] cnt = 0;` initialises the variable at time 0 in simulation, so the RTL simulates as if the counter were reset. ASIC synthesis ignores the initialiser (there is no power-on value in a standard-cell flop), and the gate-level netlist starts at X. FPGA tools do honour initialisers through the bitstream, which is why code ported from FPGA to ASIC breaks here. In ASIC RTL, reset every flop that needs a known value explicitly and let lint flag initialisers.

27.6 Tool portability

The LRM is large, and every tool implements a different subset with a different interpretation of the unclear corners. Table 27.4 summarises the practical situation for the tools most used on SoC projects; check your tool's release notes for the exact version you run, because open-source tools in particular move quickly.

Tool	Kind	Language coverage and notable behaviour
Icarus Verilog 12	4-state event-driven sim (open source)	Verilog-2005 nearly complete; SV design subset good; limited classes; no constraints, covergroups, concurrent SVA, clocking blocks or UVM
Verilator 5.x	2-state cycle-based compiler to C++ (open source)	very fast; broad SV incl. classes, queues, mailboxes, --timing; X mapped to 0/1; constraints ignored in 5.020; no covergroups; limited concurrent SVA; strong lint
VCS (Synopsys)	full 4-state sim	full IEEE 1800 incl. UVM, constraints, coverage, SVA; xprop; commercial
Xcelium (Cadence)	full 4-state sim	same class as VCS; multi-language; xprop; commercial
Questa (Siemens)	full 4-state sim	same class as VCS; strong debug and formal integration; commercial
Design Compiler / Fusion (Synopsys), Genus (Cadence)	ASIC synthesis	synthesizable SV subset (Chapter 17); ignore initial blocks and initialisers; honour unique/priority as optimisation hints; full_case/parallel_case pragmas
Vivado (AMD)	FPGA sim + synthesis	synthesis accepts a broad SV design subset and honours initialisers; simulator supports UVM with limitations
Yosys 0.x	open-source synthesis/formal	Verilog-2005 plus a growing SV subset (via plugins such as slang for full SV); used for open flows and formal (SymbiYosys)

Table 27.4 - Tool landscape (as of 2026; verify against your versions).

Behaviour	Observed in this book
Loop variable captured by <code>fork/join_none</code>	Icarus: 3,3,3 (LRM); Verilator 5.020: 0,1,2

Behaviour	Observed in this book
Concurrent calls of a static task	Icarus: shared storage (LRM); Verilator 5.020: private copies
Order of two initial blocks at time 0	Icarus and Verilator disagree
Blocking assignments between clocked processes	Icarus result depends on source order; Verilator's does not
Out-of-range read of a logic array	Icarus: X; Verilator: 0 (2-state)
Evaluation order of function-call arguments	Icarus left-to-right; Verilator right-to-left
sum() with (int'(item))	Verilator 5.020 kept the 8-bit width
typedef virtual fifo_if #(W, D)	Verilator 5.020 internal fault (Ch. 25)
Recursive package functions	Verilator 5.020 internal error; static class methods worked (Ch. 26)
randomize() with constraints	Verilator 5.020 compiles but ignores constraints

Table 27.5 - Portability differences actually hit while writing this book.

PORTABILITY HABITS THAT PAY

(1) Run at least two simulators in CI - for example Verilator for speed and lint plus a 4-state simulator for X behaviour and verification features. (2) Write race-free code (nonblocking for flops, clocking blocks or the NBA discipline in testbenches), so scheduling differences cannot matter. (3) Avoid relying on anything the LRM calls unspecified: process order, argument evaluation order, the value of a variable read at time 0. (4) Keep the synthesizable subset conservative (Chapter 17) and check every new construct against every tool in the flow, including lint, CDC, equivalence and emulation compilers, before adopting it.

27.7 Lint tools and rule categories

A linter checks source code against rules without simulating it. On SoC projects lint is a sign-off gate: RTL is not handed to synthesis until it is lint-clean against the project rule set, with every waiver reviewed. Commercial tools include Synopsys SpyGlass Lint and VC SpyGlass, Cadence Jasper Superlint and HAL, Siemens Questa Lint and Real Intent Ascent Lint. Open-source options are Verilator's `--lint-only`, Verible (a style linter and formatter from the CHIPS Alliance), slang (a fast, strict SV front end with good diagnostics) and Yosys for structural checks.

Rule category	Examples
Syntax and semantics	undeclared identifiers, implicit nets, port width/direction mismatch, duplicate declarations
Width and sign	truncation, extension, mixed signedness, unsized constants in concatenations
Synthesis-ability	initial blocks and delays in RTL, unsynthesizable constructs, real types, === in RTL
Sim/synth mismatch	incomplete sensitivity lists, full_case/parallel_case pragmas, casex, initialisers
Inference	latches, combinational loops, multiple drivers, undriven or unloaded signals, constant outputs
Clocks and resets	gated/derived clocks, async reset not synchronised, mixed edges, reset used as data
Style and naming	file/module name match, one module per file, naming conventions, default_nettype usage
CDC / RDC (separate tools)	unsynchronised crossings, reconvergence, reset-domain crossings (companion RTL guide)

Table 27.6 - Lint rule categories found in every commercial rule set.

27.8 Running lint on a buggy block

```
// buggy.sv - one small block, many classic mistakes (for lint)
module buggy (
    input  logic      clk, rst_n, en,
    input  logic [1:0] sel,
    input  logic [7:0] a, b,
    output logic [7:0] y,
    output logic [3:0] cnt,
    output logic      q2
);
    logic [7:0] m;
    logic      q1;
    logic      unused_sig;

    assign sum_ab = a + b;                // typo'd / undeclared net: implicit 1-bit wire

    always_comb                                // no default: latch on m
    case (sel)
        2'd0: m = a;
        2'd1: m = b;
        2'd2: m = sum_ab;
    endcase

    always @(posedge clk) begin            // blocking assignments in a clocked block
        q1 = en;
        q2 = q1;
    end

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) cnt <= 0;
        else      cnt <= cnt + 1;        // 32-bit expression into 4 bits

    assign y = m;
    assign y = a & b;                    // second driver of y
endmodule
```

buggy.sv

```
$ verilator --lint-only -Wall buggy.sv
%Warning-IMPLICIT: buggy.sv:14:10: Signal definition not found, creating implicitly: 'sum_ab'
%Warning-WIDTHTRUNC: buggy.sv:14:17: Operator ASSIGNW expects 1 bits on the Assign RHS,
    but Assign RHS's ADD generates 8 bits.
%Warning-WIDTHEXPAND: buggy.sv:20:15: Operator ASSIGN expects 8 bits on the Assign RHS,
    but Assign RHS's VARREF 'sum_ab' generates 1 bits.
%Warning-MULTIDRIVEN: buggy.sv:6:22: Bits [7:0] of signal 'y' have multiple combinational
    drivers
%Warning-CASEINCOMPLETE: buggy.sv:17:5: Case values incompletely covered (example pattern 0x3)
%Warning-BLKSEQ: buggy.sv:24:8: Blocking assignment '=' in sequential logic process
%Warning-BLKSEQ: buggy.sv:25:8: Blocking assignment '=' in sequential logic process
%Error: Exiting due to 7 warning(s)
```

Verilator 5.020 (context lines omitted; long lines wrapped at the indented continuation).

```
$ iverilog -g2012 -Wall -o /dev/null buggy.sv
buggy.sv:14: warning: implicit definition of wire 'sum_ab'.
buggy.sv:33: error: Unresolved net/uwire y cannot have multiple drivers.
1 error(s) during elaboration.
```

Icarus 12.


```
$ yosys -q -p "read_verilog -sv buggy2.sv; proc; check"      # buggy.sv minus line 33
buggy2.sv:14: Warning: Identifier '\sum_ab' is implicitly declared.
ERROR: Latch inferred for signal '\buggy2.\m' from always_comb process
      '\buggy2.$proc$buggy2.sv:0$2'.
```

Yosys 0.33 on a copy without the second driver of y (the module renamed buggy2).

Compare what each tool found:

Bug in buggy.sv	Verilator -Wall	iverilog -Wall	Yosys
implicit net sum_ab	IMPLICIT + width warnings	warning	warning
latch on m (missing default)	CASEINCOMPLETE (indirect)	-	ERROR: latch in always_comb
blocking in clocked block	BLKSEQ x2	-	-
two drivers of y	MULTIDRIVEN	error	(not run)
cnt + 1 into 4 bits	- (unsized constant exempt)	-	-
unused_sig never used	- (not reported in this run)	-	-

Table 27.7 - No single tool caught everything.

Three lessons. First, the tools are complementary: Verilator gave the broadest set of warnings, Icarus refused the multiple drivers outright, and only Yosys reported the latch - because inferring a latch is a synthesis question and Yosys actually synthesizes the process. Second, `always_comb` made the latch an *error* in Yosys rather than a silent inference: the SystemVerilog procedural keywords are machine-checkable statements of intent (Chapter 14). Third, a missing warning is not proof of correctness: the `cnt + 1` width change is intentional and conventional (and Verilator exempts it), and the unused signal was not reported in this run - commercial lint tools with a full rule set would flag both classes.

INTERVIEW INSIGHT: HOW TO RUN LINT ON A REAL PROJECT

Run it early and often (pre-commit, not pre-tapeout). Start from the vendor's or company's standard rule set (for RTL sign-off typically a few hundred rules), never from 'everything on'. Fix, do not waive, anything in the synthesis-mismatch, latch, multi-driver and clock/reset categories. Every waiver is written in a waiver file with the rule, the exact location, a reason and a reviewer, so that a new instance of the same rule is still reported. Track the lint count per block as a project metric.

27.9 Summary

- Expression width comes from the operands *and* the target; concatenation operands, shift counts and cast results are self-determined; one unsigned operand makes an expression unsigned; part-selects are unsigned.
- if/case are X-optimistic, ?: is X-pessimistic, 2-state types turn X into 0, and == with an X yields X: use !== in checkers and `case inside` instead of casex.
- Anything the LRM leaves unspecified - process order in a time step, argument evaluation order - differs between tools; the real runs here showed Icarus and Verilator disagreeing on initial-block order, fork loop capture, static tasks, out-of-range reads and argument order.
- Nonblocking assignments for flops, `automatic` for tasks and functions, a per-thread copy of loop variables in forks, and `$urandom` instead of `$random` remove whole classes of bugs.
- Variable initialisers are ignored by ASIC synthesis; reset explicitly.
- Lint is a sign-off gate with categories for syntax, width, synthesis-ability, mismatch, inference, clocks/resets and style; on the buggy block, Verilator, Icarus and Yosys each caught something the others missed.

27.10 Exercises

- 1 Predict, then verify with Icarus, the value and width of each: `4'hF + 4'h1` assigned to a 4-bit and to an 8-bit variable; `{4'hF + 4'h1}` assigned to 8 bits; `-4'sd1 >>> 1` and `-4'sd1 >> 1` assigned to a `logic signed [3:0]`.
- 2 Write a module where an X on a mux select produces 0 in RTL simulation but the synthesized gate-level netlist (use Yosys `synth` and `write_verilog`) produces X in Icarus. Explain which construct is X-optimistic.
- 3 Fix `fork_static.sv` so that it prints 0, 1, 2 and 'called with 1' plus 'called with 2' on every LRM-compliant simulator, and explain why each change is required.
- 4 Add the directive `default_nettype none` to `buggy.sv` and rerun both linters. Then fix every warning without waivers and confirm that `verilator --lint-only -Wall` and the Yosys `check` pass are clean.
- 5 Pick five rows of Table 27.3 and write a one-line lint rule for each (in words) that would detect the bad form. Which of them cannot be detected statically, and what would you use instead?
- 6 Add a `$finish` to `pit_x.sv` and run it on Verilator with `--x-initial unique` and `--x-assign unique` and several seeds. Which lines change, and why can none of them print `x`?

28. The Verilog/SystemVerilog Learning Roadmap for SoC and ASIC Roles

SystemVerilog is one language, but nobody on a chip team uses all of it. An RTL designer writes a strict synthesizable subset and must know it perfectly; a verification engineer lives in classes, randomization, assertions, coverage and UVM; a DFT or gate-level engineer reads netlists full of primitives, specify blocks and SDF; a formal engineer writes almost nothing but SVA. This final chapter turns the book into a plan: the stages of growth, the language subset each role needs, a week-by-week study schedule mapped to the chapters, practice projects, a free tool setup, how to use the IEEE 1800 LRM, and what interviews actually test.

28.1 Stages of growth

Stage	You can...	Language focus	Book chapters
1. Beginner	write and simulate small modules and testbenches; read waveforms	modules, ports, nets vs variables, operators, always/initial, blocking vs nonblocking	1-6
2. RTL designer	write lint-clean, synthesizable, parameterized blocks; FSMs, FIFOs, bus interfaces; avoid sim/synth mismatch	synthesizable Verilog + SV design subset: logic, always_ff/comb, enums, structs, packages, interfaces, generate, parameters	7-17, 27
3. DV engineer	build self-checking, constrained-random, coverage-driven environments; write assertions; use UVM	classes, randomization, processes/IPC, clocking blocks, SVA, covergroups, DPI, UVM	18-26
4. Senior / lead	define methodology and coding rules, review others' code, debug across tools, own sign-off (lint, CDC, formal, GLS), mentor	the whole LRM as a reference; portability; scheduling semantics; tool behaviour	all, plus the LRM

Table 28.1 - Four stages. Most engineers specialise at stage 2 or 3, then broaden at stage 4.

DEPTH BEFORE BREADTH

The fastest route to stage 2 is not reading about many features, but mastering a few rules completely: expression sizing and signedness (Chapter 4), the scheduler and nonblocking assignment (Chapter 5), and the synthesizable subset (Chapters 11 and 17). Interviewers and code reviewers probe exactly those, and almost every bug in Chapter 27 comes from one of them.

28.2 Which language subsets each role needs

```

+-----+
| UVM / methodology libraries                      (DV)                      |
+-----+
| classes, constraints, covergroups, mailboxes, DPI (DV, modelling)          |
+-----+
| SVA: immediate + concurrent                      | clocking blocks, program    |
| (DV, formal, RTL designers)                     | blocks (DV)                        |
+-----+
| SV design subset: logic, always_ff/comb, enum, struct, package,          |
| interface, unique/priority, inside, type parameters (RTL, all)          |
+-----+
| Verilog-2005 core: modules, nets/variables, operators, generate,          |
| parameters, tasks/functions, system tasks                      (everyone)  |
+-----+
| gate level: primitives, UDPs, specify, timing checks, SDF (DFT, GLS)      |
+-----+

```

Figure 28.1 - The language as layers. Every role needs the core; each role then goes deep in a different layer.

Role	Must know deeply	Should know	Can skip at first
RTL design	synthesizable Verilog-2005; SV design subset (Ch. 12-17); sizing/signedness; NBA; generate; parameters; lint rules	immediate and simple concurrent SVA; interfaces/modports; basic testbenches	classes, constraints, UVM internals, PLI/VPI
Design verification (DV)	classes and OOP; constrained random; processes and IPC; clocking blocks; SVA; covergroups; UVM	DPI-C; RTL subset (to read and debug the DUT); scheduling regions	UDPs, specify blocks, switch-level modelling
DFT and gate-level simulation	primitives, UDPs, specify blocks, timing checks, SDF annotation (Ch. 9); 4-state/X behaviour; netlist reading	RTL subset; testbenches; \$sdf_annotate and notifiers; VPI basics	UVM, covergroups
Formal verification	SVA in depth (Ch. 22): sequences, properties, assume/assert/cover, local variables, liveness; bind	RTL subset; checkers; let; coverage of properties	classes, randomization, UVM
Architecture / performance modelling	classes, queues, associative arrays, DPI-C to C++ models	TLM-style transactions; UVM basics	gate level, specify
FPGA prototyping / emulation	synthesizable subset as accepted by the FPGA/emulator compiler; portability	SVA synthesizable subset; DPI for transactors	UVM internals

Table 28.2 - Language needs by role.

28.3 A 22-week study plan

The plan assumes 8-10 hours per week alongside a job or degree. Each week has a reading target, a coding target that must run on a free tool, and a checkpoint question you should be able to answer without notes. Weeks 1-12 make an RTL-ready engineer; weeks 13-22 add verification. An RTL-only track can stop after week 12 and spend weeks 13-16 on projects; a DV-track engineer with an HDL background can compress weeks 1-8 into four.

Week	Chapters	Build and run	Checkpoint
1	1-2	Tool setup; hello-world module and testbench in Icarus and Verilator	What does each tool in Table 27.4 do?
2	3	4-state experiments; nets vs variables; strengths	Why can a wire not be assigned in an always block?
3	4	Width/sign puzzles from Ch. 27 - predict, then run	Width and sign of $a+b*c$ with mixed operands
4	5	Scheduler experiments: races, #0, NBA	Explain the Active/Inactive/NBA regions
5	6	ALU with functions and tasks; automatic vs static	When is a function static, and why does it matter?
6	7-8	Parameterized FIFO with generate; macros; include guards	localparam vs parameter vs a define macro
7	9-10	Gate-level model with specify; \$sdf_annotate reading; VCD dump	What does a \$setuphold violation do?
8	11	Synthesize with Yosys; find and fix a latch and a mismatch	Three causes of sim/synth mismatch
9	12-13	Enum FSM with struct outputs; queues and associative arrays in a TB	Packed vs unpacked; which are synthesizable?
10	14	always_comb/ff, unique case, inside, streaming	What does unique case promise the synthesizer?
11	15-16	Package + interface with modports; virtual interface in a class	Why is a virtual interface needed?

Week	Chapters	Build and run	Checkpoint
12	17, 27	Refactor an older Verilog block to clean SV; lint it to zero warnings	Which SV features are safe in every flow?
13	18	Class hierarchy for transactions; copy, compare, polymorphism	Virtual vs non-virtual method call
14	19	Constraints on EDA Playground (commercial simulator)	solve-before, dist, soft constraints
15	20-21	Mailbox pipeline; clocking-block driver	join vs join_any vs join_none; #1step
16	22	Assertions for a handshake protocol; bind to RTL	Overlapping vs non-overlapping implication
17	23	Covergroups with crosses and bins; coverage closure	Code vs functional coverage
18	24	DPI-C reference model; cocotb test of the same DUT	What types can cross DPI?
19	25	Rebuild Chapter 25's testbench for your own DUT	Where does end-of-test logic live?
20	26	Port it to UVM on EDA Playground	Factory, config_db, objections in your own words
21-22	Project	One project from Table 28.4, end to end	Present it as if in an interview

Table 28.3 - A 22-week plan mapped to this book.

28.4 Practice projects

Project	Skills exercised	Chapters	Tools
Parameterized synchronous and asynchronous FIFO with a self-checking TB	generate, parameters, Gray code, CDC, scoreboard	7, 11, 25	Icarus, Verilator
UART TX/RX with baud generator	FSMs, counters, testbench tasks, file I/O for stimulus	6, 10, 12	Icarus, GTKWave/Surfer
APB or AXI-Lite register block	structs, packages, interfaces, address decode, reset values	12-17	Verilator, Yosys
Pipelined MAC / FIR filter	signed arithmetic, sizing, saturation, DPI-C golden model	4, 24	Verilator + C
Round-robin arbiter with assertions	SVA, formal proof of fairness and mutual exclusion	22	SymbiYosys (open) or commercial
Layered class TB for the register block	classes, mailboxes, virtual interfaces, end-of-test	18, 20, 25	Verilator
UVM agent for APB with RAL	UVM, factory, sequences, RAL	26	EDA Playground
Tiny RISC-V core (RV32I subset)	large RTL, decode, pipelines, self-checking programs	11-17	Verilator, Yosys
Gate-level sim of a synthesized block	netlists, SDF-style delays, X at reset	9, 11	Yosys, Icarus
Lint-clean refactor of an open-source IP	reading real code, portability, waivers	27	Verilator, Verible

Table 28.4 - Practice projects. Publish them in a public repository with a README and a CI job that runs the tests.

28.5 Free tools setup

Tool	Purpose	Notes
Icarus Verilog	4-state event-driven simulation	best for Verilog and X behaviour; <code>iverilog -g2012 -Wall</code>

Tool	Purpose	Notes
Verilator	fast 2-state simulation, lint	<code>--binary</code> <code>--timing</code> builds a stand-alone executable; <code>--lint-only</code> <code>-Wall</code>
Yosys	synthesis, structural checks, formal front end	<code>synth</code> , <code>stat</code> , <code>check</code> ; SymbiYosys for formal
GTKWave / Surfer	waveform viewers	VCD/FST from <code>\$dumpvars</code> or Verilator <code>--trace</code>
Verible, slang	style lint/format; strict SV parsing	good complements to Verilator lint
cocotb	Python testbenches driving Icarus or Verilator	Chapter 24
EDA Playground	browser access to commercial simulators	the free way to run UVM, constraints, covergroups, full SVA

Table 28.5 - A complete free tool kit.

```
# Debian/Ubuntu (versions used in this book: Icarus 12, Verilator 5.020, Yosys 0.33)
sudo apt install iverilog verilator yosys gtkwave
pip install cocotb # optional: Python testbenches
# smoke test on the FIFO of Chapter 25
iverilog -V 2>&1 | head -1
verilator --version
yosys -V
iverilog -g2012 -Wall -o /dev/null sync_fifo.sv
verilator --lint-only -Wall sync_fifo.sv && echo "verilator lint: clean"
yosys -q -p "read_verilog -sv sync_fifo.sv; synth -top sync_fifo; tee -o stat.txt stat"
grep -E "Number of cells|DFF" stat.txt
```

Installation and a three-tool smoke test.

```
Icarus Verilog version 12.0 (stable) ()
Verilator 5.020 2024-01-01 rev (Debian 5.020-1)
Yosys 0.33 (git sha1 2584903a060)
sync_fifo.sv:30: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
verilator lint: clean
  Number of cells:          91
    $ _DFFE_PN0P_           7
    $ _DFFE_PP_             32
```

The smoke test as run. Icarus notes that it does not check unique; Yosys maps the FIFO to 7 reset flops (pointers and count) and 32 non-reset flops (the 4 x 8 storage).

Even the smoke test teaches something: each tool has a different view of the same file. Icarus simulates it but ignores the `unique` qualifier; Verilator lints it clean; Yosys shows exactly which flops have a reset - the storage array has none, as intended. Setting up a Makefile that runs all three on every change is the first step toward the multi-tool CI habit of Chapter 27.

28.6 Reading the IEEE 1800 LRM efficiently

The IEEE 1800 standard (the Language Reference Manual, LRM) is the final authority on every question in this book. IEEE 1800-2017 and 1800-2023 run to well over a thousand pages; nobody reads them cover to cover. They have been available at no cost through the IEEE GET Program, sponsored by Accellera. Use the standard as a reference, with a strategy:

- **Start from the clause map** (Table 28.6) and go straight to the clause for your question.
- **Read the normative words.** 'Shall' is a requirement on tools and code; 'may' is permission; 'should' is a recommendation. Notes and examples are informative, not normative - useful, but they do not override the text.
- **Use Annex A (formal syntax)** to settle what is legal: if a construct cannot be derived from the BNF, it is not legal, whatever a tool accepts.

- **Look for 'unspecified', 'indeterminate' and 'implementation-dependent'.** These words mark exactly where tools may differ (Chapter 27).
- **Keep a short list of key tables:** the expression bit-length rules (Table 11-21 in 1800-2017), operator precedence (Table 11-2), and the scheduling-region figure in Clause 4.
- **Check the version.** Tools lag the standard; a feature new in 1800-2023 may not be in your simulator yet. Test before adopting.

Clause (1800-2017)	Topic	This book
4	Scheduling semantics (event regions)	5, 21
5-7	Lexical conventions, data types, aggregate types	2, 3, 12, 13
8	Classes	18
9-10	Processes; assignment statements	5, 14, 20
11	Operators and expressions (sizing, signedness)	4
12-13	Procedural statements; tasks and functions	6, 14
14-15	Clocking blocks; interprocess synchronization	20, 21
16-17	Assertions; checkers	22
18-19	Constrained random; functional coverage	19, 23
20-22	Utility and I/O system tasks; compiler directives	8, 10
23-27	Modules, programs, interfaces, packages, generate	2, 7, 15, 16, 21
28-33	Gate/switch level, UDPs, specify, timing checks, SDF, configurations	7, 9
35	Direct Programming Interface (DPI)	24
36-40	PLI/VPI, object model, assertion and coverage APIs	10, 24
Annex A	Formal syntax (BNF)	Appendix A

Table 28.6 - LRM clause map (IEEE 1800-2017 numbering; 1800-2023 is organised the same way).

A 30-MINUTE LRM HABIT

Whenever this book or a colleague states a rule, find it in the LRM and read the surrounding paragraph. After a few weeks you will navigate the standard as quickly as a search engine, and you will be the person on the team who settles arguments with a clause number - a senior-level skill.

28.7 Interview preparation

Interviews for RTL and DV roles test language understanding far more than trivia. Expect to write code on a whiteboard or shared editor, predict the output of short snippets, and explain *why*. Appendix C has 100 questions with answers; Table 28.7 lists the topics that come up most.

Area	Topics that come up again and again
Core language	blocking vs nonblocking and the scheduler; wire vs reg vs logic; 4-state values; == vs ===; expression sizing and signedness puzzles; \$display vs \$strobe vs \$monitor
RTL design	latch inference; FSM coding styles (one/two/three always blocks, one-hot vs binary); synchronous vs asynchronous reset; FIFO depth and full/empty logic; clock gating; CDC synchronizers and Gray code; sim/synth mismatch; parameterization and generate
SystemVerilog design	always_comb/ff/latch; unique/priority; packed vs unpacked; structs and unions; enums; interfaces and modports; packages and scoping
Verification language	classes, inheritance, virtual methods, polymorphism, shallow vs deep copy; static members; parameterized classes; virtual interfaces; fork/join variants and disable fork; mailboxes, semaphores, events; automatic vs static

Area	Topics that come up again and again
Randomization and coverage	rand vs randc; constraint operators; solve-before; dist; soft; pre/post_randomize; covergroups, bins, crosses; code vs functional coverage
Assertions	immediate vs concurrent; implication operators; sequences, repetition, throughout, within; \$rose/\$fell/\$stable/\$past; disable iff; assume/cover; bind
UVM	component vs object; phases; factory and overrides; config_db; sequences, sequencer, driver handshake; TLM and analysis ports; objections; RAL basics
Practical	debug a failing test from a log; write a scoreboard; code a small block (arbiter, FIFO, edge detector, counter) live; explain a past project in depth

Table 28.7 - Interview topics by area.

HOW TO ANSWER A LANGUAGE QUESTION WELL

State the rule, give a two-line example, name the consequence in hardware or in the testbench, and mention the pitfall or tool difference if there is one. For 'blocking vs nonblocking': the rule (NBA updates happen in the NBA region after all RHS are evaluated), the example (a two-flop shift register), the consequence (blocking in clocked logic can collapse pipeline stages and create races), and the pitfall (race.sv in Chapter 27 gave different answers on two simulators). That structure signals understanding rather than memorisation.

28.8 Final checklist

Before you call yourself fluent in Verilog and SystemVerilog

- ☐ I can predict the width and signedness of any expression and explain every result in Chapter 27's pit_expr.sv.
- ☐ I can explain the event regions of one time step and why nonblocking assignments prevent races between flops.
- ☐ I write RTL that is lint-clean under Verilator -Wall, has no latches, no implicit nets, one driver per signal, and explicit resets.
- ☐ I use always_ff/always_comb, enums, structs, packages and interfaces in RTL, and know which SV features every tool in my flow accepts.
- ☐ I can read a gate-level netlist with primitives and specify blocks, and know what SDF annotation and timing-check notifiers do.
- ☐ I can build a layered class-based testbench with a monitor, scoreboard and end-of-test logic, and prove it fails on a planted bug.
- ☐ I can write constraints, covergroups and concurrent assertions, and know which simulator I need to run them.
- ☐ I can explain how the UVM factory, config_db, phases and objections work in terms of SystemVerilog classes and static members.
- ☐ I know the portability differences between at least two simulators and run more than one in my regression.
- ☐ I can find the answer to a language question in the IEEE 1800 LRM in a few minutes and cite the clause.

28.9 Summary

- Growth runs from beginner through RTL designer and DV engineer to senior/lead; each stage adds a layer of the language and a level of responsibility for quality.
- Every role needs the Verilog core and the scheduling and sizing rules; RTL goes deep on the synthesizable subset, DV on OOP, randomization, SVA, coverage and UVM, DFT/gate-level on primitives, specify and SDF, and formal on SVA.
- A 22-week plan maps weekly reading, a runnable exercise and a checkpoint question to the chapters of this book, followed by an end-to-end project.
- Icarus, Verilator and Yosys (plus a waveform viewer and EDA Playground for commercial-only features) form a complete free tool kit; run them together from the start.

- Use the IEEE 1800 LRM as a reference: clause map, normative words, Annex A syntax, and the words that mark tool-dependent behaviour.
- Interviews test understanding of rules and consequences; answer with rule, example, consequence and pitfall.

28.10 Exercises

- 1 Place yourself in Table 28.1 honestly. List the three chapters whose checkpoint questions you cannot yet answer without notes and schedule them for the next two weeks.
- 2 Set up a repository with a Makefile that runs Icarus, Verilator lint and Yosys synthesis on every RTL file, plus a CI job that fails on a new warning. Add the FIFO of Chapter 25 as the first block.
- 3 Choose one project from Table 28.4 for your target role and write a one-page verification plan for it before writing any RTL: features, checks, coverage goals and the tool that will run each part.
- 4 Find in the IEEE 1800 LRM the rules behind three results in Chapter 27 (for example: why cat9 is 44, why the fork prints 3,3,3, why a static function initialiser runs once) and write down the clause numbers.
- 5 Write your own answers to ten questions from Table 28.7 using the rule-example-consequence-pitfall structure, then compare them with Appendix C.

APPENDICES

Appendices

A keyword and syntax quick reference, the operator, sizing and system-task rules in table form, 100 interview questions with answers, and the standards, tools, books and code bases of the Verilog and SystemVerilog world.

Appendix A. Keywords and Syntax Quick Reference

This appendix is the page you keep open while writing code. It lists every reserved word of Verilog-2005 and every word SystemVerilog added, grouped by what the word is for, and then gives a compact, correct template for each major construct. Each template was checked by a tool: the design subset with Icarus Verilog and Verilator, and the verification-only constructs (covergroups, properties, clocking blocks, programs, constraints) by full elaboration in the open-source **slang** compiler, because neither open simulator here executes them. Chapter numbers point to the full treatment.

Verilog-2005 reserved words (IEEE 1364-2005, Annex B)

Verilog-2005 has 123 reserved words. Keywords are lower case and case-sensitive: **Module** is a legal identifier, **module** is not. A reserved word can still be used as a name by writing it as an *escaped identifier* (**\module** - backslash, the name, then white space), which is how netlist writers cope with names from other languages.

Category	Keywords
Design units and blocks	module , macromodule , endmodule , primitive , endprimitive , table , endtable , function , endfunction , task , endtask , specify , endspecify , generate , endgenerate , genvar , begin , end , fork , join
Ports, nets and variables	input , output , inout , wire , uwire , tri , tri0 , tri1 , wand , wor , triand , trior , triereg , supply0 , supply1 , reg , integer , real , realtime , time , event , signed , unsigned , scalared , vectored , automatic
Parameters	parameter , localparam , defparam , specparam
Procedural code	initial , always , assign , deassign , force , release , if , else , case , casex , casez , endcase , default , for , forever , repeat , while , wait , disable , posedge , negedge , edge
Gate and switch primitives	and , nand , or , nor , xor , xnor , not , buf , bufif0 , bufif1 , notif0 , notif1 , nmos , pmos , rnmos , rpmos , cmos , rcmos , tran , tranif0 , tranif1 , rtran , rtranif0 , rtranif1 , pullup , pulldown
Drive and charge strengths	supply0 , supply1 , strong0 , strong1 , pull0 , pull1 , weak0 , weak1 , highz0 , highz1 , small , medium , large
Specify blocks and pulse control	ifnone , pulstyle_oneevent , pulstyle_ondetect , showcancelled , noshowcancelled
Configurations (Chapter 7)	config , endconfig , design , instance , cell , use , liblist , library , incdir , include

Verilog-2005 keywords by category (**supply0/supply1** appear twice because they are both net types and strengths). **uwire** is the only word new in 1364-2005; the rest date from 1995 or 2001.

Words SystemVerilog added (IEEE 1800)

SystemVerilog is a superset of Verilog, so every word above stays reserved and about 130 more join it. The additions arrived in three waves: IEEE 1800-2005 (the bulk), 1800-2009 (temporal logic, checkers, **let**) and 1800-2012 (interface classes, soft constraints, user-defined nets). 1800-2017 added no keywords, and 1800-2023 expresses its new features with existing words (for example the **:initial**, **:extends** and **:final** method qualifiers), so a 2012 keyword list is still complete for practical work - but check Annex B of the edition your tools claim.

Category	Keywords (edition in brackets if after 2005)
Data types and declarations	bit , byte , shortint , int , longint , logic , shortreal , chandle , string , enum , struct , union , packed , tagged , typedef , type , var , void , const , static , nettype interconnect [2012], untyped [2009]

Category	Keywords (edition in brackets if after 2005)
Procedural blocks and statements	<code>always_comb</code> , <code>always_ff</code> , <code>always_latch</code> , <code>final</code> , <code>unique</code> , <code>priority</code> , <code>do</code> , <code>break</code> , <code>continue</code> , <code>return</code> , <code>foreach</code> , <code>inside</code> , <code>iff</code> , <code>join_any</code> , <code>join_none</code> , <code>wait_order</code> , <code>randcase</code> , <code>randsequence</code> , <code>matches</code> , <code>alias</code> , <code>unique0</code> [2009]
Structure and scoping	<code>interface</code> , <code>endinterface</code> , <code>modport</code> , <code>package</code> , <code>endpackage</code> , <code>import</code> , <code>export</code> , <code>program</code> , <code>endprogram</code> , <code>clocking</code> , <code>endclocking</code> , <code>bind</code> , <code>timeunit</code> , <code>timeprecision</code> , <code>extern</code> , <code>context</code> , <code>forkjoin</code> , <code>checker</code> <code>endchecker</code> <code>let</code> <code>global</code> [2009]
Classes (OOP)	<code>class</code> , <code>endclass</code> , <code>extends</code> , <code>new</code> , <code>null</code> , <code>this</code> , <code>super</code> , <code>local</code> , <code>protected</code> , <code>pure</code> , <code>virtual</code> , <code>implements</code> [2012]
Randomization	<code>rand</code> , <code>randc</code> , <code>constraint</code> , <code>solve</code> , <code>before</code> , <code>dist</code> , <code>with</code> , <code>soft</code> [2012]
Assertions (SVA)	<code>assert</code> , <code>assume</code> , <code>cover</code> , <code>expect</code> , <code>property</code> , <code>endproperty</code> , <code>sequence</code> , <code>endsequence</code> , <code>first_match</code> , <code>intersect</code> , <code>throughout</code> , <code>within</code> , <code>restrict</code> , <code>implies</code> , <code>until</code> , <code>s_until</code> , <code>until_with</code> , <code>s_until_with</code> , <code>nexttime</code> , <code>s_nexttime</code> , <code>s_always</code> , <code>eventually</code> , <code>s_eventually</code> , <code>accept_on</code> , <code>reject_on</code> , <code>sync_accept_on</code> , <code>sync_reject_on</code> , <code>strong</code> , <code>weak</code> [2009]
Functional coverage	<code>covergroup</code> , <code>endgroup</code> , <code>coverpoint</code> , <code>cross</code> , <code>bins</code> , <code>illegal_bins</code> , <code>ignore_bins</code> , <code>binsof</code> , <code>wildcard</code>

SystemVerilog keyword additions by category.

PITFALL - LEGACY CODE THAT STOPS COMPILING

Moving a Verilog file to SV mode (`.sv` suffix, `-sv`, `-g2012`) can break it because ordinary-looking names are now keywords: `bit`, `byte`, `int`, `logic`, `type`, `ref`, `do`, `final`, `class`, `new`, `this`, `soft`, `global`, `matches`, `interface`, `package`, `program`, `sequence`. Even pure Verilog bites: `small`, `medium` and `large` are charge strengths and `cell`, `design`, `instance`, `library`, `use` are configuration words (this very appendix hit `small`, `large` and `matches` while its templates were being checked).

Fix it per file by bracketing the old code with the `begin_keywords "1364-2005"` and `end_keywords` directives (each written with a leading grave accent, Chapter 8), which tell the compiler which keyword set the enclosed text was written against.

Lexical cheat sheet

Item	Form	Notes
Comments	<code>// line</code> <code>/* block */</code>	Block comments do not nest
Identifiers	<code>[a-zA-Z_][a-zA-Z0-9_\$]*</code> , escaped <code>\any_chars</code>	Case-sensitive; <code>\$</code> cannot start a user name (reserved for system tasks)
Sized literal	<code><size>'<s?><base><digits></code> : <code>8'hA5</code> , <code>4'sb1010</code> , <code>12'o7_7_7</code>	Base <code>b o d h</code> (either case); <code>x z ?</code> legal in <code>b/o/h</code>
Unsize literal	<code>42</code> , <code>'hFF</code> , <code>'b1</code>	At least 32 bits; plain decimal is signed, based is unsigned
Fill literal (SV)	<code>'0</code> <code>'1</code> <code>'x</code> <code>'z</code>	Every bit of the context width
Real	<code>3.14</code> , <code>1.5e-9</code> , <code>2E3</code>	<code>.5</code> and <code>5.</code> are illegal
Time literal (SV)	<code>10ns</code> , <code>2.5us</code> , <code>1step</code>	No space before the unit; units <code>s ms us ns ps fs</code>
String	<code>"text\n"</code>	8 bits per character when assigned to a vector; SV adds <code>\v \f \a \x41</code>
System name	<code>\$display</code> , <code>\$clog2</code>	Built-in or PLI/VPI tasks and functions
Directive	<code>define</code> , <code>ifdef</code> , <code>timescale</code> , <code>include</code> preceded by a grave accent	Processed before parsing (Chapter 8)
Attribute	<code>(* keep = 1 *)</code>	Tool hints attached to the next item

Template: package and module

A package holds shared types, parameters and functions (Chapter 15). The module header uses the ANSI style with the parameter port list first; a **localparam** in the parameter port list (legal in SystemVerilog) cannot be overridden. Imports in the header make package names visible to the port list.

```
package bus_pkg;
  parameter int AW = 32;                // package parameters are constants
  typedef logic [AW-1:0] addr_t;
  typedef enum logic [1:0] {IDLE, BUSY, DONE, ERR} state_e;
  typedef struct packed {
    addr_t      addr;
    logic [7:0] len;
    logic       write;
  } req_t;
  function automatic logic [7:0] parity8(logic [7:0] d);
    return {7'b0, ^d};
  endfunction
endpackage
```

Package template (Verilator lint and slang elaboration clean)

```
module fifo_ctrl
  import bus_pkg::*;                // import in the module header
  #(
    parameter int DEPTH = 8,          // overridable
    localparam int PW = $clog2(DEPTH) // not overridable
  ) (
    input logic      clk,
    input logic      rst_n,
    input logic      push, pop,
    input req_t      wdata,
    output req_t      rdata,
    output logic [PW:0] count
  );
  req_t      mem [DEPTH];            // unpacked array: DEPTH entries
  logic [PW-1:0] wp, rp;

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
      wp <= '0; rp <= '0; count <= '0;
    end else begin
      if (push) begin mem[wp] <= wdata; wp <= wp + 1'b1; end
      if (pop)  rp <= rp + 1'b1;
      count <= count + (PW+1)'(push) - (PW+1)'(pop);
    end

  assign rdata = mem[rp];
endmodule

module top;
  import bus_pkg::*;
  logic clk = 0, rst_n = 0, push = 0, pop = 0;
  req_t wdata = '0, rdata;
  logic [3:0] count;
  fifo_ctrl #(DEPTH(8)) u_fifo (.clk, .rst_n, .push, .pop, // .name implicit
                                .wdata, .rdata, .count(count)); // named
endmodule
```

Module template with header import, parameters, **.name** implicit connections and a named connection (Icarus, Verilator and slang clean)

Port connection style	Example	Rule
Positional	<code>fifo_ctrl u (clk, rst_n, ...);</code>	Order must match; fragile - avoid beyond primitives
Named	<code>.count(count)</code>	Explicit; unconnected ports allowed (<code>.x()</code>)
Implicit .name (SV)	<code>.clk</code>	Same name and compatible type must exist; no implicit nets created
Wildcard .* (SV)	<code>fifo_ctrl u (.*);</code>	Connects every port to a same-named signal; explicit connections override

Template: interface, modport and clocking block

```

interface bus_if #(parameter int DW = 32) (input logic clk);
    logic          valid, ready;
    logic [DW-1:0] data;

    clocking cb @(posedge clk);                // testbench view, race-free
        default input #1step output #1;
        output valid, data;
        input  ready;
    endclocking

    modport mst (output valid, data, input  ready);
    modport slv (input  valid, data, output ready);
    modport tb  (clocking cb);

    function automatic bit fire();              // methods may live in interfaces
        return valid && ready;
    endfunction
endinterface

module sink (bus_if.slv b);                    // modport-typed port
    assign b.ready = 1'b1;
endmodule

module top_if;
    logic clk = 0;
    bus_if #(.DW(16)) b (.clk);
    sink u_sink (.b(b));                      // connect instance, modport chosen above
endmodule

```

Interface template (slang clean; Verilator 5.020 rejects `modport ... (clocking cb)` as unsupported)

Instantiate an interface like a module; pass it through a port typed `bus_if.slv` (restricted by the modport) or `bus_if` (full access). A class reaches it through a *virtual interface* handle, `virtual bus_if.tb vif`; (Chapter 16).

Template: typedef, enum, struct and union

```

module t_types;
    typedef enum logic [2:0] {RD = 3'b001, WR = 3'b010, NOP = 3'b100} op_e;
    typedef struct packed {                                // packed: a 16-bit vector, first field = MSBs
        op_e      op;                                     // [15:13]
        logic [4:0] reg_id;                               // [12:8]
        logic [7:0] imm;                                  // [7:0]
    } instr_t;
    typedef union packed {                                // all members must be the same width
        instr_t    f;
        logic [15:0] raw;
    }
endmodule

```

```

} instr_u;
typedef struct {                                // unpacked: may hold any types
    string name;
    real    delay;
    int     beats[4];
} txn_t;

instr_u u;
txn_t  t;
op_e   o;
initial begin
    u.f = '{op: WR, reg_id: 5'd3, imm: 8'hA5};          // assignment pattern
    $display("raw=%h op=%s reg=%0d", u.raw, u.f.op.name(), u.f.reg_id);
    o = op_e'(3'b100);                                // static cast to the enum
    $display("o=%s first=%s num=%0d", o.name(), o.first().name(), o.num());
    o = o.next();                                      // wraps from last to first
    $display("o.next()=%s", o.name());
    t = '{name: "burst", delay: 2.5, beats: '{default: 0}}';
    $display("%s has %0d beats, $bits(instr_t)=%0d", t.name, $size(t.beats), $bits(instr_t));
    $finish;
end
endmodule

```

User-defined types (Verilator 5.020)

```

raw=43a5 op=WR reg=3
o=NOP first=RD num=3
o.next()=RD
burst has 4 beats, $bits(instr_t)=16

```

`next()` wraps from the last member to the first; the static cast `op_e'(...)` is unchecked, so use `$cast` when the value might not be a legal member (Appendix B, casting table).

Template: `always_comb`, `always_ff` and `always_latch`

```

module t_alw (
    input logic      clk, rst_n, en, g,
    input logic [1:0] sel,
    input logic [7:0] a, b, c, d,
    output logic [7:0] y, q, l
);
    always_comb begin                                // combinational: runs at time 0, full sensitivity
        unique case (sel)                            // parallel + full: warns if no/multiple match
            2'd0: y = a;
            2'd1: y = b;
            2'd2: y = c;
            2'd3: y = d;
        endcase
    end

    always_ff @(posedge clk or negedge rst_n) // exactly one event control; flops only
        if (!rst_n) q <= '0;
        else if (en) q <= q + 8'd1;

    always_latch                                // intended latch (tools check it is one)
        if (g) l <= a;
endmodule

```

The three intent-specific processes (Icarus, Verilator and slang clean)

Process	Sensitivity	Rules enforced	Infers
<code>always_comb</code>	Implicit: every value read, including inside called functions	Runs once at time 0; no timing controls; no other process may write its outputs	Combinational logic (tools warn on a latch)
<code>always_latch</code>	Implicit, as <code>always_comb</code>	Same rules	Level-sensitive latch
<code>always_ff</code>	Exactly one event control at the top	No blocking timing controls inside; outputs written by this block only	Flip-flops
<code>always @*</code>	Implicit, but not inside functions	None	Whatever you wrote - often an accidental latch
<code>always</code>	Whatever you write	None	Testbench clocks, legacy RTL

Template: function and task

```

module t_ft;
  // function: zero time, returns a value (or void); SV allows output/ref args
  function automatic int unsigned popcnt(input logic [31:0] v, input int start = 0);
    int unsigned n = 0;
    for (int i = start; i < 32; i++) n += 32'(v[i]);
    return n;
  endfunction

  function automatic void swap(ref int a, ref int b); // ref: pass by reference
    int t = a; a = b; b = t;
  endfunction

  // task: may consume time; outputs are copied back when the task returns
  task automatic pulse(output logic sig, input int unsigned width);
    sig = 1'b1; #width; sig = 1'b0;
  endtask

  int x = 3, y = 7;
  logic p;
  initial begin
    $display("popcnt=%0d popcnt(start=8)=%0d", popcnt(32'hF0F0), popcnt(32'hF0F0, 8));
    swap(x, y);
    $display("after swap x=%0d y=%0d", x, y);
    pulse(p, 5);
    $display("t=%0t p=%b", $time, p);
    void'(popcnt(1)); // discard a return value
  end
endmodule

```

Functions, tasks, default arguments and `ref` (Verilator 5.020; Icarus 12 does not support `ref` arguments)

```

popcnt=8 popcnt(start=8)=4
after swap x=7 y=3
t=5 p=0

```

Feature	function	task
Consumes time (#, @, wait)	Never	May
Returns a value	Yes, or <code>void</code>	No (use <code>output/ref</code> arguments)
Can call	Functions only (and fork/join_none in SV)	Functions and tasks
Default lifetime in a module	static	static
Argument directions	<code>input output inout ref</code> (SV)	<code>input output inout ref</code>

Feature	function	task
Synthesizable	Yes, as combinational logic	Yes if it has no timing control

Template: generate

```

module t_gen #(parameter int N = 4, parameter bit USE_FAST = 1) (
    input  logic [N-1:0] a, b,
    output logic [N-1:0] y,
    output logic [N:0]  sum
);
    // generate-for: genvar loop, named block gives hierarchical names g_bit[i]
    for (genvar i = 0; i < N; i++) begin : g_bit
        assign y[i] = a[i] ^ b[i];
    end

    // generate-if: elaboration-time choice between implementations
    if (USE_FAST) begin : g_fast
        assign sum = a + b;
    end else begin : g_ripple
        logic [N:0] c;
        assign c[0] = 1'b0;
        for (genvar i = 0; i < N; i++) begin : g_fa
            assign {c[i+1], sum[i]} = a[i] + b[i] + c[i];
        end
        assign sum[N] = c[N];
    end

    // generate-case
    case (N)
        1:      begin : g_one  initial $display("N=1");      end
        default: begin : g_many initial $display("N=%0d", N); end
    endcase
endmodule

module t_gen_tb;
    logic [3:0] a = 4'd9, b = 4'd12, y1, y2;
    logic [4:0] s1, s2;
    t_gen #(.N(4), .USE_FAST(1)) u1 (.a, .b, .y(y1), .sum(s1));
    t_gen #(.N(4), .USE_FAST(0)) u2 (.a, .b, .y(y2), .sum(s2));
    initial #1 $display("fast=%0d ripple=%0d xor=%b carries=%b", s1, s2, y2, u2.g_ripple.c);
endmodule

```

generate-for, generate-if and generate-case; the **generate/endgenerate** keywords are optional since 1364-2005 (Icarus 12)

```

N=4
N=4
fast=21 ripple=21 xor=0101 carries=10000

```

Label every generate block: the label is part of the hierarchical name (**u2.g_ripple.c**), unlabeled blocks get tool-specific **genblkN** names that break scripts, waveforms and SDC constraints.

Template: class

```

package tb_pkg;
    typedef class scoreboard;                // forward declaration

    virtual class base_item;                // abstract: cannot be constructed
        static int count;                    // one copy shared by all objects
        local int id;                        // visible only inside base_item
        function new();

```

```

        id = count++;
    endfunction
    pure virtual function string convert2string();
    function int get_id(); return id; endfunction
endclass

class bus_item extends base_item;
    rand bit [31:0] addr;
    rand bit [7:0] len;
    randc bit [1:0] kind; // cyclic random
    constraint c_len { len inside {[1:16]}; }
    constraint c_addr { addr[1:0] == 2'b00; soft addr < 32'h1000; }
    function new();
        super.new();
    endfunction
    virtual function string convert2string();
        return $sprintf("#%0d addr=%h len=%0d", get_id(), addr, len);
    endfunction
    function void post_randomize(); // callback after randomize()
    endfunction
endclass

class fifo #(type T = int, int DEPTH = 4); // parameterized class
    protected T q[$];
    function bit put(T t);
        if (q.size() >= DEPTH) return 0;
        q.push_back(t);
        return 1;
    endfunction
    extern function T get(); // out-of-block definition below
endclass
function fifo::T fifo::get();
    return q.pop_front();
endfunction

interface class printable; // SV-2012 interface class
    pure virtual function void print();
endclass

class scoreboard implements printable;
    int unsigned n_match;
    virtual function void print(); $display("n_match=%0d", n_match); endfunction
endclass
endpackage

```

Class template: abstract base, static and local members, pure virtual method, constraints, parameterized class with an **extern** method, interface class (slang clean)

Template: program block

```

program automatic test (bus_if.tb bif); // program block: runs in the Reactive region
    initial begin
        bif.cb.valid <= 1'b0;
        repeat (2) @(bif.cb);
        bif.cb.valid <= 1'b1;
        bif.cb.data <= 32'hCAFE;
        @(bif.cb iff bif.cb.ready); // wait for a sampled ready
        bif.cb.valid <= 1'b0;
    end
    // implicit $finish when all program
endprogram // initial blocks have ended

```

Program block driving through a clocking block (slang clean, elaborated together with the interface template)

Template: covergroup

```

module cov_demo (input logic clk, input logic [3:0] op, input logic [1:0] mode,
                 input logic valid);
    covergroup cg_op @(posedge clk iff valid);
        option.per_instance = 1;
        cp_op : coverpoint op {
            bins zero      = {0};
            bins low[]     = {[1:3]};           // one bin per value
            bins high      = {[4:14]};
            illegal_bins bad = {15};
        }
        cp_mode : coverpoint mode;             // automatic bins: 4 values
        x_op_mode : cross cp_op, cp_mode {
            ignore_bins no_zero = binsof(cp_op.zero);
        }
    endgroup
    cg_op cg = new();                         // must be constructed

    covergroup cg_trans with function sample(logic [1:0] m);
        coverpoint m { bins up = (0 => 1 => 2); bins wrap = (3 => 0); }
    endgroup
    cg_trans ct = new();
    always @(posedge clk) if (valid) ct.sample(mode);
endmodule

```

Covergroups with an event, explicit bins, cross with `ignore_bins`, and a `sample()` override with transition bins (slang clean; needs a commercial simulator to execute)

Template: sequence, property and assertion statements

```

module sva_demo (input logic clk, rst_n, req, gnt, done, input logic [7:0] data);
    default clocking dcb @(posedge clk); endclocking
    default disable iff (!rst_n);

    sequence s_handshake;
        req ##[1:4] gnt;                       // gnt 1 to 4 cycles after req
    endsequence

    property p_req_gets_gnt;
        req |-> s_handshake;                   // overlapping implication
    endproperty

    property p_stable_until_done(sig);
        $rose(req) |>= $stable(sig) throughout done [->1];
    endproperty

    a_gnt    : assert property (p_req_gets_gnt) else $error("req not granted");
    a_stable : assert property (p_stable_until_done(data));
    m_in     : assume property (gnt |-> req);   // constraint for formal
    c_b2b    : cover property (req ##1 !req ##1 req);
    a_known  : assert property (!$isunknown({req, gnt}));

    always_comb begin                          // immediate (non-temporal) assertion
        a_imm : assert (!(gnt && !rst_n)) else $warning("gnt in reset");
    end
endmodule

```

Concurrent and immediate assertions (slang clean; Verilator 5.020 rejects the `##[m:n]` range, run on a commercial simulator or SymbiYosys with a commercial front end)

Template: fork variants

```
module t_fork;
  task automatic worker(string name, int d);
    #d $display("%3t %s done", $time, name);
  endtask
  initial begin
    fork worker("A", 10); worker("B", 20); join          // wait for all
    $display("%3t join finished", $time);
    fork worker("C", 10); worker("D", 20); join_any      // wait for first
    $display("%3t join_any finished", $time);
    fork worker("E", 5); join_none                      // do not wait
    $display("%3t join_none returned", $time);
    wait fork;                                           // D and E still running
    $display("%3t wait fork finished", $time);
    fork worker("F", 10); worker("G", 50); join_any
    disable fork;                                       // kill G
    $display("%3t disable fork: G killed", $time);
    #100 $finish;
  end
endmodule
```

All four fork endings plus `wait fork` and `disable fork`

```
10  A done
20  B done
20  join finished
30  C done
30  join_any finished
30  join_none returned
35  E done
40  D done
40  wait fork finished
50  F done
50  disable fork: G killed
```

Identical output from Verilator 5.020 and Icarus 12

Construct	Parent continues when	Children after parent resumes
<code>fork ... join</code>	All children finished	None left
<code>fork ... join_any</code>	First child finished	Keep running
<code>fork ... join_none</code>	Immediately (children start when the parent next blocks)	Keep running
<code>wait fork;</code>	All children of the current process finished	-
<code>disable fork;</code>	Immediately	All descendants of the current process killed - wrap in <code>fork begin ... end join</code> to limit the scope

Appendix B. Operators, Sizing Rules and System Tasks Reference

The rules in this appendix are the ones that decide what a line of RTL actually computes: how wide an expression is, whether it is signed, what happens to X and Z, and which built-in tasks exist. They are condensed from IEEE 1800-2017 clause 11 (operators and expressions) and clauses 20-21 (system tasks). Chapter 4 derives them with examples; Chapter 10 covers the system tasks in depth.

Operator precedence and associativity

Level	Operators	Associativity
1 (highest)	<code>() [] :: .</code>	Left
2	Unary <code>+ - ! ~ & ~& ~ ^ ~^ ~, ++ --</code>	-
3	<code>**</code>	Left
4	<code>* / %</code>	Left
5	Binary <code>+ -</code>	Left
6	<code><< >> <<< >>></code>	Left
7	<code>< <= > >=, inside, dist</code>	Left
8	<code>== != === !== ==? !=?</code>	Left
9	Binary <code>&</code>	Left
10	Binary <code>^ ~^ ~</code>	Left
11	Binary <code> </code>	Left
12	<code>&&</code>	Left
13	<code> </code>	Left
14	<code>?:</code> (conditional)	Right
15	<code>-> <-></code> (implication, equivalence)	Right
16	<code>= += -= *= /= %= &= ^= = <<= >>= <<<= >>>=, := :/, <=</code>	None
17 (lowest)	<code>{ } { { } }</code> (concatenation, replication)	Concatenation

IEEE 1800-2017 Table 11-2. `**` is left-associative in the LRM, unlike mathematics: `2**3**2` is 64 in both Icarus and Verilator. Parenthesise.

Operator reference

Group	Operators	Result	Notes
Arithmetic	<code>+ - * / % **</code>	max(L(i), L(j)) bits	Any X/Z bit makes the whole result X; <code>/</code> truncates toward zero; sign of <code>%</code> follows the first operand
Increment (SV)	<code>i++ ++i i-- --i</code>	L(i)	Blocking side effect; undefined if the variable is used twice in one expression
Assignment ops (SV)	<code>+= -= *= /= %= &= ^= = <<= >>= <<<= >>>=</code>	L(lhs)	Blocking; <code>a += b</code> is <code>a = a + b</code> with <code>a</code> evaluated once
Relational	<code>< <= > >=</code>	1 bit	X if any operand bit is X/Z; unsigned unless both operands signed
Logical equality	<code>== !=</code>	1 bit	X/Z anywhere -> X (unless a known bit already differs -> 0/1)

Group	Operators	Result	Notes
Case equality	<code>=== !==</code>	1 bit	Compares X and Z literally; never X; not synthesizable
Wildcard equality (SV)	<code>==? !=?</code>	1 bit	X/Z in the right operand are don't-cares
Logical	<code>! && -> <-></code>	1 bit	<code>&&</code> and <code> </code> short-circuit; operands reduced to 0/1/X
Bitwise	<code>~ & ^ ~^ ^~</code>	$\max(L(i), L(j))$	Per-bit 4-state tables below
Reduction	<code>& ~& ~ ^ ~^</code>	1 bit	Unary; <code>^v</code> is even/odd parity
Logical shift	<code><< >></code>	$L(i)$	Fill with 0; shift amount self-determined, treated as unsigned; X amount -> all X
Arithmetic shift	<code><<< >>></code>	$L(i)$	<code>>>></code> fills with the sign bit only if the expression is signed
Conditional	<code>c ? a : b</code>	$\max(L(a), L(b))$	If c is X, the result merges a and b bit by bit (equal bits kept, others X)
Concatenation	<code>{a, b}</code>	$L(a) + L(b)$	Operands self-determined; unsized constants illegal; result unsigned
Replication	<code>{n{a}}</code>	$n \times L(a)$	n must be a constant; <code>{0{a}}</code> is legal only inside a larger concatenation
Streaming (SV)	<code>{<<{a}}, {>>8{a}}</code>	Sum of operand bits	<code><<</code> reverses in slices (default 1 bit); <code>>></code> keeps order
Set membership (SV)	<code>a inside {1, [4:7], arr}</code>	1 bit	Uses <code>==</code> for singular values (<code>==?</code> style for X/Z in the set)
Distribution (SV)	<code>x dist {0 := 3, [1:9] :/ 1}</code>	-	Constraints only; <code>:=</code> weight per value, <code>:/</code> weight shared by range
Cast (SV)	<code>T'(e), 8'(e), signed'(e)</code>	Target	See casting table

Expression bit-length rules (IEEE 1800-2017 Table 11-21)

Every operand is either **context-determined** (it is extended to the width of the whole expression, including the assignment target) or **self-determined** (its width is fixed by itself alone). The table gives L, the bit length of each expression form; i, j, k are operands.

Expression	Bit length	Comment
Unsized constant	Same as integer (at least 32)	
Sized constant	As given	
<code>i op j</code> , op in <code>+ - * / % & ^ ^~ ^~</code>	$\max(L(i), L(j))$	Operands context-determined
<code>op i</code> , op in <code>+ - ~</code>	$L(i)$	
<code>i op j</code> , op in <code>=== !== == != > >= < <= ==? !=?</code>	1	Operands sized to $\max(L(i), L(j))$ - to each other, not to the LHS
<code>i op j</code> , op in <code>&& -> <-></code>	1	All operands self-determined
<code>op i</code> , op in <code>& ~& ~ ^ ~^ ^~ !</code>	1	Operand self-determined
<code>i op j</code> , op in <code>>> << ** >>> <<<</code>	$L(i)$	j is self-determined
<code>i ? j : k</code>	$\max(L(j), L(k))$	i is self-determined
<code>{i, ..., j}</code>	$L(i) + \dots + L(j)$	All operands self-determined
<code>{i{j}, ..., k{j}}</code>	$i \times L(j) + \dots + L(k)$	All self-determined

Expression	Bit length	Comment
<code>lhs = rhs</code>	-	RHS evaluated at $\max(L(lhs), L(rhs))$, then truncated or extended to $L(lhs)$

```

module sz;
  logic [7:0] a = 8'd200, b = 8'd100;
  logic [8:0] s9;
  logic [7:0] s8;
  logic signed [7:0] sa = -8'sd4;
  logic [3:0] u4 = 4'd3;
  initial begin
    s9 = a + b;           // context: 9 bits -> carry kept
    $display("s9 = a + b      = %0d", s9);
    s9 = {a + b};         // concatenation: self-determined 8 bits
    $display("s9 = {a + b}    = %0d", s9);
    s9 = (a + b) >> 1;     // context 9 bits: carry kept, then shifted
    $display("s9 = (a+b)>>1    = %0d", s9);
    s8 = (a + b) >> 1;     // context 8 bits: carry lost
    $display("s8 = (a+b)>>1    = %0d", s8);
    $display("$bits(a+b)      = %0d", $bits(a + b));
    $display("sa + u4 (unsigned)= %0d", sa + u4);
    $display("sa + $signed(u4) = %0d", sa + $signed(u4));
    $display("-1 > 8'd0       = %b", -1 > 8'd0);
    $display("sa >>> 1        = %0d", sa >>> 1);
    $display("sa[7:0] >>> 1    = %0d", sa[7:0] >>> 1);
  end
endmodule

```

Width and sign rules in action (Icarus 12)

```

s9 = a + b      = 300
s9 = {a + b}    = 44
s9 = (a+b)>>1    = 150
s8 = (a+b)>>1    = 22
$bits(a+b)      = 8
sa + u4 (unsigned)= 255
sa + $signed(u4) = -1
-1 > 8'd0       = 1
sa >>> 1        = -2
sa[7:0] >>> 1    = 126

```

HOW TO READ THE OUTPUT

`{a + b}` is a concatenation, so its operand is self-determined at 8 bits and the carry is gone before the 9-bit target sees it. `(a + b) >> 1` keeps the carry only because the 9-bit LHS widens the whole context. `sa + u4` is unsigned because one operand is, so `-4` becomes 252. `sa[7:0]` is a part-select and therefore unsigned even though it selects all of a signed variable, so `>>>` zero-fills.

Signedness rules (IEEE 1800-2017 11.8)

- The type of an expression depends **only on its operands**, never on the assignment target: `logic signed [15:0] y = a + b;` with unsigned `a, b` is an unsigned addition.
- Decimal numbers without a base (`-5, 42`) are signed; based numbers are unsigned unless written with `s` (`8'sh80`).
- Bit-selects, part-selects (even of the full width), concatenations, comparison and reduction results are **unsigned**.
- If **any** operand of a context-determined operator is unsigned, the whole expression is unsigned; signed operands are then zero-extended.

- `integer`, `int`, `shortint`, `longint`, `byte` are signed; `reg`, `logic`, `bit`, `time` and nets are unsigned unless declared `signed`.
- The evaluation steps are: (1) determine the expression size by Table 11-21, (2) determine its signedness, (3) propagate size and type down to every context-determined operand, (4) extend each such operand (sign-extend if the expression is signed, zero-extend otherwise), (5) evaluate.
- If any operand is `real`, the result is real and integer operands are converted first; `shortreal` mixed with `real` gives `real`.
- `$signed(x)` and `$unsigned(x)` (or `signed'(x)`, `unsigned'(x)`) change only the interpretation, not the bits.

PITFALL - THE SILENT UNSIGNED CONTAGION

`if (count - 1 < 0)` is never true when `count` is a `logic [7:0]`: the expression is unsigned. `for (int i = N - 1; i >= 0; i--)` is fine because `int` is signed, but the same loop with an unsigned `int` or a `logic` index never terminates. Lint tools flag both (Chapter 27).

Four-state truth tables

The bitwise operators work on each bit with the tables below; `z` on an input behaves as `x`. A controlling value wins regardless of the other input: `0` for AND, `1` for OR. XOR has no controlling value, so any unknown input gives `x`. The tables were printed by the simulator itself:

```
module tt;
  logic [3:0] v = 4'bzx10;          // v[0]=0 v[1]=1 v[2]=x v[3]=z
  string n = "01xz";
  initial begin
    $display("AND | 0 1 x z    OR | 0 1 x z    XOR | 0 1 x z");
    for (int i = 0; i < 4; i++)
      $display(" %s | %b %b %b %b    %s | %b %b %b %b    %s | %b %b %b %b",
        n.substr(i,i), v[i]&v[0], v[i]&v[1], v[i]&v[2], v[i]&v[3],
        n.substr(i,i), v[i]|v[0], v[i]|v[1], v[i]|v[2], v[i]|v[3],
        n.substr(i,i), v[i]^v[0], v[i]^v[1], v[i]^v[2], v[i]^v[3]);
    $display("NOT: ~0=%b ~1=%b ~x=%b ~z=%b", ~v[0], ~v[1], ~v[2], ~v[3]);
  end
endmodule
```

Generating the 4-state tables (Icarus 12)

```
AND | 0 1 x z    OR | 0 1 x z    XOR | 0 1 x z
0   | 0 0 0 0    0   | 0 1 x x    0   | 0 1 x x
1   | 0 1 x x    1   | 1 1 1 1    1   | 1 0 x x
x   | 0 x x x    x   | x 1 x x    x   | x x x x
z   | 0 x x x    z   | x 1 x x    z   | x x x x
NOT: ~0=1 ~1=0 ~x=x ~z=x
```

Rows are the left operand, columns the right operand

Operator	0	1	x	z
Logical <code>!a</code>	1	0	x	x
Reduction <code>&a, 1'b1</code>	0	1	x	x
<code>a && 1'b0</code>	0	0	0	0
<code>a 1'b1</code>	1	1	1	1
if <code>(a)</code> takes the	else	then	else	else

Other operators with an unknown operand. An `if` treats X and Z as false - the root of X-optimism (Chapter 11).

Equality and matching variants

Form	X/Z in left operand	X/Z in right operand	Result can be X?	Synthesizable
<code>a == b, a != b</code>	Unknown -> X	Unknown -> X	Yes	Yes
<code>a === b, a !== b</code>	Must match exactly	Must match exactly	No	No
<code>a ==? b, a !=? b</code>	Unknown -> X	Don't care	Yes	Yes (constant <code>b</code>)
<code>a inside {b, ...}</code>	Unknown -> X	Don't care (like <code>==?</code>)	Yes	Yes
case item match	Exact (as <code>===</code>)	Exact	No	Yes
casez item match	<code>z/?</code> don't care	<code>z/?</code> don't care	No	Yes
casex item match	<code>x/z</code> don't care	<code>x/z</code> don't care	No	Yes - but dangerous
case (...) inside	Exact	<code>x/z/?</code> don't care	No	Yes

```

module eqv;
  logic [3:0] a = 4'b10x1, b = 4'b10x1, c = 4'b1001;
  initial begin
    $display("a==b %b  a===b %b  a!==b %b", a == b, a === b, a !== b);
    $display("a==c %b  a===c %b", a == c, a === c);
    $display("c==?a %b  a==?c %b  c==?4'b1???1 %b", c ==? a, a ==? c, c ==? 4'b1???1);
  end
endmodule

```

Equality operators with an X bit (Icarus 12)

```

a==b  x   a===b 1   a!==b 0
a==c  x   a===c 0
c==?a 1   a==?c x   c==?4'b1???1 1

```

PITFALL - CASEX TREATS X IN THE SELECTOR AS A MATCH

With **casex**, an X on the case expression (an uninitialised register, for instance) matches the first item whose other bits agree, so simulation silently takes a branch that hardware might not. Prefer **case ... inside** or **casez** with `?` in the items, never **casex**.

Casting and conversion

Cast	Example	Effect
Size cast	<code>16'(u), 4'(u)</code>	Extend (by the signedness of <code>u</code>) or truncate to N bits
Sign cast	<code>signed'(u), unsigned'(s)</code>	Same bits, new interpretation; same as <code>\$signed/\$unsigned</code>
Type cast	<code>int'(r), op_e'(3), my_t'(v)</code>	Converts to the type; real to integer rounds to nearest, ties away from zero; no range check for enums
Const cast	<code>const'(x)</code>	Value treated as constant (used in assertions/let)
Dynamic cast	<code>\$cast(dst, src)</code>	Checked at run time: enum range, class down-cast; as a function returns 0 on failure and leaves <code>dst</code> unchanged; as a task a failure is a run-time error
Bit-stream cast	<code>pkt_t'(bit_array)</code>	Between types of equal total bits (packed or unpacked, including queues/dynamic arrays of bits)
Streaming	<code>{>>{obj_fields}}</code>	Pack/unpack in chosen order (Chapter 14)
Real conversions	<code>\$rtoi(3.9), \$itor(3)</code>	<code>\$rtoi</code> truncates toward zero; contrast <code>int'()</code> which rounds

Cast	Example	Effect
Real bit patterns	<code>\$realtobits, \$bitstoreal, \$shortrealtobits, \$bitstoshortreal</code>	Move IEEE-754 bits through ports and DPI

```

module q_cast;
  typedef enum logic [1:0] {A = 0, B = 1, C = 2} abc_e;
  abc_e e;
  logic [7:0] u = 8'hF0;
  initial begin
    $display("signed'(u)=%0d 16'(u)=%h 4'(u)=%h int'(2.6)=%0d", signed'(u), 16'(u),
            4'(u), int'(2.6));
    if ($cast(e, 2)) $display("$cast(e,2) ok -> %s", e.name());
    if (!$cast(e, 3)) $display("$cast(e,3) failed, e still %s", e.name());
    e = abc_e'(3); // static cast: no check
    $display("abc_e'(3) -> value %0d, name '%s'", e, e.name());
    $finish;
  end
endmodule

```

Static and dynamic casts (Verilator 5.020)

```

signed'(u)=-16 16'(u)=00f0 4'(u)=0 int'(2.6)=3
$cast(e,2) ok -> C
$cast(e,3) failed, e still C
abc_e'(3) -> value 3, name ''

```

System tasks and functions by group

Entries marked (SV) are SystemVerilog additions. All of these are simulation constructs except the constant functions (`$clog2`, `$bits`, array queries) which synthesis evaluates at elaboration time.

Group	Tasks / functions	Notes
Display	<code>\$display \$displayb \$displayo \$displayh, \$write (+ b/o/h), \$strobe (+ b/o/h), \$monitor (+ b/o/h), \$monitoron \$monitoroff</code>	<code>\$display</code> prints now (Active region), <code>\$strobe</code> at the end of the time step (Postponed), <code>\$monitor</code> whenever an argument changes; only one <code>\$monitor</code> active at a time
String formatting	<code>\$sformat \$swrite (+ b/o/h), \$sformatf</code>	(SV) <code>\$sformatf</code> returns a string; <code>\$psprintf</code> is a non-standard alias
Severity	<code>\$fatal \$error \$warning \$info</code>	(SV) Print with location; <code>\$fatal(n, ...)</code> ends simulation with <code>\$finish(n)</code>
File I/O	<code>\$fopen \$fclose \$fdisplay \$fwrite \$fstrobe \$fmonitor \$fgetc \$ungetc \$fgets \$fscanf \$sscanf \$fread \$ftell \$fseek \$rewind \$fflush \$feof \$ferror</code>	<code>\$fopen(name)</code> returns a multichannel descriptor; <code>\$fopen(name, "r")</code> a file descriptor (modes as C <code>fopen</code>)
Memory load/dump	<code>\$readmemb \$readmemh \$writememb \$writememh</code>	Optional start/end addresses; <code>@addr</code> in the file
Waveform dump	<code>\$dumpfile \$dumpvars \$dumpon \$dumpoff \$dumpall \$dumplimit \$dumpflush \$dumpports</code>	VCD / extended VCD (Chapter 10)
Simulation control	<code>\$finish \$stop \$exit</code>	<code>\$finish(0 1 2)</code> sets the statistics printed; <code>\$exit</code> ends a program block
Time	<code>\$time \$stime \$realttime \$prnttimescale \$timeformat</code>	Scaled to the calling module's time unit; <code>\$time</code> is rounded 64-bit, <code>\$stime</code> 32-bit, <code>\$realttime</code> real

Group	Tasks / functions	Notes
Conversion	<code>\$signed \$unsigned \$rtoi \$itor</code> <code>\$realtobits \$bitstoreal</code> <code>\$shortrealtobits \$bitstoshortreal</code> <code>\$cast</code>	See casting table
Integer math	<code>\$clog2</code>	Ceiling of log2; <code>\$clog2(1) = 0</code> ; constant function
Real math	<code>\$ln \$log10 \$exp \$sqrt \$pow \$floor</code> <code>\$ceil \$sin \$cos \$tan \$asin \$acos \$atan</code> <code>\$atan2 \$hypot \$sinh \$cosh \$tanh \$asinh</code> <code>\$acosh \$atanh</code>	Real arguments and results, same as the C library
Data query	<code>\$bits \$typename \$isunbounded</code>	<code>\$bits</code> works on any type or expression; (SV) <code>\$typename</code> returns a string
Array query	<code>\$dimensions \$unpacked_dimensions \$left</code> <code>\$right \$low \$high \$increment \$size</code>	Optional dimension number (1 = leftmost unpacked); <code>\$increment</code> is 1 if <code>\$left >= \$right</code> , else -1
Bit vector	<code>\$countbits \$countones \$onehot \$onehot0</code> <code>\$isunknown</code>	<code>\$countbits(v, '1, 'x)</code> counts any listed values; used heavily in SVA
Sampled values (SVA)	<code>\$sampled \$rose \$fell \$stable \$changed</code> <code>\$past, \$past_gclk \$rose_gclk</code> <code>\$future_gclk ...</code>	Use Preponed-region values; <code>\$past(e, n, gate, clk)</code>
Assertion control	<code>\$asserton \$assertoff \$assertkill</code> <code>\$assertcontrol \$assertpasson</code> <code>\$assertpassoff \$assertfailon</code> <code>\$assertfailoff \$assertnonvacuouson</code> <code>\$assertvacuousoff</code>	Scope- and level-selectable; turn checks off during reset or error injection
Coverage	<code>\$coverage_control \$coverage_get_max</code> <code>\$coverage_get \$coverage_merge</code> <code>\$coverage_save \$get_coverage</code> <code>\$set_coverage_db_name</code> <code>\$load_coverage_db</code>	Tool support varies; covergroup methods (<code>sample</code> , <code>get_coverage</code> , <code>get_inst_coverage</code> , <code>start</code> , <code>stop</code>) are more portable
Random	<code>\$random \$urandom \$urandom_range,</code> <code>\$dist_uniform \$dist_normal</code> <code>\$dist_exponential \$dist_poisson</code> <code>\$dist_chi_square \$dist_t \$dist_erlang</code>	<code>\$random</code> is signed 32-bit with a seed variable; (SV) <code>\$urandom</code> is unsigned and thread-stable
Plusargs	<code>\$test\$plusargs \$value\$plusargs</code>	Read <code>+NAME</code> and <code>+NAME=value</code> from the command line
Misc.	<code>\$system \$stacktrace</code> , PLA tasks (<code>\$async\$and\$array ...</code>), stochastic queues (<code>\$q_initialize ...</code>)	<code>\$system</code> runs a shell command (standard in IEEE 1800); <code>\$stacktrace</code> was added in 1800-2023; the PLA and queue tasks are legacy

```

module st;
  logic [7:0] mem [1:16];
  logic [3:0][7:0] pk;
  logic [7:0] v = 8'b0010_1100;
  logic [3:0] u = 4'b01x0;
  real r;
  initial begin
    $display("$size=%0d $left=%0d $right=%0d $low=%0d $high=%0d $dimensions=%0d",
      $size(mem), $left(mem), $right(mem), $low(mem), $high(mem), $dimensions(mem));
    $display("$bits(pk)=%0d $size(pk,1)=%0d $size(pk,2)=%0d $increment(mem)=%0d",
      $bits(pk), $size(pk,1), $size(pk,2), $increment(mem));
    $display("$countones=%0d $onehot=%b $onehot0=%b $isunknown(u)=%b $clog2(17)=%0d",
      $countones(v), $onehot(v), $onehot0(8'h00), $isunknown(u), $clog2(17));
    r = $sqrt(2.0);
    $display("$sqrt(2)=%f $ln(10)=%f $pow(2,10)=%0.1f $floor(-1.5)=%0.1f", r, $ln(10.0),

```

```

        $pow(2.0, 10.0), $floor(-1.5));
    $display("$rtoi(3.9)=%0d $itor(3)=%0.1f $realtobits(1.0)=%h", $rtoi(3.9), $itor(3),
        $realtobits(1.0));
    $display("%h=%h %d=%d %0d=%0d %b=%b %o=%o %e=%e %s=%s %c=%c", v, v, v, v, v,
        1234.5, "ok", 8'h41);
end
endmodule

```

Array-query, bit-vector, math, conversion and format functions (Icarus 12)

```

$size=16 $left=1 $right=16 $low=1 $high=16 $dimensions=2
$bits(pk)=32 $size(pk,1)=4 $size(pk,2)=8 $increment(mem)=-1
$countones=3 $onehot=0 $onehot0=1 $isunknown(u)=1 $clog2(17)=5
$sqrt(2)=1.414214 $ln(10)=2.302585 $pow(2,10)=1024.0 $floor(-1.5)=-2.0
$rtoi(3.9)=3 $itor(3)=3.0 $realtobits(1.0)=3ff0000000000000
%h=2c %d= 44 %0d=44 %b=00101100 %o=054 %e=1.234500e+03 %s=ok %c=A

```

```

module q_fio;
    integer fd, n, code, addr, data;
    string line;
    initial begin
        fd = $fopen("vec.txt", "w");
        for (int i = 0; i < 3; i++) $fdisplay(fd, "%h %h", 16 * i, i * i + 1);
        $fclose(fd);
        fd = $fopen("vec.txt", "r");
        while (!$feof(fd)) begin
            code = $fgets(line, fd);           // returns chars read, 0 at EOF
            if (code && $sscanf(line, "%h %h", addr, data) == 2)
                $display("addr=%0d data=%0d", addr, data);
        end
        $fclose(fd);
        if (!$value$plusargs("SEED=%d", n)) n = 1; // run with +SEED=42
        $display("seed=%0d verbose=%0d", n, $test$plusargs("VERBOSE"));
        $finish;
    end
endmodule

```

File I/O and plusargs (Verilator 5.020, run with **+SEED=42 +VERBOSE**)

```

addr=0 data=1
addr=16 data=2
addr=32 data=5
seed=42 verbose=1

```

Format specifiers and escapes

Spec	Meaning	Spec	Meaning
%h %H %x %X	Hexadecimal	%C	ASCII character
%d %D	Decimal (padded to max width)	%S	String
%o %O	Octal	%t	Time, per <code>\$timeformat</code>
%b %B	Binary	%m	Hierarchical name of the scope (no argument)
%e %f %g	Real: exponent, fixed, shorter	%v	Net signal strength
%0d, %0h	Minimum width (no padding)	%l	Library binding (config)
%5d, %-8s	Field width, left-justify	%p	(SV) Assignment-pattern print of any aggregate
%u %z	Unformatted 2-state / 4-state binary (files)	%%	Percent sign

Escapes in strings: `\n` newline, `\t` tab, `\\` backslash, `\"` quote, `\ddd` octal character; SystemVerilog adds `\v`, `\f`, `\a` and `\xHH`. `%h` and `%d` print `x/z` when all bits of a digit are unknown and `x/Z` when only some are.

Array methods (IEEE 1800-2017 7.12)

Kind	Methods	Returns / notes
Locator	<code>find</code> <code>find_index</code> <code>find_first</code> <code>find_first_index</code> <code>find_last</code> <code>find_last_index</code> (need <code>with</code>), <code>min</code> <code>max</code> <code>unique</code> <code>unique_index</code>	Always a queue (possibly empty); <code>item</code> and <code>item.index</code> usable in the <code>with</code> clause
Ordering	<code>reverse</code> <code>sort</code> <code>rsort</code> <code>shuffle</code>	In place; <code>sort/rsort</code> accept <code>with (key)</code> ; not for associative arrays
Reduction	<code>sum</code> <code>product</code> <code>and</code> <code>or</code> <code>xor</code>	Result has the element type - <code>bit</code> arrays sum to 1 bit unless you write <code>sum()</code> with <code>(int'(item))</code>
Queue	<code>size</code> <code>insert</code> <code>delete</code> <code>push_front</code> <code>push_back</code> <code>pop_front</code> <code>pop_back</code>	<code>delete()</code> with no index empties it; <code>q[\$]</code> is the last element
Dynamic array	<code>new[n]</code> , <code>new[n](old)</code> , <code>size</code> <code>delete</code>	<code>new[n](old)</code> resizes and copies
Associative array	<code>num</code> <code>size</code> <code>delete</code> <code>exists</code> <code>first</code> <code>last</code> <code>next</code> <code>prev</code>	Traversal functions take a <code>ref</code> key and return 0/1 (or -1 if the key had to be truncated)

String methods (IEEE 1800-2017 6.16)

Method	Effect	Method	Effect
<code>len()</code>	Number of characters	<code>substr(i, j)</code>	Characters i..j (inclusive)
<code>putc(i, c)</code>	Replace character i	<code>getc(i)</code>	Character i as byte
<code>toupper()</code> <code>tolower()</code>	Case-converted copy	<code>compare(s)</code>	Like C <code>strcmp</code> (<0, 0, >0)
<code>icompare(s)</code>	Case-insensitive compare	<code>atoi()</code> <code>atohex()</code> <code>atooct()</code> <code>atobin()</code>	Parse leading digits to integer
<code>atoreal()</code>	Parse to real	<code>itoa(i)</code> <code>hextoa(i)</code> <code>octtoa(i)</code> <code>bintoa(i)</code>	Format integer into the string
<code>realtoa(r)</code>	Format a real	Operators	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code> , <code>{s1, s2}</code> , <code>{n{s}}</code> , <code>s[i]</code>

```

module am;
  int q[$] = '{5, 3, 9, 3, 7};
  int r[$];
  int aa[string];
  string s = "Hello,SoC";
  function automatic string L(int a[$]);
    string t = "";
    foreach (a[i]) t = {t, $sformatf(" %0d", a[i])};
    return t;
  endfunction
  initial begin
    r = q.find(x) with (x > 4);           $display("find >4      : %s", L(r));
    r = q.find_first_index(x) with (x == 3); $display("first idx 3 : %s", L(r));
    r = q.unique();                       $display("unique       : %s", L(r));
    r = q.min();                           $display("min         : %s", L(r));
    $display("sum=%0d product=%0d", q.sum(), q.product());
    $display("sum with (x>4)=%0d", q.sum(x) with (int'(x > 4)));
    q.sort();                             $display("sort        : %s", L(q));
  end

```

```

q.rsort();                $display("rsort      : %s", L(q));
q.reverse();              $display("reverse   : %s", L(q));
q.push_front(1); q.push_back(10); $display("push      : %s size=%0d", L(q), q.size());
$display("pop_front=%0d pop_back=%0d", q.pop_front(), q.pop_back());
q.insert(1, 42); q.delete(0);    $display("insert/del : %s", L(q));
aa["b"] = 2; aa["a"] = 1; aa["c"] = 3;
$display("num=%0d exists(\"a\")=%0d", aa.num(), aa.exists("a"));
foreach (aa[k]) $write("%s=%0d ", k, aa[k]);
$display("");
$display("len=%0d upper=%s lower=%s substr(6,8)=%s", s.len(), s.toupper(), s.tolower(),
        s.substr(6, 8));
$display("getc(0)=%s compare=%0d icompare=%0d", s.getc(0), s.compare("Hello"),
        s.icompare("HELLO,SOC"));
s = "123"; $display("atoi=%0d atohex=%0d", s.atoi(), s.atohex());
s.itoa(-77); $display("itoa=%s", s); s.hextoa(255); $display("hextoa=%s", s);
$finish;
end
endmodule

```

Queue, associative-array and string methods (Verilator 5.020)

```

find >4      : 5 9 7
first idx 3  : 1
unique       : 5 3 9 7
min          : 3
sum=27 product=2835
sum with (x>4)=3
sort         : 3 3 5 7 9
rsort        : 9 7 5 3 3
reverse      : 3 3 5 7 9
push         : 1 3 3 5 7 9 10 size=7
pop_front=1 pop_back=10
insert/del   : 42 3 5 7 9
num=3 exists("a")=1
a=1 b=2 c=3
len=9 upper=HELLO,SOC lower=hello,soc substr(6,8)=SoC
getc(0)=H compare=44 icompare=0
atoi=123 atohex=291
itoa=-77
hextoa=ff

```

`compare` is only specified by sign; Verilator returns the C `strcmp` difference (44 here), another simulator may return 1. Note that `q.sum(x) with (int'(x > 4))` counts elements: the `with` expression is summed instead of the elements.

Appendix C. 100 Verilog/SystemVerilog Interview Questions with Answers

These are the language questions asked in RTL design, design-verification and DFT interviews, grouped from screening level to on-site puzzles. Answer each one out loud before reading the answer. Answers are deliberately short; the chapter in brackets has the full story. Code with an output card was run with Icarus Verilog 12 or Verilator 5.020 as noted; constraint and assertion code was elaborated with slang and needs a commercial simulator to execute.

Verilog basics

Q1. What is the difference between a `wire` and a `reg`?

A `wire` is a net: it has no storage and continuously takes the resolved value of its drivers (continuous assignments, ports, primitives). A `reg` is a variable: it holds the last value assigned by procedural code. `reg` does **not** mean flip-flop - a `reg` written in a combinational `always` block is plain logic. SystemVerilog's `logic` is a variable type that can also be driven by one continuous assignment, which removes most of the confusion (Chapter 3).

Q2. What is the difference between `logic` and `wire` in SystemVerilog?

`logic` is a 4-state **data type**; `wire` is a **net kind**. A plain `logic x;` declares a variable, which allows either procedural writes or a single continuous driver. A net (`wire logic x;` or `wire x;`) allows multiple drivers resolved by strength - needed for tri-state buses and `inout` ports. Use `logic` everywhere except multi-driver nets.

Q3. What are the four values of Verilog logic and what does each mean?

`0` and `1` are logic levels; `x` is unknown (uninitialised, conflicting drivers, or undefined result); `z` is high impedance (undriven net). Variables start at `x`; nets with no driver are `z`. 2-state types (`bit`, `int`) hold only 0/1 and start at 0.

Q4. What is the difference between `initial` and `always`?

Both start at time 0 as independent processes. `initial` runs once; `always` loops forever, so it must contain a timing control or it hangs the simulator. Synthesis ignores `initial` for ASICs (FPGA flows may use it for power-up values).

Q5. How does the `timescale` directive work, and what is the precision for?

`timescale 1ns/10ps` (with its leading grave accent) sets the time **unit** for delays in the following modules (`#5` = 5 ns) and the **precision** to which delays are rounded. The simulator runs at the finest precision of all modules, so one `1fs` precision slows everything. SystemVerilog's `timeunit` and `timeprecision` inside a module avoid the order dependence of the directive (Chapter 8).

Q6. What does the directive `default_nettype none` do and why use it?

It turns off implicit net declaration, so a misspelled identifier in a port connection or continuous assignment becomes a compile error instead of a silently created 1-bit wire. Restore `wire` at the end of the file so third-party code that relies on implicit nets still compiles.

Q7. What are `parameter`, `localparam`, `defparam` and `specparam`?

`parameter`: overridable per instance with `#(...)`. `localparam`: derived constant, not overridable. `defparam`: overrides a parameter by hierarchical path from anywhere - deprecated, avoid. `specparam`: timing value inside a `specify` block, can be back-annotated from SDF (Chapters 7 and 9).

Q8. What is the difference between a task and a function?

A function executes in zero time, cannot contain `#`, `@` or `wait`, and returns a value (or `void`). A task may consume time, returns results through `output/inout/ref` arguments and can call tasks. Both are static by

default in modules; declare them `automatic` for recursion or concurrent calls (Chapter 6).

Q9. What is the difference between `==` and `===`?

`==` returns `x` if any compared bit is `x` or `z` and the result is otherwise ambiguous; `===` compares all four values literally and always returns 0 or 1. `===` is for testbenches (e.g. `if (bus === 'z)`); synthesis either rejects it or treats it as `==`.

Q10. What is a hierarchical reference, and where is it acceptable?

A dotted path such as `tb.dut.u_fifo.count` that reads or writes a signal anywhere in the design. It is fine in testbenches, `bind` files and assertions, but not in synthesizable RTL (most synthesis tools reject cross-module references), and it couples code to the hierarchy.

Data types and sizing

Q11. What is the difference between packed and unpacked arrays?

Packed dimensions (before the name, `logic [3:0][7:0] w`) form one contiguous vector that can be sliced, used in arithmetic and assigned as a whole. Unpacked dimensions (after the name, `logic [7:0] m [256]`) are an array of separate elements - memories, register files. Only packed types can be packed struct members (Chapter 13).

Q12. Compare fixed, dynamic, associative arrays and queues.

Fixed: size set at elaboration, synthesizable. Dynamic `[]`: size set at run time with `new[n]`, contiguous. Associative `[key_t]`: sparse, indexed by any type, only written entries exist - ideal for a memory model. Queue `[$]`: ordered, grows at both ends with `push_*/pop_*` - ideal for scoreboards. Only fixed arrays are synthesizable.

Q13. What is `$bits(8'd1 + 8'd1 + 32'd0)`?

32. The expression width is the maximum operand width (Table 11-21), so the three operands are all evaluated at 32 bits.

Q14. `a` and `b` are `logic [7:0]` with values 200 and 100. What is `sum` for `logic [8:0] sum = a + b`; and for `sum = {a + b}`?

300 and 44. In the first the 9-bit LHS makes the addition 9 bits wide and keeps the carry. In the second the concatenation makes `a + b` self-determined (8 bits), so the carry is lost before assignment (Appendix B shows the simulator output).

Q15. Why can `-1 > 8'd0` be true?

Mixing a signed operand (`-1`, a 32-bit signed decimal) with an unsigned one makes the whole comparison unsigned. `-1` becomes `32'hFFFF_FFFF`, which is larger than 0. Keep both operands signed (`8'sd0`) or cast.

Q16. Is a part-select of a signed variable signed?

No. Bit-selects and part-selects - even `s[7:0]` covering the whole of a signed 8-bit `s` - are unsigned, as are concatenations. Wrap them in `$signed()` when the sign matters (for example before `>>>`).

Q17. What is the difference between `int` and `integer`?

`integer` is Verilog's 4-state signed 32-bit variable; `int` is SystemVerilog's 2-state signed 32-bit type. `int` is faster in simulation and starts at 0, but hides X: an X assigned to an `int` becomes 0.

Q18. Why are 2-state types risky in RTL?

X is how simulation tells you something is uninitialised or multiply driven. A `bit` register without reset simply reads 0, so reset bugs and X-propagation problems disappear in RTL simulation and reappear in gate-level simulation or silicon. Use 4-state `logic` for design; 2-state types are for testbench data.

Q19. What are `'0`, `'1`, `'x` and `'z`?

SystemVerilog fill literals: every bit of the context width is set to that value, whatever the width. `'1` is all ones - unlike `-1` or `1`, it never depends on sign extension or 32-bit truncation, so it is the safe way to write a parameterized all-ones constant.

Q20. What is a packed union, and what is a tagged union?

A packed union overlays members of **equal** width on the same bits, giving several views of one vector (e.g. raw word versus decoded fields); it is synthesizable. A tagged union stores which member was last written and type-checks reads (**matches** in **case**); it is a verification construct with patchy tool support.

Blocking, non-blocking and scheduling

Q21. Explain blocking versus non-blocking assignment. What does this print?

`=` updates the variable immediately; `<=` evaluates the RHS now and schedules the update for the NBA region, after all Active events of the time step. So the blocking pair copies (both become 2) while the non-blocking pair swaps. `$display` runs in the Active region and still sees the old `c`, `d`; `$strobe` runs at the end of the time step and sees the swap.

```
module q_nba;
  logic [3:0] a = 4'd1, b = 4'd2, c = 4'd1, d = 4'd2;
  initial begin
    a = b; b = a;           // blocking: both end up 2
    c <= d; d <= c;         // non-blocking: true swap
    $display("t=0 display: a=%0d b=%0d c=%0d d=%0d", a, b, c, d);
    $strobe ("t=0 strobe : a=%0d b=%0d c=%0d d=%0d", a, b, c, d);
  end
endmodule
```

```
t=0 display: a=2 b=2 c=1 d=2
t=0 strobe : a=2 b=2 c=2 d=1
```

Icarus 12

Q22. List the SystemVerilog scheduling regions in order.

For each time slot: **Preponed** (assertion sampling), **Active** (blocking assignments, continuous assignments, `$display`), **Inactive** (`#0` events), **NBA** (non-blocking updates), **Observed** (concurrent assertion evaluation), **Reactive** (program blocks, assertion action blocks), **Re-Inactive**, **Re-NBA** (clocking-block drives, NBAs from programs), **Postponed** (`$strobe`, `$monitor`, read-only). Active through NBA iterate until empty, then the reactive set, then back if new active events appear (Chapter 5).

Q23. What are the coding rules that avoid simulation races in RTL?

(1) Sequential logic: non-blocking. (2) Combinational `always`: blocking. (3) Never mix `=` and `<=` in one block. (4) Never assign a variable from more than one `always` block. (5) Use `$strobe` or check after the edge to display NBA results. (6) No `#0`. These are Cummings' guidelines (SNUG 2000) and `always_ff/always_comb` enforce several of them.

Q24. What is a race condition? Give an example.

The result depends on which of two processes scheduled in the same region runs first, which the LRM leaves undefined. Example: `always @(posedge clk) a = b;` and `always @(posedge clk) b = a;` - the pair may swap, or both may get the same value, depending on the simulator. Using `<=` in both removes the race.

Q25. Why is `#0` considered harmful?

It moves a process to the Inactive region, which fixes one ordering and creates another; code that needs it usually has a race. In testbenches, clocking blocks and program blocks provide a defined ordering instead.

Q26. What is the difference between `$display`, `$write`, `$strobe` and `$monitor`?

`$display` prints immediately with a newline, `$write` without. `$strobe` prints at the end of the time step (Postponed region), so it shows NBA results. `$monitor` prints whenever any of its arguments changes; only one is active at a time.

Q27. How does `always @(a or b)` differ from `always @*` and `always_comb`?

A hand-written list misses signals added later: simulation then behaves as a latch while synthesis builds combinational logic - a sim/synth mismatch. `@*` infers the list from the block. `always_comb` also includes signals read inside called functions, runs once at time 0 (so outputs are valid even if inputs never change), forbids timing controls and forbids other processes writing its outputs.

Q28. What does intra-assignment delay `a = #5 b;` do compared with `#5 a = b;`?

`a = #5 b;` samples `b` now and assigns it 5 units later (the process blocks). `#5 a = b;` waits 5 units, then samples and assigns. `a <= #5 b;` samples now and schedules the update 5 units later without blocking - it models transport delay. None are synthesizable; synthesis ignores delays.

Q29. When should you use static versus automatic variables? What does this print?

Variables in module-level functions and tasks are static by default: one copy, initialised once at time 0 and shared by every call and every concurrent caller. `automatic` gives each call its own storage, required for recursion, re-entrant tasks and fork loops. Classes' methods are automatic by default. The LRM requires the explicit `static` keyword when a variable in a static function has an initialiser.

```
module q_static;
  function int cnt_s();                // static by default in a module
    static int n = 0;                 // initialised ONCE, at time 0
    n++;
    return n;
  endfunction
  function automatic int cnt_a();      // fresh storage per call
    int n = 0;
    n++;
    return n;
  endfunction
  initial begin
    repeat (3) $write("%0d ", cnt_s());
    repeat (3) $write("%0d ", cnt_a());
    $display("");
  end
endmodule
```

```
1 2 3 1 1 1
```

Icarus 12

Synthesis

Q30. How is a latch inferred unintentionally, and how do you prevent it?

A combinational block that does not assign an output on every path (an `if` without `else`, a `case` without `default`) must remember the old value, so synthesis builds a latch. Prevent it with default assignments at the top of the block, complete `case/if`, and `always_comb` so tools warn.

Q31. Which constructs are not synthesizable?

Delays (`#`, ignored), `initial` (ASIC), `force/release`, `fork/join`, `wait` on arbitrary expressions, `==/!=`, real arithmetic, dynamic/associative arrays, queues, classes, strings, file I/O and most system tasks. Loops are synthesizable only with a static bound (they are unrolled).

Q32. What are `full_case` and `parallel_case`, and what replaced them?

Synthesis pragmas: `full_case` says unlisted values never occur (outputs may be don't-care), `parallel_case` says items never overlap (no priority logic). Because simulation ignores comments, they cause sim/synth mismatches (Cummings: 'the evil twins'). SystemVerilog's `unique` and `priority` state the same intent and make simulation check it.

Q33. Explain `unique`, `unique0` and `priority`.

unique case/unique if: exactly one item must match - a run-time violation warning if none or more than one does; synthesis may build parallel logic and treat unmatched values as don't-care. **unique0:** at most one may match (no warning for none). **priority:** first match wins, but at least one must match. All checks happen in simulation after the inputs settle in the time step.

Q34. How does synthesis treat a **for** loop?

It unrolls it at elaboration: each iteration becomes a copy of the hardware. The bound must be a constant. A loop is therefore a way to describe replicated or reduction logic, never a sequence in time.

Q35. What is a simulation/synthesis mismatch? Name four causes.

RTL simulation behaves differently from the synthesized netlist. Causes: incomplete sensitivity lists; **full_case/parallel_case** pragmas; X-optimism in **if/case** (X takes the else branch in RTL but propagates in gates); **initial** values; delays in RTL; blocking assignments in sequential blocks read before written; **casex** with X inputs. Gate-level simulation and equivalence checking catch them (Chapter 11).

Q36. Synchronous versus asynchronous reset - how is each coded?

Synchronous: `always_ff @(posedge clk) if (!rst_n) q <= '0; else ...` - reset is just another data input. Asynchronous: `always_ff @(posedge clk or negedge rst_n) if (!rst_n) ...` - the reset must be the first **if** and the only other event. Asynchronous assertion needs synchronised deassertion (a reset synchronizer) to avoid recovery/removal violations.

Q37. What is X-optimism and X-pessimism?

X-optimism: RTL treats X as a known value, e.g. **if (x)** takes the **else** branch, hiding an X that hardware would propagate. X-pessimism: gate-level simulation produces X where real hardware would be deterministic, e.g. a mux with equal inputs and an X select. Both waste debug time; reset every control flop and use X-propagation modes or assertions (`$isunknown`).

Q38. Why must a combinational block not read a variable before writing it?

Reading first implies the old value is needed - a latch or a combinational loop in hardware, and simulation that depends on execution order. Assign defaults first, then compute; `always_comb` plus lint catch the pattern.

SystemVerilog design features

Q39. What are interfaces and modports for?

An interface bundles the signals (and optionally tasks, functions, assertions and clocking blocks) of a protocol so a port list shrinks to one line. A modport gives each side its direction view (`mst`, `slv`), which is checked at compile time. Interfaces synthesize; the signals are flattened into ports (Chapter 16).

Q40. What is a virtual interface and why is it needed?

A class variable that holds a handle to an interface **instance**. Classes are created at run time and cannot have ports, so a virtual interface is the bridge from dynamic testbench objects to static design signals. UVM passes it through `uvm_config_db`.

Q41. Package versus the **include** directive - which and why?

A package is a named scope compiled once; importing it gives every user the **same** types, so type checking across modules works. **include** pastes text, so each includer gets its own copy of a typedef - two identical declarations are different types in different scopes. Use packages for shared types, **include** only for macros and class files inside a package.

Q42. What is the compilation-unit scope `$unit`?

Declarations outside any module, interface or package go into `$unit`. Whether each file is its own unit or all files share one is tool-dependent, so code that relies on it breaks between tools. Put declarations in packages.

Q43. Wildcard import versus explicit import?

`import p::*;` makes names visible only when used and not otherwise declared locally; a local declaration silently wins. `import p::name;` imports that name immediately and conflicts with a local declaration. Explicit imports and `p::name` references are clearer in large SoCs.

Q44. How do you reverse the bits of a vector in one line?

With the streaming operator: `rev = {<<{x}};` `{<<8{w}}` reverses bytes (endianness swap). Streaming is synthesizable when widths are static. The coding-puzzles section shows it running.

Q45. What is `let` and how does it differ from a macro?

`let name(args) = expr;` (1800-2009) declares a scoped, typed expression template that follows normal name resolution; a macro is textual substitution with no scope. `let` is ideal for reusable expressions in assertions and RTL.

Q46. How are type parameters used?

`module fifo #(parameter type T = logic [7:0])` lets one module store any type, including structs. Combined with `$bits(T)`, it replaces width parameters for bus payloads (Chapter 17).

Q47. What does the `inside` operator do in RTL?

`if (op inside {ADD, SUB, [8:11]})` tests set membership, with ranges and array elements; X/Z in the set values are don't-cares. `case (x) inside` gives range items in a `case`. Both are synthesizable.

Object-oriented programming

Q48. What is the difference between a class handle and an object?

The handle is a typed reference variable (initially `null`); `new()` allocates an object and returns a handle to it. Assigning a handle copies the reference, not the object. Objects are freed automatically when no handle refers to them.

Q49. Shallow copy versus deep copy - what does this print, and why is `kind()` 'animal'?

`new a` copies the properties of `a`; a nested object handle is copied as a handle, so both copies share the inner object - that is a shallow copy. A deep copy must also construct new nested objects. (`new this` inside a method is the usual idiom; Verilator 5.020 rejects it, so the method below copies fields explicitly.) `speak()` is virtual, so the base handle dispatches to `dog::speak`; `kind()` is not virtual, so the handle type decides.

```
module q_oop;
  class inner; int v = 1; endclass
  class outer;
    int    x = 5;
    inner  h = new();
    function outer deep_copy();
      outer c = new();
      c.x = x;
      c.h = new h;                // copy the nested object too
      return c;
    endfunction
  endclass

  class animal;
    virtual function string speak(); return "..."; endfunction
    function string kind(); return "animal"; endfunction    // NOT virtual
  endclass
  class dog extends animal;
    virtual function string speak(); return "woof"; endfunction
    function string kind(); return "dog"; endfunction
  endclass

  initial begin
    outer a = new(), s, d;
```

```

animal an;
dog    dg = new();
s = new a;                // shallow: s.h and a.h are the same object
d = a.deep_copy();
a.x = 9; a.h.v = 7;
$display("shallow: x=%0d h.v=%0d    deep: x=%0d h.v=%0d", s.x, s.h.v, d.x, d.h.v);
an = dg;                  // base handle, derived object
$display("speak=%s kind=%s", an.speak(), an.kind());
$finish;
end
endmodule

```

```

shallow: x=5 h.v=7    deep: x=5 h.v=1
speak=woof kind=animal

```

Verilator 5.020

Q50. What does `virtual` mean for a method, and can a constructor be virtual?

A virtual method is dispatched on the **object** type at run time; once virtual, it stays virtual in all derived classes and overrides must keep the same signature. Constructors cannot be virtual; factories (as in UVM) provide the equivalent of a virtual constructor.

Q51. What is an abstract class and a pure virtual method?

`virtual class` cannot be instantiated. A `pure virtual` method has no body and must be implemented by every non-abstract derived class. Used for base transactions and components that define an API.

Q52. What do `local` and `protected` mean?

`local` members are visible only inside the class itself; `protected` members also in derived classes. Everything else is public. `const` properties are read-only after construction.

Q53. What is `$cast` used for with classes?

Down-casting: assigning a base-class handle to a derived-class handle. `if (!$cast(pkt, base_h))` checks at run time that the object really is a `pkt` (or derived) and returns 0 otherwise. Up-casting needs no cast.

Q54. How do static members behave, including in parameterized classes?

A static property has one copy per class, shared by all objects; a static method can only touch static members. Each **specialization** of a parameterized class is a separate class with its own statics - UVM's factory registration relies on this.

Q55. What is an interface class?

(1800-2012) A class containing only pure virtual methods, types and parameters. A class `implements` any number of them, giving multiple inheritance of API without inheritance of implementation, as in Java.

Q56. What are `this` and `super`, and when is `typedef class` needed?

`this` refers to the current object (to disambiguate a property from an argument of the same name); `super` to the parent class's members, e.g. `super.new()` must be the first statement of a derived constructor if the parent's constructor takes arguments. `typedef class c;` forward-declares a class used before its definition.

Constrained randomization

Q57. `rand` versus `randc`?

`rand` values are independent draws, so repeats are possible. `randc` cycles through every value of its range in random order before repeating; it is meant for small ranges (tools limit its width) and is solved before `rand` variables.

Q58. What is the difference between `:=` and `:/` in `dist`?

`[1:3] := 30` gives weight 30 to **each** value (total 90); `[1:3] :/ 30` shares 30 across the range (10 each). `dist` is a soft preference shape, not a hard guarantee in any single draw.

Q59. What does `solve a before b` do?

It changes the probability distribution, not the solution space: `a` is picked uniformly among its legal values first. Without it, the solver picks uniformly over all legal (`a`, `b`) pairs, so for `is_long -> len > 64` the rare `is_long == 0` combinations dominate. It cannot make an unsolvable set solvable and does not apply to `randc`.

Q60. How do you write constraints for a packet class? Show common idioms.

The class below uses a weighted `dist`, implications, `solve ... before`, an array size constraint, `foreach`, a `soft` default that an inline `with` constraint overrides, and the run-time switches `constraint_mode` and `rand_mode`. (Elaborated with slang; Verilator 5.020 ignores constraints, so run it on a commercial simulator.)

```
class packet;
  rand bit [3:0]  kind;
  rand bit [7:0]  len;
  rand bit [7:0]  payload[];
  rand bit       is_long;
  constraint c_kind { kind dist {0 := 60, [1:3] := 30, [4:15] := 10}; }
  constraint c_len  { is_long -> len > 64;                // implication
                    !is_long -> len inside {[1:16]};
                    solve is_long before len; }          // shape distribution only
  constraint c_pay  { payload.size() == int'(len);
                    foreach (payload[i]) payload[i] != 8'hFF; }
  constraint c_def  { soft len < 32; }                    // yields to inline constraints
endclass

module q_rand;
  initial begin
    packet p;
    p = new();
    if (!p.randomize() with { len == 100; }) $error("randomize failed");
    p.c_def.constraint_mode(0);                          // switch a block off
    p.kind.rand_mode(0);                                  // freeze one variable
  end
endmodule
```

Q61. What happens when `randomize()` fails?

It returns 0 and leaves all random variables unchanged; nothing else happens unless you check. Always write `if (!obj.randomize()) $fatal(...)` or an `assert` - but not `assert(obj.randomize())` if assertions might be disabled, because the call would disappear with it.

Q62. What are `pre_randomize` and `post_randomize`?

Callbacks the solver calls automatically before and after solving, recursively for rand sub-objects. Use them to set up state variables or to compute derived fields (a CRC, a parity) from the random values.

Q63. What is random stability?

Every thread and object has its own random number generator seeded from its parent, so adding a new `randomize` call or thread elsewhere does not change the sequence seen by existing ones. That is why `$urandom` and `randomize()` are preferred to `$random`, and why a failing test reproduces with the same seed.

Q64. How do you randomize a variable that is not in a class?

`std::randomize(x, y) with { x < y; };` randomizes local or module variables with inline constraints. `$urandom_range(max, min)` covers simple uniform ranges.

Processes and inter-process communication

Q65. Explain `fork...join`, `join_any` and `join_none`.

`join` waits for all children; `join_any` for the first (the others keep running); `join_none` does not wait (children start when the parent blocks or finishes). `wait fork` waits for all descendants; `disable fork` kills them. Appendix A shows all of them running.

Q66. What does this loop print, and how do you fix it?

IEEE 1800 semantics: the three threads of the first loop are created by `join_none` but only start when the parent blocks at `#2`; by then the single loop variable `i` is 3, so all print `A3`. The fix declares an `automatic` copy per iteration, captured when the fork executes, giving `B0 B1 B2`. Commercial simulators print `A3 A3 A3` and `B0 B1 B2`. Beware tool differences: Verilator 5.020 prints `A0 A1 A2` for the first loop and Icarus 12 does not support the `automatic` declaration here - so no output card is shown.

```
module q_forkloop;
  initial begin
    for (int i = 0; i < 3; i++)
      fork
        #1 $write("A%d ", i);          // all threads see the final i
      join_none
    #2 $display("");
    for (int i = 0; i < 3; i++) begin
      automatic int k = i;             // per-iteration copy captured by the thread
      fork
        #1 $write("B%d ", k);
      join_none
    end
    #2 $display("");
    $finish;
  end
endmodule
```

Q67. What is the difference between `@(ev)` and `wait(ev.triggered)`?

`@(ev)` blocks until the next trigger, so if `->ev` executes earlier in the same time step the waiter misses it and may hang. `wait(ev.triggered)` is true for the whole time step in which `ev` was triggered, removing that race. `->>ev` triggers in the NBA region.

Q68. How do mailboxes and semaphores work? What does this print?

A mailbox is a FIFO between processes: `put` (blocks when a bounded mailbox is full), `get` (blocks when empty), `try_put/try_get/peek` (non-blocking). `mailbox #(T)` is type-checked. A semaphore is a bucket of keys: `get(n)` blocks until `n` keys are available, `put(n)` returns them. With a bound of 2 the producer stalls on its third item until the consumer frees space; the semaphore serialises P1 and P2.

```
module q_mbx;
  mailbox #(int) mb = new(2);          // bounded: put() blocks when 2 items queued
  semaphore      key = new(1);
  initial begin
    fork
      for (int i = 1; i <= 4; i++) begin mb.put(i); $display("%0t put %0d", $time, i); end
      repeat (4) begin int v; #10 mb.get(v); $display("%0t got %0d", $time, v); end
    join
    fork
      begin key.get(1); $display("%0t P1 has key", $time); #5 key.put(1); end
      begin key.get(1); $display("%0t P2 has key", $time); #5 key.put(1); end
    join
    $finish;
  end
endmodule
```



```

0 put 1
0 put 2
10 got 1
10 put 3
20 got 2
20 put 4
30 got 3
40 got 4
40 P1 has key
45 P2 has key

```

Verilator 5.020

Q69. How do you implement a timeout around a task call?

Race the task against a delay inside an isolating fork:

```
fork begin fork do_xfer(); begin #1us; $error("timeout"); end join_any disable fork; end
join - the outer fork begin ... end join limits disable fork to these two threads.
```

Q70. What does `disable` do on a named block versus `disable fork`?

`disable blk`; terminates the named block (or task) - in every process executing it, which surprises people with re-entrant tasks. `disable fork`; terminates all child processes of the calling process. `break` and `continue` (SV) replace `disable` for loop control.

Q71. How do you get a handle to a process and control it?

`process p = process::self();` inside the thread; then `p.status()`, `p.kill()`, `p.suspend()`, `p.resume()`, `p.await()`, `p.srandom(seed)`. This gives finer control than `disable fork`.

SystemVerilog Assertions

Q72. Immediate versus concurrent versus deferred assertions?

Immediate (`assert (expr)`) is procedural, checks at once, no clock. Concurrent (`assert property (...)`) is clocked, samples values in the Preponed region and can span many cycles. Deferred immediate (`assert #0 / assert final`) reports only after the values settle in the time step, avoiding glitches from zero-delay ordering.

Q73. `|->` versus `|=>`?

`a |-> b` (overlapping): `b` checked in the same cycle as the end of `a`. `a |=> b` (non-overlapping): one cycle later; it equals `a |-> ##1 b`.

Q74. What is a vacuous pass?

An implication whose antecedent never matches passes vacuously. An assertion that only ever passes vacuously checked nothing, so pair important assertions with `cover property` on the antecedent.

Q75. Explain `[*n]`, `[->n]` and `[=n]`.

`a[*3]`: `a` on 3 **consecutive** cycles. `a[->3]` (goto): 3 not necessarily consecutive occurrences, ending **on** the third. `a[=3]` (non-consecutive): 3 occurrences, and the match may extend past the third while `a` stays low.

Q76. How do `$rose`, `$fell`, `$stable` and `$past` evaluate?

They compare the sampled value in the current clock tick with the previous one: `$rose(a)` means the LSB went to 1. `$past(a, n)` returns the value `n` ticks ago (X or the initial value before `n` ticks exist). All use Preponed-region samples, so they see values before the clock edge updates.

Q77. What does `disable iff` do?

It asynchronously aborts an in-flight evaluation when its condition is true (typically reset), so neither pass nor fail is reported. `default disable iff (!rst_n);` applies it to every assertion in the scope.

Q78. How do you attach assertions to a design without editing it?

`bind dut_module sva_checker u_chk (.*);` instantiates the checker inside every instance of `dut_module` (or a specific instance path). Verification engineers keep assertions in separate files that way.

Q79. `assert`, `assume`, `cover`, `restrict` - differences in formal and simulation?

`assert`: must hold - a proof target. `assume`: environment constraint; formal uses it to limit inputs and simulation checks it like an `assert`. `cover`: must be reachable - proves a scenario is possible. `restrict` (2009): constrains formal only and is ignored in simulation.

Functional coverage

Q80. Code coverage versus functional coverage?

Code coverage (line, branch, condition, toggle, FSM) is collected automatically and tells you which code executed. Functional coverage is written by the engineer from the verification plan and tells you which **features** were exercised. 100% code coverage with missing functional coverage means untested scenarios; the reverse means missing features or dead code.

Q81. What are bins, and what are `illegal_bins` and `ignore_bins`?

Bins are counters for values or ranges of a coverpoint (`bins b[] = {[0:3]}` creates one per value). `ignore_bins` excludes values from the coverage goal; `illegal_bins` excludes them and reports an error if hit.

Q82. How is a covergroup sampled?

By a clocking event in its declaration (`covergroup cg @(posedge clk);`), by calling `cg_inst.sample()`, or by a user-defined `sample` function with arguments (`with function sample(...)`). A covergroup must be constructed with `new()` or nothing is collected.

Q83. What are transition bins and cross coverage?

Transition bins cover sequences of values: `(0 => 1 => 2)`, `(3 => 0)`, `(1 [*3])`. `cross` counts combinations of coverpoints; use `binsof` with `ignore_bins` to remove impossible combinations, otherwise the cross can never reach 100%.

Q84. What do `option.per_instance`, `at_least` and `goal` control?

`per_instance = 1` keeps separate coverage per covergroup instance (else only the merged type coverage is reported); `at_least` is the hit count for a bin to be covered (default 1); `goal` the target percentage.

Q85. How do you know when verification is done?

When the functional coverage model derived from the plan is closed (or each gap is waived), code coverage holes are explained, all assertions have covered antecedents, bug rate has flattened and regressions pass across seeds. Coverage without checkers proves nothing - a scenario must be both reached and checked.

DPI and how UVM uses the language

Q86. How do you call C from SystemVerilog and vice versa?

`import "DPI-C"` declares a C function callable from SV; `export "DPI-C"` makes an SV function or task callable from C. Arguments map directly for C-compatible types (`int -> int`, `real -> double`, `string -> const char*`, `chandle -> void*`) and through `svBit`, `svLogic`, `svBitVecVal`, `svLogicVecVal` for packed types. A C function that calls an export must be imported `context`. The example was compiled with Verilator 5.020, which builds the C file as C++, hence the `extern "C"` guard.

```

module q_dpi;
  import "DPI-C" function int c_add(input int a, input int b); // C: int c_add(int, int)
  import "DPI-C" context function void c_run(input string name); // may call exports
  export "DPI-C" function sv_log; // callable from C
  function void sv_log(string msg); $display("[from C] %s", msg); endfunction
  initial begin
    $display("c_add(2, 3) = %0d", c_add(2, 3));
    c_run("dma_test");
    $finish;
  end
endmodule

```

```

#include <stdio.h>
#include "svdpi.h"
#ifdef __cplusplus
extern "C" { /* Verilator compiles this file as C++ */
#endif
extern void sv_log(const char *msg); /* prototype of the SV export */
int c_add(int a, int b) { return a + b; }
void c_run(const char *name) {
  char buf[64];
  snprintf(buf, sizeof buf, "running %s", name);
  sv_log(buf); /* legal: import is 'context' */
}
#ifdef __cplusplus
}
#endif

```

```

c_add(2, 3) = 5
[from C] running dma_test

```

Verilator 5.020: `verilator --binary q_dpi.sv q_dpi.c`

Q87. What is the difference between pure and context imports?

pure: result depends only on inputs, no side effects - the simulator may optimise or skip calls. **context:** the C code may call exported SV functions or access the calling scope via `svGetScope` and VPI; the simulator must preserve scope information, which costs speed. Default is neither.

Q88. How does the UVM factory work in language terms?

The macro call `uvm_component_utils(my_drv)` expands to a `typedef uvm_component_registry #(my_drv, "my_drv") type_id;` whose static member registers a proxy object with the factory at time 0 (static initialisation of a parameterized class specialization). `type_id::create` asks the factory, which checks overrides and calls the proxy's `create`, which calls `new` of the right class - a virtual constructor built from parameterized classes and statics (Chapter 26).

Q89. How does UVM get a virtual interface to a driver?

The top module calls `uvm_config_db #(virtual bus_if)::set(null, "uvm_test_top.*", "vif", bus_if_inst);` and the driver's `build_phase` calls `get` with the same type and name. The config DB is a parameterized class built on a static resource database keyed by scope and name strings.

Q90. Why are UVM phases functions or tasks?

`build_phase`, `connect_phase` and the other setup/report phases are functions because they must take zero time; `run_phase` and the runtime sub-phases are tasks because they consume simulation time. Objections (`raise_objection/drop_objection`) decide when the time-consuming phases end.

Q91. How do TLM ports connect components in UVM?

Ports, exports and imps are parameterized classes (`uvm_analysis_port #(T)`); `connect()` stores the target handle and `write(t)` calls the implementing component's `write` method through it - plain handles and virtual methods, no signals.

Q92. What is a `uvm_sequence` in language terms?

A class whose `body()` task generates items; `start_item/finish_item` handshake with the sequencer (which arbitrates with semaphores/queues) and the driver's `get_next_item/item_done`. The `uvm_do` family of macros wraps `create`, `randomize` and that handshake.

Coding puzzles**Q93. Test whether `x` is a power of two, isolate its lowest set bit, reverse its bits, swap bytes, convert binary to Gray and count ones - one line each.**

`x & (x - 1)` clears the lowest set bit, so it is 0 exactly for powers of two (exclude `x == 0`). `x & (~x + 1)` (that is `x & -x`) keeps only the lowest set bit. Streaming reverses bits or bytes. Gray is `x ^ (x >> 1)`. `$countones` counts (synthesizable in most tools).

```
module q_bits;
  logic [7:0] x = 8'b0110_1000;
  initial begin
    $display("pow2(x)=%b pow2(64)=%b", (x != 0) && ((x & (x - 8'd1)) == 0),
            (8'd64 & (8'd64 - 8'd1)) == 0);
    $display("lowest set bit   = %b", x & (~x + 8'd1));
    $display("bit-reverse     = %b", {<<{x}});
    $display("byte-swap       = %h", {<<8{16'hABCD}});
    $display("bin->gray       = %b", x ^ (x >> 1));
    $display("popcount        = %0d", $countones(x));
    $finish;
  end
endmodule
```

```
pow2(x)=0 pow2(64)=1
lowest set bit   = 00001000
bit-reverse     = 00010110
byte-swap       = cdab
bin->gray       = 01011100
popcount        = 3
```

Verilator 5.020

Q94. How do `case`, `casez` and `casex` treat an X in the selector?

`case` matches X only against an item with X in the same position. `casez` treats `z/?` in either side as don't-care, but X is still an exact value. `casex` treats X as don't-care, so `2'b1x` matches `2'b10`. An `if (s == 2'b10)` gets X and takes the else branch.

```
module q_case;
  logic [1:0] s = 2'b1x;
  initial begin
    case (s) 2'b10: $display("case : 10"); 2'b1x: $display("case : 1x");
              default: $display("case : default"); endcase
    casez (s) 2'b1?: $display("casez: 1?"); default: $display("casez: default"); endcase
    casex (s) 2'b10: $display("casex: 10"); default: $display("casex: default"); endcase
    if (s == 2'b10) $display("if   : taken"); else $display("if   : else (s==10 is x)");
  end
endmodule
```

```
case : 1x
casez: 1?
casex: 10
if   : else (s==10 is x)
```

Icarus 12

Q95. Write a parameterized N-bit synchronous counter with enable and active-low async reset.

One `always_ff`: `always_ff @(posedge clk or negedge rst_n) if (!rst_n) cnt <= '0; else if (en) cnt <= cnt + 1'b1; with logic [N-1:0] cnt. '0 scales with N; 1'b1 extends to N bits in context.`

Q96. How do you write a combinational priority encoder that synthesizes cleanly?

In `always_comb`: default the outputs (`idx = '0; valid = 1'b0;`), then `for (int i = 0; i < N; i++) if (req[i]) begin idx = i[IW-1:0]; valid = 1'b1; end` - the last match wins, so this gives highest-index priority; loop downward for lowest. The loop unrolls into a mux chain.

Q97. Swap two variables without a temporary in RTL.

`a <= b; b <= a;` in the same clocked block - non-blocking assignments sample both RHS values before either update. In procedural testbench code, `{a, b} = {b, a};` also works because the RHS concatenation is evaluated first.

Q98. What is wrong with `assign y = sel ? a : 8'bz;` driving an internal bus?

Nothing in simulation, but internal tri-states are not supported in most ASIC flows (and poorly in FPGAs): synthesis converts them to muxes or fails. Keep `z` for top-level pads and use muxes internally.

Q99. Generate a 50 MHz clock and a reset that deasserts after 3 edges in a testbench.

`initial clk = 0; always #10ns clk = ~clk;` (20 ns period), and `initial begin rst_n = 0; repeat (3) @(posedge clk); rst_n <= 1; end`. The non-blocking release avoids a race with flops sampling `rst_n` on the same edge.

Q100. Detect a rising edge of an input `d` synchronously.

Register it and compare: `always_ff @(posedge clk) d_q <= d;` and `assign rise = d & ~d_q;`. If `d` is asynchronous, pass it through a 2-flop synchronizer first and detect the edge on the synchronized value.

Appendix D. Standards, Tools and Resources

This appendix maps the documents, tools and code bases you will meet when working with Verilog and SystemVerilog professionally. Product names and ownership change often (vendors merge and rename tools); the standards numbers are the stable anchors.

Verilog: IEEE 1364

Edition	Key content
Pre-IEEE (1984-1995)	Gateway Design Automation created Verilog-XL in 1984; Cadence acquired Gateway (1989) and released the language to Open Verilog International (OVI) in 1990, which led to standardisation.
IEEE 1364-1995	First standard: modules, nets and <code>reg</code> , gate primitives and UDPs, <code>specify</code> blocks, the PLI (tf/acc routines and VPI).
IEEE 1364-2001	The big update (Verilog-2001): ANSI-style ports, <code>generate/genvar</code> , <code>signed</code> types and <code>\$signed</code> , <code>**</code> , <code>>>>/<<<</code> , multi-dimensional arrays, <code>always @*</code> , <code>localparam</code> , configurations, file I/O (<code>\$fopen</code> modes, <code>\$fscanf...</code>), the <code>ifndef</code> and <code>elsif</code> directives, <code>automatic</code> tasks/functions, an extended VPI.
IEEE 1364-2005	Corrections and clarifications; <code>uwire</code> , the keyword-version directives, generate-scope naming rules; the old tf/acc PLI routines deprecated in favour of VPI. The last standalone Verilog standard.
IEEE 1364.1-2002	RTL synthesis subset: what synthesis tools must accept and how they interpret it.
After 2009	IEEE 1364 was merged into IEEE 1800-2009 and is no longer maintained; 'Verilog' today means the Verilog subset of 1800.

SystemVerilog: IEEE 1800

Edition	Key additions
Accellera SV 3.0 / 3.1 / 3.1a (2002-2004)	Donations from Co-Design (Superlog), Synopsys (Vera verification language and OpenVera assertions), Novas (debug API) and others were merged into Verilog by Accellera, then handed to IEEE.
IEEE 1800-2005	First IEEE SystemVerilog, published as an extension to 1364-2005: <code>logic</code> , 2-state types, enums, structs, unions, typedefs, packages, interfaces and modports, <code>always_comb/ff/latch</code> , <code>unique/priority</code> , classes, constrained random, dynamic and associative arrays, queues, strings, mailboxes/semaphores/events, clocking and program blocks, SVA, covergroups, DPI.
IEEE 1800-2009	Merged with 1364 into a single language document. Checkers, <code>let</code> , global clocking, strong/weak properties and LTL operators (<code>until</code> , <code>s_eventually</code> , <code>nexttime</code> , <code>accept_on...</code>), deferred immediate assertions, <code>unique0</code> , macros with default arguments.
IEEE 1800-2012	Interface classes (<code>implements</code> , multiple inheritance of API), <code>soft</code> constraints, <code>unique</code> constraints, user-defined nettypes and <code>interconnect</code> , covergroup enhancements (<code>with</code> clause on bins, cross-bin select expressions), <code>\$assertcontrol</code> , real number modelling improvements. Free download through the IEEE GET program, sponsored by Accellera.
IEEE 1800-2017	Corrections and clarifications only - no significant new features. The reference most tools and this book target; free through IEEE GET.
IEEE 1800-2023	Published early 2024 (also free through IEEE GET). Additions include method override qualifiers (<code>:initial</code> , <code>:extends</code> , <code>:final</code>), triple-quoted multi-line string literals, boolean expressions in the <code>ifdef</code> directive, coverpoints of <code>real</code> type, and further class and constraint refinements; tool support lags by several years, so check your simulators before using them.

Related standards

Standard	What it is	Where you meet it
IEEE 1800.2 (UVM)	Universal Verification Methodology class library API: 1800.2-2017 and 1800.2-2020. Accellera maintains the open-source reference implementation (UVM 1.1d and 1.2 were the pre-IEEE Accellera releases; the '2017-x.y' and '2020-x.y' releases track the IEEE editions).	Every commercial DV environment (Chapter 26)
IEEE 1801 (UPF)	Unified Power Format: power domains, supply networks, isolation, retention, level shifters (editions 2009, 2013, 2015, 2018 and later).	Power-aware simulation, synthesis, P&R
IEEE 1735	Recommended practice for encryption and rights management of IP (<code>pragma protect</code> envelopes).	Encrypted vendor IP, memory models
IEEE 1497 (SDF)	Standard Delay Format for back-annotating delays and timing checks.	Gate-level simulation (Chapter 9)
IEEE 1685 (IP-XACT)	XML description of IP: ports, bus interfaces, register maps.	SoC integration, register generation
Accellera SystemRDL 2.0	Register description language from which RTL, UVM register models, C headers and docs are generated.	CSR blocks (tools: PeakRDL, commercial generators)
IEEE 1850 (PSL)	Property Specification Language; largely superseded by SVA for SV users.	Older VHDL/mixed flows
IEEE 1666 (SystemC)	C++ class library for system-level and TLM modelling.	Architecture models, virtual prototypes
Accellera PSS	Portable Test and Stimulus Standard: abstract scenario models retargeted to UVM, C tests and post-silicon.	SoC-level test generation
Accellera UCIS	Unified Coverage Interoperability Standard (coverage database API).	Coverage merging across tools

Simulators

Simulator	Type	Notes
Synopsys VCS	Commercial	Full SV/UVM, constraint solver, coverage, SVA; Verdi debugger
Cadence Xcelium	Commercial	Full SV/UVM, multi-core; SimVision/Verisium debug
Siemens Questa (and ModelSim)	Commercial	Full SV/UVM in Questa; ModelSim editions shipped with FPGA tools have limited SV verification support
Aldec Riviera-PRO / Active-HDL	Commercial	Full SV/UVM; popular in FPGA and mixed-language work; available on EDA Playground
AMD Vivado simulator (xsim)	Free with Vivado	SV design and much of verification including UVM
Altair DSIm (formerly Metrics)	Commercial, free tier	Cloud/desktop SV/UVM simulator
Verilator	Open source	Compiles SV to multithreaded C++; fastest open simulator; 2-state; strong on design, growing on verification (classes, timing, fork); limited SVA, no constraint solving or covergroups in 5.020
Icarus Verilog	Open source	Event-driven 4-state interpreter; excellent Verilog-2005, partial SV (limited classes; no <code>ref</code> arguments or unpacked structs in v12)
CVC / Tachyon	Open source	Compiled Verilog-2005 simulator with full 4-state and timing

Front ends, lint and formal

Tool	Type	Use
slang	Open source	Complete, fast SV parser and elaborator with excellent diagnostics; pyslang Python bindings (used to check this book's templates)
Surelog / UHDM	Open source	SV front end producing a universal hardware data model for other tools
sv2v	Open source	Converts SV design code to Verilog-2005 for older tools
Verible	Open source	Parser, style linter, formatter and language server (from Google/CHIPS Alliance)
svlint	Open source	Configurable rule-based SV linter
Verilator --lint-only -Wall	Open source	Quick width/latch/multi-driver lint
Yosys + SymbiYosys (sby)	Open source	Synthesis, equivalence and formal property checking (immediate assertions and a subset of SVA in the open front end)
EBMC	Open source	Bounded model checker for SV with SVA support
Synopsys SpyGlass / VC SpyGlass	Commercial	Lint, CDC, RDC, constraints
Siemens Questa Lint / CDC / Formal	Commercial	Lint, clock-domain crossing, property checking
Cadence Jasper	Commercial	Formal apps (FPV, connectivity, sequential equivalence, X-prop)
Synopsys VC Formal	Commercial	Formal property and equivalence apps
Real Intent Ascent / Meridian	Commercial	Lint, X-propagation, CDC, RDC
Cadence Conformal / Synopsys Formality	Commercial	Logical equivalence checking RTL vs netlist

Online playgrounds and references

Resource	What it offers
EDA Playground (edaplayground.com)	Browser-based editor with free access to commercial and open simulators, UVM libraries and waveform viewing; ideal for trying a construct on several tools and sharing a reproducible example.
CHIPS Alliance sv-tests	Per-LRM-clause test suite with a public table of which open tools support which SV feature - check here before relying on a construct in an open flow.
HDLBits	Graded Verilog practice problems from gates to FSMs.
Verification Academy (Siemens)	UVM, coverage and formal courses and the UVM cookbook.
Accellera and IEEE GET	Free IEEE 1800-2017/2023 and 1800.2 PDFs; UVM reference implementation downloads.
Sunburst Design paper archive	Cummings' SNUG and DVCon papers (see below).
DVCon and SNUG proceedings	Industrial papers on SV, UVM, SVA and coverage methodology; the best source for 'how teams really do it'.
Doulos KnowHow, ChipVerify	Short tutorials and examples for most language features.

Books

Book	Authors	Why read it
Verilog HDL: A Guide to Digital Design and Synthesis (2nd ed.)	Samir Palnitkar	Classic Verilog-2001 introduction with gate-level, switch-level and PLI coverage
SystemVerilog for Design (2nd ed.)	Stuart Sutherland, Simon Davidmann, Peter Flake	The design subset of SV explained by members of the standards committee
RTL Modeling with SystemVerilog for Simulation and Synthesis	Stuart Sutherland	Modern synthesizable coding, with tool results
Verilog and SystemVerilog Gotchas	Stuart Sutherland, Don Mills	101 language traps with explanations - interview gold
SystemVerilog for Verification (3rd ed.)	Chris Spear, Greg Tumbush	The standard DV textbook: OOP, randomization, IPC, coverage
SVA: The Power of Assertions in SystemVerilog (2nd ed.)	Eduard Cerny, Surrendra Dudani, John Havlicek, Dmitry Korchemny	Definitive, formal-semantics-level treatment of SVA by its designers
SystemVerilog Assertions and Functional Coverage	Ashok B. Mehta	Practical, example-driven SVA and covergroup guide
The UVM Primer	Ray Salemi	Gentle path from SV OOP to UVM with a small complete testbench
Writing Testbenches using SystemVerilog	Janick Bergeron	Verification methodology foundations
A Practical Guide to Adopting the Universal Verification Methodology	Sharon Rosenberg, Kathleen Meade	UVM architecture from Cadence authors

Clifford Cummings' SNUG papers worth reading

- *Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!* (SNUG 2000) - the origin of the blocking/non-blocking rules (Chapter 5).
- *"full_case parallel_case", the Evil Twins of Verilog Synthesis* (SNUG 1999) - why pragmas cause mismatches (Chapter 11).
- *Synchronous Resets? Asynchronous Resets? I am so confused!* (with Don Mills, SNUG 2002) and its sequel *Asynchronous and Synchronous Reset Design Techniques - Part Deux* (with Mills and Steve Golson, SNUG 2003).
- *Simulation and Synthesis Techniques for Asynchronous FIFO Design* (SNUG 2002) and *Clock Domain Crossing (CDC) Design and Verification Techniques Using SystemVerilog* (SNUG 2008).
- *Verilog Nonblocking Assignments With Delays, Myths and Mysteries* (SNUG 2002) - scheduling details behind # in NBAs.
- *SystemVerilog Logic Specific Processes for Synthesis - Benefits and Proper Usage* (SNUG 2016) - [always_comb/always_ff](#) in depth.
- Papers on SV implicit port connections, UVM transaction coding styles and the UVM factory - all at [sunburst-design.com](#).

Open-source SystemVerilog code bases to read

Project	What to learn from it
lowRISC Ibex	Small, production-quality RV32 core in clean SV; its DV uses UVM-style environments, riscv-dv random instruction generation and formal; read with the lowRISC Verilog coding style guide.
OpenTitan	Complete open silicon root-of-trust SoC: composable IP template, register generation from HJSON, TL-UL bus, DV methodology with UVM, assertions and coverage at scale.

Project	What to learn from it
PULP platform (ETH Zurich / Bologna)	<code>common_cells</code> (FIFOs, arbiters, CDC), the <code>axi</code> library with type-parameterized structs, Snitch and CV32E40P cores - excellent SV design idioms (Chapter 17).
CVA6 (OpenHW Group)	64-bit application-class RISC-V core (formerly Ariane) with MMU and caches; industrial verification in core-v-verif.
Verilator test suite (<code>test_regress/t</code>)	Thousands of tiny, self-checking SV programs, one feature each - the fastest way to see how a construct is supposed to behave in an open tool.
Accellera UVM reference implementation	The <code>uvm-core</code> source: how factories, config DB, phasing and TLM are built from classes, parameterization and static members (Chapter 26).
CHIPS Alliance sv-tests and slang tests	Minimal examples keyed to LRM clauses.

How to use these resources

- 1 Keep the free IEEE 1800-2017 (or 2023) PDF open and look up the clause whenever a rule in this book matters to your code.
- 2 Try every unfamiliar construct on at least two tools (EDA Playground makes that free); if they disagree, the LRM decides - and your code should avoid the construct until both agree.
- 3 Lint continuously (Verilator, Verible or a commercial linter) and read each warning as a lesson in the sizing and signedness rules of Appendix B.
- 4 Read one real code base end to end - Ibex is the right size - before writing your first large block.
- 5 Revisit the interview questions of Appendix C after each part of the book; the ones you cannot answer point to the chapter to reread.